Software Manual

System Release Package

User's Manual: Hardware and Software

Renesas Microprocessor
RZ G/V Series

All information contained in these materials, including products and product specifications, represents information on the product at the time of publication and is subject to change by Renesas Electronics Corp. without notice. Please review the latest information published by Renesas Electronics Corp. through various means, including the Renesas Electronics Corp. website (http://www.renesas.com).

Renesas Electronics
www.renesas.com

Rev.3.30 Sep.11.26

Notice

1. Descriptions of circuits, software and other related information in this document are provided only to illustrate the operation of semiconductor products and application examples. You are fully responsible for the incorporation or any other use of the circuits, software, and information in the design of your product or system. Renesas Electronics disclaims any and all liability for any losses and damages incurred by you or third parties arising from the use of these circuits, software, or information.
2. Renesas Electronics hereby expressly disclaims any warranties against and liability for infringement or any other claims involving patents, copyrights, or other intellectual property rights of third parties, by or arising from the use of Renesas Electronics products or technical information described in this document, including but not limited to, the product data, drawings, charts, programs, algorithms, and application examples.
3. No license, express, implied or otherwise, is granted hereby under any patents, copyrights or other intellectual property rights of Renesas Electronics or others.
4. You shall be responsible for determining what licenses are required from any third parties, and obtaining such licenses for the lawful import, export, manufacture, sales, utilization, distribution or other disposal of any products incorporating Renesas Electronics products, if required.
5. You shall not alter, modify, copy, or reverse engineer any Renesas Electronics product, whether in whole or in part. Renesas Electronics disclaims any and all liability for any losses or damages incurred by you or third parties arising from such alteration, modification, copying or reverse engineering.
6. Renesas Electronics products are classified according to the following two quality grades: “Standard” and “High Quality”. The intended applications for each Renesas Electronics product depends on the product’s quality grade, as indicated below.
   "Standard": Computers; office equipment; communications equipment; test and measurement equipment; audio and visual equipment; home electronic appliances; machine tools; personal electronic equipment; industrial robots; etc.
   "High Quality": Transportation equipment (automobiles, trains, ships, etc.); traffic control (traffic lights); large-scale communication equipment; key financial terminal systems; safety control equipment; etc.

   Unless expressly designated as a high reliability product or a product for harsh environments in a Renesas Electronics data sheet or other Renesas Electronics document, Renesas Electronics products are not intended or authorized for use in products or systems that may pose a direct threat to human life or bodily injury (artificial life support devices or systems; surgical implantations; etc.), or may cause serious property damage (space system; undersea repeaters; nuclear power control systems; aircraft control systems; key plant systems; military equipment; etc.). Renesas Electronics disclaims any and all liability for any damages or losses incurred by you or any third parties arising from the use of any Renesas Electronics product that is inconsistent with any Renesas Electronics data sheet, user’s manual or other Renesas Electronics document.
7. No semiconductor product is absolutely secure. Notwithstanding any security measures or features that may be implemented in Renesas Electronics hardware or software products, Renesas Electronics shall have absolutely no liability arising out of any vulnerability or security breach, including but not limited to any unauthorized access to or use of a Renesas Electronics product or a system that uses a Renesas Electronics product. RENESAS ELECTRONICS DOES NOT WARRANT OR GUARANTEE THAT RENESAS ELECTRONICS PRODUCTS, OR ANY SYSTEMS CREATED USING RENESAS ELECTRONICS PRODUCTS WILL BE INVULNERABLE OR FREE FROM CORRUPTION, ATTACK, VIRUSES, INTERFERENCE, HACKING, DATA LOSS OR THEFT, OR OTHER SECURITY INTRUSION (“Vulnerability Issues”). RENESAS ELECTRONICS DISCLAIMS ANY AND ALL RESPONSIBILITY OR LIABILITY ARISING FROM OR RELATED TO ANY VULNERABILITY ISSUES. FURTHERMORE, TO THE EXTENT PERMITTED BY APPLICABLE LAW, RENESAS ELECTRONICS DISCLAIMS ANY AND ALL WARRANTIES, EXPRESS OR IMPLIED, WITH RESPECT TO THIS DOCUMENT AND ANY RELATED OR ACCOMPANYING SOFTWARE OR HARDWARE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE.
8. When using Renesas Electronics products, refer to the latest product information (data sheets, user’s manuals, application notes, “General Notes for Handling and Using Semiconductor Devices” in the reliability handbook, etc.), and ensure that usage conditions are within the ranges specified by Renesas Electronics with respect to maximum ratings, operating power supply voltage range, heat dissipation characteristics, installation, etc. Renesas Electronics disclaims any and all liability for any malfunctions, failure or accident arising out of the use of Renesas Electronics products outside of such specified ranges.
9. Although Renesas Electronics endeavors to improve the quality and reliability of Renesas Electronics products, semiconductor products have specific characteristics, such as the occurrence of failure at a certain rate and malfunctions under certain use conditions. Unless designated as a high reliability product or a product for harsh environments in a Renesas Electronics data sheet or other Renesas Electronics document, Renesas Electronics products are not subject to radiation resistance design. You are responsible for implementing safety measures to guard against the possibility of bodily injury, injury or damage caused by fire, and/or danger to the public in the event of a failure or malfunction of Renesas Electronics products, such as safety design for hardware and software, including but not limited to redundancy, fire control and malfunction prevention, appropriate treatment for aging degradation or any other appropriate measures. Because the evaluation of microcomputer software alone is very difficult and impractical, you are responsible for evaluating the safety of the final products or systems manufactured by you.
10. Please contact a Renesas Electronics sales office for details as to environmental matters such as the environmental compatibility of each Renesas Electronics product. You are responsible for carefully and sufficiently investigating applicable laws and regulations that regulate the inclusion or use of controlled substances, including without limitation, the EU RoHS Directive, and using Renesas Electronics products in compliance with all these applicable laws and regulations. Renesas Electronics disclaims any and all liability for damages or losses occurring as a result of your noncompliance with applicable laws and regulations.
11. Renesas Electronics products and technologies shall not be used for or incorporated into any products or systems whose manufacture, use, or sale is prohibited under any applicable domestic or foreign laws or regulations. You shall comply with any applicable export control laws and regulations promulgated and administered by the governments of any countries asserting jurisdiction over the parties or transactions.
12. It is the responsibility of the buyer or distributor of Renesas Electronics products, or any other party who distributes, disposes of, or otherwise sells or transfers the product to a third party, to notify such third party in advance of the contents and conditions set forth in this document.
13. This document shall not be reprinted, reproduced or duplicated in any form, in whole or in part, without prior written consent of Renesas Electronics.
14. Please contact a Renesas Electronics sales office if you have any questions regarding the information contained in this document or Renesas Electronics products.

(Note1) “Renesas Electronics” as used in this document means Renesas Electronics Corporation and also includes its directly or indirectly controlled subsidiaries.

(Note2) “Renesas Electronics product(s)” means any product developed or manufactured by or for Renesas Electronics.

(Rev.5.0-1 October 2020)

Corporate Headquarters

TOYOSU FORESIA, 3-2-24 Toyosu,
Koto-ku, Tokyo 135-0061, Japan
www.renesas.com

Contact information

For further information on a product, technology, the most up-to-date version of a document, or your nearest sales office, please visit:
www.renesas.com/contact/

Trademarks

Renesas and the Renesas logo are trademarks of Renesas Electronics Corporation. All trademarks and registered trademarks are the property of their respective owners.

© 2026 Renesas Electronics Corporation. All rights reserved.

Trademarks (continued)

For the “Cortex” notation, it is used as follows;
— Arm® Cortex®-A55
— Arm® Cortex®-M33
Note that after this page, they may be noted as Cortex-A55 and Cortex-M33 respectively.

Examples of trademark or registered trademark used in the RZ/G2L SMARC Module Board RTK9744L23C01000BE User's Manual: Hardware;
CoreSight™: CoreSight is a trademark of Arm Limited.
MIPI®: MIPI is a registered trademark of MIPI Alliance, Inc.
eMMC™: eMMC is a trademark of MultiMediaCard Association.

Note that in each section of the Manual, trademark notation of ® and TM may be omitted.
All other trademarks and registered trademarks are the property of their respective owners.

© 2026 Renesas Electronics Corporation. All rights reserved.

General Precautions in the Handling of Microprocessing Unit and Microcontroller Unit Products

The following usage notes are applicable to all Microprocessing unit and Microcontroller unit products from Renesas. For detailed usage notes on the products covered by this document, refer to the relevant sections of the document as well as any technical updates that have been issued for the products.

1. Precaution against Electrostatic Discharge (ESD)
   A strong electrical field, when exposed to a CMOS device, can cause destruction of the gate oxide and ultimately degrade the device operation. Steps must be taken to stop the generation of static electricity as much as possible, and quickly dissipate it when it occurs. Environmental control must be adequate. When it is dry, a humidifier should be used. This is recommended to avoid using insulators that can easily build up static electricity. Semiconductor devices must be stored and transported in an anti-static container, static shielding bag or conductive material. All test and measurement tools including work benches and floors must be grounded. The operator must also be grounded using a wrist strap. Semiconductor devices must not be touched with bare hands. Similar precautions must be taken for printed circuit boards with mounted semiconductor devices.
2. Processing at power-on
   The state of the product is undefined at the time when power is supplied. The states of internal circuits in the LSI are indeterminate and the states of register settings and pins are undefined at the time when power is supplied. In a finished product where the reset signal is applied to the external reset pin, the states of pins are not guaranteed from the time when power is supplied until the reset process is completed. In a similar way, the states of pins in a product that is reset by an on-chip power-on reset function are not guaranteed from the time when power is supplied until the power reaches the level at which resetting is specified.
3. Input of signal during power-off state
   Do not input signals or an I/O pull-up power supply while the device is powered off. The current injection that results from input of such a signal or I/O pull-up power supply may cause malfunction and the abnormal current that passes in the device at this time may cause degradation of internal elements. Follow the guideline for input signal during power-off state as described in your product documentation.
4. Handling of unused pins
   Handle unused pins in accordance with the directions given under handling of unused pins in the manual. The input pins of CMOS products are generally in the high-impedance state. In operation with an unused pin in the open-circuit state, extra electromagnetic noise is induced in the vicinity of the LSI, an associated shoot-through current flows internally, and malfunctions occur due to the false recognition of the pin state as an input signal become possible.
5. Clock signals
   After applying a reset, only release the reset line after the operating clock signal becomes stable. When switching the clock signal during program execution, wait until the target clock signal is stabilized. When the clock signal is generated with an external resonator or from an external oscillator during a reset, ensure that the reset line is only released after full stabilization of the clock signal. Additionally, when switching to a clock signal produced with an external resonator or by an external oscillator while program execution is in progress, wait until the target clock signal is stable.
6. Voltage application waveform at input pin
   Waveform distortion due to input noise or a reflected wave may cause malfunction. If the input of the CMOS device stays in the area between V_{IL} (Max.) and V_{IH} (Min.) due to noise, for example, the device may malfunction. Take care to prevent chattering noise from entering the device when the input level is fixed, and also in the transition period when the input level passes through the area between V_{IL} (Max.) and V_{IH} (Min.).
7. Prohibition of access to reserved addresses
   Access to reserved addresses is prohibited. The reserved addresses are provided for possible future expansion of functions. Do not access these addresses as the correct operation of the LSI is not guaranteed.
8. Differences between products
   Before changing from one product to another, for example to a product with a different part number, confirm that the change will not lead to problems. The characteristics of a microprocessing unit or microcontroller unit products in the same group but having a different part number might differ in terms of internal memory capacity, layout pattern, and other factors, which can affect the ranges of electrical characteristics, such as characteristic values, operating margins, immunity to noise, and amount of radiated noise. When changing to a product with a different part number, implement a system-evaluation test for the given product.

Renesas RZ Family / RZ G/V Series

Renesas System Release Package

Introduction

This software manual describes the unified system release package. The system release package contains supported hardware and software.

The result is a consistent experience across the different platforms. This streamlines the development effort for user applications.

Package Contents

The system release package contains the following:

- Multiple Images that are geared to general baseline use cases.
- Yocto build scripts.
- Host-side tools.
- Environmental files.
- SDKs for all images
- Documentation, which includes a user manual and copyright & license information

Features

The following are the general features of the system release package.

- Architected to support multiple platforms with the same image and tools over time.
- Common frameworks.
- Open-source packages using GPLv2 and GPLv3 packages.
- Carefully considered base images that allow for a quick starting point to build a product.
- Complete set of features working out of the box.
- Seamless out-of-the-box experience.
- Automated Yocto build scripts that can rebuild the entire package with only a few commands.
- Host tools to flash the firmware in multiple processes.
- Tools supporting both Linux and Windows workflows.
- Docker-friendly build scripts.
- Extensive documentation covering the hardware, software, and application development and deployment.

Contents

Introduction 1

Package Contents 1

Features 1

Glossary 11

1. Overview 13
1.1 Supported Distributions 13
1.1.1 Yocto Images 13
1.1.2 Renesas Custom Images 13
1.1.3 Ubuntu Images 14
1.2 Supported Platforms 15

2. Introduction 16
2.1 Package Hierarchy 17
2.2 Source Repositories 19
2.3 Release Software Versions 21

3. Required Resources 22
3.1 Development Tools and Software 22
3.2 Hardware 22
3.2.1 Common Hardware Requirements 22
3.2.2 Board-Specific Hardware 22

4. Quick Start 25
4.1 SD/MMC Card Flashing 25
4.2 RZ/G2L-SBC 26
4.2.1 Hardware Requirements 26
4.2.2 Essential Hardware Setup 27
4.2.3 Complete Hardware Setup 28
4.2.4 Booting 28
4.2.5 Known Hardware and Functional Limitations on RZ/G2L-SBC 29
4.3 RS-G2L100 31
4.3.1 Overview of Connectors 31
4.3.2 Hardware Requirements 32
4.3.3 Essential Hardware Setup 32
4.3.4 Complete Hardware Setup 35
4.3.5 Booting 35

4.3.6 Known Hardware and Functional Limitations 36
4.3.7 Solutions and Workarounds 36
4.4 RZ/G2L-EVK and RZ/V2L-EVK 38
4.4.1 Hardware Requirements 38
4.4.2 Essential Hardware Setup 39
4.4.3 Complete Hardware Setup 42
4.4.4 Booting 42
4.4.5 Known Hardware and Functional Limitations 45
4.5 RZ/V2H-EVK 45
4.5.1 Hardware Requirements 45
4.5.2 Essential Hardware Setup and Booting 46
4.5.3 Complete Hardware Setup 49
4.5.4 Booting 49
4.5.5 Known Hardware and Functional Limitations 52
4.6 RZ/V2H-RDK 52
4.6.1 Hardware Requirements 52
4.6.2 Essential Hardware Setup and Booting 53
4.6.3 Complete Hardware Setup 55
4.6.4 Booting 56
4.6.5 Known Hardware and Functional Limitations 56
4.7 IMDT-V2H-SBC 57
4.7.1 Overview of Connectors 57
4.7.2 Hardware Requirements 59
4.7.3 Essential Hardware Setup and Booting 60
4.7.4 Complete Hardware Setup 61
4.7.5 Booting 61
4.7.6 Known Hardware and Functional Limitations 63
4.8 Boot Notice 63

5. General Operational Flow 64
5.1 Arm Exception Levels 64
5.2 Secure and Non-Secure Runtime 65
5.3 Arm Trusted Firmware-A (TF-A) 65
5.3.1 Components of Boot 65
5.3.2 Trusted Boot Flow 68

6. Build System 69
6.1 Build Host Environment Setup 69
6.2 Builders 71
6.2.1 rz_builder.sh 72

6.2.2 Ubuntu Builder 73
6.2.3 DRP/AI Enablement 75
6.3 Yocto OE Build 76
6.3.1 Initiate Yocto Build 76
6.3.2 Collect the Build Output 76
6.4 Ubuntu OE Build 82
6.4.1 Ubuntu Build Host Environment Setup 82
6.4.2 Initial Ubuntu Build 84
6.4.3 Collect the Build Output 85

7. Programming / Flashing Images 91
7.1 Prerequisites 92
7.1.1 Linux Setup 92
7.1.2 Windows Setup 92
7.2 Hardware Setup 94
7.2.1 RZ/G2L-SBC 94
7.2.2 RS-G2L100 95
7.2.3 RZ/G2L-EVK and RZ/V2L-EVK 95
7.2.4 RZ/V2H-EVK 96
7.2.5 RZ/V2H-RDK 96
7.2.6 IMDT-V2H-SBC 96
7.3 Universal Flashing Script 96
7.3.1 flash_images.json – File Overview and Usage 97
7.3.2 Usage and Flashing Operations 99
7.4 Flashing the Bootloader 107
7.4.1 Hardware Connection 107
7.4.2 Flashing Procedure 107
7.5 Flashing the Bootloader from the U-Boot Console (uLoad-bootloader) 108
7.5.1 Prerequisites 108
7.5.2 Collect the Output and Prepare Binaries in the SD Card 109
7.5.3 Hardware Connection 110
7.5.4 Flashing Procedure 110
7.6 Flashing the SD Card Image 111
7.6.1 Hardware Connection 111
7.6.2 Flashing Procedure 112

8. Accessing Supported Features 115
8.1 Supported Features in Yocto Images 115
8.1.1 U-Boot Environment 115
8.1.2 Common Yocto Features 118

8.1.3 RZ/G2L-SBC Yocto Features 157
8.1.4 IMDT V2H-SBC 178
8.2 Supported Features in Ubuntu Images 182
8.2.1 Accessing Supported Features in Ubuntu LXDE 183
8.2.2 Accessing Supported Features in Ubuntu Core 197
8.3 Real-time (RT) Systems and OS 198
8.3.1 Overview 198
8.3.2 What an RTOS is not 199
8.3.3 Types of Real-time Systems 199
8.3.4 Linux RT Patch 200
8.3.5 What does this mean for our Linux 201
8.3.6 Selecting RT or Normal Image in U-Boot 201
8.4 DRP/AI (DRP-AI) 202
8.4.1 Supported RZ/V Devices 202
8.4.2 Core Hardware concepts 202
8.4.3 Overview 203
8.4.4 DRP-AI & DRP-AI3 208
8.4.5 DRP is a Shared Resource (System-Level Constraint) 208
8.4.6 RZ/V DRP-AI Software Stack 210
8.5 Multi-OS / Heterogeneous Multi-Core Operation 228
8.5.1 Supported Devices and Core Mapping 230
8.5.2 Overview 230
8.5.3 Basic Processing Models 230
8.5.4 Communication Concepts 234
8.5.5 Firmware Load Methods 237
8.5.6 Running the RPMsg Sample Application 245

9. Network Boot and TFTP 250
9.1 TFTP Server Setup 250
9.2 NFS Server Setup 251
9.3 U-Boot DHCP IP Configuration 251
9.4 TFTP Boot 252
9.4.1 Prepare boot images from the release package 253
9.4.2 Setup U-Boot boot arguments 255
9.4.3 Configure U-Boot boot commands 255

10. Using SSH and SCP for Remote Access and File Transfers 258
10.1 Differences Between Dropbear and OpenSSH 258
10.2 Using OpenSSH 258
10.3 SSH Access 259

10.3.1 SSH from Windows Host 259
10.3.2 SSH from Linux Host 260
10.4 SCP (Secure Copy Protocol) 261
10.4.1 SCP from Windows Host 261
10.4.2 SCP from Linux Host 263

11. Building the eSDK 264

12. Application Building, Packaging, and Running 266
12.1 How to extract the eSDK 266
12.2 Build a sample application using the eSDK with CMake 267
12.3 Package Programs with Cpack 269
12.3.1 Package a C Program 269
12.3.2 Package a Python Program 272
12.4 Run Sample Applications 275
12.5 Install and Run Debian application packages by using DPKG 275
12.6 Using devtool in the Yocto eSDK 277
12.6.1 Overview 277
12.6.2 Prerequisites 277
12.6.3 Common Usage Scenarios 277

13. Remote Debugging using GDBServer 281
13.1 Prepare GDB on the Host Machine 281
13.2 Install GDBServer on RZ 281
13.3 Remote Debugging Example 282
13.3.1 Remote Debugging on CLI 282
13.3.2 Remote Debugging on Visual Studio Code 285
13.3.3 Remote Debugging on Eclipse IDE 289
13.4 Postmortem Analysis Example 295
13.4.1 Postmortem Analysis on CLI 295
13.4.2 Postmortem Analysis on Visual Studio Code 297
13.4.3 Postmortem Analysis on Eclipse 299

14. Functional Overview 301
14.1 MPU Architecture 301
14.1.1 RZ/G2L 301
14.1.2 RZ/V2L 303
14.1.3 RZ/V2H 304
14.2 RZ/G2L-SBC Board 305
14.2.1 Functional Overview 307
14.2.2 Physical View 307

14.2.3 Overview of Connectors 308
14.2.4 Power Supply 311
14.2.5 Power Management Integrated Circuit- PMIC 312
14.2.6 RESET Control 313
14.2.7 Clock Configuration 314
14.2.8 Peripheral Interface 315
14.2.9 Memory 324
14.3 RS-G2L100 Board 326
14.3.1 Notice 327
14.3.2 Overview 327
14.3.3 Overview of Connectors 330
14.3.4 Power Supply 330
14.3.5 RESET Control 332
14.3.6 Clock Configuration 332
14.3.7 Memory 332
14.4 RZ/G2L-EVK & RZ/V2L-EVK Board 334
14.4.1 Overview 334
14.4.2 Layout Parts 335
14.4.3 Overview of Connectors 336
14.4.4 Power Supply 337
14.4.5 Power Management Integrated Circuit (PMIC) 338
14.4.6 RESET Control 339
14.4.7 Clock Configuration 340
14.4.8 Power Management Integrated Circuit (PMIC) 341
14.4.9 Peripheral Interface 341
14.4.10 Memory 348
14.5 RZ/V2H-EVK Board 349
14.5.1 Overview 349
14.5.2 Part Layout 351
14.5.3 Overview of Connectors 353
14.5.4 Power Supply 353
14.5.5 RESET Control 354
14.5.6 Clock Configuration 354
14.5.7 Peripheral Interface 357
14.5.8 Memory 363
14.6 RZ/V2H-RDK Board 364
14.6.1 Overview 364
14.6.2 Part Layout 366
14.6.3 Overview of Connectors 367

14.6.4 Power Supply 367
14.6.5 Power Management Integrated Circuit (PMIC) 368
14.6.6 Clock Configuration 368
14.6.7 Peripheral Interface 371
14.6.8 Memory 374
14.7 IMDT V2H-SBC Board 376
14.7.1 Notice 376
14.7.2 Overview 376
14.7.3 Overview of Connectors 380
14.7.4 Power Supply 382
14.7.5 Power Management Integrated Circuit (PMIC) 382
14.7.6 Power Tree / Internal Power Architecture 382
14.7.7 Clock Configuration 383
14.7.8 Peripheral Interface 387
14.7.9 Memory 391

15. Adding a New Board (Porting Guide) 394
15.1 Board Identification 394
15.1.1 End-to-End Flow 394
15.1.2 Host-side JSON Specification 395
15.1.3 Host-Side Binary Generation 397
15.1.4 Target-Side Flashing 400
15.1.5 Target-Side Runtime Usage 400
15.2 Flash Writer 401
15.2.1 What must be ported for a new board 401
15.2.2 DDR bring-up for the Flash Writer 403
15.2.3 Add a New SPI Flash 408
15.3 TF-A (BL2/BL31) 410
15.3.1 Firmware Configuration Framework 410
15.3.2 Tool Location and Layout 411
15.3.3 Prerequisites 411
15.3.4 Adding a New Board 412
15.3.5 Usage 413
15.4 U-Boot (BL33) 414
15.4.1 Initialization Model 414
15.4.2 How soc_id and board_id are used 415
15.4.3 Steps to Add a Board 415
15.4.4 Board-Specific Code Hooks 417
15.4.5 Adding BSP Drivers 418
15.5 Linux Kernel 419

15.5.1 Steps to Add a Board 419
15.5.2 Kernel Device Tree Selection 420

16. Appendix 421
16.1 Factory Firmware Flashing Using Serial Downloader (SCIF) Mode 421
16.1.1 RZ/G2L-SBC 421
16.1.2 RS-G2L100 422
16.1.3 RZ/G2L-EVK and RZ/V2L-EVK 422
16.1.4 RZ/V2H-EVK 423
16.1.5 RZ/V2H-RDK 424
16.1.6 IMDT V2H-SBC 426
16.2 Boot Mode Reference (Non-SCIF) 426
16.2.1 RS-G2L100 426
16.2.2 RZ/G2L-EVK & RZ/V2L-EVK 427
16.2.3 RZ/V2H-EVK 428
16.2.4 RZ/V2H-RDK 429
16.2.5 IMDT V2H-SBC 430
16.3 Prepare the eMMC root filesystem 431
16.4 How To Get the Console After Bootup 434
16.5 BSP Interfaces 434
16.6 Creating Remote RPMsg Firmware 434
16.6.1 Prerequisites and Software Requirements 435
16.6.2 RPMsg Example Project Workflow 437

17. Troubleshooting 446
17.1 Unable To Support Scripts for Bootloader/Firmware Flashing on Linux 446
17.2 Flashing Tools Failing Halfway 446
17.3 DHCP Failure 446
17.4 ‘Ifconfig’ doesn’t list the Wi-Fi interface 446
17.5 IP Configuration 446
17.6 Stuck in U-boot with error “Bad Linux ARM64 Image magic!” 446

18. References 448
18.1 Git Repositories 448
18.2 RZ/G MPU Series 448
18.3 RZ/V MPU Series 448
18.4 External Resources 448
18.4.1 Confluence Page 448
18.4.2 QT Development 448
18.4.3 Yocto Project 448
18.4.4 Linux Kernel Documentation 448

18.4.5 Arm Developer Documentation 448
18.4.6 JEDEC DDR4 449
18.4.7 PMOD Specification 449
18.4.8 Essential Linux Tutorial 449
18.4.9 Packaging 449
18.4.10 Using Extensible SDK 449
18.4.11 Install Eclipse IDE 449
18.4.12 Linux Kernel Development 449
18.4.13 Linux Kernel Driver Development 449
18.4.14 Real-time kernel 450
18.4.15 IMD Technologies 450
18.4.16 Renesas RZ Multi-OS package 450

Revision History 451

Glossary

Terms	Description
802.11 - Wi-Fi	The technical name of the standard specification for Wi-Fi is 802.11. This is also the working group that develops and maintains the standards for Wi-Fi that everyone conforms to.
ADC – Analog to digital converter	A hardware unit that converts an input analog signal to a digital value by measuring its immediate voltage at a fixed resolution.
BSP – Board Support Package	BSP is an essential software package that has bootloaders, Linux kernel, a minimal user space and programming tools, allowing the device to boot. This core software allows the system to boot into an operating system, enables all the features and allows application development.
CAN – Controller area network	This is a standardized communication protocol used widely on automotive and aerospace systems. It connects various ECU's known as nodes and uses two wires / lines as a pair carrying differential signals. This method of signaling allows long length cables to interface different systems on the machine with reliable signals. The CAN protocol has multiple specifications and is an ISO standard. It supports flexible data rates reaching as high as 8Mbps. Most automobiles have CAN networks in them, and it is a part of OBD-2 specification which is mandatory law in most of the world for automotive machines like cars.
DAC – Digital to analog converter	A hardware unit that takes digital value and exerts a corresponding analog voltage on an output line.
Firmware	For the scope of this document, the term 'firmware' refers to the low-level software that runs before an OS takes over. This includes arm trust zone, optee & u-boot at the very least. It also refers to the standalone binaries that run on the embedded real-time core like the CM33.
FCONF – Firmware Configuration Framework	FCONF stands for Firmware Configuration Framework, used primarily in Arm Trusted Firmware-A (TF-A). It provides a structured way to pass configuration data (such as hardware descriptions, SoC parameters, memory layout, boot parameters, etc.) into different firmware stages (BL1/BL2/BL31).
I^2C - Inter Integrated circuit protocol:	This is a communication protocol used to implement digital communication between two devices (chips / board) using only two wires. It is a standardized specification and is used widely to implement low to medium data rate data transfers both among devices on the same circuit board as well as external add on peripheral boards. I^2C can be implemented across a few meters in distance. I^2C is half duplex meaning only one device can communicate at a time. Speeds range from 100 Kbps to 3Mbps while 100 / 400 Kbps are the typical operating mode. The other major advantage of this protocol is that it allows many devices to be on the same two lines reducing the cost of the interfacing. This is ideal when there are many devices like sensors that transfer limited amounts of data periodically. I^2C can support up to 127 independent directly addressable devices on the same channel. I^2C : Superscript notation is used while referencing to the interface spec. I2C: This format is used to specify actual interfaces because we can't write the superscript in code and most hardware design tools don't support it either.
IEEE- Institute of Electrical and Electronics Engineers	IEEE is the world's largest technical professional organization dedicated to advancing technology for the benefit of humanity. It is a major technical organization covering vast fields of engineering and a major standards organization.
MCU – Micro controller unit	A micro controller unit is a self-contained unit that has the core processing as well as core memory within the same device. It often contains the core software programmed into the chip itself. This allows the device to start executing with minimal external devices / circuitry. Some microcontrollers can be powered on a mere breadboard.
MPU – Micro processing unit	An MPU is a processing unit: a CPU that contains only the processing core and interfaces for external peripherals. A microprocessor is usually a powerful CPU in its class. However, it requires a very large number of external circuitries to achieve its functionality like external memory, disk drives, etc.
PMIC – Power management IC	This is a specific chip on the board that manages multiple power supply lines at various levels. It manages the respective supplies along with sequences which control power on and power off cycles.

SBC – Single board computer	It is a standard term that means a tiny computer in the form factor of a single circuit board usually just inches in area. This board is self-sufficient in every way and can give you a usable computer with just a power supply, keyboard, mouse, and display.
EVK – Evaluation Kit	It is a reference hardware platform provided by the silicon vendor. EVKs are designed to help developers evaluate SoC features, validate software, test peripherals, and accelerate early development before moving to custom hardware.
RDK – Robot Development Kit	It is a hardware + software platform intended for robotics development, typically providing motor control, sensors, compute modules, and pre-integrated frameworks for rapid prototyping and testing robotic applications.
SiP – System in Package	SiP is a device where multiple silicon IP's are combined to form a single device. It is one of the densest chips where the external devices like flash memory, DDR RAM and even Wi-Fi module are all packaged into a single chip. These are used in very niche application that require ultra small size and low thermal requirement.
SoC- System on Chip	A system on chip is a complete hardware platform packaged on to a single chip. It contains the CPU, internal fast memory, interrupt controllers, pin controllers, ROM memory, and a few other peripherals and sensors; all packaged into the same IC. An SoC despite the high level of integration does not necessarily power on and run by itself. Microcontrollers are often independent SoC's that can work on their own. However, SoC's often combine MPU's and MCU's into the same chip. This allows very powerful systems to be built in a compact form factor but requires external supporting peripherals like DDR RAM and flash memory and power management IC's.
SMARC	SMARC is a standardized interface to connect a core module to a common platform carrier board. It's basically a connector with specific pin outs. It allows the reuse of the carrier board to support a variety of modules for evaluation.
SPI - Serial Peripheral interface	SPI is another standard interface used to interface other devices on the board or attaching peripheral boards. It specifies 3 wires / lines to achieve fast full duplex data transfer. Two devices can send / receive data at the same time in this protocol. The protocol is also a high-speed protocol where typical operating speeds start at 5Mbps and go over 50Mbps. This high speed allows interfacing high speed devices like memory, Wi-Fi, subsystems made of independent microcontrollers, etc. While only 3 lines are needed to interface two devices, a fourth line is used as a device selector allowing multiple devices to share the same interface. However, only two devices may communicate at a time.

1. Overview

The Renesas System Release Package is a unified software package that aims to provide an easy-to-use yet comprehensive software platform for the Renesas RZ series of MPU-based boards. It aims to provide fully functional base images for supported reference designs, along with easy-to-use development and programming tools that help users quickly get started on their application development. This package aims to provide a standardized and familiar workflow for a similar experience across a variety of Renesas RZ MPU-based product platforms.

This package provides comprehensive documentation, quick-start guides, multiple Linux-based distribution images, automated tools and scripts, and an ongoing expansion of supported products.

1.1 Supported Distributions

The System Release package supports a set of both Yocto images and custom images to enable the user to start quickly on their embedded end application. The large collection of images in prebuilt format provides a wide set of capabilities. This release focuses on Yocto images.

1.1.1 Yocto Images

This section lists the standard Yocto images, offering a variety of configurations that cater to different embedded use cases. From a minimal bootable environment to fully graphical systems, these images provide the essential building blocks for embedded Linux development.

Table 1. Yocto images

Distribution	Image file	Version	Description
Yocto minimal	core-image-minimal	styhead-5.1.4	A basic image that contains the minimal set of components required to boot the device. It focuses on essential system functions without extra tools or features.
Yocto BSP	core-image-bsp	styhead-5.1.4	Extends core-image-minimal with additional utilities and tools, providing a lightweight environment for system validation, hardware diagnostics, and basic development.
Yocto weston	core-image-weston	styhead-5.1.4	A standard graphical image with Wayland and Weston support for embedded GUI applications.

1.1.2 Renesas Custom Images

This section presents Renesas-specific custom images, which are customized and optimized for Renesas products. These images offer specialized features, including fast booting and tailored environments for both graphical and CLI-based applications.

Table 2. Renesas custom images

Distribution	Image file	Version	Description
Renesas CLI Base	renesas-core-image-cli	styhead-5.1.4	Based on core-image-bsp, this image offers a CLI environment for Renesas hardware development without graphical interfaces. Besides the useful tools inherited from the core-image-bsp, this image also contains new packages for SBC (Single Board Computer) development. For example, package managers (apt, dpgk), network utilities for Bluetooth and Wi-Fi.
Renesas Quickboot CLI	renesas-quickboot-cli	styhead-5.1.4	This image has the same system functionality as the renesas-core-image-cli but with Quickboot enabled, allowing for faster boot times and efficient system validation in a CLI environment.
Renesas Weston (Qt6)	renesas-core-image-weston	styhead-5.1.4	Renesas customized core image based on the core-image-weston, with Qt6 framework support (no QT demo apps included). This image offers a full graphical environment for Renesas hardware development and all the useful tools from the renesas-core-image-cli.
Renesas Quickboot Wayland	renesas-quickboot-wayland	styhead-5.1.4	This image has the same system functionality as the renesas-core-image-weston, but with Quickboot enabled, allowing for faster boot times and efficient system validation in a graphical environment.

Note: Quickboot is a trade term that refers to the specific optimizations that are performed to achieve ultra-low start-up times in specific images. Depending on the board architecture, the startup time can be as low as 2s. While there is no assurance of the startup time in these images for every platform, these images are the most optimized on our platforms.

1.1.3 Ubuntu Images

This section presents custom Ubuntu-based images tailored for embedded systems, offering a variety of configurations to suit both headless and graphical environments. These images are optimized for performance and ease of use, providing a solid foundation for deploying embedded applications on Renesas platforms.

Table 3. Renesas Ubuntu images

Distribution	Image file	Version	Description
Ubuntu Core	ubuntu-core-image	ubuntu-base-24.04	A minimal, headless Ubuntu image tailored for embedded systems.
Ubuntu LXDE	ubuntu-lxde-image	ubuntu-base-24.04	A lightweight Ubuntu image featuring the LXDE desktop environment, providing a graphical

			interface while maintaining low resource consumption. This image also includes Qt framework support for GUI development.

1.2 Supported Platforms

Table 4. Supported Platforms

Platform	MPU	OPN	Description
RZ/G2L-SBC	RZ/G2L	US157-G2LSBCPOCZ	RZ/G2L-based Pi-compatible SBC.
RZ/G2L-EVK	RZ/G2L	RTK9744L23S01000BE	Evaluation Board Kit for RZ/G2L MPU
Geniatech RS-G2L100	RZ/G2L	R9A07G044L24GBG#BC0	Geniatech AHAURA RS-G2L100
RZ/V2L-EVK	RZ/V2L	RTK9754L23S01000BE	Evaluation Board Kit for RZ/V2L MPU
RZ/V2H-EVK	RZ/V2H	R9A09G057H48GBG#AC0 R9A09G057H48GBG#BC0	Evaluation Board Kit for RZ/V2H MPU
RZ/V2H-RDK	RZ/V2H	WS125-V2HRDKREFZ	RZ/V2H Robot Development Kit (RDK): AI-powered robotics kit for autonomous robots.
IMDT V2H SBC	RZ/V2H	N/A	Compact AI SBC board with RZ/V2H and rich connectivity.

Note: Boards without a product number (currently not for sale) are marked as N/A.

2. Introduction

The system release package provides a unified and consistent experience across multiple RZ platforms by providing prebuilt binaries that are as universal as possible, along with all the tools and documentation necessary to work with these platforms. To enable the platforms, the package contains a variety of images that provide the most common starting points for embedded application development. The workflow envisioned is provided below.

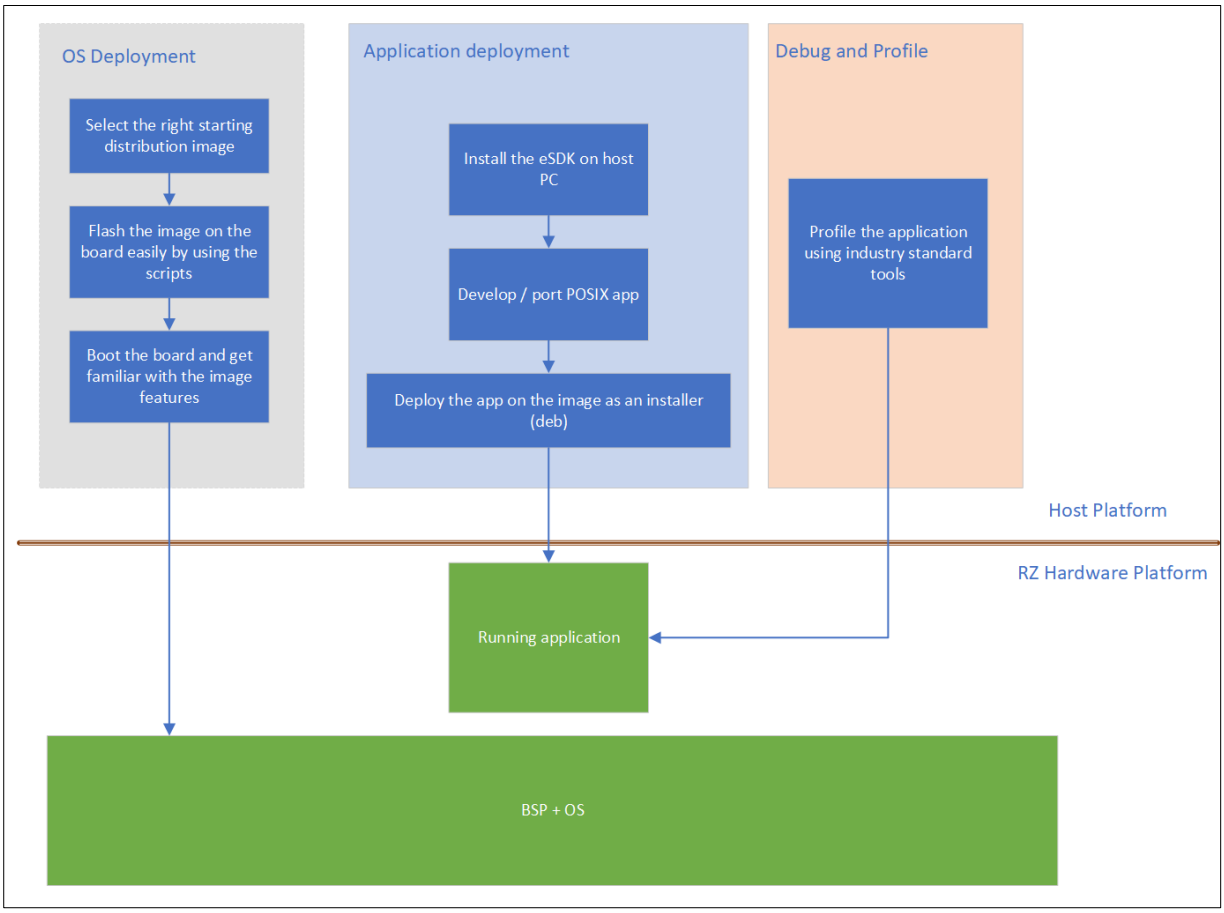


Figure 1. Embedded application workflow for RZ System Release Package

The package also comes with automated scripts that let users rebuild the entire package on the user end, as well as modify the existing images as needed. The eSDKs and usage of the wic file formats for the images allow the tweaking and generation of new images without rebuilding the entire package. This lets users focus on the embedded applications instead of the platform's intricacies.

2.1 Package Hierarchy

The System Release Package is organized into two archives. The first archive is the primary package itself, containing the images, build scripts, documentation, etc. The second archive is the SDK archive. The package is arranged into an intuitive file hierarchy that is easy to follow. There are 'Readme.md' files at every location to help with understanding the contents. The Readme.md file at the root of the hierarchy is a comprehensive guide that gives an overview of the entire package and how to use it.

Below is an overview of the package hierarchy, followed by a description of the contents and purpose of each directory/file:

```
.
├─── host
│    ├─── build
│    ├─── env
│    ├─── Readme.md
│    ├─── src
│    └─── tools
├─── license
│    └─── Disclaimer051.pdf
├─── <code>-rz-cmn-srp-um-quick-start-guide.pdf
├─── <code>-rz-cmn-srp-um.pdf
├── README.md
├── RZ_System_Release_Package_Evaluation_license.pdf
└── target
    ├── env
    ├── images
    └── Readme.md

9 directories, 9 files
```

host/: This directory holds all the tools, scripts, and artifacts needed on the host machine for building and preparing the system images.

- **build/:** Contains build artifacts (manifests and test data).
  Key files:
  - Manifest file: Files like core-image-bsp-<timestamp>.rootfs.manifest lists the contents of the generated root file system.
  - Test data: Files with the *.testdata.json extension that contain metadata or test results of the said image.
- **env/:** Provides environment configuration files used during the build or runtime.
  Key files:
  - .env Files: Examples include core-image-bsp.env or core-image-minimal.env, which define variables and configuration parameters for different image variants.
- **src/:** Holds build scripts, source code, and patches that are used to build the package.
  Key files:
  - rz-cmn-srp/: The folder that contains artifacts to build Yocto and Ubuntu images.
    - Patches: Located in the patches/ subdirectory, these files (For example, 0001-...patch) contain necessary modifications.

        - Build scripts: The master script rz_builder.sh automates the build process for both Ubuntu and Yocto packages, handling setup, configuration, and image generation based on user-selected build options.
        - Configuration files: site.conf, which is used to set up a specific build tag.
        - config.json: Contains the available build image options grouped by build type, including Yocto images, Ubuntu images, and static image collections (all-yocto-images, all-ubuntu-images, all-supported-images).
        - git_patch.json: Contains JSON keys and repository configuration, such as url, branch, tag, commit, repo type, and patch paths to apply.
        - layer_override.json: Pre-build customization file: add/remove meta-layers in BBLAYERS, append/exclude image packages, and select GPU mode (none/panfrost/mali). Applied before BitBake runs.
    - ubuntu/: Main folder for Ubuntu-based image generation for RZ boards.
        - config/: The folder that holds configuration files for different Ubuntu variants.
        - docs/: Contains documentation detailing supported features and usage instructions for each Ubuntu image variant.
        - Include/: Contains scripts related to Ubuntu (e.g., for creating WIC files, packaging the root filesystem, preparing the environment, etc.)
        - script/: The folder that contains all scripts related to Ubuntu image creation.
        - config.ini: Configuration file that defines key parameters for the Ubuntu image build process, such as the Ubuntu variant, base image, output filenames, and system settings.
        - setup_ubuntu_environment.sh: Main entry-point script (acts like a dispatcher/header). It sources and sequences logic from the modular scripts under script/. It does not build anything by itself.
- **tools/:** Provides utilities to assist with tasks such as bootloader flashing, uLoad bootloader flashing, and SD card image creation. All tools are designed to support both Linux and Windows environments.
  Key files:
    - bin/: This directory contains precompiled tools required for the firmware build and packaging process. They are organized by operating system to support cross-platform usage.
    - firmware_compile/: Contains a helper script for building Renesas RZ family firmware artifacts, including BL2, Boot Parameter (BP) files, U-Boot binaries, and FIP packages.
    - bootloader-flasher/: Contains a script and README.md for flashing the main bootloader. Works on both Linux and Windows.
    - sd-creator/: Provides scripts to create SD card images from .wic files. Compatible with both Linux and Windows.
    - uload-bootloader: Includes automated host-side scripts to flash the qspi boot firmware (IPL) using the images from the SD card.
    - universal_flash.py: A master Python script that unifies all flashing workflows (bootloader, uLoad, SD card) under a single interface. Cross-platform and recommended for most use cases.
    - flash_images.json: Contains predefined image mappings for supported devices. User configuration should be defined in this file. All image files referenced for flashing must be located in: </path/to/your/yocto/package>/target/images>
    - config: Contains scripts that use a boards_flash_config.toml file to define flash layout and bootloader addresses for supported boards. The configuration is structured per board and boot type (e.g., qspi, emmc), and other information.

license/: Contains all the supporting legal documents and licensing agreements related to the release package.

target/: This directory includes all the files needed for deploying the system on target hardware.

- **env/:** Contains environment configuration files that are used during boot-up on the target device.
  Key file:
  - uEnv.txt: A file that holds boot configuration parameters.
- **images/:** Holds the final system images and associated files required for the target device.
  Key files:
  - atf: The RZ Common Arm Trusted-firmware (TF-A) directory contains BL2, BL31 binaries, and TF-A configuration DTBs (FDTS).
  - u-boot/: The RZ Common U-Boot directory contains the U-Boot binaries and device tree blobs (DTBs) used across all supported boards.
  - linux/: The directory contains the Linux kernel and device trees for the target images.
  - System images: Files with the '.wic' extension corresponding to different build variants (BSP, minimal, Weston, Renesas images).
  - rootfs folder: Compressed archives (For example, core-image-bsp.tar.bz2) contain the root file system for each image.
  - Flash Writer binaries per board, e.g:
    - Flash_Writer_SCIF_<board-name>.mot
    - Flash_Writer_SCIF_<board-name>_PMIC.mot (includes PMIC init)
  - Board identification binaries per board and version, e.g:
    - <board>-<version>-platform-settings.bin
    - <board>-<version>-platform-settings.srec
- README.md (root level): This is the comprehensive guide that provides an overview of the entire release package, including instructions on how to use, build, and deploy the system.

2.2 Source Repositories

The system release package is maintained in public repositories that hold all the latest work that has been released. The table below describes the public repositories that are the basis for the system release package.

Table 5. Public repositories for the system release package

Name	Type	URL	Description
rz-build-scripts	Yocto build automation	Renesas-SST/rz-build-scripts: Build scripts for rz projects	Custom Yocto build script that downloads the base Yocto package and other downloaded zip files, arranges the layers, applies relevant meta layers, sets up the environment, and initiates a build.
meta-renesas	Yocto meta layer	Renesas-SST/meta-renesas: Yocto meta layer for Renesas System Solutions	Yocto meta layer supports RZ-based platforms.
linux-rz	Linux kernel	Renesas-SST/linux-rz: Linux kernel for System and Solutions Products	This repo contains the kernel fork with RZ-based patches.
u-boot	Boot loader	Renesas-SST/u-boot: A u-boot suporting System & Solutions Products	This repository contains the U-Boot bootloader source code customized for

			Renesas RZ devices. It includes board-specific configurations and patches required to boot Linux on supported RZ-based platforms.
rz-atf	Arm Trusted Firmware-A	Renesas-SST/rz-atf: Arm Trusted Firmware implementation for System & Solutions products	Arm trusted the firmware repo with RZ-based patches.
flash-writer	Firmware flashing tool	Renesas-SST/flash-writer: Serial flashing utility to load into blank boards supporting System & Solutions Products	This repository contains the code for an essential tool that is used as the base for flashing blank boards in the factory for the first time or recovering bricked boards.
rz-utils	Flashing tools	Renesas-SST/rz-utils: Collection of utilities for various workflows related to Renesas RZ-based devices	Collection of utilities for various workflows related to Renesas RZ-based devices
rzv_drp-ai_driver	DRP-AI meta-layer for RZ/V MPUs	Renesas-SST/rzv_drp-ai_driver: RZ/V DRP-AI Driver	This repository contains the recipes and patches of DRP-AI for RZ/V MPUs such as RZ/V2H-EVK or RZ/V2L-EVK
rzv2h_opencv_accelerator	RZ/V2H OpenCV Accelerator	Renesas-SST/rzv2h_opencv_accelerator: RZ/V2H OpenCV Accelerator	RZ/V2H OpenCV Accelerator layer
meta-rz-multi-os	RZ Multi-OS layer	Renesas-SST/meta-rz-multi-os: yocto meta layer for Renesas RZ Multi-OS Package	Yocto meta layer for Renesas RZ Multi-OS

While the public repositories are mostly open source, the RZ MPUs contain IPs that are licensed differently, and those functionalities require specific packages to be downloaded from the Renesas website through login. These are mostly free-to-download packages and contain their licenses, which are non-standard. The build scripts can identify and point to the missing packages and their download URLs. You can download these scripts manually and copy them to the workspace.

2.3 Release Software Versions

This table summarizes the software component versions included in this release, such as U-Boot, ATF, Linux kernel, and other relevant platform packages. It serves as a reference for configuration control, traceability, and release verification.

Table 6. Release software versions

Component	Version
U-Boot	2026.04
Trusted-Firmware-A	2.14.2
Linux Kernel	6.18.20
Yocto	Styhead (5.1.4)

3. Required Resources

3.1 Development Tools and Software

The following tools are used for development:

- SEGGER JLink software (SEGGER - The Embedded Experts - Downloads - J-Link / J-Trace).
- Tera Term (TeraTerm Project) on Windows PC for accessing the UART.
- Minicom on the Ubuntu host machine for accessing the UART on Linux.
- Balena Etcher

3.2 Hardware

The hardware requirements vary slightly depending on the Renesas RZ board in use. Refer to the corresponding subsection below for board-specific details.

3.2.1 Common Hardware Requirements

These items are required for all supported Renesas RZ boards:

- Windows PC with Tera Term software and admin privileges.
- Ubuntu 24.04 host environment (native install, VM, or Docker) for working with Yocto distros.
- Ethernet cables for networking.
- USB keyboard and mouse

3.2.2 Board-Specific Hardware

3.2.2.1 RZ/G2L-SBC

The following hardware would be needed to work with the RZ/G2L-SBC:

- RZ/G2L-SBC - RZ/G2L Single Board Computer
- SD/MMC card (minimum 8 GB)
- UART TTL cables (Raspberry Pi compatible) featuring FTDI chipset. We do not recommend PL2302-based UART TTL cables, as they have demonstrated issues with Windows drivers.
- Jumper wires/plugs.
- Mini-HDMI to HDMI display interface cable.
- Power supply that can provide 5V at 3 A with USB-C pins. (not included in the package).
- Waveshare 5” DSI display module with a capacitive touch interface (optional: not included in the hardware package).
- OV5640 camera module (optional: not included in the hardware package).

3.2.2.2 RS-G2L100

The RS-G2L100 is an industrial single-board computer based on the Renesas RZ/G2L processor. The board is manufactured by Geniatech, and its full product name is Geniatech AHAURA RS-G2L100. It is officially supported by Renesas and intended for the evaluation and development of RZ/G2L-based Linux systems.

To work with the RS-G2L100 board, the following hardware components are required.

- RS-G2L100
- SD/MMC card (minimum 8 GB)
- UART TTL cables (Raspberry Pi compatible) featuring an FTDI chipset. We do not recommend PL2302-based UART TTL cables, as they have demonstrated issues with Windows drivers.
- Power supply that can provide 12V at 3 A with USB-C pins. (not included in the package).

- 1080p HDMI display
- Ethernet cables.
- USB keyboard and mouse

3.2.2.3 RZ/G2L-EVK & RZ/V2L-EVK

The following hardware is required for working with the RZ/G2L-EVK or RZ/V2L-EVK. Most items are identical for both boards:

- RZ/G2L SMARC module board or RZ/V2L SMARC module board and common carrier board.
- SD/MMC card (minimum 8 GB)
- USB Type-A to Micro USB Type-B cable
- Micro-HDMI to HDMI display interface cable.
- 1080p HDMI display.
- Power supply that can provide 5V at 3 A with USB-C pins. (not included in the package).
- OV5645 camera module (optional: not included in the hardware package).
- 3.5mm Audio Stereo Y Splitter extension cable (optional: not included in the hardware package).

3.2.2.4 RZ/V2H-EVK

The following hardware is needed to work with the RZ/V2H-EVK:

- RZ/V2H Quad-core Vision AI MPU Evaluation Kit
- SD/MMC card (minimum 8 GB)
- USB Type-A to Micro USB Type-B cable.
- HDMI to HDMI display interface cable.
- 1080p HDMI display.
- Power supply that can provide up to 100W via USB-C PD (not included in the package).
- OV5645 camera module (optional: not included in the hardware package).
- Camera conversion 22-pin to 24-pin FPC adapter (included in the hardware package).
- 3.5mm Audio Stereo Y Splitter extension cable (optional: not included in the hardware package)
- M2_M-Key + M.2_E-Key PCI-E X1 Expansion Card (required when using PCIe)

3.2.2.5 RZ/V2H-RDK

The following hardware is needed to work with the RZ/V2H-RDK:

- RZ/V2H Quad-core Robot Development Kit
- SD/MMC card (minimum 8 GB)
- USB Type-A to Micro USB Type-B cable.
- Micro HDMI to HDMI display interface cable.
- 1080p HDMI display.
- Power supply: DC power supply, 12 V, ≥ 1 A (12 W or higher), regulated (AC input 100–240 V)
- OV5645 camera module (optional: not included in the hardware package).
- Camera conversion 22-pin to 24-pin FPC adapter (optional: not included in the hardware package).
- PCIe to M.2-E KEY HAT (required when using PCIe)
- Wireless AX210 NIC – Gigabit Tri-Band Wi-Fi 6E, 802.11ax, Bluetooth 5.3 (optional; required when using wireless connectivity)

3.2.2.6 IMDT V2H-SBC

The basic hardware setup consists of the following:

- IMDT-V2H-SBC
- SD/MMC card (minimum 8 GB)
- USB Type-A to Micro USB Type-C cable
- B-LCDAD-HDMI DSI to HDMI adapter
- 1080p HDMI display.
- Power supply DC9V/3A via USB-C PD (not included in the package) or DC Barrel jack with DC12V- 24V/2A minimum.
- Ethernet cables.
- 3.5mm Headphone with microphone

4. Quick Start

This section describes how to quickly get set up and start running the supported platforms with this release. The following are the essential steps for an SD/MMC card-based boot:

1. Select an image from the list of available images in Section 1.1. Supported Distributions
2. Prepare an SD MMC card that has the image programmed onto it.
3. Prepare the hardware with power and debug UART interface. Displaying the connection to one of the HDMI interfaces is highly recommended, but not essential.
4. Program the firmware using the appropriate scripts and process in the ‘host/tools’ directory of the package.
5. Boot normally with the SD MMC card.

4.1 SD/MMC Card Flashing

The Linux bootable SD card creation is a very simple process. The idea is to use any filesystem imaging tool (etcher) to burn the required image’s ‘.wic’ file (core-image-weston.wic, for example) located in the ‘target/images’ directory of the release to the SD/MMC card. We recommend installing Balena etcher, which is available for Linux, MacOS, and Windows.

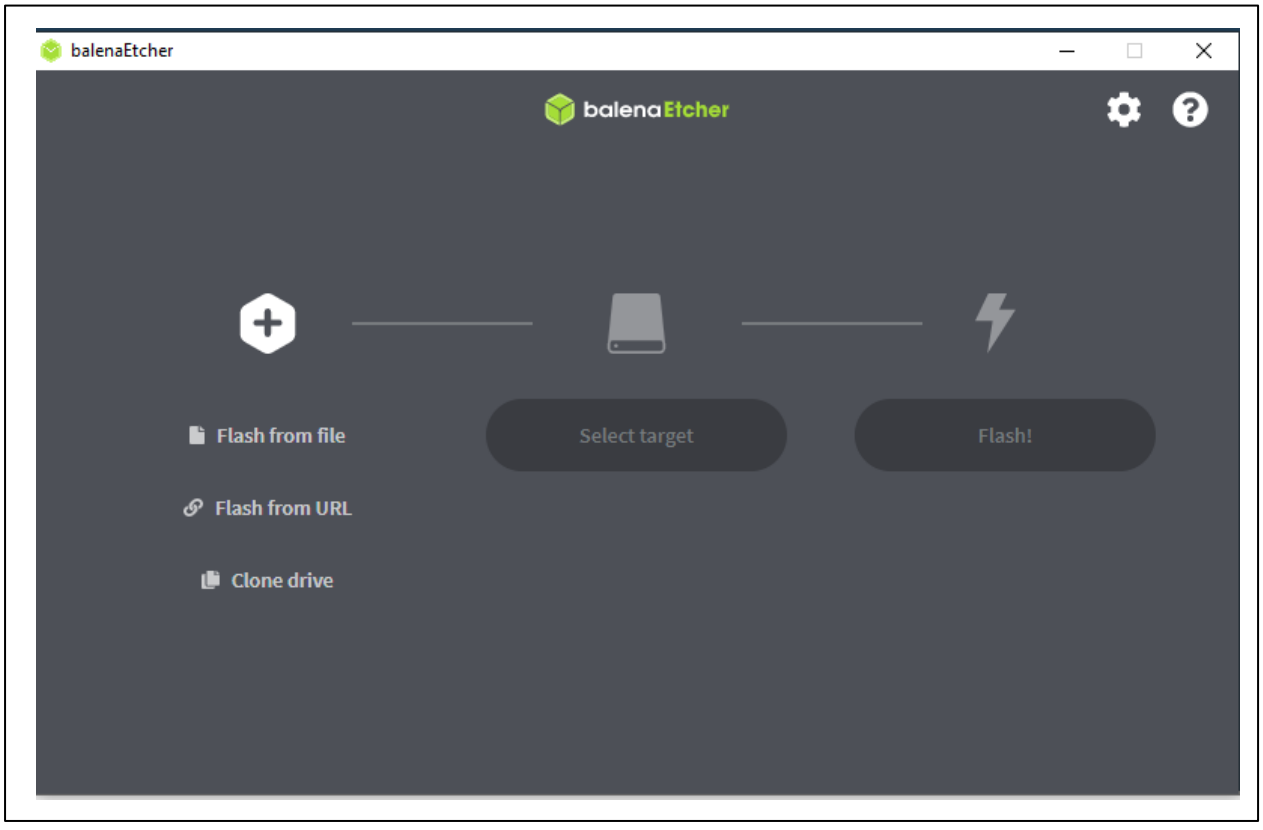


Figure 2. Balena etcher UI

Steps:

1. Select “Flash from File”.
2. In the pop-up window, navigate to your release and select one of the chosen image files (core-image-weston.wic).
3. Then click on ‘Select target,’ and it will list all available devices.
4. Select the appropriate card from the list.
   Be mindful not to select your primary laptop/desktop hard drive.

5. Select ‘Flash’.
6. When flashing is completed, it will automatically dismount the card from the host.
7. Insert the SD card into the SD/MMC card slot on the RZ board.

4.2 RZ/G2L-SBC

This section describes the hardware-specific processes for the RZ/G2L-SBC (Single-board Computer).

Note:

- The release consists of images that have user desktop and display support.
- At least one basic display, like a 1080p HDMI monitor, must be available for those images.
- You can also use the DSI touch panel described in the MIPI DSI Display Touch Panel.
- It is recommended to use an FTDI cable for the UART and not any other converter chip.

4.2.1 Hardware Requirements

The hardware setup consists of the following:

Table 7. Accessory List for RZ/G2L-SBC

Item	Required / Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	RZ/G2L-SBC	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD	Used for Linux image
Debug cable	Required	FTDI RS232 UART cable	UART header	FTDI recommended, avoid PL2302
Power supply	Required	USB-C 5V 3A+ power supply	USB-C power	Required for power source
Display cable	Required for graphical images	Mini-HDMI to HDMI cable	HDMI	Required for HDMI display path
Display	Required for graphical images	1080p HDMI monitor	HDMI	Recommended for Weston / GUI images
DSI Display	Optional / recommended	1080p HDMI display/Waveshare 5” MIPI DSI display touch panel	MIPI DSI	Supported alternative to HDMI
CSI camera	Optional	OV5640 MIPI CSI camera	MIPI CSI	Required for camera usage

Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Required for audio usage

4.2.2 Essential Hardware Setup

Figure 3. Essential minimum interfaces show the basic hardware setup. We expect a UART cable and an HDMI display to be available.

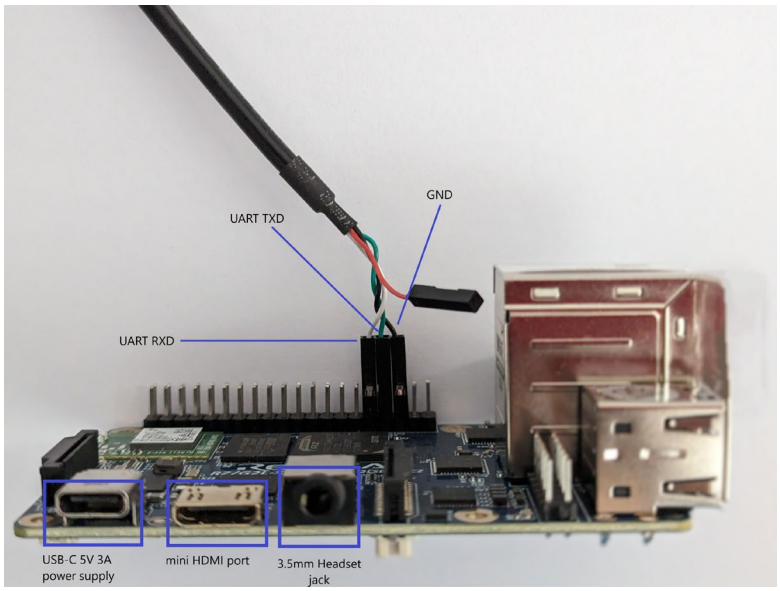


Figure 3. Essential minimum interfaces for RZ/G2L-SBC

4.2.3 Complete Hardware Setup

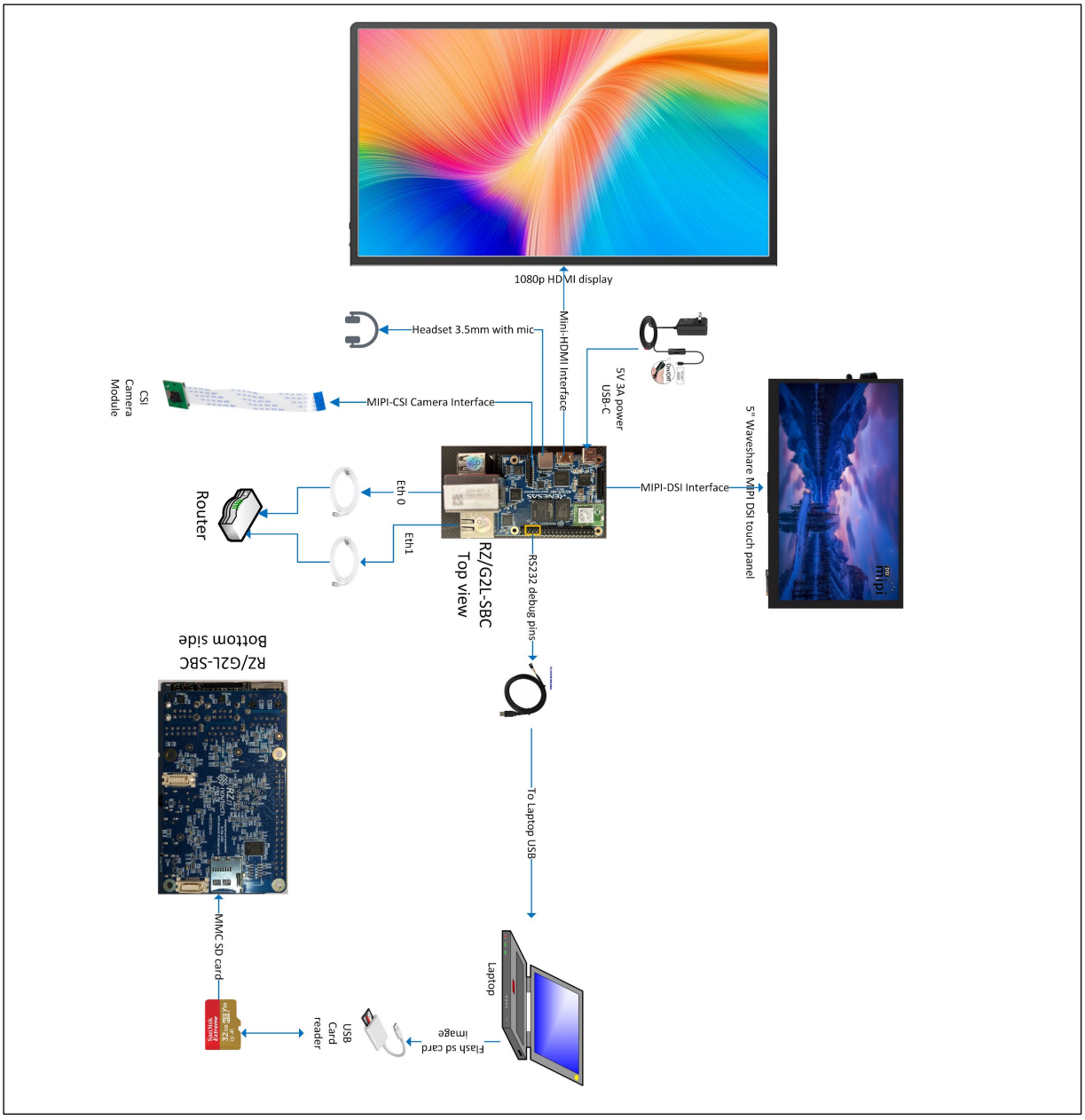


Figure 4. Complete setup for RZ/G2L-SBC board

4.2.4 Booting

Booting is straightforward.

1. Insert the MMC card into the MMC port on the bottom side of the RZ/G2L-SBC.
2. Connect the keyboard, mouse, and HDMI display; then insert the USB-C power supply and turn the power on.
3. You should see the boot log on the UART console and the Weston desktop on the HDMI screen.
4. Click on any of the applications and interact with them.

The image is fully featured and has powerful desktop-grade features. Read further to learn more about the features packed into the Linux image.

Notes: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in Section 16.1.1

4.2.5 Known Hardware and Functional Limitations on RZ/G2L-SBC

4.2.5.1 Linux (CA55) Side Known Issues

1. HDMI Audio

- Status: Unverified
- Description: The functionality of the HDMI audio output has not been tested yet, and its behavior remains uncertain. Additional development and testing are required to assess its reliability and performance on the RZ/G2L-SBC.

2. Audio Sampling Rate Limitation

- Status: Currently limited to 48 kHz (validated)
- Description: The board's clock design deviates from the standard RZ reference, which prevents the existing driver frameworks from generating the proper clocks when the SoC operates as I²S/TDM master. As a result, only 48 kHz sampling has been successfully validated so far. This is not a fundamental hardware restriction — codec-master mode has not yet been evaluated, and wider sampling-rate support may be possible but remains unsupported.

3. Onboard Bluetooth (BT) Functionality

- Status: Non-functional (onboard BT only)
- Description: The onboard Bluetooth functionality is currently non-operational due to a schematic symbol error in the Laird Wi-Fi/BT module. The Bluetooth interface is missing from the module's schematic design, preventing Bluetooth connectivity. However, USB Bluetooth functionality remains operational. This issue requires a hardware revision to enable full Bluetooth functionality on the onboard module.

4.2.5.2 FreeRTOS/FSP (CM33) Side Known Issues

1. MIPI-CSI2 Camera and Peripherals Accessing Shared I2C1 Bus

In the RZ/G2L-SBC, the MIPI CSI Camera interface, HDMI Bridge, and MIPI DSI all share the same I2C1 channel. Due to this hardware constraint, controlling one of these devices may impact the functionality of the others.

Limitations:

- I2C1 can only be accessed by one core at a time, which can prevent both the camera and display from functioning simultaneously.
- Any device using I2C1 must be managed carefully to avoid conflicts with other peripherals.
- This limitation should be considered when designing the system to ensure both peripherals can operate as required.

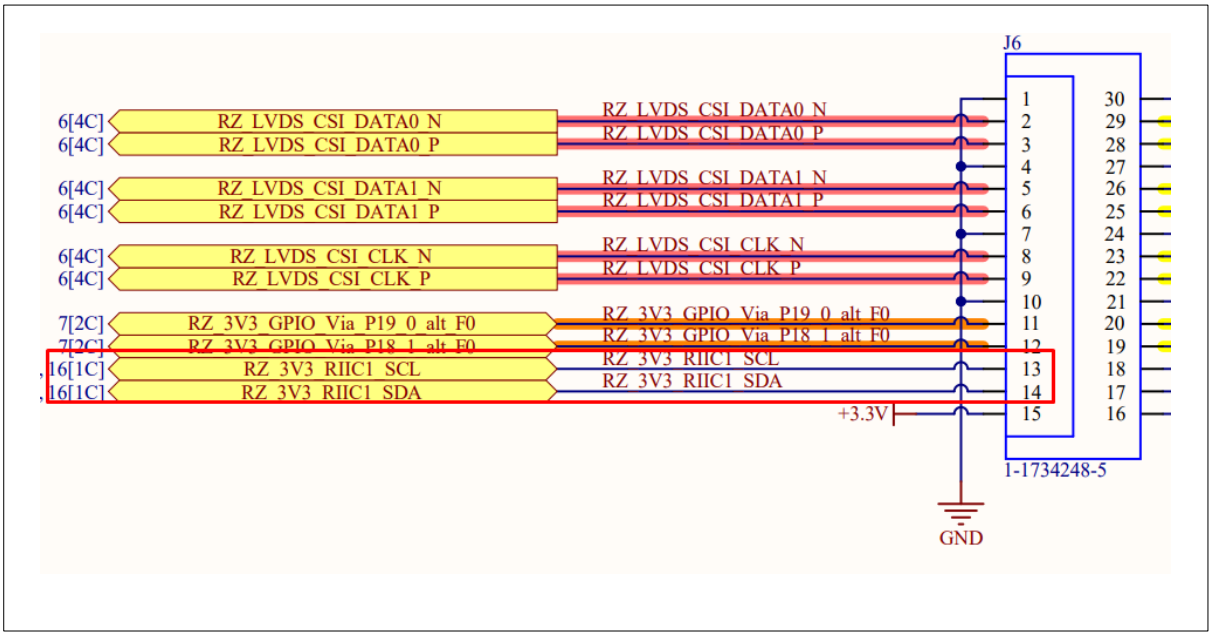


Figure 5. CSI using shared I2C1 bus

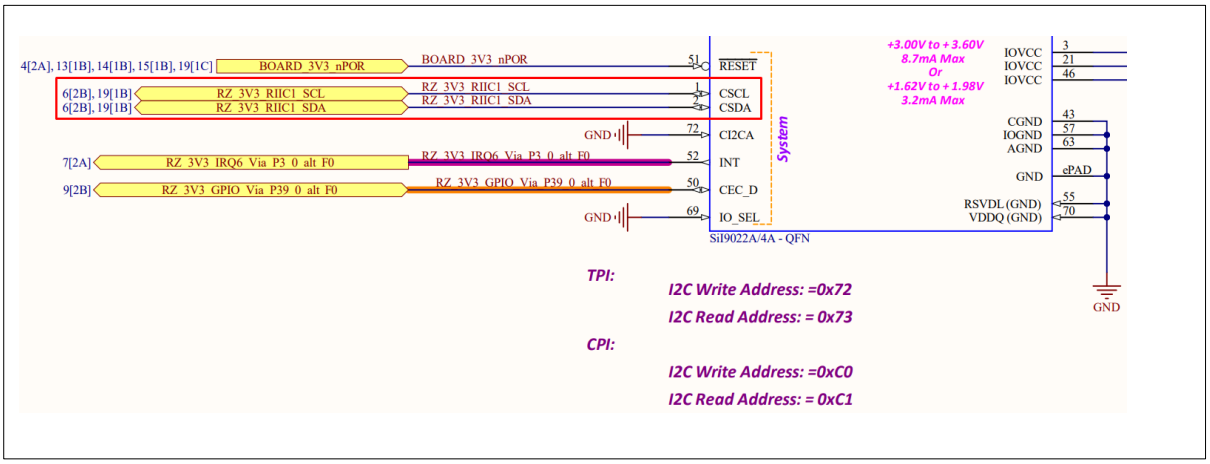


Figure 6. HDMI using the shared I2C1 bus

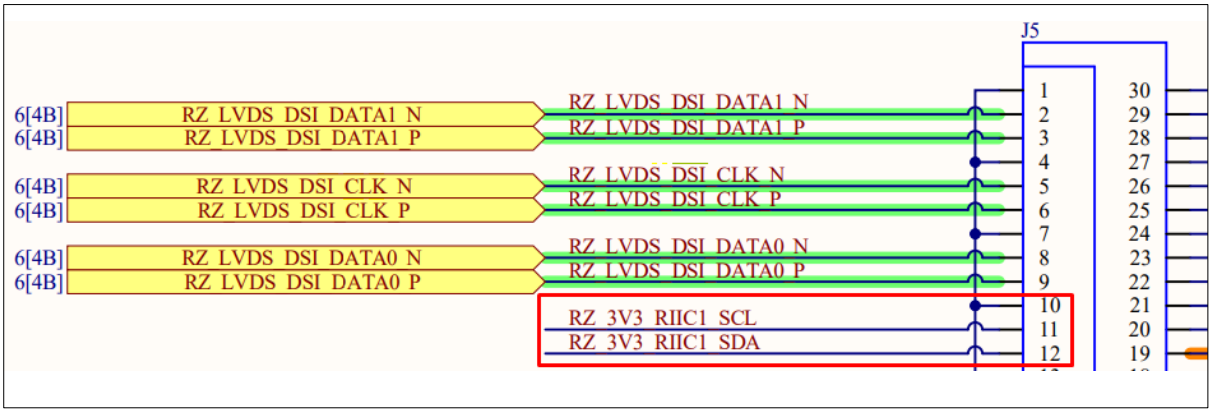


Figure 7. DSI using shared I2C1 bus

As shown in the three figures above, the shared I2C1 bus is used by multiple peripherals, which may lead to conflicts if both cores are used simultaneously. To avoid issues, users should ensure that only

one core accesses I2C1 at a time or consider alternative methods for managing communication between peripherals.

2. Limited SCIF Availability for Multi-Core Development

However, a limitation exists in the number of available SCIF (Serial Communication Interface with FIFO) channels, which impacts debugging and logging functionality for multi-core development.

Limitations:

- Single SCIF Channel: Only SCIF0 is available for serial communication, and it is exclusively allocated to the CA55 core.
- Restricted logging for CM33: Since SCIF0 is dedicated to CA55, the CM33 core lacks direct access to an SCIF channel, making it challenging to perform independent serial logging or debugging.

This limitation should be considered when designing multi-core applications, especially those requiring real-time logging, debugging, or inter-core communications.

4.3 RS-G2L100

This section describes the hardware-specific setup and booting process for the RS-G2L100

4.3.1 Overview of Connectors

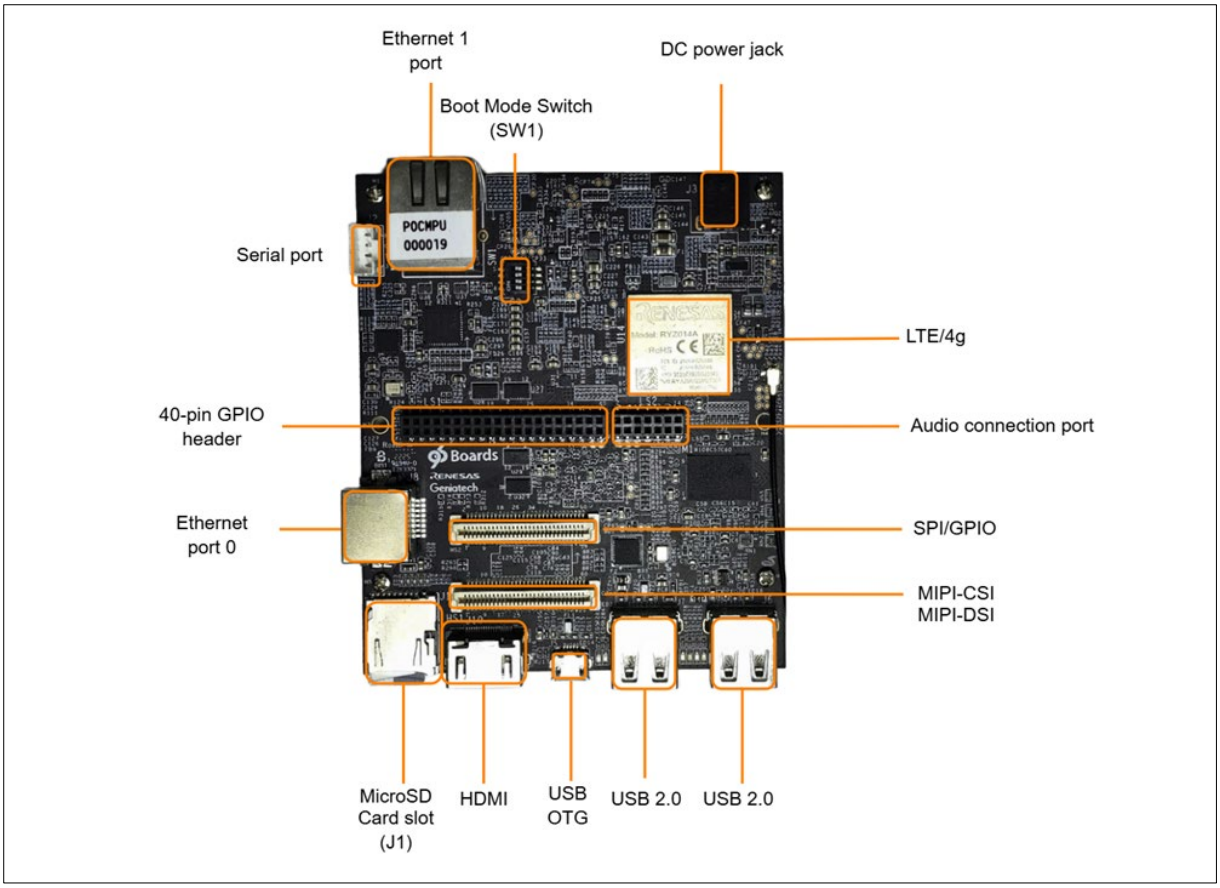


Figure 8. Overview of Connectors - RS-G2L100

4.3.2 Hardware Requirements

The hardware setup consists of the following:

Table 8. Accessory List for RS-G2L100

Item	Required / Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	RS-G2L100	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD	Used for Linux image
Debug cable	Required	FTDI RS232 UART cable	UART header	FTDI recommended, avoid PL2302
Power supply	Required	DC 12V/3A adapter	DC jack	Required for power source
Display cable	Required for graphical images	HDMI to HDMI cable	HDMI	Required for HDMI display path
Display	Required for graphical images	1080p HDMI monitor	HDMI	Needed for display output
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Only HDMI audio is supported

4.3.3 Essential Hardware Setup

The figure below shows the basic essential hardware setup for RS-G2L100. Follow these steps to prepare and power on the board:

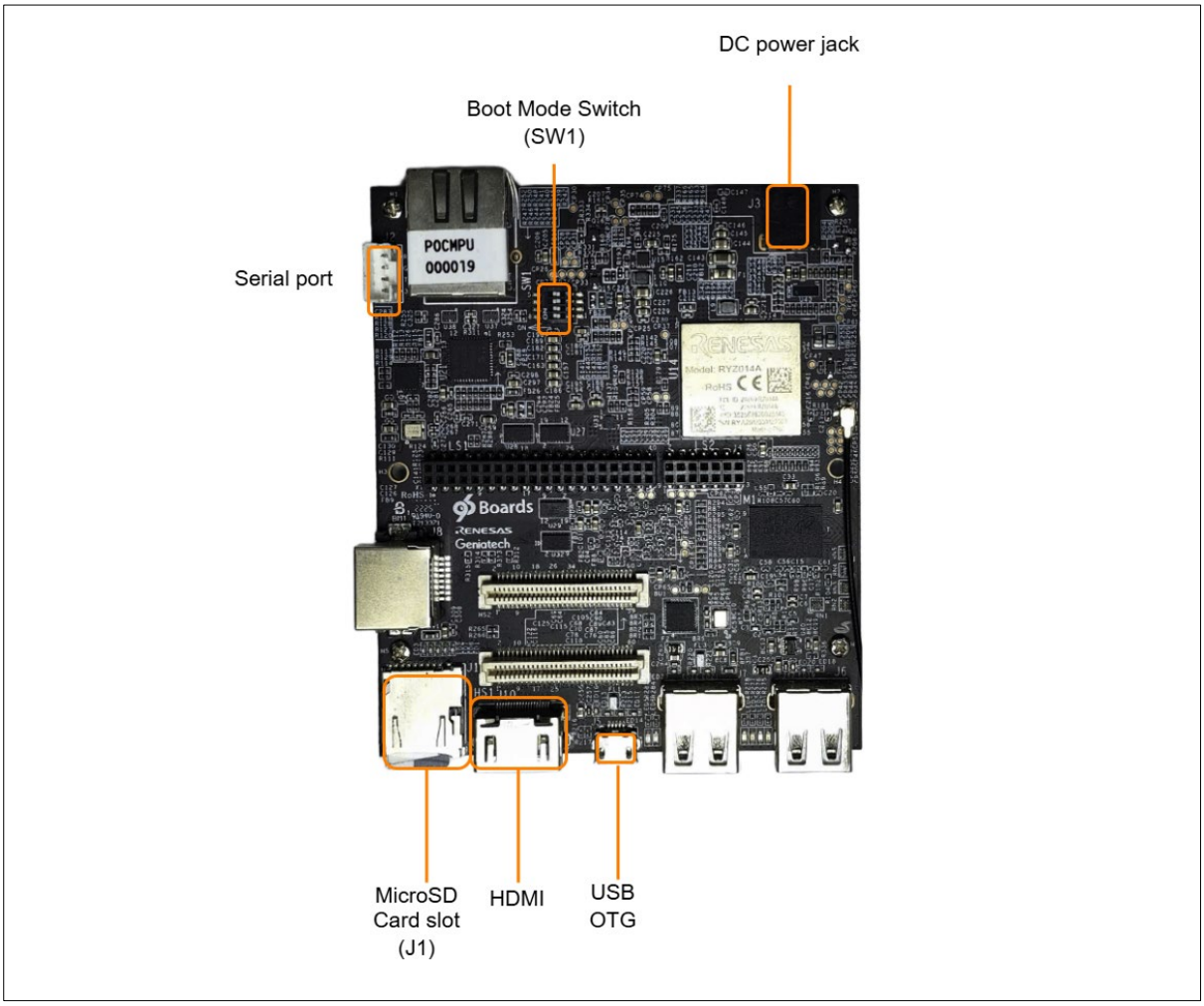


Figure 9. Essential minimum interfaces for RS-G2L100

1. **Prepare the microSD Card**

   Flash the provided image to the microSD card using the procedure in section Flashing the SD Card **Image** or using a user-friendly tool such as Balena Etcher, as described in section SD/MMC Card Flashing for a simplified flashing experience.

2. **Insert the microSD Card**

   Insert the prepared SD card into J1. This slot is for rootfs/data only; it cannot be used to load BL2/FIP.

3. **Configure the Boot Mode**

   In this section, we focus on QSPI boot as the default boot method.

   - SW1 (on the carrier board): Set to QSPI boot.

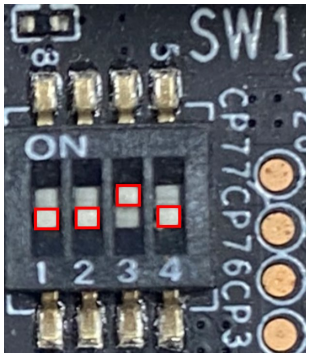


Figure 10. RS-G2L100 xSPI boot switch

4. **Connect Peripherals**
   - Attach an HDMI display
5. **Connect UART**
   - Use an FTDI RS232 UART cable to connect the board's UART console port to the host PC.
6. **Power on the Board**
   - Connect the Adapter power 12V 3A jack DC to the board
   - Wait for the UART serial device to appear on the host PC.
   - Open a terminal program on the host PC to monitor the boot log.

Once powered on, the boot log should appear on the UART console, and the Weston desktop will display on the HDMI screen.

4.3.4 Complete Hardware Setup

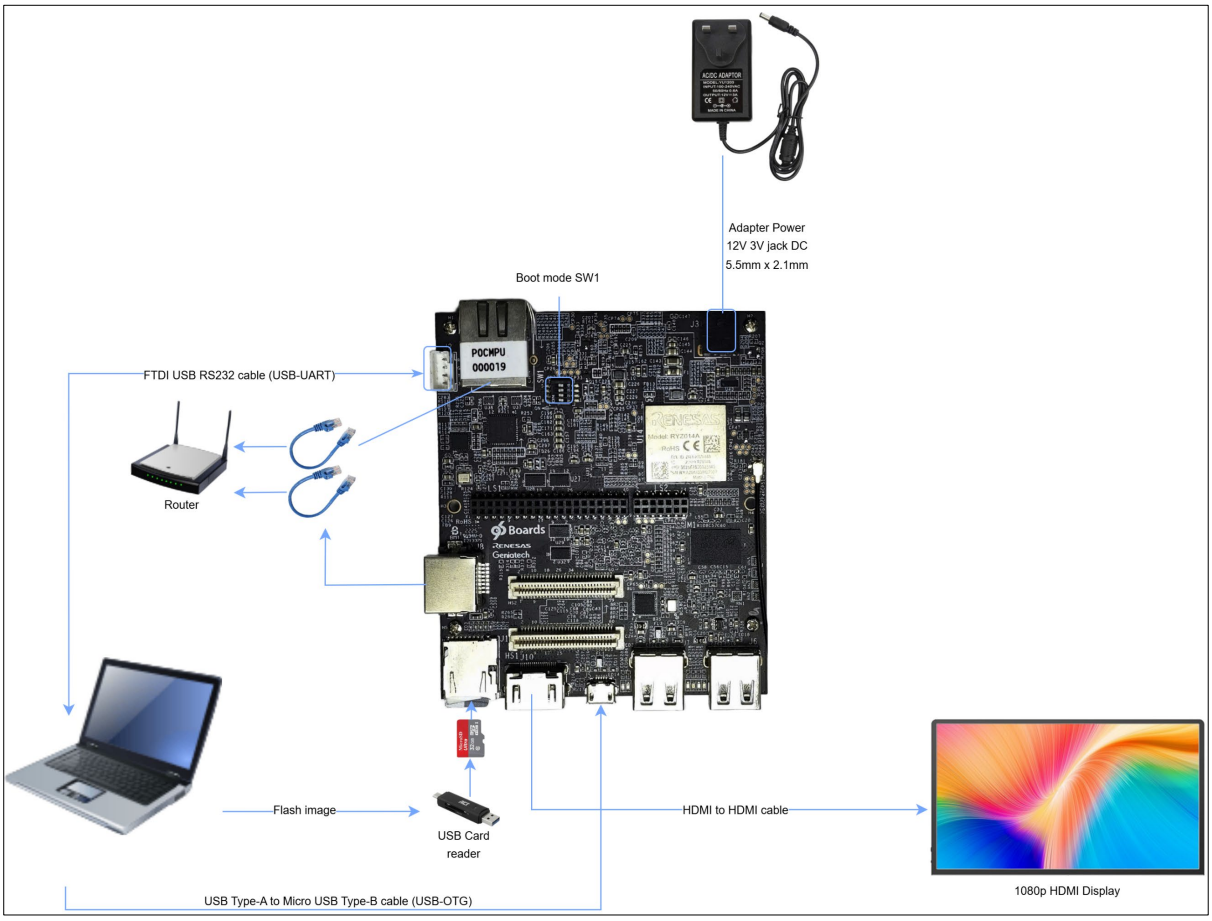


Figure 11. Complete setup for RS-G2L100

4.3.5 Booting

The boot process is straightforward:

1. Insert the prepared SD card into J1.
2. Connect a keyboard, mouse, HDMI display, and other devices. Then connect the DC power supply.
3. The boot log should appear on the UART console, and the Weston desktop should display on the HDMI screen.
4. Open any of the available applications to interact with the system.

The image provided is feature-complete, offering a desktop-grade experience.

If a different boot device is required, such as QSPI or eMMC, adjust the DIP switch settings as described in the picture below to boot from QSPI. The selected boot device determines where on-chip ROM code loads the Boot Loader stage 2 (BL2) and Firmware Image Package (FIP):

- QSPI boot loads BL2 and FIP from the onboard QSPI flash.
- eMMC boot is not supported in this release.

Notes: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in Section 16.1.2

4.3.6 Known Hardware and Functional Limitations

1. eMMC boot is not supported in this release.
2. Camera / CSI – DSI: RS-G2L100 does not support a standalone MIPI CSI-2 connection. The camera and CSI–DSI feature is not supported in this release.
3. Audio: Audio is non-functional on-board version 2 due to the absence of an audio codec in both the schematic and the hardware. Only HDMI audio is tested and verified.
4. Wifi: The Wi-Fi module interfaces with the RZ/G2L processor via the QSPI1 interface. However, the DA16600 driver only supports SPI and SDIO communication for Wi-Fi. There is no implementation for QSPI, so the Wi-Fi feature is not supported in this release.
5. SIM – LTE: The SIM–LTE feature is unsupported because the RYZ014A LTE module is no longer in production and has reached EOL.
6. Bluetooth: The Bluetooth module DA16600 hardware design only includes a 2-pin UART layout, making it impossible to support functionality.
7. Ethernet: The KSZ9131 PHY address configuration pins (PHYAD1 and PHYAD0) are left floating on both PHY0 and PHY1. Pull-down resistors to GND are required on PHY0 (PHYAD1, PHYAD0), and pull-up resistors to VDD are required on PHY1 (PHYAD1, PHYAD0) to properly set the PHY addresses to 4 and 7, respectively. Without these resistor modifications, Ethernet functionality will be unreliable or non-functional.

4.3.7 Solutions and Workarounds

4.3.7.1 Ethernet Hardware Fix

The RS-G2L100 board requires manual resistor installation to configure the PHY addresses correctly so that PHY0 will have address 4 and PHY1 will have address 7, as required by the RS-G2L100 device tree and driver configuration.

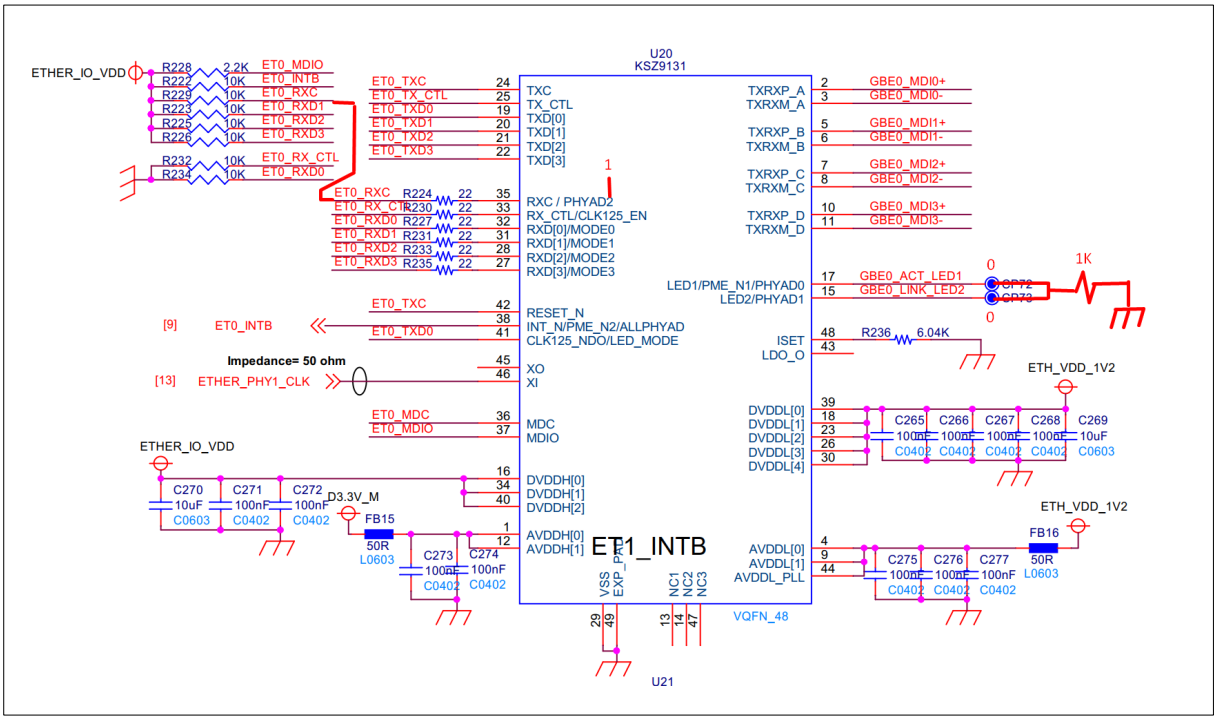


Figure 12. RS-G2L100 - Ethernet 0 Hardware Modifications

The schematic above shows the modification for Ethernet 0 by connecting GBE0_ACT_LED1 (Pin 17) and GBE0_LINK_LED2 (Pin 15) to GROUND through 1kΩ pull-down resistors to properly configure PHY0 address to 4.

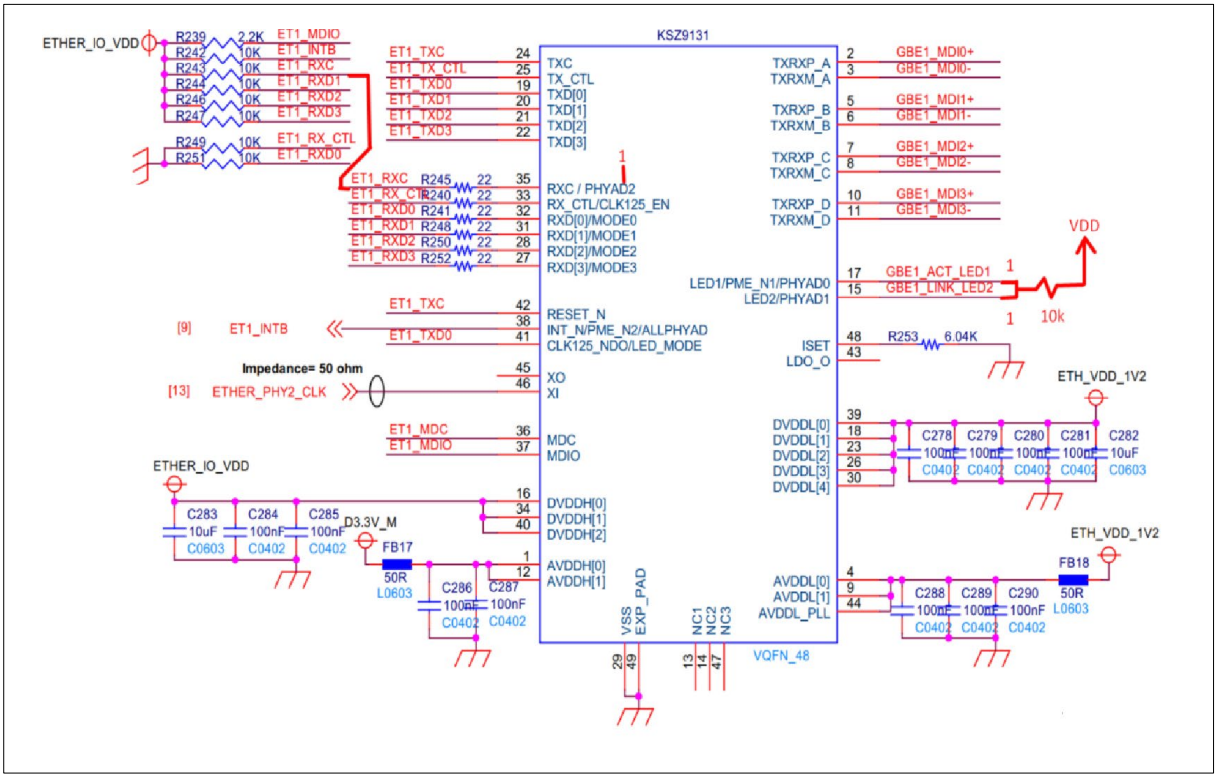


Figure 13. RS-G2L100 – Ethernet 1 Hardware Modifications

The schematic above shows the modification for Ethernet 1 by connecting GBE1_ACT_LED1 (Pin 17) and GBE1_LINK_LED2 (Pin 15) to VDD through 10kΩ pull-up resistors to properly configure PHY1 address to 7.

Without these resistor modifications, Ethernet functionality will be unreliable or non-functional.

4.4 RZ/G2L-EVK and RZ/V2L-EVK

Both RZ/G2L-EVK and RZ/V2L-EVK platforms feature robust Linux images with desktop and display support. Their hardware setup and booting procedures are nearly identical, allowing a common approach for setup and evaluation.

Note:

- The release consists of images that have desktop and display support.
- At least one basic display, like a 1080p HDMI monitor, must be available for those images.

4.4.1 Hardware Requirements

The hardware setup consists of the following:

Table 9. Accessory List for RZ/G2L-EVK and RZ/V2L-EVK

Item	Required / Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	RZ/G2L-EVK or RZ/V2L-EVK	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD	Used for Linux image
Debug cable	Required	USB Type-A to Micro USB Type-B cable	UART / debug serial (CN14)	—
Power supply	Required	USB-C 5V 3A+ power supply	USB-C power	Required for power source
Display cable	Required for graphical images	micro-HDMI to HDMI cable	HDMI	Required for HDMI display path
Display	Required for graphical images	1080p HDMI monitor	HDMI	Recommended for Weston / GUI images
CSI camera	Optional	OV5645 camera module	MIPI CSI	Required for camera usage
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use

USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Required for audio usage
Audio splitter	Optional	3.5mm Audio Stereo Y Splitter extension cable	Audio jack	Needed for separate mic/headphone use

4.4.2 Essential Hardware Setup

The figure below shows the basic essential hardware setup for RZ/G2L-EVK & RZ/V2L-EVK. Follow these steps to prepare and power on the board:

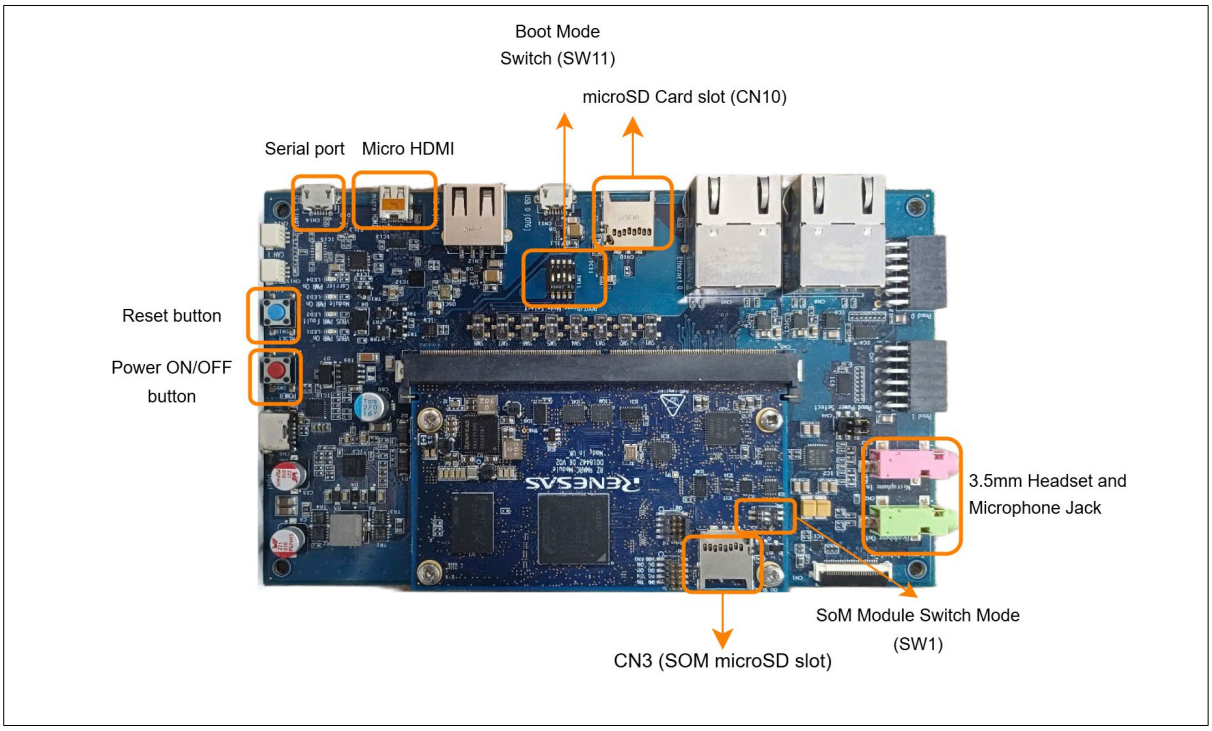


Figure 14. Essential minimum interfaces for RZ/G2L-EVK & RZ/V2L-EVK

Note:

- Boot source selection
  - In this release, all boot flows use QSPI or eMMC as selected by SW11.
- CN10 (Carrier microSD slot)
  - CN10 is not a boot device.
  - CN10 can be used only for rootfs or data storage after the system has booted.
  - No SW1 configuration is required for CN10.
- CN3 (SoM microSD slot)
  - BL2/FIP cannot be loaded from CN3.
  - When the board boots from QSPI or eMMC, CN3 is accessible in U-Boot/Linux (commonly as mmc device 0).
  - CN3 may be used as a rootfs or additional storage device if enabled via SW1-2.

Follow these steps to prepare and power on the board:

1. **Prepare the microSD Card**

   Flash the provided image to the microSD card using the procedure in section **Flashing the SD Card Image** or using a user-friendly tool such as Balena Etcher, as described in section SD/MMC Card Flashing for a simplified flashing experience.

2. **Insert the microSD Card**

   Insert the prepared SD card into CN10. This slot is for rootfs/data only; it cannot be used to load BL2/FIP.

3. **Configure the Boot Mode**

   In this section, we focus on QSPI boot as the default boot method. The switch configuration below loads BL2 and FIP from QSPI flash, with the root filesystem provided from eMMC or the SD card in slot CN10/CN3.

   Default setup (QSPI boot):

   - SW11 (on the carrier board): Set to QSPI boot.

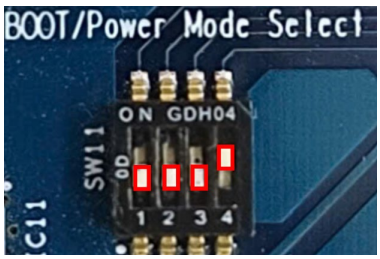


Figure 15. RZ/G2L-EVK & RZ/V2L-EVK QSPI boot mode

- SW1-2 (on the SoM): Controls access to storage devices on the SOM (Optional)
  - OFF: Enables access to the onboard eMMC
  - ON: Enables access to the microSD slot CN3

Table 10. SW1 Boot Mode Switch Settings for eMMC and SD Card Operation

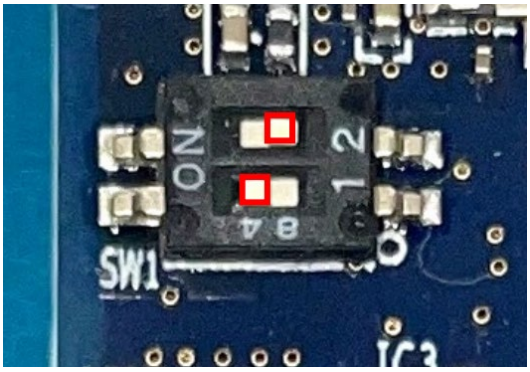


Figure 16. eMMC selection with normal operation

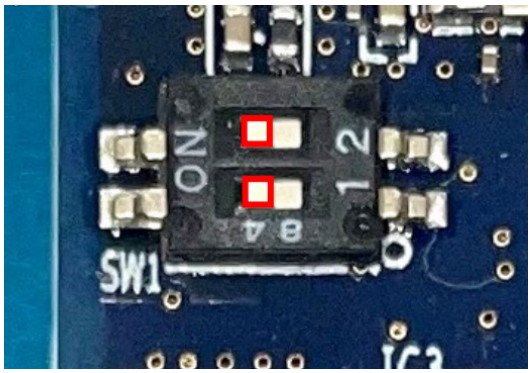


Figure 17. Enable the SD slot CN3 with normal operation

- Root filesystem: Use the SD card in slot CN10 (carrier board). CN10 remains accessible regardless of SW1-2.

Alternative setup (eMMC boot):

- SW11 (on the carrier board): Set to eMMC boot
- SW1-2 (on the SoM): Same as above for QSPI boot.

Refer to section 16.2.2 for other boot switch positions. In these modes, the board fetches BL2 and FIP from the chosen device.

4. **Connect Peripherals**

   - Attach an HDMI display

5. **Connect UART**

   - Use a USB Type-A to Micro-USB Type-B cable to connect the board's UART console port to the host PC.

6. **Power on the Board**

   - Connect the USB-C power supply (5V, 3A).
   - Press the red power button to turn on the board.
   - Wait for the UART serial device to appear on the host PC.
   - Open a terminal program on the detected serial port to monitor the boot log.

Once powered on, the boot log should appear on the UART console, and the Weston desktop will display on the HDMI screen.

4.4.3 Complete Hardware Setup

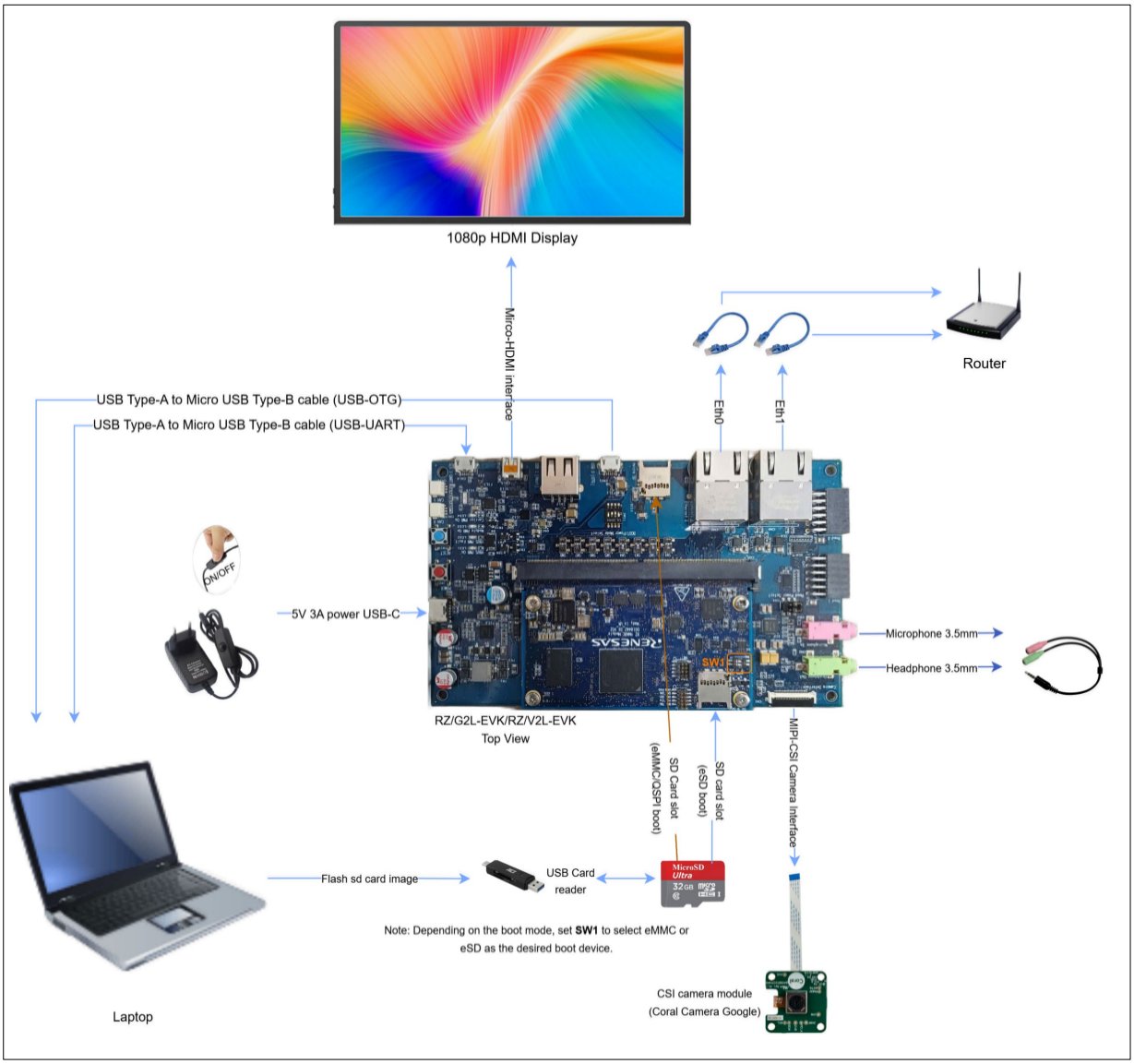


Figure 18. Complete setup for RZ/G2L-EVK & RZ/V2L-EVK

4.4.4 Booting

The boot process is straightforward:

1. Insert the prepared SD card into CN10.
2. Connect a keyboard, mouse, HDMI display, and other devices. Then connect the USB-C power supply and press the red power button to turn the board on.
3. The boot log should appear on the UART console, and the Weston desktop should display on the HDMI screen.
4. Open any of the available applications to interact with the system.

The image provided is feature-complete, offering a desktop-grade experience.

If a different boot device is required, such as QSPI, eMMC, or SD card, adjust the DIP switch settings as described in section 16.2.2 before powering on. The selected boot device determines where on-chip ROM code loads the Boot Loader stage 2 (BL2) and Firmware Image Package (FIP):

- QSPI boot loads BL2 and FIP from the onboard QSPI flash.
- eMMC boot loads BL2 and FIP from the onboard eMMC device.

The chosen boot device can be written or updated using the Universal Flashing Script; see section 7.3 for detailed instructions.

Notes: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in section 16.1.3

4.4.4.1 Default Boot Behavior

When booting from QSPI or eMMC, the U-Boot environment defaults to using the SD card (CN10) as the root filesystem.

- The SD card on CN10 is always available as mmc1.
- The onboard eMMC is always available as mmc0.
- Unless reconfigured, Linux will mount the rootfs from CN10 (e.g., /dev/mmcblk1p1).

4.4.4.2 Using eMMC (kernel/DTB and/or root filesystem)

The onboard eMMC may be used for:

- kernel/DTB only (root filesystem on SD/CN10),
- root filesystem only (kernel/DTB on SD),
- both kernel/DTB and root filesystem.

Steps:

1. Enable eMMC on the SoM: Set SW1-2 = OFF (see section 16.2.2 for SW1 description).
2. Prepare the eMMC rootfs. Refer to section 16.3 for detailed information
3. Reboot the board. During startup, output similar to the following will appear:

```
NOTICE: BL2: v2.9(release):<release-tag>
NOTICE: BL2: Booting BL31
NOTICE: BL31: v2.9(release): <release-tag>

U-Boot 2021.10 <Date>

CPU: Renesas Electronics CPU rev 1.0
Model: <board-name>
DRAM: 1.9 GiB MMC: sd@11c00000: 0, sd@11c10000: 1
Loading Environment from SPIFlash... SF: Detected mt25qu512a ...
*** Warning - bad CRC, using default environment

Hit any key to stop autoboot: 0
=>
```

   - When the message “Hit any key to stop autoboot” appears, press Enter (or any key) to interrupt the countdown.
   - The => prompt indicates that you are in the U-Boot console.
4. Choose one configuration and apply the corresponding commands:

   Detect current environment

```
=> printenv mmcdev mmc_args
```

Device mapping

- mmcdev
  - mmcdev=0 → eMMC
  - mmcdev=1 → SD (CN10)
- mmc_args
  - root=/dev/mmcblk0p2 → root filesystem from eMMC (MMC device 0, partition 2)
  - root=/dev/mmcblk1p2 → root filesystem from SD card (CN10) (MMC device 1, partition 2)

Platform default (typical): kernel/DTB load from SD; rootfs points to /dev/mmcblk1p2 = SD, partition 2.

```
=> printenv mmcdev mmc_args
mmcdev=1
mmc_args=setenv bootargs rw rootwait earlycon root=/dev/mmcblk1p2
```

Choose one of the following configurations

- Root filesystem on eMMC (kernel, DTB from SD/CN10)

```
=> setenv mmcdev 1
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> boot
```

- Kernel/DTB on eMMC (root filesystem on SD/CN10)

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk1p2'
=> boot
```

- Kernel/DTB and root filesystem on eMMC

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> boot
```

4.4.4.3 Using CN3 (eSD slot) as Root Filesystem

eSD boot from CN3 is supported in this release. The board can load the boot components from the SD card inserted in the CN3 slot, and CN3 can also be used as the root file system device in Linux.

Steps to use CN3 as the root filesystem:

1. Insert the SD card into the CN3 slot
2. Enable eSD on the SoM: Set SW1-2 = ON (see section 16.2.2 for SW1 description).
3. Reboot the board. During startup, output similar to the following will appear:

```
NOTICE:  BL2: <version>(release):<release-tag>
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: <version>(release): <release-tag>

U-Boot <Version> <Date>

CPU:   Renesas Electronics CPU rev 1.0
Model: <board-name>
DRAM:  1.9 GiB
MMC:   sd@11c00000: 0, sd@11c10000: 1
Loading Environment from SPIFlash... SF: Detected mt25qu512a ...
*** Warning - bad CRC, using default environment

Hit any key to stop autoboot:  0
=>
```

- When the message “Hit any key to stop autoboot” appears, press Enter (or any key) to interrupt the countdown.
- The => prompt indicates that the system is now in the U-Boot console.

4. Update the U-Boot environment by changing the default device from 1 (CN10) to 0 (CN3):

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> saveenv
=> boot
```

4.4.5 Known Hardware and Functional Limitations

1. No onboard Wi-Fi/Bluetooth

   These EVKs do not include an on-chip or on-board Wi-Fi/Bluetooth module. Wireless connectivity must be provided through external modules (e.g., USB or SDIO-based Wi-Fi dongles).

4.5 RZ/V2H-EVK

This section describes the hardware-specific setup and booting process for the RZ/V2H-EVK

This EVK supports rich multimedia, multi-camera, and AI inference workloads. Advanced features may require DIP switch configuration. For full details, refer to the RZ/V2H Evaluation Board Kit Hardware Manual.

4.5.1 Hardware Requirements

The hardware setup consists of the following:

Table 11. Accessory List for RZ/V2H-EVK

Item	Required / Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	RZ/V2H-EVK	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD (default SD slot 0)	Used for Linux image
Debug cable	Required	USB Type-A to Micro USB Type-B cable	UART / debug serial (CN12)	—
Power supply	Required	Power supply that can provide up to 100W via USB-C PD	USB-C power	Required for power source
Display cable	Required for graphical images	HDMI to HDMI cable	HDMI	Required for HDMI display path
Display	Required for graphical images	1080p HDMI monitor	HDMI	Recommended for Weston / GUI images
CSI camera	Optional	OV5645 camera module	MIPI CSI	Required for camera usage
CSI converter	Optional	Camera conversion 22-pin to 24-pin FPC adapter	MIPI CSI	Required for camera connection
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Required for audio usage
Audio splitter	Optional	3.5mm Audio Stereo Y Splitter extension cable	Audio jack	Needed for separate mic/headphone use

4.5.2 Essential Hardware Setup and Booting

The figure below shows the basic essential hardware setup for RZ/V2H-EVK. Follow these steps to prepare and power on the board:

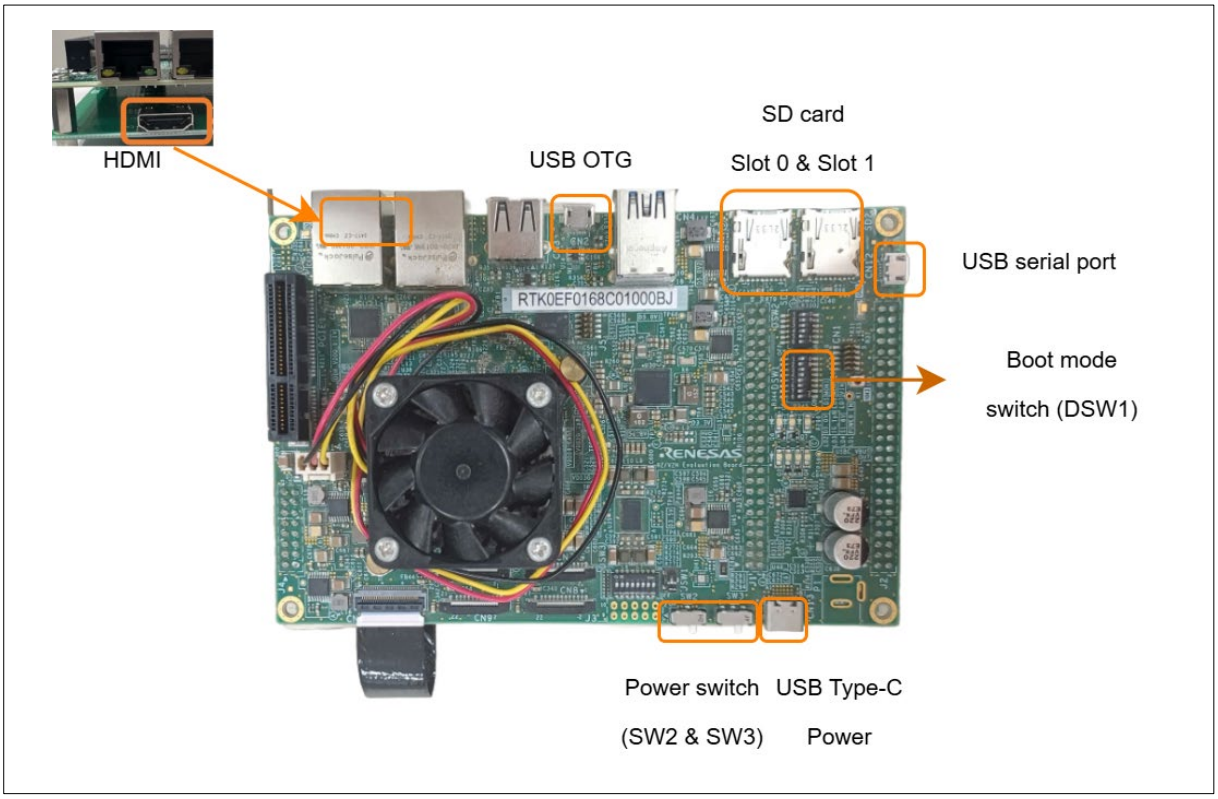


Figure 19. Essential minimum interfaces for RZ/V2H-EVK

1. **Prepare the microSD card**

   Flash the provided image to the microSD card using the procedure in section Flashing the SD Card **Image** or using a user-friendly tool such as Balena Etcher, as described in section SD/MMC Card Flashing for a simplified flashing experience.

2. **Insert the microSD Card**

   Insert the prepared microSD card into the SD card slot on the underside of the board.

3. **Configure the Boot Mode**

   Set the DIP switches (DSW1) according to section 16.2.3 to select the xSPI boot source.

   For this configuration, DSW1 is set to boot from xSPI flash

   - BL2 and FIP are loaded from xSPI.
   - Rootfs can be located on eMMC or SD card, as defined in the U-Boot environment.

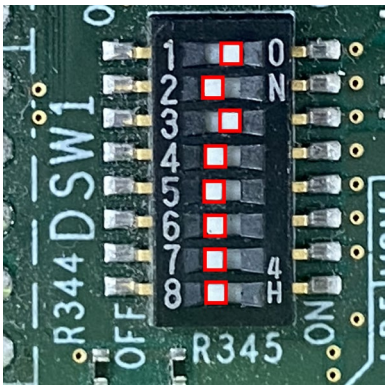


Figure 20. RZ/V2H-EVK xSPI boot mode

Note: Booting from eMMC is currently not supported in this release. This option may be enabled in future updates.

4. **Connect Peripherals**

   Attach an HDMI display.

5. **Connect UART**
   - Use a USB Type-A to Micro-USB Type-B cable to connect the board's UART console port to the host PC.
6. **Power on the board**
   - Connect the USB-C power supply (100W).
   - Turn SW3 on first, then turn on SW2
   - Open a terminal program on the host PC to monitor the boot log.

Once powered on, the boot log should appear on the UART console, and the Weston desktop will display on the HDMI screen.

4.5.3 Complete Hardware Setup

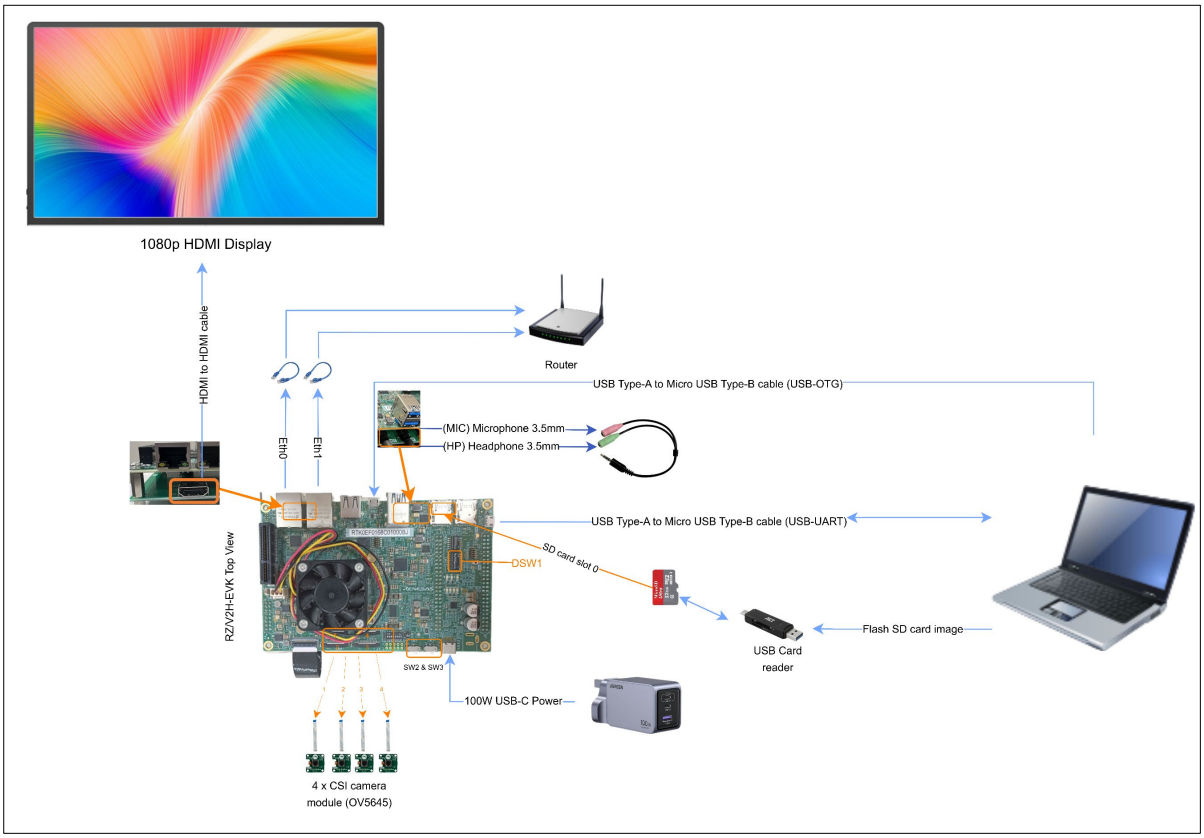


Figure 21. Complete setup for RZ/V2H-EVK board

Note: The RZ/V2H-EVK is available in two hardware versions, which differ in their storage options:

- **Version 1**: Equipped with two SD card slots and no onboard eMMC.
- **Version 2**: Equipped with one onboard eMMC (default boot device) and one SD card slot for alternate booting or data storage.

The complete setup shown above applies to Version 1. For Version 2, the setup procedure is the same as Version 1, with the only difference being the use of one eMMC and one SD card instead of two SD cards.

4.5.4 Booting

The boot process for RZ/V2H-EVK is very similar to RZ/G2L-EVK and RZ/V2L-EVK, as the boot mode must be selected using the DIP switches before power-on. Follow the steps below to boot the board:

1. Insert the SD card into the SD card slot (located on the underside of the carrier board; see the complete setup picture in the previous section).
2. Connect the keyboard, mouse, and HDMI display. Then connect the 100 W USB-C power supply and toggle two power switches next to the USB-C port (SW2 and SW3)
3. The boot log will appear on the UART console, and the Weston desktop will display on the HDMI screen.
4. Launch any of the available applications to interact with the system.

The provided image is feature-complete and offers a desktop-grade user experience.

As with other RZ/G2L-EVK & RZ/V2L-EVK, the RZ/V2H-EVK requires selecting a boot device — such as xSPI, eMMC, or SD Card before powering on. The default boot mode used by the flashing script is xSPI. Adjust the DIP switch settings as described in Section 16.2.3 for other boot modes.

The chosen boot device can be written or updated using the Universal Flashing Script; see Section 7.3.2 for detailed instructions.

Note: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in section 16.1.4

4.5.4.1 Power Switch Handling Precautions for RZ/V2H-EVK

Use caution when configuring the RZ/V2H-EVK power switches. Incorrect switch operation may damage the board or connected devices.

The RZ/V2H requires a defined power-on and power-off sequence. When using the RZ/V2H-EVK, follow these rules:

Power on

1. Confirm that power slide switches SW2 and SW3 are OFF.
2. Connect the USB Type-C cable to the Type-C connector (CN13). Do not connect the USB Type-C cable while SW2/SW3 are ON.
3. Turn SW3 ON
4. Turn SW2 ON

Power off

- Perform the software-controlled power-off sequence before switching off hardware power.
- After the software power-off sequence completes, turn SW2 OFF, then turn SW3 OFF, then disconnect the USB Type-C cable from CN13.
- Do not remove the USB Type-C cable while SW2/SW3 are ON, as this may cause device damage.

4.5.4.2 Default Boot Behavior

When booting from QSPI, the U-Boot environment defaults to using the SD card.

RZ/V2H-EVK hardware variants:

- Version 1 — SD-only (two SD slots, no eMMC): default boot device is SD0.
- Version 2 — SD + eMMC: default boot device is the SD card; it can be reconfigured to boot from eMMC if required.

4.5.4.3 Selecting the Root Filesystem Device

The RZ/V2H-EVK has two hardware variants, and the MMC device mapping differs depending on the variant. Select the root filesystem device according to the target board version.

Steps:

1. Prepare the rootfs:
    - For RZ/V2H-EVK Version 1, prepare an SD1 card by following the procedure in SD/MMC Card Flashing to flash the required image to another SD card.

- For RZ/V2H-EVK Version 2, prepare a root filesystem on eMMC, refer to Prepare the eMMC root filesystem for detailed instructions.

2. Reboot the board. During startup, output similar to the following will appear:

```
NOTICE:  BL2: <version>(release):<release-tag>
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: <version>(release): <release-tag>

U-Boot <version> <Date>

CPU:   Renesas Electronics CPU rev 1.0
Model: <board-name>
DRAM:  1.9 GiB
MMC:   sd@11c00000: 0, sd@11c10000: 1
Loading Environment from SPIFlash... SF: Detected mt25qu512a ...
*** Warning - bad CRC, using default environment

Hit any key to stop autoboot:  0
=>
```

- When the message “Hit any key to stop autoboot” appears, press Enter (or any key) to interrupt the countdown.
- The => prompt indicates that we are in the U-Boot console.

3. Choose one configuration and apply the corresponding commands:

Check the current environment

```
=> printenv mmcdev mmc_args
```

Table 12. RZ/V2H-EVK board variant mmcdev mapping

Board version	mmcdev=0	mmcdev=1	Default configuration
RZ/V2H-EVK version 1	SD0 card	SD1 card	SD0
RZ/V2H-EVK version 2	eMMC	SD1 card	SD1

Select the device used to load the kernel image and DTB by setting mmcdev.

```
=> setenv mmcdev <device_number>
```

For example, to load from mmc1, run the following command

```
=> setenv mmcdev 1
```

Use the following table to select the root filesystem device.

Table 13. RZ/V2H-EVK board variant mmc_args mapping

Board version	Root filesystem location	Settings
RZ/V2H-EVK version 1	SD0, partition 2	root=/dev/mmcblk0p2
RZ/V2H-EVK version 1	SD1, partition 2	root=/dev/mmcblk1p2
RZ/V2H-EVK version 2	eMMC, partition 2	root=/dev/mmcblk0p2
RZ/V2H-EVK version 1	SD1, partition 2	root=/dev/mmcblk1p2

Update mmc_args so that the root= value points to the required root filesystem device.

Generic command format:

```
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk<X>p2'
```

For example, on RZ/V2H-EVK Version 2, to use the root filesystem from eMMC, use:

```
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
```

4.5.5 Known Hardware and Functional Limitations

1. No on-board Wi-Fi/Bluetooth:

   These EVKs do not include an on-chip or on-board Wi-Fi/Bluetooth module. Wireless connectivity must be provided through external modules (e.g., USB or SDIO-based Wi-Fi dongles).
2. eMMC boot is not supported in this release.

   The RZ/V2H board exists in two hardware versions, one of which does not include eMMC. As a result, eMMC flash support was not provided.

4.6 RZ/V2H-RDK

4.6.1 Hardware Requirements

The basic hardware setup consists of the following:

Table 14. Accessory List for RZ/V2H-RDK

Item	Required / Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	RZ/V2H-RDK	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD	Used for Linux image

Debug cable	Required	USB Type-A to Micro USB Type-B cable	UART / debug serial (CN12)	—
Power supply	Required	Power supply, DC power supply: 12 V, ≥ 1 A (12 W or higher), regulated (AC input 100–240 V)	DC jack	Required for power source
Display cable	Required for graphical images	micro-HDMI to HDMI cable	HDMI	Required for HDMI display path
Display	Required for graphical images	1080p HDMI monitor	HDMI	Recommended for Weston / GUI images
CSI camera	Optional	OV5645 camera module	MIPI CSI	Required for camera usage
CSI converter	Optional	Camera conversion 22-pin to 24-pin FPC adapter	MIPI CSI	Required for camera connection
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Required for audio usage

4.6.2 Essential Hardware Setup and Booting

The figure below shows the basic essential hardware setup for RZ/V2H-RDK. Follow these steps to prepare and power on the board:

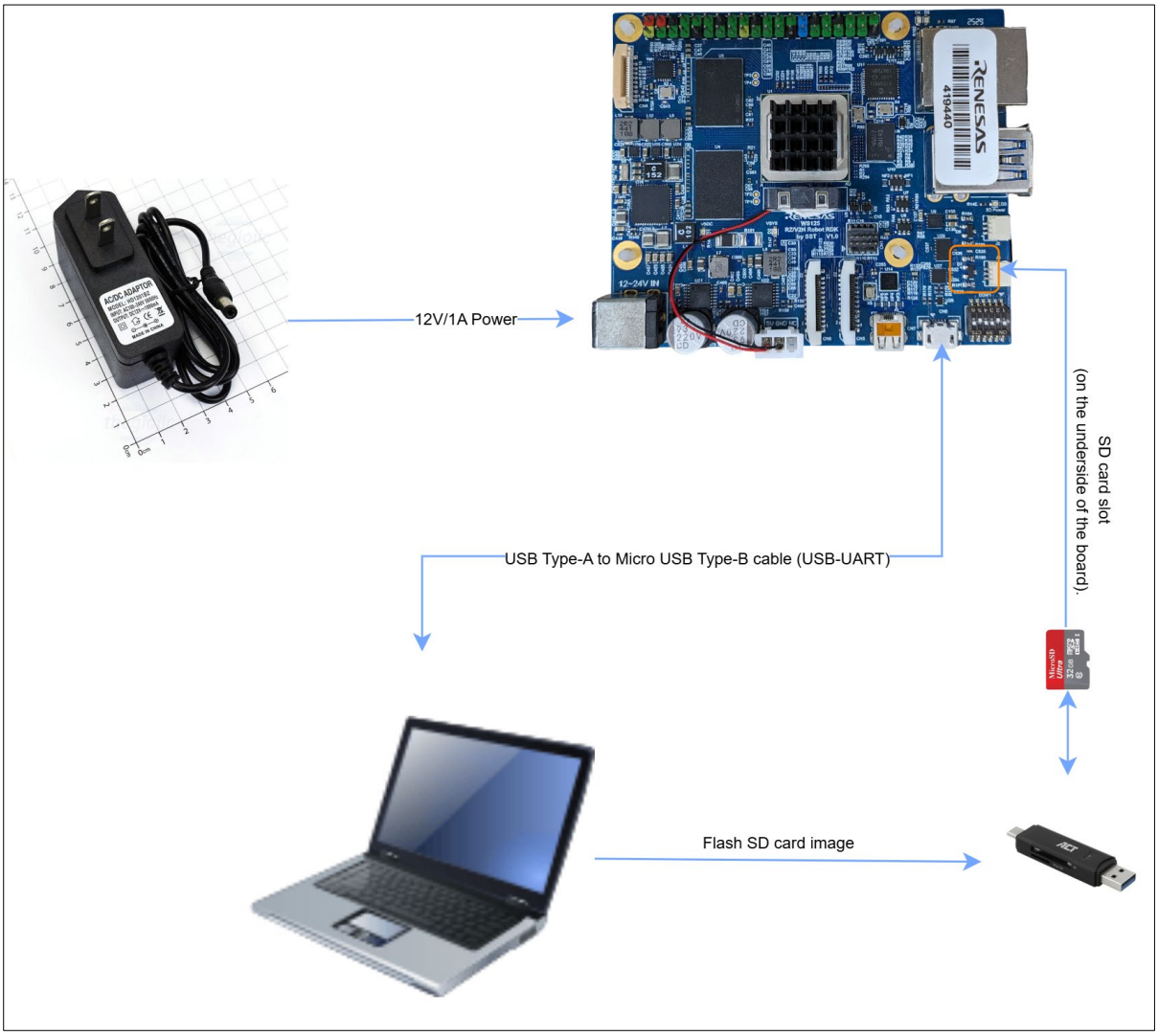


Figure 22. Minimum interfaces for RZ/V2H-RDK

1. **Prepare the microSD card**

   Flash the provided image to the microSD card using the procedure in section **Flashing the SD Card Image** or using a user-friendly tool such as Balena Etcher, as described in section SD/MMC Card Flashing for a simplified flashing experience.

2. **Configure the boot mode**

   Set the DIP switches (DSW1) to select the boot source. For this release, configure DSW1 to boot from QSPI flash:

   - BL2 and FIP are loaded from QSPI.
   - Rootfs can be located on eMMC or SD card, as defined in the U-Boot environment.

Figure 23. RZ/V2H-RDK xSPI boot switch

3. **Connect peripherals**
   - Attach the HDMI display, USB keyboard, and USB mouse.
4. **Connect UART**
   - Use a USB Type-A to Micro-USB Type-B cable to connect the board's UART console port to the host PC.
5. **Power on the board**
   - Connect the 12V/1A power supply.
   - Open a terminal program on the host PC to monitor the boot log.

Once powered on, the boot log should appear on the UART console, and the Weston desktop will display on the HDMI screen.

4.6.3 Complete Hardware Setup

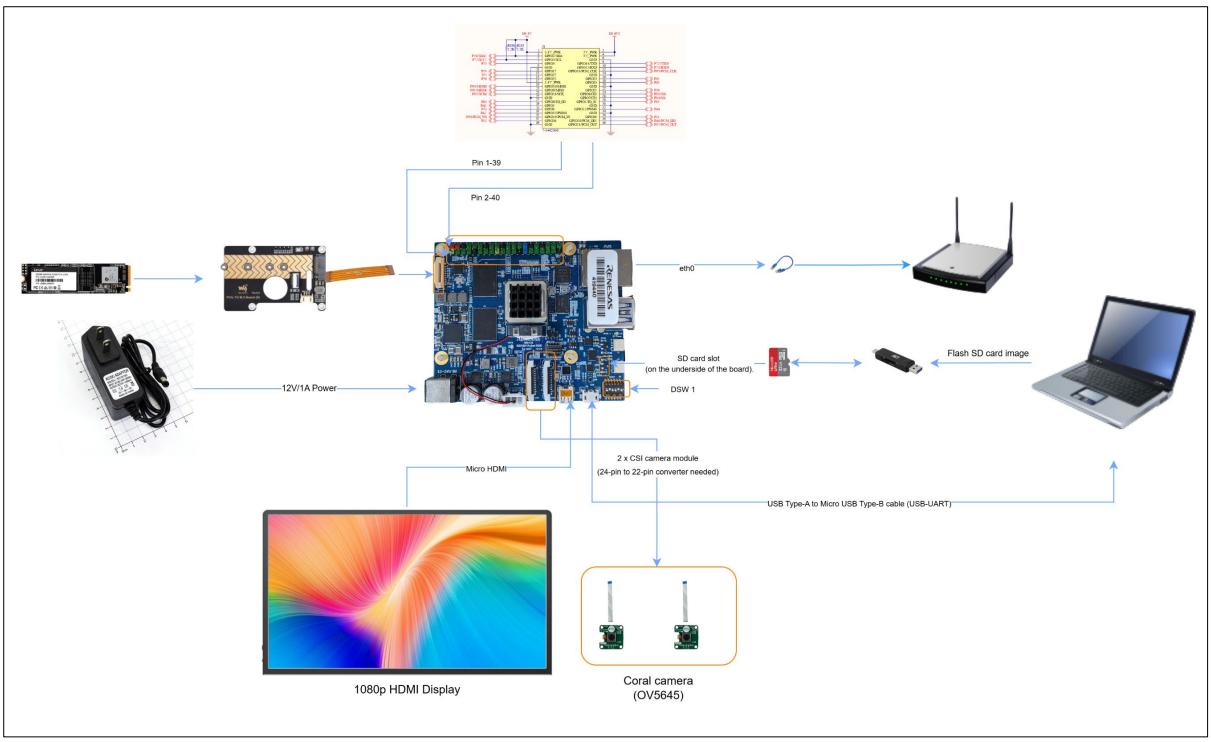


Figure 24. Complete Hardware Setup for RZ/V2H-RDK

4.6.4 Booting

The boot process for RZ/V2H-RDK is very similar to RZ/G2L-EVK, RZ/V2L-EVK, and RZ/V2H-EVK, as the boot mode must be selected using the DIP switches before power-on. Follow the steps below to boot the board:

1. Insert the SD card into the SD card slot.
2. Connect the keyboard, mouse, and HDMI display. Then connect the power supply.
3. The boot log will appear on the UART console, and the Weston desktop will display on the HDMI screen.
4. Launch any of the available applications to interact with the system.

The provided image is feature-complete and offers a desktop-grade user experience.

As with other RZ/G2L-EVK & RZ/V2L-EVK and RZ/V2H-EVK, the RZ/V2H-RDK requires selecting a boot device — such as xSPI, eMMC, or SD Card before powering on. The default boot mode used by the flashing script is xSPI. Adjust the DIP switch settings as described in section 16.2.4 for other boot modes.

The chosen boot device can be written or updated using the Universal Flashing Script; see Section 7.3.2 for detailed instructions

Note: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in section 16.1.5

4.6.5 Known Hardware and Functional Limitations

1. No on-board Wi-Fi/Bluetooth:

   The RZV2H-RDK does not include an on-chip or on-board Wi-Fi/Bluetooth module. Wireless connectivity must be provided through external modules (e.g., USB or SDIO-based Wi-Fi dongles).
2. The PCIe port on the RZV2H-RDK board does not supply power to the Endpoint, so external power must be provided to the Endpoint.

4.7 IMDT-V2H-SBC

This section describes the hardware-specific setup and booting process for the IMDT-V2H-SBC

4.7.1 Overview of Connectors

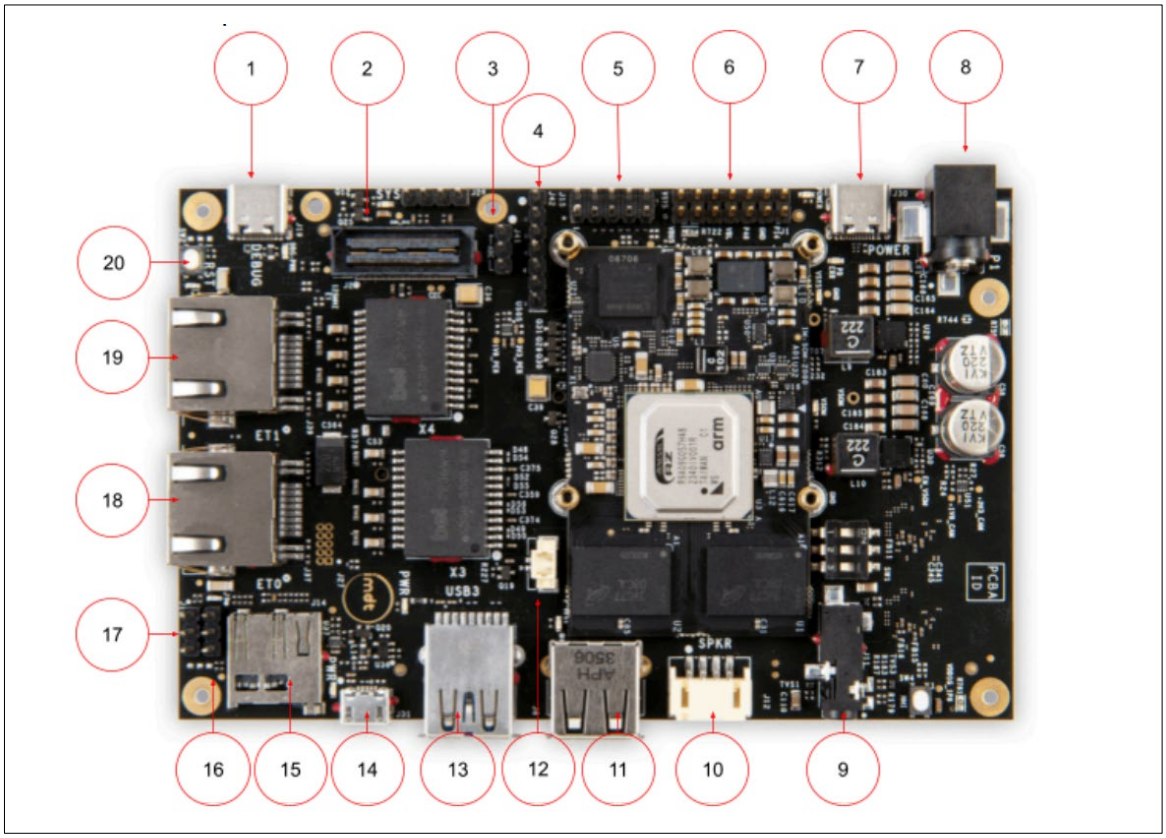


Figure 25. IMDT-V2H-SBC Top view

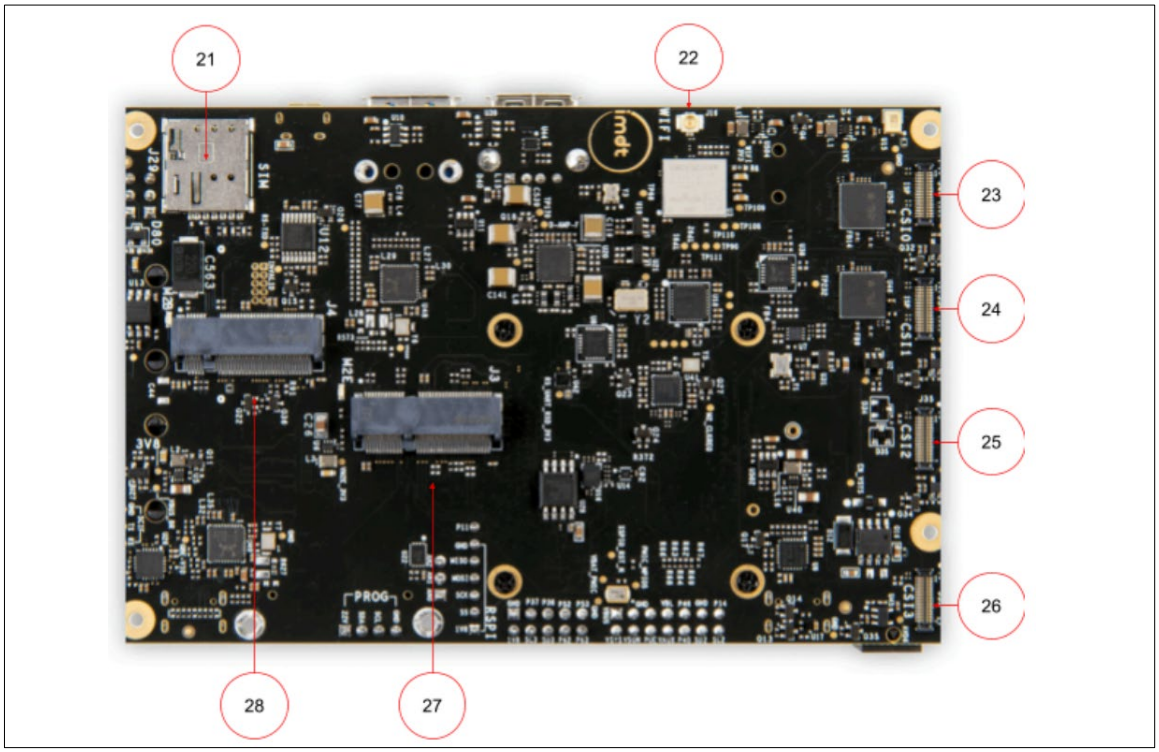


Figure 26. IMDT-V2H-SBC Bottom view

The following table shows details of the board functionality.

Table 15. Board Functionality Overview

Number	Functionality
1	DEBUG
2	Display
3	Analog Input
4	Auxiliary 1
5	Auxiliary 2
6	PSM (Power Supply Module)
7	Power In - USB-PD
8	Power In - DC Jack
9	3.5mm Audio Jack
10	Speaker Out
11	USB 2
12	Fan
13	USB 3
14	USB Micro-B
15	Micro SD Card
16	RS-232
17	CAN
18	Ethernet 0
19	Ethernet 1
20	Reset Button
21	Nano SIM Card
22	WiFi Antenna
23	CSI 0 Connector
24	CSI 1 Connector
25	CSI 2 Connector
26	CSI 3 Connector

27	M.2 Key-E
28	M.2 Key-B

4.7.2 Hardware Requirements

The hardware setup consists of the following:

Table 16. Accessory List for IMDT-V2H-SBC

Item	Required / Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	IMDT-V2H-SBC	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD	Used for Linux image
Debug cable	Required	USB Type-A to Micro USB Type-C cable	UART / debug serial (J10)	—
Power supply	Required	Power supply DC9V/3A via USB-C PD (not included in the package) or DC Barrel jack with DC12V- 24V/2A minimum.	DC jack	Required for power source
Display cable	Required for graphical images	micro-HDMI to HDMI cable	HDMI	Required for HDMI display path
DSI to HDMI adapter	Required for graphical images	B-LCDAD-HDMI DSI to HDMI adapter	MIPI-DSI to HDMI	Converts the board's DSI display output to HDMI for an external monitor
Display	Required for graphical images	1080p HDMI monitor	HDMI	Recommended for Weston / GUI images
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Required for audio usage

4.7.3 Essential Hardware Setup and Booting

1. **Prepare the microSD card**

   Flash the provided image to the microSD card using the procedure in section Flashing the SD Card **Image** or using a user-friendly tool such as Balena Etcher, as described in section SD/MMC Card Flashing for a simplified flashing experience.

2. **Configure the boot mode**

   Set the DIP switches according to the picture below to select the xSPI boot mode.

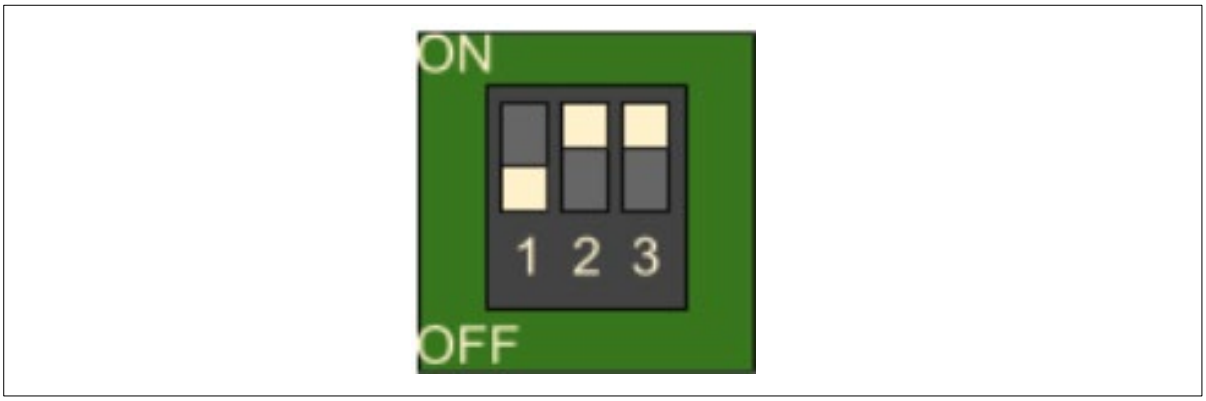


Figure 27. IMDT-V2H-SBC xSPI boot switch

   - eMMC is not supported in this release.
   - QSPI (xSPI) for the bootloader and an SD card for the kernel image.

3. **Connect peripherals**
   - Connect the DSI-to-HDMI adapter and HDMI monitor, then connect the USB keyboard and USB mouse.
4. **Connect UART**
   - Use a USB Type-A to USB Type-C cable to connect the board's UART console port to the host PC.
   - Open a terminal program on the host PC to monitor the boot log.
5. **Power on the board**
   - The SBC can be powered in two ways:
     - DC Barrel Jack: 12–24V / 2A minimum
     - USB-C PD: 9V / 3A minimum

Once powered on, the boot log should appear on the UART console.

4.7.4 Complete Hardware Setup

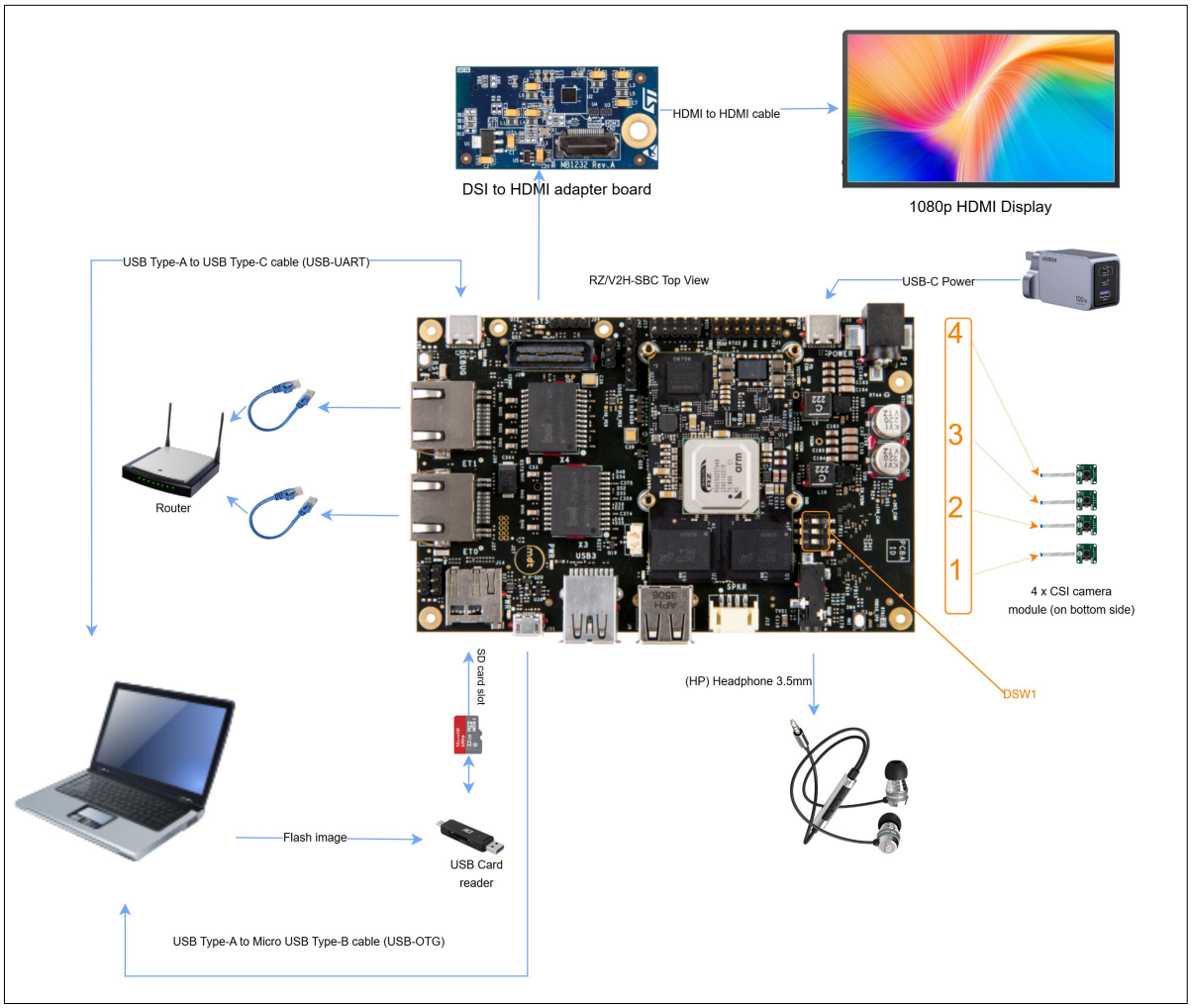


Figure 28. Complete setup for IMDT-V2H-SBC board

4.7.5 Booting

The boot process is straightforward, as the boot mode must be selected using the DIP switches before power-on. Follow the steps below to boot the board:

1. Insert the SD card into the SD card slot (located on the underside of the carrier board; see the complete setup picture in the previous section).
2. Connect the DSI screen, keyboard, and mouse. Then connect the USB-C power or DC barrel jack.
3. The boot log will appear on the UART console, and the Weston desktop will display on the HDMI screen.
4. Launch any of the available applications to interact with the system.

The provided image is feature-complete and offers a desktop-grade user experience.

Before powering the IMDT V2H SBC, set the DIP switches to select the desired boot device for xSPI as the default boot mode. The board reads these switch positions at power-up to determine the boot source; changing switches while the board is powered may be ignored or cause undefined behavior. Refer to Section 16.2.5 for the exact switch map for other boot modes.

Note: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in section 16.1.5

4.7.5.1 Default Boot Behavior

When booting from QSPI, U-Boot can use either the SD card or eMMC as the root filesystem..

- The onboard eMMC is always available as mmc0.
- The SD card is always available as mmc1.
- Unless reconfigured, Linux will mount the rootfs from the SD Card (e.g., /dev/mmcblk1p1).

4.7.5.2 Using an eMMC Root Filesystem

The onboard eMMC may be used for:

- kernel/DTB only (root filesystem on SD card),
- root filesystem only (kernel/DTB on SD card),
- both kernel/DTB and root filesystem.

Steps:

1. Prepare the eMMC rootfs. Refer to section 16.3 for detailed information
2. Reboot the board. During startup, output similar to the following will appear:

```
NOTICE:  BL2: <version>(release):<release-tag>
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: <version>(release): <release-tag>

U-Boot <Version> <Date>

CPU:   Renesas Electronics CPU rev 1.0
Model: <board-name>
DRAM:  <ram-amount> GiB
MMC:   sd@11c00000: 0, sd@11c10000: 1
Loading Environment from SPIFlash... SF: Detected mt25qu512a ...

Hit any key to stop autoboot:  0
=>
```

   - When the message “Hit any key to stop autoboot” appears, press Enter (or any key) to interrupt the countdown.
   - The => prompt indicates that you are in the U-Boot console.
3. Choose one configuration and apply the corresponding commands:

   Detect current environment

```
=> printenv mmcdev mmc_args
```

   Device mapping
   - mmcdev
     - mmcdev=0 → eMMC
     - mmcdev=1 → SD card
   - mmc_args

- root=/dev/mmcblk0p2 → root filesystem from eMMC (MMC device 0, partition 2)
- root=/dev/mmcblk1p2 → root filesystem from SD card (CN10) (MMC device 1, partition 2)

Platform default (typical): kernel/DTB load from SD; rootfs points to /dev/mmcblk1p2 = SD, partition 2.

```
=> printenv mmcdev mmc_args
mmcdev=1
mmc_args=setenv bootargs rw rootwait earlycon root=/dev/mmcblk1p2
```

Choose one of the following configurations

- Root filesystem on eMMC (kernel, DTB from SD card)

```
=> setenv mmcdev 1
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> saveenv
=> boot
```

- Kernel/DTB on eMMC (root filesystem on SD card)

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk1p2'
=> saveenv
=> boot
```

- Kernel/DTB and root filesystem on eMMC

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> saveenv
=> boot
```

4.7.6 Known Hardware and Functional Limitations

1. eMMC boot is not supported in this release
2. Ethernet operation in U-Boot may be unstable; if UDP-based flashing is required, Ethernet must be initialized in the Linux kernel first, and the system rebooted without disconnecting power; otherwise, OTG should be used for flashing.
3. Bluetooth: The on-board Bluetooth is non-operational. The Bluetooth module is not documented in the schematic; therefore, Bluetooth support is not currently developed. This feature may be implemented in a future release

4.8 Boot Notice

On every reboot, U-Boot imports the fields listed in Table 74. Auto-managed U-boot environment variables and automatically resets the corresponding U-Boot environment variables to their default values. These variables are managed by the system and are reloaded at boot time.

If a custom environment is required, do not rely on one-off setenv/saveenv at the prompt—those changes will be overwritten on the next boot.

Running **env default -a** (reset environment to default) will clear all environment variables, including the board identification variables loaded at runtime. After running this command, the board must be rebooted for the board identification file to be loaded again and restore these auto-managed variables.

5. General Operational Flow

The diagram below shows the operational flow of most of the RZ-based systems during power ON.

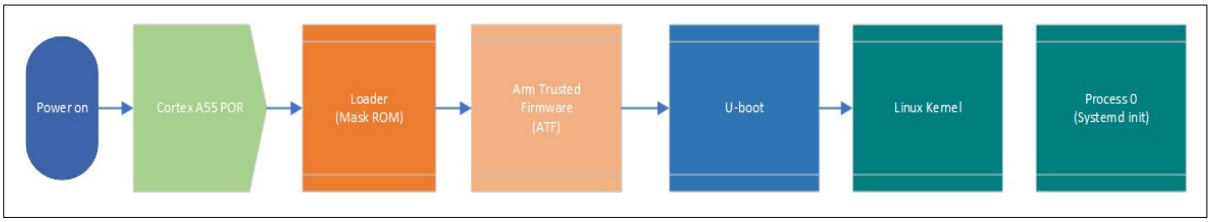


Figure 29. RZ-based systems boot operational flow

By default, the board is in the power OFF state. When the power is supplied, the PMIC immediately cycles power and puts the Cortex A55 into a POR state. This kickstarts the boot process with the Loader and ends with Linux booting into user space.

While U-Boot passes full control to the Linux kernel, the ARM trust zone remains active along with op-tee within the ARM core's trust zone of operations.

The exact boot time depends on the boot environment and the number of services in the initialization process.

5.1 Arm Exception Levels

In order to explain the booting and running of software, it is necessary to understand the ARM core's exception levels. Exception levels refer to the different privilege modes of operation. These are different levels at which the CPU operates, and each level is a layer of software that remains active throughout the runtime of the SoC.

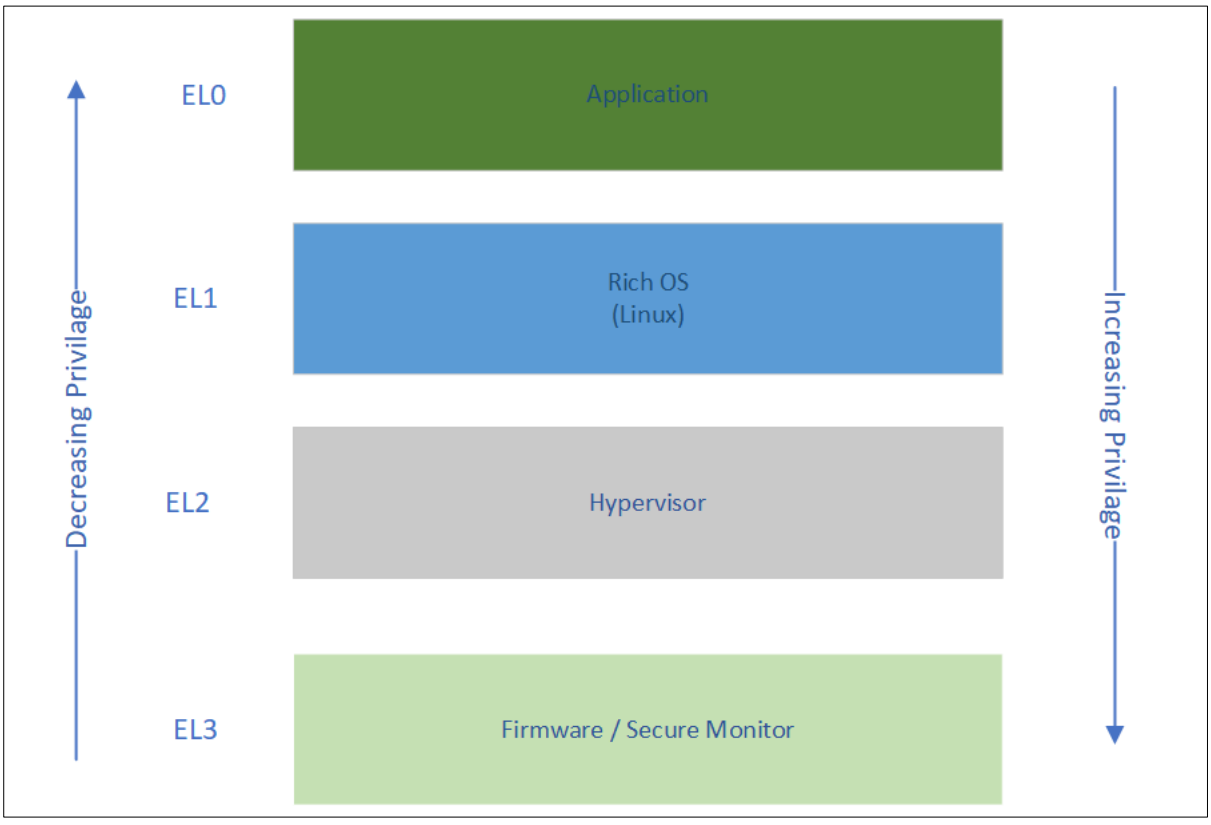


Figure 30. ARM exception levels

The general layout of this is intuitive. However, each level has multiple implementations and layer stacks of its own. In most of our implementations, we do not deploy a Hypervisor. Hence, EL2 is bypassed. The levels we are concerned with are mostly EL3 and EL1. EL3 is complex, having its own stack of layers, which will be discussed subsequently in this section.

5.2 Secure and Non-Secure Runtime

The modern ARM64 has two execution modes, both active at the same time. These are called secure and non-secure worlds. The secure world comes with its own OS, storage management, exception handling, bootloaders, etc. The previous diagram shows the boot time model. The diagram below shows the runtime model of the system levels.

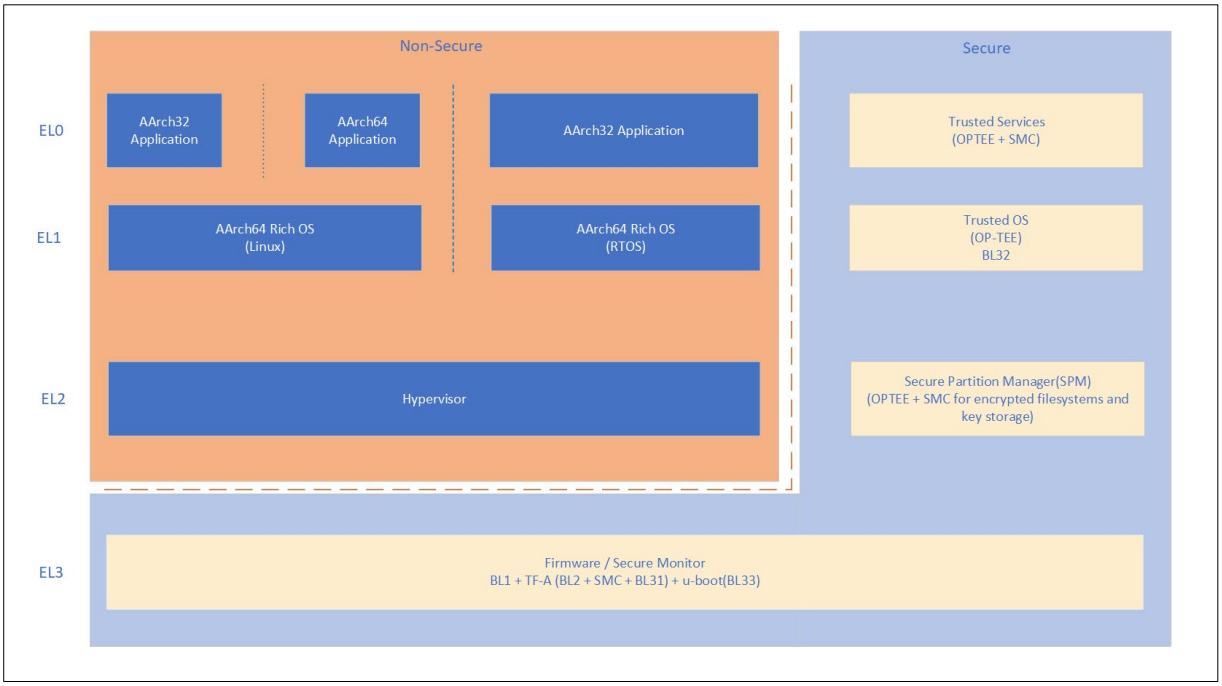


Figure 31. Runtime Exception levels with their states

5.3 Arm Trusted Firmware-A (TF-A)

The Arm Trusted Firmware-A (TF-A project) is a reference implementation of the ARM architecture's EL3 layer. It consists of the entire stack of firmware in the EL3 level except for u-boot, which is the primary high-level boot loader.

Unlike in previous implementations of ARM, where there were just stage 1 and stage 2 bootloaders, the current generations operate with a more complex hierarchy that abstracts hardware access and initializations into different components of the boot. Even DDR configurations, which were traditionally done by u-boot and later managed by the Linux kernel, are now done in the TF-A.

The TF-A project is also what is used by Renesas as its EL3 Firmware source. The forks of TF-A maintained by Renesas contain necessary hardware-specific changes on top of ARM's reference implementation.

The reference TF-A version we use in our current release is 2.14.2.

5.3.1 Components of Boot

This section provides an overview of the entire boot process used in the RZ-based system as per the TF-A paradigm. The process is divided into several distinct stages that collectively transition the

hardware from a power-off state to a fully operational system running the operating system. Each stage is handled by a different component:

Table 17. RZ-specific TF-A implementation

Component	Description
BL1 (Boot Loader Stage 1 – AP Trusted ROM):	• BL1 is embedded in the Boot ROM of the SoC and is the first code executed when power is applied (POR). • It initializes the most basic hardware components (such as memory and essential peripherals) and sets the stage for a secure boot.
BL2 (Bootloader Stage 2 – Trusted boot firmware)	• BL2 is the next stage in the boot process, executed by the Trusted Boot Firmware that was loaded in BL1. • It loads two key components into DRAM: the EL3 Runtime Software (BL31) and U-Boot.
BL31 (Boot Loader stage 3-1 – EL3 Runtime Software)	• BL31 is the EL3 Runtime Software running at the highest privilege level (EL3). • It is part of the Arm Trusted Firmware-A and is responsible for configuring Arm TrustZone to create a secure execution environment.
BL32 (Boot Loader stage 3-2 – Secure-EL1 Payload)	• This is optional; it is used to load a secure payload (such as OP-TEE) that provides a Trusted Execution Environment. This is also where the encrypted filesystem is handled and contains the keys. This layer is also known as “Trust Zone” or “Trusted Execution Environment” (TEE), such as the ‘TEE’ in ‘OP-TEE’
BL33 (Non-Trusted-firmware – u-boot)	• The final boot stage, where the u-boot is loaded, takes control of the system and boots the operating system into user space.

Note: TF-A is highly flexible and has plenty of optional layers. This section covers the specific implementation used in the RZ series of SoCs’ reference implementation and hence does not use (bypasses) many of the optional levels, such as BL3-0 SCP Firmware load. The components described here are the final implementation and supersede the reference ARM TF-A.

5.3.1.1 BL1

BL1 is the first program that runs on POR. While TF-A mentions all the layers and provides reference implementations till BL32, BL1 is often a proprietary implementation that is strictly under NDA requirements.

The core functions of BL1 include:

- Execution of code from the POR reset vector, which contains an XIP ROM within the SoC.
- Determine between warm and cold reset boots. In the case of a warm boot, the secondary cores will go to a separate entry point while the primary boot core will continue the cold boot. This is platform-specific.
- Platform initialization:
  - Enable trusted watchdog.
  - Base Clock configuration and enable the system timer.
  - Console initialization.
  - Enable SoC interconnect like AHB bridges.
  - Enable MMU and the memory map.
  - Enable the peripherals that are potential sources for the BL2 image.
- Check if there is a firmware update required, and if so, proceed with the firmware update process. This is usually the case with a warm reset with a setting passed in.
- Read bootstrapping pins and cycle through boot sources.
- BL2 image load.
- Pass control to BL2.

Note: BL1 functions are platform dependent. In aarch64 architectures, BL1 contains a limited set of SMC call implementations, allowing some back and forth between EL1, BL2, and BL1.

Unlike traditional systems, modern systems have all layers of software remain active in their own quasi-virtual world and allow some interactions through the SMC call mechanism. These are critical for firmware updates and control passing across layers, along with initialization data like ID strings and timing. The Linux kernel has a secure component that interacts with all the EL3 components through SMC calls.

For more information, refer to TF-A's documentation for further details.

5.3.1.2 BL2

The BL2 layer is provided by the Renesas TF-A implementation. The TF-A provides bl2.bin, which is the implementation used here.

Functions of BL2:

- Initialize console.
- Configure platform storage to load further images. This includes DDR, SD, MMC, etc.
- MMU initialization and mapping.
- Set up security components.
- Populate shared memory to be passed to other components.
- Define memory regions and timing.
- Device tree load and address passing.
- BL2 loads multiple images to the DDR memory.
    - EL1 Runtime: This is the secure TEE OS that runs on the secure side of EL1.
    - U-boot: Primary BL33 bootloader.

5.3.1.3 BL31

BL31 is the EL3 runtime software responsible for managing secure monitor calls (SMCs) and switching between secure and non-secure execution worlds. It is provided as part of the TF-A implementation.

Functions of BL31:

- Initialize console.
- Configure the Interconnect to enable hardware coherency.
- Enable the MMU and map the memory it needs to access.

To initialize the generic interrupt controller (GIC):

- Initialize the power controller device.
- Detect the system topology.
- Manage execution handoff between secure (BL32) and non-secure (BL33) environments.
- If a Trusted OS (BL32) is present, transfer execution to BL32; otherwise, transfer execution directly to BL33.

5.3.1.4 BL33

BL33 refers to the non-secure world in ARM's TEE. It is the first non-secure code loaded by TF-A after the secure world (BL31) and optionally BL32 (Trusted OS).

In ARM-based systems, BL33 is typically the secondary stage bootloader (SSBL), responsible for further system initialization and loading the operating system.

For RZ-based systems, U-Boot is used as the default SSBL, preparing the hardware and loading the operating system into memory. The reference U-Boot version we use in our current release is 2026.04.

Functions of BL33 (U-Boot):

- Initializes key hardware components such as the CPU, memory, and storage devices.
- Configures system peripherals like UART, I²C, SPI, and others, ensuring they are ready for the OS.
- Loads essential images, including the kernel and device tree, into memory from boot sources such as SD cards or network locations using Network Boot and TFTP.
- Sets the necessary environment variables and passes boot arguments to the kernel.
- Finally, after loading the required images, BL33 hands off control to the operating system kernel to complete the boot process.

5.3.2 Trusted Boot Flow

The Trusted Boot process ensures secure initialization of the system through the following sequence:

- Power-on and BL1 execution: When power is applied, the boot ROM immediately executes BL1. It performs minimal hardware initialization and basic security checks.
- BL2 initialization: BL1 loads BL2 into memory. BL2 then verifies the system's integrity, establishes additional secure boot mechanisms, and loads the next critical components (BL31 and U-Boot).
- Security setup via BL31: EL3 Runtime Software executes and sets up system-level security before handing off control.
- Transition to normal operation: U-Boot takes over from BL31 to finalize system initialization, prepare hardware interfaces, and manage device drivers as needed.
- Launching the operating system (BL33): Finally, U-Boot loads the Linux kernel (BL33). The kernel then starts the operating system, transitioning the system into full user-mode operation, and completes the boot process.

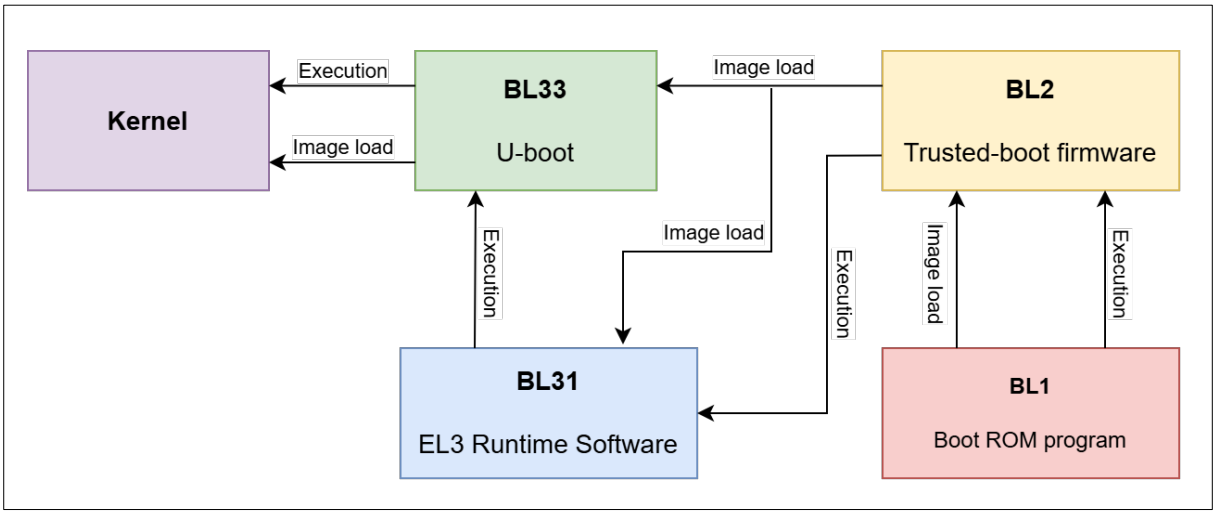


Figure 32. Trusted boot flow

6. Build System

The System release package comes with a copy of the build system that allows users to build the yocto as well as the Ubuntu images on their own. This is located under the 'host/src/rz-cmn-srp/' path under the release package. The GitHub project 'rz-build-scripts', as noted in Table 5. Public repositories for the system release package is the original reference. The appropriate branch can be investigated for further reference. The branch names usually are of the format <yocto-name>/<machine-id>.

The repository hosts builder infra that allows the user to rebuild the release using their own host machines. All projects concerned are available on GitHub and can be cloned for further development by end users, restricted only by their respective end-user licenses.

One thing to be noted here is that Yocto is the basis for much of the build's internal workings. Hence, the Yocto OE build must always be set up regardless of what else is being built.

6.1 Build Host Environment Setup

This section describes how to prepare a host system, install required dependencies, and perform a full Yocto build. The same environment is also used later when building Ubuntu images. The following sub-sections provide a step-by-step guide for setting up and completing a successful Yocto/Ubuntu build.

Requirements

- Ubuntu 24.04 LTS (64-bit) is recommended as a build environment, as we are building the 'Styhead' version of yocto.
- The RZ System release package must be available on the host

Prerequisite files

The files listed in Table 18. Prerequisite files from the release package (Yocto build only) are part of the release package. These are essential files to be used for the RZ-based system Yocto build and are located under 'host/src/' in the release package.

Table 18. Prerequisite files from the release package (Yocto build only)

File	Description
rz-cmn-srp/	Main folder for Yocto/Ubuntu build environment for RZ platforms
rz_builder.sh	Custom master build script that downloads required packages and ZIP files, configures meta layers, sets up the environment, and builds for both Yocto and Ubuntu target images. Execute it with no arguments for a helpful description.
site.conf	An override file that targets a specific build version.

patches	This is a folder that contains additional patches that are needed for the Yocto eSDK build. The patches are organized as follows:
	• meta-summit-radio/
	  • 0001-rz-sbc-meta-summit-radio-Support-build-in-yocto-styh.patch
	  • 0002-rz-sbc-summit-radio-support-eSDK-build.patch
	• meta-rz-features/meta-rz-codecs
	  • 0001-support-codec-for-linux-6.10-and-yocto-styhead.patch
	  • 0002-kernel-module-uvcs-drv-update-following-the-kernel-6.patch
	• poky
	  • 0001-uboot-config-Fix-devtool-modify.patch

files_to_add	A folder containing additional files that need to be added to the meta layer. These files are specified in the add_files field of git_patch.json, which defines their source locations and target destinations.
	• meta-rz-features /
	  • 0001-rzg2l-sbc-Bring-compat_alloc_user_space-back.patch
	  • 0004-rzg2l-sbc-Get-interrupt-number.patch
	  • 0005-src-lkm-support-Linux-6.18-kernel-APIs.patch

config.json	Contains the config options, including available build image options grouped by build type, including Yocto images, Ubuntu images, and static image collections (all-yocto-images, all-ubuntu-images, all-supported-images).
git_patch.json	A configuration file that contains JSON keys and repository configuration, such as: url, branch, tag, commit, repo type, and patch paths to apply.
layer_override.json	Pre-build customization file to add/remove meta-layers in BBLAYERS, append/exclude image packages, and select GPU mode (none/panfrost/mali). Applied before BitBake runs.
jq-linux-amd64	A lightweight and flexible shell tool that supports parsing of JSON data.
README.md	A README file describing all the necessary info about the building process.

Optional References:

- Yocto documentation (Yocto Project Documentation) – background and deeper details.
- Yocto quick build guide (Yocto Project Quick Build — The Yocto Project ®5.1.4 documentation) – alternative quick-start overview.

Install packages on the Ubuntu Host.

1. Update the Ubuntu package manager.
```
$ sudo apt update
```
2. Install the necessary packages and tools that are used by the Yocto build.
```
$ sudo apt install -y gawk wget git-core diffstat unzip texinfo gcc-multilib \
build-essential chrpath socat cpio python python3 python3-pip python3-pexpect \
xz-utils debianutils iputils-ping libsdl1.2-dev xterm p7zip-full libyaml-dev \
rsync curl locales bash-completion zstd lz4
```
3. Configure a local git account for the user.
```
$ git config --global user.name “Your Name”
$ git config --global user.email “you@example.com”
```
4. Download the following packages provided by Renesas.

Table 19. List of packages to manually download for Yocto Build

File name	Version	Download Link	Comments
RTK0EF0045Z15001ZJ-v1.1.0_EN.zip	1.1.0	rz-mpu-video-codec-library-evaluation-version	Video codec package

5. Assume all the downloaded zip files from Table 19. List of packages to manually download for Yocto Build are placed in ‘~/Downloads/renesas-yocto’. If locations differ, substitute the corresponding paths in subsequent steps.

6. Copy all the above downloaded zip files to a build folder (For example, ~/renesas/rz-cmn-srp as shown below) in the Ubuntu Host PC.

```
$ cd ~/Downloads/renesas-yocto
$ mkdir -p ~/renesas/rz-cmn-srp
$ mv *.zip ~/renesas/rz-cmn-srp
```

7. Copy all the contents in the folder 'host/src/rz-cmn-srp/' from the release package into the '~/renesas/rz-cmn-srp' folder. (This example assumes the pre-requisite files that are described are located at the package unpacked location ~/Downloads/renesas-yocto/rz-cmn-srp)

```
$ cd ~/Downloads/renesas-yocto/rz-cmn-srp/host/src/rz-cmn-srp
$ cp -r * ~/renesas/rz-cmn-srp/
```

8. Eventually, all the necessary files for the Yocto build should be present in the '~/renesas/rz-cmn-srp folder as shown below.

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ tree -L 2
.
├── config.json
├── files_to_add
│   └── meta-rz-features
├── git_patch.json
├── jq-linux-amd64
├── patches
│   ├── meta-rz-features
│   ├── meta-summit-radio
│   └── poky
├── README.md
├── rz_builder.sh
└── ubuntu
    ├── config
    ├── config.ini
    ├── docs
    ├── include
    ├── README.md
    ├── script
    └── setup_ubuntu_environment.sh
```

6.2 Builders

The release package provides a copy of the build system from the rz-build-scripts repo that was used for creating this release. The general idea is to have a single script that takes inputs and performs all subsequent actions. While most of the build is automated, it is incapable of downloading some essential licensed components at the very beginning. It has a very good CLI and is intelligent enough to point to the missing components and their download URLs. It is even able to identify the wrong checkouts and correct the branches to a limited capacity. It is governed by the parameters in the config.json along with the inputs provided on the command line while executing the builds.

The build infrastructure is split into two sections:

- **Yocto** – This part is the core foundation to build the BSP and all the yocto images. A build won't produce usable artifacts without it. Hence, the yocto build is integrated into the primary build script, rz_builder.sh itself.

- **Ubuntu** – This Ubuntu build script / infra is placed under its own directory called ‘ubuntu’. However, it should ideally be invoked using the master script rz_builder.sh to ensure that the BSP components are built in the yocto OE section and then transferred into the Ubuntu image that is built under the Ubuntu builder.

The key components are further described in the subsequent sections before heading over to the final build invocation.

6.2.1 rz_builder.sh

The rz_builder.sh script is the primary builder that performs all actions from setting up the build directory to collecting all the final build artifacts and packaging them together into a final release package.

It is capable of invoking the Ubuntu build as well.

Inputs:

- config.json — authoritative build menu:
  - Supported machines ("machine").
  - Supported images ("images.yocto" and "images.ubuntu").
  - Static groups ("images.static") that expand to multiple images (e.g., all-yocto-images).
  - Defaults ("defaults"), such as MACHINE and IMAGE.
- layer_override.json: Feature customization (features) applied before BitBake runs:
  - features.layers: adds/removes meta-layers in BBLAYERS (either a specific layer path containing conf/layer.conf, or a folder containing multiple layers).
  - features.libraries: adjusts image package lists (add appends packages; remove excludes packages and marks them as bad recommendations).
  - features.gpu: selects graphics mode. Available values are "none" and "panfrost", "mali" is not supported in this release and is treated the same as "none".
- git_patch.json — repository set and customization plan:
  - Repository origin and revision (url / type / branch, tag, or commit)
  - Patch application list (paths under patches/)
  - File overlays (paths under files_to_add/ → destination in target repos)

Invoking with no parameters prints usage, supported machines/images/groups, and examples.

```
renesas@builder-pc:~/$ cd ~/renesas/rz-cmn-srp/
renesas@builder-pc:~/renesas/rz-cmn-srp$ chmod a+x rz_builder.sh
renesas@builder-pc:~/renesas/rz-cmn-srp$ ./rz_builder.sh
```

Typical Operations:

Exact flags may vary by branch; rely on ./rz_builder.sh help for authoritative syntax.

```
# Environment and arguments
MACHINE=<machine> IMAGE=<image|group> DISTRO=<distro> ./rz_builder.sh <target-build>
<target-dir>
```

- MACHINE — target machine from config.json.machine.
- IMAGE — a single image (Yocto/Ubuntu) or a static group from config.json.images.*.
- DISTRO — defaults to poky for Yocto images; automatically set to ubuntu-tiny for Ubuntu images.
- <target_build> — build or build-sdk
- <build_dir> — build workspace name; default directory name (yocto_rzcmn_board) used if omitted.

Common examples:

```
# Yocto: single image (defaults: MACHINE=rz-cmn, DISTRO=poky, IMAGE=core-image-
weston)
$ ./rz_builder.sh build ~/build-workspace

#Ubuntu: single image (DISTRO auto-overridden to ubuntu-tiny)
$ MACHINE=rz-cmn IMAGE=ubuntu-lxde \
  ./rz_builder.sh build ~/build-workspace

# Build all Yocto images (static group)
$ MACHINE=rz-cmn IMAGE=all-yocto-images \
  ./rz_builder.sh build ~/build-workspace

#Build all Ubuntu images (static group)
$ MACHINE=rz-cmn IMAGE=all-ubuntu-images \
  ./rz_builder.sh build ~/build-workspace

#Build the full set (Yocto then Ubuntu, in order)
$ MACHINE=rz-cmn IMAGE=all-supported-images \
  ./rz_builder.sh build ~/build-workspace

#SDK build for a Yocto image
$ MACHINE=rz-cmn IMAGE=renesas-core-image-cli \
  ./rz_builder.sh build-sdk ~/build-workspace

#Build full set of SDKs (static group)
$ MACHINE=rz-cmn IMAGE=all-supported-images \
  ./rz_builder.sh build-sdk ~/build-workspace
```

Note: When using build-sdk as <target_build>, the orchestrator builds the selected image first and then generates its matching SDK.

6.2.2 Ubuntu Builder

The Ubuntu builder assembles Ubuntu images for RZ platforms by reusing BSP components exported from the Yocto build. It is a source helper that is loaded and driven by rz_builder.sh, ensuring the required Yocto artifacts exist and that image assembly proceeds in the correct sequence. It is not intended to operate as an independent CLI.

Location:

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ tree ubuntu -L 1
.
├── config                          # per-variant static configs
├── config.ini                      # configuration (image geometry, policy,..)
├── docs                            # Reference and usage for each Ubuntu image
├── include                         # common and variant functions
├── README.md                       # main README for Ubuntu build
├── script                          # chroot-run helpers script
└── setup_ubuntu_environment.sh     # sourced entry; orchestration helpers
```

6.2.2.1 Ubuntu Builder Configuration (ubuntu/config.ini)

This file defines the core parameters for Ubuntu image assembly. It controls partition sizing, output filenames, base image selection, timezone, user credentials, and optional features such as enabling or disabling the Weston compositor.

The builder automatically discovers and downloads the latest Ubuntu Base (arm64) tarball from the configured Ubuntu CDImage index using URL and filename patterns. If a suitable tarball is already present locally, it will be reused instead of downloading again.

Users may adjust config.ini before running a build to customize system behavior (for example: enlarge the boot partition, change the default username and password, or set the correct timezone). These changes directly affect the generated Ubuntu Core or LXDE image.

Core parameters (summary):

- **UBUNTU_TYPE**: Type of target Ubuntu. Available types are "**CORE**", "**LXDE**", and "**ALL**". ("ALL" option will build all Ubuntu types)
- **BOOT_SIZE_MB**: Size of the boot partition in MB. It should be larger than 100MB.
- **WIC_ROOTFS_PARTITION_OVERHEAD_FACTOR**: Overhead factor for the rootfs partition in WIC. The default is 1.3 (30%), which is a common default for WIC. Use 1.0 to disable overhead.
- **ROOTFS_INTERNAL_FREE_SPACE_MB**: Default extra free space to add inside the root filesystem (in MB). This space is available to the user/system after booting.
- **UBUNTU_BASE_URL**: The primary URL to use for landing at the download page.
- **UBUNTU_BASE_URL_FILE_PATTERN**: The regex pattern used to filter out the requisite image file from the download page.
- **UBUNTU_BASE_FILE_PATTERN**: The file pattern used to search for the local CD image archive previously downloaded. It's passed on to a find command.
- **UBUNTU_LATEST_IMAGE**: A composite variable that automatically determines the image to download from those available at the release page.
- **DOWNLOAD_DIR**: Specify the path to place the downloaded CD image archive. DOWNLOAD_DIR can be set here, but will default to './' during execution if left empty here.
- **LOCAL_FILE**: A shorthand variable that points to where the previous CD image archive might be present.
- **OUTPUT_ROOTFS**: The output file name for the rootfs.

- **OUTPUT_WIC**: The output file name for the wic image.
- **TIME_ZONE_AREA**: The time zone area (e.g., "Asia").
- **TIME_ZONE_CITY**: The time zone city (e.g., "Ho_Chi_Minh").
- **IS_WESTON_ENABLE**: Set to 0 to disable Weston compositor.
- **USERNAME:** The default username for logging into the system (e.g., "rz"). This account is used for user login during system access.
- **PASSWORD**: The password associated with the default **USERNAME** (e.g., "1"). This password is required to authenticate the user during login.

6.2.3 DRP/AI Enablement

DRP/AI (DRP-AI) is controlled by the features section in layer_override.json, which is described in the section rz_builder.sh. The builder applies these settings before BitBake runs by updating BBLAYERS and image configuration. DRP/AI is enabled when the DRP-AI meta-layer is included in features.layers.add, and it is disabled when the same layer is listed in features.layers.remove.

The following layer_override.json example enables DRP/AI by adding the DRP-AI feature layer:

```
rzv_drp-ai_driver/meta-rz-features/meta-rz-drpai
```

In addition, the OpenCV accelerator layer is enabled (platform-dependent):

```
rzv2h_opencv_accelerator/meta-rz-features/meta-rz-opencva
```

To enable DRP/AI, edit features.layers and add the DRP/AI meta-layer to the add field. The features.layers.add list may already contain other meta-layers that are not related to DRP/AI; those entries should be kept unchanged.

```
"features": {
  "layers": {
    "add": [
      "rzv2h_opencv_accelerator/meta-rz-features/meta-rz-opencva",
      "rzv_drp-ai_driver/meta-rz-features/meta-rz-drpai"
    ],
    "remove": []
  },
  ...
}
```

To disable DRP/AI for the build, move the DRP-AI layer from features.layers.add to features.layers.remove:

```
"features": {
  "layers": {
    "add": [],
    "remove": [
      "rzv2h_opencv_accelerator/meta-rz-features/meta-rz-opencva",
      "rzv_drp-ai_driver/meta-rz-features/meta-rz-drpai"
    ]
  },
  ...
}
```

6.3 Yocto OE Build

6.3.1 Initiate Yocto Build

Add execute permission to rz_builder.sh

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ chmod a+x rz_builder.sh
```

Commence build.

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ IMAGE=<target-images> ./rz_builder.sh
build
```

See Section 1.1 Supported Distributions for the full list of available images. To build core-image-bsp, use the following command:

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ IMAGE=core-image-bsp ./rz_builder.sh
build
```

Note: This build requires internet access and takes several hours. Use a build system with high core count, a lot of RAM memory, and a fast SSD to have quicker builds.

6.3.2 Collect the Build Output

After building Yocto with the ‘all-supported-images’ option, which builds all images at once, the output folder should be located at: `~/renesas/rz-cmn-srp/yocto_rzcmn_board/build/tmp/deploy/images/rz-cmn`
The output folder outline should look as follows:

```
renesas@builder-pc:~/renesas/rz-cmn-
srp/yocto_rzcmn_board/build/tmp/deploy/images/rz-cmn$ tree
.
├── host
│   ├── build
│   │   ├── <image-name>-<timestamp>.rootfs.manifest
│   │   ├── <image-name>-<timestamp>.testdata.json
│   │   ├── <image-name>.manifest -> <image-name>-<timestamp>.rootfs.manifest
│   │   └── <image-name>.testdata.json -> <image-name>-<timestamp>.testdata.json
│   ├── env
│   │   ├── core-image-bsp.env
│   │   ├── core-image-minimal.env
```

```
|   |   ├── core-image-weston.env
|   |   ├── Readme.md
|   |   ├── renesas-core-image-cli.env
|   |   ├── renesas-core-image-weston.env
|   |   ├── renesas-quickboot-cli.env
|   |   └── renesas-quickboot-wayland.env
|   ├── Readme.md
|   ├── src
|   |   └── rz-cmn-srp
|   |       ├── config.json
|   |       ├── layer_override.json
|   |       ├── files_to_add
|   |       |   └── meta-rz-features
|   |       |       ├── 0001-rzg2l-sbc-Bring-compat_alloc_user_space-back.patch
|   |       |       └── 0004-rzg2l-sbc-Get-interrupt-number.patch
|   |       ├── git_patch.json
|   |       ├── jq-linux-amd64
|   |       ├── patches
|   |       |   ├── meta-rz-features
|   |       |   |   └── 0001-support-codec-for-linux-6.10-and-yocto-styhead.patch
|   |       |   ├── meta-summit-radio
|   |       |   |   ├── 0001-rz-sbc-meta-summit-radio-Support-build-in-yocto-
styh.patch
|   |       |   |   └── 0002-rz-sbc-summit-radio-support-eSDK-build.patch
|   |       |   └── poky
|   |       |       └── 0001-uboot-config-Fix-devtool-modify.patch
|   |       ├── README.md
|   |       ├── rz_builder.sh
|   |       └── ubuntu
|   |           ├── config
|   |           |   ├── common
|   |           |   |   ├── hostapd.conf
|   |           |   |   ├── hostapd.service
|   |           |   |   ├── moal.conf
|   |           |   |   └── resolved.conf
|   |           |   ├── ubuntu_core
|   |           |   |   ├── audio-init-core.sh
|   |           |   |   ├── network_interfaces.conf
|   |           |   |   └── NetworkManager.conf
|   |           |   └── ubuntu_lxde
|   |           |       ├── audio-init-lxde.sh
|   |           |       ├── connman-gtk.desktop
|   |           |       ├── force-display-xorg.sh
|   |           |       ├── force-xorg-display.service
|   |           |       ├── interfaces
|   |           |       ├── lightdm.conf
|   |           |       ├── NetworkManager.conf
```

```
|   |           |       ├── panel
|   |           |       ├── rsyslog
|   |           |       ├── ttyS0.conf
|   |           |       ├── v4l2-init.service
|   |           |       └── v4l2-init.sh
|   |           ├── config.ini
|   |           ├── docs
|   |           |   ├── ubuntu_core
|   |           |   |   └── README.md
|   |           |   └── ubuntu_lxde
|   |           |       ├── Pictures
|   |           |       |   ├── audacity.png
|   |           |       |   ├── audio_settings.png
|   |           |       |   ├── bluetooth_0.png
|   |           |       |   ├── bluetooth_1.png
|   |           |       |   ├── bluetooth_2.png
|   |           |       |   ├── bluetooth_3.png
|   |           |       |   ├── bluetooth_4.png
|   |           |       |   ├── csi_0.png
|   |           |       |   ├── csi_1.png
|   |           |       |   ├── csi_2.png
|   |           |       |   ├── eth_1.png
|   |           |       |   ├── eth_2.png
|   |           |       |   ├── eth_3.png
|   |           |       |   ├── eth_4.png
|   |           |       |   ├── eth_5.png
|   |           |       |   ├── eth.png
|   |           |       |   ├── save_audio_0.png
|   |           |       |   ├── save_audio_1.png
|   |           |       |   ├── save_audio_2.png
|   |           |       |   ├── vlc_open_0.png
|   |           |       |   ├── vlc_open_1.png
|   |           |       |   ├── vlc_open_2.png
|   |           |       |   ├── vlc.png
|   |           |       |   ├── vlc_video_1.png
|   |           |       |   ├── vlc_video.png
|   |           |       |   ├── web_1.png
|   |           |       |   ├── web_2.png
|   |           |       |   ├── web_lxterm_htop.png
|   |           |       |   ├── web.png
|   |           |       |   └── wifi_0.png
|   |           |       └── README.md
|   |           ├── include
|   |           |   ├── common
|   |           |   |   ├── allow_empty_password.sh
|   |           |   |   ├── create_wic.sh
|   |           |   |   ├── install_gstreamer.sh
```

```
|   |           |   |   ├── install_imdt_utils.sh
|   |           |   |   ├── install_weston.sh
|   |           |   |   ├── mount.sh
|   |           |   |   ├── prepare_env_rootfs.sh
|   |           |   |   ├── prepare_env.sh
|   |           |   |   ├── prepare_ubuntu_base.sh
|   |           |   |   ├── setup_conf.sh
|   |           |   |   └── yocto_working.sh
|   |           |   ├── ubuntu_core
|   |           |   |   ├── prepare_conf.sh
|   |           |   |   ├── prepare_env.sh
|   |           |   |   └── prepare_rootfs.sh
|   |           |   └── ubuntu_lxde
|   |           |       ├── create_swap.sh
|   |           |       ├── prepare_conf.sh
|   |           |       └── prepare_rootfs_qt.sh
|   |           ├── README.md
|   |           ├── script
|   |           |   ├── common
|   |           |   |   ├── dpkg-install-lock-fix.sh
|   |           |   |   ├── enable_ping.sh
|   |           |   |   └── setup_dns_and_time.sh
|   |           |   ├── ubuntu_core
|   |           |   |   ├── apt_install_base.sh
|   |           |   |   ├── link_to_leagcy_iptables.sh
|   |           |   |   └── set_root_password.sh
|   |           |   └── ubuntu_lxde
|   |           |       ├── apt_audio_video.sh
|   |           |       ├── apt_blueman.sh
|   |           |       ├── apt_install_base.sh
|   |           |       ├── apt_lxde_desktop.sh
|   |           |       ├── apt_wifi_ble.sh
|   |           |       ├── create_user.sh
|   |           |       ├── enable_service.sh
|   |           |       ├── set_root_password.sh
|   |           |       ├── set_swap_enable.sh
|   |           |       └── setup-set-permissions.sh
|   |           └── setup_ubuntu_environment.sh
|   └── tools
|       ├── bin
|       |   ├── linux
|       |   |   ├── bpgen
|       |   |   ├── fiptool
|       |   |   ├── libcrypto.so.1.1
|       |   |   ├── OPENSSL_LICENSE.txt
|       |   |   └── Readme.md
|       |   ├── Readme.md
```

```
│       │   └── windows
│       │       ├── bpgen.exe
│       │       ├── fiptool.exe
│       │       ├── GNU_BINUTILS_LICENSE.txt
│       │       ├── libcrypto-3-x64.dll
│       │       ├── libwinpthread-1.dll
│       │       ├── LIBWINPTHREAD_LICENSE.txt
│       │       ├── objcopy.exe
│       │       ├── OPENSSL_LICENSE.txt
│       │       └── Readme.md
│       ├── bootloader_flasher
│       │   ├── bootloader_flash.py
│       │   └── README.md
│       ├── config
│       │   ├── boards_flash_config.toml
│       │   └── README.md
│       ├── firmware_compile
│       │   ├── firmware_compile.py
│       │   └── Readme.md
│       ├── flash_images.json
│       ├── README.md
│       ├── requirements.txt
│       ├── sd_creator
│       │   ├── README.md
│       │   ├── sd_flash.py
│       │   └── tools
│       │       ├── AdbWinApi.dll
│       │       ├── AdbWinUsbApi.dll
│       │       ├── fastboot.exe
│       │       └── NOTICE.txt
│       ├── uload_bootloader
│       │   ├── README.md
│       │   └── uload_bootloader_flash.py
│       └── universal_flash.py
├── license
│   └── Disclaimer051.pdf
├── <code>-rz-cmn-srp-um-quick-start-guide.pdf
├── <code>-rz-cmn-srp-um.pdf
├── README.md
├── RZ_System_Release_Package_Evaluation_license.pdf
└── target
    ├── env
    │   ├── Readme.md
    │   └── uEnv.txt
    ├── images
    │   ├── atf
    │   │   ├── bl2-rz-cmn.bin
```

```
    │   │   ├── bl31-rz-cmn.bin
    │   │   ├── fdts
    │   │   │   ├── <board-name>.dtb
    │   │   │   └── Readme.md
    │   │   └── Readme.md
    │   ├── core-image-bsp.wic
    │   ├── core-image-minimal.wic
    │   ├── core-image-weston.wic
    │   ├── Flash_Writer_SCIF_<board-name>.mot
    │   ├── Flash_Writer_SCIF_<board-name>_PMIC.mot
    │   ├── linux
    │   │   ├── dtbs
    │   │   │   ├── overlays
    │   │   │   │   ├── Readme.md
    │   │   │   │   └── <board-name>-<rev.major>.<rev.minor>-<feature>.dtbo
    │   │   │   ├── <board-name>--<kernel-version>-rz-cmn-<timestamp>.dtbo
    │   │   │   ├── <board-name>.dtb -> <board-name>--<kernel-version>-rz-cmn-
<timestamp>.dtbo
    │   │   │   └── Readme.md
    │   │   ├── Image -> Image--<kernel-version>-rz-cmn-<timestamp>.bin
    │   │   ├── Image--<kernel-version>-rz-cmn-<timestamp>.bin
    │   │   └── Readme.md
    │   ├── Readme.md
    │   ├── renesas-core-image-cli.wic
    │   ├── renesas-core-image-weston.wic
    │   ├── renesas-quickboot-cli.wic
    │   ├── renesas-quickboot-wayland.wic
    │   ├── rootfs
    │   │   ├── core-image-bsp.tar.bz2
    │   │   ├── core-image-minimal.tar.bz2
    │   │   ├── core-image-weston.tar.bz2
    │   │   ├── Readme.md
    │   │   ├── renesas-core-image-cli.tar.bz2
    │   │   ├── renesas-core-image-weston.tar.bz2
    │   │   ├── renesas-quickboot-cli.tar.bz2
    │   │   └── renesas-quickboot-wayland.tar.bz2
    │   ├── <board>-<version>-platform-settings.bin
    │   ├── <board>-<version>-platform-settings.srec
    │   └── u-boot
    │       ├── dtbs
    │       │   ├── Readme.md
    │       │   └── <board-name>.dtb
    │       ├── Readme.md
    │       └── u-boot-nodtb-rz-cmn.bin
    └── Readme.md
```

6.4 Ubuntu OE Build

This section describes how to build a custom Ubuntu Core image for the RZ-based system. The process involves using the chroot method, which creates an isolated environment and utilizes a compressed Ubuntu-based file as the foundation. The overall build process relies on a Yocto-based environment to generate the necessary system files. The steps outlined below cover the entire process, including setting up the build host environment and creating the Ubuntu image for the RZ-based system.

What is chroot?

chroot (change root) is a Unix command that changes the apparent root directory for a process and its children, effectively isolating the environment for building or running applications. This is useful for creating custom system images, as it ensures that the environment is clean and separated from the host system.

6.4.1 Ubuntu Build Host Environment Setup

To begin the build process, it is essential to set up the build host environment. This involves configuring the necessary tools and dependencies to prepare for the Ubuntu image build.

The preparation steps for the Ubuntu build host are the same as those described for the Yocto build in the section Build Host Environment Setup The same workspace, dependencies, and prerequisite files must be in place.

Table 20. Prerequisite files from the release package for the Ubuntu build

File	Description
rz-cmn-srp/	Main folder for Yocto/Ubuntu build environment for RZ platforms.
rz_builder.sh	Custom master build script that downloads required packages and ZIP files, configures meta layers, sets up the environment, and builds for both Yocto and Ubuntu target images.
site.conf	An override file that targets a specific build version.

patches	This is a folder that contains additional patches that are needed for the Yocto eSDK build. The patches are organized as follows:
	• meta-summit-radio/
	  • 0001-rz-sbc-meta-summit-radio-Support-build-in-yocto-styh.patch
	  • 0002-rz-sbc-summit-radio-support-eSDK-build.patch
	• meta-rz-features/meta-rz-codecs
	  • 0001-support-codec-for-linux-6.10-and-yocto-styhead.patch
	  • 0002-kernel-module-uvcs-drv-update-following-the-kernel-6.patch
	• poky
	  • 0001-uboot-config-Fix-devtool-modify.patch

files_to_add	A folder containing additional files that need to be added to the meta layer. These files are specified in the add_files field of git_patch.json, which defines their source locations and target destinations.
	• meta-rz-features /
	  • 0001-rzg2l-sbc-Bring-compat_alloc_user_space-back.patch
	  • 0004-rzg2l-sbc-Get-interrupt-number.patch
	  • 0005-src-lkm-support-Linux-6.18-kernel-APIs.patch

git_patch.json	A configuration file that contains JSON keys and repository configuration, such as: url, branch, tag, commit, repo type, and patch paths to apply.
config.json	Contains the config options, including available build image options grouped by build type, including Yocto images, Ubuntu images, and static image collections (all-yocto-images, all-ubuntu-images, all-supported-images).
layer_override.json	Pre-build customization file to add/remove meta-layers in BBLAYERS, append/exclude image packages, and select GPU mode (none/panfrost/mali). Applied before BitBake runs.
jq-linux-amd64	A lightweight and flexible tool that supports parsing JSON files.
README.md	A README file describing all the necessary information about the building process.
ubuntu/	Main folder for Ubuntu-based image generation for the RZ platform.
ubuntu/config	The folder that holds configuration files for different Ubuntu variants.
ubuntu/docs	Contains documentation detailing supported features and usage instructions for each Ubuntu image variant.
ubuntu/include	Contains scripts related to Ubuntu (e.g., for creating WIC files, packaging the root filesystem, preparing the environment, etc.)
ubuntu/script	The folder that contains all scripts related to Ubuntu image creation.
ubuntu/setup_ubuntu_environment.sh	Main entry-point script (acts like a dispatcher/header). It sources and sequences logic from the modular scripts under script/. It does not build anything by itself.
ubuntu/config.ini	A configuration file that defines key parameters for the Ubuntu image build process, such as the Ubuntu variant, base image, output filenames, and system settings.

Note: The Ubuntu build process relies on artifacts generated by the Yocto environment, such as the kernel, bootloader, and device tree. These are produced through the included Yocto build scripts and are required for creating a functional Ubuntu image for the RZ/G2L-SBC.

Eventually, all the necessary files for the Ubuntu build should be present in the ‘renesas/rz-cmn-srp’ folder as shown below.

```
renesas@builder-pc:~/renesas/$ tree -L 3
.
└── rz-cmn-srp
    ├── config.json
    ├── files_to_add
    │   └── meta-rz-features
    ├── git_patch.json
    ├── jq-linux-amd64
    ├── patches
    │   ├── meta-rz-features
    │   ├── meta-summit-radio
    │   └── poky
    ├── README.md
    ├── rz_builder.sh
    └── ubuntu
        ├── config
        ├── config.ini
        ├── docs
        ├── include
        ├── README.md
        ├── script
        └── setup_ubuntu_environment.sh
```

6.4.2 Initial Ubuntu Build

Before starting the build, review the Ubuntu Builder Configuration (ubuntu/config.ini) file and adjust any required settings such as partition sizes, credentials, or base image selection. Once configured, add execute permission to rz_builder.sh and run it with the appropriate options to generate the desired Ubuntu image.

> Note: Host PC with Ubuntu 24.04 is recommended for the build. Prepare environment for building package and local build environment

To perform a build, first go to the rz-cmn-srp folder

```
renesas@builder-pc:~/$ cd ~/renesas/rz-cmn-srp
```

Add execute permission to rz_builder.sh.

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ chmod a+x rz_builder.sh
```

Before running the build script, please ensure that this source belongs to a regular user (not root or a privileged user), and the user executing this must have sudo/root privileges.

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ IMAGE=<target-image> ./rz_builder.sh build
```

Run the following command with the appropriate option:

- ubuntu-core: Build Ubuntu core image
- ubuntu-lxde: Build Ubuntu LXDE image (with graphic stacks).
- all-ubuntu-images: Build both Ubuntu LXDE and Ubuntu Core

For example, to build ubuntu-lxde image:

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ IMAGE=ubuntu-lxde ./rz_builder.sh build
```

Note: Please note that this build requires internet access and will take several hours. Use a build system with high core count, lot of RAM memory and a fast SSD to have quicker builds.

6.4.3 Collect the Build Output

After building Ubuntu LXDE and Ubuntu Core, the output folder should be located at: `~/renesas/rz-cmn-srp/yocto_rzcmn_board/build/tmp/deploy/images/rz-cmn`
The output folder outline should look as follows:

```
renesas@builder-pc:~/renesas/rz-cmn-
srp/yocto_rzcmn_board/build/tmp/deploy/images/rz-cmn$ tree
.
├── host
│   ├── build
│   │   ├── <image-name>-<timestamp>.rootfs.manifest
│   │   ├── <image-name>-<timestamp>.testdata.json
│   │   ├── <image-name>.manifest -> <image-name>-<timestamp>.rootfs.manifest
│   │   └── <image-name>.testdata.json -> <image-name>-<timestamp>.testdata.json
│   ├── env
│   │   ├── Readme.md
│   │   └── renesas-ubuntu.env
│   ├── Readme.md
│   ├── src
│   │   └── rz-cmn-srp
│   │       ├── config.json
│   │       ├── layer_override.json
│   │       ├── files_to_add
│   │       │   └── meta-rz-features
│   │       │       ├── 0001-rzg2l-sbc-Bring-compat_alloc_user_space-back.patch
│   │       │       └── 0004-rzg2l-sbc-Get-interrupt-number.patch
│   │       ├── git_patch.json
│   │       ├── jq-linux-amd64
│   │       ├── patches
│   │       │   ├── meta-rz-features
│   │       │   │   └── 0001-support-codec-for-linux-6.10-and-yocto-styhead.patch
│   │       │   ├── meta-summit-radio
│   │       │   │   ├── 0001-rz-sbc-meta-summit-radio-Support-build-in-yocto-
styh.patch
│   │       │   │   └── 0002-rz-sbc-summit-radio-support-eSDK-build.patch
│   │       │   └── poky
│   │       │       └── 0001-uboot-config-Fix-devtool-modify.patch
```

```
|   |       ├── README.md
|   |       ├── rz_builder.sh
|   |       └── ubuntu
|   |           ├── config
|   |           |   ├── common
|   |           |   |   ├── hostapd.conf
|   |           |   |   ├── hostapd.service
|   |           |   |   ├── moal.conf
|   |           |   |   └── resolved.conf
|   |           |   ├── ubuntu_core
|   |           |   |   ├── audio-init-core.sh
|   |           |   |   ├── network_interfaces.conf
|   |           |   |   └── NetworkManager.conf
|   |           |   └── ubuntu_lxde
|   |           |       ├── audio-init-lxde.sh
|   |           |       ├── connman-gtk.desktop
|   |           |       ├── force-display-xorg.sh
|   |           |       ├── force-xorg-display.service
|   |           |       ├── interfaces
|   |           |       ├── lightdm.conf
|   |           |       ├── NetworkManager.conf
|   |           |       ├── panel
|   |           |       ├── rsyslog
|   |           |       ├── ttyS0.conf
|   |           |       ├── v4l2-init.service
|   |           |       └── v4l2-init.sh
|   |           ├── config.ini
|   |           ├── docs
|   |           |   ├── ubuntu_core
|   |           |   |   └── README.md
|   |           |   └── ubuntu_lxde
|   |           |       ├── Pictures
|   |           |       |   ├── audacity.png
|   |           |       |   ├── audio_settings.png
|   |           |       |   ├── bluetooth_0.png
|   |           |       |   ├── bluetooth_1.png
|   |           |       |   ├── bluetooth_2.png
|   |           |       |   ├── bluetooth_3.png
|   |           |       |   ├── bluetooth_4.png
|   |           |       |   ├── csi_0.png
|   |           |       |   ├── csi_1.png
|   |           |       |   ├── csi_2.png
|   |           |       |   ├── eth_1.png
|   |           |       |   ├── eth_2.png
|   |           |       |   ├── eth_3.png
|   |           |       |   ├── eth_4.png
|   |           |       |   ├── eth_5.png
```

```
|   |           |       |   ├── eth.png
|   |           |       |   ├── save_audio_0.png
|   |           |       |   ├── save_audio_1.png
|   |           |       |   ├── save_audio_2.png
|   |           |       |   ├── vlc_open_0.png
|   |           |       |   ├── vlc_open_1.png
|   |           |       |   ├── vlc_open_2.png
|   |           |       |   ├── vlc.png
|   |           |       |   ├── vlc_video_1.png
|   |           |       |   ├── vlc_video.png
|   |           |       |   ├── web_1.png
|   |           |       |   ├── web_2.png
|   |           |       |   ├── web_lxterm_htop.png
|   |           |       |   ├── web.png
|   |           |       |   └── wifi_0.png
|   |           |       └── README.md
|   |           ├── include
|   |           |   ├── common
|   |           |   |   ├── allow_empty_password.sh
|   |           |   |   ├── create_wic.sh
|   |           |   |   ├── install_gstreamer.sh
|   |           |   |   ├── install_imdt_utils.sh
|   |           |   |   ├── install_weston.sh
|   |           |   |   ├── mount.sh
|   |           |   |   ├── prepare_env_rootfs.sh
|   |           |   |   ├── prepare_env.sh
|   |           |   |   ├── prepare_ubuntu_base.sh
|   |           |   |   ├── setup_conf.sh
|   |           |   |   └── yocto_working.sh
|   |           |   ├── ubuntu_core
|   |           |   |   ├── prepare_conf.sh
|   |           |   |   ├── prepare_env.sh
|   |           |   |   └── prepare_rootfs.sh
|   |           |   └── ubuntu_lxde
|   |           |       ├── create_swap.sh
|   |           |       ├── prepare_conf.sh
|   |           |       └── prepare_rootfs_qt.sh
|   |           ├── README.md
|   |           ├── script
|   |           |   ├── common
|   |           |   |   ├── dpkg-install-lock-fix.sh
|   |           |   |   ├── enable_ping.sh
|   |           |   |   └── setup_dns_and_time.sh
|   |           |   ├── ubuntu_core
|   |           |   |   ├── apt_install_base.sh
|   |           |   |   ├── link_to_leagcy_iptables.sh
|   |           |   |   └── set_root_password.sh
```

```
│   │           │   └── ubuntu_lxde
│   │           │       ├── apt_audio_video.sh
│   │           │       ├── apt_blueman.sh
│   │           │       ├── apt_install_base.sh
│   │           │       ├── apt_lxde_desktop.sh
│   │           │       ├── apt_wifi_ble.sh
│   │           │       ├── create_user.sh
│   │           │       ├── enable_service.sh
│   │           │       ├── set_root_password.sh
│   │           │       ├── set_swap_enable.sh
│   │           │       └── setup-set-permissions.sh
│   │           └── setup_ubuntu_environment.sh
│   └── tools
│       ├── bin
│       │   ├── linux
│       │   │   ├── bpgen
│       │   │   ├── fiptool
│       │   │   ├── libcrypto.so.1.1
│       │   │   ├── OPENSSL_LICENSE.txt
│       │   │   └── Readme.md
│       │   ├── Readme.md
│       │   └── windows
│       │       ├── bpgen.exe
│       │       ├── fiptool.exe
│       │       ├── GNU_BINUTILS_LICENSE.txt
│       │       ├── libcrypto-3-x64.dll
│       │       ├── libwinpthread-1.dll
│       │       ├── LIBWINPTHREAD_LICENSE.txt
│       │       ├── objcopy.exe
│       │       ├── OPENSSL_LICENSE.txt
│       │       └── Readme.md
│       ├── bootloader_flasher
│       │   ├── bootloader_flash.py
│       │   └── README.md
│       ├── config
│       │   ├── boards_flash_config.toml
│       │   └── README.md
│       ├── firmware_compile
│       │   ├── firmware_compile.py
│       │   └── Readme.md
│       ├── flash_images.json
│       ├── README.md
│       ├── requirements.txt
│       ├── sd_creator
│       │   ├── README.md
│       │   ├── sd_flash.py
│       │   └── tools
```

```
│       │       ├── AdbWinApi.dll
│       │       ├── AdbWinUsbApi.dll
│       │       ├── fastboot.exe
│       │       └── NOTICE.txt
│       ├── uload_bootloader
│       │   ├── README.md
│       │   └── uload_bootloader_flash.py
│       └── universal_flash.py
├── license
│   └── Disclaimer051.pdf
├── <code>-rz-cmn-srp-um-quick-start-guide.pdf
├── <code>-rz-cmn-srp-um.pdf
├── README.md
├── RZ_System_Release_Package_Evaluation_license.pdf
└── target
    ├── env
    │   ├── Readme.md
    │   └── uEnv.txt
    ├── images
    │   ├── atf
    │   │   ├── bl2-rz-cmn.bin
    │   │   ├── bl31-rz-cmn.bin
    │   │   ├── fdts
    │   │   │   ├──  <board-name>.dtb
    │   │   │   └──  Readme.md
    │   │   └── Readme.md
    │   ├── Flash_Writer_SCIF_<board-name>.mot
    │   ├── Flash_Writer_SCIF_<board-name>_PMIC.mot
    │   ├── linux
    │   │   ├── dtbs
    │   │   │   ├── overlays
    │   │   │   │   ├── Readme.md
    │   │   │   │   └── <board-name>-<rev.major>.<rev.minor>-<feature>.dtbo
    │   │   │   ├── <board-name>--<kernel-version>-rz-cmn-<timestamp>.dtbo
    │   │   │   ├── <board-name>.dtb -> <board-name>--<kernel-version>-rz-cmn-
<timestamp>.dtbo
    │   │   │   └── Readme.md
    │   │   ├── Image -> Image--<kernel-version>-rz-cmn-<timestamp>.bin
    │   │   ├── Image--<kernel-version>-rz-cmn-<timestamp>.bin
    │   │   └── Readme.md
    │   ├── Readme.md
    │   ├── rootfs
    │   │   ├── Readme.md
    │   │   ├── renesas-ubuntu.tar.bz2
    │   │   ├── ubuntu-core-image.tar.bz2
    │   │   └── ubuntu-lxde-image.tar.bz2
    │   ├── <board>-<version>-platform-settings.bin
```

```
    │   ├── <board>-<version>-platform-settings.srec
    │   ├── u-boot
    │   │   ├── dtbs
    │   │   │   ├── Readme.md
    │   │   │   └── <board-name>.dtb
    │   │   ├── Readme.md
    │   │   └── u-boot-nodtb-rz-cmn.bin
    │   ├── ubuntu-core-image.wic.gz
    │   └── ubuntu-lxde-image.wic.gz
    └── Readme.md
```

7. Programming / Flashing Images

This section explains how to program and flash various firmware and root file system images onto Renesas boards. It covers firmware components, prerequisites, hardware setup for each board, and usage of the universal flashing script for seamless flashing workflows.

This package contains only the following firmware components:

Table 21. Firmware description

Module	Binary	Stack layer	Description
ROM code	N/A	BL1	This is the internal ROM code that the Arm Cortex SoC's primary core executes at POR.
Flash writer	Flash_Writer_SCIF_ <board>.mot	BL2	This is meant for serial load in factory environments, which is directly loaded onto the SRAM by the BL1 (ROM code) through UART SCIF0. It is then executed to acquire another image on UART SCIF0 to directly flash onto qspi or emmc into the boot sector. It provides a command-based UI.
Arm trusted Firmware-A	bl2-rz-cmn.bin bl31-rz-cmn.bin <board>.dtb	BL2 and BL31	Minimal Trusted Firmware-A implementation without a device tree. The flashing script will combine bl2-rz-cmn.bin with the device tree dynamically during flashing. It comes in only .bin format: • .bin – for raw flashing for native in-system flashing
U-Boot (BL33)	u-boot-nodtb-rz-cmn.bin <board>.dtb	BL33	U-Boot (nodtb) binary and matching device tree. The flashing script combines these into the FIP.
Board Identification	<board>-platform-settings.bin		This binary stores key platform settings for Renesas boards, like model IDs, revisions, memory locations, and image sizes, enabling firmware and bootloaders to identify hardware and locate boot components efficiently during startup or flashing.

Note: This release does **not** ship a prebuilt FIP. Instead, the flashing script will automatically build a valid FIP image at flash time by combining:

- bl31-rz-cmn.bin
- u-boot-nodtb-rz-cmn.bin
- <board>.dtb

It also dynamically merges bl2-rz-cmn.bin with <board>.dtb to create the BL2 binary that is flashed to the boot sector.

7.1 Prerequisites

Before flashing any images, ensure the following system requirements are met on your host PC and that the necessary files and tools are available.

- Operating System:
  - Linux (Ubuntu 20.04 or newer recommended)
  - Windows 10 or newer
- Software Requirements:
  - Python 3.8 or later
  - Firmware release package with images and tools
- Hardware requirements:
  - Required cables: USB, UART debug cables
  - SD card (8GB or larger)

7.1.1 Linux Setup

Follow these steps to prepare a Linux host:

1. Install Python, Binutils, and build tools

```
renesas@builder-pc:~# sudo apt update
renesas@builder-pc:~# sudo apt install python3 python3-pip build-essential
android-tools-fastboot
```

   - python3, python3-pip: Required to run host scripts
   - build-essential (optional): Installs gcc, g++, and make for rebuilding firmware
   - android-tools-fastboot: Install to get the fastboot binary

2. Install Python Dependencies

   If Python 3.12 is in use, set up a virtual environment first.

```
renesas@builder-pc:~/rz-cmn-srp/host/tools# sudo apt install python3.12-venv
renesas@builder-pc:~/rz-cmn-srp/host/tools# python3 -m venv .venv
renesas@builder-pc:~/rz-cmn-srp/host/tools# source .venv/bin/activate
```

   After the virtual environment is active, choose one of the two install methods:

   - Option 1 – Use **requirements.txt** (recommended)

```
renesas@builder-pc:~# cd <path/to/the/package>/host/tools/
renesas@builder-pc:~/rz-cmn-srp/host/tools$ pip3 install -r requirements.txt
```

   - Option 2 – Install manually

```
renesas@builder-pc:~# pip install pyserial
renesas@builder-pc:~# pip install dataclasses (if using python <3.7)
```

7.1.2 Windows Setup

Follow these steps to prepare a Windows host:

1. Install Python3
   - Download and install Python 3 from python.org
   - During installation, enable “Add Python to environment variables.”
2. If pip is missing, repair your Python installation or download get-pip.py and run:

```
PS C:\Users\renesas> py get-pip.py
```

3. Install Python Dependencies: Open one of the following terminals with Administrator privileges:
   - PowerShell

- Git Bash

The example below uses PowerShell, but the same applies to other terminals

- Option 1 – Use **requirements.txt** (recommended)

```
PS C:\Users\renesas> cd <path/to/the/package/host/tools>
PS C:\Users\renesas\rz-cmn-srp\host\tools> py -m pip install -r requirements.txt
```

- Option 2 – Install manually
  - Using the Python launcher:

```
PS C:\Users\renesas> py -m pip install pyserial
PS C:\Users\renesas> py -m pip install tomli
PS C:\Users\renesas> py -m pip install dataclasses       # Only if Python < 3.7
```

  - Or using pip directly (if already in PATH)

```
PS C:\Users\renesas> pip install pyserial
PS C:\Users\renesas> pip install tomli
PS C:\Users\renesas> pip install dataclasses # required only if using Python
versions older than 3.7
```

4. OTG Flashing setup

Fastboot over USB OTG on Windows requires the device's Fastboot / USB-download interface to be bound to WinUSB.

- Put the board into Fastboot (from U-Boot)
  - Connect the board's USB-to-serial to the PC and open Tera Term (115200 8-N-1).
  - Power on and interrupt autoboot to get the U-Boot> prompt.
  - Connect the USB OTG port on the board to the PC.
  - At U-Boot, run the following commands to go to OTG download mode

```
=> setenv serial# Renesas_RZ_CMN
=> saveenv
=> fastboot usb 27
```

- Bind WinUSB with Zadig
  - Download and run Zadig (no installation required)
  - Click Options → List All Devices.
  - In the dropdown, select the Fastboot/bootloader interface (USB Download gadget).
  - On the right, choose WinUSB → click Install Driver (or Replace Driver).

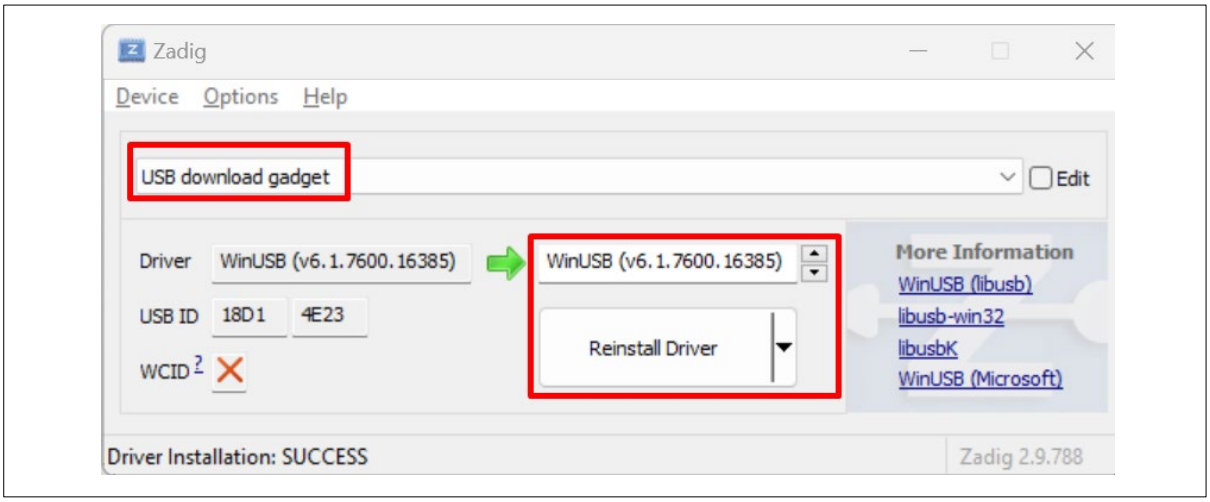


Figure 33. Install WinUSB via Zadig

7.2 Hardware Setup

Flashing methods depend on the type of image being flashed. Follow the setup that matches the operation.

RZ boards are shipped with pre-programmed firmware. However, in scenarios such as factory flashing, image corruption recovery, or when deploying custom builds, re-flashing may be necessary. The Renesas System Package includes tools to perform firmware and root filesystem flashing across different host OS environments.

7.2.1 RZ/G2L-SBC

The RZ/G2L-SBC images consist of:

1. Trusted firmware
2. Multi-stage bootloaders.
3. Linux demo distribution.

The SBC board is designed to boot from a QSPI EEPROM containing the trusted firmware and bootloaders. However, the SBC does not have eMMC storage, and the Linux image is expected to be available on an SD card or a TFTP server.

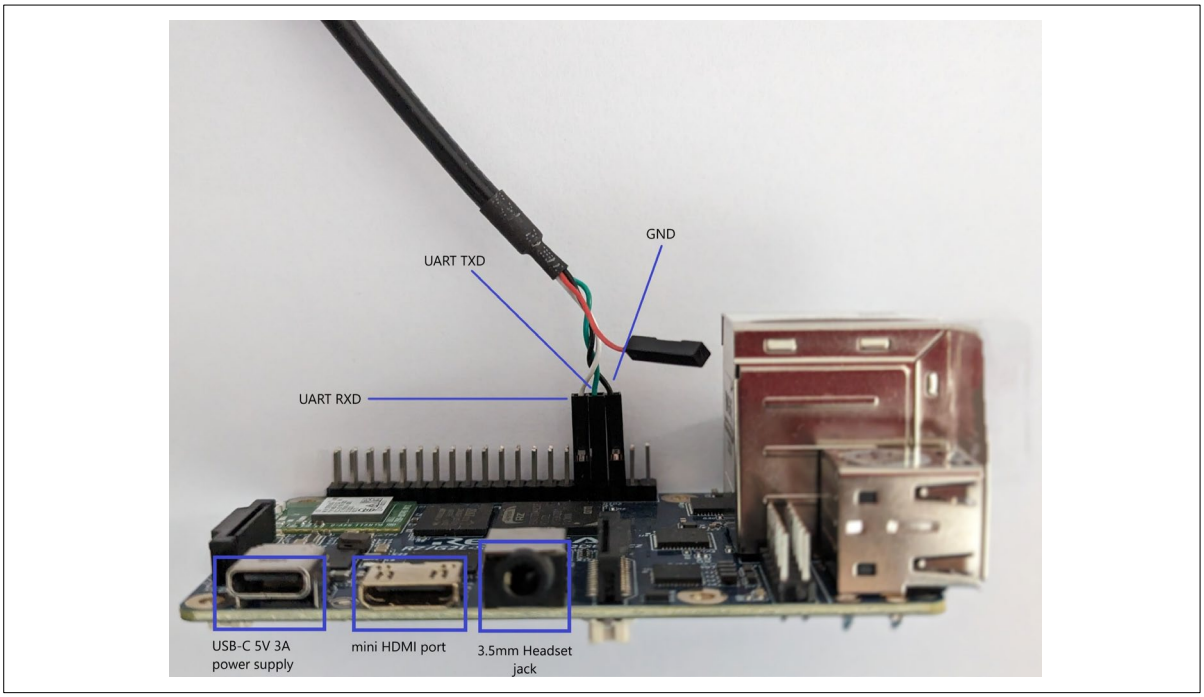


Figure 34. Cortex A55 debug UART cable interface

1. The SD card with the Linux boot image from the release.
2. A USB cable connected to the host PC for flashing and debugging
3. A 5V 3A USB-C power supply.

Other interfaces are not necessary for this purpose.

7.2.2 RS-G2L100

RS-G2L100 development kits ship with pre-programmed firmware and Linux images and include onboard eMMC storage. This allows both firmware and root filesystem images to be flashed directly onto the board, making them suitable for development and production scenarios where frequent updates or customization are required.

Firmware and root filesystem flashing can be done directly onto the eMMC, facilitating faster development cycles and easier updates.

Typical flashing requires:

- The SD card with the Linux boot image from the release.
- A USB cable connected to the host PC for flashing and debugging
- A 12V 3A DC power supply.

7.2.3 RZ/G2L-EVK and RZ/V2L-EVK

Both the RZ/G2L-EVK and RZ/V2L-EVK development kits ship with pre-programmed firmware and Linux images and include onboard eMMC storage. This allows both firmware and root filesystem images to be flashed directly onto the board, making them suitable for development and production scenarios where frequent updates or customization are required.

Firmware and root filesystem flashing can be done directly onto the eMMC, facilitating faster development cycles and easier updates.

Typical flashing requires:

- The SD card with the Linux boot image from the release.
- A USB Type-A to Micro USB Type-B cable connected to the host PC for flashing and debugging
- A USB Type-C charger (65 W)
    - Supports USB PD (Power only)
    - Output specification: 5V3A,9V3A,15V3A,20V3.25A

Other interfaces are not necessary for this purpose.

7.2.4 RZ/V2H-EVK

The RZ/V2H-EVK development kit typically ships with pre-programmed firmware and Linux images. However, updating or customizing these images is common in development or production workflows.

Typical flashing requires:

- The SD card with the Linux boot image from the release.
- A USB Type-A to Micro USB Type-B cable connected to the host PC for flashing and debugging
- Power supply that can provide up to 100W via USB-C PD (not included in the package).

Other interfaces are not necessary for this purpose.

7.2.5 RZ/V2H-RDK

The RZ/V2H-RDK development kit typically ships with pre-programmed firmware and Linux images. However, updating or customizing these images is common in development or production workflows.

Typical flashing requires:

- The SD card with the Linux boot image from the release.
- A USB Type-A to Micro USB Type-B cable connected to the host PC for flashing and debugging
- Power supply, DC power supply: 12 V, ≥ 1 A (12 W or higher), regulated (AC input 100–240 V)

Other interfaces are not necessary for this purpose.

7.2.6 IMDT-V2H-SBC

The IMDT-V2H-SBC development kit typically ships with pre-programmed firmware and Linux images.

Typical flashing requires:

- The SD card with the Linux boot image from the release.
- A USB Type-A to USB Type-C cable connected to the host PC for flashing and debugging.
- Power supply: DC9V/3A via USB-C PD (not included in the package) or DC Barrel jack with DC12V- 24V/2A minimum.

Other interfaces are not necessary for this purpose.

7.3 Universal Flashing Script

The universal_flash.py is a cross-platform script designed to simplify various flashing workflows for Renesas boards. It utilizes a board configuration JSON file to manage image mappings and provides a unified interface for different flashing operations. This is the recommended tool for most flashing tasks.

It's located in the Yocto build output: </path/to/your/yocto/package>/host/tools/universal_flash.py

The script then guides the user through the available options, including board selection and flashing method. Based on the selections, it performs the corresponding flashing operation and displays any required power-cycle or boot-mode steps.

7.3.1 flash_images.json – File Overview and Usage

The flash_images.json file is a board-to-image mapping configuration used by the flashing framework. It serves as a single source of truth that lists:

- Which images (BL2, FIP, U-Boot DTBs, rootfs, etc.) belong to each supported board
- Where those images are in the release package
- Which flashing methods (e.g., xSPI vs eMMC, UDP vs OTG) are applicable to that board

This file is mandatory for universal_flash.py, which parses it to select the correct binaries and flashing procedure for a given board. In short, it links boards → binaries → flashing operations.

Location:

- The file itself must be in the same directory as universal_flash.py.
- Referenced images are expected to be present under the release package with a path following the pattern <path_to_release_package>/target/images/ directory, but may also be organized into subfolders. For example:
  - target/images/
  - target/images/atf/
  - target/images/u-boot/dtbs/

The following section details which files are expected in each location.

7.3.1.1 JSON Structure (Schema)

The file is a JSON object whose keys are board identifiers (e.g., rzg2l-evk, rzg2l-sbc), and whose values are objects with the following fields:

Table 22. flash_images.json schema

Key	Description	Allowed / Example Values
soc	SoC/MPU family identifier	g2l, v2l, v2h
bl2	BL2 (Boot Loader stage 2) image. This BL2 must contain the FCONF device tree.	bl2_bp_<board-name>.srec
board_identification	Board-info binary from binmake tool (see Section 15.1. Board identification JSON Specification). Not the JSON itself.	<board-name>-platform-settings.bin
fip	FIP (TF-A package) image, contains BL31 and u-boot-nodtb.bin with its dtb.	fip_<board-name>.srec
atf_fdts	FCONF DTB for BL2, compiled from DTS (see Section 15.3. TF-A (BL2/BL31)).	<board-name>.dtb
uboot_dtb	U-Boot device tree blob	<board-name>.dtb
flash_writer	Flash Writer binary for low-level programming	Flash_Writer_SCIF_<board-name>.mot
ipl_flash_method	IPL media used for flashing	xspi, emmc
rootfs	Root filesystem image	core-image-minimal.wic
rootfs_flash_method	Method to flash rootfs image.	udp, otg

The table below shows, for each supported board, which IPL flash method the IPL uses to write boot components (xspi or emmc) and which rootfs flash method is used to write the .wic image (udp or otg) as defined in flash_images.json is currently supported

Table 23. Supported IPL flash and rootfs method

Board	SoC/MPU	ipl_flash_method	Default	rootfs_flash_ method	Default
rzg2l-sbc	g2l	xspi	xspi	udp	udp
rs-g2l100	g2l	xspi	xspi	udp, otg	otg
rzg2l-evk	g2l	xspi, emmc or esd	xspi	udp, otg	otg
rzv2l-evk	v2l	xspi, emmc or esd	xspi	udp, otg	otg
rzv2h-evk	v2h	xspi or esd	xspi	udp, otg	otg
rzv2h-rdk	v2h	xspi	xspi	udp	udp
imdt-v2h-sbc	v2h	xspi	xspi	udp, otg	otg

Note:

- IPL flash method: emmc for rzv2h-evk is not supported yet.
- The RZ/G2L-SBC board does not provide a USB OTG port; accordingly, OTG is not supported
- IMDT-V2H-SBC UDP-based flashing: Ethernet operation in U-Boot may be unstable; if UDP-based flashing is required, Ethernet must be initialized in the Linux kernel first and the system rebooted without disconnecting power, otherwise OTG should be used for flashing.

7.3.1.2 Adding a New Board

To add a custom board, create a new top-level entry following the schema described in Section 7.3.1.1 (JSON Structure)

- Provide the correct soc value.
- Generate the board-info binary via binmake and set board_identification to that binary filename (not the JSON source).
- Create/compile the FCONF DTB for BL2 and set atf_fdts accordingly (see Section 15.3).
- Provide the board's U-Boot DTB via uboot_dtb.
- Choose valid methods for ipl_flash_method (xspi/emmc) and rootfs_flash_method (udp/otg).
- Verify bl2, fip, flash_writer, and rootfs filenames match the delivered artifacts.

Example: A new board entry for RZ/G2L-CUSTOM may be defined as follows:

```
{
  "rzg2l-custom": {
    "soc": "g2l",
    "bl2": "bl2_bp_rzg2l-custom.srec",
    "board_identification": "rzg2l-custom-platform-settings.bin",
    "fip": "fip_rzg2l-custom.srec",
    "atf_fdts": "rzg2l-custom.dtb",
    "uboot_dtb": "rzg2l-custom.dtb",
    "flash_writer": "Flash_Writer_SCIF_rzg2l-custom.mot",
    "ipl_flash_method": "xspi",
    "rootfs": "core-image-minimal.wic",
    "rootfs_flash_method": "udp"
  }
}
```

7.3.2 Usage and Flashing Operations

The universal_flash.py serves as the main entry point for performing all flashing operations. It uses the board configuration defined in flash_images.json to select the appropriate images and procedures.

All required images (bl2, bl31, u-boot-nodtb, DTBs, rootfs) must be placed under:

Table 24. Required Image Locations and Resolution Priority

Component	Expected Location	Purpose	Override (from flash_images.json)
BL2/BL31	• </path/to/yocto/package>/target/images/atf/bl2-rz-cmn.bin • </path/to/yocto/package>/target/images/atf/bl31-rz-cmn.bin	Bootloader stages are responsible for early initialization (BL2) and runtime services (BL31).	Not overridable – always taken from the default location.
BL2 FCONF Device Trees	</path/to/yocto/package>/target/images/atf/fdts/	Configuration device trees for BL2 (DDR, PFC, CPG, etc.).	**atf_fdts** can override with a per-board FCONF DTB
U-Boot (no DTB)	</path/to/yocto/package>/target/images/u-boot/u-boot-nodtb-rz-cmn.bin	Per-board U-Boot device trees. Selected automatically by the flashing script.	Not overridable – always taken from the default location.
U-Boot DTBs	</path/to/yocto/package>/target/images/u-boot/dtbs/	Per-board U-Boot device trees. Selected automatically by the flashing script.	**uboot_dtb** can override and select the exact DTB for the board.
Root filesystem	</path/to/yocto/package>/target/images/(e.g., core-image-weston.wic	Complete root file system image written to SD card or eMMC.	**rootfs** and **rootfs_flash_method** override the image and flashing method (e.g., udp, otg).
Flash writer	</path/to/yocto/package>/target/images/Flash-writer-<board>.mot	Initial loader for XSPI/eMMC flashing.	**flash_writer** must define the correct image for the board
Board identification	- </path/to/yocto/package>/target/images/<board>-platform-settings.srec - </path/to/yocto/package>/target/images/<board>-platform-settings.bin	Per-board platform/ID blob used during flashing.	**board_identification** can override and select the file.

After all the required images are set up in their expected locations (and flash_images.json is configured if needed), proceed to run the script.

In general, the release images already include all necessary files. Only when replacing a specific file or adding a new board is it necessary to follow the expected directory layout above and update flash_images.json accordingly.

- **On Linux:** Open a terminal and run the following commands

```
renesas@builder-pc:~$ cd ~/renesas/rz-cmn-srp/host/tools/
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/$ python3 universal_flash.py
```

- **On Windows:** Open one of the following terminals with Administrator privileges:
  - PowerShell
  - Git Bash
  - MobaXterm

  Navigate to the host/tools/ directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> cd C:\Users\renesas\rz-cmn-srp\host\tools\
PS C:\Users\renesas\rz-cmn-srp\host\tools\> py universal_flash.py
```

Upon execution, the script will present an interactive menu to choose the desired flashing operation.

7.3.2.1 Workflow Example

The example below describes the interactive workflow of universal_flash.py using the RZ/V2H-RDK as the reference board. During execution, the script prompts the user to select the target board, serial port, baud rate, and the flashing operations to perform, such as programming the IPL/bootloader and the root file-system.

To run the universal flashing process, follow these steps.

Step 1. Connect the board to the host PC

Step 2. Temporarily power on the board in its current mode so the serial interface can enumerate.
- If the serial port is not visible on the host PC after connecting the USB cable, power on the board and wait for the serial interface to enumerate. On many boards, the UART-over-USB interface appears only after power is applied (e.g., RZ/G2L-EVK, RZ/V2L-EVK, RZ/V2H-RDK, RZ/V2H-EVK)

Step 3. Run the flashing tool
- Start universal_flash.py. For details on how to run it on Windows or Linux, refer to section 7.3.2.

Step 4. Select the target board
The tool displays a numbered list of supported boards. Enter the number corresponding to the target board; do not enter the board name. For example, to select the RZ/V2H-RDK, enter 6 and press **Enter**

```
Available boards:
1. rzv2l-evk
2. rzg2l-evk
3. rzg2l-sbc
4. rs-g2l100
5. rzv2h-evk
6. rzv2h-rdk
7. imdt-v2h-sbc
Select board by number: 6
Selected board: rzv2h-rdk
```

Step 5. The tool enumerates the serial ports detected by the host. Enter the number corresponding to the required port. Pressing Enter without entering a value selects the first port shown in.

```
Available serial ports:
0: COM15
1: COM3
Select a port by number (Default COM15): 0
Selected port [COM15] with baud rate: 115200
```

Step 6. When asked “Write IPL method:”, choose:

```
Write IPL method:
1. BootloaderFlash
2. UloadFlash
Select write IPL method by number:
```

- 1 - BootloaderFlash → Use for blank boards, factory provisioning, recovery of a non-bootable or bricked board, and complete or major boot-firmware updates. This method uses SCIF download mode and Flash Writer to directly erase and program the IPL boot flash.
- 2 – UloadFlash → requires prerequisites (see Sections 7.5.1–7.5.2). Use this for a routine firmware update on a working board that can boot to a valid U-Boot update path. The required BL2, FIP, and board-identification files must be prepared and stored in /uload-bootloader/ on partition 1 (FAT32) of the SD card.

Confirm the choice and press Enter.

Step 7. If option 1 or 2 in **Step 6** above is selected:

- Please power off the board.
- Set the DIP switches to:
  i. SCIF download mode (if option 1 was selected)
  ii. Normal boot mode (if option 2 was selected)
- Power the board back on; now the flashing will begin automatically.

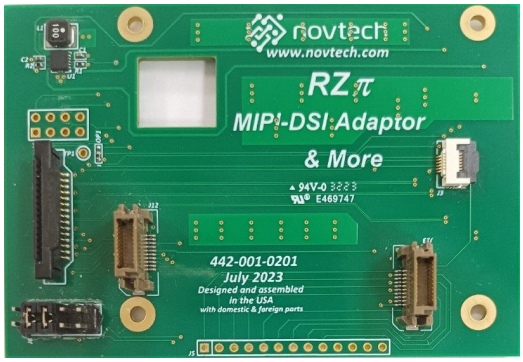
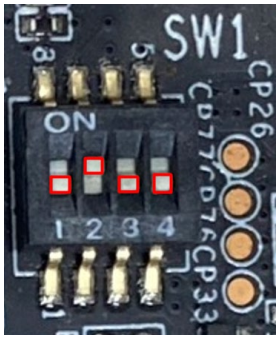
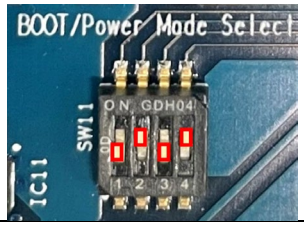
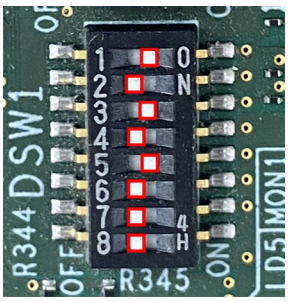
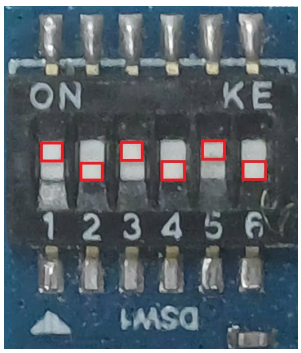
(1) *SCIF Download Mode Configuration (if option 1 was selected)*

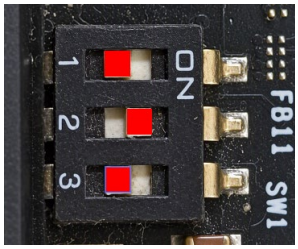
Use the SCIF download mode configuration when 1 – BootloaderFlash is selected in Step 6.

SCIF download mode is required for blank-board programming, factory provisioning, recovery of a non-bootable board, and major IPL/bootloader updates.

Power off the board before changing any DIP switch or jumper setting.

Table 25. SCIF Download Mode for all supported boards

Board	SCIF Download mode switch	Note
RZ/G2L-SBC		This board uses connector-based boot-mode selection rather than a standard DIP switch. Configure the required boot mode using connectors J12 and J13 as shown in the figure before powering on the board
RS-G2L/100		—
RZ/V2L-EVK RZ/G2L-EVK		—
RZ/V2H-EVK		—
RZ/V2H-RDK		—

IMDT-V2H-SBC		—

(2) Normal Boot Mode (If option 2 was selected)

Use the normal boot mode configuration when Option 2: UloadFlash is selected in universal_flash.py, when flashing the root file system, and when booting Linux after flashing is complete.

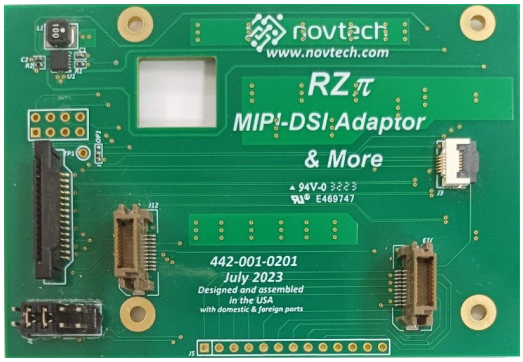
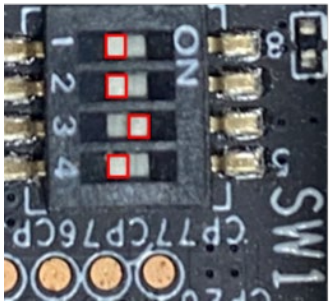
Normal boot mode is required when the board is already able to boot through a valid U-Boot update path. It is also required for rootfs flashing and for normal Linux startup after all flashing operations are complete.

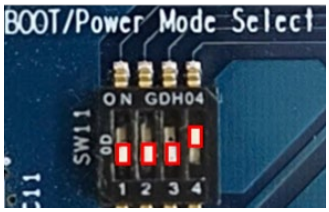
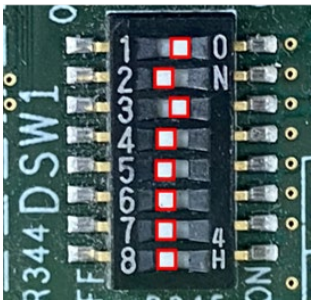
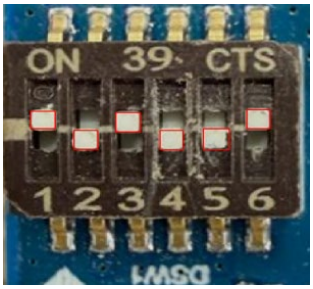
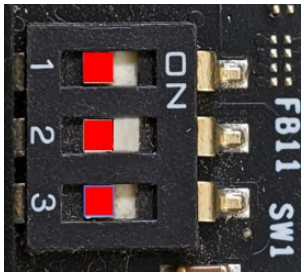
The normal boot mode configurations in this section use xSPI/QSPI boot as the reference boot mode.

Power off the board before changing any DIP switch, jumper, or connector setting. Do not change boot-mode settings while the board is powered on. Set the target board to the normal boot mode configuration shown in the table below, then power the board back on when instructed by the script.

For normal boot configurations using boot media other than xSPI, refer to **Boot Mode Reference (Non-SCIF)** for details information

Table 26. Normal Boot Mode

Board	Normal Boot Mode	Note
RZ/G2L-SBC		Remove the jumpers from J12 and J13 to restore normal boot mode before powering on the board.
RS-G2L/100		Set DIP switch SW1 to the normal boot mode position to set **xSPI** boot mode before powering on the board.

RZ/V2L-EVK RZ/G2L-EVK		Set DIP switch SW11 to the normal boot mode position to set **xSPI** boot mode before powering on the board.
RZ/V2H-EVK		Set DIP switch DSW1 to the normal boot mode position to set **xSPI** boot mode before powering on the board.
RZ/V2H-RDK		Set DIP switch DSW1 to the normal boot mode position to set **xSPI** boot mode before powering on the board.
IMDT-V2H-SBC		Set DIP switch SW1 to the normal boot mode position to set **xSPI** boot mode before powering on the board.

After the board is powered on with the required boot-mode configuration, universal_flash.py starts the selected IPL/bootloader flashing operation automatically. Wait until the IPL/bootloader flashing operation completes successfully. The script then prompts the user to select whether to write the root file system.

Step 8. When asked “Do you want to write the rootfs? (y/n)”.
- y → to flash the root filesystem
- n → to skip rootfs flashing

Step 9. If ‘y’ is selected in **Step 8**
- Please power off the board.
- Set the board to normal boot mode using the configuration described in (2) Normal Boot Mode Configuration.
- Power the board back on; now the flashing rootfs will begin automatically, no other steps are needed.

After the bootloader/IPL flashing operation completes successfully, output similar to the following is displayed. The script writes the firmware files, closes the serial port, and reports the total elapsed time.

```
===== Please Input Qspi Save Address ===
Send . to cancel
Please Input : H
'5F300
Send . to stop
please send !
Writing board identification...
 (binary)
Erase SPI Flash memory...
Erase Completed
Write to SPI Flash memory.
======= Qspi Save Information =================
SpiFlashMemory Stat Address : H'0005F300
SpiFlashMemory End Address : H'0005FB0E
============================================================

>
Board identification write completed.

Closed serial port.
Elapsed time: 83.783759 seconds
```

After the serial port is closed, power off the board, restore the DIP switches to the correct normal boot mode for your board, then power on again to use.

If the board is left in SCIF download mode, it will not boot Linux and may return to the serial downloader/XMODEM prompt.

For the correct DIP switch settings, please refer to the section Factory Firmware Flashing Using Serial Downloader (SCIF) Mode for SCIF download mode and section Boot Mode Reference (Non-SCIF) for normal boot mode

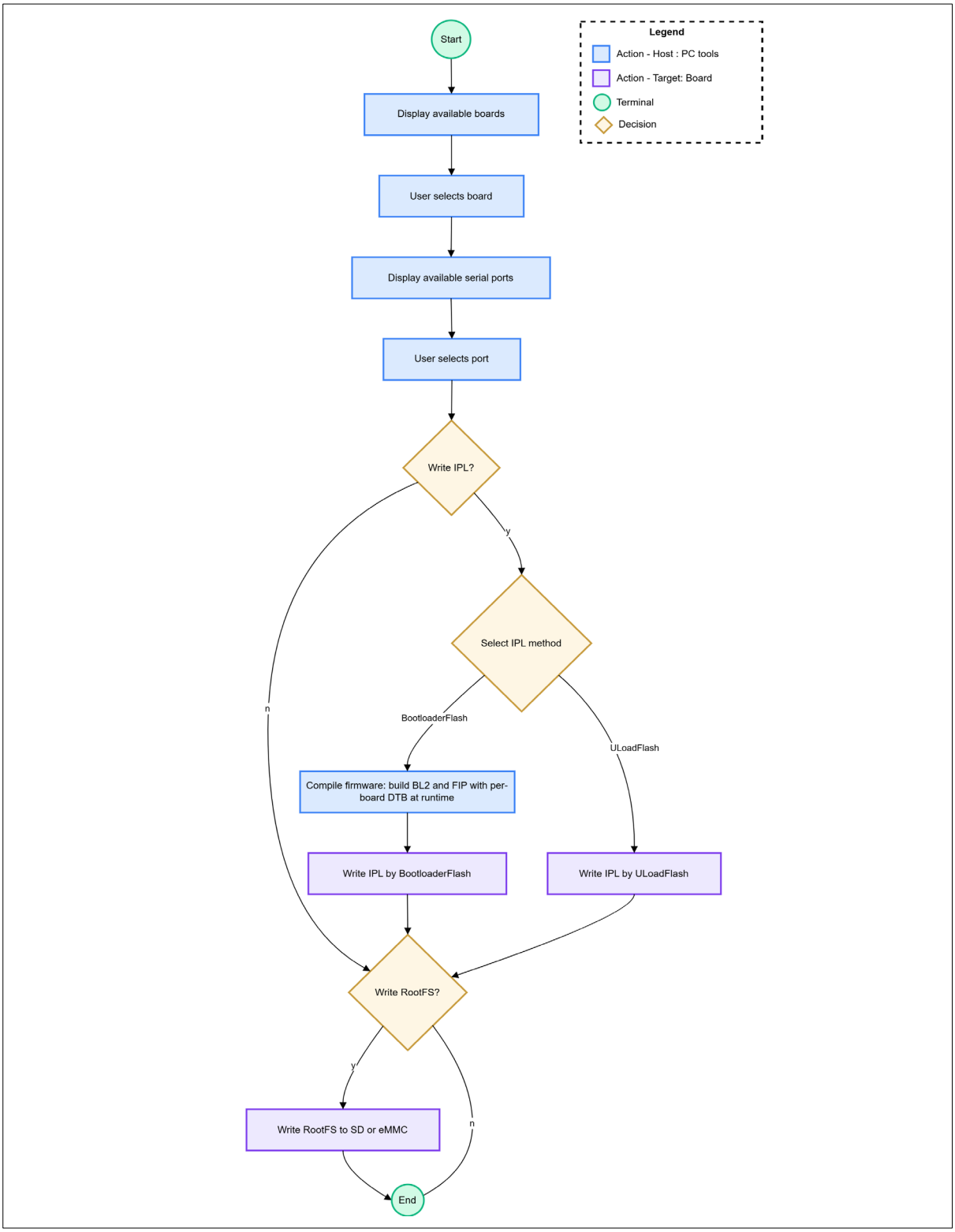


Figure 35. Universal Flashing Script — Interactive Workflow

As shown in the above figure, the script guides the user through the available options and performs the corresponding flashing operation based on the selected board and flashing method. The required power-cycle and boot-mode steps are indicated by the script.

7.4 Flashing the Bootloader

This section describes the dedicated bootloader flashing script, intended for writing the initial bootloader image to the board over a serial interface. It is suitable for first-time provisioning, factory recovery, or focused solely on IPL/bootloader updates.

The universal flashing script (see Section 7.3.2. Usage and Flashing Operations) is the master, multi-operation tool that handles rootfs, and IPL flashing. The bootloader-only script is a streamlined utility that performs bootloader flashing only, with minimal prompts and faster turnaround.

Location:

```
host/tools/bootloader_flasher/bootloader_flash.py
```

7.4.1 Hardware Connection

Before starting the flashing process, ensure that the required hardware interfaces are properly connected between the board and the host PC.

- Connect the debug serial port of the board to the host PC.
- Place the board into **SCIF (serial) download mode before flashing**. The exact DIP-switch or jumper settings are **board-specific**.
    - Refer to Section 16.1. Factory Firmware Flashing Using Serial Downloader (SCIF) Mode for the required switch positions and steps for the target board.

7.4.2 Flashing Procedure

To begin using the bootloader_flash.py, execute it from the host machine:

- **On Linux:** Open a terminal and run the following commands

```
renesas@builder-pc:~$ cd ~/renesas/rz-cmn-srp/host/tools/bootloader_flasher
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/bootloader_flasher$ python3
bootloader_flash.py
```

- **On Windows:** Open one of the following terminals with Administrator privileges:
    - PowerShell
    - Git Bash
    - MobaXterm

  Navigate to the host/tools/bootloader_flasher directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> cd C:\Users\renesas\rz-cmn-srp\host\tools\bootloader_flasher
PS C:\Users\renesas\rz-cmn-srp\host\tools\bootloader_flasher> py
bootloader_flash.py
```

If no arguments are provided, the script will use the following default values:

- Board name: rzg2l-sbc
- Flash method: xspi
- Serial port: most recently connected port (E.g., COM8 in Windows or /dev/ttyUSB0 in Linux)
- Serial port baud: 115200
- Flash Writer Image: /path/to/universal-scripts/target/images/Flash_Writer_SCIF_rzg2l-sbc.mot
- BL2 Image: /path/to/universal-scripts/target/images/bl2_bp_rzg2l-sbc.srec
- FIP Image: /path/to/universal-scripts/target/images/fip_rzg2l-sbc.srec

- Board identification Image: /path/to/universal-scripts/target/images/rzg2l-sbc-platform-settings.bin

Ensure that these files are present in the current directory before executing the script.

To specify custom file paths or override the defaults, the following arguments can be passed:

- **--board_name:** Board name to flash bootloader.
- **--flash_method:** Flash method to use (xspi or emmc).
- **--serial_port:** Serial port to use for communication with the board.
- **--serial_port_baud:** Baud rate for the serial port.
- **--image_writer:** Path to the Flash Writer image.
- **--image_bl2:** Path to the BL2 image.
- **--image_fip:** Path to the FIP image.
- **--image_bid:** Path to the board identification image.

Example command:

- **On Linux:**

```
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/bootloader_flasher$ py
bootloader_flash.py --board_name rzg2l-evk --flash_method emmc --serial_port
COM11 --serial_port_baud 115200 --image_writer
D:\custom_images\Flash_Writer_SCIF_rzg2l-sbc.mot --image_bl2
D:\custom_images\bl2_bp_rzg2l-sbc.srec --image_fip D:\custom_images\fip-rzg2l-
sbc.srec --image_bid D:\custom_images\rzg2l-evk-platform-settings.bin
```

- **On Windows:**

```
PS C:\Users\renesas\rz-cmn-srp\host\tools\bootloader_flasher> python3
bootloader_flash.py --board_name rzg2l-evk --flash_method emmc --serial_port
/dev/ttyUSB0 --serial_port_baud 115200 --image_writer
/home/renesas/custom_images/Flash_Writer_SCIF_rzg2l-sbc.mot --image_bl2
/home/renesas/custom_images/bl2_bp_rzg2l-sbc.srec --image_fip
/home/renesas/custom_images/fip-rzg2l-sbc.srec --image_bid
/home/renesas/custom_images/rzg2l-evk-platform-settings.bin
```

7.5 Flashing the Bootloader from the U-Boot Console (uLoad-bootloader)

This section describes the ULoad-bootloader flow for programming the bootloader from the U-Boot console. It supports QSPI/xSPI flashing and is intended for cases where the device can boot to U-Boot and program flash using images stored on the removable media.

Location:

```
host/tools/uload_bootloader/uload_bootloader_flash.py
```

7.5.1 Prerequisites

The release does not include prebuilt ULoad images on the SD card. The ULoad flow requires BL2 and FIP artifacts that are rebuilt for the selected board with the correct DTB/FCONF and configuration.

Run the firmware_compile.py script to generate these artifacts, then copy them to partition 1 (FAT32) under /uload-bootloader/ before running the ULoad flasher. This ensures the programmed bootloader matches the exact board and release configuration, minimizing risk of mismatch.

To begin with, compile uLoad images; execute them from the host machine:

- **On Linux:** Open a terminal and run the following commands

```
renesas@builder-pc:~$ cd ~/renesas/rz-cmn-srp/host/tools/firmware_compile
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/firmware_compile$ python3
firmware_compile.py
```

- **On Windows:** Open one of the following terminals with Administrator privileges:
  - PowerShell
  - Git Bash
  - MobaXterm

  Navigate to the host/tools/firmware_compile directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> cd C:\Users\renesas\rz-cmn-srp\host\tools\ firmware_compile
PS C:\Users\renesas\rz-cmn-srp\host\tools\firmware_compile> py
firmware_compile.py
```

If no CLI options are supplied, the script uses the following defaults:

Table 27. Firmware compiled CLI options

Option	Default	Description
--board	rzg2l-sbc	Target board name (must exist in boards_flash_config.toml and flash_images.json).
--soc	g2l	Target SoC family (g2l, v2l, v2h).
--method	xspi	Flash method (xspi, emmc or esd).
--bl2	auto from images	Path to BL2 binary (override default).
--atf-fdts	auto from JSON	TF-A FDT(s) to append to BL2.
--uboot-dtbs	auto from JSON	U-Boot DTB(s) to append to U-Boot nodtb.
--bl31	auto from images	Path to BL31 binary (override default).
--u-boot-nodtb	auto from images	Path to U-Boot (nodtb) binary (override default).
--bootparameter	auto search	Path to bpgen tool (override search path).
--fiptool	auto search	Path to fiptool tool (override search path).
--objcopy	auto search	Path to objcopy tool (override search path).
--fip-align	16	FIP alignment.
--fip-vma	from TOML	Override virtual memory address (VMA) for FIP .srec.
--bl2-bp-vma	from TOML	Override VMA for BL2+BP .srec.

Notes:

- auto from images: uses the **common artifacts** produced in target/images/… (not per-board overrides).
- auto from JSON: resolves per-board filenames/DTBs via flash_images.json.
- from TOML: uses addresses/layout defined in boards_flash_config.toml for the selected --board and --method

7.5.2 Collect the Output and Prepare Binaries in the SD Card

This step packages the artifacts built by firmware-compile.py and places them on the removable media so the ULoad-bootloader script (U-Boot console flow) can program QSPI/xSPI.

From <path/to/yocto/package/target/images/>, gather the per-board files:

- bl2: bl2_bp_<board>.bin.

- fip_<board>.bin
- <board>-<version>-platform-settings.bin

Place all files on partition 1 (FAT32) of the SD card under this directory.

```
/uload-bootloader
```

7.5.3 Hardware Connection

Before starting the flashing process, ensure that the required hardware interfaces are properly connected between the board and the host PC.

- Connect the debug serial port of the board to the host PC.
- Place the board into normal boot mode. The exact DIP-switch or jumper settings are board-specific.
  - Refer to Section 16.1. Factory Firmware Flashing Using Serial Downloader (SCIF) Mode for the required switch positions and steps for the target board.

7.5.4 Flashing Procedure

To begin using the uload_bootloader.py, execute it from the host machine:

- **On Linux:** Open a terminal and run the following commands

```
renesas@builder-pc:~$ cd ~/renesas/rz-cmn-srp/host/tools/uload_bootloader
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/uload_bootloader$ python3
uload_bootloader_flash.py
```

- **On Windows:** Open one of the following terminals with Administrator privileges:
  - PowerShell
  - Git Bash
  - MobaXterm

  Navigate to the host/tools/uload_bootloader directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> cd C:\Users\renesas\rz-cmn-srp\host\tools\uload_bootloader
PS C:\Users\renesas\rz-cmn-srp\host\tools\uload_bootloader> py
uload_bootloader_flash.py
```

If no arguments are provided, the script will use the following default values for the RZ/G2L-SBC board.

- bl2 binary file: bl2_bp_<board-name>.bin
- fip file: fip_<board-name>.bin
- board identication file: <board-name>-platform-settings.bin

Ensure that these files are present in the current directory before executing the script.

To specify custom file paths or override the defaults, the following arguments can be passed:

- **--serial_port:** Serial port to use for communication with the board.
- **--serial_port_baud:** Baud rate for the serial port.
- **--image_bl2:** Path or filename of the BL2 image
- **--image_fip:** Path or filename of the FIP image
- **--image_bid:** Path or filename of the board-identification file

Example command:

- **On Linux:**

```
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/uload_bootloader$ python3
uload_bootloader_flash.py \
  --serial_port /dev/ttyUSB0 \
  --serial_port_baud 115200 \
  --image_bl2 /mnt/sd/uload-bootloader/bl2_bp-rzg2l-sbc.bin \
  --image_fip /mnt/sd/uload-bootloader/fip-rzg2l-sbc.bin \
  --image_bid /mnt/sd/uload-bootloader/rzg2l-sbc-platform-settings.bin
```

- **On Windows:**

```
PS C:\Users\renesas\rz-cmn-srp\host\tools\uload_bootloader> py
uload_bootloader_flash.py \
  --serial_port /dev/ttyUSB0 \
  --serial_port_baud 115200 \
  --image_bl2 /mnt/sd/uload-bootloader/bl2_bp-rzg2l-sbc.bin \
  --image_fip /mnt/sd/uload-bootloader/fip-rzg2l-sbc.bin \
  --image_bid /mnt/sd/uload-bootloader/rzg2l-sbc-platform-settings.bin
```

Note:

- If only a filename is provided (no path), the script searches the default directory (e.g., /uload-bootloader on partition 1, FAT32).
- Ensure the filenames match the board that was built with firmware-compile.py.

7.6 Flashing the SD Card Image

This section explains how to use the dedicated SD flashing script, sd_creator.py, which helps create bootable SD cards using a fastboot-like approach.

```
host/tools/sd_creator/
```

Hierarchy:

```
sd_creator
├── README.md
├── sd_flash.py
└── tools
    ├── AdbWinApi.dll
    ├── AdbWinUsbApi.dll
    ├── fastboot.exe
    └── NOTICE.txt
```

7.6.1 Hardware Connection

Before starting the flashing process, ensure that the required hardware interfaces are properly connected between the board and the host PC. The type of connection depends on the flashing method selected.

- Connect the debug serial port of the board to the host PC for console access and monitoring.
- Connect the appropriate interface based on the fastboot method:

- [UDP] Connect the Ethernet port of the board to the host PC.
- [OTG] Connect the USB OTG port of the board to the host PC.

Table 28. SD card flash - Hardware connection

Fastboot Type	Connection to Host	Board Ports Used	Notes
UDP	Ethernet cable + USB cable	RJ45 + Debug Serial	Requires an IP address and Ethernet port index
OTG	USB cable	USB OTG + Debug Serial	No Ethernet/IP required

U-Boot fastboot-udp uses a single active Ethernet MAC per board. If multiple RJ45/PHY ports are present, only one is active (board-dependent). Select the interface with --ether_port. This option maps to the U-Boot Ethernet device index and will be explained in detail in a later section.

Table 29. UDP Ethernet Port Index per board

Board	Ethernet port to use
RZ/G2L-SBC	1
RS-G2L100	0, 1
RZ/V2L-EVK	0
RZ/G2L-EVK	0
RZ/V2H-EVK	0, 1
RZ/V2H-RDK	0
IMDT V2H-SBC	0, 1

Both fastboot-otg and fastboot-udp write to U-Boot's current MMC device (typically mmc0). Depending on board revision, mmc0 may point to the SD card or eMMC.

Table 30. Fastboot target device mapping

Board/Rev	Fastboot Method	Typical mmc0 target	How to change target
RZ/G2L-SBC	UDP	Carrier SD (board default)	N/A (single device)
RS-G2L100	UDP, OTG	eMMC	N/A (single device)
RZ/V2L-EVK	UDP, OTG	SD (CN3 on SOM or eMMC device depending on SW1)	Set SW1-2 ON to SD and OFF to eMMC
RZ/G2L-EVK	UDP, OTG	SD (CN3 on SOM or eMMC device depending on SW1)	Set SW1-2 ON to SD and OFF to eMMC
RZ/V2H-EVK (Rev 1 – 2 SD cards)	UDP, OTG	SD card slot 0	N/A (single device)
RZ/V2H-EVK (Rev 2 – SD & eMMC)	UDP, OTG	eMMC	N/A (single device)
RZ/V2H-RDK	UDP	SD card	N/A (single device)
IMDT V2H-SBC	UDP, OTG	eMMC	N/A (single device)

7.6.2 Flashing Procedure

To begin using sd_creator.py, execute it from the host machine:

- **On Linux:** Open a terminal and run the following commands

```
renesas@builder-pc:~$ cd ~/renesas/rz-cmn-srp/host/tools/sd_creator
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/sd_creator$ python3
sd_flash.py
```

- **On Windows:** Open one of the following terminals with Administrator privileges:

- PowerShell
- Git Bash
- MobaXterma

Navigate to the host/tools/sd_creator directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> cd C:\Users\renesas\rz-cmn-srp\host\tools\sd_creator
PS C:\Users\renesas\rz-cmn-srp\host\tools\sd_creator> py sd_flash.py
```

If no arguments are provided, the script will use the following default values:

- Fastboot type: udp
- IP address: 169.254.187.89
- Serial port: most recently connected port (e.g., COM8 in Windows or /dev/ttyUSB0 in Linux)
- Serial port baud: 115200
- WIC file: </path/to/your/package>/target/images/core-image-minimal.wic

Ensure that these files are present in the current directory before executing the script.

To specify custom file paths or override the defaults, the following arguments can be passed:

- **--board_name**: Board name to flash bootloader. Default is rzg2l-sbc.
- **--fastboot_type**: Fastboot type to use (udp or otg). Default is udp.
- **--ether_port**: [Only used in fastboot UDP] Ethernet port used for board communication. Defaults to 1.
- **--ip_address**: [Only used in fastboot UDP] Ethernet IP address used for board communication. Defaults to 169.254.187.89.
- **--serial_port**: Serial port to use for communication with the board. Default is the most recently connected port (E.g, COM8 in Windows or /dev/ttyUSB0 in Linux).
- **--serial_port_baud**: Baud rate for the serial port. Must be 115200.
- **--image_rootfs**: Path to the root filesystem image.

Example command

For UDP flash:

- **On Linux:**

```
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/sd_creator$ python3 \
sd_flash.py --board_name rzg2l-evk --fastboot_type udp --ip_address \
169.254.187.9 --ether_port 1 --serial_port /dev/ttyUSB0 \
--serial_port_baud 115200 \
--image_rootfs /home/renesas/custom_images/core-image-weston.wic
```

- **On Windows:**

```
PS C:\Users\renesas\rz-cmn-srp\host\tools\sd_creator> py sd_flash.py \
--board_name rzg2l-evk --fastboot_type udp --ip_address 169.254.187.9 \
--ether_port 1 --serial_port COM11 --serial_port_baud 115200 \
--image_rootfs D:\custom_images\core-image-weston.wic
```

For OTG flash:

- **On Linux:**

```
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/sd_creator$ python3 \
sd_flash.py --fastboot_type otg --serial_port /dev/ttyUSB0 \
--serial_port_baud 115200 --image_rootfs \
/home/renesas/custom_images/core-image-weston.wic
```

- **On Windows:**

```
PS C:\Users\renesas\rz-cmn-srp\host\tools\sd_creator> py sd_flash.py \
--fastboot_type otg --serial_port COM11 --serial_port_baud 115200 \
--image_rootfs D:\custom_images\core-image-weston.wic
```

Note:

- For OTG, `--ip_address` and `--ether_port` are not applicable.
- For UDP, choose the correct `--ether_port` for the target board (see the Table 18. UDP Ethernet Port Index per board)
- IMDT-V2H-SBC UDP-based flashing: Ethernet operation in U-Boot may be unstable; if UDP-based flashing is required, Ethernet must be initialized in the Linux kernel first and the system rebooted without disconnecting power, otherwise OTG (default option) should be used for flashing.

8. Accessing Supported Features

This section explores the key features and interfaces available across both Yocto and Ubuntu images on all supported platforms in this release.

8.1 Supported Features in Yocto Images

8.1.1 U-Boot Environment

The uEnv.txt file contains boot configuration settings for both the U-Boot bootloader and the Linux kernel. It resides in the boot partition (Partition 1) of the storage media, which is formatted as FAT32. This format allows the partition to be easily read by most operating systems, including Windows. When the Linux system is running, this partition is typically mounted at /boot.

While the U-Boot environment contains many settings, this guide focuses on the essential ones for the RZ Common System. Each board has its own specific overlay settings; overlays intended for other boards may have no effect.

Device tree overlay loading follows the pattern ${model_string}-${revision_major}.${revision_minor}. These U-Boot environment variables are automatically set by U-Boot during runtime. Details:

- ${model_string}: Board model string (e.g., rzg2l-sbc)
- ${revision_major}: Board major revision
- ${revision_minor}: Board minor revision

For more detailed information, these fields correspond to the board information that is programmed during the bootloader flashing process for each board (see Section Board Identification for detailed information).

Sample rzg2l-sbc device tree overlay list:

- rzg2l-sbc-1.0-ext-i2c.dtbo
- rzg2l-sbc-1.0-ext-spi.dtbo
- rzg2l-sbc-1.0-can.dtbo
- rzg2l-sbc-1.0-dsi.dtbo
- rzg2l-sbc-1.0-ov5640.dtbo

The table below provides a list of all the overlay options available in the provided kernel.

Table 31. Boot configuration settings

Config	Value if set	Loading	Description	Board supported
enable_overlay_i2c	1 or 'yes'	${model_string}-${revision_major}.${revision_minor}-ext-i2c.dtbo	Enables I²C driver enumeration and reconfigures the relevant I/O pins to connect to the I²C peripheral.	RZ/G2L-SBC
enable_overlay_spi	1 or 'yes'	${model_string}-${revision_major}.${revision_minor}-ext-spi.dtbo	Enables the SPI driver enumeration and reconfigures the relevant IO pins to connect to the SPI peripheral.	RZ/G2L-SBC RZ/V2H-RDK

enable_overlay_can	1 or 'yes'	${model_string}-${revision_major}.${revision_minor}-can.dtbo	Enables the CAN driver enumeration and reconfigures the relevant IO pins to connect to the CAN peripheral.	RZ/G2L-SBC RZ/V2H-RDK
enable_overlay_dsi	1 or 'yes'	${model_string}-${revision_major}.${revision_minor}-dsi.dtbo	Enables MIPI-DSI to enumerate the touch panel driver and route video output to the DSI display.	RZ/G2L-SBC, IMDT-V2H-SBC
enable_overlay_csi_ov5640	1 or 'yes'	${model_string}-${revision_major}.${revision_minor}-ov5640.dtbo	Enables the OV5640 CSI camera driver enumeration and loads the v4l2 pipelines.	RZ/G2L-SBC
enable_overlay_csi_ov5645	1 or 'yes'	${model_string}-${revision_major}.${revision_minor}-cru-csi-ov5645.dtbo	Enables the OV5645 CSI camera driver enumeration and loads the v4l2 pipelines.	RZ/G2L-EVK RZ/V2L-EVK
enable_overlay_audio_codec	1 or 'yes'	${model_string}-${revision_major}.${revision_minor}-audio_codec.dtbo	Enable on-board analog audio codec: Enables the integrated analog audio codec for playback and recording via the on-board line-out, headphone, and microphone interfaces. HDMI audio is disabled due to a hardware limitation, as both share the SSI0 bus; using an alternative SSI bus is not supported due to Raspberry Pi compatibility.	RZ/V2H-RDK
enable_overlay_csi22_ar1335	1 or ‘yes’	${model_string}-${revision_major}.${revision_minor}-cru-csi22-ar1335.dtbo	Enable MIPI-CSI2 AR1335 camera on slot 3 (CSI2-2)	IMDT-V2H-SBC
enable_overlay_csi23_ar1335	1 or ‘yes’	${model_string}-${revision_major}.${revision_minor}-cru-csi23-ar1335.dtbo	Enable MIPI-CSI2 AR1335 camera on slot 4 (CSI2-3)	IMDT-V2H-SBC

A Readme.md file with potentially more up-to-date descriptions of the FDT overlays can also be found in the boot partition.

8.1.1.1 How to edit uEnv.txt

Because the ‘uEnv.txt’ file is on a FAT32 partition, it can be modified easily using one of two methods.

Method 1: Editing from a Windows PC (Recommended for simplicity)

This method is straightforward, following the steps below.

1. Access the SD Card: Power down the SBC and remove its SD card. Insert the card into a reader connected to a Windows PC.
2. Open the Boot Partition: Windows will automatically mount the FAT32 partition (Partition 1), which will appear as a removable drive. Open this drive.
3. Edit the File: Locate and open uEnv.txt with a plain text editor like Notepad or Notepad++.
4. Enable a Peripheral: To activate an overlay, set the value of enable_overlay_variable to 1 or 'yes'.
5. Save and Eject: Save the changes. Use the "Safely Remove Hardware and Eject Media" option in Windows before physically removing the SD card to prevent data corruption.
6. Boot the Device: Return the SD card to the SBC and power it on.

Method 2: Editing from the Root File System (Linux)

This method ensures a reliable way to modify the file by manually mounting the boot partition.

1. Identify the boot partition

   Run the lsblk command to list your storage devices. Look for the small FAT32 partition on the SD card.

```
root@rz-cmn:~# lsblk
```

Table 32. Boot Media for all supported boards

Board	Boot media	MMC device	Boot partition device
RZ/G2L-SBC	SD card	MMC 0	/dev/mmcblk0p1
RZ/G2L-EVK **RZ/V2L-EVK**	CN3 SD card or eMMC	MMC 0	/dev/mmcblk0p1
	CN10 SD card slot	MMC 1	/dev/mmcblk1p1
RZ/V2H-EVK	SD card slot 0 or eMMC	MMC 0	/dev/mmcblk0p1
	SD card slot 1	MMC 1	/dev/mmcblk1p1
RZ/V2H-RDK	SD card	MMC 0	/dev/mmcblk0p1
RS-G2L100	eMMC	MMC 0	/dev/mmcblk0p1
	SD card	MMC 1	/dev/mmcblk1p1
IMDT V2H-SBC	eMMC	MMC 0	/dev/mmcblk0p1
	SD card	MMC 1	/dev/mmcblk1p1

2. Create a mount point: Make a temporary directory to mount the partition.

```
root@rz-cmn:~# mkdir -p /mnt/boot_part
```

3. Mount the boot partition identified in the previous step. Replace /dev/mmcblk0p1 with the appropriate device name as listed in Table 32. Boot Media for all supported boards.

```
root@rz-cmn:~# mount /dev/mmcblk0p1 /mnt/boot_part
```

4. Modify the file: Edit the uEnv.txt file using a text editor.

```
root@rz-cmn:~# vi /mnt/boot_part/uEnv.txt
```

5. Save and Sync: After saving the file, you must run the sync command to ensure the changes are written from memory to the SD card.

```
root@rz-cmn:/mnt/boot_part# sync
```

6. Unmount the partition: unmount the partition before rebooting

```
root@rz-cmn:/mnt/boot_part# cd ~
root@rz-cmn:~# umount
root@rz-cmn:~# sync
root@rz-cmn:~# reboot
```

Device tree file changes require the board to be rebooted to take effect.

Method 3: Set the environment directly in U-Boot prompt:

This method allows setting overlay variables directly from the U-Boot prompt without modifying files on the SD card. It is useful when the SD card is not easily accessible or for quick one-time changes.

1. Power on the board and interrupt autoboot:

```
Hit any key to stop autoboot: 0
=>
```

2. Set the desired overlay variable and save:

```
=> setenv <overlay_variable> 1
=> saveenv
=> boot
```

For example, to enable the DSI display overlay:

```
=> setenv enable_overlay_dsi 1
=> saveenv
=> boot
```

Note: Variables set via **saveenv** persist across reboots. To disable an overlay, set the variable to empty and save:

```
=> setenv enable_overlay_dsi
=> saveenv
=> boot
```

Refer to Table 31. Boot configuration settings for the full list of available overlay variables and their supported boards.

8.1.2 Common Yocto Features

8.1.2.1 Generic USB Bluetooth Framework

The RZ boards support the generic USB Bluetooth framework, which is back-ported from the Linux kernel mainline. TP-Link UB500 Bluetooth 5.0 Nan o USB Adapter (Realtek chipset) has been tested and proven to work on the boards.

(1) Establishing a Bluetooth Connection

Note: Ensure you have internet access before running the commands. If the firmware is downloaded for the first time, a reboot of the board is required to ensure the TP-Link UB500 adapter functions properly.

The following steps will guide you on how to enable the TP-Link UB500 adapter:

1. Download the appropriate firmware for the TP-Link UB500 adapter and store it on the target board. This will ensure it is loaded each time the board boots (one-time setup).

```
root@rz-cmn:~# mkdir -p /lib/firmware/rtl_bt
root@rz-cmn:~# curl -s https://raw.githubusercontent.com/Realtek-
OpenSource/android_hardware_realtek/rtk1395/bt/rtkbt/Firmware/BT/rtl8761b_fw -o
/lib/firmware/rtl_bt/rtl8761bu_fw.bin
```

2. By default, Bluetooth is blocked by RFKILL. To unblock it, use the command 'rfkill unblock bluetooth'

```
root@rz-cmn:~# rfkill list
0: hci0: Bluetooth
        Soft blocked: yes
        Hard blocked: no
root@rzg2l-sbc:~# rfkill unblock bluetooth
root@rzg2l-sbc:~# rfkill list
0: hci0: Bluetooth
        Soft blocked: no
        Hard blocked: no
```

3. Verify whether the TP-Link UB500 adapter is properly attached and is running.

Run the following command to ensure that the system has recognized the TP-Link UB500 adapter:

```
rz-cmn:~# hciconfig -a

hci0:   Type: Primary  Bus: USB
        BD Address: E8:48:B8:C8:20:00  ACL MTU: 1021:6  SCO MTU: 255:12
        UP RUNNING
        RX bytes:1773 acl:0 sco:0 events:142 errors:0
        TX bytes:13029 acl:0 sco:0 commands:142 errors:0
        Features: 0xff 0xff 0xff 0xfe 0xdb 0xfd 0x7b 0x87
        Packet type: DM1 DM3 DM5 DH1 DH3 DH5 HV1 HV2 HV3
        Link policy: RSWITCH HOLD SNIFF PARK
        Link mode: PERIPHERAL ACCEPT
        Name: rz-cmn
        Class: 0x000000
        Service Classes: Unspecified
        Device Class: Miscellaneous,
        HCI Version: 5.1 (0xa)  Revision: 0x97b
        LMP Version: 5.1 (0xa)  Subversion: 0xec43
        Manufacturer: Realtek Semiconductor Corporation (93)
```

The TP-Link UB500 adapter is now ready to connect.

3. Connect the Bluetooth device.

Use bluetoothctl to connect to a Bluetooth device:

```
rz-cmn:~# bluetoothctl

[bluetooth]# power on
[bluetooth]# pairable on
[bluetooth]# agent on
[bluetooth]# default-agent
```

Set the target board to be discoverable by other Bluetooth devices:

```
[bluetooth]# discoverable on
```

Enable and disable scan function:

```
[bluetooth]# scan on
[bluetooth]# scan off
```

Pair and connect the device:

```
[bluetooth]# pair FC:02:96:A5:80:97
[bluetooth]# trust FC:02:96:A5:80:97
[bluetooth]# connect FC:02:96:A5:80:97
```

‘FC:02:96:A5:80:97’ is the address of the Bluetooth device. Change it to match your device’s address.

Exit bluetoothctl.

```
[bluetooth]# exit
```

(2) Transferring Files over Bluetooth

To share files between the RZ board and another Bluetooth-enabled device (e.g., a laptop or smartphone), start the obexctl daemon on the RZ board and initiate a connection:

```
root@rz-cmn:~# export $(dbus-launch)

root@rz-cmn:~# /usr/libexec/bluetooth/obexd -r /home/root -a -d & obexctl
[1] 595
[NEW] Client /org/bluez/obex
[obex]#
[obex]# connect FC:02:96:A5:80:97
Attempting to connect to FC:02:96:A5:80:97
[NEW] Session /org/bluez/obex/client/session0 [default]
[NEW] ObjectPush /org/bluez/obex/client/session0
Connection successful
```

‘FC:02:96:A5:80:97’ is the address of the Bluetooth device. Change it to match your device’s address.

Then, to send files, use the ‘send’ command while connected to the OBEX Object Push profile.

```
[FC:02:96:A5:80:97]# send /boot/uEnv.txt
Attempting to send /boot/uEnv.txt to /org/bluez/obex/client/session0
[NEW] Transfer /org/bluez/obex/client/session0/transfer0
Transfer /org/bluez/obex/client/session0/transfer0
        Status: queued
        Name: uEnv.txt
        Size: 2069
        Filename: /boot/uEnv.txt
        Session: /org/bluez/obex/client/session0
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Status: complete
[DEL] Transfer /org/bluez/obex/client/session0/transfer0
[FC:02:96:A5:80:97]# quit
```

8.1.2.2 Generic USB Wi-Fi Framework

The system supports the generic USB WiFi framework. A wide range of common USB WiFi adapters are supported, including those based on the following chipsets (module support is indicated in parentheses):

Table 33. Supported USB Wi-Fi

Vendor	Chipsets / Examples
MediaTek (MTK)	MT7601U, MT76x0U, MT76x2U, MT7663U, MT7921U (Wi-Fi 6), MT7925U (Wi-Fi 6E)
Realtek (RTL)	RTL8187, RTL8192CU, RTL8XXXU family, RTW88 family (RTL8822BU, RTL8822CU, RTL8723DU, RTL8821CU)
Ralink (RT2x00)	RT2500USB, RT73USB, RT2800USB (RT3573/53XX/55XX variants)
Broadcom (BRCM)	BCM43xx / BCM43xxx USB variants
Atheros / Qualcomm	CARL9170, ATH6KL-USB, AR5523
Libertas	Marvell USB (“THINFIRM”)
Atmel	AT76C50X USB
ZyDAS	ZD1211 / ZD1211B

Note: For many chipsets (especially Realtek and Broadcom), operation requires providing the necessary proprietary firmware files to the system.

The following steps describe how to enable support for a USB WiFi adapter that is not supported by default:

Step 1: Download and install the appropriate firmware

Each WiFi chipset requires a specific firmware file that the kernel loads during initialization. Some public firmware files are available from the official Linux firmware repository: linux-firmware

If they are missing there, please download the latest firmware files from the manufacturer's website. Example command:

```
root@rz-cmn:~# mkdir -p /lib/firmware
root@rz-cmn:~# curl -s\
https://git.kernel.org/pub/scm/linux/kernel/git/firmware/linux-
firmware.git/plain/<firmware_file_name> -o /lib/firmware/<firmware_file_name>
```

After that, store the firmware file in the system firmware directory so it can be loaded automatically:

```
root@rz-cmn:~# cp /lib/firmware/<firmware_file_name> /lib/firmware/$(uname -r)/
root@rz-cmn:~# chmod 644 /lib/firmware/<firmware_file_name>\
/lib/firmware/$(uname -r)/<firmware_file_name>
```

Note:

- Only follow these steps if the firmware is missing.
- Ensure the board has internet access before running the commands.
- If the firmware is downloaded for the first time, a reboot may be required for proper initialization.
- For a customized kernel that requires the Wi-Fi driver to be built-in (=y), embedding the firmware using CONFIG_EXTRA_FIRMWARE="<firmware_file_name>" in the kernel configuration is recommended.

Step 2: Verify firmware loading and device recognition

After connecting the USB WiFi adapter, verify that the kernel has recognized it and successfully loaded the firmware:

```
root@rz-cmn:~# dmesg | tail -n 100
```

If an error such as "Direct firmware load failed with error -2" appears, confirm that the firmware file exists in /lib/firmware/ and /lib/firmware/$(uname -r)/.

In this example, an MT7601U USB Wi-Fi adapter is used. The kernel log below shows the device being detected and initialized successfully:

```
[  585.319094] usb 1-1: new high-speed USB device number 3 using ehci-platform

[  585.639376] usb 1-1: reset high-speed USB device number 3 using ehci-platform

[  585.819387] mt7601u 1-1:1.0: ASIC revision: 76010001 MAC revision: 76010500

[  585.831377] mt7601u 1-1:1.0: Firmware Version: 0.1.00 Build: 7640 Build time:
201302052146____

[  586.259966] mt7601u 1-1:1.0: EEPROM ver:0d fae:00

[  586.508715] ieee80211 phy1: Selected rate control algorithm 'minstrel_ht'

[  586.817026] mt7601u 1-1:1.0 wlu1: renamed from wlan0
```

Based on these logs, the Wi-Fi network interface is renamed from wlan0 to wlu1.

Step 3: Connect to a Wi-Fi network:

Once the device is detected and the firmware is loaded, verify that the network interface is up and ready for use.

```
root@rz-cmn:~# ifconfig wlu1 up

SIOCSIFFLAGS: Operation not possible due to RF-kill
```

If the interface is blocked by rfkill, use the following command to unblock it:

```
root@rz-cmn:~# rfkill list
1: phy1: Wireless LAN
        Soft blocked: yes
        Hard blocked: no


# Unblock the interface
root@rz-cmn:~# rfkill unblock all
root@rz-cmn:~# rfkill list
1: phy1: Wireless LAN
        Soft blocked: no
        Hard blocked: no
```

Next, use connmanctl to establish a connection.

```
root@rz-cmn:~# connmanctl
connmanctl> enable wifi
Enabled wifi
connmanctl> agent on
Agent registered
connmanctl> scan wifi
Scan completed for wifi
connmanctl> services
    xDredme10zW          wifi_0025ca329da3_78447265646d6531307a57_managed_psk
                         wifi_0025ca329da3_hidden_managed_psk
    REL-GLOBAL           wifi_0025ca329da3_52454c2d474c4f42414c_managed_ieee8021x
    R-GUEST              wifi_0025ca329da3_522d4755455354_managed_none
    RVC-WLS              wifi_0025ca329da3_5256432d574c53_managed_ieee8021x
connmanctl> connect wifi_0025ca329da3_78447265646d6531307a57_managed_psk
Agent RequestInput wifi_0025ca329da3_78447265646d6531307a57_managed_psk
  Passphrase = [ Type=psk, Requirement=mandatory ]
Passphrase? nFjey48aT9pk
connmanctl> exit
```

To confirm the Wi-Fi is connected, ping the outside world:

```
root@rz-cmn:~# ping www.google.com
PING www.google.com(hkg07s39-in-x04.1e100.net (2404:6800:4005:813::2004)) 56 data bytes
64 bytes from hkg07s39-in-x04.1e100.net (2404:6800:4005:813::2004): icmp_seq=1 ttl=57 time=43.2 ms
64 bytes from hkg07s39-in-x04.1e100.net (2404:6800:4005:813::2004): icmp_seq=2 ttl=57 time=81.1 ms
64 bytes from hkg07s39-in-x04.1e100.net (2404:6800:4005:813::2004): icmp_seq=3 ttl=57 time=124 ms
```

Note: Ethernet interfaces may interfere with routing through Wi-Fi. If issues occur, first check which Ethernet interfaces are active and then disable them:

List all active interfaces; this will show all interfaces that are currently UP.

```
root@rz-cmn:# ip -o link show | grep 'state UP'
```

Identify Ethernet interfaces (look for names starting with end). Disable all active Ethernet interfaces.

```
root@rz-cmn:~# ifconfig <interface-name> down
```

For example, to disable end0 interface, run:

```
root@rz-cmn:~# ifconfig end0 down
```

8.1.2.3 PCIe Wi-Fi Module

This section describes how to enable and use an M.2 Key-E PCIe Wi-Fi module on supported boards. The physical connection depends on the target board:

- A board with a native M.2 Key-E socket can accept a compatible module directly.
- A board without a native M.2 Key-E socket requires a compatible PCIe-to-M.2 Key-E adapter or HAT.

The Wi-Fi module is enumerated through the PCIe subsystem and appears in Linux as a PCIe network controller. After the device is detected and its driver and firmware are loaded, the wireless interface can be configured using standard Linux networking tools.

The general bring-up flow is:

1. Install the M.2 Key E Wi-Fi module into the supported socket.
2. Connect the antenna(s).
3. Boot Linux and verify that the module is detected.
4. Confirm that the correct driver is loaded.
5. Install firmware if required.
6. Bring up the wireless interface and connect to a Wi-Fi network.

(1) Supported devices

The currently validated Wi-Fi solution for this release is the Waveshare AX210 M.2 Key E Wi-Fi module

Figure 36. AX210 Wi-Fi Module

Some supported boards, such as RZ/V2H-RDK, do not provide a native M.2 Key E slot, an M.2 Key E HAT is required to connect the module to the board's PCIe interface. The complete hardware setup in such case consists of:

- M.2 Key E HAT – provides the PCIe interface required to connect an M.2 Key E module to the RZ/V2H-RDK.
- M2 M-Key + M.2 E-Key PCI-E X1 Expansion Card, required to connect an M.2 Key-E module to the RZ/V2H-EVK.
- Waveshare AX210 M.2 Key E Wi-Fi module – the actual Wi-Fi/Bluetooth module installed on the M.2 Key E HAT.

The HAT and the AX210 module are therefore not interchangeable. The HAT provides the physical PCIe connection, while the AX210 provides the wireless functionality.

(2) Hardware setup

Before using the Wi-Fi module, install the module and connect the required antenna cable.

The installation method depends on the target board. Some boards provide an onboard M.2 Key E socket, while others require an adapter board or HAT.

1. Power off the board.
2. Insert the M.2 Key E Wi-Fi module into the supported M.2 Key E socket.
3. Secure the module with the appropriate mounting screw.
4. Connect the required antenna cable(s).
5. If required, connect the M.2 Key E HAT or adapter to the target board.
6. Power on the board.

(A) RZ/V2H-RDK

The RZ/V2H RDK does not provide an onboard M.2 Key E socket. Use the supported PCIe-to-M.2 Key E HAT to connect the Wi-Fi module.

1. Connect the PCIe-to-M.2 Key E HAT to the 40-pin header on the RZ/V2H RDK: connect the USB cable and external power supply as shown in (1), then connect the PCIe cable as shown in (2) in the figure below.

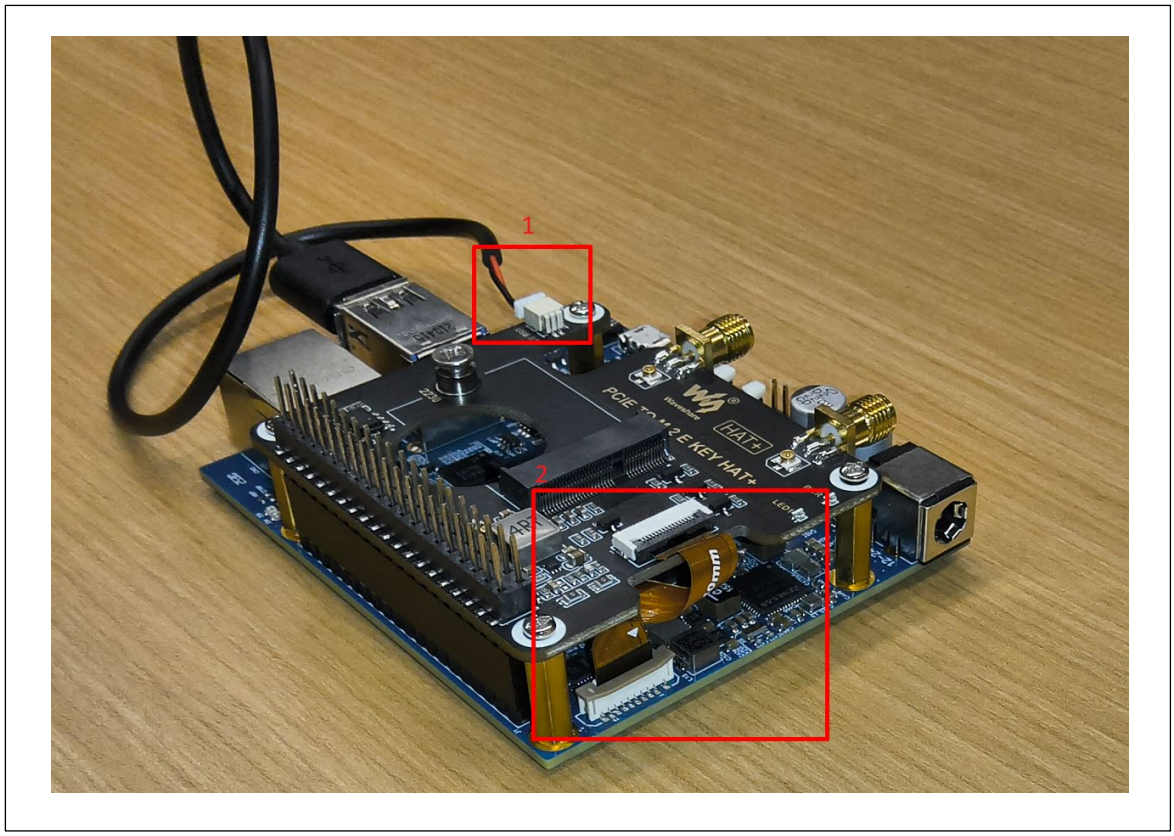


Figure 37. Connect the HAT to RZ/V2H-RDK

2. Insert the Wi-Fi module into the M.2 Key E socket on the HAT.
3. Secure the Wi-Fi module using the appropriate mounting screw.
4. Connect the required antenna cable(s) to the Wi-Fi module.

After installation, the hardware setup should appear as shown in the following figure.

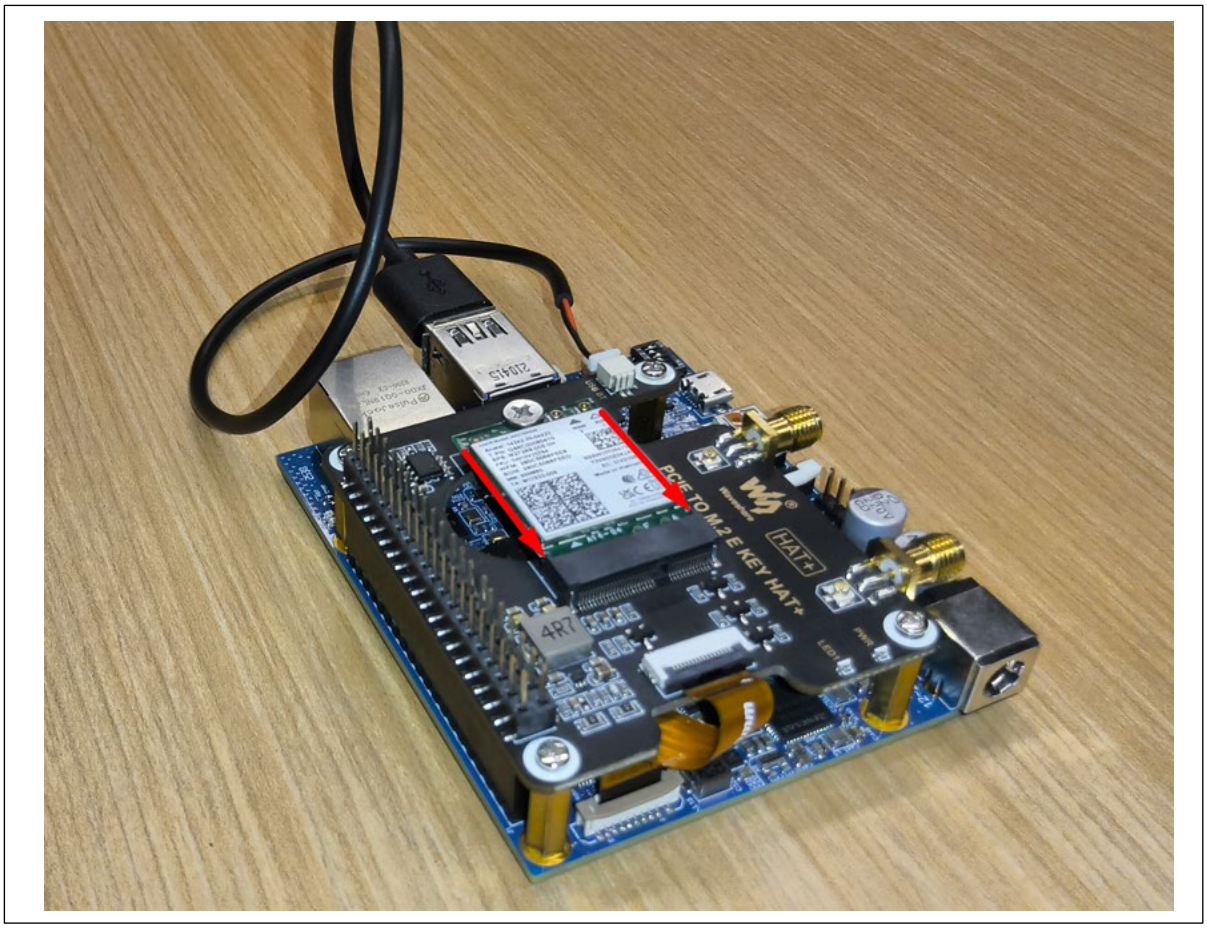


Figure 38. RZ/V2H-RDK after installing the AX210

(B) RZ/V2H-EVK

The RZ/V2H-EVK also needs the converter, the below device is used for this board.

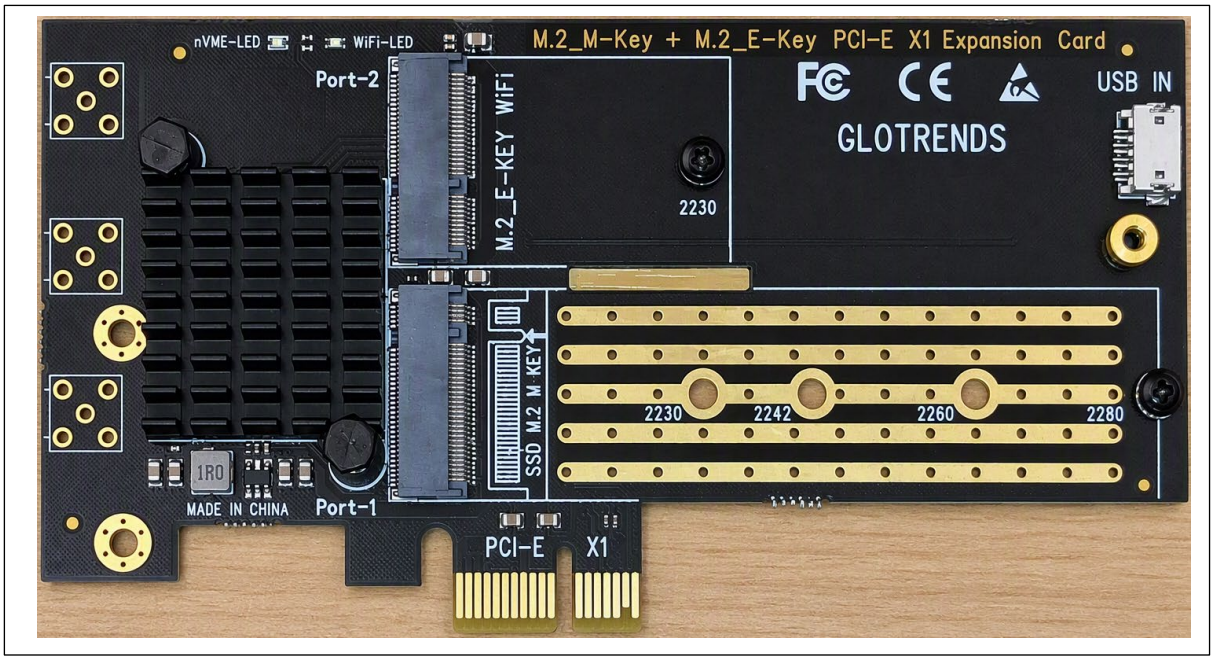


Figure 39. M.2_M-Key + M2_E-Key PCI-E X1 Expansion Card use for RZ/V2H-EVK PCIe

Connect the Wi-Fi module to the Key-E expansion card slot and secure it using the appropriate mounting screw.

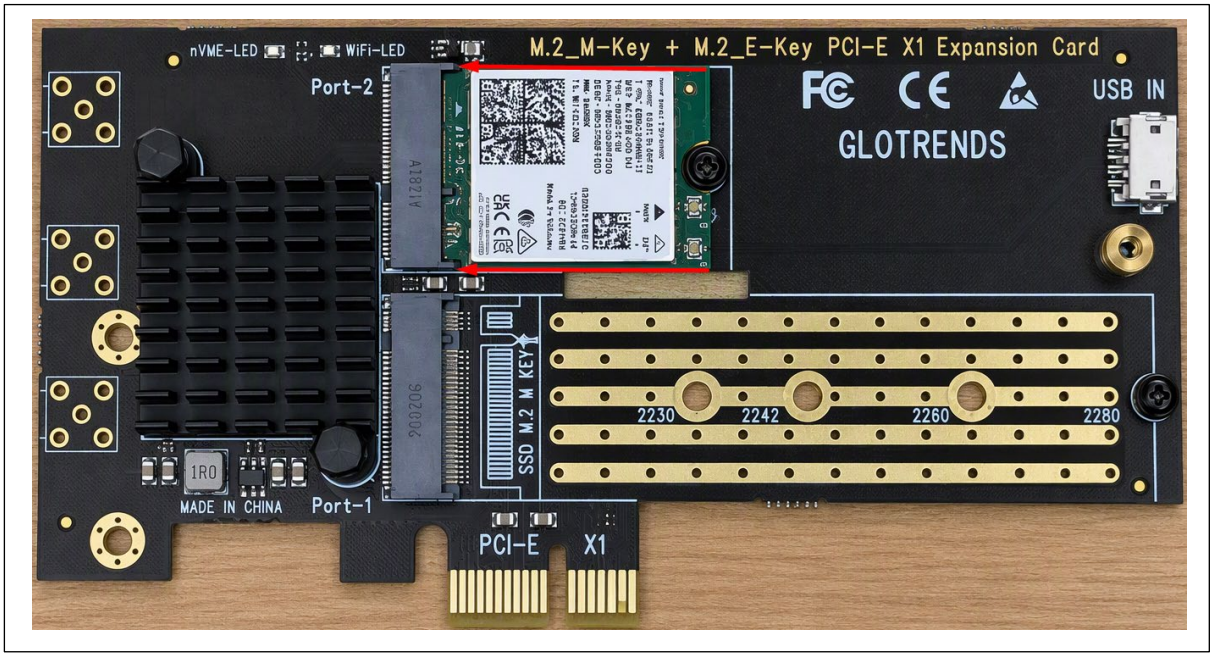


Figure 40. Connect the Wi-Fi Module Key-E to the Expansion Card

After the Wi-Fi module is secured, connect the expansion card to the PCIe interface on the board.

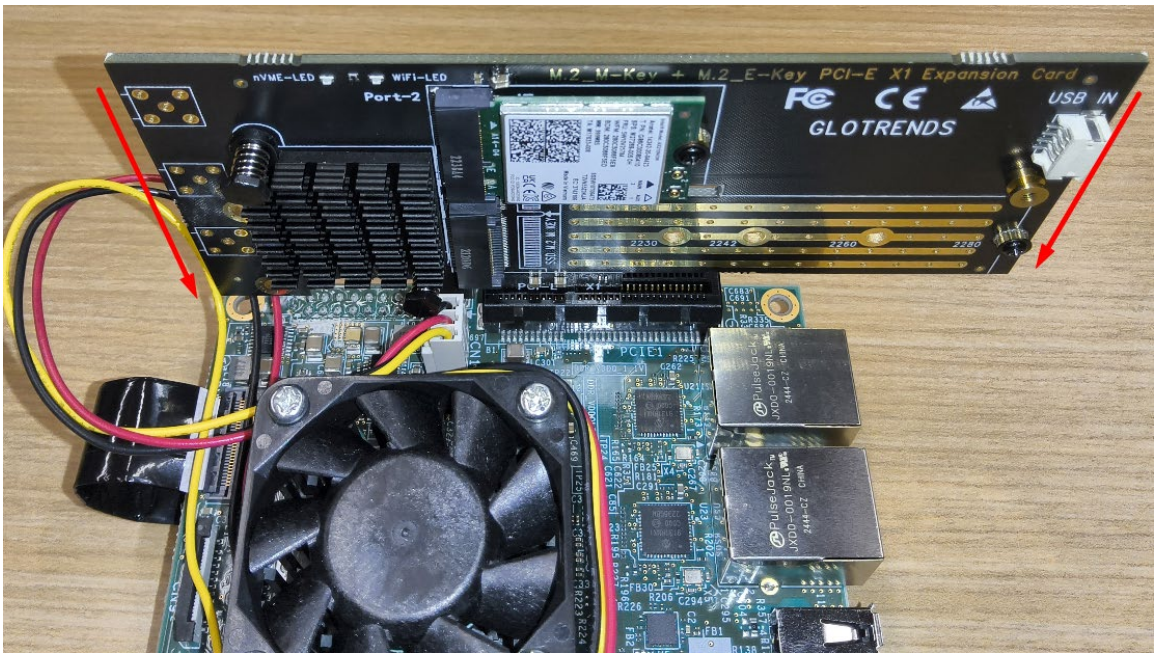


Figure 41. Connect the Expansion Card to PCIe Interface on the board

(C) IMDT V2H-SBC

The IMDT V2H-SBC features an onboard M.2 Key-E slot. Connect the Wi-Fi module directly to this interface.

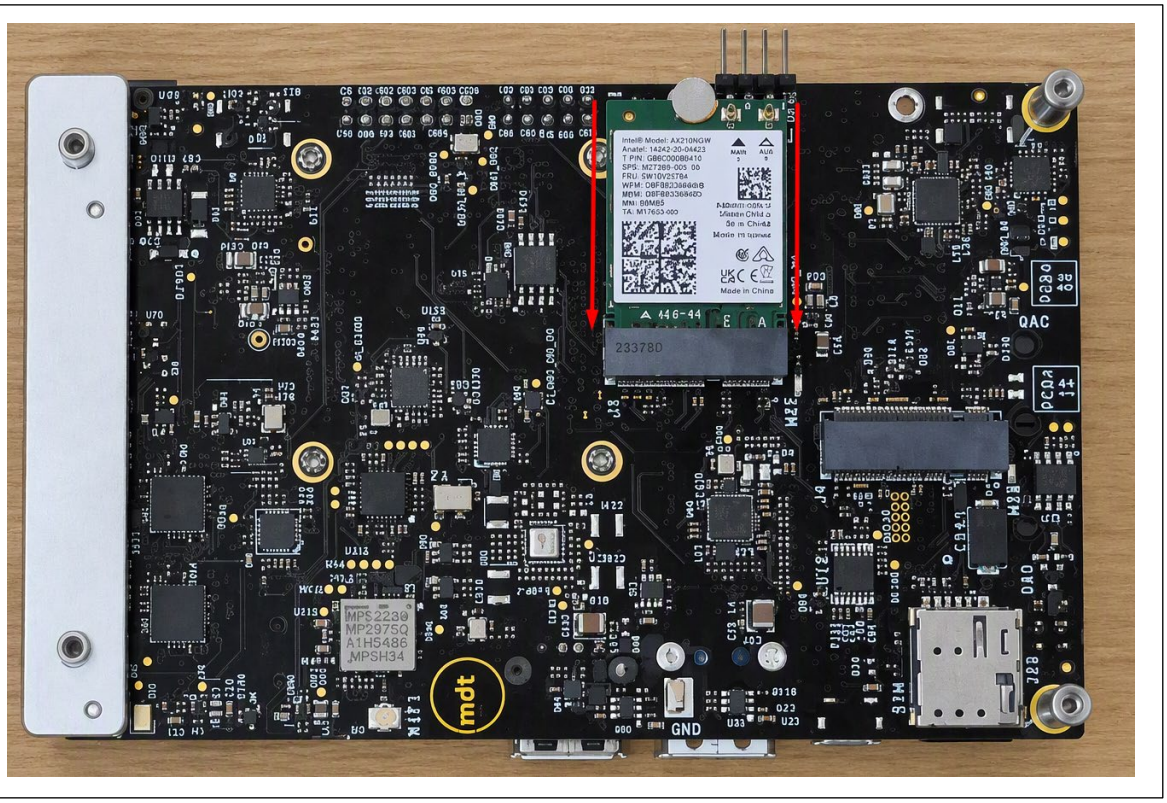


Figure 42. Connect the Wi-Fi Module to the M.2-KeyE on IMDT-V2H-SBC board

(3) Verify PCIe detection

After Linux boots, verify that the module is detected by the PCIe subsystem.

Run:

```
root@rz-cmn:~# lspci
```

Example output:

```
root@rz-cmn:~# lspci

00:00.0 PCI bridge: Renesas Technology Corp. Device 003b

01:00.0 Network controller: Intel Corporation Wi-Fi 6E(802.11ax) AX210/AX1675*
2x2 [Typhoon Peak] (rev 1a)
```

The output confirms that:

- 00:00.0 PCI bridge: the RZ/V2H PCIe root complex
- 01:00.0 Network controller: the Intel AX210 endpoint

If the AX210 does not appear in lspci:

- confirm that the module is inserted correctly,
- confirm that the board supports the M.2 Key E socket being used,
- inspect PCIe logs:

```
root@rz-cmn:~# dmesg | grep -i pcie
```

A successfully enumerated module should appear as a PCIe Network controller before the Wi-Fi driver is considered.

```
root@rz-cmn:~# dmesg | grep -i iwlwifi

iwlwifi 0000:01:00.0: enabling device (0000 -> 0002)

iwlwifi 0000:01:00.0: Detected crf-id 0x400410, cnv-id 0x400410 wfpm id
0x80000000

iwlwifi 0000:01:00.0: PCI dev 2725/0024, rev=0x420, rfid=0x10d000

iwlwifi 0000:01:00.0: Detected Intel(R) Wi-Fi 6E AX210 160MHz

iwlwifi 0000:01:00.0: loaded firmware version 89.202a2f7b.0 ty-a0-gf-a0-89.ucode
op_mode iwlmvm

iwlwifi 0000:01:00.0: Detected RF GF, rfid=0x10d000

iwlwifi 0000:01:00.0: base HW address: 28:0c:50:69:67:b6

iwlwifi 0000:01:00.0 wlp1s0: renamed from wlan0
```

The firmware version and MAC address vary by module and release. The important messages are:

```
Detected Intel(R) Wi-Fi 6E AX210 160MHz

loaded firmware version ...

wlp1s0: renamed from wlan0
```

In the validated setup, Linux renamed the wireless interface from wlan0 to wlp1s0. Always use the interface name reported by the target system.

(4) Check driver loading

After confirming that the AX210 is visible on the PCIe bus, verify that the Linux kernel has bound the device to the expected Wi-Fi driver.

For the currently validated Waveshare AX210 module, the Linux Wi-Fi driver is:

```
iwlwifi
```

For a different M.2 Key E Wi-Fi module, the required driver depends on the chipset used by that module. The driver can be identified from the PCIe device information and kernel messages.

To display the driver associated with the AX210, run:

```
root@rz-cmn:~# lspci -k

01:00.0 Network controller: Intel Corporation Wi-Fi 6E (802.11ax) AX210/AX1675*
2x2 [Typhoon Peak] (rev 1a)

Kernel driver in use: iwlwifi

Kernel modules: iwlwifi
```

Kernel driver in use: iwlwifi confirms that the AX210 is detected and bound to the correct Linux driver.

The PCI revision and descriptive text may differ slightly. The important results are that the AX210 appears as a network controller and that iwlwifi is shown as the driver in use.

To filter for network-related devices:

```
root@rz-cmn:~# lspci -nn | grep -i -E "network|wireless"
```

A successfully detected Wi-Fi module typically appears as a **Network controller** entry.

This command shows:

- The detected PCIe device,
- The kernel driver currently in use,
- Other kernel modules that may support the device.

Check kernel messages related to driver probing:

```
root@rz-cmn:~# dmesg | grep -i -E "iwlwifi|wifi|wlan|firmware|pci"
```

List loaded modules:

```
root@rz-cmn:~# lsmod
```

If required, try loading the expected driver manually:

```
root@rz-cmn:~# modprobe <driver_name>
```

Examples of common driver families:

- Intel: iwlwifi
- Qualcomm/Atheros: ath10k_pci
- MediaTek: mt76
- Realtek: rtw88_pci

If the correct driver is not known, first identify the module chipset or PCI vendor/device ID using **lspci -nn**, then match it to the corresponding Linux driver.

If the required kernel module is not present in the running kernel or root filesystem, the Wi-Fi module cannot be used in the current software configuration. In such cases, support for the required driver must be added to the kernel configuration, and the image must be regenerated. For instructions on modifying the source and rebuilding the image with the Yocto eSDK, refer to Section **Using devtool in the Yocto eSDK**

(5) Install firmware if required

Many M.2 Key E Wi-Fi modules require firmware files in `/lib/firmware/`.

Check for firmware-related messages:

```
root@rz-cmn:~# dmesg | grep -i firmware
```

If firmware is missing, the kernel log may show an error similar to:

```
Direct firmware load failed with error -2
```

In such cases, obtain the firmware file corresponding to the Wi-Fi module or chipset. Public firmware files may be available from the Linux firmware repository

If the required firmware is not available there, refer to the module or chipset vendor’s distribution source or support website.

Then place the firmware file in:

```
/lib/firmware/
```

(6) Verify wireless interface creation

Once the module, driver, and firmware are functioning correctly, verify that a wireless network interface has been created.

List interfaces:

```
root@rz-cmn:~# ip link
```

Typical interface names may include:

- wlan0
- wlp1s0
- wlp2s0

The interface name depends on the Linux naming policy and the detected hardware.

To inspect interface wlan0 capabilities, run the following command:

```
root@rz-cmn:~# ip link show wlan0
```

(7) Unblock Wi-Fi if blocked

If the wireless interface is blocked by rfkill, unblock it before use.

Check rfkill status:

```
root@rz-cmn:~# rfkill list
```

Example,

```
root@rz-cmn:~# rfkill list
0: phy0: Wireless LAN
        Soft blocked: no
        Hard blocked: no
1: hci0: Bluetooth
        Soft blocked: yes
        Hard blocked: no
```

The Soft blocked status indicates whether the wireless device is disabled by software, while Hard blocked indicates whether the device is disabled by hardware or a physical control.

For Wi-Fi, both Soft blocked and Hard blocked should be ‘no’ before use. In the example above, the Wi-Fi interface (phy0) is not blocked, while Bluetooth (hci0) is soft-blocked.

If Wi-Fi is soft-block, unblock it with:

```
root@rz-cmn:~# rfkill unblock wifi
```

If Bluetooth is also used through the same module, it may be unblocked separately:

```
root@rz-cmn:~# rfkill unblock bluetooth
```

After the M.2 Key E Wi-Fi module has been detected and the wireless interface has been created, the Wi-Fi setup and connection procedure is the same as described in Section **Generic USB Wi-Fi Framework**

8.1.2.4 Audio (Codec and Interfaces)

Audio capability is board-dependent. Some RZ boards provide an onboard audio codec with an analog connector (for example, a 3.5 mm jack). Other boards expose only a digital audio interface (I^2S or PCM/TDM) on header pins and require an external codec/breakout to obtain analog headphone and microphone connections.

Typical interfaces:

- Audio data interface: SSI (I^2S or PCM/TDM, depending on the board)

- Control interface: I²C (codec configuration, depending on the board)

(1) Hardware connection

(A) BOARD CAPABILITY SUMMARY

Table 34. Board compatibility summary

Board	Analog audio on board	Digital audio on the header	HDMI Audio	Notes
RZ/G2L-SBC	3.5mm jack	Not provided	Not supported	If the jack is TRRS, a TRRS headset or TRRS-to-dual-TRS Y-adapter may be required for microphone use.
RZ/G2L-EVK RZ/V2L-EVK	3.5mm jack	Not provided	Supported	If the jack is TRRS, a TRRS headset or TRRS-to-dual-TRS Y-adapter may be required for microphone use.
RS-G2L100	Not supported	Not supported	Supported	-
RZ/V2H-EVK	3.5mm jack	Not provided	Supported	If the jack is TRRS, a TRRS headset or TRRS-to-dual-TRS Y-adapter may be required for microphone use.
RZ/V2H-RDK	Not provided	I²S/PCM (SSI0 & SSI9) + I²C (codec control)	Supported (default). Disabled when the codec overlay is enabled	External codec/breakout required for headphone/mic.
IMDT-V2H-SBC	3.5mm jack	Not provided	Not supported	If the jack is TRRS, a TRRS headset or TRRS-to-dual-TRS Y-adapter may be required for microphone use.

(B) ANALOG CONNECTION

Boards with an analog jack typically support headphone/line-out and may support microphone input if the jack is wired for headset operation.

Accessory note (headset microphone support):

- TRS (3-conductor): stereo headphone/line-out only.
- TRRS (4-conductor): headset (headphone + microphone).

For separate headphone and microphone plugs, a TRRS-to-dual-TRS Y-adapter (headset splitter) may be required for a better experience.

(C) DIGITAL CONNECTION

Some boards in this release (for example, RZ/V2H-RDK) do not provide an on-board 3.5 mm audio jack. Audio is exposed on the 40-pin expansion header as a digital serial audio interface. These header signals carry the audio bitstream and codec control; they are not directly compatible with analog headphones or microphones. An external audio codec/breakout board is required to provide analog line-out/headphone output and, if supported by the hardware routing, microphone input.

Validation was performed using the DA7219 codec devkit connected to the expansion header. In this configuration, the device tree configures the digital audio link as **I²S over SSI0**, with codec control via I²C and an optional interrupt for jack detect.

Other external codec/breakout hardware can be used following the same connection approach (digital audio + I²C control), provided it is electrically compatible, and the device-tree/overlay configuration matches the external hardware (codec type, digital audio format, clocking, I²C address, and any required GPIO/interrupt signals).

The diagram below shows an example wiring between the RZ/V2H-RDK 40-pin expansion header and the DA7219 codec devkit.

Signal description (validated example):

- I²C (control): SCL, SDA - Used to configure codec registers.
- Digital audio (I²S/SSI0): BCLK (SSI0_SCK), LRCLK/WS (SSI0_WS), SDATA (SSI0_SDATA – Audio path), SSI9_SDATA – Record path; carries the audio data stream between the SoC and the codec
- Interrupt (optional): INT - Used for codec events such as jack detect
- Power/GND: connect the required supply and at least one ground reference

Note: VDD_IO and VDD_MIC should be supplied with 3.3 V to match the external I/O and microphone bias requirements, while VDD must remain at 1.8 V for the internal core supply.

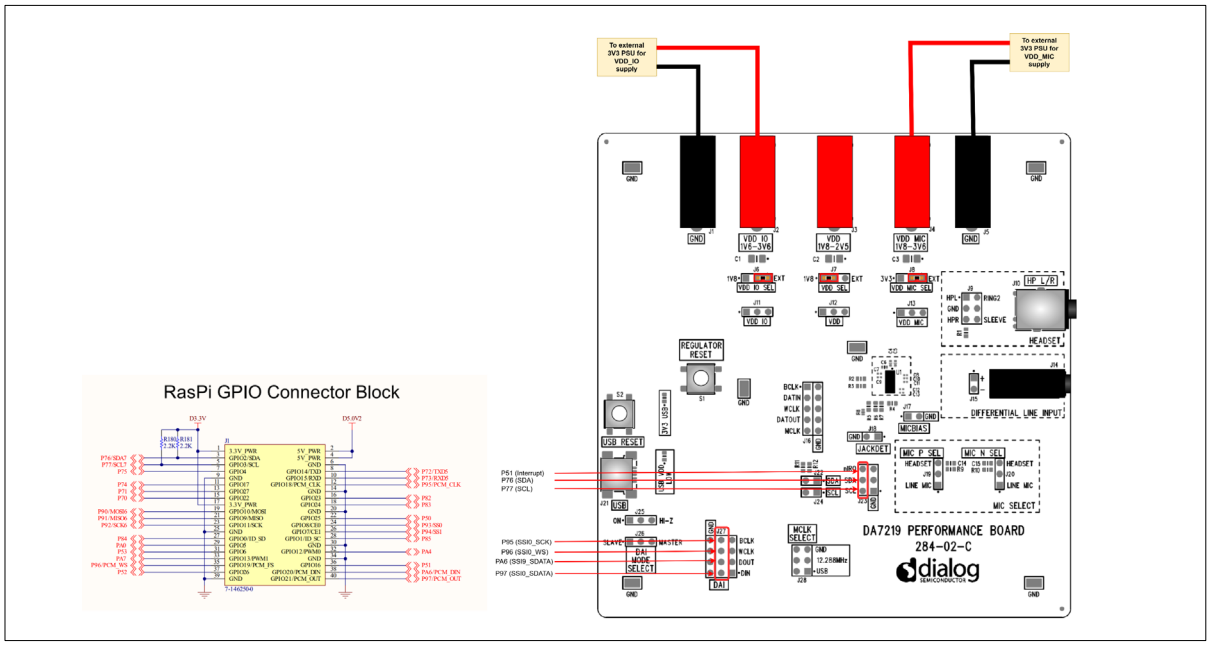


Figure 43. Example connection diagram: RZ/V2H-RDK expansion header to DA7219 codec devkit

(D) HDMI AUDIO

HDMI audio is available only on boards where the display interface supports audio and the connected monitor/TV advertises audio capability (EDID). HDMI audio is typically output-only. Refer to the table Table 34. Board compatibility summary for details on supported boards.

(2) Software usage

On RZ/V2H-RDK, analog audio is provided through an external audio codec/breakout connected to the expansion header. Before running playback or recording commands, the audio codec overlay must be enabled so that the codec is instantiated by the kernel.

Step 1. Enable the audio overlay in U-Boot (if applicable)

Set the U-Boot environment variable ***enable_overlay_audio_codec***. Refer to U-Boot Environment for details. After updating the U-Boot environment, reboot the board to apply the overlay

Then, follow the steps below:

Step 1: Discover available audio interfaces

Before playback or recording, list all ALSA devices and their properties:

```
root@rz-cmn:~# aplay -l      # List available playback devices
root@rz-cmn:~# arecord -l    # List available recording devices
root@rz-cmn:~# aplay -L      # List all supported PCM devices and formats
```

This step ensures that the onboard codec is recognized and identifies the correct device index (e.g., hw:0,0).

Step 2: Prepare Audio files

Prepare the required audio files and copy them into the target filesystem (e.g., /home/root/audio/).

- The aplay tool supports only the WAV (.wav) format.
- For additional formats such as MP3 and AAC, use the pre-installed GStreamer framework, which provides compatibility with multiple codecs.

Step 3: Playback

Examples:

- WAV playback (ALSA/PCM):

```
root@rz-cmn:~# aplay -D hw:0,0 /home/root/audios/test.wav
```

    - -D specifies the ALSA device to use.
    - hw:0,0 means card 0, device 0, which corresponds to the onboard audio codec (as shown in the aplay -l output).
    - If the board reports a different index, replace hw:0,0 with the correct value (e.g., hw:1,0).
- WAV, MP3, AAC playback (Gstreamer):

```
root@rz-cmn:~# gst-play-1.0 /home/root/audios/test.wav
root@rz-cmn:~# gst-play-1.0 /home/root/audios/test.mp3
root@rz-cmn:~# gst-play-1.0 /home/root/audios/test.aac
```

Step 4: Recording

To capture audio through the onboard codec:

```
root@rz-cmn:~# arecord -f S16_LE -r 48000 audio_capture.wav
```

In the above command:

- -f S16_LE: audio format (signed 16-bit little endian)
- -r 48000: sample rate of the audio file (48KHz)

Press Ctrl + C to stop recording.

Verify the recording

```
root@rz-cmn:~# aplay audio_capture.wav
```

To adjust the level of the audio record/playback, use the following command to open the ALSA mixer GUI on the debug console:

```
root@rz-cmn:~# alsamixer
```

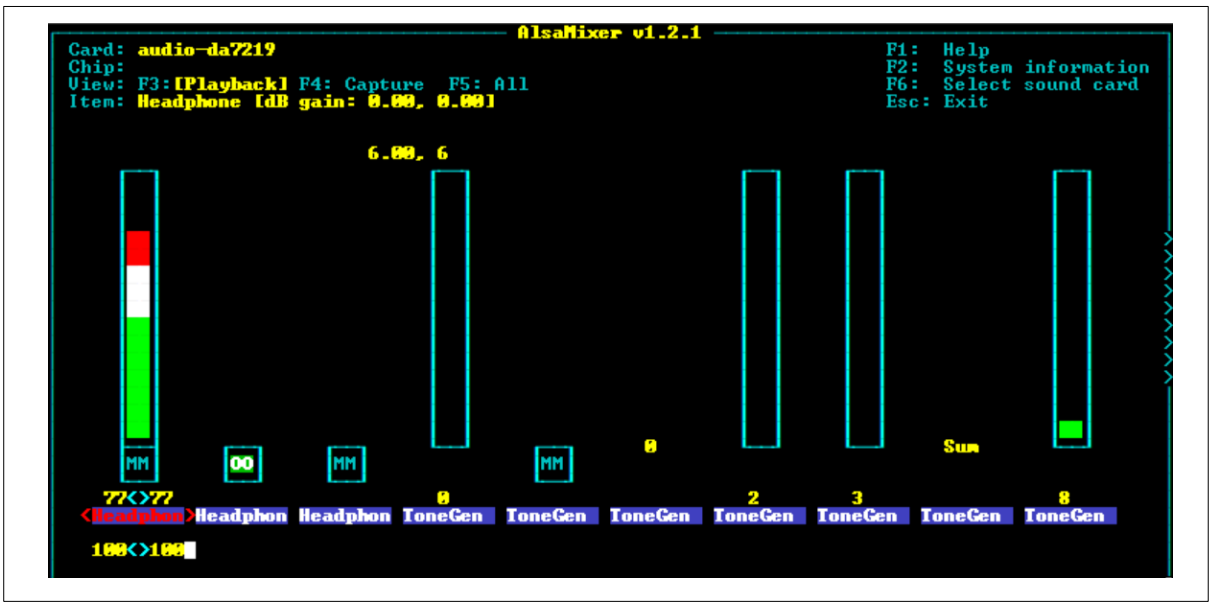


Figure 44. Example ALSA Mixer GUI displayed on the RZ/G2L-SBC debug console

8.1.2.5 MIPI CSI-2 Cameras

This section describes how to enable and use MIPI CSI-2 cameras across the supported boards. All boards share a common camera initialization script, but each board requires its own compatible camera module and driver configuration.

(1) Board-Specific Setup

Before initializing the camera, complete the setup for the target board.

This includes connecting the correct camera module, enabling the appropriate device tree overlay or built-in driver, and rebooting the board.

(A) RZ/G2L-SBC

The Arducam 5 MP (OV5640) camera module is easily installed into the RZ/G2L-SBC.

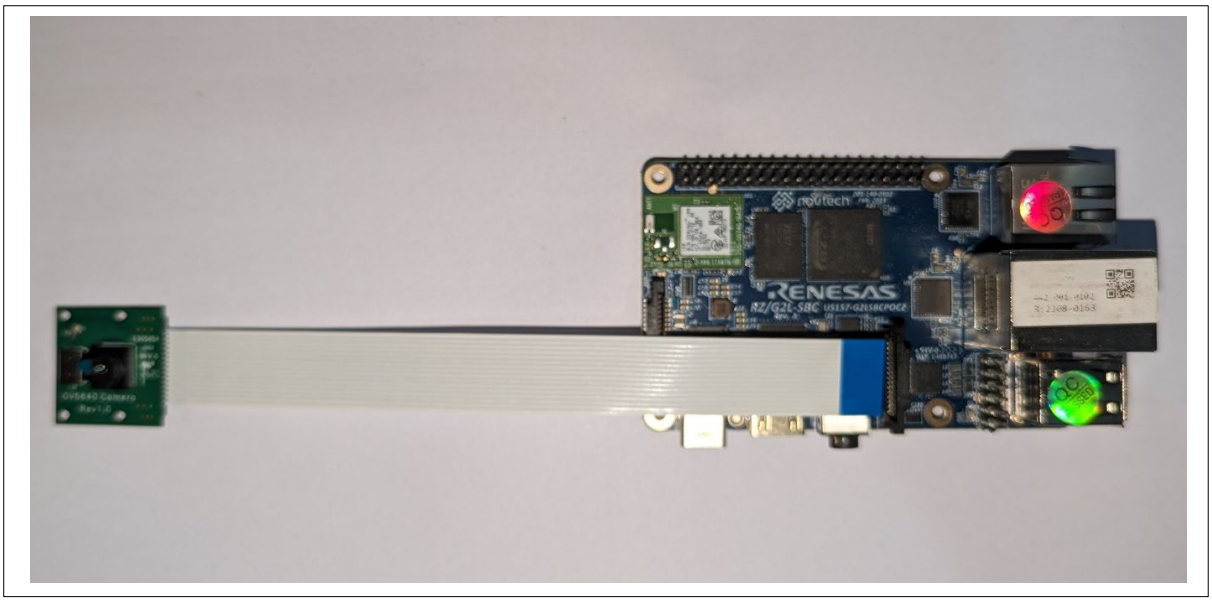


Figure 45. Orientation of the camera module. Blue stripes upward.

The black notch must be pulled up to unlock it.

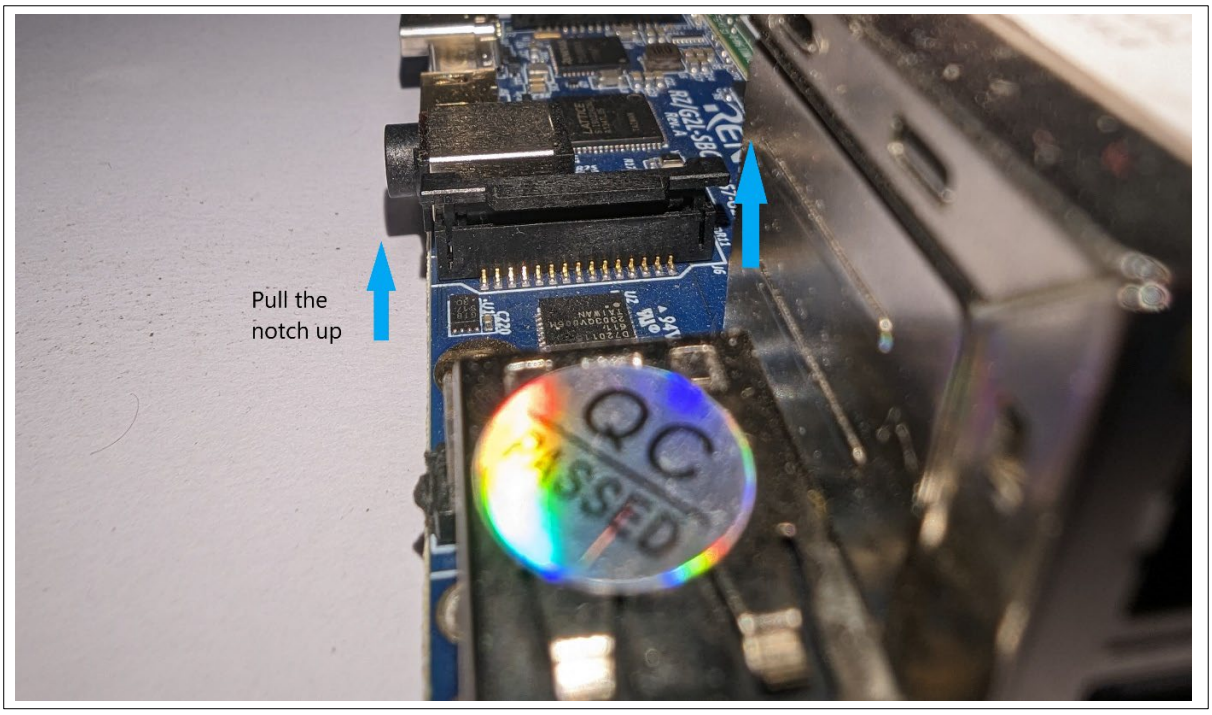


Figure 46. Pull the notch up to unlock it.

Insert the flat cable in the correct orientation, as depicted in the pictures.

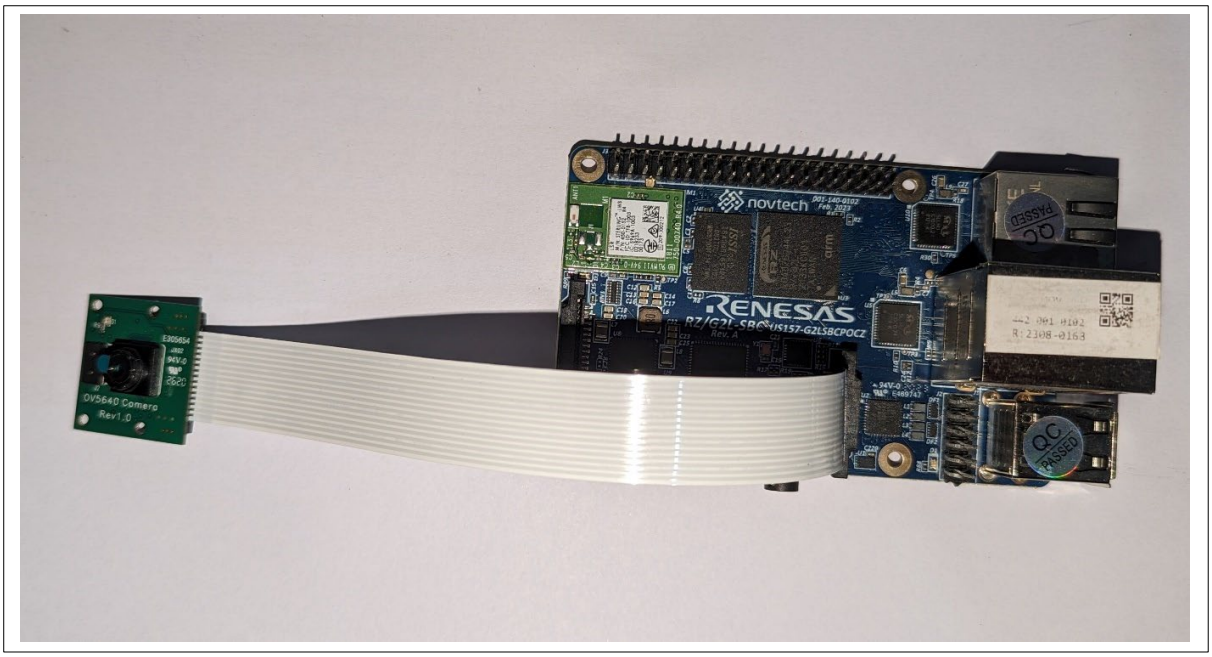


Figure 47. The CSI module is inserted.

Push down on the notch to lock it with the flat cable inserted.

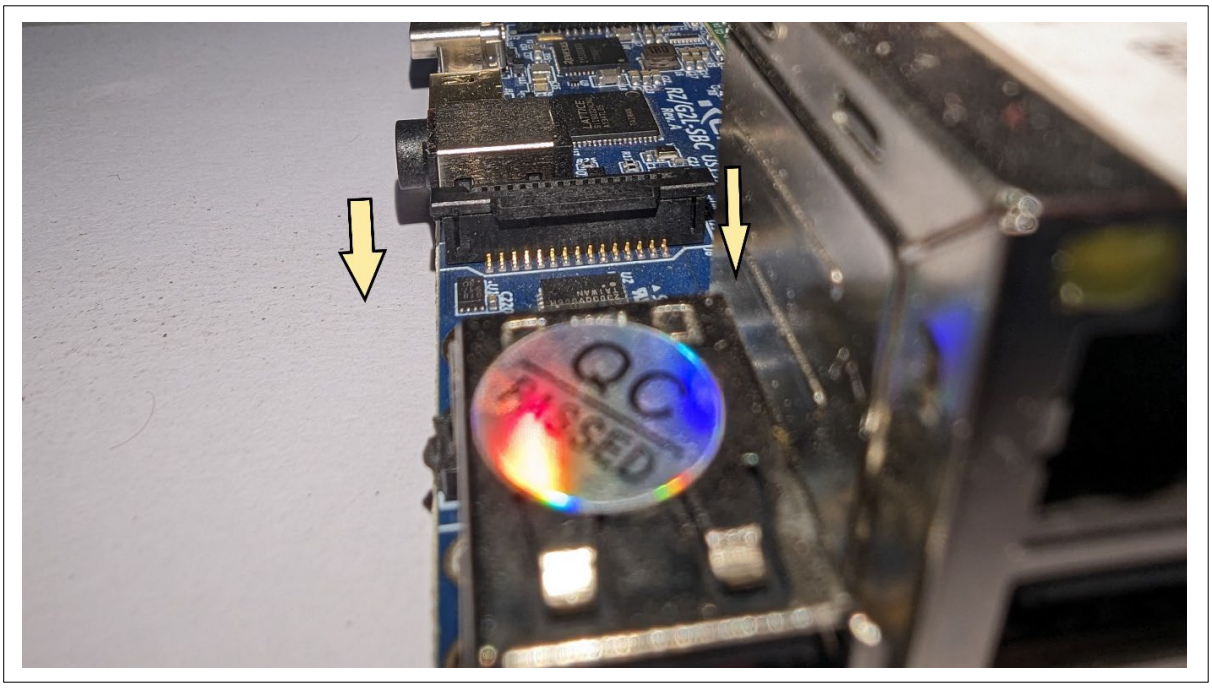


Figure 48. Push down the notch to lock it when you have inserted the flat cable

After completing the hardware setup, enable the camera by editing uEnv.txt in partition 1 and enabling the following line:

```
#enable_overlay_csi_ov5640=1
```

To

```
enable_overlay_csi_ov5640=1
```

Reboot the board.

(B) RZ/G2L-EVK AND RZ/V2L-EVK

These boards use the Coral camera that contains the OmniVision OV5645 sensor.

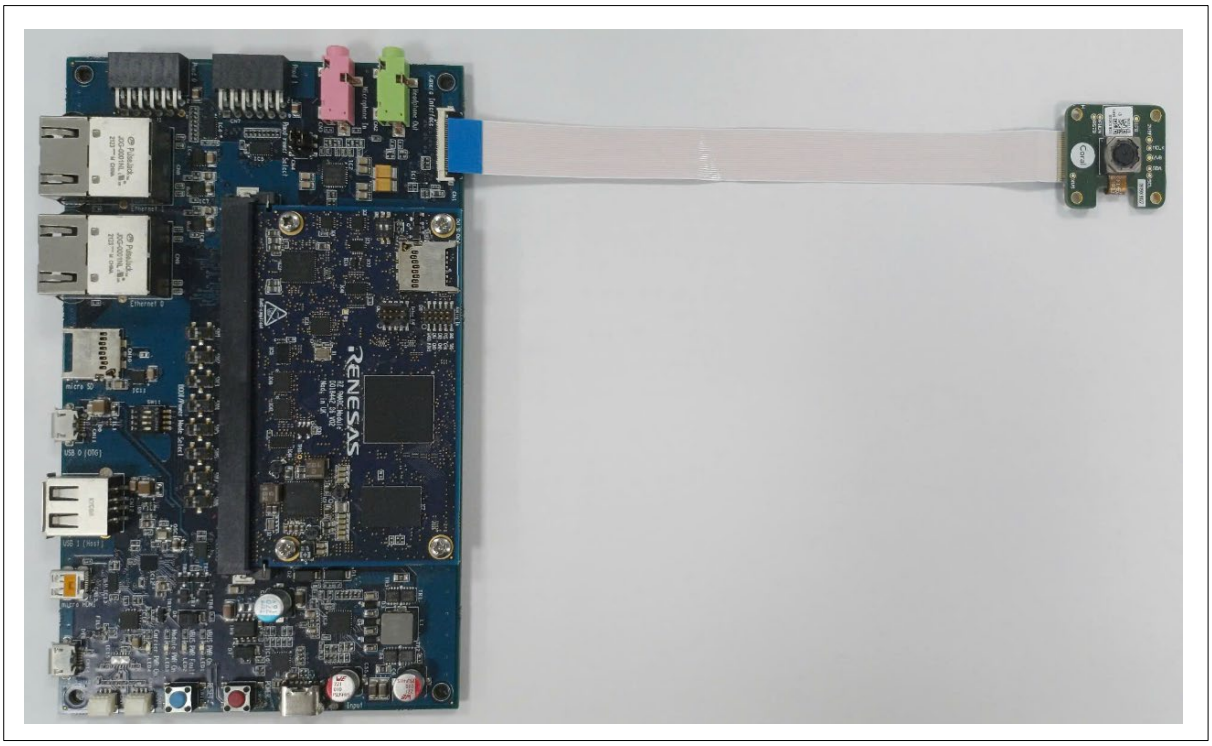


Figure 49. Orientation of the camera module. Blue striped upward

The black notch must be pulled up to unlock it.

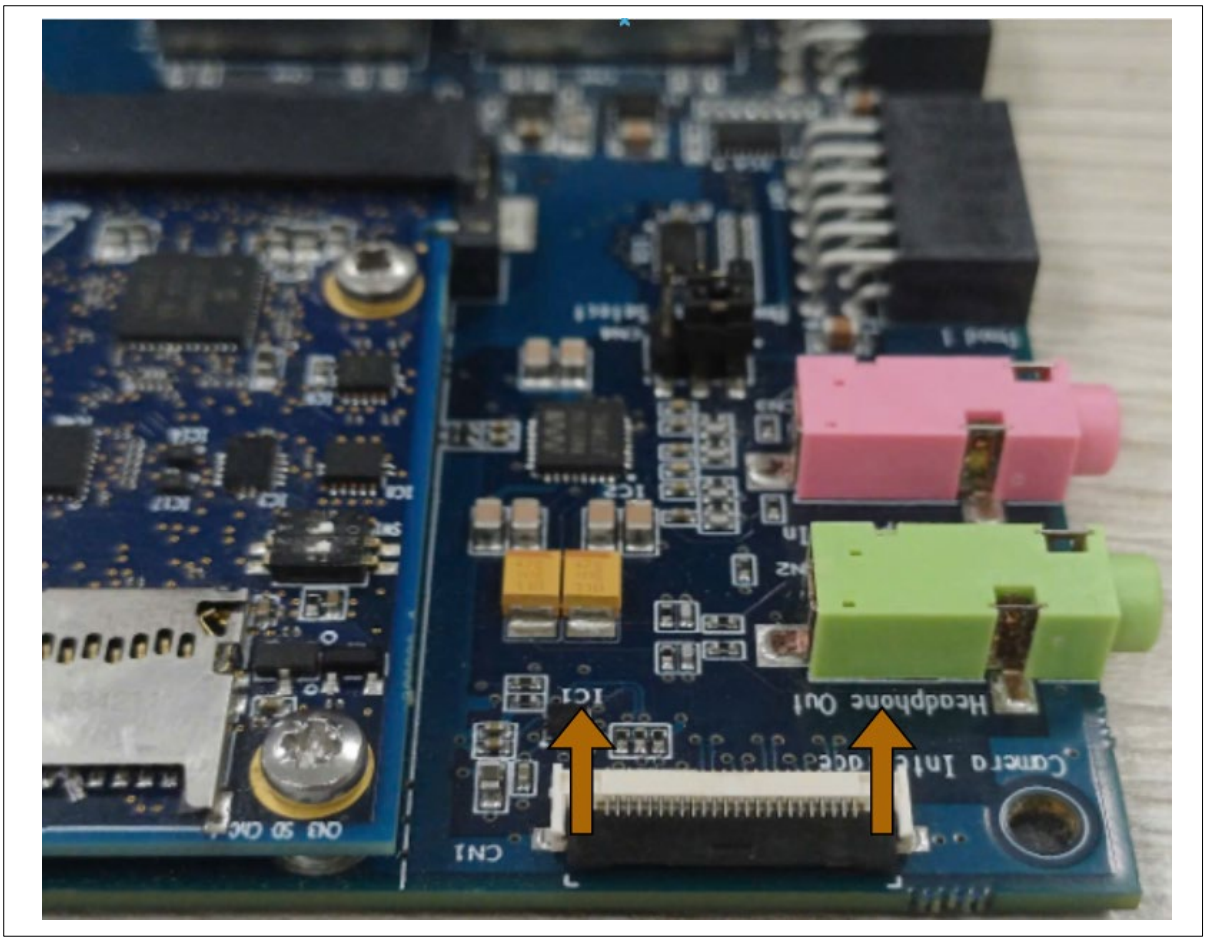


Figure 50. Pull up the notch to unlock

Insert the flat cable in the correct orientation, as depicted in the pictures.

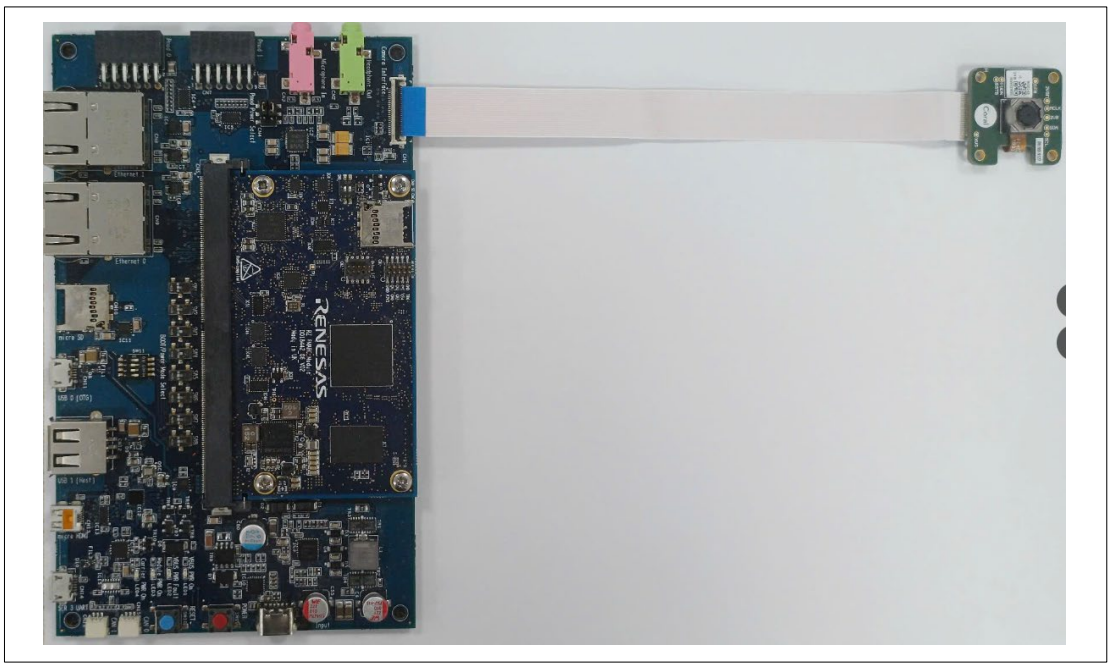


Figure 51. The CSI camera is inserted

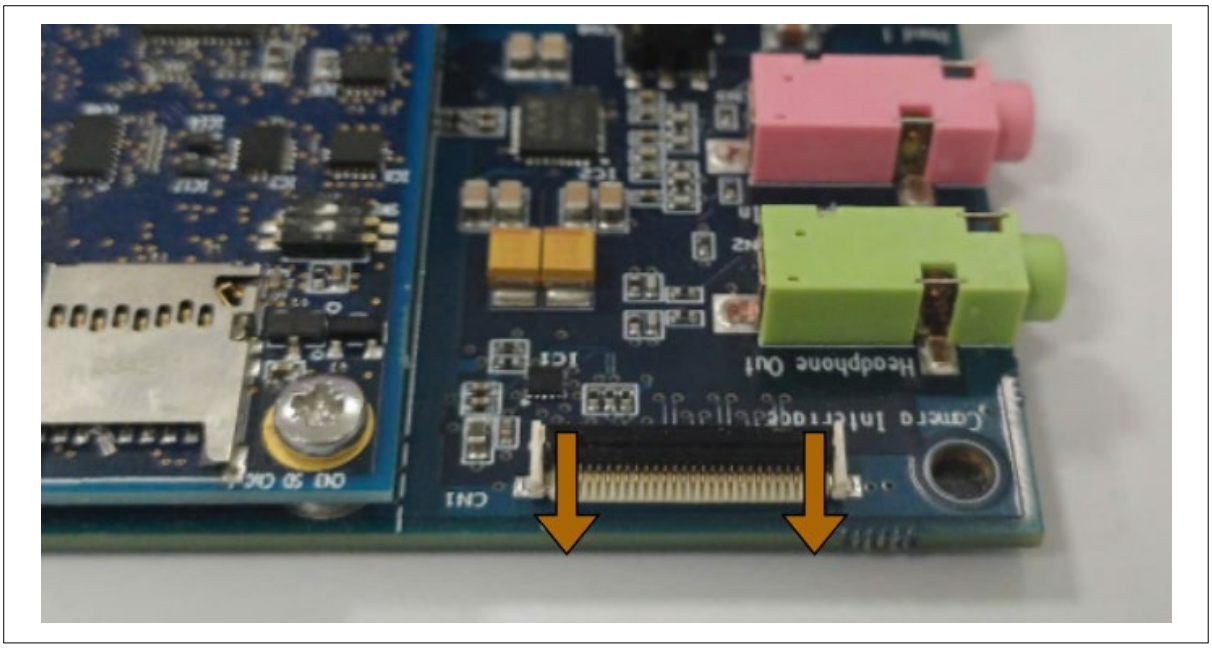


Figure 52. Push down the notch to lock it when you have inserted the flat cable

After connecting the camera hardware, enable the camera by editing uEnv.txt in partition 1 and uncommenting the line that enables the OV5645 overlay:

```
#enable_overlay_csi_ov5645=1
```

To

```
enable_overlay_csi_ov5645=1
```

Then, reboot the board.

(C) RZ/V2H-EVK

The RZ/V2H-EVK supports the same Coral camera (OV5645) as the RZ/G2L-EVK and RZ/V2L-EVK, but requires a 22-pin → 24-pin CSI conversion cable to connect the camera.

This conversion cable is included with the RZ/V2H-EVK package.

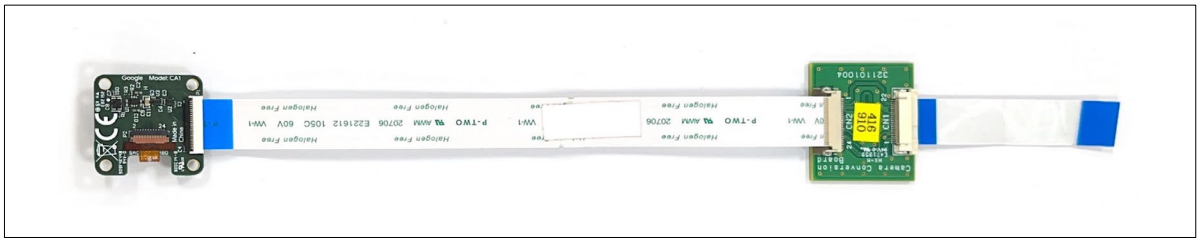


Figure 53. 22-pin to 24-pin CSI conversion

The board has four CSI camera ports, and the image below shows an example of one port. The other ports are similar.

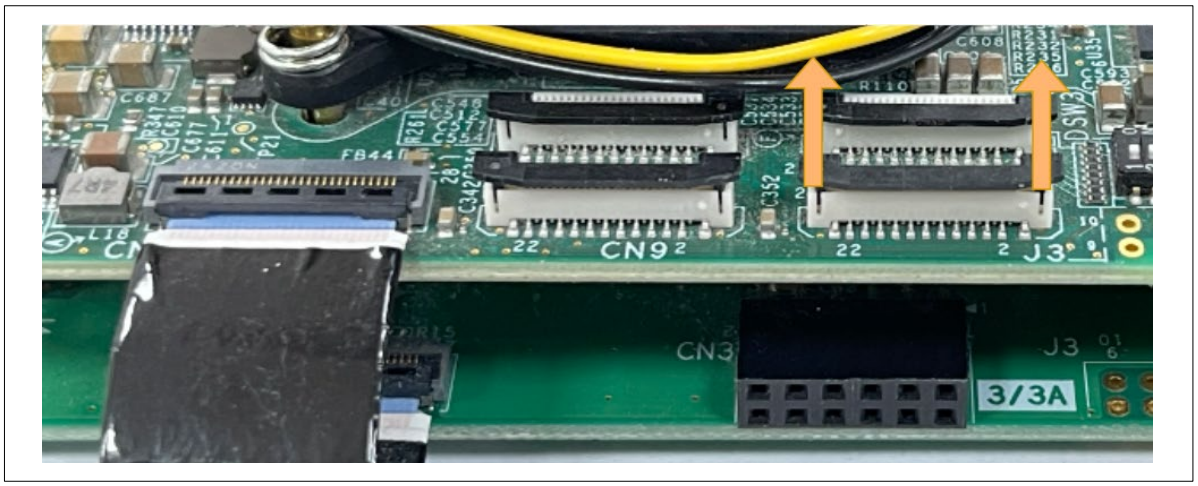


Figure 54. Pull up the notch to unlock

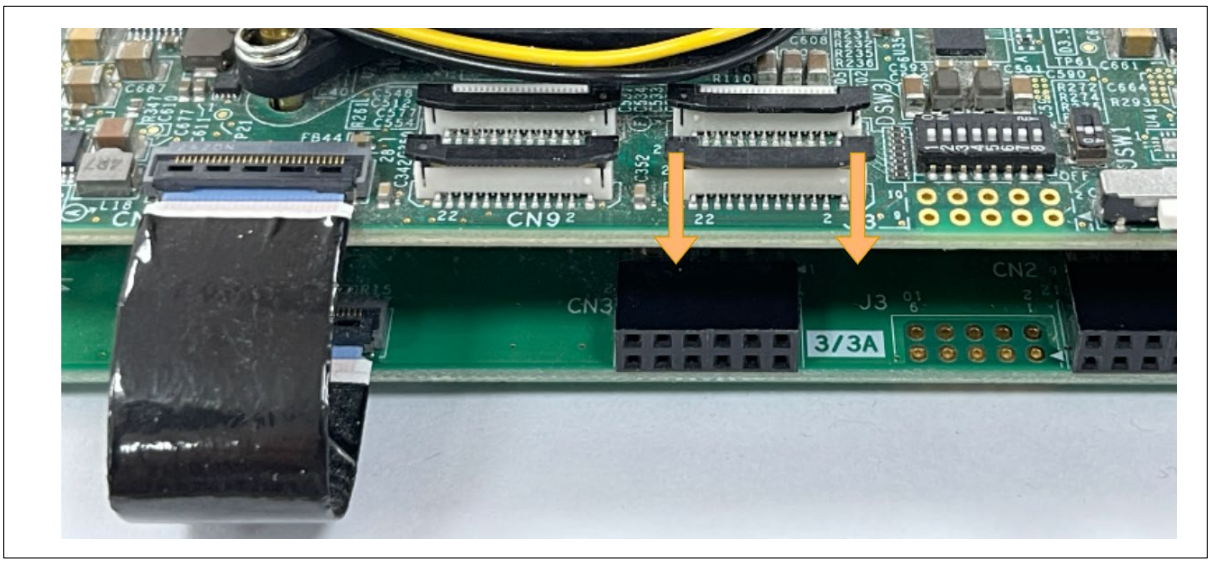


Figure 55. Push down the notch to lock it when you have inserted the flat cable

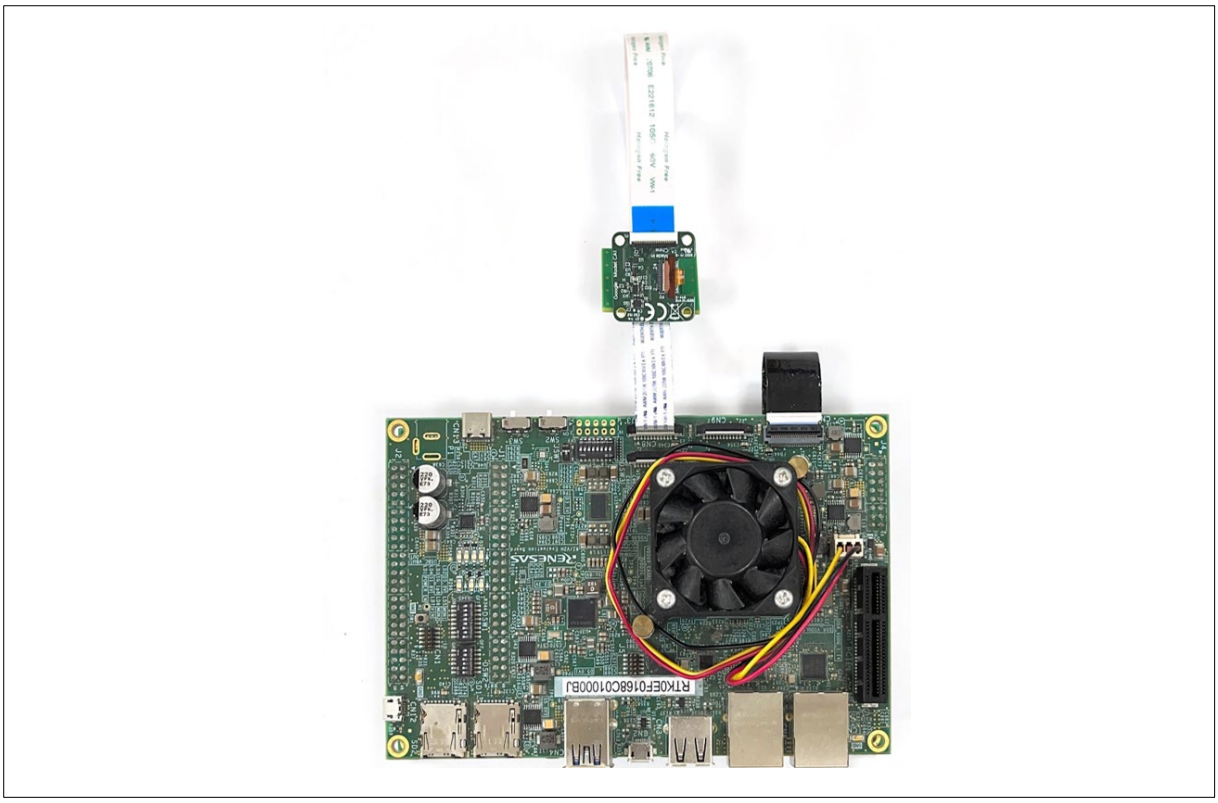


Figure 56. The CSI camera is inserted

(D) RZ/V2H-RDK

The RZ/V2H-RDK is similar to RZ/V2H-EVK; it supports the same Coral camera (OV5645) and requires a 22-pin → 24-pin CSI conversion cable to connect the camera.

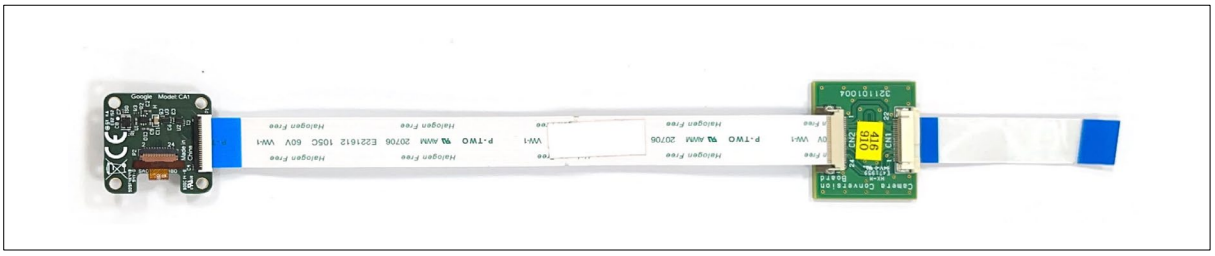


Figure 57. CSI conversion cable (22-pin to 24-pin)

The board has two CSI camera connectors, CN5 and CN6. Insert the camera into either connector.

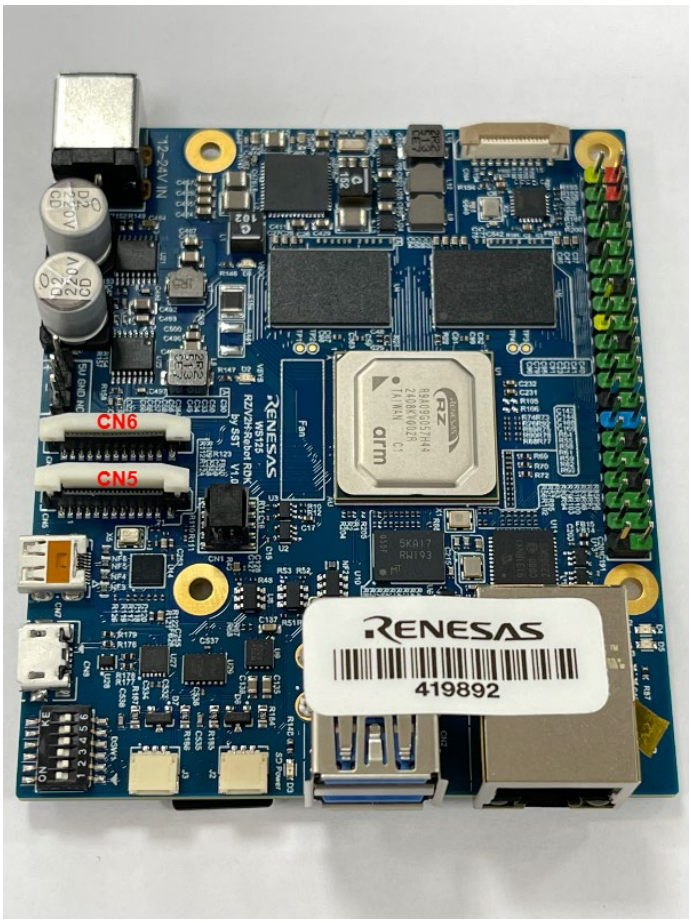


Figure 58. RZ/V2H-RDK CSI cameras (CN5 and CN6)

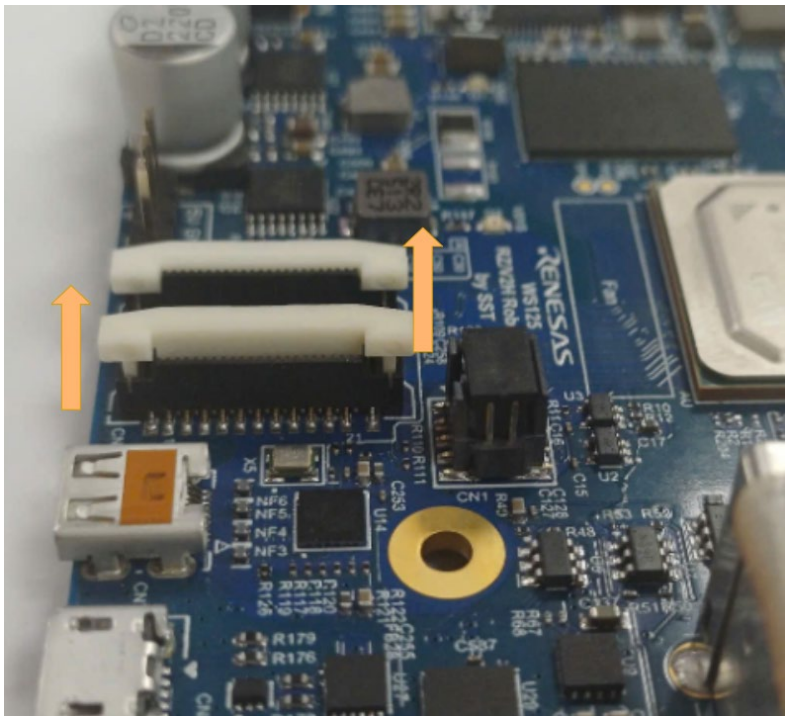


Figure 59. Pull up the notch of RZ/V2H-RDK

The white notch must be pulled up to unlock it.

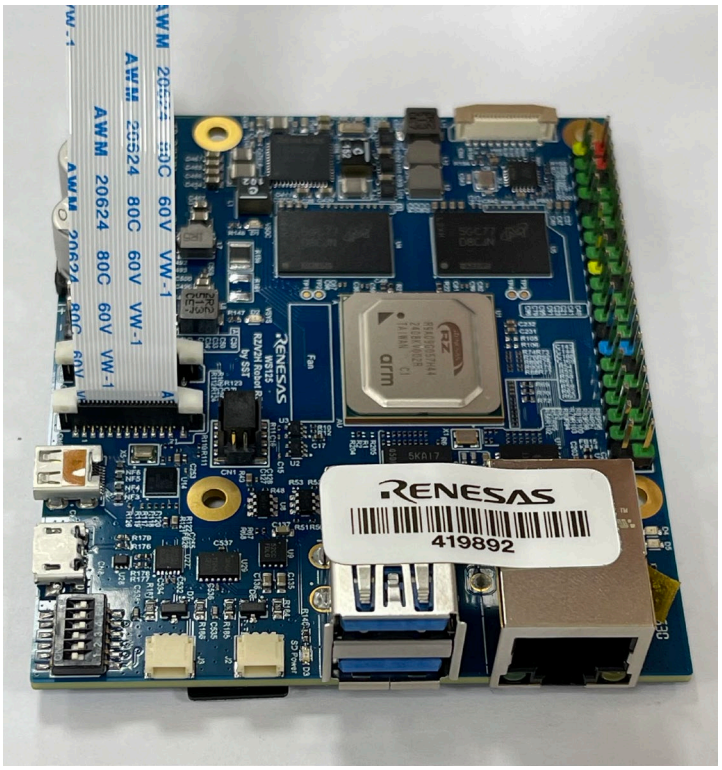


Figure 60. The CSI cable is inserted

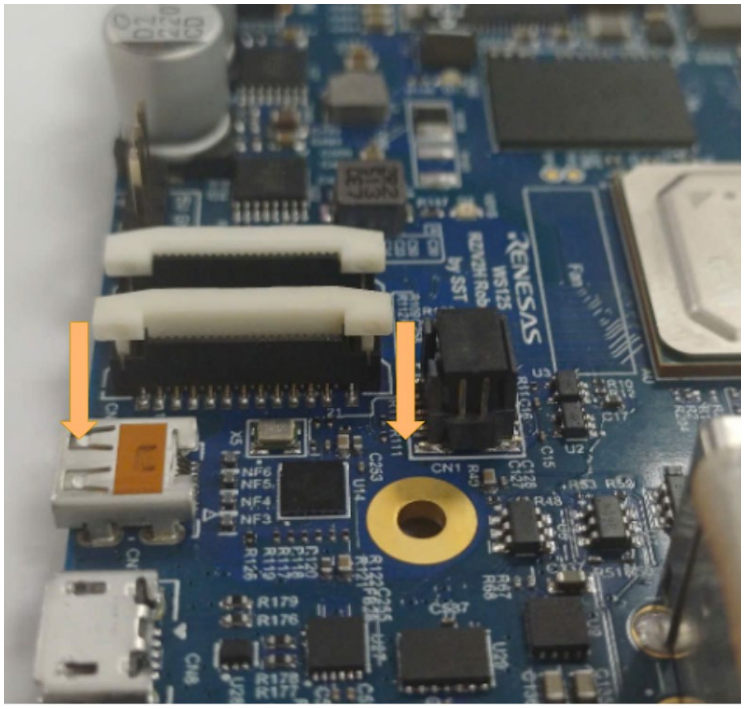


Figure 61. Push down the white notch to lock it when you have inserted the flat cable.

(E) IMDT V2H-SBC

This section describes the camera modules currently supported on the IMDT V2H-SBC, along with the steps required to enable each camera.

The following cameras are supported based on the available drivers and device tree entries:

Table 35. List of cameras with available drivers for IMDT V2H-SBC

Camera Sensor	Support Status
AR1335 (on board)	Supported
AR0521	Not verified
IMX135	Not verified
IMX219	Not verified
IMX462	Not verified
IMX274	Not verified
IMX377	Not verified

The IMDT V2H-SBC provides four CSI camera slots. The interfaces are not equivalent in terms of ISP integration:

- CSI0 (Slot 1) and CSI1 (Slot 2) — Connected via on-board OnSemi AP1302 ISPs. These are enabled by default and require no overlay.
- CSI2 (Slot 3) and CSI3 (Slot 4) — Connected directly to the SoC. These must be enabled via device tree overlay.

To enable CSI2 (Slot 3) AR1335: edit uEnv.txt in partition 1 (FAT32) and uncomment the following line:

```
#enable_overlay_csi22_ar1335=1
```

To enable CSI3 (Slot 4) AR1335: edit uEnv.txt in partition 1 (FAT32) and uncomment the following line:

```
#enable_overlay_csi23_ar1335=1
```

Reboot the board after editing uEnv.txt for the overlay to take effect.

(2) Camera Initialization

Before initializing the camera capture, it needs to be enabled and configured. The Linux distribution has a helper script (v4l2-init.sh) in the /home/root directory to enable and configure the camera.

```
root@rz-cmn:~# cd /home/root/
root@rz-cmn:~# ./v4l2-init.sh --device DEVICE_ID --width WIDTH --height HEIGHT
```

Options:

- --help: show help message and exit
- --device DEVICE_ID: The ID of the media device, where 0 corresponds to /dev/media0
- --width WIDTH: Width of capture image
- --height HEIGHT: Height of capture image

```
root@rz-cmn:~# ./v4l2-init.sh --help
```

If no resolution is specified or an invalid resolution is provided, the default resolution (1920x1080) will be used. For example:

When using a valid resolution, for example, on RZ/G2L-SBC using OV5640 sensor:

```
root@rz-cmn:~# ./v4l2-init.sh --device 0 --width 1280 --height 720
Detected cameras: ov5640 1-003c
Default camera: ov5640 1-003c
Default CSI2: csi-10830400.csi2
Default CRU: cru-ip-10830000.video
Pipeline successfully configured for 1280x720
```

When no resolution is specified:

```
root@rz-cmn:~# ./v4l2-init.sh
Detected cameras: ov5640 1-003c
Default camera: ov5640 1-003c
Default CSI2: csi-10830400.csi2
Default CRU: cru-ip-10830000.video
Pipeline successfully configured for 1920x1080
```

The `v4l2-init.sh` script helps enable the CSI-2 module and select the camera's supported display resolution.

After running the script, initiate a video capture session using the matching width and height:

```
root@rz-cmn:~# gst-launch-1.0 v4l2src device=/dev/video0 ! video/x-
raw,width=1920,height=1080 ! videoconvert ! waylandsink
```

Ensure that the width and height values in the GStreamer pipeline match the resolution specified in v4l2-init.sh. This command starts a continuous stream of the camera feed to the active video display.

8.1.2.6 Quickboot Images and Network Configurations

Renesas provides custom Quickboot images optimized for faster boot times. These images include necessary systemd optimizations and a streamlined kernel to minimize boot delays.

By default, systemd services for networking, D-Bus, and other non-essential components are disabled, leaving only the core boot services active.

The details of these images are provided in section 1.1 Supported Distributions:

Table 36. Custom Quickboot images

Images	Description
renesas-quickboot-cli	A minimal Linux image with Quickboot enabled, offering only a CLI without a desktop environment. It supports HDMI/DSI output but lacks graphical components and a desktop, which makes it ideal for fast-booting command-line-based systems.
renesas-quickboot-wayland	A Quickboot-enabled Linux image with Wayland and Qt support, featuring the Weston graphical desktop environment. It provides a basic graphical desktop, allowing users to develop and integrate custom GUI applications. No desktop applications are included by default. However, the QT framework is available, allowing the user to install and run any QT application.

(1) Enable Networking Stack

For both Quickboot CLI and Quickboot Wayland images, networking (including Wi-Fi, Bluetooth, and SSH services) is disabled by default and must be enabled manually. The required scripts are in /home/root/network-management/.

To see available options before enabling any services, run the help command:

```
root@rz-cmn:~# cd network-management
root@rz-cmn:~/network-management# ./enable_networking_stack.sh help
```

This command displays the usage information along with the following options:

- Wi-Fi: Enable Wi-Fi services.
- Bluetooth: Enable Bluetooth services.
- SSHD: Enable SSH/SCP services.
- all: Enable all network-related services (Wi-Fi, Bluetooth, SSHD).

Run the following command with the appropriate option:

```
root@rz-cmn:~/network-management# ./enable_networking_stack.sh <service>
```

For example, to enable Wi-Fi, run:

```
root@rz-cmn:~/network-management# ./enable_networking_stack.sh wifi
```

To enable all networking services:

```
root@rz-cmn:~/network-management# ./enable_networking_stack.sh all
```

(2) Disable Networking Stack

To restore the default Quickboot behavior and disable unused network services, use the provided script. This removes systemd service symlinks and masks services related to networking, Wi-Fi, Bluetooth, and SSH.

Run the following command with the appropriate option to disable unused services:

```
root@rz-cmn:~/network-management# ./disable_networking_stack.sh <service>
```

For example, to disable Bluetooth:

```
root@rz-cmn:~/network-management# ./disable_networking_stack.sh bluetooth
```

To fully restore Quickboot's default behavior by disabling all networking services:

```
root@rz-cmn:~/network-management# ./disable_networking_stack.sh all
```

(3) Kernel Optimization

By default, the release package does not optimize the kernel. This is purposefully done to allow kernel debugging and have more verbose logs.

If an optimized kernel is required, it becomes necessary to rebuild a kernel through the SDK or yocto. The optimization setting is configured in the local.conf file within the Yocto build environment (typically located under build/conf/local.conf.)

Set the variable OPTIMIZE_KERN in local.conf to enable kernel optimization. This configuration disables unused features and converts certain built-in modules (USB, touchscreen, CANFD, etc.) into loadable modules. The result is a smaller kernel, faster boot time, and improved resource utilization.

To optimize the kernel, follow these steps to modify the local.conf:

3. Open the local.conf file in the Yocto build configuration.
4. Set the 'OPTIMIZE_KERN' from "0" to "1".
   ```
   # Optimized Linux Kernel Support: Build with optimizations for the Linux kernel
   # Default: 0 - Disable
   # Set to: 1 - Enable
   OPTIMIZE_KERN = "1"
   ```
   This will ensure that unnecessary kernel features are disabled, and certain modules are built as loadable, leading to a more efficient system.
5. Rebuild and deploy the image to apply the changes.

8.1.2.7 Playing Video Files on the RZ board

Use gst-launch-1.0 to play video files. The playbin element in GStreamer makes it easy to play multimedia content. Prepare an MP4 file and run the following command:

```
root@rz-cmn:~# gst-launch-1.0 playbin uri=file:///<path/to/your/video/path>
```

For example,

```
root@rz-cmn:~# gst-launch-1.0 playbin uri=file:///home/root/videos/h264-hd-30.mp4
```

This will start an MP4 video and display it on the screen.

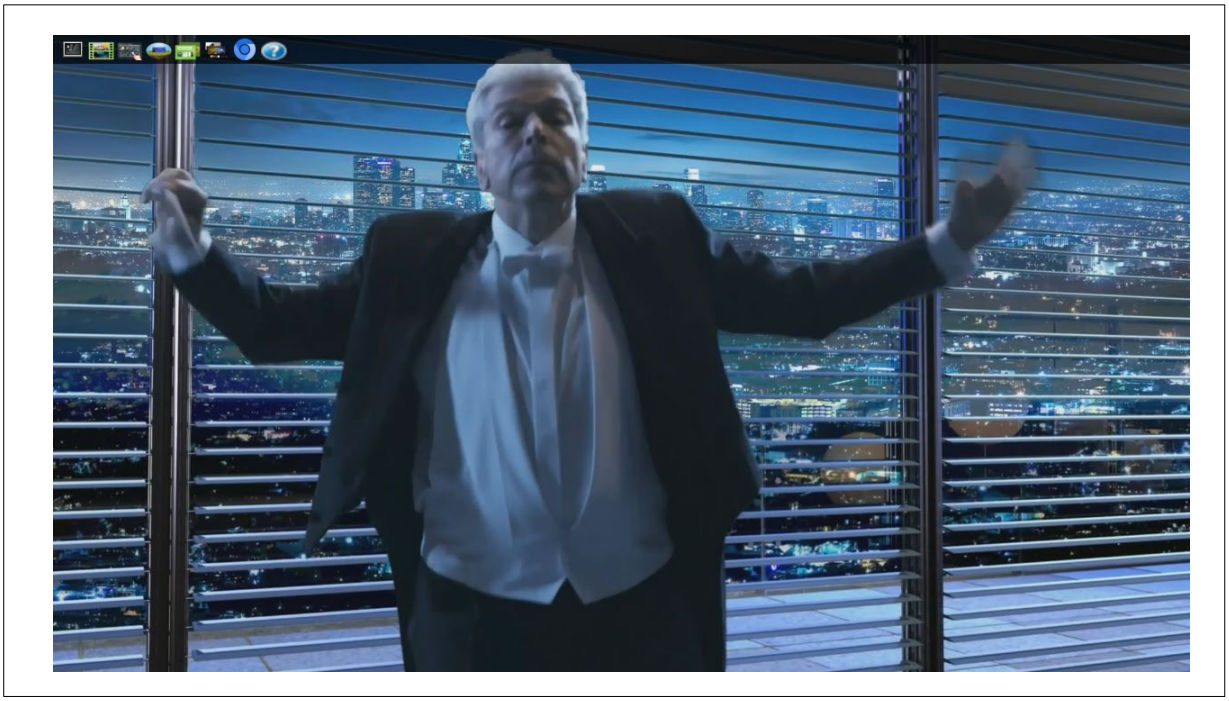


Figure 62. Playing an MP4 video on the RZ board

8.1.2.8 Package Management

The distribution comes with the Debian package manager ‘apt-get’ and ‘dpkg’ for binary package handling.

Follow the steps below to modify the Debian package repository and install packages according to your needs.

1. Add/modify sources.list file to address the package repository:
   The ‘sources.list’ is a critical configuration file for package installation and updates used by package managers on Debian-based Linux distributions. The ‘sources.list’ file contains a list of URLs for repository addresses where the package manager can find software packages. These repositories may be maintained by the Linux distribution itself or by third-party individuals or organizations.

   Currently, the default `sources.list`, which is located in /etc/apt/sources.list.d/sources.list/ directory is as follows.

```
deb [arch=arm64] http://old-releases.ubuntu.com/ubuntu/ oracular main multiverse universe
deb [arch=arm64] http://old-releases.ubuntu.com/ubuntu/ oracular-security main multiverse universe
deb [arch=arm64] http://old-releases.ubuntu.com/ubuntu/ oracular-backports main multiverse universe
deb [arch=arm64] http://old-releases.ubuntu.com/ubuntu/ oracular-updates main multiverse universe
```

2. Update the defined package index for apt-get.

```
root@rz-cmn:~# apt-get update
```

   Ensure you have internet access before running apt-get update.

   In the contents of the sources.list file, each line has [arch=arm64]. This is because the RZ/G2L SoC is an ARM 64 (aarch64) core. This can be verified by the lscpu command:

```
root@rz-cmn:~# lscpu
Architecture:                    aarch64
CPU op-mode(s):                  32-bit, 64-bit
Byte Order:                      Little Endian
CPU(s):                          2
...
Vendor ID:                       ARM
```

   By specifying [arch=arm64] in sources.list file, apt-get will filter for the proper binary packages in the repository. This will limit the existing APT sources to ARM64 type only. However, if we use a repository that only hosts ARM 64-bit (aarch64) packages, we do not need to specify [arch=arm64] in the sources.list entry. For example:

```
deb http://deb.debian.org/debian trixie main contrib non-free
```

   Remember that sources do not have to be a single origin. It is very common to add multiple repositories and sources for packages and manage them using keys.

   The source management is beyond the scope of this document.

3. Installing packages using apt-get:
   To install a package using apt-get, use the following command:

```
root@rz-cmn:~# apt-get install <package-name>
```

Note: The release currently uses Ubuntu Oracular as the default APT repository source. Modifying the APT sources (e.g., switching to Debian or using third-party repositories) may break the boot or cause installation issues for some applications due to changes in package versions, availability, or dependencies. Proceed with caution if you plan to alter the default APT configuration.

(1) Docker Installation Setup

This guide walks you through enabling Docker support at the kernel level, installing Docker, configuring firewall compatibility, and verifying the installation.

Step 1: Enable Docker support in kernel build

Docker support is disabled by default for Yocto images. To enable Docker integration at the kernel level, set the following option in your `local.conf` build as below:

```
DOCKER_SUPPORT = "1"  # Set to "1" to enable; "0" to disable (default)
```

Note: After enabling Docker support, you must rebuild the kernel and replace the existing kernel image with the newly built one for the changes to take effect.

Step 2: Install Docker

Ensure the device has internet access, then update package lists and install Docker:

```
root@rz-cmn:~# apt-get update
root@rz-cmn:~# apt-get install docker.io
```

Step 3: Configure firewall compatibility. Docker supports only iptables-legacy and iptables-nft. Directly using nftables firewall rules is incompatible. Switch to legacy iptables with:

```
root@rz-cmn:~# update-alternatives --set iptables /usr/sbin/iptables-legacy
root@rz-cmn:~# update-alternatives --set ip6tables /usr/sbin/ip6tables-legacy
```

Restart Docker to apply these changes:

```
root@rz-cmn:~# systemctl restart docker
```

Step 4: Verify Docker Installation by running:

```
root@rz-cmn:~# docker run hello-world
```

A successful run will display a message confirming Docker is working properly.

(2) Using DPKG to Install Packages

The utility ‘dpkg’ is the low-level package manager for Debian-based systems. It is the local system-wide package manager. It handles installation, removal, provisioning, indexing, and other aspects of packages installed on the system. However, it does not perform any cloud operations. Dpkg also does not handle dependency resolution. This is another task handled by a high-level manager like ‘apt-get’. In fact, ‘dpkg’ is the backend for ‘apt-get’. While ‘apt-get’ handles fetching and indexing, the local installations and management of the packages are performed by the ‘dpkg’ manager.

Basic dpkg commands:

- dpkg -i <package.deb>: Installs a package.deb package.
- dpkg -r <package>: Removes a package.
- dpkg -l <pattern>: Lists installed packages matching <pattern>.
- dpkg -s <package>: Provides information about an installed package.

You can install any <package>.deb (where '<package>' is a placeholder for the name of the real package being installed) using dpkg with the following command:

```
root@rz-cmn:~# dpkg -i <package>.deb
```

After installing a package using dpkg, if you need to resolve dependency issues, use the following command:

```
root@rz-cmn:~# apt-get install -f
```

8.1.2.9 Python GUI Programming with Tkinter

This section outlines the steps for creating a basic graphical user interface (GUI) application using Tkinter, the standard Python interface to the Tk GUI toolkit. Tkinter is included with Python by default, so no additional libraries are required. It offers a straightforward way to develop desktop applications, making it a practical option for many use cases.

> Note: Running graphical applications such as Tkinter requires access to the X11 display server, which is provided by Xwayland in this setup. Therefore, the application must be run as the weston user (not as root), because only that user has permission to access the running Xwayland display session (DISPLAY=:0).

The following steps will show how to create a new Tkinter application:

1. Switch to user 'weston'

```
root@rz-cmn:~# su - weston
```

2. Create a working directory on the RZ board to develop and store the Python application.

```
rz-cmn:~$ mkdir ~/python_apl
rz-cmn:~$ cd ~/python_apl
```

3. Create a new Python file (For example, main.py) in the work directory.

```
rz-cmn:~/python_apl$ vi main.py
```

4. Develop a Simple Python GUI Application with Tkinter.

- Import the tkinter module:
  ```
  import tkinter as tk
  ```
  This statement imports the Tkinter module, allowing access to its classes and functions for creating GUI elements.
- Create a main window.
  ```
  root = tk.Tk()
  ```
  This creates the main application window.
- Change the window title and resolution as desired.
  ```
  root.title("Sample application")
  root.geometry("200x100")
  ```

- Create and place a label.

```
label = tk.Label(root, text="Press the button", width=20, height=2)
label.pack()
```

- Create and place a button.

```
button = tk.Button(root, text="Click Me", command=on_button_click, width=10,height=2)
button.pack()
```

This creates a button with the text "Click Me" and associates it with the on_button_click function.

When the button is pressed, the function is called.

- Define a user function that helps to handle the on-click event and shows “Hello, Tkinter!” on the application’s window.

```
def on_button_click():
    label.config(text="Hello, Tkinter!")
```

- Run the application

```
root.mainloop()
```

This starts the Tkinter event loop, which waits for user interactions and updates the UI accordingly.

- The completed Python program: “main.py”.

```
import tkinter as tk

def on_button_click():
    label.config(text="Hello, Tkinter!")

root = tk.Tk()
root.title("Sample application")
root.geometry("200x100")

# Create a label
label = tk.Label(root, text="Press the button", width=20, height=2)
label.pack()

# Create a button
button = tk.Button(root, text="Click Me", command=on_button_click, width=10,height=2)
button.pack()

# Run the application
root.mainloop()
```

4. Run the application

- Ensure the RZ board is connected to an external display. If the display is not set automatically, set the DISPLAY environment variable as follows:
-

```
rz-cmn:~$ export DISPLAY=:0
```

- Run the Python application:

```
rz-cmn:~$ python3 main.py
```

Table 37. Python TKinter application GUI

Figure 63. Initial GUI layout

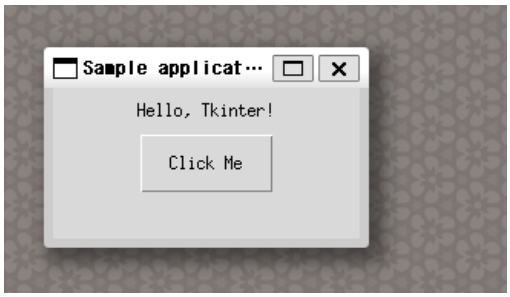


Figure 64. After the button 'Click me' is clicked

8.1.2.10 Install Packages Using Python3-Pip

The distribution includes Python 3 along with useful libraries/modules/packages such as Pip3, Numpy, Pandas, PySerial, Matplotlib, etc. This section will focus on using Pip3, the package installer for Python 3, to manage additional packages.

Python3-pip allows you to install, update, and manage Python packages from the Python Package Index (PyPI) and other repositories.

To install a new package using pip3, use the following command:

```
root@rz-cmn:~# pip3 install <package_name>
```

For example, to install the `requests` package, you would run:

```
root@rz-cmn:~# pip3 install requests
```

To verify that the `requests` package (or any other installed package) is correctly installed, you can use:

```
root@rz-cmn:~# pip3 show requests
```

This command provides details about the requests package, including its version and installation location.

Alternatively, you can list all installed packages and check if the `requests` package is included:

```
root@rz-cmn:~# pip3 list
```

This will confirm that the package is installed and available for use.

8.1.3 RZ/G2L-SBC Yocto Features

8.1.3.1 40-Pin IO Expansion Interface

The 40-pin IO Expansion Interface on RZ/G2L-SBC has support for:

- I²C channel 0
- I²C channel 3
- SPI channel 0
- SCIF channel 0
- CAN channel 0
- CAN channel 1

- GPIO pin-function (default).

Notes:

- The GPIO pin array is multiplexed with peripheral IO lines.
- By default, I²C channel 0 and SCIF channel 0 are enabled.
- The rest of the pins are GPIOs by default.
- Enable the other functions by editing the uEnv.txt on the SD card and enabling the appropriate device tree overlay file (DT overlays). This is also how some of the dedicated drivers are enabled, like the display.
- Reboot the board for the overlay to take effect.

(1) GPIO (General Purpose I/O pins) with libgpiod

By default, most pins are configured as GPIOs on the SBC's 40-pin GPIO pin header. This section details how to identify and control these pins using the libgpiod library and its associated command-line tools. The IO pins are explored in detail in Figure 94. 40 PIN GPIO map with orientation details.

Unlike the deprecated sysfs interface, libgpiod provides a standardized and kernel-integrated method for GPIO management. It interacts with GPIO character devices (e.g., /dev/gpiochip0, /dev/gpiochip1) to offer a more efficient and flexible control over individual GPIO lines.

All GPIO pins on the 40-pin header are exposed through /dev/gpiochip0.

Table 38. GPIO pins and functions

GPIO Line	Function	Group	Pin	J3 PINs		Pin	Group	Function	GPIO Line
				Left side	Right side				
	3.3V			**1**	**2**			5V	
370	I²C3 SDA	46	2	**3**	**4**			5V	
371	I²C3 SCL	46	3	**5**	**6**			GND	
184	GPIO	23	0	**7**	**8**	0	38	SCIF0 TX	304
	GND			**9**	**10**	1	38	SCIF0 RX	305
456	GPIO	42	0	**11**	**12**	2	7	GPIO	58
336	GPIO	27	0	**13**	**14**			GND	
225	GPIO	28	1	**15**	**16**	0	8	GPIO	64
	3.3V			**17**	**18**	0	15	GPIO	120
345	SPI0 MOSI	43	1	**19**	**20**			GND	
346	SPI0 MISO	43	2	**21**	**22**	1	14	GPIO	113
344	SPI0 CK	43	0	**23**	**24**	3	43	SPI0 CS	347
	GND			**25**	**26**	1	11	GPIO	89
	I²C0 SDA			**27**	**28**			I²C0 SCL	

32	GPIO	4	0	**29**	**30**			GND	
33	GPIO	4	1	**31**	**32**	0	32	GPIO	256
177	GPIO	22	1	**33**	**34**			GND	
337	CAN0 TX	42	1	**35**	**36**	1	23	GPIO	**185**
88	CAN0 RX	11	0	**37**	**38**	0	46	CAN1 TX	**368**
	GND			**39**	**40**	1	46	CAN1 RX	**369**

(A) UNDERSTANDING LIBGPIOD'S CONCEPTS: CHIPS AND LINES

Instead of a single, linear pin number system, libgpiod organizes GPIOs around two key concepts:

- GPIO Chips: These represent the physical GPIO controllers on your system. Each chip manages a specific set of GPIO lines. You'll typically see them identified as gpiochip0, gpiochip1, and so on.
- GPIO Lines: Each chip contains a number of individual GPIO lines, identified by an offset within that chip (e.g., line 0, line 1, line 2, etc.).

(B) IDENTIFYING GPIO CHIPS AND LINES

Before GPIO control can be initiated, the specific chip and line offset corresponding to the desired pin must be identified.

The gpiodetect command lists all GPIO controllers on the system:

```
root@rz-cmn:# gpiodetect
```

Output will be similar to

```
root@rz-cmn:# gpiochip0 [chip_name_0] (XX lines)
root@rz-cmn:# gpiochip1 [chip_name_1] (YY lines)
# ... additional chips
```

(C) INSPECT LINES ON A SPECIFIC GPIO CHIP:

Detailed information about individual lines on a chip is obtained using gpioinfo:

```
root@rz-cmn:# gpioinfo gpiochip0
```

Please replace gpiochip0 with the relevant chip name or number identified via gpiodetect. This command lists each line, including its offset, any assigned name, and its current state. This output is essential for mapping physical pins to libgpiod's chips and offsets.

(D) CONFIGURING GPIO PINS

To set the state of a GPIO pin, use the gpioset command with the following syntax:

```
gpioset -c <chip> <pin> = value
```

- chip: Specifies the GPIO chip (e.g., gpiochip0).
- pin: Refers to the specific GPIO pin number on that chip.
- value: Sets the pin state (0 for low, 1 for high).

Examples:

To set GPIO pin 0 on gpiochip0 to a low state:

```
root@rz-cmn:~# gpioset -c gpiochip0 0=0
```

To set GPIO pin 0 on gpiochip0 to a high state:

```
root@rz-cmn:~# gpioset -c gpiochip0 0=1
```

(E) USING LIBGPIOD IN PYTHON

libgpiod provides an official Python API that allows applications to control GPIO lines using the same chip / line model used by the command-line tools.

This interface is stable, easy to use, and suitable for scripting, automation, or application integration.

First, install the latest supported version (example: gpiod-2.4.0) using pip3:

```
root@rz-cmn:~# pip3 install gpiod
```

Expected a successful installation message

```
root@rz-cmn:~# pip3 install gpiod
Collecting gpiod
  Downloading gpiod-2.4.0-cp312-cp312-manylinux2014_aarch64.manylinux_2_17_aarch
64.manylinux_2_28_aarch64.whl.metadata (4.1 kB)
Downloading gpiod-2.4.0-cp312-cp312-manylinux2014_aarch64.manylinux_2_17_aarch64
.manylinux_2_28_aarch64.whl (104 kB)
Installing collected packages: gpiod
Successfully installed gpiod-2.4.0
WARNING: Running pip as the 'root' user can result in broken permissions and con
flicting behaviour with the system package manager, possibly rendering your syst
em unusable.It is recommended to use a virtual environment instead: https://pip.
pypa.io/warnings/venv. Use the --root-user-action option if you know what you ar
e doing and want to suppress this warning.
```

Once gpiod-2.4.0 is installed, create a Python file (for example: gpio_usage.py) to work with GPIO lines.

```
root@rz-cmn:~# vi gpio_usage.py
```

1. Importing the library, gpiod, and time

```
import gpiod
import time
```

2. Define a GPIO chip and a GPIO line. A GPIO chip represents a hardware controller (for example: /dev/gpiochip0). A GPIO line is an individual pin within that chip, identified by a unique offset number.

```
GPIO_CHIP = "/dev/gpiochip0"
GPIO_LINE = 0
```

3. Configuring Direction and Output State: A line must be requested with a configuration before use.

Set as output

```
config = gpiod.LineSettings(
    direction=gpiod.line.Direction.OUTPUT,
    output_value=gpiod.line.Value.ACTIVE   # initial active
)
```

Or, set as input

```
config = gpiod.LineSettings(
    direction=gpiod.line.Direction.INPUT,
)
```

4. Requesting the Line

The configuration is applied when creating a request:

```
request = gpiod.request_lines(
    GPIO_CHIP,
    config={GPIO_LINE: config},        # GPIO line 0
    consumer="gpio-example"
)
```

The request holds ownership of the line

5. Setting Output Values

Output lines can be written using:

```
request.set_value(GPIO_LINE, gpiod.line.Value.ACTIVE) # set active for GPIO_LINE
request.set_value(GPIO_LINE, gpiod.line.Value.INACTIVE) # set inactive for
GPIO_LINE
```

6. Examples

The following example drives a GPIO line HIGH, waits for user input, and then releases the state.

```
import gpiod
import time

GPIO_CHIP = "/dev/gpiochip0"
GPIO_LINE = 0

config = gpiod.LineSettings(
    direction=gpiod.line.Direction.OUTPUT,
    output_value=gpiod.line.Value.ACTIVE    # initial active
)

request = gpiod.request_lines(
    GPIO_CHIP,
    config={GPIO_LINE: config},         # GPIO line 0
    consumer="gpio-example"
)

request.set_value(GPIO_LINE, gpiod.line.Value.ACTIVE)    # Set active GPIO line 0
input()      # Wait
```

While the program is running, the line is actively held HIGH.

When the program exits (after pressing Enter), the framework releases the line, and its state returns to the default.

Note:

- When a Request object (or request_lines context) is released—either by exiting a with block or when the program terminates—the GPIO line is released back to the kernel.
- The line may then revert to the default state defined in the device tree or pinctrl.
- It is not guaranteed to stay HIGH or LOW after exit unless explicitly configured in the device tree.

(2) Enabling I²C Function (Channel 3 – RIIC3)

Edit `uEnv.txt` and uncomment the line as follows to enable I²C channel 3 on the 40 IO expansion interface:

Change the following line:

```
#enable_overlay_i2c=1
```

To

```
enable_overlay_i2c=1
```

Then reboot the RZ/G2L-SBC.

To check if I²C channel three is enabled, run the following command and check the result:

```
root@rz-cmn:~# i2cdetect -l
i2c-3   i2c             Renesas RIIC adapter                    I2C adapter
i2c-1   i2c             Renesas RIIC adapter                    I2C adapter
i2c-4   i2c             i2c-1-mux (chan_id 0)                   I2C adapter
i2c-0   i2c             Renesas RIIC adapter                    I2C adapter
root@rz-cmn:~#
```

To map out all the devices present on the I²C channel 3, execute the following command:

```
root@rz-cmn:~# i2cdetect -y -r 3
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: 50 -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

In the i2cdetect output, the row labels (e.g., 00:, 10:, 20:, …) indicate the most significant nibble (high 4 bits) of the I²C address, while the column headers (0 1 2 3 … f) indicate the least significant nibble (low 4 bits).

In the example above, the device at 0x50 is found in row 50 and column 0.

(A) EXAMPLE: READING/WRITING I²C DATA IN PYTHON

1. Install the I²C library: the release package already includes this library, but it can be installed manually if needed.

```
root@rz-cmn:# pip3 install smbus2
```

2. Create and open the Python file i2c_example.py using vi/vim

```
root@rz-cmn:# vi i2c_example.py
```

3. Import libraries

```
from smbus2 import SMBus
import time
```

4. Define the I²C channel, address, and register, and then open the I²C bus

```
I2C_CHANNEL = 3       # corresponds to /dev/i2c-3
DEVICE_ADDR = 0x50    # example device detected via i2cdetect
REG = 0x01  # target register on the device for read/write operations

# Open I2C bus
bus = SMBus(I2C_CHANNEL)
```

5. Write a single byte

```
# Single-byte write to register REG (0x01)
bus.write_byte_data(DEVICE_ADDR, REG, 0xAB)
print(f"Wrote 0xAB to register 0x{REG:02X}")
time.sleep(0.01)
```

6. Read a single byte

```
# Single-byte read
data = bus.read_byte_data(DEVICE_ADDR, REG)
print(f"Data read from register 0x{REG:02X}: 0x{data:02X}")
```

7. Close the I²C bus

```
bus.close()
```

8. Run

```
root@rz-cmn:~# python3 i2c_example.py
Wrote 0xAB to register 0x01
Data read from register 0x01: 0xAB
```

These are all the required APIs and steps. The following code provides a complete implementation, illustrating how to use them in the examples described above

```
from smbus2 import SMBus
import time

I2C_CHANNEL = 3       # corresponds to /dev/i2c-3
DEVICE_ADDR = 0x50    # example device detected via i2cdetect
REG = 0x01  # target register on the device for read/write operations

bus = SMBus(I2C_CHANNEL)

# Single-byte write
bus.write_byte_data(DEVICE_ADDR, REG, 0xAB)
print(f"Wrote 0xAB to register 0x{REG:02X}")
time.sleep(0.01)

# Single-byte read
data = bus.read_byte_data(DEVICE_ADDR, REG)
print(f"Data read from register 0x{REG:02X}: 0x{data:02X}")

bus.close()
```

(3) SPI function (Channel 0 – RSPI0)

Edit `uEnv.txt` as follows to enable SPI channel 0 on the 40 IO expansion interface:

Change the following line:

```
#enable_overlay_spi=1
```

To

```
enable_overlay_spi=1
```

This will enable the SPI module.

Run the following command to configure the SPI:

```
root@rz-cmn:~# spi-config -d /dev/spidev0.0 -q
/dev/spidev0.0: mode=0, lsb=0, bits=8, speed=2000000, spiready=0
```

Connect Pin 19 (RSPI0 MOSI) to Pin 21 (RSPI0 MISO), then run the command below and check the result. The idea is to transmit on MOSI and read back on MISO to validate the transfer.

```
root@rz-cmn:~# echo -n -e "1234567890" | spi-pipe -d /dev/spidev0.0 -s 10000000 |
hexdump
0000000 3231 3433 3635 3837 3039
000000a
```

(A) EXAMPLE: TRANSMIT SPI DATA IN PYTHON

This example demonstrates how to communicate with an SPI device using Python. For verification, the MOSI pin is connected to the MISO pin (echo mode) to validate data transmission. In real applications, MOSI connects to an SPI slave device. The example shows step-by-step how to open the SPI channel, configure settings, transmit data, and read back the response.

1. Install SPI

```
root@rz-cmn:~# pip3 install spidev
Collecting spidev
  Downloading spidev-3.8.tar.gz (13 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: spidev
  Building wheel for spidev (pyproject.toml) ... done
  Created wheel for spidev: filename=spidev-3.8-cp312-cp312-linux_aarch64.whl
size=47912
sha256=18739eede95a522c9de83f6b055c079392ec0cd43abff4b855cd22d6e53bf130
  Stored in directory:
/home/root/.cache/pip/wheels/52/52/d0/de1ca6c54f08a7ec8c99844a6adff825366800f3e06
d78ee05
Successfully built spidev
Installing collected packages: spidev
Successfully installed spidev-3.8
```

2. Create a new script spi_example.py using vi, vim.

```
root@rz-cmn:# vi spi_example.py
```

3. Import libraries

```
import spidev
import time
```

4. Create SPI object

```
# Create SPI object
spi = spidev.SpiDev()
```

5. Open SPI channel 0, device 0. This corresponds to /dev/spidev0.0 (RSPI0)

```
# Open SPI channel 0, device 0
spi.open(0, 0)
```

6. Configure SPI speed and mode

```
# Configure SPI
spi.max_speed_hz = 500000  # 500 kHz
spi.mode = 0               # SPI mode 0
```

7. Prepare the data to transmit

```
# Prepare data to transmit
data_out = [0x01, 0x02, 0x03, 0xAB]
```

8. Transmit data and receive response

```
# Transmit data and receive response (echo mode)
data_in = spi.xfer2(data_out)

# Display transmitted and received data
print(f"Transmitted data: {data_out}")
print(f"Received data   : {data_in}")
```

9. Close SPI bus

```
spi.close()
```

10. Run

```
root@rz-cmn:~# python3 spi_example.py
Transmitted data: [1, 2, 3, 171]
Received data   : [1, 2, 3, 171]
```

These are all the required APIs and steps. The following code provides a complete implementation, illustrating how to use them in the examples described above

```
import spidev
import time

# Create SPI object
spi = spidev.SpiDev()

# Open SPI channel 0, device 0
spi.open(0, 0)

# Configure SPI
spi.max_speed_hz = 500000   # 500 kHz
spi.mode = 0                # SPI mode 0

# Prepare data to transmit
data_out = [0x01, 0x02, 0x03, 0xAB]

# Transmit data and receive response (echo mode)
data_in = spi.xfer2(data_out)

# Display transmitted and received data
print(f"Transmitted data: {data_out}")
print(f"Received data   : {data_in}")

# Close SPI bus
spi.close()
```

(4) CAN Function (Channel 0,1 - CAN 0, CAN 1)

Edit `uEnv.txt` as follows to enable CAN channel 0,1 on 40 IO expansion interface:

Change the following line:

```
#enable_overlay_can=1
```

To

```
enable_overlay_can=1
```

To verify that the CAN channels are enabled, run the following command and check the result:

```
root@rz-cmn:~# ip a | grep can
3: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
4: can1: <NOARP,ECHO> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
root@rzg2l-sbc:~#
```

Then set up for CAN devices. Now you can go up/down the interface or send data over CAN channels.

The example below shows the communication between two CAN channels.

```
root@rz-cmn:~# ip link set can0 down
root@rz-cmn:~# ip link set can0 type can bitrate 500000
root@rz-cmn:~# ip link set can0 up
[   48.120419] IPv6: ADDRCONF(NETDEV_CHANGE): can0: link becomes ready
root@rz-cmn:~# ip link set can1 down
root@rz-cmn:~# ip link set can1 type can bitrate 500000
root@rz-cmn:~# ip link set can1 up
[   69.906039] IPv6: ADDRCONF(NETDEV_CHANGE): can1: link becomes ready
root@rz-cmn:~# candump can0 & cansend can1 123#01020304050607
[1] 271
  can0  123   [7]  01 02 03 04 05 06 07
root@rz-cmn:~# candump can1 & cansend can0 123#01020304050607
[2] 273
  can0  123   [7]  01 02 03 04 05 06 07
  can1  123   [7]  01 02 03 04 05 06 07
root@rz-cmn:~#
```

(A) EXAMPLE: SEND DATA BETWEEN TWO CAN CHANNELS IN PYTHON

This example demonstrates how to enable and communicate over the CAN bus. CAN channels can0 and can1 correspond to the 40-pin I/O expansion interface. The procedure shows how to enable the CAN modules, verify their state, and send/receive messages between the two channels.

Important: Ensure the CAN interfaces are up before running the Python script:

```
root@rz-cmn:~# ip link set can0 up type can bitrate 500000
root@rz-cmn:~# ip link set can1 up type can bitrate 500000
```

1. Install CAN library

```
root@rz-cmn:~# pip3 install python-can
```

2. Create a new script spi_example.py using vi, vim.

```
root@rz-cmn:# vi spi_example.py
```

3. Import libraries

```
import can
import time
```

4. Open CAN buses. Example here uses can0 as RX and can1 as TX

```
# Use 'interface' instead of deprecated 'bustype'
bus0 = can.Bus(channel='can0', interface='socketcan')  # RX
bus1 = can.Bus(channel='can1', interface='socketcan')  # TX
```

5. Prepare the CAN frame (message)

```
# Data for frame: 123#01020304050607
msg = can.Message(
    arbitration_id=0x123,
    data=[0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07],
    is_extended_id=False
)
```

- arbitration_id: CAN identifier (0x123 in this example).
- data: list of bytes to transmit. (In this script, the array contains seven bytes.)
- is_extended_id=False indicates a standard 11-bit identifier; set True for 29-bit extended IDs.

6. Send the frame

```
print("Sending frame on CAN1...")
bus1.send(msg)
print("Frame sent.")
```

- bus1.send(msg) transmits the prepared CAN frame on can1.
- If the interface is down, send will fail with CanOperationError: Network is down. Ensure interfaces are up first.
- The send is non-blocking at the application level; kernel handles retries/ACKs on the bus.

7. Receive a frame

```
rx_msg = bus0.recv(timeout=1)   # 1-second timeout
print(f"Received on CAN0: ID=0x{rx_msg.arbitration_id:X}\
    Data={rx_msg.data.hex()}")
```

- bus0.recv(timeout=1) waits up to 1 second for a frame on can0.
- If a frame is received, rx_msg contains arbitration_id, data (bytes), timestamp, etc.
- Important: recv() can return None on timeout. To avoid errors, check if rx_msg is None before accessing fields. Example safe pattern:

```
rx_msg = bus0.recv(timeout=1)
if rx_msg is None:
    print("No message received within timeout")
else:
    print(f"Received on CAN0: ID=0x{rx_msg.arbitration_id:X}\
          Data={rx_msg.data.hex()}")
```

8. Shutdown / cleanup

```
bus0.shutdown()
bus1.shutdown()
```

9. Run

```
root@rz-cmn:~# python3 can_example.py
Sending frame on CAN1...
Frame sent.
Received on CAN0: ID=0x123          Data=01020304050607
```

These are all the required APIs and steps. The following code provides a complete implementation, illustrating how to use them in the examples described above

```
#!/usr/bin/env python3

import can
import time

# Use 'interface' instead of deprecated 'bustype'
bus0 = can.Bus(channel='can0', interface='socketcan')  # RX
bus1 = can.Bus(channel='can1', interface='socketcan')  # TX

# Data for frame: 123#01020304050607
msg = can.Message(
    arbitration_id=0x123,
    data=[0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07],
    is_extended_id=False
)

print("Sending frame on CAN1...")
bus1.send(msg)
print("Frame sent.")

rx_msg = bus0.recv(timeout=1)
if rx_msg is None:
    print("No message received within timeout")
else:
    print(f"Received on CAN0: ID=0x{rx_msg.arbitration_id:X}\
          Data={rx_msg.data.hex()}")


bus0.shutdown()
bus1.shutdown()
```

8.1.3.2 Wi-Fi 802.11 Module

RZ/G2L-SBC comes equipped with an onboard wireless 802.11 module. The image is ready with all the necessary tools to connect to Wi-Fi. The Wi-Fi can be configured on the command line, which can be either on the desktop UI or the UART tty from the host.

The following shows how to enable the 802.11 Wi-Fi module and connect to a network.

```
root@rz-cmn:~# connmanctl
connmanctl> enable wifi
Enabled wifi
connmanctl> agent on
Agent registered
connmanctl> scan wifi
Scan completed for wifi
connmanctl> services
    xDredme10zW          wifi_0025ca329da3_78447265646d6531307a57_managed_psk
                         wifi_0025ca329da3_hidden_managed_psk
    REL-GLOBAL           wifi_0025ca329da3_52454c2d474c4f42414c_managed_ieee8021x
    R-GUEST              wifi_0025ca329da3_522d4755455354_managed_none
    RVC-WLS              wifi_0025ca329da3_5256432d574c53_managed_ieee8021x
connmanctl> connect wifi_0025ca329da3_78447265646d6531307a57_managed_psk
Agent RequestInput wifi_0025ca329da3_78447265646d6531307a57_managed_psk
  Passphrase = [ Type=psk, Requirement=mandatory ]
Passphrase? nFjey48aT9pk
connmanctl> exit
```

To confirm the Wi-Fi is connected, ping the outside world:

```
root@rz-cmn:~# ping www.google.com
PING www.google.com(hkg07s39-in-x04.1e100.net (2404:6800:4005:813::2004)) 56 data bytes
64 bytes from hkg07s39-in-x04.1e100.net (2404:6800:4005:813::2004): icmp_seq=1 ttl=57
time=43.2 ms
64 bytes from hkg07s39-in-x04.1e100.net (2404:6800:4005:813::2004): icmp_seq=2 ttl=57
time=81.1 ms
64 bytes from hkg07s39-in-x04.1e100.net (2404:6800:4005:813::2004): icmp_seq=3 ttl=57 time=124
ms
```

Note: The Ethernet interfaces may potentially interfere with the routing of communication through Wi-Fi. If issues start appearing, use the following commands to disable the Ethernet ports.

```
root@rz-cmn:~# ifconfig end0 down
root@rz-cmn:~# ifconfig end1 down
```

8.1.3.3 MIPI DSI Display Touch Panel

The RZ/G2L-SBC has an MIPI DSI interface that supports both a display module and a touch interface. The DSI port supports dual-channel DSI and one I²C interface in the connector.

(1) Hardware Interfacing

Given below are pictures of the Waveshare 5” DSI display panel with touch screen assembly. The pictures are self-explanatory.

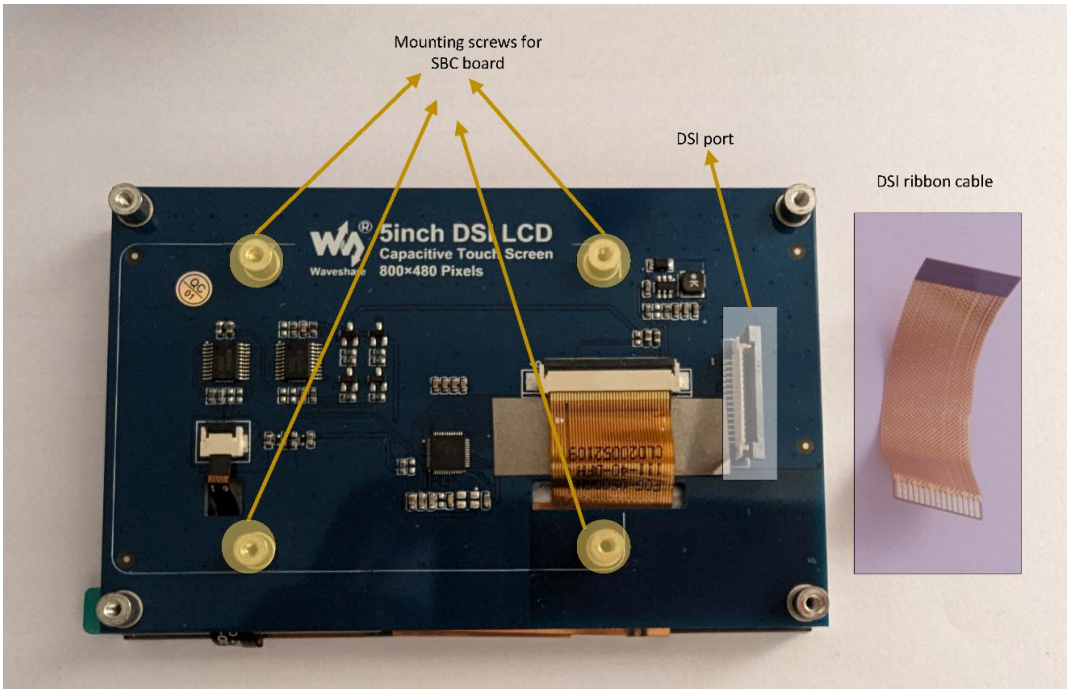


Figure 65. Waveshare 5" DSI touch panel read side picture with flat ribbon cable.

FPC connector locking and unlocking are done by pulling up the black notch or pushing it down. Unlock the connector by pulling up the notch, as shown below.

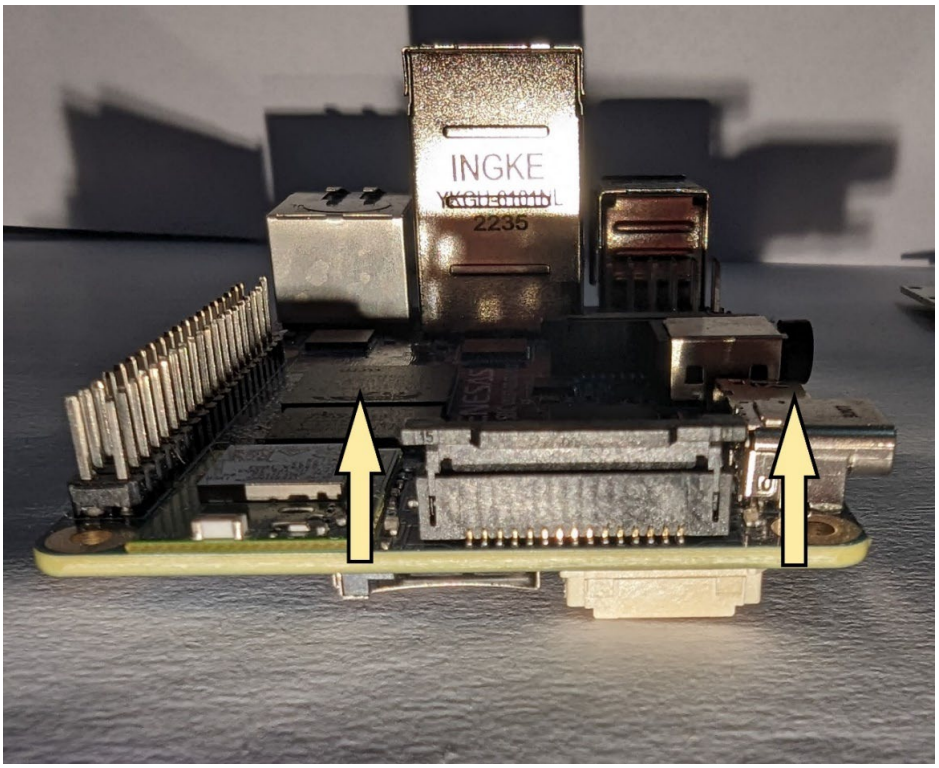


Figure 66. DSI port notch lock opens by pulling it up

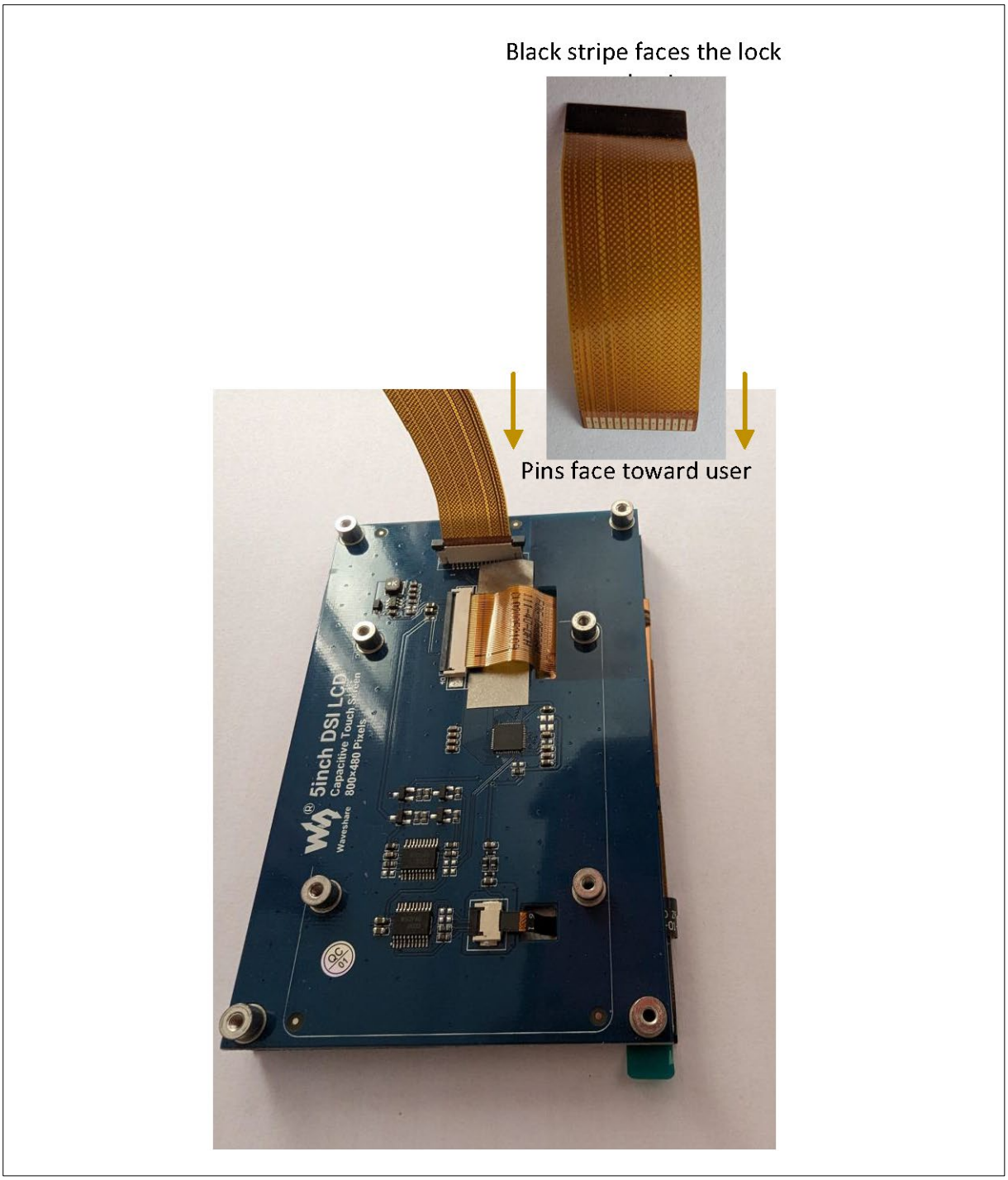


Figure 67. Waveshare DSI touch display DSI port interfacing cable orientation.

Mount the RZ/G2L-SBC onto the rear end of the display panel.

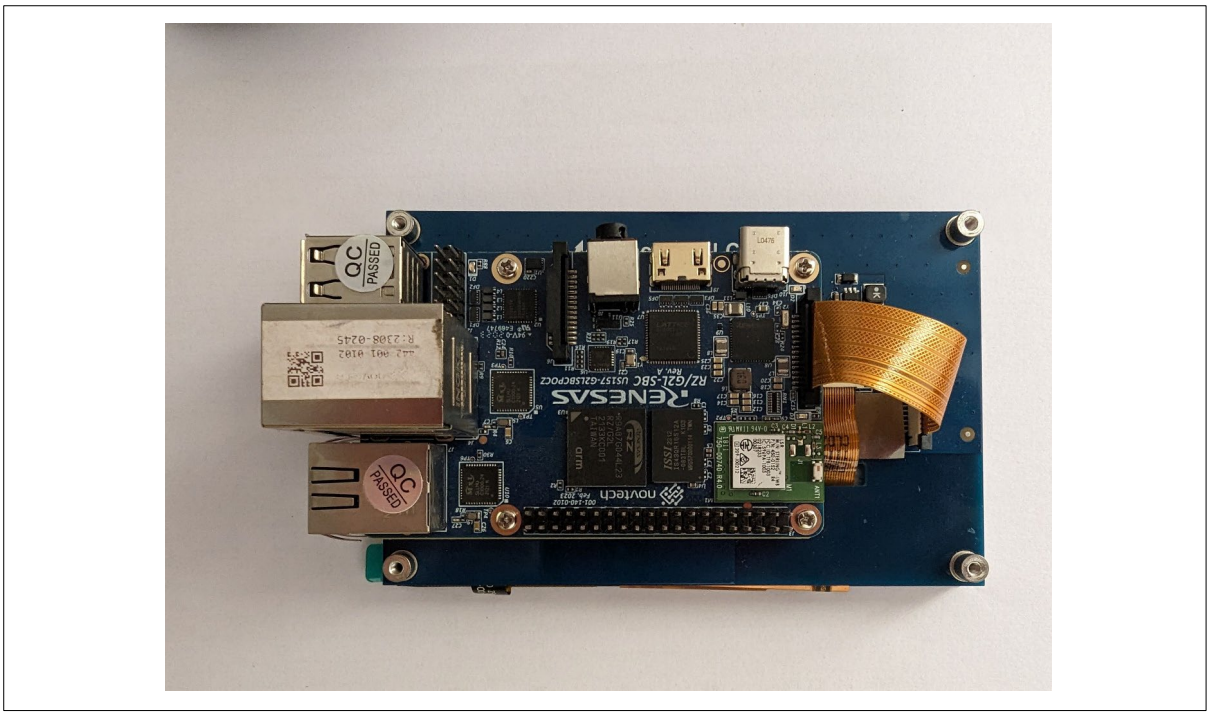


Figure 68. RZ/G2L-SBC mounted to the Waveshare DSI panel and interfaced.

Insert the other end of the FPC cable into the RZ/G2L-SBC DSI port and lock it. The locking mechanism is shown below.

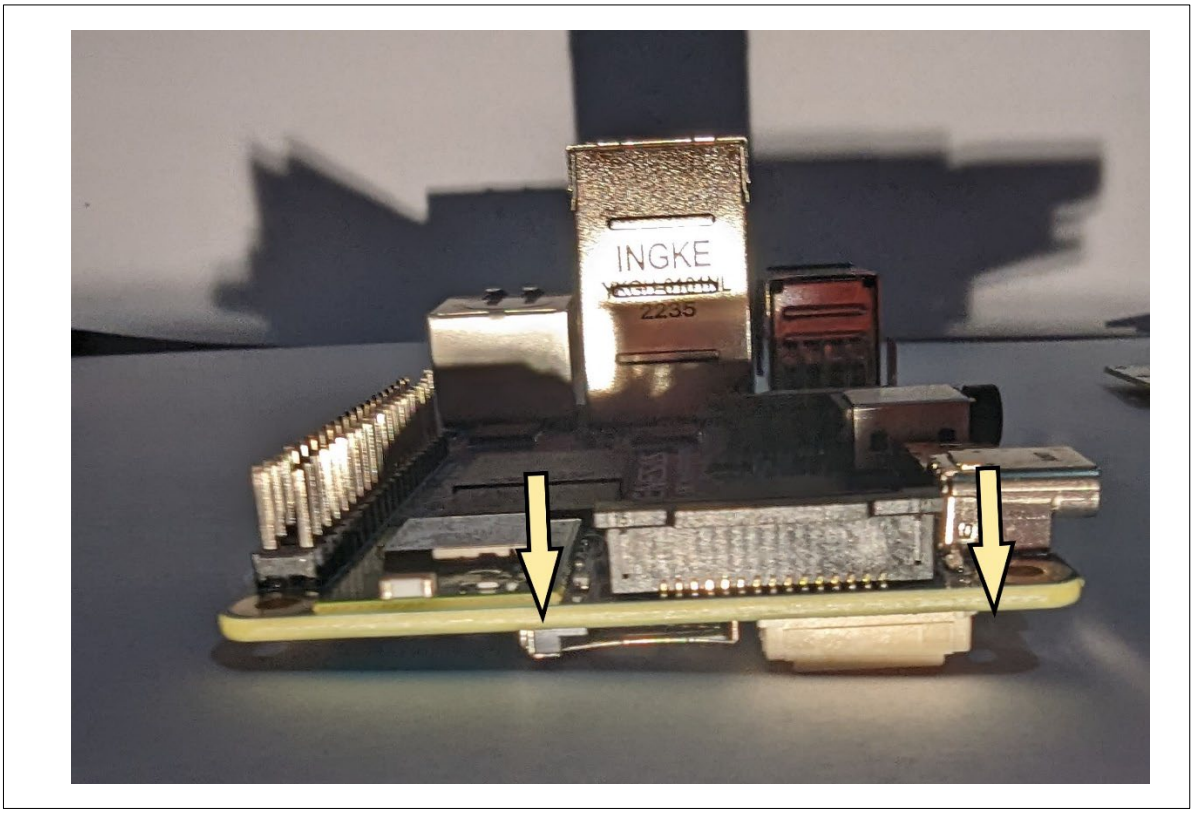


Figure 69. DSI port notch in the lock position. The cable is not shown to keep the notch in clear view.

Figure 70. Metal support screws supplied by Waveshare

The Waveshare DSI display panel comes with four metal supports that raise the display, along with the rear-attached SBC, off the surface to provide sturdy support with clearance. However, these are not high enough for the RZ/G2L-SBC due to the SBC having dual Ethernet ports, where one port is too high, sitting on top of the two USB ports. We still recommend that you use a support stand, even an off-market custom one, to ensure that the DSI cable is off the ground.

Remember that the DSI port includes an I²C two-wire interface that supports a touch panel interface without any extra cabling.

Note: The dark, solid stripe on the flat cable always faces the black locking mechanism of the connector. Do not insert the cable in reverse, as this could potentially damage the board due to incorrect electrical connections.

(2) Enabling DSI Panel Drivers

The Linux distribution supports the Waveshare 5-inch Touchscreen MIPI-DSI LCD capacitive touch panel.

By default, the video output is directed toward the mini-HDMI port. To enable the panel drivers and reroute the display to the DSI panel, you need to enable the panel driver DT overlay in uEnv.txt.

Open the `uEnv.txt` and change the following line:

```
#enable_overlay_dsi=1
```

To

```
enable_overlay_dsi=1
```

Reboot the SBC board.

Note: Enabling the MIPI DSI panel overlay disables the HDMI display. You can only use one at a time.

8.1.3.4 Accessing PWM Timers

The RZG2L-SBC provides PWM (Pulse Width Modulation) timers, which can be used for various applications, including motor control, LED dimming, and signal generation for external devices. PWM allows for precise control over voltage levels by adjusting the duty cycle, making it useful in scenarios requiring variable power output.

(1) Overview

The RZ/G2L-SBC's device tree source (DTS) includes two GPT channels by default, providing PWM functionality for three pins.

- GPT4: Supports two PWM channels (channel_A and channel_B).
- GPT5: Supports a signal PWM channel A.

However, these channels are disabled by default because the GPT4 pins are assigned to SPI, and the GPT5 pins are used for DSI. If these resources are repurposed for PWM, then the default functions (SPI or DSI) will no longer be available.

```
&gpt4 {
	pinctrl-0 = <&gpt4_pins>;
	pinctrl-names = "default";
	channel = "both_AB";
	poeg = <&poega &poegb &poegc &poegd>;
	status = "disabled";
};

&gpt5 {
	pinctrl-0 = <&gpt5_pins>;
	pinctrl-names = "default";
	channel="channel_A";
	poeg = <&poegd>;
	status = "disabled";
};
```

To enable the use of PWM, follow the steps in the next subsection:

(2) Enabling GPT Channels for PWM Use

This section explains how to enable GPT channels for PWM on the RZ/G2L-SBC. By default, the GPT channels are disabled in the device tree, so they need to be enabled manually.

Note: Ensure you have internet access before running the commands.

1. Install the device tree compiler tool.

```
root@rz-cmn:~# apt-get update
root@rz-cmn:~# apt-get install device-tree-compiler
```

2. Decompile the dtb file into a dts file.

The Device Tree Blob (DTB) is typically stored on a dedicated boot partition, which is often formatted as FAT32. This partition needs to be mounted to access its contents.

Create a temporary mount point and mount the boot partition (partition 1 – FAT32)

```
root@rz-cmn:~# mkdir -p /mnt/boot_partition
root@rz-cmn:~# mount /dev/mmcblk0p1 /mnt/boot_partition/
```

Following successful mounting, the specific DTB file can be located within `/mnt/boot_partition/``

```
root@rz-cmn:~# dtc -I dtb -O dts -f /mnt/boot_partition/dtb/renesas/rzg2l-sbc.dtb
-o rzg2l-sbc.dts
```

3. Modify the DTS file.

Open the rzg2l-sbc.dts file in a text editor.

```
root@rz-cmn:~# vi rzg2l-sbc.dts
```

For GPT4, locate gpt@10048400

For GPT5, locate gpt@10048500

Change the status property of the node you want to enable from "disabled" to "okay". Save the file after making the changes.

4. Recompile the DTS file back into a DTB file.

```
root@rz-cmn:~# dtc -I dts -O dtb -f rzg2l-sbc.dts -o new_rzg2l-sbc.dtb
```

5. Deploy the new DTB file:

Replace the original DTB file with the newly compiled one.

Note: It is recommended to back up the original DTB file beforehand. After recompiling the DTS into a DTB and deploying it to /mnt/boot_partition/dtb/renesas/rzg2l-sbc.dtb in partition 1, ensure that the file retains its original name. If the DTB file is missing or renamed, the boot process may fail.

```
root@rz-cmn:~# cp new_rzg2l-sbc.dtb /mnt/boot_partition/dtb/renesas/rzg2l-sbc.dtb
```

6. Sync and umount the partition

```
root@rz-cmn:~# cd ~
root@rz-cmn:~# sync
root@rz-cmn:~# umount /mnt/boot_partition
root@rz-cmn:~# sync
```

7. Reboot the system to apply the changes.

After booting up, if everything is configured correctly, the PWM device file will be automatically generated in /sys/class/pwm/pwmchipX, where X can be 0, 1, 2, and so on.

(3) Enable PWM channels

Before using PWM, the channels need to be exported to the system.

For example, to use PWM chip 0 and export channel 0, the following steps are required.

```
root@rz-cmn:~# cd /sys/class/pwm/pwmchip0/
root@rz-cmn:/sys/class/pwm/pwmchip0# echo 0 > export
```

(4) Configuring PWM

To configure a single PWM channel (For example, from GPT5), follow these steps:

In this example, the period is set to 1,000,000 nanoseconds, and the duty cycle is configured to 500,000 nanoseconds, which is 50% of the period. Adjust these values as needed to achieve the desired PWM output.

1. Modify the duty cycle and period.

Set the period (in nanoseconds).

```
root@rz-cmn:/sys/class/pwm/pwmchip0/# cd pwm0
root@rz-cmn:/sys/class/pwm/pwmchip0/pwm0# echo 1000000 > period
```

Set the duty cycle (in nanoseconds).

```
root@rz-cmn:/sys/class/pwm/pwmchip0/pwm0# echo 500000 > duty_cycle
```

2. Enable the PWM to start output.

```
root@rz-cmn:/sys/class/pwm/pwmchip0/pwm0# echo 1 > enable
```

For devices like GPT4 that provide two PWM channels (channel A and channel B), each channel needs to be configured separately.

1. Modify the period.

Define the period for both channels in nanoseconds. For example, to set the period to 100,000 nanoseconds, use the following command:

```
root@rz-cmn:/sys/class/pwm/pwmchip0/pwm0# echo 1000000 > period
```

2. Enable the PWM to start output

```
root@rz-cmn:/sys/class/pwm/pwmchip0/pwm0# echo 1 > enable
```

3. Modify the duty cycle for each channel.

Navigate to the device directory to configure the duty cycles for both channels.

```
root@rz-cmn:/sys/class/pwm/pwmchip0/pwm0# cd /sys/class/pwm/pwmchip0/device
root@rz-cmn:/sys/class/pwm/pwmchip0/device# echo 1000000 > buffA0
root@rz-cmn:/sys/class/pwm/pwmchip0/device# echo 500000 > buffB0
```

In this example, channel A is set to a duty cycle of 1,000,000 nanoseconds, while channel B is set to 500,000 nanoseconds. Adjust these values as needed for the desired PWM output.

8.1.4 IMDT V2H-SBC

8.1.4.1 USB

The IMDT V2H-SBC provides the USB connectors/ports listed below. The table summarizes each port and its supported operating mode.

Table 39. Supported USB features (IMDT V2H-SBC)

Connector	Feature
Type A USB2	Supported (Host)
Micro USB (USB2)	Supported (OTG)
M2 Key B USB (USB3)	Supported (Host)
Type A USB3 (J8)	Supported (Host)

Note: Automatic USB gadget enumeration is not enabled by default. The OTG port can switch the hardware role (host/device), but the system still must explicitly configure which USB gadget function to expose (for example, CDC ACM serial, Ethernet/RNDIS, or mass storage) and bind it to the USB Device Controller (UDC). The provided script performs this configuration via ConfigFS and activates the gadget by binding it to the UDC.

USB gadget (CDC ACM) setup script:

Save the following as usb_gadget_acm.sh on the target and run it as root:

```
#!/bin/sh
set -e

# Mount ConfigFS if needed
if ! mount | grep -q '/sys/kernel/config type configfs'; then
  mount -t configfs none /sys/kernel/config
fi

# Load composite gadget framework (if built as a module)
modprobe libcomposite 2>/dev/null || true

G=/sys/kernel/config/usb_gadget/g1
mkdir -p "$G"
cd "$G"

# Basic USB device parameters
echo 0x1d6b > idVendor
echo 0x0101 > idProduct
echo 0x0200 > bcdDevice
echo 0x0200 > bcdUSB

# Optional identification strings
mkdir -p strings/0x409
echo "IMDT" > strings/0x409/manufacturer
echo "CDC ACM Gadget" > strings/0x409/product
echo "0001" > strings/0x409/serialnumber

# Create CDC ACM (USB serial) function
mkdir -p functions/acm.usb0

# Create a configuration and link the function
mkdir -p configs/c.1/strings/0x409
echo "ACM" > configs/c.1/strings/0x409/configuration
echo 120 > configs/c.1/MaxPower
ln -sf functions/acm.usb0 configs/c.1/

# Bind to the USB Device Controller (UDC)
UDC_NAME="${1:-15820000.usb}"
echo "$UDC_NAME" > UDC
```

Usage:

```
root@rz-cmn:~# chmod +x usb_gadget_acm.sh
root@rz-cmn:~# ./usb_gadget_acm.sh
```

Verification:

- Target: *cat /sys/kernel/config/usb_gadget/g1/UDC* should output 15820000.usb
- Host (Linux): a new /dev/ttyACM* device should appear after connecting the OTG port

8.1.4.2 Wi-Fi 802.11 module

The IMDT V2H-SBC includes an integrated Wi-Fi module supporting 802.11 b/g/n. Wi-Fi is managed by systemd-networkd. Helper scripts are available under /opt/imdt/wifi to simplify common operations such as switching modes and connecting to an access point.

(1) Overview of Wi-Fi operating modes

The Wi-Fi interface can be used in the following modes:

- STA (Station) mode: The SBC connects to an existing Wi-Fi network (for example, a home/office router). Use this mode when the SBC needs LAN/Internet connectivity through an access point.
- AP (Access Point) mode: The SBC creates its own Wi-Fi network (SSID). A laptop or phone can connect directly to the SBC. Use this mode for local setup or when no external Wi-Fi network is available.
- OFF: The Wi-Fi interface is disabled.

(2) Getting the current Wi-Fi operating mode

Use get-wifi-mode.sh to check the current mode. Supported modes are AP, STA, or OFF.

```
root@rz-cmn:~# /opt/imdt/wifi/get-wifi-mode.sh
```

Example output:

```
OFF
```

(3) Station mode (STA): scan and connect to a Wi-Fi network

Step 1: Switch the Wi-Fi interface to Station (STA) mode:

```
root@rz-cmn:~# /opt/imdt/wifi/connect-to-access-point.sh
```

Step 2: Scan for available Wi-Fi networks:

```
root@rz-cmn:~# iwlist wlan0 scan | grep ESSID
```

Example output:

```
ESSID:"RZ_Device"
```

Step 3: Connect to a Wi-Fi network:

Replace <ssid> and <passphrase> with the target network credentials:

```
root@rz-cmn:~# /opt/imdt/wifi/connect-to-access-point.sh <ssid> <passphrase>
```

(4) Access Point (AP) mode

By default, AP mode uses:

- SSID: imdt-v2h-sbc-ap
- Password: imdtpass

To enable AP mode with the default SSID/password as above. Run the following command:

```
root@rz-cmn:~# /opt/imdt/wifi/create-access-point.sh
```

To enable AP mode with a custom SSID and password. Run the following command:

```
root@rz-cmn:~# /opt/imdt/wifi/create-access-point.sh <ssid> <passphrase>
```

Get the number of connected clients, run:

```
root@rz-cmn:~# /opt/imdt/wifi/get-number-of-ap-clients.sh
```

(5) Disable/re-enable Wi-Fi

To disable Wi-Fi, run the following command:

```
root@rz-cmn:~# /opt/imdt/wifi/disable-wifi.sh
```

To re-enable Wi-Fi

Use either Station mode or AP mode script. If <ssid> / <passphrase> are omitted, the last saved configuration is used.

```
root@rz-cmn:~# /opt/imdt/wifi/connect-to-access-point.sh
```

or

```
root@rz-cmn:~# /opt/imdt/wifi/create-access-point.sh
```

Confirm status again

```
root@rz-cmn:~# /opt/imdt/wifi/get-connection-status.sh
```

8.1.4.3 CAN / CAN FD

The IMDT V2H SBC provides a CAN interface with CAN FD capability. The CAN signals are available on connector J10 (see the IMDT board silkscreen / pinout diagram).

(1) Bring up the CAN interface

By default, the script configures the CAN bitrate to 250 kbit/s. If a different bitrate is required, update the script accordingly.

```
root@rz-cmn:~# /opt/imdt/can/start-can.sh
```

(2) Monitor CAN traffic

To display received frames:

```
root@rz-cmn:~# candump can0
```

(3) Send a CAN frame

Example transmit command:

```
root@rz-cmn:~# cansend can0 123#DEADBEEF
```

(4) Bring down the CAN interface

To bring down the CAN interface, run the following command:

```
root@rz-cmn:~# /opt/imdt/can/stop-can.sh
```

8.1.4.4 Display (and HDMI)

The IMDT V2H-SBC provides a MIPI-DSI connector that supports two display output options:

- ST DSI-to-HDMI Adapter — B-LCDAD-HDMI1 (default)
- IMDT 7" DSI Touchscreen Display

The B-LCDAD-HDMI1 is an ST adapter board that provides a DSI input port and HDMI output port, allowing the board's DSI interface to drive a standard HDMI monitor. By default, the board is configured to output video through this adapter.

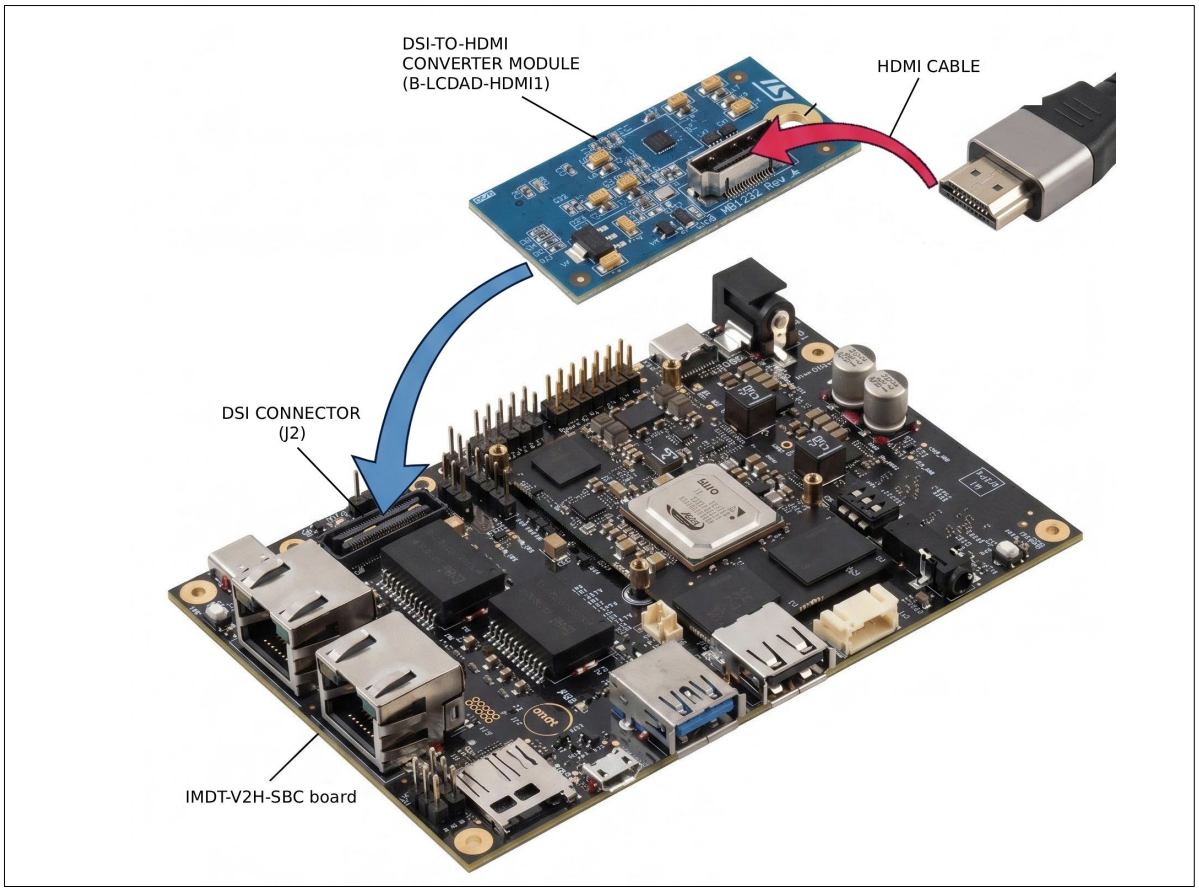


Figure 71. Use DSI-to-HDMI Adapter on IMDT-V2H-SBC

To use the IMDT 7" touchscreen display instead of the HDMI adapter, the DSI display overlay must be enabled by editing uEnv.txt in partition 1 (FAT32) and uncomment the following line:

```
#enable_overlay_dsi=1
```

or it can be set via U-Boot console

```
=> setenv enable_overlay_dsi 1
```

Note: Enabling the DSI display overlay will disable HDMI output via the B-LCDAD-HDMI1 adapter. Only one display output can be active at a time. To revert to HDMI output, comment out or remove enable_overlay_dsi=1 in uEnv.txt, or clear the variable in U-Boot and save the environment.

8.2 Supported Features in Ubuntu Images

Before accessing the features available in both the Ubuntu Core and Ubuntu LXDE images on the supported platforms, please log in using the default credentials:

- **Username:** rz
- **Password:** 1

After logging in, the supported features can be explored and interacted with, as detailed below.

8.2.1 Accessing Supported Features in Ubuntu LXDE

8.2.1.1 Selecting the LXDE session

The LXDE desktop environment is enabled by default:

1. On first login, the system automatically launches LXDE as the desktop environment.
2. No manual selection is required, providing a seamless user experience.

If you wish to use a different desktop environment, click the gear icon in the bottom-right corner of the login screen and choose an alternative. However, note that this may result in a different experience from LXDE's lightweight and responsive interface.

This default configuration ensures that users can immediately take advantage of LXDE's performance and simplicity without requiring additional setup.

8.2.1.2 Audacity

Audacity is a free, open-source, cross-platform audio software that is used for recording, editing, and producing audio. It allows users to capture live audio, convert tapes and records into digital recordings, and edit audio files in a variety of formats. Audacity is widely used for tasks such as podcasting, music production, and audio analysis due to its user-friendly interface and powerful editing tools. It supports multi-track editing, numerous audio effects, and plugins, making it a popular choice for both amateurs and professionals.

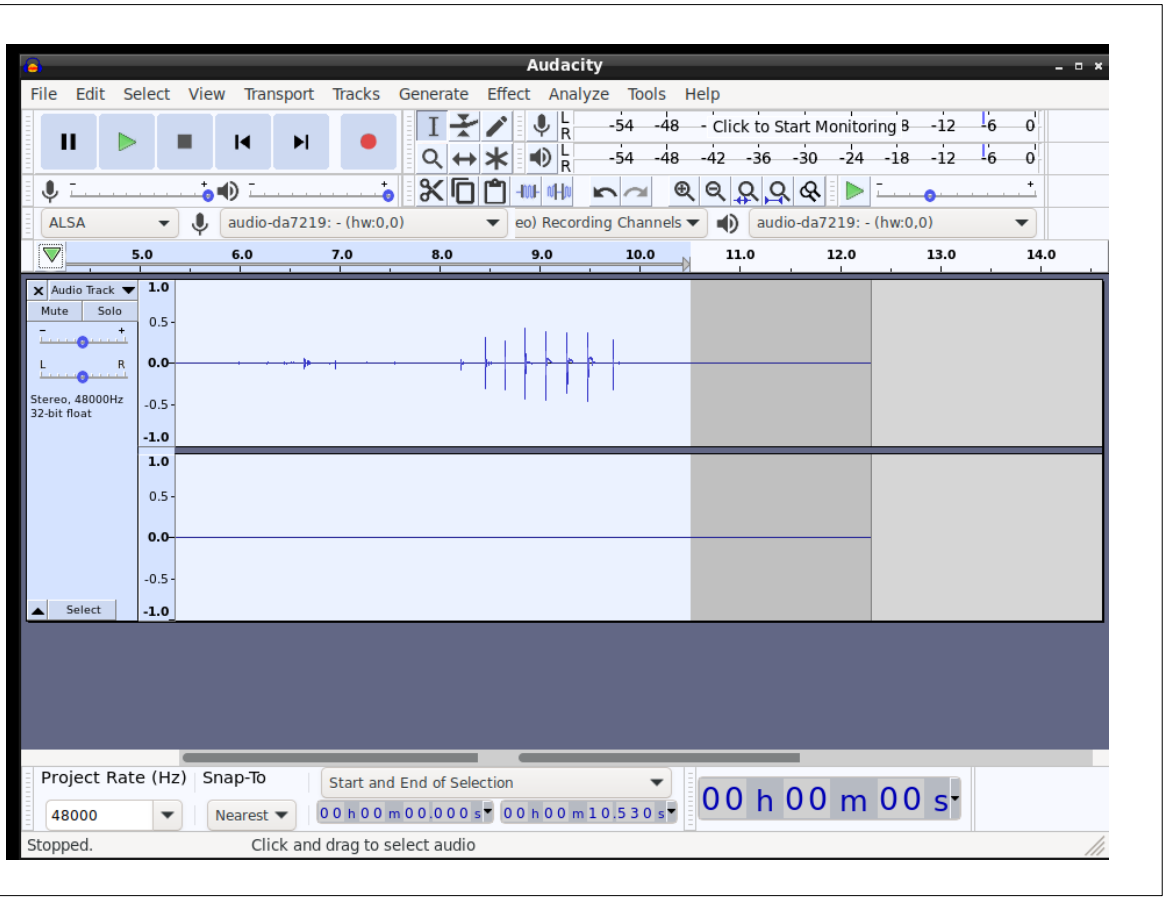


Figure 72. Audacity graphic user interface

To properly configure Audacity for the system:

1. Open Audio Settings: In Audacity, click Audio Setup in the top-right corner, then select Audio Settings.
2. Select the correct playback and recording devices: In the window that opens, set both the Playback and Recording Device to the appropriate sound card for the current board in use (e.g., for the RZG2L-SBC, select the corresponding device).

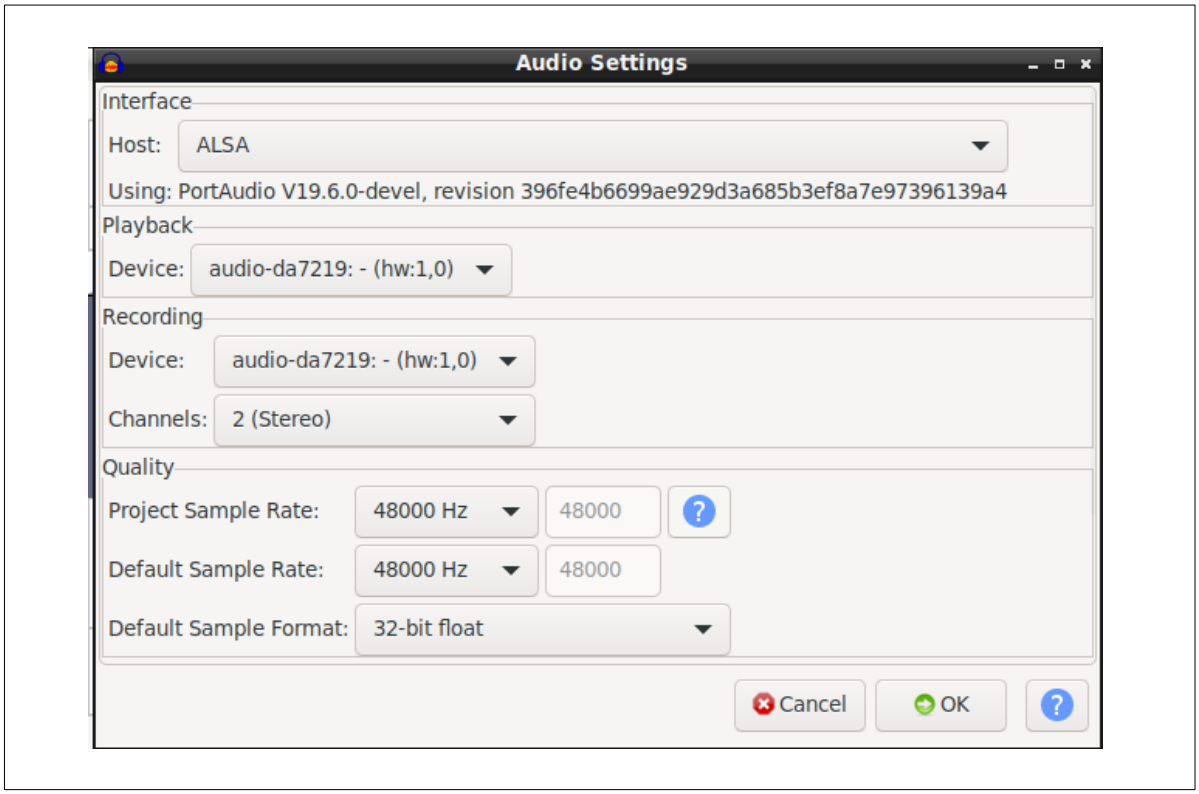


Figure 73. Audacity - Audio Settings

3. Set the sample rate: At the bottom left of the main Audacity window, set both the Project Sample Rate and Default Sample Rate to 48000 to match the hardware requirements.
4. Increase buffer length if audio problems occur: If audio issues such as glitches, dropouts, or latency are encountered:
   - Open Edit → Preferences.
   - Increase the Buffer Length to a value greater than the default 100 ms (for example, 10000 ms is recommended).
   - A larger buffer allows more time for the system to process audio data, which can improve performance on embedded platforms, systems under high CPU load, or when using less optimized audio drivers.

Click OK to save the settings. Then, click the red circle button to begin recording.

To export the recording as an MP3, follow the steps outlined in the images below.

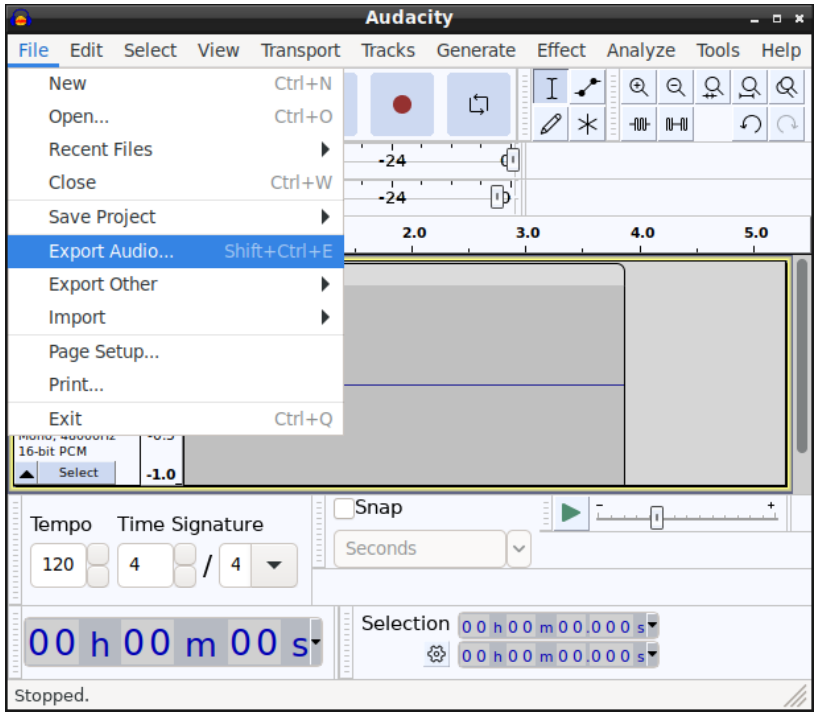


Figure 74. Audacity export video as MP3

Then, metadata can be filled in for the audio as follows:

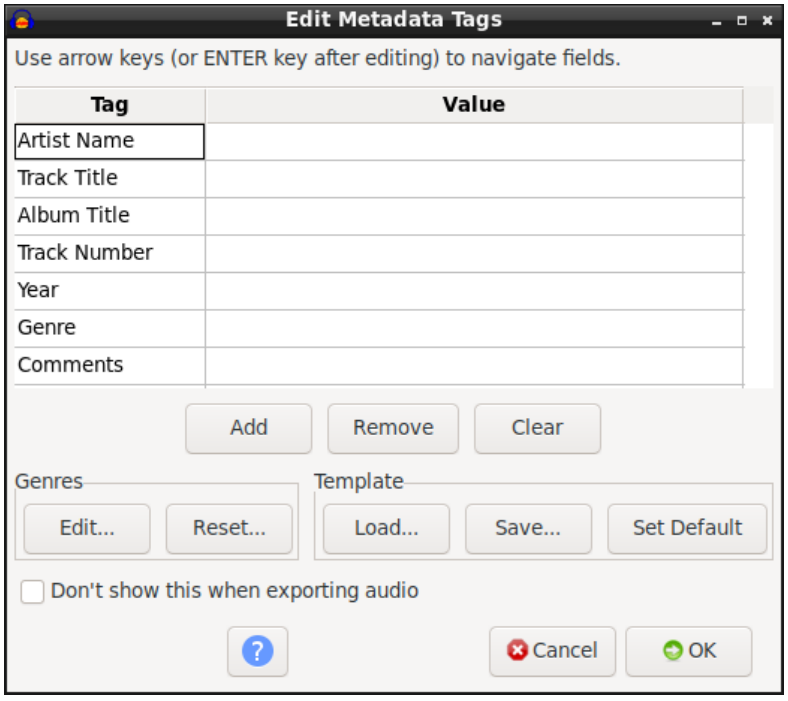


Figure 75. Audacity metadata tags to fill for the audio

Select OK to finish editing the metadata tags.

Once the audio file is edited, it can be renamed (e.g., song.mp3). Then, choose the desired directory and click Save to store the file.

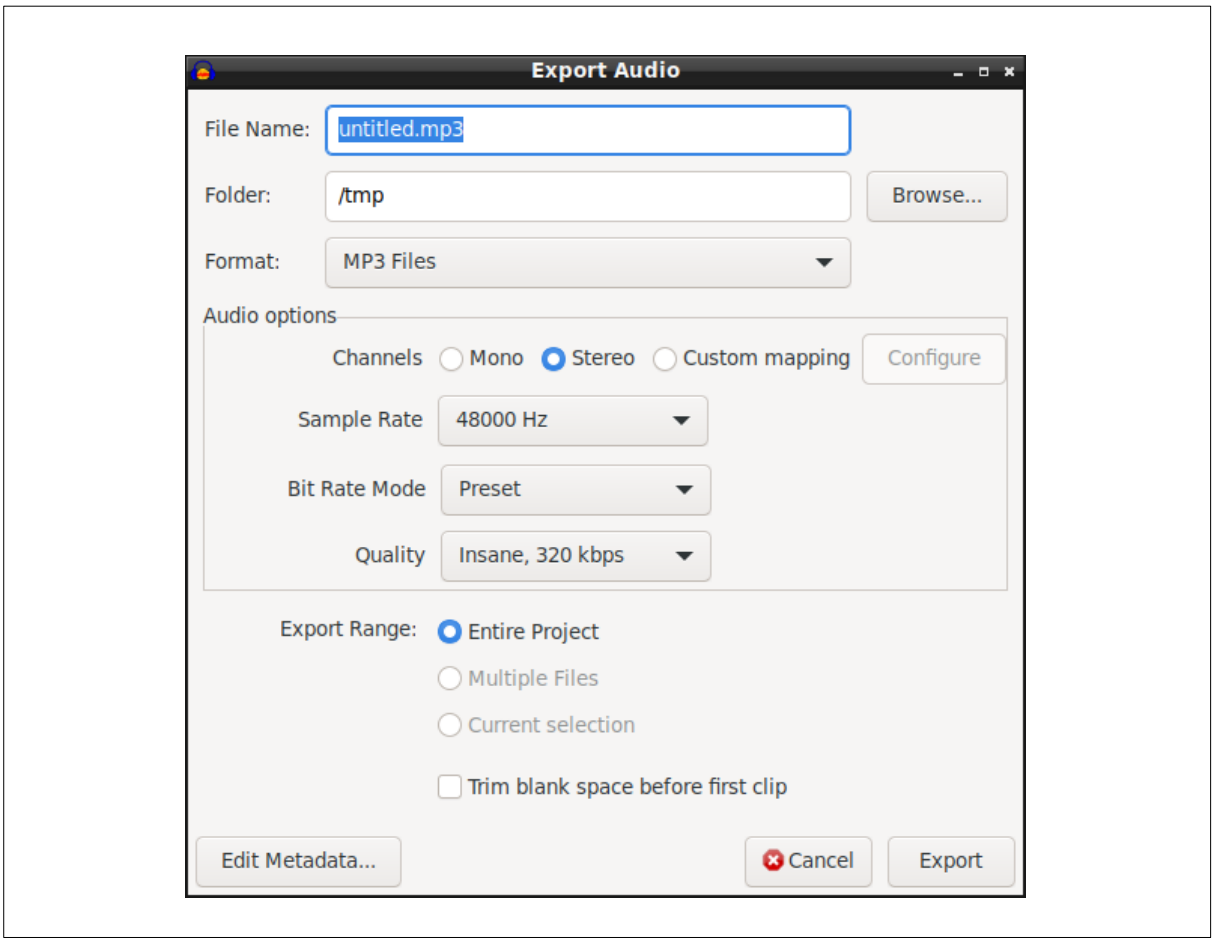


Figure 76. Saving the recorded audio

8.2.1.3 VLC Media Player

VLC Media Player is a free and open-source multimedia player that supports a wide range of audio and video formats. To play music, simply open VLC and follow these steps:

1. Launch VLC Media Player

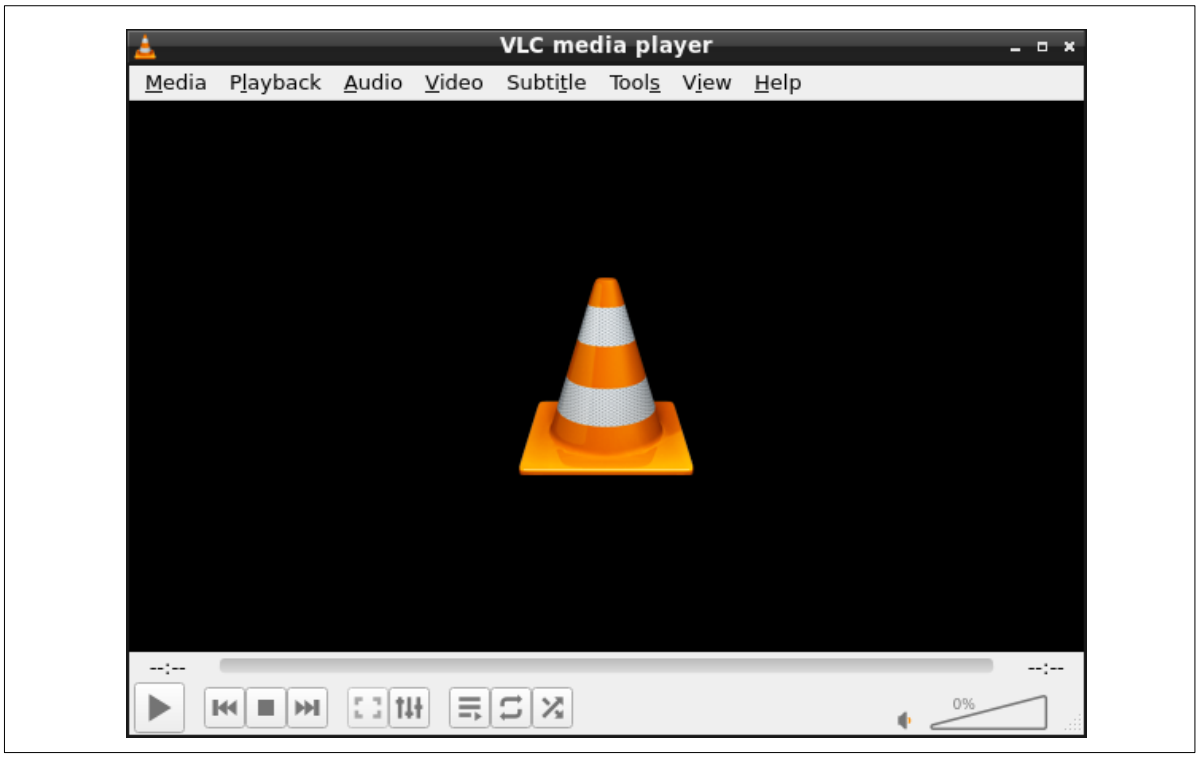


Figure 77. VLC Media Player

2. Click on “Media” in the top Menu, then select “Open File”

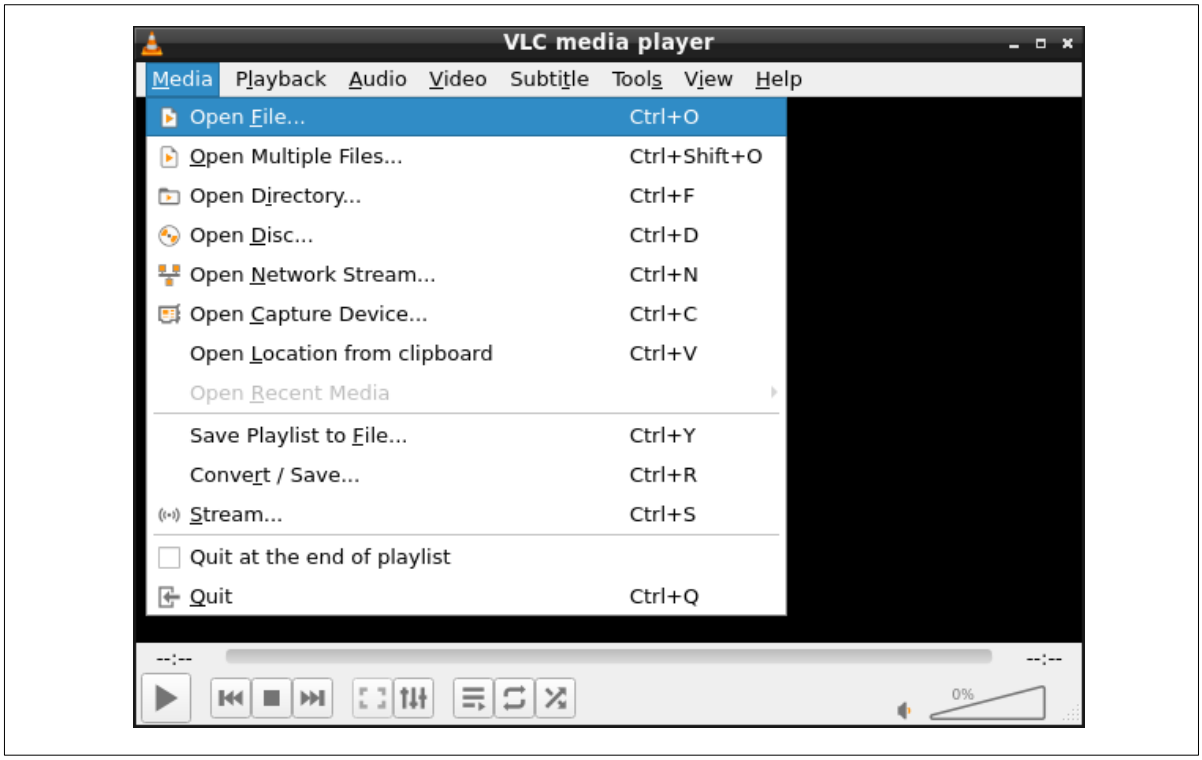


Figure 78. Open the file in VLC Media Player

3. Browse to the location of the MP3/MP4 file, select it, and click Open to start playing.

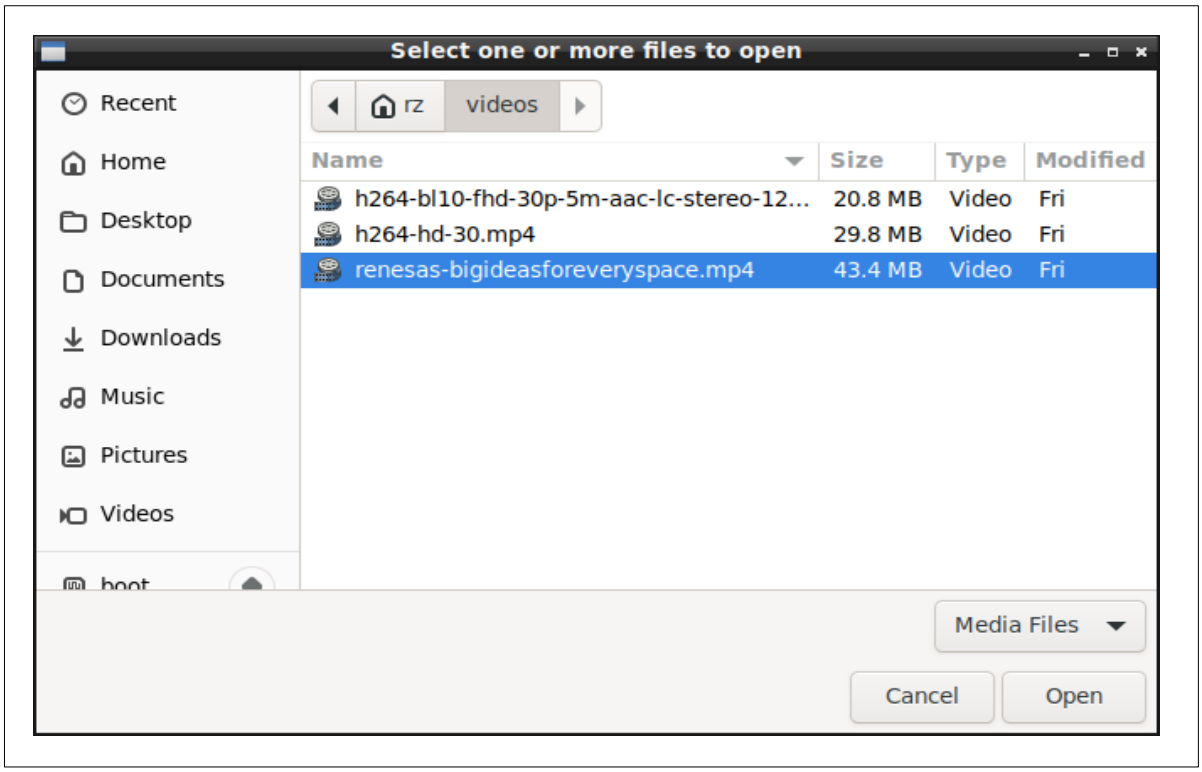


Figure 79. Select an MP3/MP4 file to play in VLC Media Player

4. Now, the media can be played using VLC.

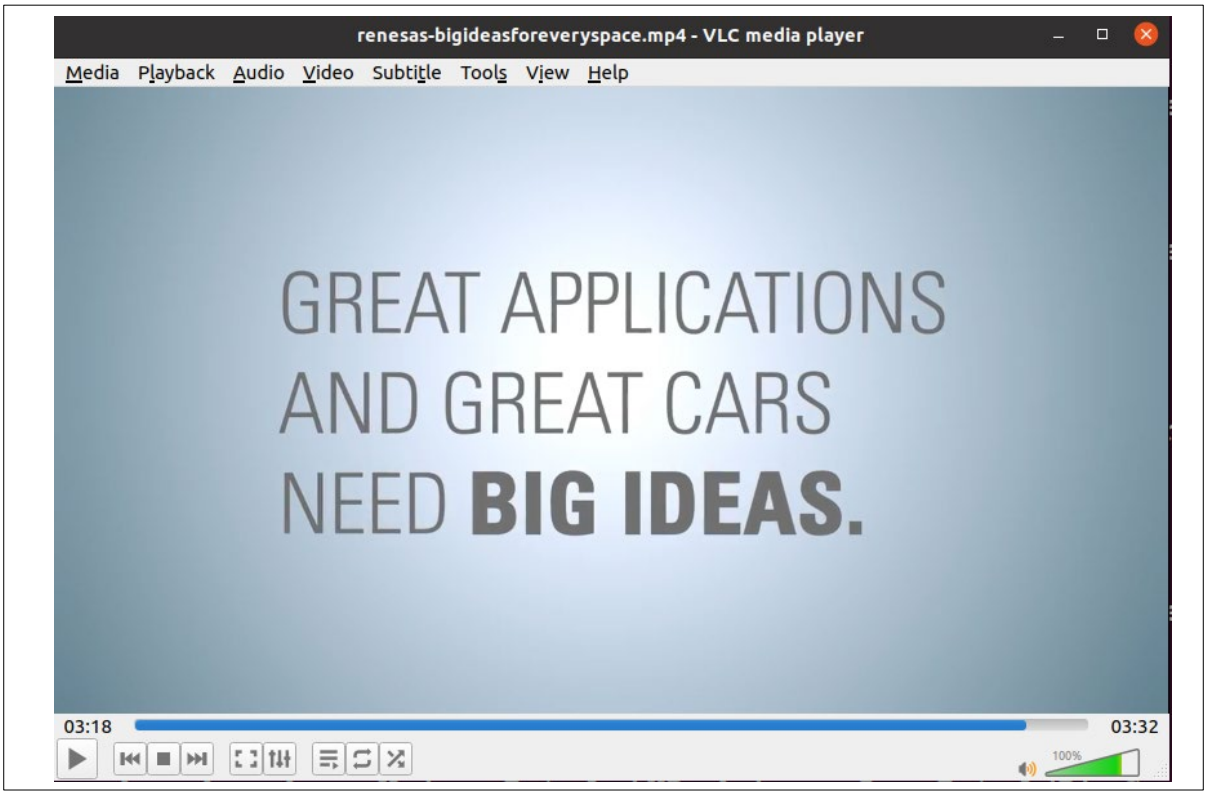


Figure 80. Video playback in VLC Media Player

8.2.1.4 Using the CSI Camera with VLC

CSI (Camera Serial Interface) is an interface standard used to connect cameras to a device, commonly used in embedded systems like Raspberry Pi and other single-board computers. It allows for high-speed data transfer between the camera and the system, enabling the capture of high-quality video and images.

You can use VLC Media Player to capture and view live videos from a CSI camera. Here's how you can do it:

1. Connect the Camera: Make sure your CSI camera is connected to the CSI port on your device.
2. Open VLC Media Player:
    - Launch VLC from the application menu.

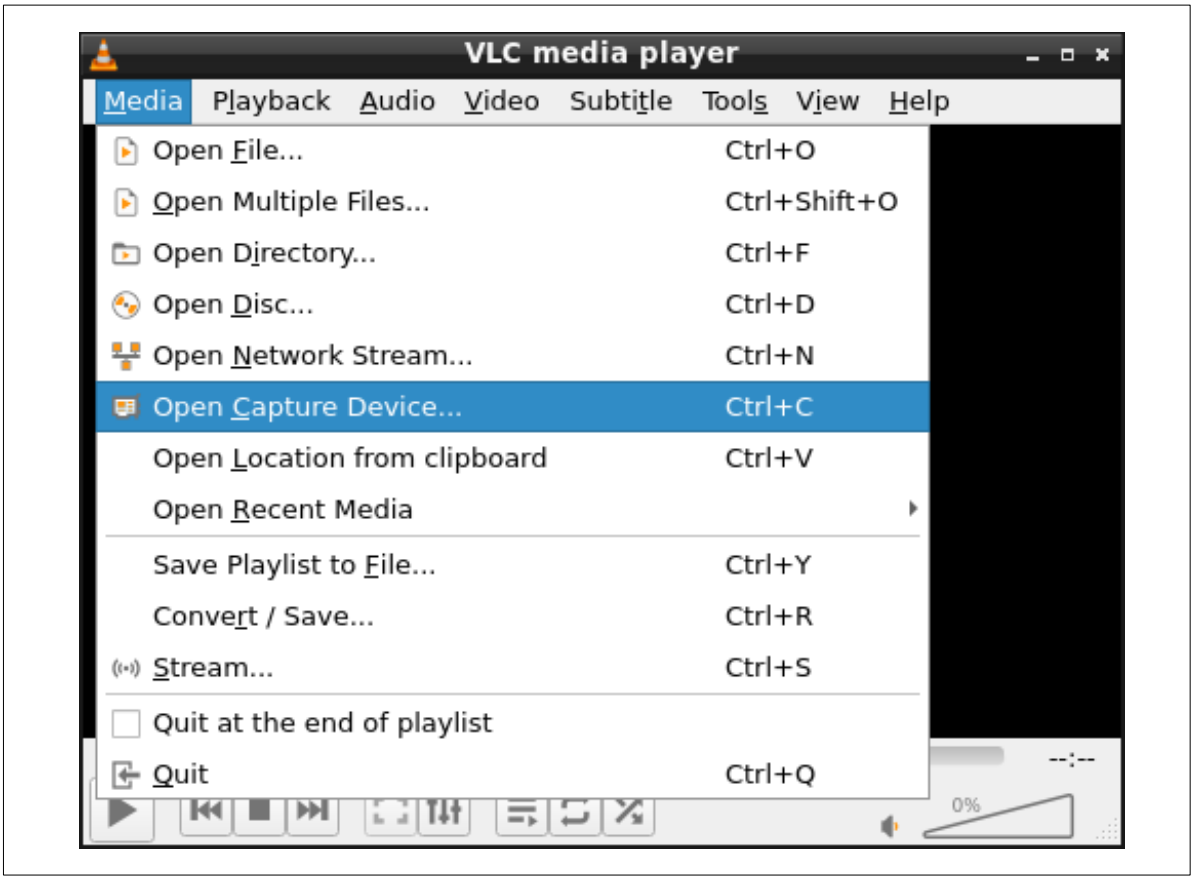


Figure 81. Capture Device in VLC Media Player

3. Open Capture Device:

    - In VLC, click on the Media menu and select Open Capture Device....
    - In the Capture Device tab, choose the Video device name that corresponds to your CSI camera (it might be listed as /dev/video or something similar).

4. Configure the Capture Settings:

    - Choose the desired video format (e.g., MJPEG or YUY2) and resolution (e.g., 640x480, 1280x720) based on your camera capabilities.

5. Click Play:

- Once you've selected the correct capture device and settings, click Play to start viewing the live video feed from your CSI camera.

Live video from the CSI camera should now be visible in VLC.

8.2.1.5 Web Browser

Ubuntu LXDE comes with a default web browser pre-installed. This browser provides essential features for browsing the internet and is lightweight, making it suitable for low-resource systems.

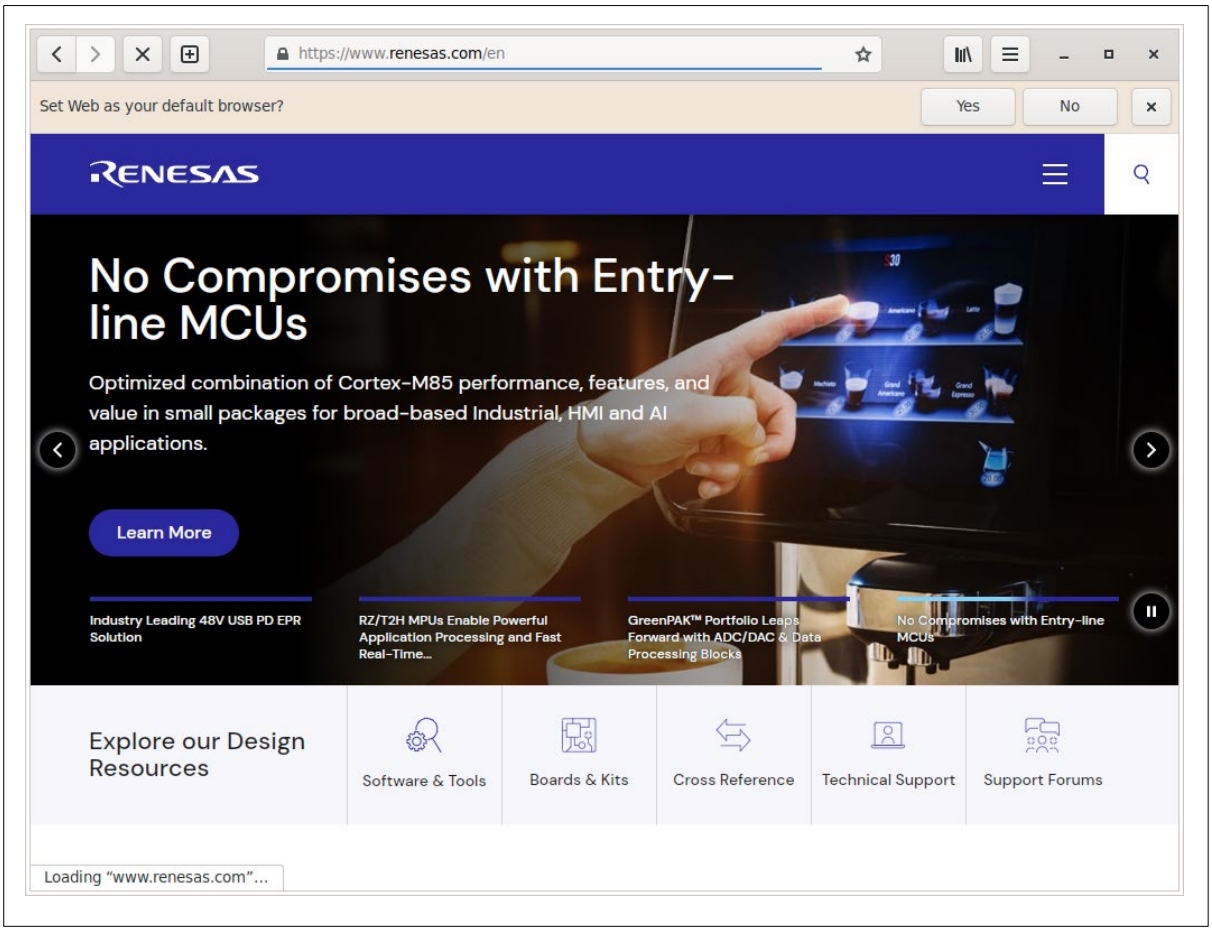


Figure 82. Ubuntu LXDE web browser pre-installed

8.2.1.6 LXTerminal

LXTerminal is a VTE-based terminal emulator with support for multiple tabs. It is completely desktop-independent and does not have any unnecessary dependencies. To reduce memory usage and improve performance, all terminal instances share a single process.

Features:

- Lightweight and fast terminal emulator.
- Supports multiple tabs.
- Desktop-independent, reducing resource consumption.
- Optimized for performance with a single shared process for all instances.

Example Usage: Monitor Swap Memory with htop while browsing renesas.com

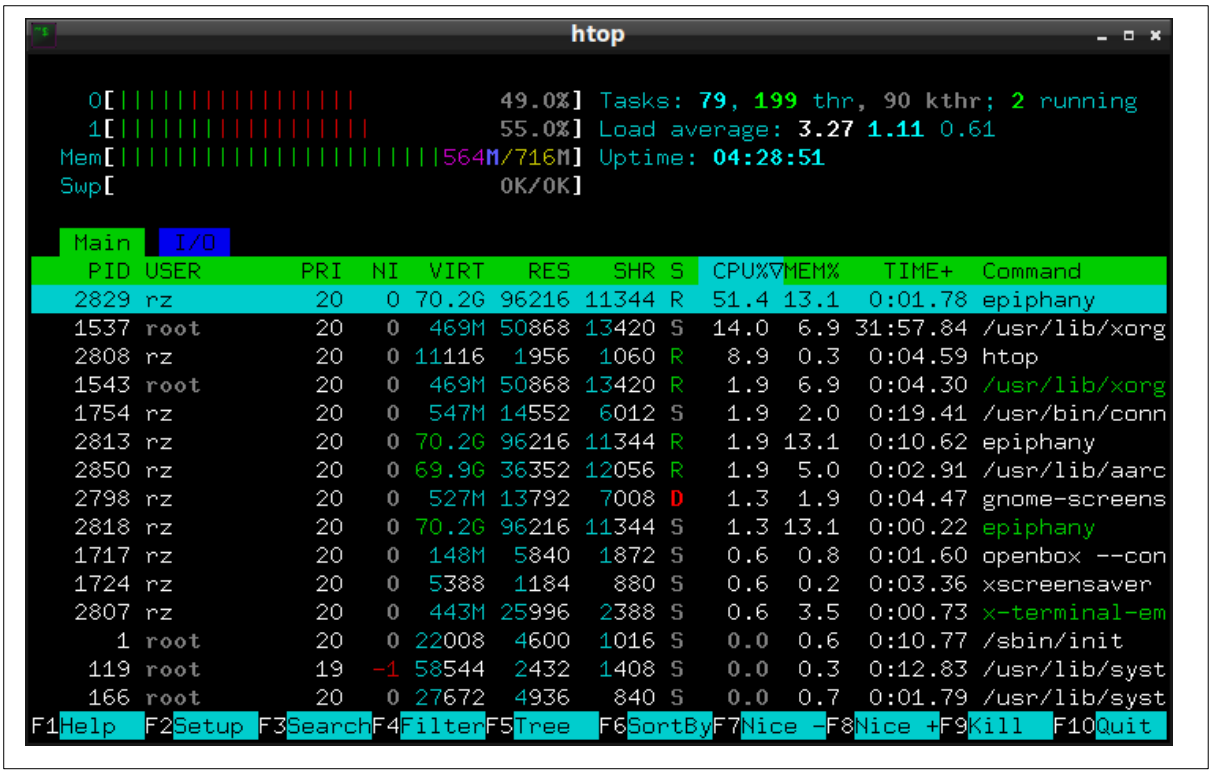


Figure 83. htop in Ubuntu-LXDE

8.2.1.7 Ethernet

Ethernet, also known as a wired network, is a widely used method to connect a device to a Local Area Network (LAN) or the internet using physical cables. On Ubuntu LXDE, connecting to an Ethernet network can be easily done through the Network Manager, a powerful and user-friendly network management tool.

Follow these simple steps to connect to an Ethernet network using the Network Manager UI:

1. Open the Network Manager:
   - At the bottom-right corner of the screen, click on the network icon, then choose Edit connection....

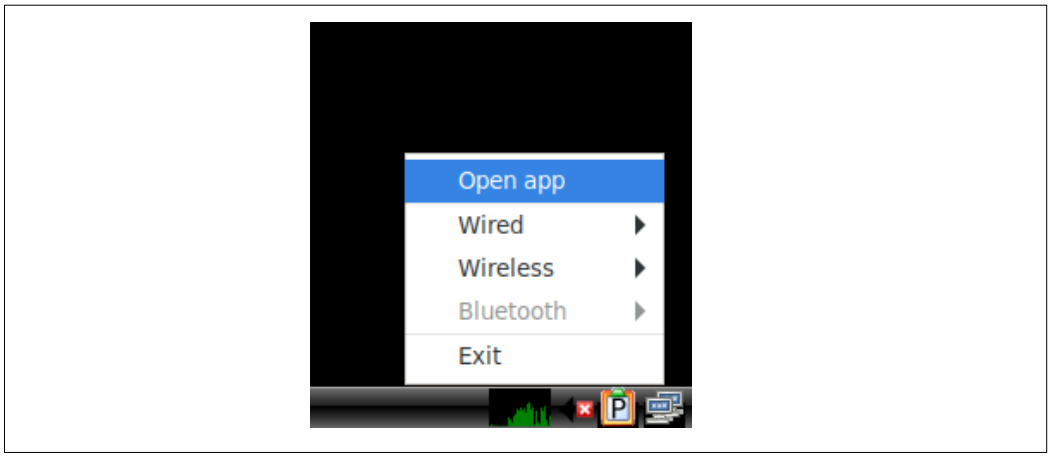


Figure 84. Bluetooth icon Ubuntu-LXDE taskbar

2. Choose Your Ethernet Network:
   - In the Network Manager menu, you should see Wired Networks listed. Simply click on your Ethernet connection or manually configure it as described below (if not automatically connected).

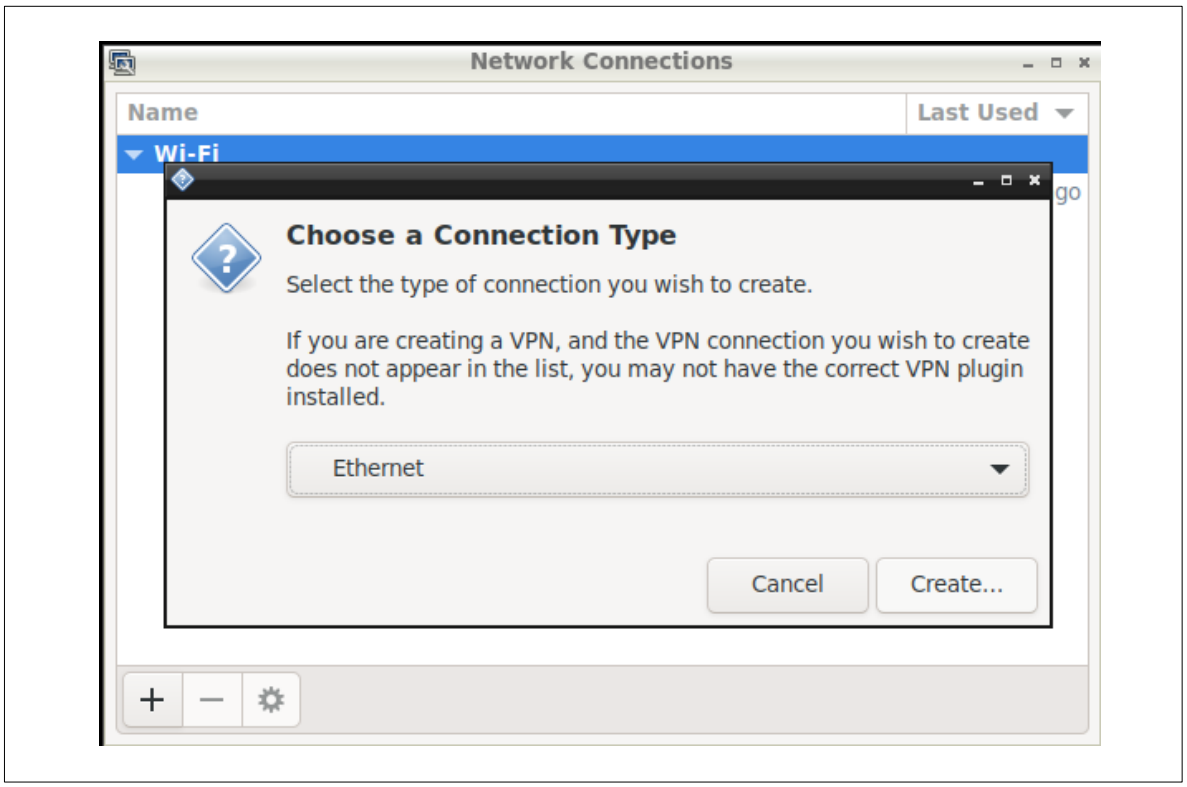


Figure 85. Choose Network Connections

3. Configure the Connection:
   - If the connection is not automatically established, you can configure network settings such as IP addresses, DNS servers, etc.

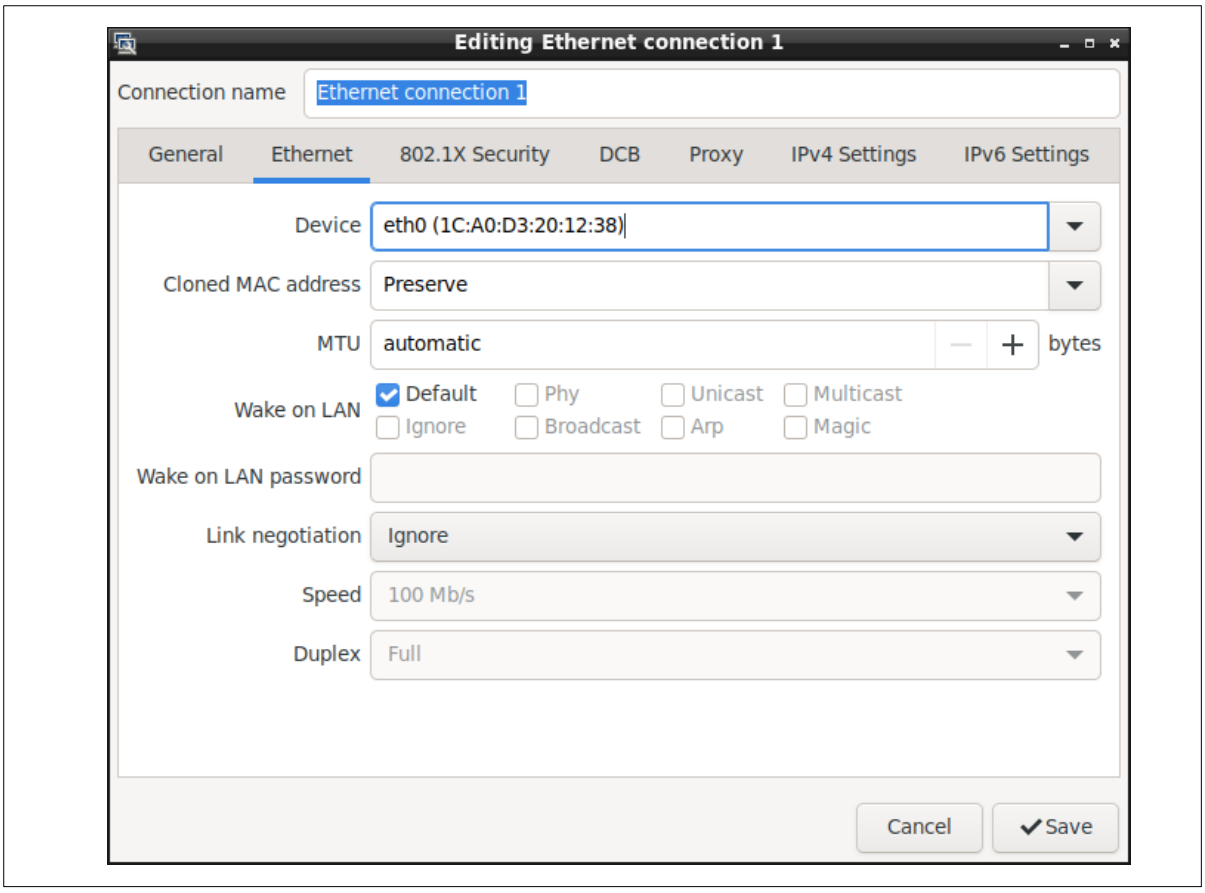


Figure 86. Edit Ethernet connection 1

4. Connect: Once the connection settings are confirmed, the Ethernet connection should be ready for use. The network icon will update to indicate a successful connection.

8.2.1.8 Wi-Fi Network

Ubuntu LXDE provides an easy way to connect to Wi-Fi networks. Follow these simple steps to get connected:

1. Click on the Network Icon: In the lower-right corner of the screen, you will find the network icon. Click on this icon.
2. Choose Your Wi-Fi Network: A list of available Wi-Fi networks will appear. Find and click on your desired Wi-Fi network from the list.

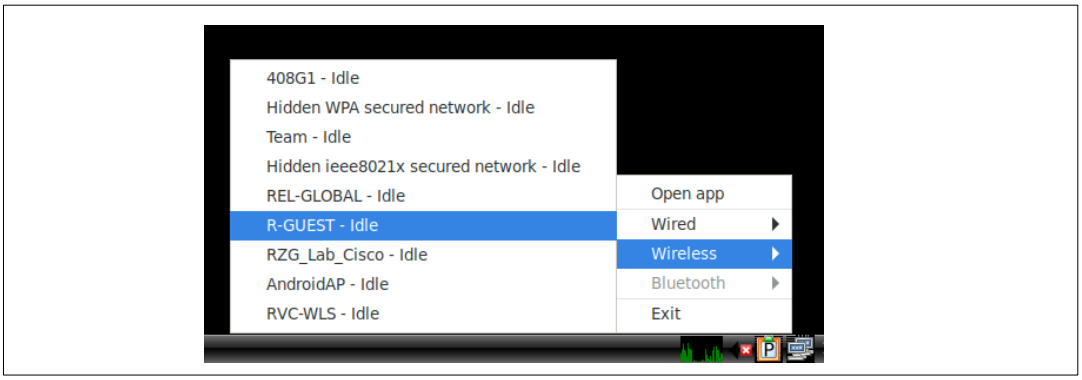


Figure 87. Wi-Fi selection in Ubuntu-LXDE

3. Enter the Password: After selecting the network, a prompt will appear asking for the Wi-Fi password. Type in the password and click Connect.
4. Connected: Once the password is verified, your system will be connected to the Wi-Fi network.

8.2.1.9 PCIe Wi-Fi Module

Wi-Fi can be provided via a PCIe Wi-Fi module. Follow the instructions in this section **PCIe Wi-Fi Module** to configure or set up the Wi-Fi module

8.2.1.10 Bluetooth

Ubuntu LXDE provides an easy way to connect to Bluetooth devices. Follow these simple steps to get connected:

1. Open Bluetooth Manager: Click the LXDE icon in the lower-left corner of the screen, go to Preferences, and select Bluetooth Manager to access Bluetooth settings.

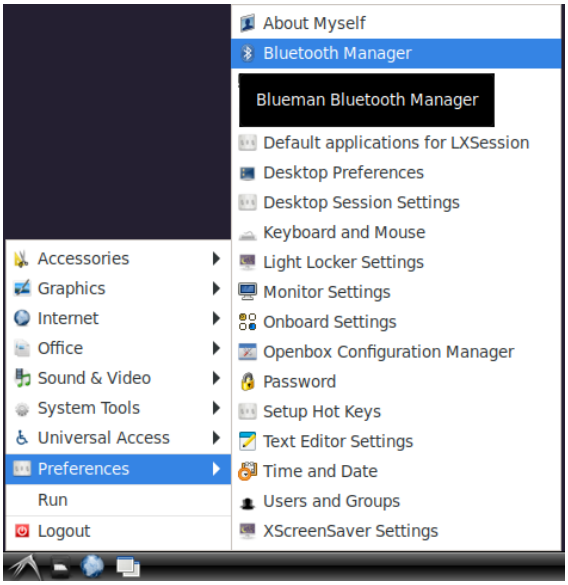


Figure 88. Bluetooth icon in Ubuntu LXDE taskbar

2. Enable Bluetooth: If Bluetooth is not already enabled, click the “Turn Bluetooth On” option to activate it.
3. Search for Devices: Select Adapter and click Search to view a list of available Bluetooth devices.

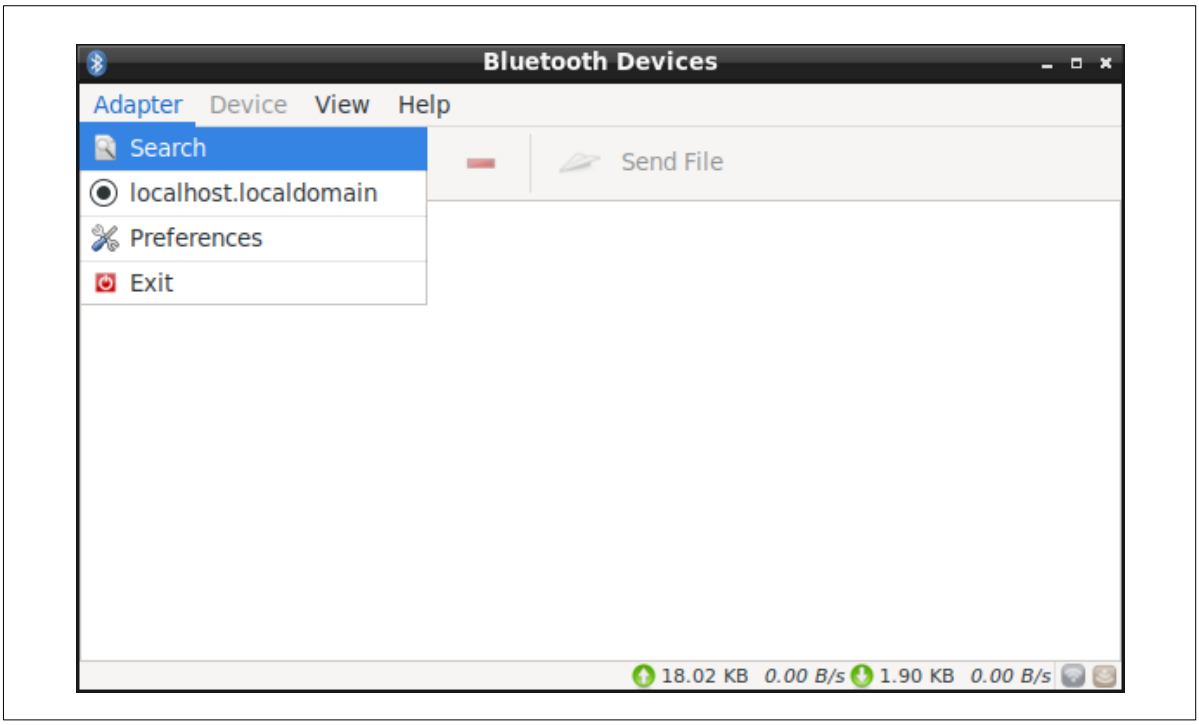


Figure 89. Search for available Bluetooth devices

4. Select the device: From the list of available Bluetooth devices, select the desired device to connect to.

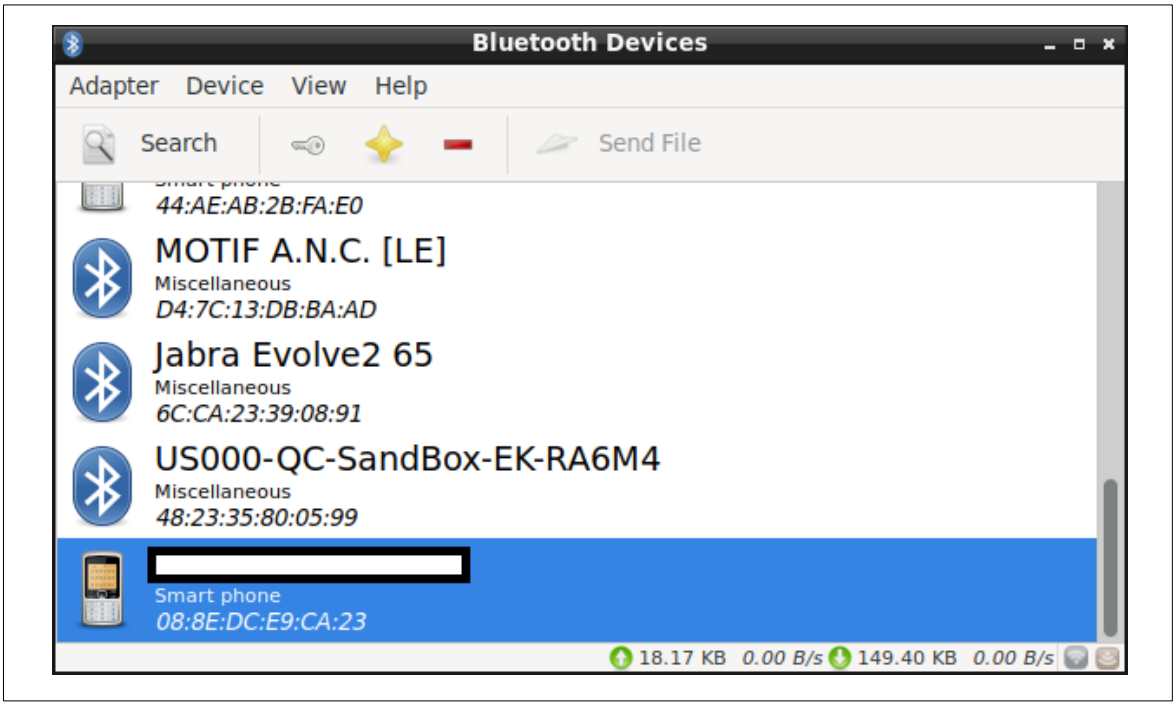


Figure 90. Select a Bluetooth device

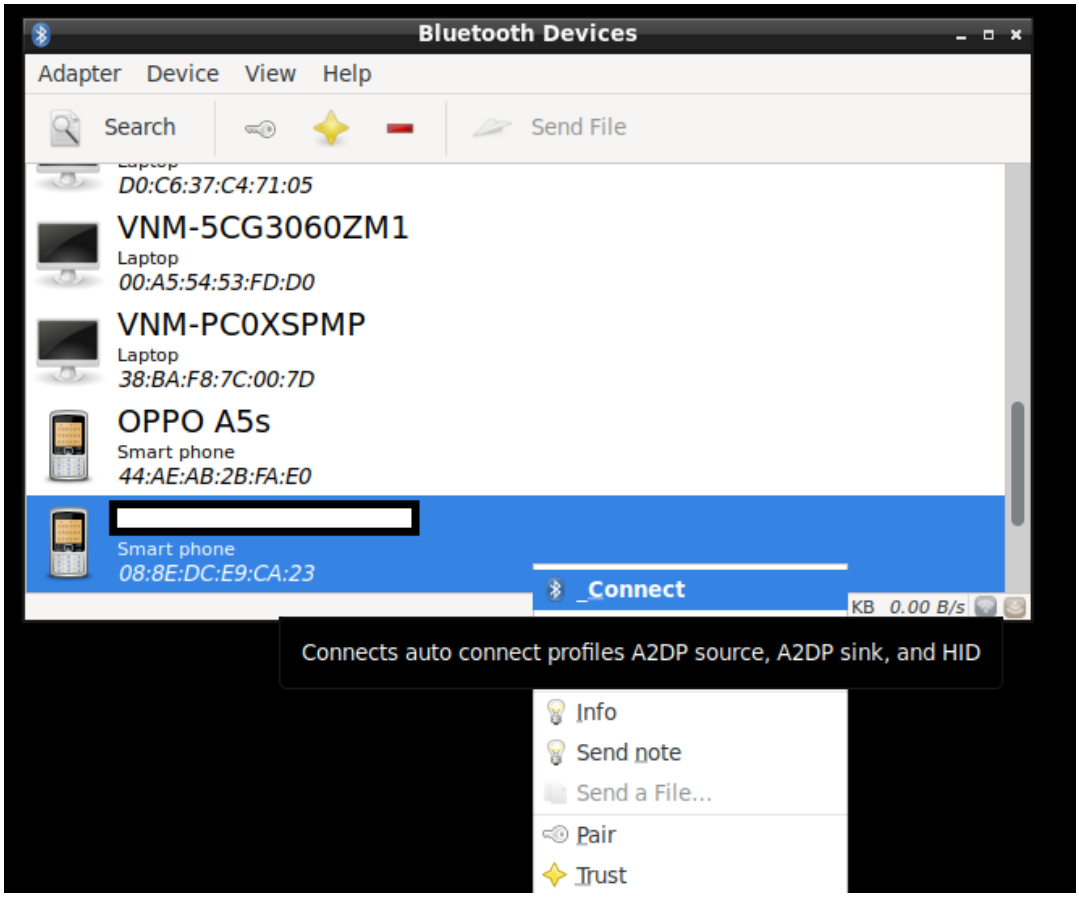


Figure 91. Connect to a Bluetooth device

5. Pair the Device: If prompted, confirm the pairing request and enter the required pairing code or PIN if necessary. After confirming, the devices will be paired.

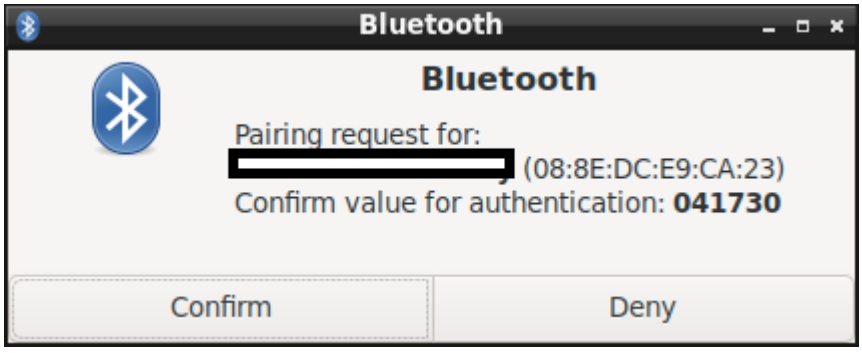


Figure 92. Bluetooth connection confirmation

6. Connection Established: Once the pairing process is complete, the device will be successfully connected.

8.2.2 Accessing Supported Features in Ubuntu Core

Ubuntu Core provides similar feature support to Yocto-based images, offering a headless environment for command-line operations. Feature usage and functionality align closely with those available in Yocto images. For details on supported features and their usage, refer to section Supported Features in Yocto Images

8.2.2.1 Configure the Network in Ubuntu Core

The Ubuntu installer configures the system to obtain network settings via DHCP by default. To switch to a static IP address, modify the network configuration using Netplan. The configuration file /etc/network/interfaces is no longer used. Instead, edit /etc/netplan/00-installer-config.yaml to set a static IP address. For example, the following configuration assigns the IP address 192.168.0.100 and specifies the DNS servers 8.8.4.4 and 8.8.8.8.

To open the network configuration file, use:

```
root@localhost:~# sudo vi /etc/netplan/00-installer-config.yaml
```

After installation, the system uses DHCP, and the network configuration file appears as follows:

```
# This is the network config written by 'subiquity'

network:
  ethernets:
    ens33:
      dhcp4: true
  version: 2
```

To assign a static IP address (192.168.0.100), modify the file as follows:

```
# This file describes the network interfaces available on your system

# For more information, see netplan(5).
network:
 version: 2
 renderer: networkd
 ethernets:
   ens33:
     dhcp4: no
     dhcp6: no
     addresses: [192.168.0.100/24]
     routes:
      - to: default
        via: 192.168.0.1
     nameservers:
       addresses: [8.8.8.8,8.8.4.4]
```

Then the hosts file needs to be updated to reflect the new hostname and IP address:

```
root@localhost:~# sudo vi /etc/hosts
```

Modify the file by adding the following entries:

```
127.0.0.1 localhost

192.168.0.100 rz-cmn.example.com rz-cmn

# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Next, change the hostname; run the following commands:

```
root@localhost:~# sudo echo rz > /etc/hostname
root@localhost:~# sudo hostname rz
```

The first command updates /etc/hostname, which is read during boot. The second command applies the change immediately without requiring a reboot.

As an alternative to the two commands above. Instead of manually updating the hostname file, the hostnamectl command (part of systemd) can be used:

```
root@localhost:~# sudo hostnamectl set-hostname rz
```

Afterward, run:

```
root@localhost:~# hostname
root@localhost:~# hostname -f
```

The first command returns the short hostname, while the second command shows the fully qualified domain name:

```
root@localhost:/home/root# hostname
localhost

root@localhost:/home/root# hostname -f
localhost.example.com

root@localhost:/home/root#
```

8.2.2.2 PCIe Wi-Fi Module

Wi-Fi can be provided via a PCIe Wi-Fi module. Follow the instructions in this section **PCIe Wi-Fi Module** to configure or set up the Wi-Fi module

8.3 Real-time (RT) Systems and OS

This section describes the core fundamentals of real-time concepts followed by introducing the real-time capability provided by this release package. It explains the difference between hard, firm, soft real-time, and how to select the RT or Normal (default) image in U-Boot and how to confirm the running mode after boot.

8.3.1 Overview

Real-time operation is one where a response is required to an event within a deterministic window of time. It is a key design consideration for critical systems.

Imagine a car's electronic brake system. When the driver presses the brake pedal, the system must respond within a known time regardless of any environmental factors. Failing to respond creates catastrophic consequences. In a similar manner, control response to nuclear reactor events needs to be in real-time.

In such systems, the OS along with the rest of the hardware and business logic is required to respond within a bounded time to events such as interrupts, periodic timers, or external I/O. In Linux-based systems, real-time behavior depends on kernel configuration, system load, interrupt handling, and application design. An OS kernel enforcing such behavior is called a real-time operating system (RTOS).

A real-time OS is characterized by two main characteristics:

- Pre-emption: The ability of the operating system to interrupt a currently running task and regain control when a higher-priority task or time-critical event must be handled. The scheduler's ability to take control of the system is nearly guaranteed, and most of the OS kernel itself is often pre-emptible. This prevents one application from holding the CPU indefinitely and allows urgent work to be scheduled promptly.
- Determinism: The scheduler is designed in a manner such that the performance of the system is deterministic, meaning it's properly responding to events within a margin of error. This is usually a major design aspect where no "unknowns" are tolerated. The system designer knows and has considered every eventuality and the response to it, along with the granular timing of events across the full system and not just the OS.

8.3.2 What an RTOS is not

There is a common misunderstanding of Real-time systems as low-latency and high-speed systems. This is fundamentally false. Most traditional operating systems are designed for maximum throughput, i.e lowest idle CPU. In raw processing ability, the everyday OS often outperforms an RTOS due to it being optimized for throughput.

RTOS systems provide predictability and deterministic behavior, and in doing so often allow larger idling of CPU. Low latency is a design trick used to achieve determinism but not an end goal. Low latency also is not a proxy substitute for raw throughput. Low latency guarantees fast response to events but does not guarantee maximum throughput.

8.3.3 Types of Real-time Systems

There are three types of Real-time systems:

- Hard Real-time — Tolerance for a missed deadline is extremely low and often catastrophic. An example is car brakes or flight controls, where missing an event can crash the system and cost lives.
- Firm Real-time — Missed deadlines cause reduced qualitative experience to the point of being unusable. The missed deadline isn't catastrophic, but the results are invalid and useless. Examples include financial trading, video conferencing, and satellite tracking, resulting in glitchy behavior which can make them unusable.
- Soft Real-time — Missed deadlines are recoverable, and the system, though it degrades in performance, continues to be functional. Examples include UI responses like mouse and keyboard and Ticketing systems where missed events are creating a noticeably bad experience, but the system completes the task regardless.

In the case of operating systems, the only real-time classes are Hard, soft, and none.

The OS is either not a real-time OS or hard / soft real-time. There is no firm real-time OS, as this is more related to systems-level design and not OS design.

Note: In reality, having a real-time OS alone does not guarantee real-time behavior of the whole system. A proper real-time system is designed by a professional with careful consideration and very strict protocols where there are no unknowns and many convenient features are avoided on purpose. Real-time systems and operating systems are fields of study and execution in their own and beyond the scope of this manual. Additional reference materials are provided in the References section for those interested.

8.3.4 Linux RT Patch

The Real-time kernel patch is a major change to the fundamentals of the Linux kernel, merged in the heat of the COVID era, and fundamentally alters how Linux handles interrupts, clocks, events, and handoffs.

This release package provides three boot images:

- Normal (default) image — Soft real-time: Intended for general-purpose usage. It can satisfy many time-sensitive workloads; however, occasional deadline misses may occur depending on system load and configuration.
- RT image — Hard real-time (target use): Intended for applications that require improved determinism and reduced worst-case latency/jitter compared with the default image.
- Non-Preemptive Kernel: An environment in which kernel preemption is disabled. This variant is intended for special use cases such as debugging preemption-related behavior or environments that prefer strictly non-preemptive kernel execution. This was also how Linux behaved before the real-time patch in 2020.

RT capability is supported on all boards covered by this release package. Achievable latency depends on the system configuration and workload.

The following sections describe soft real-time and hard real-time behavior in more detail.

8.3.4.1 Soft Real-Time (Normal / Default Image)

Soft real-time systems aim to meet timing targets most of the time, but occasional deadline misses may be tolerated.

Characteristics

- Suitable for mixed workloads and general-purpose Linux operation
- Timing behavior is generally stable under light to moderate load
- Latency spikes may occur due to background services, storage/network I/O, interrupt activity, CPU frequency scaling, or thermal throttling

Typical Use Cases

- General application development and evaluation
- UI, networking, logging, and service workloads
- Multimedia/vision workloads where occasional frame drops are acceptable
- Periodic tasks without strict worst-case requirements

8.3.4.2 Hard Real-Time (RT Image)

Hard real-time systems require deadlines to be met within a strict bound; missing a deadline is considered a failure from the application perspective. The RT image is provided to improve determinism and reduce worst-case latency.

Characteristics

- Improved worst-case latency and jitter compared to the Normal (default) image

- Best results require system-level tuning and workload control (CPU/IRQ affinity, limiting background services, controlled I/O)

Typical Use Cases

- Industrial control loops with strict cycle-time requirements
- Deterministic actuator control and time-critical I/O handling
- Robotics control workloads (for example, motor/servo control loops and time-bounded actuator coordination) such as those targeted by the RZ/V2H-RDK (Robot Development Kit)
- Applications that define worst-case latency requirements and validate against them

Note: Even with an RT kernel, hard real-time behavior depends on the overall system design. Validation against application-specific acceptance criteria is required.

8.3.4.3 Non-Real-Time / Traditional kernel

This is the original Linux kernel with Softirqs and other mechanisms that prevented Linux from ever being a real-time system for decades but also ran the most high-performance clusters (supercomputers) in the world. This old kernel was designed with one and only consideration — raw processing throughput.

8.3.5 What does this mean for our Linux

The RT patch in the Linux kernel was a major redesign of core mechanisms such as interrupt and exception handling, the kernel scheduler, and memory management. By choosing the type of real-time kernel, you are effectively choosing what type of scheduling, event management, interrupt mechanism, and syscall handling will play out. The same application will show phenomenal differences in response and raw throughput. This choice will determine if your end application in embedded systems will meet a potential real-time requirement of your product. The difference between your application being functional and failing to respond or glitching might simply be this one selection setting at boot time.

8.3.6 Selecting RT or Normal Image in U-Boot

The kernel image loaded by U-Boot can be selected using the *image_flavor* environment variable. When *image_flavor* is set, U-Boot selects the corresponding kernel image filename (for example, Image, Image-preempt_rt) and loads it from the boot (FAT) partition.

Table 40. image_flavor options and corresponding kernel images for real-time kernel

image_flavor	Kernel Image	Notes
normal	Image	Default Image - Soft real-time (general purpose)
preempt_rt	Image-preempt-rt	Hard real-time (PREEMPT_RT)
nonpreempt	Image-nonpreempt	Non-preemptive

If *image_flavor* is not set, the default is normal.

To select a kernel image, perform the following steps:

1. Reboot the board or power it on.
2. When the message below is displayed, press any key to stop automatic boot and enter the U-Boot prompt:

   Hit any key to stop autoboot

3. Set *image_flavor* to the corresponding value. For example, to select the hard real-time (PREEMPT_RT) kernel, set *image_flavor* to *preempt_rt*.
```
=> setenv image_flavor preempt_rt
```
4. To keep the selected kernel image across reboots.
```
=> saveenv
```
5. Enter 'boot' to boot to the kernel
```
=> boot
```
6. Verification: After Linux boots, confirm the running kernel variant (example for PREEMPT_RT)
```
root@rz-cmn:~# uname -v
#1 SMP PREEMPT_RT <DATE> <UTC TIME> <YEAR>
```

8.4 DRP/AI (DRP-AI)

This section describes the DRP/AI BSP software support provided by this release package. It introduces the DRP and DRP-AI technologies at a high level, lists the supported MPUs and boards, and summarizes the software interfaces used to access DRP-AI functionality on supported RZ/V devices.

8.4.1 Supported RZ/V Devices

The RZ/V series is a Renesas MPU family designed for vision AI applications. In this context, the "V" represents Vision. RZ/V devices are intended for systems that process camera or image data and perform AI-based recognition, detection, classification, or analysis.

DRP-AI functionality is supported only on devices that include the required DRP-AI hardware block and corresponding software support. Therefore, this release supports DRP-AI-related functionality on the following RZ/V platforms:

- RZ/V2L:
  - RZ/V2L-EVK
- RZ/V2H:
  - RZ/V2H-EVK
  - RZ/V2H-RDK
  - IMDT-V2H-SBC

The available hardware blocks and software interfaces may differ depending on the target device.

8.4.2 Core Hardware concepts

There are a few important hardware concepts we need to establish before proceeding to explain DRP-AI.

8.4.2.1 Switch

A digital switch is basically a logical element that controls an io line. It connects data paths and is controlled by a digital selection line. It's practically like the everyday switches we use but much smaller and fully digital.

8.4.2.2 Processing element — PE

A processing element (PE) is basically a hardened arithmetic unit capable of operations such as multiply, add, shift, etc. It is fundamentally the same as an ALU but has substantially reduced abilities and is not programmable in the traditional sense. The function is hard-wired by design, and a switch matrix controls the interconnects and triggers. While the individual function may be complex, it does not have a vast set of operations or any flexibility with regard to programming them.

8.4.2.3 Multiply and Accumulate unit — MAC unit

A Multiply and Accumulate (MAC) unit is a basic digital logic unit made of three parts:

- A multiplier that multiplies two inputs.
- An Adder that adds the product to a running total.
- An Accumulator, which is a single register that stores the running total for in between the operations.

MAC units became popular with Digital signal processors from the pre-2005 era, where these were used extensively to perform floating-point operations in signal processing. However, these found new applications in modern MPU's. These are the basic building blocks of modern NPU's. An array of MAC units arranged in an NxN configuration allows processing of matrix computations, which is fundamental to AI applications.

8.4.2.4 DMA & Buffering

DMA stands for Direct Memory Access, which is a standalone unit whose main job is to move data between the memory and the execution unit independent of the CPU and can handle large blocks of data at once.

A large block of memory held for storing data for processing is called a buffer, and the process of collecting this data in the memory is called buffering. Buffers are heavily used in every computer application, and often the first place data goes from a sensor or storage and the last place it's held before storage into secondary memory like hard drives and SD Cards.

8.4.3 Overview

Embedded vision systems use image data from cameras or image sensors to understand what is happening around the device. Instead of only capturing and displaying images, these systems analyze the image data and use the result to make decisions or trigger actions.

A typical case of use is a camera-based inspection or detection system. The system captures an image, prepares the image data, runs an AI model, and then uses the result to determine the next action. This may include detecting a person, recognizing an object, inspecting a product, reading a barcode, or classifying an image.

These operations can require significant processing. Image preparation may involve resizing, cropping, color conversion, or normalization. AI inference may involve running a trained neural network model on the prepared input data. If these tasks are performed only by the CPU, the system may have limited throughput, higher latency, or increased power consumption.

To address this, Renesas RZ/V devices provide hardware acceleration using DRP and DRP-AI. These hardware blocks are designed to offload processing-heavy workloads from the CPU and execute them more efficiently. At a high level:

- CPU controls the application, prepares memory, starts hardware processing, and handles the results.
- DRP accelerates image and data processing. This is an NPU for all practical purposes.
- DRP-AI extends this concept for AI applications, especially embedded vision systems.

In simple terms, DRP/AI helps the system process camera data faster, reduce CPU workload, and improve power efficiency. Renesas describes DRP as a proprietary accelerator technology that provides software flexibility with hardware performance.

8.4.3.1 DRP — Dynamically Reconfigurable Processor

The Dynamically Reconfigurable Processor (DRP) is a Renesas hardware accelerator technology for image and vision processing. It is built to deliver hardware-level speed while keeping the ability to change processing behavior through configuration data instead of redesigning hardware

A CPU provides high software flexibility, but it normally executes operations sequentially. An FPGA provides configurable hardware, but it usually requires hardware-level design and implementation. DRP provides another approach by dynamically reconfiguring internal hardware resources to create efficient processing paths for specific tasks. This allows DRP to achieve hardware-like acceleration while still allowing processing behavior to be changed through configuration data.

The DRP is a Coarse-Grained Reconfigurable Array (CGRA). It contains an array of pre-built functional processing elements (PE's) — adders, multipliers, comparators — whose interconnections can be rewired at runtime by loading a stored "context." This context defines both the function of each element and the wiring between them, effectively making the silicon behave as a different circuit for each pipeline stage.

Key characteristic: Context switching is single-cycle. Multiple contexts are stored in on-chip SRAM, and the array flips between them every clock cycle with zero stall overhead. This makes it practical for real-time video pipelines where each frame passes through several distinct processing stages in sequence.

The DRP handles all pre- and post-processing around the neural network: demosaicing, color space conversion, noise reduction, resize, normalization before inference, and bounding-box decoding or other post-processing afterward.

Analogy: The DRP is conceptually similar to an FPGA with pre-compiled configuration bitstreams that can be swapped in one cycle — but with coarser-grained logic blocks (complete arithmetic units rather than LUTs), giving ASIC-level efficiency within each block and FPGA-like reconfigurability between them.

Scope: The DRP is optimized for structured image-processing operations: color space conversion, demosaicing, filtering, morphological operations, resize, stereo matching (SGM). It is not a general-purpose compute engine and does not run neural network inference directly.

Key idea:

A complex algorithm is divided into smaller hardware configurations called contexts. The DRP hardware can switch among these contexts while processing, allowing one hardware area to perform multiple stages of an algorithm over time

This provides:

- high throughput through pipelined hardware execution
- flexibility through context switching
- efficient silicon usage through time-multiplexing

The following image shows an example of dynamic reconfiguration:

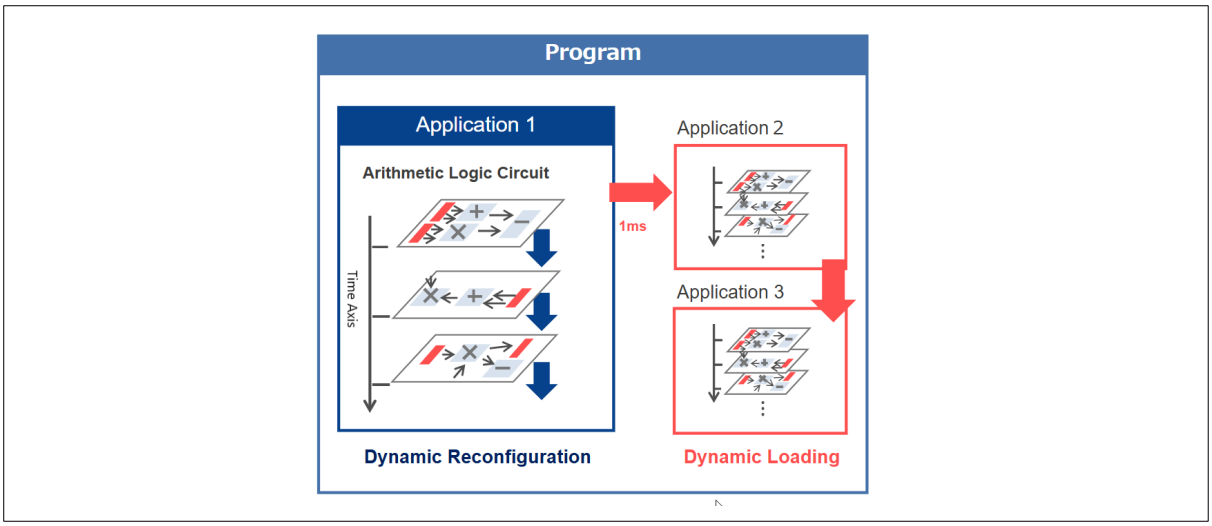


Figure 93. DRP Dynamic Reconfiguration

This figure illustrates how the DRP changes its hardware behavior over time by reconfiguring the arithmetic logic circuit.

On the left side, Application 1 is already loaded into the DRP. The stacked plates represent different hardware configurations, or contexts, used by the same application. As processing moves along the time axis, the DRP switches from one context to the next. Each context changes how the internal processing elements, such as adders, multipliers, comparators, and data paths, are connected. In effect, the same physical hardware becomes a different processing circuit at each step of the algorithm.

For example, one context may perform filtering, the next may perform arithmetic transformation, and another may perform comparison or thresholding. Instead of implementing all stages as separate dedicated hardware blocks, the DRP reuses the same hardware area and changes its configuration dynamically.

The right side shows Application 2 and Application 3 being loaded dynamically. This represents switching to a different set of contexts for a different processing task. The red arrows indicate that new application configuration data can be loaded when the required processing changes. The figure shows this dynamic loading taking around 1 ms, while the context switching inside an already loaded DRP application occurs much faster, typically at cycle-level granularity depending on the device and implementation.

This is the key benefit of DRP, as the hardware is not fixed to one operation. It can behave like different arithmetic circuits at different points in time, enabling high-speed processing while using silicon area efficiently.

8.4.3.2 AI-MAC — Multiply-Accumulate Array

DRP-AI is the Renesas hardware accelerator built into supported RZ/V MPUs. It is designed for embedded vision applications that process image data and use Artificial Intelligence (AI) to interpret the input.

In this context, AI inference means running a trained AI model, such as a neural network, to analyze input data and produce a result. For example, the AI model may determine whether a person is present in an image, identify an object, classify a product, detect a barcode, or recognize a pattern. This step is called inference.

DRP-AI can be understood as a hardware accelerator that helps execute AI-related processing efficiently. The DRP-related processing is well suited for image and data preparation, while the AI acceleration hardware executes inference using the prepared data to generate a meaningful result.

In simple terms, DRP-AI helps supported RZ/V devices run trained AI models efficiently for embedded vision applications.

What it is: A fixed-function multiply-accumulate array optimized for dense matrix operations — the same architectural idea as any mainstream NPU. It handles convolutional layers, fully connected layers, and activation functions by streaming weights and activations through fixed hardware pipelines.

Important: The AI-MAC is functionally equivalent to an NPU. Renesas bundles it with the DRP under the “DRP-AI” product name, which is more of a trade term encompassing the overall design. Comparing “DRP-AI vs NPU” is therefore not a like-for-like comparison — the AI-MAC inside DRP-AI is an NPU. The DRP is the differentiator.

8.4.3.3 DRP-AI — The Product Name

DRP-AI is the marketing and product label Renesas uses for our AI accelerator IP. In the first-generation devices (RZ/V2M, RZ/V2L), DRP-AI is an integrated block that internally contains both a DRP and an AI-MAC. In later devices, the blocks were physically separated on-chip into independent accelerators, but the DRP-AI label continues to refer to the neural-inference-facing accelerator.

8.4.3.4 DRP-AI Architecture

On supported RZ/V devices, the AI accelerator is referred to as DRP-AI. In general, DRP-AI combines a DRP block and an AI-MAC block to execute AI inference efficiently.

The DRP-AI hardware uses the system memory hierarchy to fetch input data, execute the inference workload, and store the output results. DRP-AI is designed to execute inference independently of the CPU. In a typical software flow, the DRP-AI driver reads descriptor information generated from a trained AI model, configures the hardware, and starts execution. The CPU remains responsible for control, memory preparation, and result handling.

On some devices, a separate DRP block is also available for computer vision acceleration in addition to the DRP-AI block. This separate DRP is used for image and vision pre-processing tasks and is distinct from the DRP-AI inference engine.

DRP and AI-MAC responsibilities

- DRP-AI: The AI accelerator contains a DRP and an AI-MAC. These blocks work together to execute AI inference and store the results using the hierarchical memory system.
- DRP: The DRP is a dynamically reconfigurable processor used for computer vision processing acceleration. It performs data-processing operations efficiently using reconfigurable hardware and the system memory hierarchy.

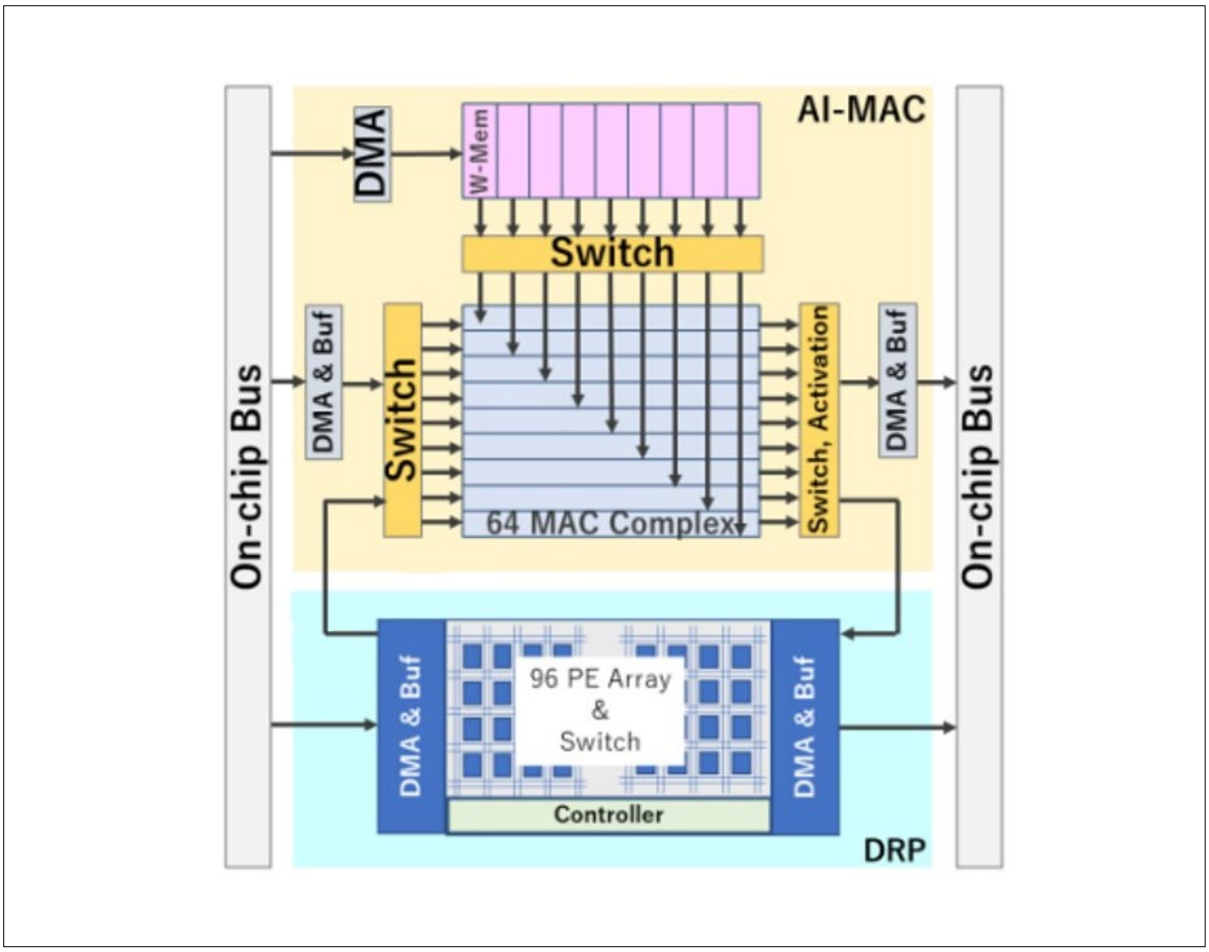


Figure 94. DRP-AI Driver Architecture

8.4.3.5 DRP-AI Driver Execution Flow

The DRP-AI Driver manages the sequence required to execute AI inference on the DRP-AI hardware. The driver receives input data from the application, prepares the data for AI processing, executes inference, and returns the processed result.

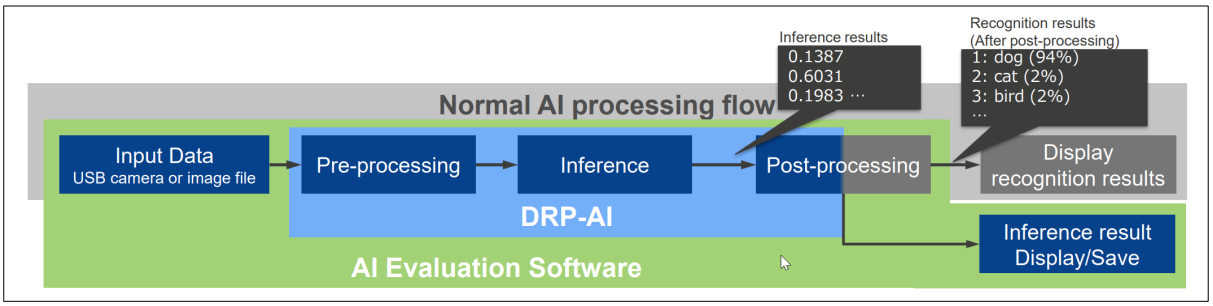


Figure 95. DRP-AI Driver Execution Flow

The execution flow consists of the following stages:

1. Pre-processing

   Input data is converted into the format required by the AI model. This may include resizing, cropping, format conversion, and normalization.

2. Inference Execution

   The DRP-AI hardware executes the AI model using the pre-processed input data and generates output data.

3. Post-processing

   The output data is interpreted and converted into the final inference result. This may include label extraction, score calculation, threshold processing, or coordinate conversion.

4. Application Use

   The application uses the result for display, communication, logging, or system control.

This execution flow allows the application to use DRP-AI through a driver-based interface while the hardware accelerator performs heavy processing tasks.

8.4.4 DRP-AI & DRP-AI3

The Renesas products as of this writing have two distinct DRP configurations, and an SoC part may contain either both or none of the DRP blocks.

Gen1 — Integrated DRP-AI (pre 2022 designs)

The Gen 1 AI-MAC operates on 16-bit floating-point values and is paired with the image processing DRP.

Gen 3 — DRP-AI3 (INT8 + N:M pruning): The DRP-AI3 AI-MAC switches to 8-bit integer arithmetic. Halving the numeric precision reduces both arithmetic unit area and memory bandwidth by 2x, which translates to a 2x power improvement from quantization and up to 10 times power efficiency overall. In addition, the DRP-AI3 adds workflow support for flexible N:M pruning, allowing fine control over pruning. This unit is better at running a sparse model after pruning.

8.4.5 DRP is a Shared Resource (System-Level Constraint)

This section describes the single most important system-level constraint on RZ/V chips, and the one most likely to surprise a developer reading the marketing material. The DRP block is one physical piece of silicon used by multiple software paths at different times — they cannot all use it simultaneously.

The DRP is a single CGRA block that holds only one context at a time, and therefore only one user at a time. On a typical RZ/V2H system, the DRP has the following potential users:

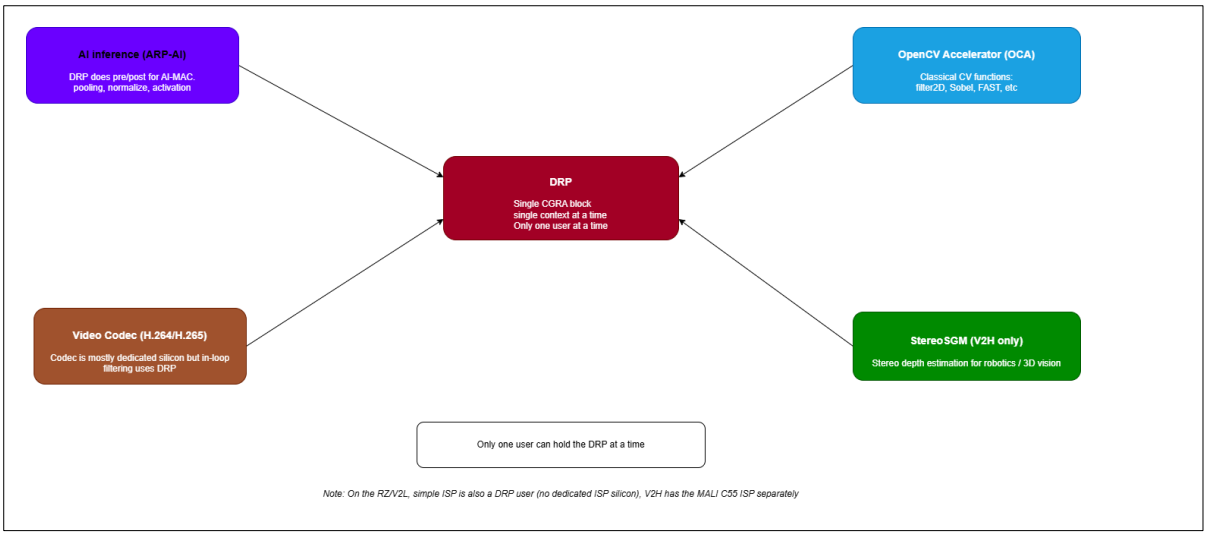


Figure 96. DRP is a shared resource

Table 41. DRP's users on the RZ/V2H

User	Role	Notes
AI Inference (DRP-AI)	Pre/post-processing for AI-MAC: pooling, normalization, activation, output decoding	Active whenever a neural network is running
OpenCV Accelerator (OCA)	Classical CV functions (filter2D, Sobel, FAST, edge detection, color conversion, morphological ops)	Renesas library that intercepts standard OpenCV calls and routes compatible ones to DRP
Video Codec (H.264/H.265)	The codec is mostly dedicated silicon, but in-loop filtering uses DRP	This is why decoding video and running AI inference simultaneously will fail
StereoSGM (RZ/V2H only)	Stereo depth estimation for robotics / 3D vision	Implemented as a DRP context
Simple ISP (RZ/V2L only)	Camera input pipeline: color correction, noise reduction	RZ/V2L has no dedicated ISP silicon, so every camera capture competes for the DRP. RZ/V2H uses the separate Mali-C55 ISP and avoids this conflict

System Design Implication

You **cannot** pipeline AI inference and video decoding (or AI inference and OCA, or video decoding and OCA) in parallel on the same DRP. They must be scheduled sequentially. This is the central system-level constraint when designing on the RZ/V family.

Note: This constraint is documented in the OCA library manual under "DRP conflict" rather than in the datasheet, so it is easily missed during early system design.

8.4.6 RZ/V DRP-AI Software Stack

This section provides a software-oriented view of DRP-AI on supported RZ/V devices. It summarizes the main hardware cooperation model and the low-level driver APIs used to control DRP-AI execution.

The APIs described in this section are driver-level interfaces. They are typically used by the software stack to initialize the hardware, start execution, handle interrupts, check status, and recover from errors. They are included here to help users understand the software structure and execution sequence provided by this release package.

8.4.6.1 DRP-AI End-to-End Workflow

DRP-AI development is split into two distinct environments: a host workstation where the model is prepared, and the target RZ/V device where the model is executed. Understanding this split is essential before looking at the driver APIs, because the driver APIs only cover the target-side runtime — the host-side steps happen offline using a separate toolchain.

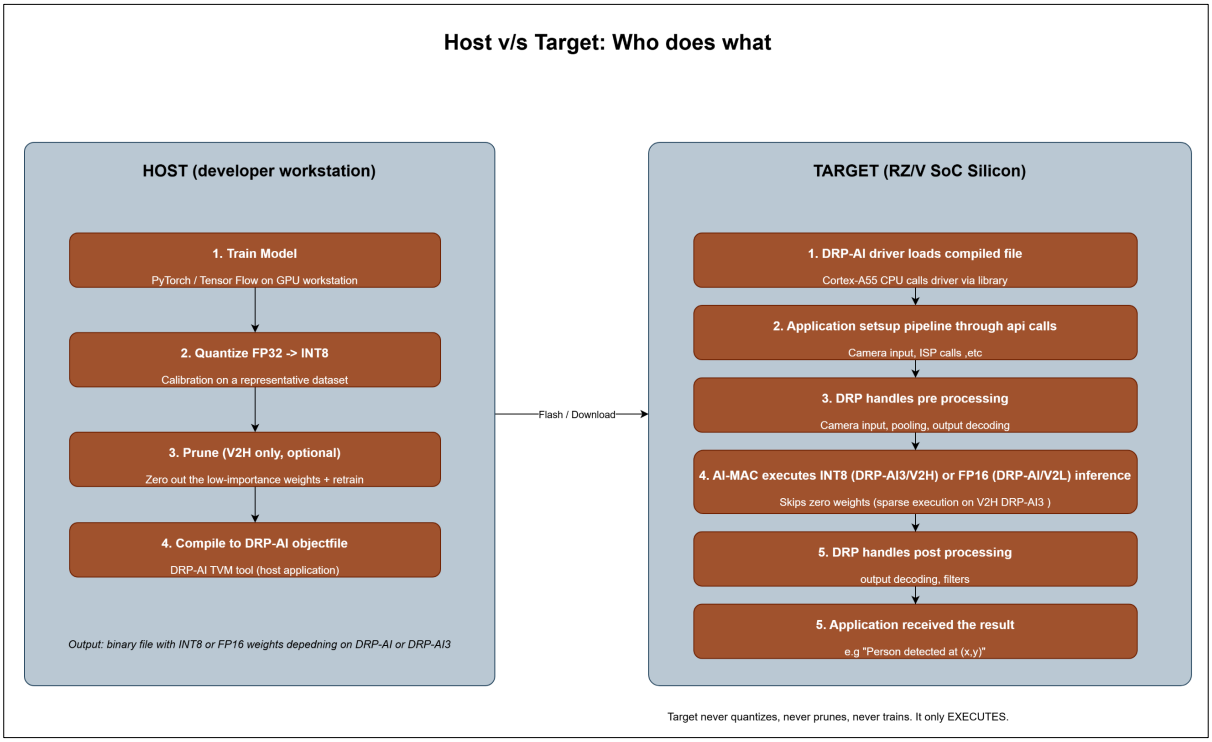


Figure 97. Workflow Overview

(1) What the Host Does

All model preparation happens on a developer's Linux workstation, not on the RZ/V chip. The host performs:

- **Training** — PyTorch or TensorFlow on a workstation GPU
- **Quantization** — converting FP32 weights to INT8 using calibration data
- **Pruning** — zeroing out low-importance weights and fine-tuning to recover accuracy
- **Compilation** — producing a binary file containing the quantized (and optionally pruned) weights, the DRP context sequence, and runtime metadata

Renesas provides three host-side tools for this workflow:

- **DRP-AI Translator** — older, simpler
- **DRP-AI TVM Compiler** — current, based on Apache TVM
- **DRP-AI Extension Pack** — pruning tool

All three run on Ubuntu. None of them run on the RZ/V chip.

(2) What the Target Does

The RZ/V chip only ever runs the binary file produced by the host. At runtime, the target:

- Loads the compiled binary via the DRP-AI Linux driver
- Streams pre-quantized INT8 (or FP16) weights into the AI-MAC
- Executes the DRP context sequence for pre- and post-processing
- Returns the inference result to the application

The driver APIs documented in the next sub-sections

(3) Pruning: A Host–Target Collaboration (RZ/V2H DRP-AI3 only)

Pruning illustrates why the host/target split matters. Effective sparse-model acceleration on the RZ/V2H requires both halves of the workflow.

- Host side — create the zeros: The pruning tool zeros out low-importance weights and retrains the model to recover accuracy. The output is a sparse weight matrix.
- Target side — skip the zeros at runtime: The DRP-AI3 AI-MAC contains zero-detect logic that recognizes a zero weight and skips the multiply-accumulate cycle entirely.

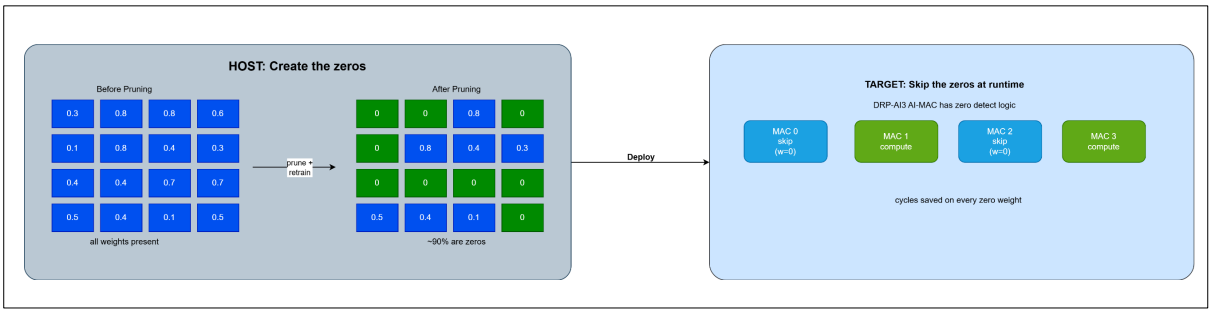


Figure 98. A Host-Target Collaboration (RZ/V2H DRP-AI3 only)

Why is neither half alone sufficient? Pruning improves runtime only when the model is made sparse on the host and the target hardware can exploit that sparsity during execution. If either part is missing, the target either processes zero-valued weights without gaining a performance benefit or has no zeros to skip. The combinations table below shows the outcome scenarios:

Table 42. Host–Target Pruning Combinations and Outcomes

Scenario	Result
Prune model, deploy on RZ/V2L	No speedup. RZ/V2L AI-MAC has no zero-skip logic; it multiplies every zero weight.
Deploy non-pruned model on RZ/V2H	No sparse speedup. Zero-skip logic detects no zeros; only dense performance is achieved.
Prune model and deploy on RZ/V2H	Full sparse-execution speedup. Both halves are in play.

8.4.6.2 Example Workflow: Deploying a First AI Model on RZ/V Devices

This section walks through the complete end-to-end process of taking a publicly available pre-trained model and running it on an RZ/V evaluation board. It covers both the RZ/V2L (DRP-AI, FP16) and the RZ/V2H (DRP-AI3, INT8) targets. The two paths share the same compiler frontend repository on GitHub but use different backend Translator tools and different compilation scripts because the underlying hardware accepts different numeric formats.

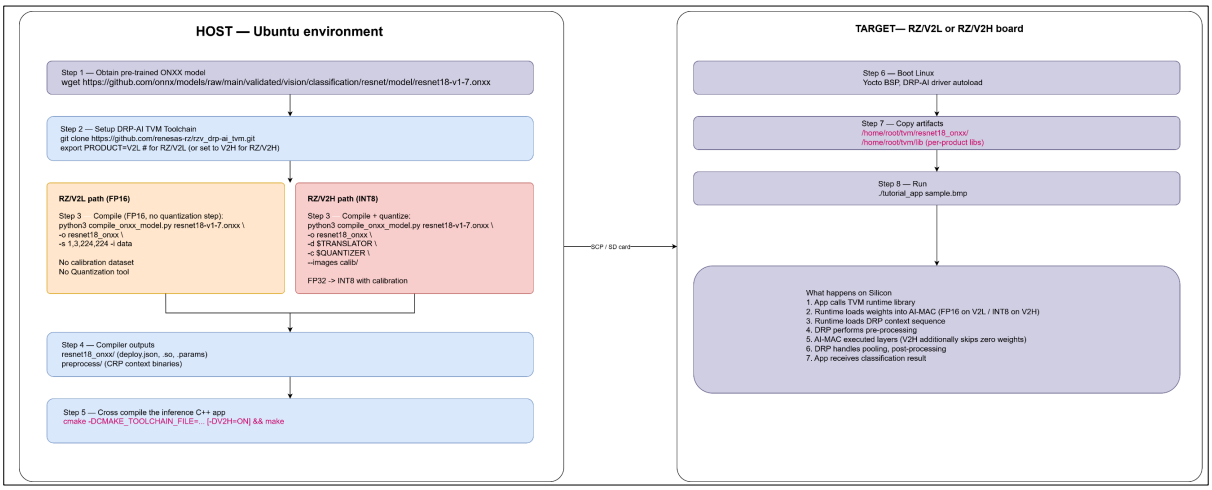


Figure 99. End-to-end developer workflow: ONXX model to running inference on RZ/V2L or RZ/V2H

(1) Prerequisites

Prepare the following items before starting.

- Host PC:
  - Use a Linux development machine that supports the required SDK and tools for the target RZ/V device.
  - For RZ/V2H RDK, use a Docker-based DRP-AI TVM environment where possible. After the environment is created, all compilation work should be performed inside the Docker container.
- Target Board
  - Prepare the target RZ/V board, for example:
    - RZ/V2L Evaluation Board Kit
    - RZ/V2H Evaluation Board Kit
- Required Software: The required software depends on the target device.
  - For RZ/V2L:
    - DRP-AI TVM compiler frontend

        - DRP-AI Translator for FP16 conversion
        - SDK for cross-compilation
    - For RZ/V2H:
        - DRP-AI TVM compiler frontend
        - DRP-AI Translator i8
        - DRP-AI Quantizer
        - SDK for cross-compilation

The FP16 Translator and the INT8 Translator i8 are separate products and cannot be used interchangeably

(2) Choosing a Starter Model

For a first deployment on either device, the recommended model is ResNet18 from the ONNX Model Zoo — the same model used in the official Renesas tutorials. It is a standard ImageNet image classification network with three properties that make it ideal for learning the toolchain:

- **Public pre-trained weights** — no training step required. Single direct download.
- **Verified by Renesas** — listed in the official model compatibility tables for both V2L and V2H.
- **Modest size** — around 11 million weights. Compiles in minutes and runs in milliseconds.

After running the tutorial successfully, the same workflow extends to other models on the compatibility lists: MobileNetV2, YOLOX-S, HRNet, and others. ResNet18 is just the canonical entry point.

(3) Download Software

Four pieces of software are needed. Download them all before starting the install — the install process expects them to be present on disk. All Renesas downloads require a My Renesas account login.

- For both targets
    - DRP-AI TVM compiler frontend — the GitHub repository at DRP-AI TVM. This is the only piece that is publicly cloneable without a Renesas login. Same repository for both targets.
- For RZ/V2L specifically
    - DRP-AI Translator (v1.90 or later) — from DRP-AI Translator | Renesas. This is the FP16 Translator. The installer file is named *DRP-AI_Translator-v1.xx-Linux-x86_64-Install* and is available inside the downloaded zip file.
- For RZ/V2H specifically
    - DRP-AI Translator i8 (v1.11 or later) — from DRP-AI Translator i8 | Renesas. This is a different product from the FP16 Translator above; you need this one for the V2H because it produces INT8 output and includes a Quantizer. The installer file is named *DRP-AI_Translator_i8-v1.xx-Linux-x86_64-Install* and is available inside the downloaded zip file.

(4) Install the Host Toolchain

The TVM compiler frontend is open source and can be cloned directly. The Translator is installed by running their respective installers as root. The recommended path is a Docker container that automates the installation steps but, more importantly, gives you a specific package environment that addresses the dependencies such as Python 3.8 for DRP-AI and 3.9 for DRP-AI3. Docker ensures that the correct versions are in place without contaminating the host.

Step 1. Create the work directory and prepare the Dockerfile

```
$ mkdir -p ~/drp-ai-docker-build
$ cd ~/drp-ai-docker-build
$ wget https://raw.githubusercontent.com/renesas-rz/rzv_drp-ai_tvm/main/Dockerfile -O
Dockerfile
```

Step 2. Prepare the DRP-AI Translator i8 for RZ/V2H or DRP-AI Translator v1.9 or later for RZ/V2L and the SDK installer file (in the rz-cmn-srp-sdk release package) in the work directory. The hierarchy should be like this:

```
user@renesas:~/drp-ai-docker-build$ tree
.
├── Dockerfile
├── DRP-AI_Translator*<type>-<version>-Linux-x86_64-Install
└── poky-glibc-x86_64-core-image-weston-cortexa55-rz-cmn-toolchain-ext-5.1.4.sh

1 directory, 3 files
```

Note: The Dockerfile is set to copy all shell scripts from this directory and execute them inside the image build. Ensure that no other shell scripts are present in the work directory.

Step 3. If the SDK is a standard yocto SDK, the Dockerfile is good to go.

Step 4. If the SDK is a yocto eSDK (as is with our package here), the default Dockerfile would try to install it as root user, which causes the script to exit and the image build to fail. Due to this, the Dockerfile must be modified to remove the SDK installation steps and manually install the eSDK into the container.

- The following lines should be removed from Dockerfile as shown below. They are essentially the lines that are under the “# Install SDK” section that execute the SDK installer. However, retain the line that copies the SDK binary to the image.
  Before change:

```
# Install SDK
COPY ./*.sh /opt
RUN chmod a+x /opt/*.sh
RUN cd /opt && yes "" | ./*.sh
RUN rm /opt/*.sh
```

  After change:

```
# Install SDK
COPY ./*.sh /opt
RUN chmod a+x /opt/*.sh
```

- In addition to the SDK installation part, an environment variable also needs to be removed. The line to be removed is marked with ‘-‘ at the beginning.

```
# Set environment variables
WORKDIR $TVM_ROOT
-RUN echo 'export SDK="`find /opt/ -name "sysroots"`/../"' >> ~/.bashrc
ENV PYTHONPATH="/opt/DRP-AI_Translator_i8/drpAI_Quantizer/"
-ENV LD_LIBRARY_PATH="$LD_LIBRARY_PATH:$LIBRARY_PATH"
-ENV LIBRARY_PATH="$LD_LIBRARY_PATH"
```

  In summary, the SDK variable and the LD_LIBRARY* variables need to be unset because they would originally come from the SDK environment. Since SDK is not installed here, these variables will get set to ‘:’ making the system unusable.

Step 5. Build the image using the Docker build command below — replace <docker-image> with a chosen name for the image.

```
user@renesas:~/drp-ai-docker-build$ docker build -t <docker-image>:5.1.4 .
```

Step 6. Create and start Docker

```
user@renesas:~/drp-ai-docker-build$. docker run --name <container-name> \
--security-opt seccomp=unconfined -dt <docker-image>:5.1.4
user@renesas:~/drp-ai-docker-build$ docker exec -it <container-name> /bin/bash
```

After starting the Docker container, all remaining actions from Step 7 are dedicated to the eSDK setup.

```
# Install missing dependencies for eSDK
root@<container-id>:/# apt-get update
root@<container-id>:/# apt-get install -y locales diffstat gawk cpio
root@<container-id>:/# locale-gen en_US.UTF-8

# Create developer user
root@<container-id>:/# useradd -m -s /bin/bash developer
root@<container-id>:/# chown -R developer:developer /opt
```

Step 7. Switch to the developer user (non-root) to install the eSDK. During installation, you will be prompted to enter a target directory. Any location where the non-root user has write permissions (to create files and directories) will work. In this guide, **/opt/poky_sdk** is used as the default installation path.

```
root@<id>:/# su - developer

# Switch to the 'developer' user

developer@<id>:~$ cd /opt
developer@<id>:~$ ./poky-glibc-x86_64-core-image-weston-cortexa55-rz-cmn-toolchain\
-ext-5.1.4.sh
```

Step 8. Source the eSDK environment and install necessary package

```
developer@<id>:~$ source /opt/poky_sdk/environment-setup-cortexa55-poky-linux
developer@<id>:~$ devtool sdk-install opencv drpai mmngr-user-module \
mmngrbuf-user-module
```

Step 9. After the installation is complete, switch back to the root user.

```
developer@<id>:~$ exit
```

All remaining commands in this section must be executed as root. Set environment variables in each new session terminal based on the target board:

```
# For RZ/V2L
root@<id>:/# export PRODUCT=V2L

# For RZ/V2H
root@<id>:/# export PRODUCT=V2H

# Common (same for both targets)
root@<id>:/# export SDK=/opt/poky_sdk
```

One-time setup (run only once after eSDK installation): Run the following commands to

configure the sysroot structure and resolve compiler path conflicts:

```
# Create the sysroots directory if it doesn't exist
root@<id>:/# mkdir -p /opt/poky_sdk/sysroots

# Clean up any partial/old links
root@<id>:/# rm -rf /opt/poky_sdk/sysroots/cortexa55-poky-linux
root@<id>:/# rm -rf /opt/poky_sdk/sysroots/aarch64-poky-linux
root@<id>:/# rm -rf /opt/poky_sdk/sysroots/x86_64-pokysdk-linux

# Target sysroot (the rootfs the cross-compiled code will run against)
root@<id>:/# ln -s /opt/poky_sdk/tmp/sysroots/rz-cmn \
/opt/poky_sdk/sysroots/cortexa55-poky-linux
root@<id>:/# ln -s /opt/poky_sdk/tmp/sysroots/rz-cmn \
/opt/poky_sdk/sysroots/aarch64-poky-linux

# Host SDK tools (the cross-compiler binaries)
root@<id>:/# ln -s /opt/poky_sdk/tmp/sysroots/x86_64 \
/opt/poky_sdk/sysroots/x86_64-pokysdk-linux
```

(5) Download the Pre-Trained Model

The model download is identical for both targets:

```
root@<id>:/# cd ${TVM_ROOT}/tutorials/
root@<id>:/# wget https://github.com/onnx/models/raw/main/validated/vision/\
classification/resnet/model/resnet18-v1-7.onnx
```

The downloaded file is roughly 44 MB of FP32 weights. The V2L path will convert it to FP16 during compilation; the V2H path will quantize it to INT8.

(6) Calibration dataset (RZ/V2H only)

Skip this section if your target is the RZ/V2L. The V2L runs models in FP16 natively without quantization, so no calibration is required. For the RZ/V2H, INT8 quantization needs sample input data to determine the typical range of activation values at each layer.

The DRP-AI Translator i8 install ships with a sample calibration dataset that works for the ResNet18 tutorial, which has no need to assemble your own for the first run:

```
# The sample calibration dataset is at:
root@<id>:/# ls $TRANSLATOR/../GettingStarted/tutorials/calibrate_sample/
# Contains ~50 representative images from ImageNet
```

For your own application later, you would replace this directory with images representative of your actual deployment scenario. A factory inspection camera would use real production-line images, not the ImageNet samples.

(7) Compile the Model

This is the step that differs most between the two targets, because each invokes a different compilation script with different arguments.

RZ/V2L compilation:

```
root@<id>:/# cd ${TVM_ROOT}/tutorials/

root@<id>:/drp-ai_tvm/tutorials# python3 compile_onnx_model.py \
./resnet18-v1-7.onnx \
-o resnet18_onnx \
-s 1,3,224,224 \
-i data
```

RZ/V2H compilation:

```
root@<id>:/# cd ${TVM_ROOT}/tutorials/

root@<id>:/drp-ai_tvm/tutorials# python3 compile_onnx_model_quant.py \
    ./resnet18-v1-7.onnx \
    -o resnet18_onnx \
    -t "$SDK" \
    -d "$TRANSLATOR" \
    -c "$QUANTIZER" \
    --images "$TRANSLATOR/../GettingStarted/tutorials/calibrate_sample/"
```

Note that the script name has a `_quant` suffix and takes three additional arguments not present in the V2L invocation: `-t` (SDK root for cross-compilation), `-c` (Quantizer location), and `--images` (calibration dataset). These reflect the INT8 quantization step that does not exist on the V2L path.

Compilation takes 2–5 minutes for the V2L path and 5–15 minutes for the V2H path (the V2H is slower because it runs calibration). On success, the script creates the output directory containing:

```
resnet18_onnx/
├── deploy.json          # graph structure
├── deploy.params        # weights (FP16 for V2L, INT8 for V2H)
├── deploy.so            # CPU-side runtime code
└── preprocess/          # DRP context binaries
    ├── aimac_desc.bin
    └── ...
```

(8) Build the inference application

The compiled model is data — it cannot run by itself. You need a small C++ application that links against the DRP-AI TVM runtime library, loads the compiled files, feeds input data, and reads back the output. The Renesas repository provides `tutorial_app` as a working example you can use as-is:

This release package ships SoC-specific DRP-AI kernel headers (drpai_v2l.h and drpai_v2h.h) instead of a generic drpai.h. Before building, alias the correct one for the target:

```
root@<id>:/# export SYSROOT_INC=/opt/poky_sdk/tmp/sysroots/rz-cmn/usr/include/linux/

# For RZ/V2H:
root@<id>:/# ln -sf $SYSROOT_INC/drpai_v2h.h $SYSROOT_INC/drpai.h

# For RZ/V2L:
root@<id>:/# ln -sf $SYSROOT_INC/drpai_v2l.h $SYSROOT_INC/drpai.h
```

```
root@<id>:/# cd $TVM_ROOT/apps
root@<id>:/drp-ai_tvm/apps# mkdir build && cd build

# For RZ/V2L:
root@<id>:/drp-ai_tvm/apps/build# cmake -DCMAKE_TOOLCHAIN_FILE=./toolchain/\
runtime.cmake ..
root@<id>:/drp-ai_tvm/apps/build# make -j$(nproc)

# For RZ/V2H (add -DV2H=ON):
root@<id>:/drp-ai_tvm/apps/build# cmake -DCMAKE_TOOLCHAIN_FILE=./toolchain\
/runtime.cmake -DV2H=ON ..
root@<id>:/drp-ai_tvm/apps/build# make -j$(nproc)
```

The CMake toolchain file uses the Yocto SDK's cross-compilation toolchain you installed in step 3.3, so it produces aarch64 binaries suitable for the target board. The output is a single binary named `tutorial_app` in the build directory.

The application source (`tutorial_app.cpp` in the apps directory) is roughly 200 lines of C++. Its job is to: read a BMP file from disk, format the pixel data for the model, call the TVM runtime API to load `deploy.so`, trigger inference, read back the output probabilities, look up the top class label, and print it.

The application code itself does not need to know whether it is running on a V2L or V2H — the runtime library abstracts that.

(9) Deploy to the board

Refer to **Quick Start** section and **Programming / Flashing Images** to bring up the supported board with one of the images from this release, preferably through microSD boot. The user needs to copy the model and application binaries into the root filesystem of the target board. One could simply copy the prepared artifacts to the SD card or use a flash drive with the artifacts to transfer and/or mount on the target device. In this section, we explore the network copy method and show you how to prepare all the artifacts on the host side and then copy them to the target. This is also the most practical method, as the artifacts are present inside the Docker container.

Note: It's possible to mount the host filesystem to a container and have the files accessible on the host machine side. However, that is beyond the scope of this guide as it leans into technically complex usage of Docker containers.

Once the board is up and you have network or serial console access, copy the artifacts over. The pre-built runtime libraries you need are different per target:

```
# From host, assuming the board is reachable at 192.168.1.10:

root@<id>:/# cd $TVM_ROOT/apps
root@<id>:/drp-ai_tvm/apps# mkdir -p tvm_deploy

# For RZ/V2L, copy V2L runtime:
root@<id>:/drp-ai_tvm/apps# cp $TVM_ROOT/obj/build_runtime/v2m/lib/* tvm_deploy/

# For RZ/V2H, copy V2H runtime libs:
root@<id>:/drp-ai_tvm/apps# cp $TVM_ROOT/obj/build_runtime/v2h/lib/* tvm_deploy/

# Common (same for both targets):
root@<id>:/drp-ai_tvm/apps# cp $TVM_ROOT/apps/exe/sample.bmp tvm_deploy/
root@<id>:/drp-ai_tvm/apps# cp $TVM_ROOT/apps/exe/synset_words_imagenet.txt \
tvm_deploy/
root@<id>:/drp-ai_tvm/apps# cp $TVM_ROOT/apps/build/tutorial_app* tvm_deploy/
root@<id>:/drp-ai_tvm/apps# cp -r $TVM_ROOT/tutorials/resnet18_onnx tvm_deploy/

# Transfer to the board:
root@<id>:/drp-ai_tvm/apps# tar czf tvm_deploy.tar.gz tvm_deploy/
root@<id>:/drp-ai_tvm/apps# scp tvm_deploy.tar.gz root@192.168.1.10:/home/root/
```

On the target board, unpack the archive into the expected location:

```
# On the target:
$ cd /home/root
$ tar xzf tvm_deploy.tar.gz
$ mv tvm_deploy tvm
```

The final structure on the board:

```
/home/root/tvm/
├── libtvm_runtime.so                    # (V2L)  OR  V2H lib files
├── libmera2_runtime.so                  # (V2L)  OR  V2H meta2 runtime lib
├── resnet18_onnx/                       # compiled model
├── sample.bmp                           # test input
├── synset_words_imagenet.txt  # class label list
└── tutorial_app (or tutorial_app_v2ml)    # the inference binary depends on target
board
```

(10) Run the Inference

The command for running is the same on both targets:

```
# Common (same for both targets)
root@rz-cmn:/# cd /home/root/tvm_deploy
root@rz-cmn:~/tvm_deploy# export LD_LIBRARY_PATH=\
/home/root/tvm_deploy:$LD_LIBRARY_PATH

# For RZ/V2H
root@rz-cmn:~/tvm_deploy# ./tutorial_app sample.bmp

# For RZ/V2L
root@rz-cmn:~/tvm_deploy# ./tutorial_app_v2ml sample.bmp
```

Expected output format — same on both targets, but with different latency:

```
[TVM Runtime] Loading model from resnet18_onnx
[TVM Runtime] Loaded model in 23 ms
[TVM Runtime] Running inference...

# On RZ/V2L:
[Inference] Completed in ~24 ms

# On RZ/V2H:
[Inference] Completed in ~6 ms

Result ----------------------
  Top 1 [ 66.0%] : [beagle]
  Top 2 [ 13.0%] : [English foxhound]
  Top 3 [ 11.4%] : [Walker hound, Walker foxhound]
  Top 4 [  5.9%] : [basset, basset hound]
  Top 5 [  0.7%] : [bloodhound, sleuthhound]
```

That output — a successful classification with non-trivial latency — is your end-to-end confirmation that the toolchain, model, application, and board are all working together. The ~4× latency difference between V2L and V2H reflects the 16× dense TOPS difference reduced by real-world I/O and memory overhead.

8.4.6.3 RZ/V2L DRP-AI

RZ/V2L DRP-AI cooperation with:

- MAC: Hardware for high-speed processing of 16-bit floating-point (FP16) matrix operations.
- Local memory (SRAM for this unit): Holds matrix data and the weight and bias values from the learning results.
- DMAC: Transfers matrix data and weight and bias values from DDR3L/DDR4 SDRAM to the local memory.
- Transfers the result of the multiply-and-accumulate operations output by the MAC to DDR3L/DDR4 SDRAM.

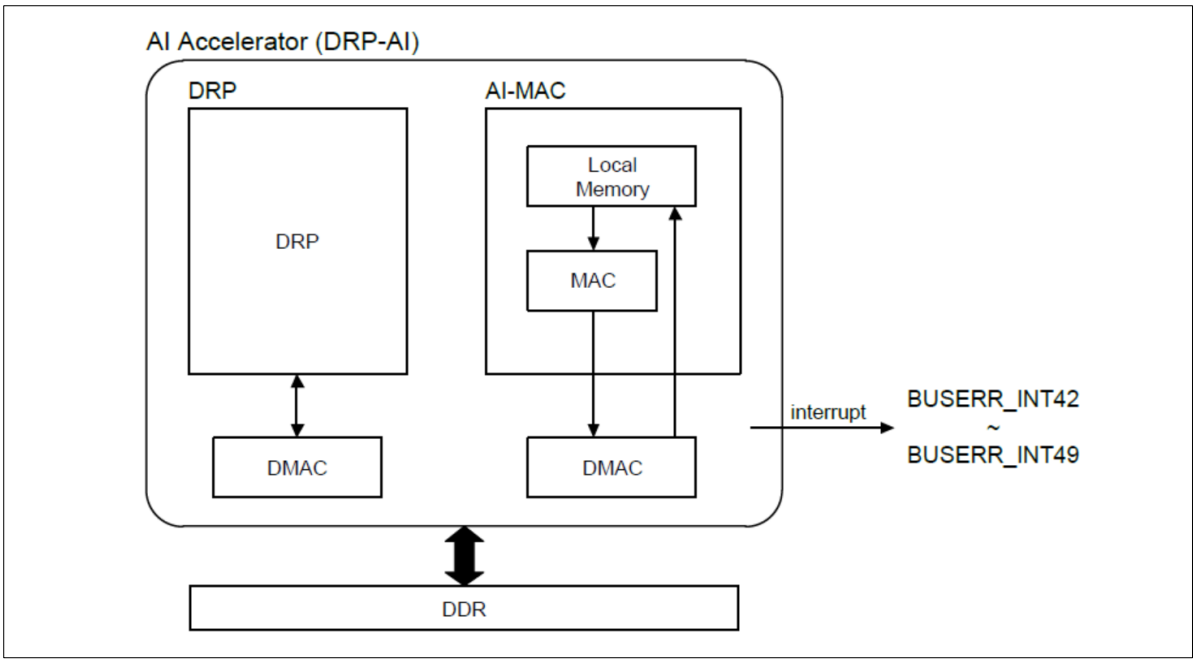


Figure 100. RZ/V2L DRP-AI (DRP0)

(1) API Overview

The RZ/V2L software interface is divided into three groups:

- DRP Driver APIs, which control DRP-side hardware operation,
- AI-MAC Driver APIs, which control AI-MAC-side processing, and
- Common DRP-AI APIs, which provide shared functions such as status monitoring and reset control.

The table below lists the DRP driver APIs for RZ/V2L. These functions are used to initialize DRP operation, start descriptor-based processing, handle completion or error interrupts, and reset the DRP block when required.

Table 43. RZ/V2L DRP/AI API

API	Description	Parameters	Return Value
R_DRPAI_DRP_Open	Initializes and enables the DRP hardware block. This includes clock enable, soft reset release, DMA channel setup, interrupt unmasking, and configuration initialization.	• void __iomem *drp_base_addr: DRP register base address • int32_t ch: DRP channel index • spinlock_t *lock: Spinlock for register protection	• R_DRPAI_SUCCESS: Initialization successful • R_DRPAI_ERR_INVALID_ARG: Invalid channel
R_DRPAI_DRP_Start	Starts DRP processing by setting the descriptor address and initiating descriptor prefetch.	• void __iomem *drp_base_addr: DRP register base address • int32_t chg.: DRP channel index • uint32_t desc: Descriptor start address	• R_DRPAI_SUCCESS: Start successful • R_DRPAI_ERR_INVALID_ARG: Invalid channel

R_DRPAI_DRP_Nmlint	Handles DRP normal-completion interrupts. Clears interrupt flags and reads ODIF interrupt counters.	• void __iomem *drp_base_addr: DRP register base address • int32_t ch: DRP channel index • drpai_odif_intcnto_t *odif_intcnto: Interrupt counter output	0 (fixed return value)
R_DRPAI_DRP_Errint	Handles DRP error interrupts. Dumps descriptor pointers and error status registers, and clears error conditions.	• void __iomem *drp_base_addr: DRP register base address • void __iomem *aimac_base_addr: AI-MAC base address • int32_t ch: DRP channel index	0 (fixed return value)
R_DRPAI_DRP_Reset	Performs a forced DRP reset sequence. Stops descriptor prefetch, halts DMA channels, clears sync tables, and issues a soft reset.	• void __iomem *drp_base_addr: DRP register base address • void __iomem *aimac_base_addr: AI-MAC base address • int32_t ch: DRP channel index spinlock_t *lock: Spinlock for reset protection	• R_DRPAI_SUCCESS: Reset successful • R_DRPAI_ERR_INVALID_ARG: Invalid channel • R_DRPAI_ERR_RESET: Reset timeout/failure

The table below lists the AI-MAC driver APIs for RZ/V2L. These functions are used to initialize the AI-MAC hardware, start processing, handle interrupts, and reset the AI-MAC block when necessary.

Table 44. RZ/V2L AI-MAC Driver APIs

API	Description	Parameters	Return Value
R_DRPAI_AIMAC_Open	Initializes the AI-MAC hardware block. Enables clocks, releases resets, configures DMA channels, and unmasks interrupts.	• void __iomem *aimac_base_addr: AI-MAC register base address int32_t ch: AI-MAC channel index	• R_DRPAI_SUCCESS: Initialization successful R_DRPAI_ERR_INVALID_ARG: Invalid channel
R_DRPAI_AIMAC_Start	Starts AI-MAC processing by setting the descriptor address and triggering descriptor fetch.	• void __iomem *aimac_base_addr: AI-MAC register base address int32_t ch: AI-MAC channel index uint32_t desc: Descriptor start address spinlock_t *lock: Spinlock for interrupt data protection	• R_DRPAI_SUCCESS: Start successful R_DRPAI_ERR_INVALID_ARG: Invalid channel
R_DRPAI_AIMAC_Nmlint	Handles AI-MAC normal-completion interrupts. Clears interrupt flags and captures completion status.	• void __iomem *aimac_base_addr: AI-MAC register base address int32_t ch: AI-MAC channel index	0 (fixed return value)

R_DRPAI_AIMAC_Errint	Handles AI-MAC error interrupts. Dumps descriptor pointers, error cause registers, and clears error status bits.	• void __iomem *drp_base_addr: DRP register base address void __iomem *aimac_base_addr: AI-MAC register base address int32_t ch: AI-MAC channel index	0 (fixed return value)
R_DRPAI_AIMAC_Reset	Performs a forced AI-MAC reset. Stops descriptor fetch, halts DMA channels across EXD blocks, clears sync tables, and resets internal state.	• void __iomem *aimac_base_addr: AI-MAC register base address int32_t ch: AI-MAC channel index	• R_DRPAI_SUCCESS: Reset successful R_DRPAI_ERR_INVALID_ARG: Invalid channel R_DRPAI_ERR_RESET: Reset timeout/failure

The table below lists common DRP-AI APIs. These functions provide access to runtime status information and common reset-related control shared by the DRP-AI subsystem.

Table 45. Common DRP-AI APIs

API	Description	Parameters	Return Value
R_DRPAI_Status	Retrieves DRP-AI runtime status including descriptor pointers, interrupt status, synchronization data, and error information.	• void __iomem *drp_base_addr: DRP register base address void __iomem *aimac_base_addr: AI-MAC register base address int32_t ch: Channel index drpai_status_t *drpai_status: Status output structure	• R_DRPAI_SUCCESS: Status read successful R_DRPAI_ERR_INVALID_ARG: Invalid channel
R_DRPAI_CPG_Reset	Controls reset of DRP-AI related clock/reset domains through the CPG reset controller.	• struct reset_control *rst_ctrl: Reset control handle	• R_DRPAI_SUCCESS: Success Negative error code: Failure

8.4.6.4 RZ/V2H DRP-AI

On RZ/V2H, the AI accelerator contains a DRP (DRP0) and an AI-MAC, and these are collectively referred to as DRP-AI. It uses the hierarchical memory system to perform AI inference and store the results.

The DRP-AI on RZ/V2H can execute AI inference independently of the CPU. In a typical execution flow, the DRP-AI driver reads descriptor information generated from a trained AI model and starts the hardware accordingly.

In addition to DRP-AI, RZ/V2H also includes a separate DRP1 block for computer vision processing acceleration. This block is distinct from the DRP-AI inference engine and can be used for image-processing operations.

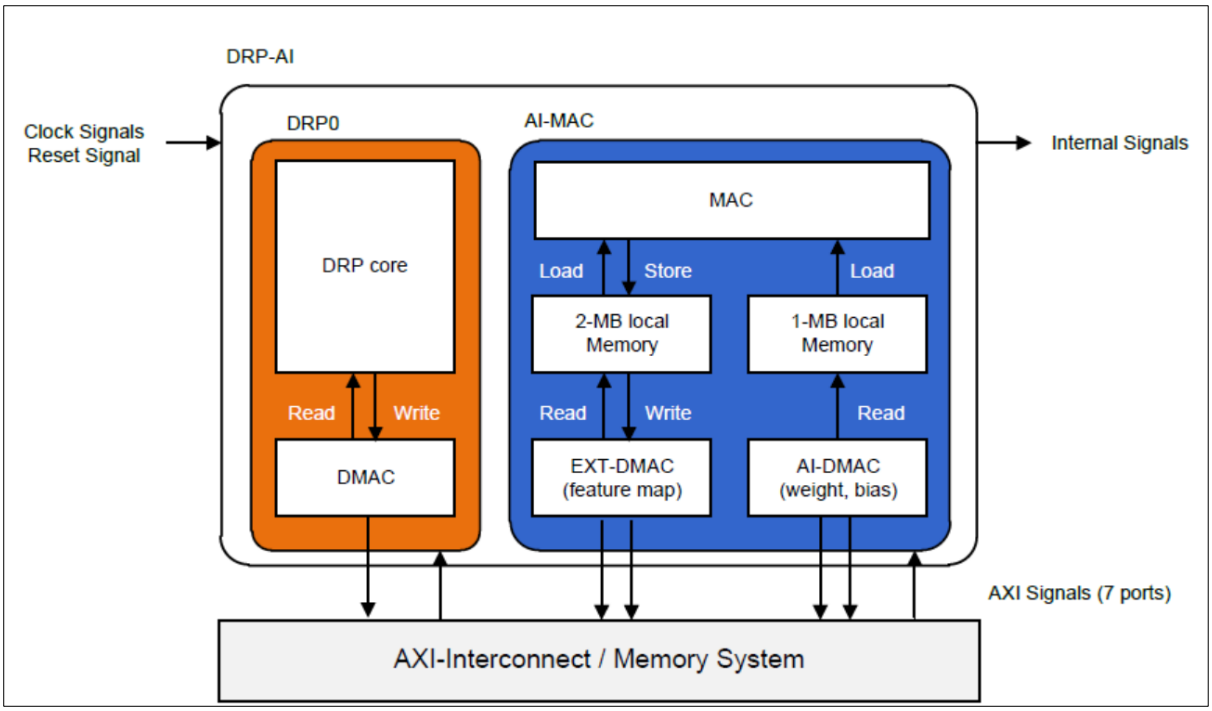


Figure 101. RZ/V2H DRP (DRP0)

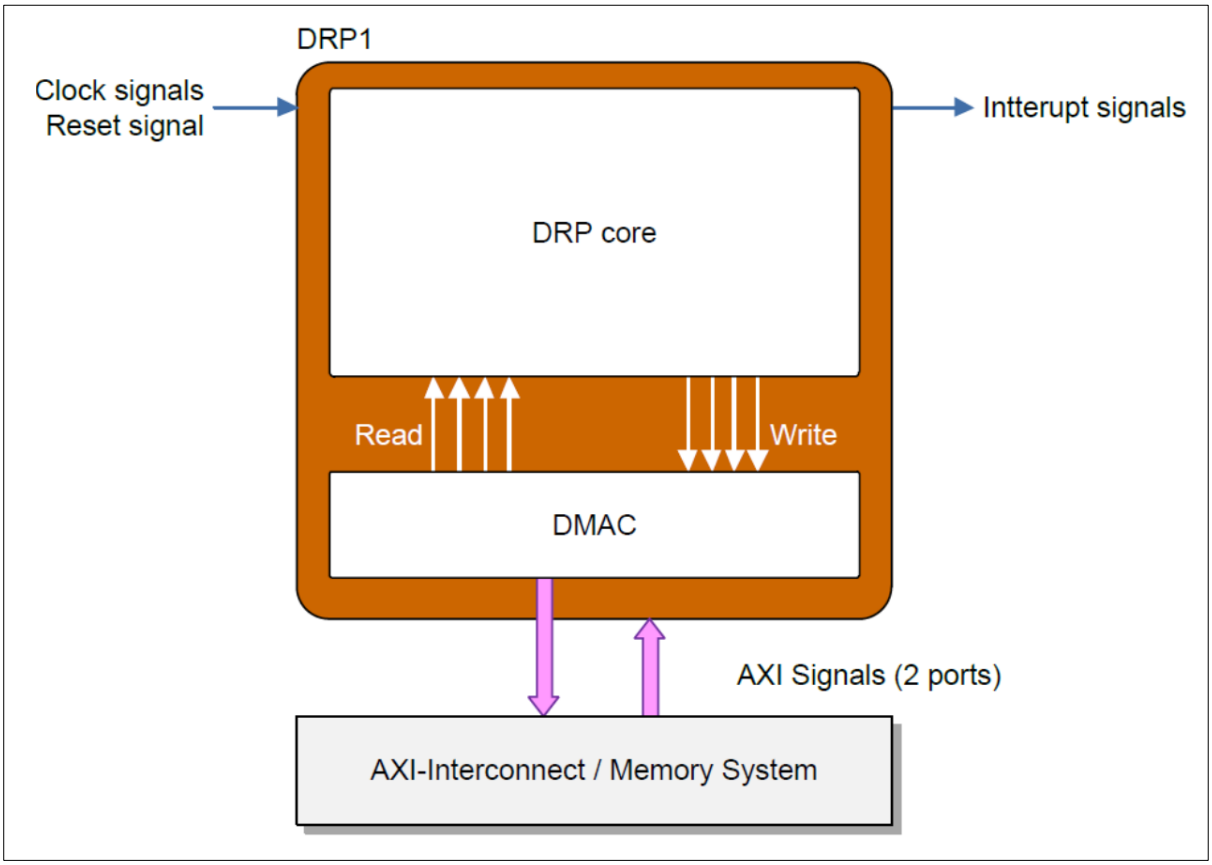


Figure 102. RZ/V2H DRP (DRP1)

The table below summarizes the main differences between RZ/V2H and RZ/V2L

Table 46. Differences in DRP aspects between RZ/V2H and RZ/V2L

Aspect	RZ/V2L	RZ/V2H
DRP-AI Composition	MAC + local SRAM + DMAC for FP16 matrix operations	Integrated DRP (DRP0) + AI-MAC as a unified DRP-AI accelerator
Primary Focus	High-speed FP16 matrix multiply-and-accumulate	General AI inference with hierarchical memory awareness
DRP Availability	N/A (Not available)	Additional DRP1 for computer vision acceleration
Execution Model	Fixed-function style processing driven mainly by MAC	Descriptor-driven execution using DRP-AI translator output
CPU Dependency	Heavily coordinated with CPU-side control	Can execute AI inference more independently of the CPU
Memory Architecture	Local SRAM tightly coupled to MAC + DMAC transfers	Hierarchical memory system with flexible data placement
Model Deployment	MAC-centric workflow	Translator-based workflow for model conversion and execution

This comparison is important because the software model on RZ/V2H is more descriptor-driven and includes a separate DRP block for vision acceleration, while RZ/V2L is more centered on MAC-based processing.

(1) API Overview

The table below lists the DRP driver APIs for RZ/V2H. These functions are used to initialize DRP operation, start descriptor processing, control address relocation, manage operating frequency, read or write registers, and recover from errors.

Table 47. RZ/V2H DRP Driver APIs

API	Description	Parameters	Return Value
R_V2H_DRPAI_DRP_Open	Initializes and boots the DRP hardware sequence.	• drp_base_addr: Base address of DRP register map • ch: DRP channel number	• void
R_V2H_DRPAI_DRP_Start	Sets the DRP descriptor and starts descriptor prefetch.	• drp_base_addr: Base address of DRP register map • ch: DRP channel number desc: Descriptor address	• void
R_DRPAI_DRP_Stop	Stops DRP descriptor prefetch.	• drp_base_addr: Base address of DRP register map ch: DRP channel number	• R_DRPAI_SUCCESS on success • R_DRPAI_ERR_STOP on failure

R_DRPAI_DRP_SetMaxFreq	Sets the maximum operating frequency divider for DRP.	• drp_base_addr: Base address of DRP register map ch: DRP channel number mindiv: Divider value (2–127)	• R_DRPAI_SUCCESS on success R_DRPAI_ERR_INVALID_ARG if out of range
R_DRPAI_DRP_EnableAddrReloc	Enables address relocation using a translation table.	• drp_base_addr: Base address of DRP register map tbl: Address relocation table pointer	• R_DRPAI_SUCCESS on success • R_DRPAI_ERR_INVALID_ARG if tbl is NULL
R_DRPAI_DRP_DisableAddrReloc	Disables DRP address relocation.	• drp_base_addr: Base address of DRP register map	• void
R_V2H_DRPAI_DRP_Reset	Stops DRP prefetch and finalizes DRP state.	• drp_base_addr: Base address of DRP register map ch: DRP channel number	• R_DRPAI_SUCCESS on success • R_DRPAI_ERR_RESET on failure
R_V2H_DRPAI_DRP_Errint	Handles DRP error interrupts and logs status.	• drp_base_addr: Base address of DRP register map aimac_base_addr: Base address of AI-MAC register map ch: DRP channel number	• void
R_DRPAI_DRP_RegRead	Reads a DRP register value.	• drp_base_addr: Base address of DRP register map offset: Register offset pvalue: Pointer to store read value	• R_DRPAI_SUCCESS on success • R_DRPAI_ERR_INVALID_ARG if pvalue is NULL
R_DRPAI_DRP_RegWrite	Writes a value to a DRP register.	• drp_base_addr: Base address of DRP register map offset: Register offset value: Value to write	• void
R_DRPAI_DRP_IsActFieldOfDsccDctlZero	Checks if the ACT field of DSCC_DCTL is zero.	• drp_base_addr: Base address of DRP register map	• 0 if zero • -1 if not zero

The table below lists the AI-MAC driver APIs for RZ/V2H. These functions initialize the AI-MAC block, start command and parameter descriptor processing, control address relocation, manage frequency, handle interrupts, and support error recovery.

Table 48. RZ/V2H AI-MAC Driver APIs

API	Description	Parameters	Return Value
R_V2H_DRPAI_AIMAC_Open	Initializes and boots the AI-MAC hardware sequence.	• aimac_base_addr: Base address of AI-MAC register map	• void

		ch: AI-MAC channel number	
R_V2H_DRPAI_AIMAC_Start	Sets command and parameter descriptors and starts prefetch.	• aimac_base_addr: Base address of AI-MAC register map ch: AI-MAC channel number cmd_desc: Command descriptor address param_desc: Parameter descriptor address	• void
R_DRPAI_AIMAC_Stop	Stops AI-MAC descriptor prefetch.	• aimac_base_addr: Base address of AI-MAC register map ch: AI-MAC channel number	• R_DRPAI_SUCCESS on success R_DRPAI_ERR_STOP on failure
R_DRPAI_AIMAC_SetFreq	Sets AI-MAC operating frequency divider.	• aimac_base_addr: Base address of AI-MAC register map ch: AI-MAC channel number divfix: Divider value (1–127)	• R_DRPAI_SUCCESS on success R_DRPAI_ERR_INVALID_ARG if out of range
R_DRPAI_AIMAC_EnableAddrReloc	Enables AI-MAC address relocation using a translation table.	• aimac_base_addr: Base address of AI-MAC register map tbl: Address relocation table pointer	• R_DRPAI_SUCCESS on success R_DRPAI_ERR_INVALID_ARG if tbl is NULL
R_DRPAI_AIMAC_DisableAddrReloc	Disables AI-MAC address relocation.	• aimac_base_addr: Base address of AI-MAC register map	• void
R_V2H_DRPAI_AIMAC_Reset	Stops prefetch and finalizes AI-MAC state.	• aimac_base_addr: Base address of AI-MAC register map ch: AI-MAC channel number	• R_DRPAI_SUCCESS on success R_DRPAI_ERR_RESET on failure
R_V2H_DRPAI_AIMAC_Nmlint	Handles AI-MAC normal interrupts and collects status.	• aimac_base_addr: Base address of AI-MAC register map ch: AI-MAC channel number reserved: Status output buffer	• R_DRPAI_SUCCESS on success R_DRPAI_ERR_INT on abnormal status
R_V2H_DRPAI_AIMAC_Errint	Handles AI-MAC error interrupts and logs status.	• drp_base_addr: Base address of DRP register map aimac_base_addr: Base	• void

		address of AI-MAC register map ch: AI-MAC channel number	
R_DRPAI_AIMAC_RegRead	Reads an AI-MAC register value.	• aimac_base_addr: Base address of AI-MAC register map offset: Register offset pvalue: Pointer to store read value	• R_DRPAI_SUCCESS on success R_DRPAI_ERR_INVALID_ARG if pvalue is NULL
R_DRPAI_AIMAC_RegWrite	Writes a value to an AI-MAC register.	• aimac_base_addr: Base address of AI-MAC register map offset: Register offset value: Value to write	• void

The table below lists the common DRP-AI APIs for RZ/V2H. These functions provide runtime status and diagnostic access for the DRP-AI subsystem.

Table 49. Common / DRP-AI APIs

API	Description	Parameters	Return Value
R_V2H_DRPAI_Status	Reads DRP-AI status and diagnostic registers.	• drp_base_addr: Base address of DRP register map aimac_base_addr: Base address of AI-MAC register map ch: Channel number reserved: Output status buffer	• void

8.5 Multi-OS / Heterogeneous Multi-Core Operation

This section provides an overview of the Multi-OS support provided by this release package for Renesas RZ/G2L, RZ/V2L, and RZ/V2H platforms.

The Renesas SoC contains several processor cores that fundamentally differ in architecture and purpose. Such an architecture of dissimilar processor cores is called a Heterogenous Multicore processor commonly abbreviated as HMP. Most of the RZ line of processors are HMP cores except for a few specific products.

Most of our RZ processors have three primary core architectures from Arm:

Cortex A55 (1 to 4 cores) — Primary application processor that runs the Linux operating system and the primary business application.

Cortex M33 (1 to 2 cores) — Secondary processor that is meant to implement system monitoring and soft real-time applications.

Cortex R8 (1 to 2 cores) — Secondary processor that is meant for real-time applications. This core is optimized with lockstep and other design elements to ensure real-time and fault tolerant performance of the deployed firmware.

While all these cores are somewhat independent, they do share memory.

Linux runs on the primary Cortex-A55 processor cores. A separate firmware image can run on another processor core, such as Cortex-M33 or Cortex-R8. Linux and the remote firmware are independent, but they can exchange messages using shared memory. This is useful when Linux handles large application tasks and the remote firmware handles smaller real-time or hardware-control tasks and they must exchange the data.

In this release, Multi-OS means that Linux runs on the Cortex-A55 cores while another independent software image runs on a remote processor core.

The remote software may be:

- an RTOS, such as FreeRTOS, or
- bare-metal firmware.

More technically, this is a heterogeneous multi-core AMP architecture.

AMP means Asymmetric Multiprocessing. In an AMP system, different processor cores run independent software images. These software images communicate using mechanisms such as shared memory, mailbox interrupts, OpenAMP, and RPMsg.

A simplified view is shown below:

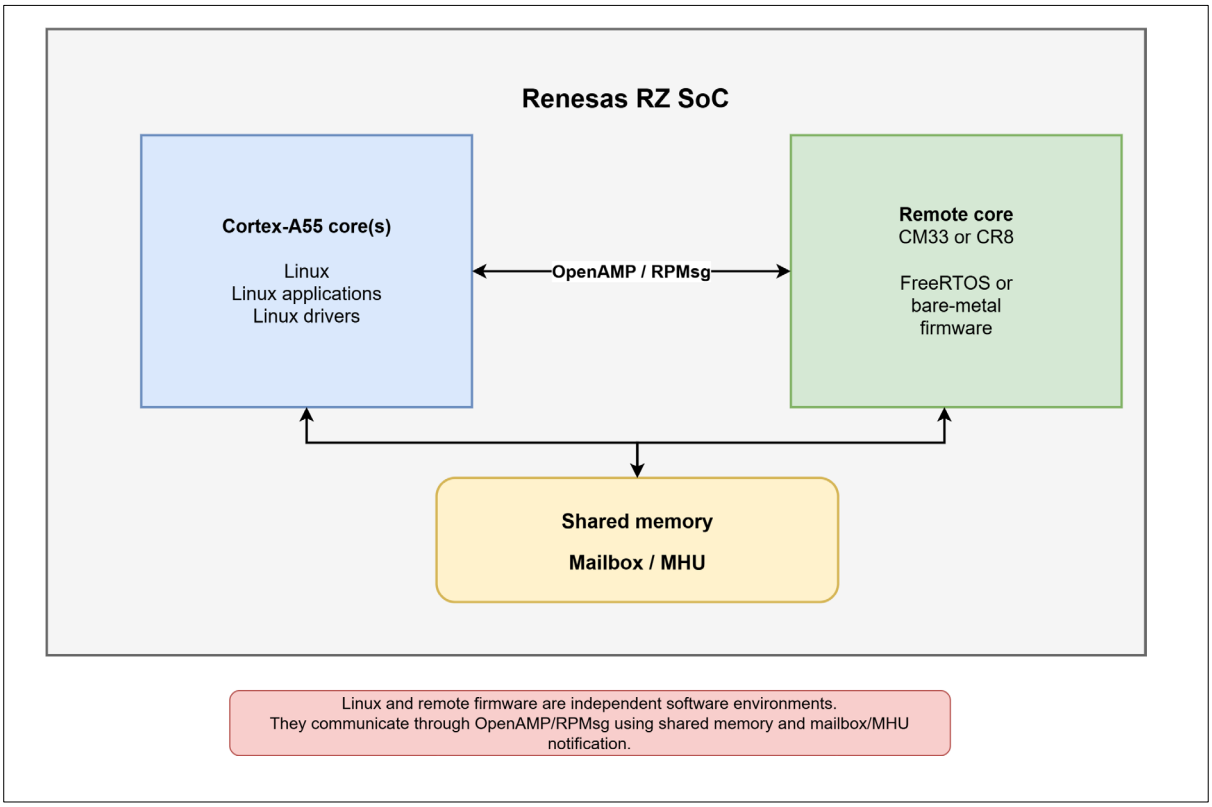


Figure 103. Linux and remote firmware operation on Renesas RZ SoC

Linux and the remote firmware do not run as one operating system. They are separate software environments running on different processor cores.

8.5.1 Supported Devices and Core Mapping

The available remote cores depend on the target device in this release.

Table 50. Supported RZ devices and Core mapping

Device	Linux Core	Remote Firmware Core
RZ/G2L	Cortex-A55	Cortex-M33
RZ/V2L	Cortex-A55	Cortex-M33
RZ/V2H	Cortex-A55	Cortex-M33, Cortex-R8 core 0, Cortex-R8 core1

RZ/G2L and RZ/V2L support a Cortex-M33 remote processor for AMP operation.

RZ/V2H supports a Cortex-M33 and Cortex-R8 cores. Depending on the sample application and configuration, the remote firmware may run on one of these remote cores.

Not every sample application necessarily supports every remote core. Refer to the corresponding sample application section for the exact supported board, core, and firmware combination.

8.5.2 Overview

Multi-OS / AMP operation is useful when one embedded product needs both a high-level Linux environment and a smaller independent firmware environment.

For example, consider an AI inspection camera used in a factory. The camera is installed beside a conveyor belt. Products pass in front of the camera one by one. The system captures an image, runs AI inference, determines whether the product is acceptable, logs the result, and may send the result to another factory system over Ethernet.

Linux is a good fit for these high-level tasks because it provides camera frameworks, AI software support, file systems, networking, graphics, and user-space applications.

At the same time, the product may also need a smaller dedicated firmware task. For example, firmware running on a Cortex-M33 or Cortex-R8 core may monitor a trigger sensor, control an LED tower light, drive a buzzer, or output a signal to an external reject mechanism when Linux reports a failed inspection result.

In this type of system, Linux handles complex application-level work, while the remote firmware handles focused hardware-control tasks.

8.5.3 Basic Processing Models

The terms “multi-core,” “SMP,” “AMP,” and “Multi-OS” are related, but they do not mean the same thing. In addition, this section uses the terms “remote processor” and “remote firmware” when describing the independent software running outside the Linux domain.

Table 51. Multi-core, SMP, AMP, Multi-OS, and related remote processor terms

Term	Meaning	Example
Multi-core	The SoC has more than one CPU core	RZ/V2H has Cortex-A55, Cortex-M33, and Cortex-R8 cores

SMP	One operating system controls multiple similar processor cores.	One Linux kernel runs on multiple Cortex-A55 cores
AMP	Different cores run independent software images	Linux on Cortex-A55, firmware on CM33
Multi-OS	Different cores run different operating systems	Linux on Cortex-A55, FreeRTOS on CM33
Remote Processor	A processor core that runs independently from Linux. In this release, the remote processor is a Cortex-M33 or Cortex-R8 core.	CM33 on RZ/G2L, CM33 on RZ/V2L, CM33 or CR8 on RZ/V2H.
Remote Firmware	The software image executed by the remote processor. It may be bare-metal firmware or RTOS-based firmware.	FreeRTOS firmware on CM33, bare-metal control firmware on CR8.
Bare-metal firmware	Firmware that runs directly on the remote processor hardware without an operating system.	A CM33 firmware image that initializes hardware and processes RPMsg commands without FreeRTOS.
RTOS firmware	Firmware that runs on the remote processor with a real-time operating system.	A FreeRTOS-based RPMsg example running on CM33.

In this release, the term Multi-OS is used broadly for Linux running on Cortex-A55 together with independent software running on a remote processor. If the remote software uses an RTOS such as FreeRTOS, the system is strictly Multi-OS. If the remote software is bare-metal firmware, the system is still AMP, although it is not strictly multiple operating systems.

In the following sections, the term remote processor refers to a Cortex-M33 or Cortex-R8 core that runs independently from Linux, and remote firmware refers to the software image executed on that core.

8.5.3.1 Multi-core

Multi-core means that the SoC contains more than one processor core.

For example, an RZ/V2H device contains:

- Cortex-A cores,
- Cortex-M33 core,
- Cortex-R8 cores.

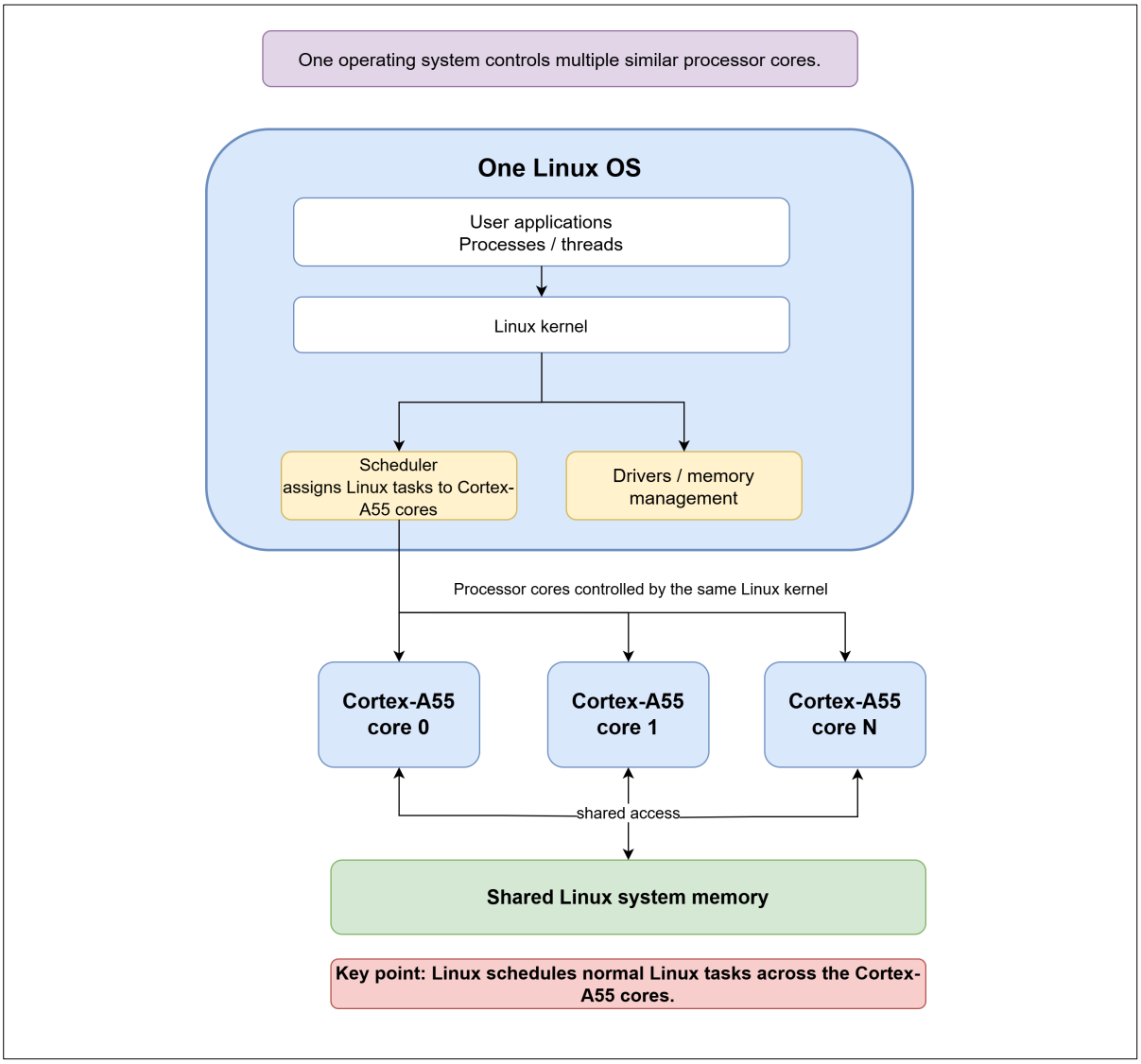
Multi-core is a hardware feature. By itself, it does not define how the software is organized, or it does not automatically mean Multi-OS.

A multi-core device may run software in different ways. It may run one operating system across several similar cores, or it may run different software images on different cores.

8.5.3.2 SMP

SMP stands for Symmetric Multiprocessing. In an SMP system, one operating system controls multiple similar processor cores.

A common example is Linux running on multiple Cortex-A55 cores. The following figure shows the SMP Linux model



In this model, there is one Linux operating system. Linux applications run as normal Linux processes or threads. These applications are managed by the Linux kernel.

The Linux kernel includes components such as:

- The scheduler
- Device drivers
- Memory management

The Linux scheduler assigns Linux tasks to the available Cortex-A55 cores. All participating Cortex-A55 cores are controlled by the same Linux kernel and share the same Linux system memory.

This is multi-core operation, but it is not Multi-OS, because there is only one operating system: Linux.

8.5.3.3 AMP

AMP stands for Asymmetric Multiprocessing. In an AMP system, different processor cores run independent software images.

This is different from SMP. In SMP, one operating system, such as Linux, controls and schedules work across multiple similar cores. In AMP, each processor domain runs its own software image, and Linux does not schedule normal Linux tasks on the remote core.

This is the architecture used by the Multi-OS examples in this release package.

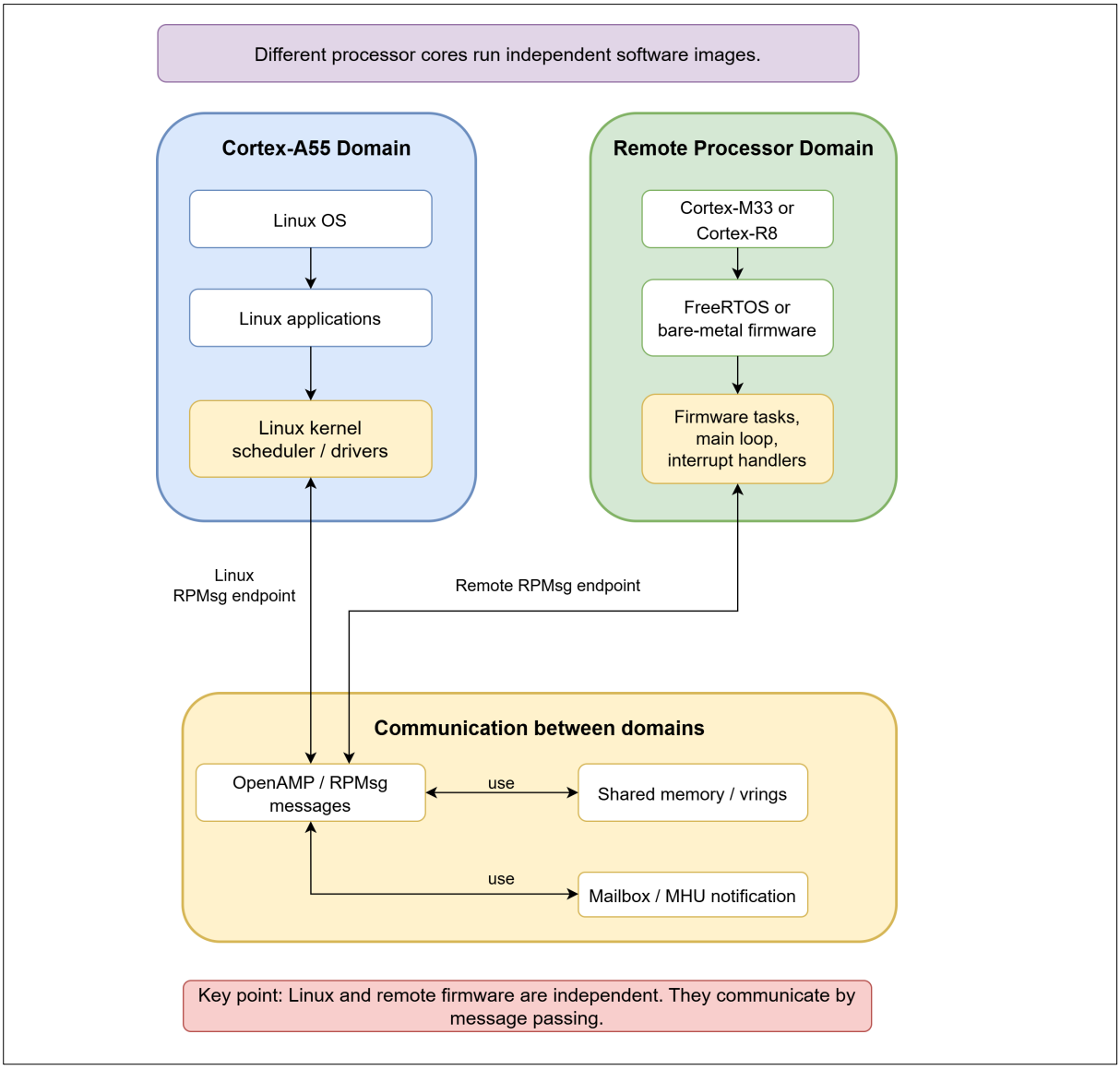


Figure 104. Multi-OS / AMP- Asymmetric Multiprocessing model

In this model, the system is divided into separate processor domains.

The Cortex-A55 domain runs Linux. This domain contains Linux applications, the Linux kernel, Linux scheduler, and Linux drivers.

The remote processor domain runs its own firmware image. Depending on the device and sample application, the remote core may be Cortex-M33 or Cortex-R8. The remote firmware may use FreeRTOS or may be bare-metal firmware.

Linux does not schedule normal Linux tasks on the Cortex-M33 or Cortex-R8 core. The remote processor is not used as an extra Linux CPU. Instead, the remote core runs its own independent firmware.

Because the Linux domain and the remote firmware domain are independent, they communicate through a defined mechanism such as OpenAMP/RPMsg.

8.5.3.4 Multi-OS

Multi-OS means that different processor cores run different operating system environments.

This is Multi-OS because Linux and FreeRTOS are separate operating system environments.

If the remote side runs bare-metal firmware instead of an RTOS, the system is still AMP, but it is not strictly multiple operating systems.

In this release, the term Multi-OS is used broadly to describe this system model: Linux on Cortex-A55 and independent firmware on a remote processor core

The remote firmware may use FreeRTOS or may be bare-metal firmware.

8.5.4 Communication Concepts

The AMP model introduced earlier showed that Linux and the remote firmware communicate through OpenAMP/RPMsg

This section expands that communication path and explains the main components used for message passing between the Linux domain and the remote firmware domain.

- Shared memory,
- Mailbox or MHU notification,
- OpenAMP,
- RPMsg,
- Vrings.

A simplified communication model is shown below:

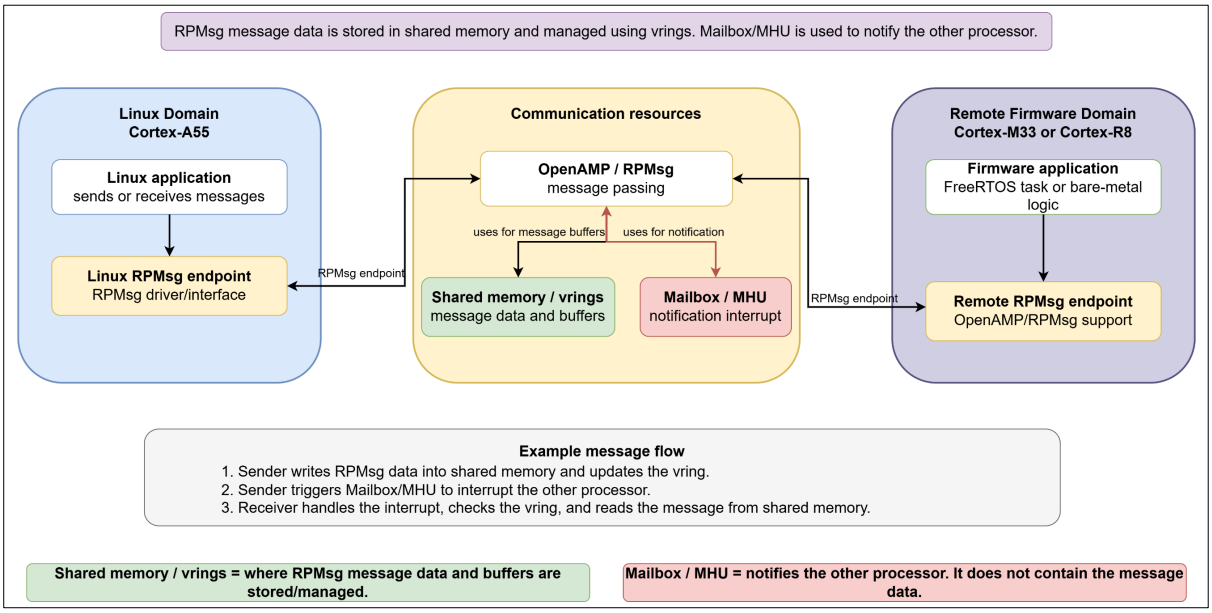


Figure 105. OpenAMP/RPMsg communication between Linux and remote firmware

The figure shows how messages are exchanged between the Linux domain and the remote firmware domain.

On the Linux side, a Linux application sends or receives messages through a Linux RPMsg endpoint or RPMsg driver interface.

On the remote firmware side, the firmware application sends or receives messages through a remote RPMsg endpoint provided by the OpenAMP/RPMsg firmware support.

The actual message data is placed in shared memory. RPMsg uses vrings to manage the message buffers in that shared memory.

After a message is placed in shared memory, mailbox/MHU notification is used to interrupt the other processor. The mailbox/MHU does not contain the message data. It only tells the other processor to check the shared memory and vring.

The following sections explain each communication concept.

8.5.4.1 OpenAMP

OpenAMP is a software framework used for communication between processors.

In this package, OpenAMP helps Linux communicate with firmware running on Cortex-M33 or Cortex-R8.

OpenAMP is not the message payload itself. It is the framework that helps the Linux side and the remote firmware side communicate in a structured way.

OpenAMP commonly works together with:

- RPMsg,
- Shared memory,
- Resource tables,
- Mailbox or interrupt mechanisms,
- Remote processor management.

8.5.4.2 RPMsg

RPMsg stands for Remote Processor Messaging. RPMsg is the message-passing mechanism used between Linux and the remote firmware.

The message exchange can be represented as follows:

1. Linux sends a message.
2. Remote firmware receives it.
3. Remote firmware sends a reply.
4. Linux receives the reply.

RPMsg allows software on the Linux side and software on the remote firmware side to exchange messages. The two sides are not two normal Linux processes running under the same Linux kernel. They are separate software environments running on different processor cores.

Underneath RPMsg, the system uses shared memory, vrings, and notification mechanisms such as mailbox/MHU.

For example, Linux may send a simple command to the remote firmware:

“TURN_LED_ON”

The remote firmware may process the command and send back:

“OK”

The exact message format depends on the sample application or product design.

8.5.4.3 Shared memory

Shared memory is a memory region that can be accessed by both processor domains.

That means:

- Linux can access the shared memory region.
- The remote firmware can also access the shared memory region.
- RPMsg uses this shared memory to store message buffers.

Shared memory is where the message data is placed.

However, shared memory alone is not enough. The receiving processor also needs to know that a new message is available. That notification is usually done by a mailbox or MHU interrupt.

8.5.4.4 Mailbox / MHU

Shared memory stores the RPMsg message data, but shared memory alone does not notify the other processor that new data is available. Therefore, a separate notification mechanism is needed. This notification mechanism is provided by a mailbox or MHU (Message Handling Unit).

In this OpenAMP/RPMsg communication model, the full RPMsg message payload is stored in shared memory. The mailbox/MHU is used to notify the other processor that it should check the shared memory and vring.

Depending on the platform implementation, the mailbox/MHU notification may also include a small value, such as:

- Vring ID
- An event ID,
- A channel ID,
- Another small notification value.

This small value helps the receiving side know what communication resources to check. However, the full RPMsg message data is not carried in the mailbox/MHU.

Conceptually:

- Shared memory = where the message is stored
- Mailbox / MHU = the notification that a message is ready

The next section explains vrings, which are used by RPMsg to manage message buffers in shared memory.

8.5.4.5 Vring

A vring is a virtio ring buffer used by RPMsg. It can be understood as a shared queue used to manage RPMsg message buffers between Linux and the remote firmware.

RPMsg uses vrings in shared memory to manage which message buffers are:

- available,
- used,
- ready to be processed.

Usually, RPMsg communication uses two message paths:

- Linux to remote firmware,
- remote firmware to Linux.

The following figure shows the basic vring concept.

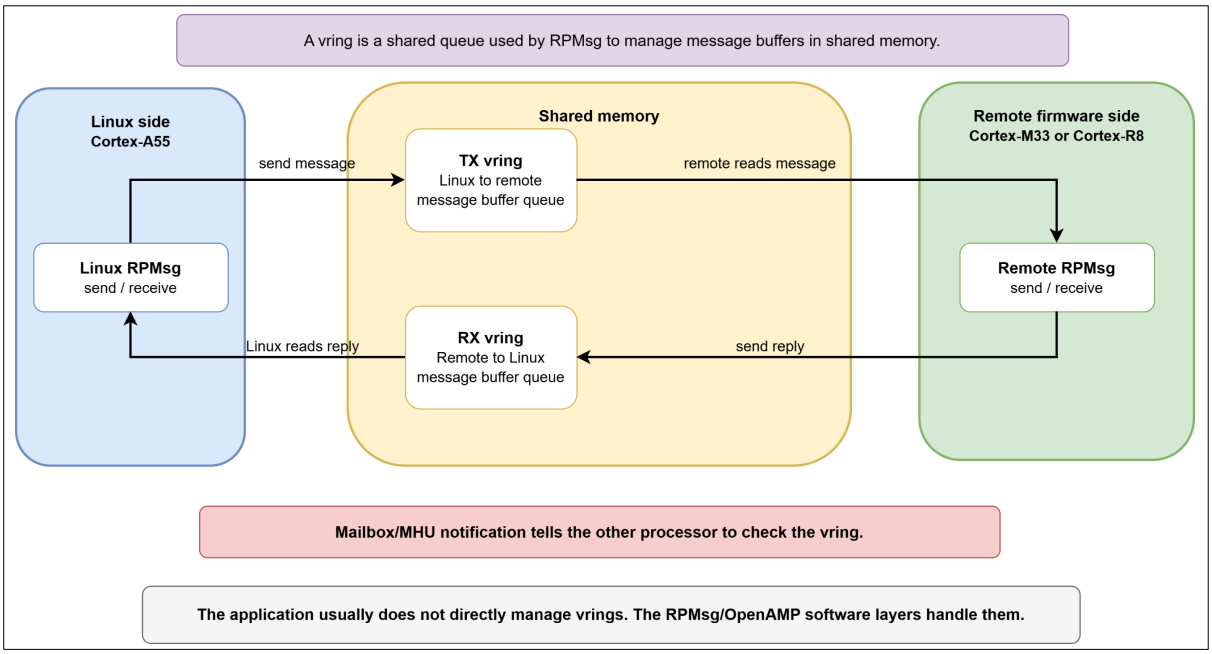


Figure 106. RPMsg vring concept

In the figure, one vring is used for messages sent from Linux to the remote firmware. Another vring is used for messages sent from the remote firmware back to Linux.

The application usually does not directly manage the vring. The RPMsg/OpenAMP software layers handle it.

8.5.5 Firmware Load Methods

Before Linux can communicate with a remote processor, the firmware for that processor must be prepared, loaded into the correct memory region, and the remote processor must be started.

In this section, remote firmware means the executable software image used by the remote processor. The remote firmware may be:

- bare-metal firmware, or
- RTOS-based firmware, such as FreeRTOS.

Depending on the target processor and how the firmware was built, the firmware may be provided in different binary forms. For example, it may be provided as:

- a single binary file,
- multiple binary files split into secure and non-secure regions for Cortex-M33,
- or multiple binary files split by memory section, such as ITCM, SRAM, and SDRAM, for Cortex-R8.

These binary files must match the memory map used when the firmware was built. Therefore, the load addresses described in this section must not be changed unless the firmware is rebuilt with a corresponding memory layout.

For this release package, the required firmware binaries for the Multi-OS demonstration have already been prepared. These binaries are provided for the supported remote processors and are used by the loading methods described in the following sections.

Depending on the target device, the prepared firmware may include:

- CM33 firmware for RZ/G2L and RZ/V2L,
- CM33 firmware for RZ/V2H,
- CR8 core 0 firmware for RZ/V2H,
- CR8 core 1 firmware for RZ/V2H.

These firmware images are used together with the Linux RPMsg sample applications to demonstrate communication between Linux and the remote processors.

This release package supports two firmware loading methods:

- loading firmware by U-Boot,
- loading firmware by Linux.

The method to use depends on when the remote firmware must start and how the system is being used.

- U-Boot loading is suitable when the remote firmware must start early, or when the product boot flow requires the remote core to be running before Linux user-space software starts.
- Linux loading is suitable when firmware is being developed, tested, restarted, or replaced more frequently.

In addition to firmware loading, the complete control flow may include:

- loading the firmware image,
- starting the remote processor,
- stopping the remote processor,
- restarting the remote processor,
- reloading a modified firmware image.

Where Linux remoteproc support is available, these control operations can be performed from Linux user space.

If remote firmware creation or modification is required instead of using the prebuilt demonstration binaries included in this release package, follow the firmware development process described in Section **Creating Remote RPMsg Firmware**.

8.5.5.1 Load firmware by U-Boot

In this method, U-Boot loads the prepared firmware binary files into memory and starts the remote processor before Linux runs the user application.

The figure below shows boot flow of this method:

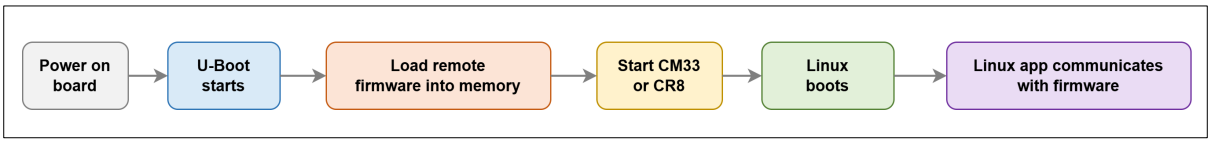


Figure 107. U-Boot Firmware Loading Flow

This method is suitable when:

- the remote firmware must start early,
- the product boot flow always requires the remote core to be running,

- early hardware initialization or monitoring is required.

To load remote firmware into memory in U-Boot. First power on the board, then stop the board at the U-Boot prompt:

```
Hit any key to stop autoboot
=>
```

The following subsections provide the exact commands and load addresses for each supported target.

(1) MMC device used by U-Boot

In the U-Boot environment, *mmcdev* is the variable that identifies the MMC device used for boot media access on the current board.

The value of *mmcdev* depends on the target board. The firmware loading commands in this section use *$mmcdev* so that the same command sequence can be reused across boards without manually editing the MMC device number in each command.

For reference, the current board mapping is:

Table 52. mmcdev mapping based on supported board

Target board	mmcdev
RZ/G2L-SBC, RZ/V2H-EVK, RZ/V2H-RDK	0
RZ/G2L-EVK, RZ/V2L-EVK, RS-G2L100, IMDT-V2H-SBC	1

(2) RZ/G2L and RZ/V2L CM33 firmware

The Cortex-M33 supports Arm TrustZone, so the prepared firmware is divided into secure and non-secure parts. Each part contains a vector table and a code section.

- The vector table contains the reset and interrupt entry information used when the firmware starts or handles exceptions.
- The code section contains the executable instructions for that firmware region.

As a result, four binary files are provided:

- secure vector,
- secure code,
- non-secure vector,
- non-secure code

The firmware files and load addresses are:

Table 53. RZ/G2L and RZ/V2L CM33 firmware files load address

Firmware File	Load Address
rzg2l_cm33_rpmsg_linux_rtos_example_secure_vector.bin	0x0001FF80
rzg2l_cm33_rpmsg_linux_rtos_example_secure_code.bin	0x42EFF440
rzg2l_cm33_rpmsg_linux_rtos_example_non_secure_vector.bin	0x00010000
rzg2l_cm33_rpmsg_linux_rtos_example_non_secure_code.bin	0x40010000

If needed, the current value can be checked at the U-Boot prompt:

```
=> printenv mmcdev
```

Use the following commands to load the firmware into memory region:

```
=> setenv cm33start 'dcache off;
fatload mmc $mmcdev:1 0x0001FF80 rzg2l_cm33_rpmsg_linux_rtos_example_secure_vector.bin;
fatload mmc $mmcdev:1 0x42EFF440 rzg2l_cm33_rpmsg_linux_rtos_example_secure_code.bin;
fatload mmc $mmcdev:1 0x00010000 rzg2l_cm33_rpmsg_linux_rtos_example_non_secure_vector.bin;
fatload mmc $mmcdev:1 0x40010000 rzg2l_cm33_rpmsg_linux_rtos_example_non_secure_code.bin;
cm33 start_debug 0x1001FF80 0x00010000;
dcache on;'
=> saveenv
=> run cm33start
```

This sequence prepares CM33, loads the firmware sections into the required memory regions, and then continues the normal Linux boot flow.

```
=> boot
```

(3) RZ/V2H CM33 firmware

For RZ/V2H, the CM33 firmware is provided as a single binary for U-Boot loading.

The firmware file and load address are:

Table 54. RZ/V2H CM33 firmware file load address

Firmware File	Load Address
rzv2h_cm33_rpmsg_linux_rtos_example.bin	0x08001e00

Use the following U-Boot commands to load this firmware file into memory region:

```
=> setenv cm33start 'dcache off; \
mw.l 0x10420D2C 0x02000000; \
mw.l 0x1043080c 0x08003000; \
mw.l 0x10430810 0x18003000; \
mw.l 0x10420604 0x00040004; \
mw.l 0x10420C1C 0x00003100; \
mw.l 0x10420C0C 0x00000001; \
mw.l 0x10420904 0x00380008; \
mw.l 0x10420904 0x00380038; \
fatload mmc $mmcdev:1 0x08001e00 rzv2h_cm33_rpmsg_linux_rtos_example.bin; \
mw.l 0x10420C0C 0x00000000; \
dcache on;'
=> saveenv
=> run cm33start
=> boot
```

The mw.l commands write values directly to SoC control registers. These register writes prepare the remote core and related memory or reset settings. The fatload command then loads the CM33 firmware into the required memory location.

(4) RZ/V2H CR8 firmware

RZ/V2H provides two Cortex-R8 remote processor cores:

- CR8 core 0
- CR8 core 1

The prepared firmware structure is the same for both cores. Each core uses three binary files corresponding to the itcm, sram, and sdram memory sections. However, the firmware file names and load addresses are different for core 0 and core 1.

In this naming:

- itcm refers to Instruction Tightly Coupled Memory (ITCM), used for instruction storage and fast code execution,
- sram refers to on-chip SRAM,
- sdram refers to external system memory (SDRAM).

These load addresses are fixed by the prepared firmware image and should not be changed unless the firmware is rebuilt with a matching memory layout.

For CR8 core 0, use the following firmware files and load addresses:

Table 55. RZ/V2H CR8 core 0 firmware files load addresses

Firmware File	Load Address
rzv2h_cr8_core0_rpmsg_linux_rtos_example_itcm.bin	0x12040000
rzv2h_cr8_core0_rpmsg_linux_rtos_example_sram.bin	0x08180000
rzv2h_cr8_core0_rpmsg_linux_rtos_example_sdram.bin	0x40800000

Use the following U-Boot commands:

```
=> setenv cr80start 'dcache off; \
mw.l 0x10420D24 0x04000000; \
mw.l 0x10420600 0xE000E000; \
mw.l 0x10420604 0x00030003; \
mw.l 0x10420908 0x1FFF0000; \
mw.l 0x10420C44 0x003F0000; \
mw.l 0x10420C14 0x00000000; \
mw.l 0x10420908 0x10001000; \
mw.l 0x10420C48 0x00000020; \
mw.l 0x10420908 0x1FFF1FFF; \
mw.l 0x10420C48 0x00000000; \
fatload mmc $mmcdev:1 0x12040000 rzv2h_cr8_core0_rpmsg_linux_rtos_example_itcm.bin;\
fatload mmc $mmcdev:1 0x08180000 rzv2h_cr8_core0_rpmsg_linux_rtos_example_sram.bin;\
fatload mmc $mmcdev:1 0x40800000 rzv2h_cr8_core0_rpmsg_linux_rtos_example_sdram.bin;\
mw.l 0x10420C14 0x00000003; \
dcache on;'
=> saveenv
=> run cr80start
```

This sequence prepares CR8 core 0, loads the three firmware sections into the required memory regions, and then continues the normal Linux boot flow.

For CR8 core 1, use the following firmware files and load addresses

Table 56. RZ/V2H CR8 core 1 firmware files load addresses

Firmware File	Load Address
rzv2h_cr8_core1_rpmsg_linux_rtos_example_itcm.bin	0x12080000
rzv2h_cr8_core1_rpmsg_linux_rtos_example_sram.bin	0x081C0000
rzv2h_cr8_core1_rpmsg_linux_rtos_example_sdram.bin	0x41800000

Use the following U-Boot commands:

```
=> setenv cr81start 'dcache off; \
mw.l 0x10420D24 0x04000000; \
mw.l 0x10420600 0xE000E000; \
mw.l 0x10420604 0x00030003; \
mw.l 0x10420908 0x1FFF0000; \
mw.l 0x10420C44 0x003F0000; \
mw.l 0x10420C14 0x00000000; \
mw.l 0x10420908 0x10001000; \
mw.l 0x10420C48 0x00000020; \
mw.l 0x10420908 0x1FFF1FFF; \
mw.l 0x10420C48 0x00000000; \
fatload mmc $mmcdev:1 0x12080000 rzv2h_cr8_core1_rpmsg_linux_rtos_example_itcm.bin;\
fatload mmc $mmcdev:1 0x081C0000 rzv2h_cr8_core1_rpmsg_linux_rtos_example_sram.bin;\
fatload mmc $mmcdev:1 0x41800000 rzv2h_cr8_core1_rpmsg_linux_rtos_example_sdram.bin;\
mw.l 0x10420C14 0x00000003; \
dcache on;'
=> saveenv
=> run cr81start
```

This sequence prepares CR8 core 1, loads the three firmware sections into the required memory regions, and then continues the normal Linux boot flow.

The CR8 core 0 and CR8 core 1 command sequences are intended for starting each core individually. Running the two per-core start sequences one after another does not correctly start both CR8 cores together. To run both cores simultaneously, a dedicated combined start-up sequence must be used, which loads both firmware images and releases both cores together.

```
=> setenv cr8both 'dcache off; mw.l 0x10420D24 0x04000000;\
mw.l 0x10420600 0xE000E000; mw.l 0x10420604 0x00830083;\
mw.l 0x10420908 0x1FFF0000; mw.l 0x10420C44 0x003F0000;\
mw.l 0x10420C14 0x00000000; mw.l 0x10420908 0x10001000;\
mw.l 0x10420C48 0x00000020; mw.l 0x10420908 0x1FFF1FFF;\
mw.l 0x10420C48 0x00000000;\
fatload mmc $mmcdev:1 0x12040000 rzv2h_cr8_core0_rpmsg_linux_rtos_example_itcm.bin;\
fatload mmc $mmcdev:1 0x08180000 rzv2h_cr8_core0_rpmsg_linux_rtos_example_sram.bin;\
fatload mmc $mmcdev:1 0x40800000 rzv2h_cr8_core0_rpmsg_linux_rtos_example_sdram.bin;\
fatload mmc $mmcdev:1 0x12080000 rzv2h_cr8_core1_rpmsg_linux_rtos_example_itcm.bin;\
fatload mmc $mmcdev:1 0x081C0000 rzv2h_cr8_core1_rpmsg_linux_rtos_example_sram.bin;\
fatload mmc $mmcdev:1 0x41800000 rzv2h_cr8_core1_rpmsg_linux_rtos_example_sdram.bin;\
mw.l 0x10420C14 0x00000003; dcache on;'
=> saveenv
=> run cr8both
```

8.5.5.2 Load firmware by Linux

In addition to U-Boot loading, Linux can load and control remote firmware using the remoteproc framework.

In this method, Linux boots first. The firmware image is then selected from Linux user space, loaded into the target remote processor, and the remote processor is started through the remoteproc sysfs interface.

This method is useful when:

- firmware is being tested during development,
- different firmware versions are used frequently,
- the remote processor needs to be started manually after Linux boots,
- RPMsg communication is being debugged from Linux,
- the remote firmware must be stopped and reloaded dynamically.

At a high level, the flow is:

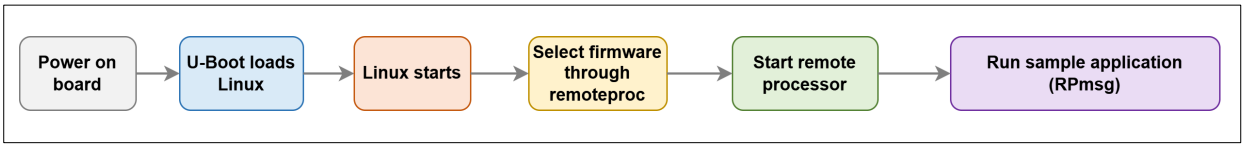


Figure 108. Linux firmware loading flow

Where supported by the platform driver, Linux remoteproc can also be used to:

- stop the remote processor,
- restart the remote processor,
- reload another firmware image.
- This is useful during development and repeated test cycles.

Use the corresponding remoteprocX (/sys/class/remoteproc/remoteprocX) device in the table below when selecting, starting, stopping, or reloading firmware from Linux.

Table 57. Linux Remote Processor Mapping

Linux remoteproc device	Remote Processor
remoteproc0	Cortex-M33
remoteproc1	Cortex-R8 Core 0
remoteproc2	Cortex-R8 Core1

To kick CM33 core from Linux, run following commands as below:

```
root@rz-cmn:~# cd /usr/lib/firmware/

# Kick RZ/V2H CM33 core
root@rz-cmn:/usr/lib/firmware# echo rzv2h_cm33_rpmsg_linux_rtos_example.elf \
> /sys/class/remoteproc/remoteproc0/firmware

# Kick RZ/G2L or RZ/V2L CM33 core
root@rz-cmn:/usr/lib/firmware# echo rzg2l_cm33_rpmsg_linux_rtos_example.elf \
> /sys/class/remoteproc/remoteproc0/firmware

# Start the processor
root@rz-cmn:/usr/lib/firmware# echo start > /sys/class/remoteproc/remoteproc0/state
```

To kick CR8 core from Linux (only for RZ/V2H), run the following commands as below:

```
root@rz-cmn:~# cd /usr/lib/firmware/

# Kick RZ/V2H CR8 core0
root@rz-cmn:/usr/lib/firmware# echo rzv2h_cr8_core0_rpmsg_linux_rtos_example.elf \
> /sys/class/remoteproc/remoteproc1/firmware

# Start the CR8 core 0 processor
root@rz-cmn:/usr/lib/firmware# echo start > /sys/class/remoteproc/remoteproc1/state

# Kick RZ/V2H CR8 core1
root@rz-cmn:/usr/lib/firmware# echo rzv2h_cr8_core1_rpmsg_linux_rtos_example.elf \
> /sys/class/remoteproc/remoteproc2/firmware

# Start the CR8 core 1 processor
root@rz-cmn:/usr/lib/firmware# echo start > /sys/class/remoteproc/remoteproc2/state
```

Then run the application normally by following the next section

In addition, in Linux, the remoteproc framework exposes each supported remote processor through the sysfs interface. Through this interface, users can select a firmware image, start the remote processor, stop it, and reload a different firmware image when needed.

```
# Select the firmware image
root@rz-cmn:# echo <firmware-name> > /sys/class/remoteproc/remoteprocX/firmware

# Start the remote processor
root@rz-cmn:# echo start > /sys/class/remoteproc/remoteprocX/state

# Stop the remote processor
root@rz-cmn:# echo stop > /sys/class/remoteproc/remoteprocX/state

# Reload a different firmware image
root@rz-cmn:# echo stop > /sys/class/remoteproc/remoteprocX/state
root@rz-cmn:# echo <new-firmware-name> > /sys/class/remoteproc/remoteprocX/firmware
root@rz-cmn:# echo start > /sys/class/remoteproc/remoteprocX/state
```

Replace remoteprocX with the remoteproc device corresponding to the target core, and replace <firmware-name> with the required firmware file name.

8.5.6 Running the RPMsg Sample Application

After the remote firmware has been loaded and started, the Linux RPMsg sample application can be used to verify communication between Linux and the remote processor.

The sample application is a demonstration program provided with this release. It runs on Linux user space and uses RPMsg/OpenAMP to communicate with the remote firmware running on CM33 or CR8.

Its purpose is to confirm that:

- The remote firmware is running,
- The communication path is initialized correctly,
- Linux can send messages to the remote firmware,

- The remote firmware can return messages back to Linux.

The sample performs a simple communication test. After the user selects a menu option, the application sends test payloads to the selected remote core and channel. The remote firmware returns the payloads, and the Linux application checks that the returned data matches the transmitted data.

The sample applications provided in this release are:

- RZ/G2L and RZ/V2L: rpmsg_sample_client
- RZ/V2H: rpmsg_sample_client_v2h

The menu displayed by the application allows the user to select which remote core and channel will be used for the communication test.

That should come before the platform-specific run steps.

8.5.6.1 RZ/G2L and RZ/V2L

After the CM33 firmware has been loaded and started by U-Boot or Linux, log in as root and run the sample application:

```
rz-cmn login: root
root@rz-cmn:~# rpmsg_sample_client
```

Then choose option 1, the expected output is as below:

```
Successfully probed IPI device
metal: info: metal_uio_dev_open: No IRQ for device 42f00000.rsctbl.
Successfully open uio device: 42f00000.rsctbl.
Successfully added memory device 42f00000.rsctbl.
metal: info: metal_uio_dev_open: No IRQ for device 43000000.vring-ctl0.
Successfully open uio device: 43000000.vring-ctl0.
Successfully added memory device 43000000.vring-ctl0.
metal: info: metal_uio_dev_open: No IRQ for device 43200000.vring-shm0.
Successfully open uio device: 43200000.vring-shm0.
Successfully added memory device 43200000.vring-shm0.
metal: info: metal_uio_dev_open: No IRQ for device 43100000.vring-ctl1.
Successfully open uio device: 43100000.vring-ctl1.
Successfully added memory device 43100000.vring-ctl1.
metal: info: metal_uio_dev_open: No IRQ for device 43500000.vring-shm1.
Successfully open uio device: 43500000.vring-shm1.
Successfully added memory device 43500000.vring-shm1.
metal: info: metal_uio_dev_open: No IRQ for device 42f01000.mhu-shm.
Successfully open uio device: 42f01000.mhu-shm.
Successfully added memory device 42f01000.mhu-shm.
Initialize remoteproc successfully.
Initialize remoteproc successfully.
**************************************
* rpmsg communication sample program *
**************************************
1. communicate with RZ/V2L CM33 ch0
2. communicate with RZ/V2L CM33 ch1
e. exit
please input
>
```

8.5.6.2 RZ/V2H

Similar to RZ/G2L and RZ/V2L, after the remote firmware has been loaded and the processor has been started, run the following commands to start the RPMsg sample application.

```
rz-cmn login: root
root@rz-cmn:~# rpmsg_sample_client_v2h
```

Then choose the option to run, the expected output is as below:

```
[xxx] Initialize remoteproc successfully.
[xxx] proc_id:1 rsc_id:1 mbx_id:1
[xxx] Initialize remoteproc successfully.
[xxx] proc_id:0 rsc_id:0 mbx_id:2
[xxx] Initialize remoteproc successfully.
[xxx] proc_id:1 rsc_id:1 mbx_id:2
[xxx] Initialize remoteproc successfully.
[xxx] proc_id:0 rsc_id:0 mbx_id:3
[xxx] Initialize remoteproc successfully.
[xxx] proc_id:1 rsc_id:1 mbx_id:3
[xxx] Initialize remoteproc successfully.
**************************************
* rpmsg communication sample program *
**************************************
1. communicate with CM33 ch0
2. communicate with CM33 ch1
3. communicate with CR8 core0 ch0
4. communicate with CR8 core0 ch1
5. communicate with CR8 core1 ch0
6. communicate with CR8 core1 ch1
7. communicate with CM33 ch0 and CR8 core0 ch1
8. communicate with CM33 ch0 and CR8 core1 ch1
9. communicate with CR8 core0 ch0 and CR8 core1 ch1
e. exit
please input
>
```

8.5.6.3 Note Running the RPMsg Sample Application

(1) Restart processor

Depending on the behavior of the demonstration firmware, the remote processor may need to be restarted before repeating a communication test.

In the provided RPMsg/OpenAMP demonstration, some firmware images may terminate the test application or stop the communication service after one test sequence has completed. In this case, if the user wants to run the same test again, the corresponding remote processor must be restarted before launching the Linux sample application again.

The restart can be done from Linux through the remoteproc sysfs interface.

```
# For example, on CM33:
root@rz-cmn:~# echo stop > /sys/class/remoteproc/remoteproc0/state
root@rz-cmn:~# echo start > /sys/class/remoteproc/remoteproc0/state

# On CR8 Core 0
root@rz-cmn:~# echo stop > /sys/class/remoteproc/remoteproc1/state
root@rz-cmn:~# echo start > /sys/class/remoteproc/remoteproc1/state

# On CR8 Core 1
root@rz-cmn:~# echo stop > /sys/class/remoteproc/remoteproc2/state
root@rz-cmn:~# echo start > /sys/class/remoteproc/remoteproc2/state
```

(2) Prebuilt Firmware Limitation

The current firmware built for CM33 is specific to thread 0, which is used for channel 0, while CR8 uses channel 1. To change the channel assignment, please rebuild the firmware.

(3) Order to invoke remote processor

For option 7: CM33 ch0 + CR8 core0 ch1

For option 8: CM33 ch0 + CR8 core1 ch1

When running a two-core communication test (option 7 or 8), you must invoke the CR8 RPMsg sample first and then invoke the CM33 sample.

The CR8 remote processor must be running and ready before the CM33 sample runs the two-core test. The Linux sample validates each remote processor once, when it starts, and skips any that is not yet ready. In addition, the CR8 firmware closes and then re-establishes its RPMsg connection after each test sequence. If the CR8 side is not running and ready beforehand, the two-core test may not run or may fail.

Note: This order only matters for the combined two-core options (7 and 8). For single-core options, the invocation order does not matter.

9. Network Boot and TFTP

This section outlines the process for network booting using TFTP (Trivial File Transfer Protocol). It includes configuration steps and commands necessary for a successful setup.

Network booting allows devices to boot from an image stored on a network server rather than relying on local storage. In this setup, the MMC SD card is not required for booting Linux.

9.1 TFTP Server Setup

This subsection covers the setup of a TFTP server on the host side. This is necessary for the device to retrieve the boot images over the network. We use an x86-based pc for this, but the instructions are the same for any server.

Prerequisites:

- x86 based PC or rack server (non x86 systems can be used at user discretion).
- An Ubuntu / Debian-based OS loaded onto the server.
- A router that performs DHCP.
- Target board; refer to section Supported Platforms for the currently supported platforms - depending on the board being used.
- FTDI-based USB-to-UART cable, or a USB Type-A to Micro USB cable (depending on the board's UART interface).
- Ethernet cables.

Note: This requires a wired connection and cannot be performed on Wi-Fi.

1. Install a TFTP server using the following command:

```
$ sudo apt update
$ sudo apt install tftpd-hpa
```

2. Create a TFTP directory and set the appropriate permissions.

```
$ sudo mkdir /tftpboot
$ sudo chmod 755 /tftpboot
```

3. Edit the TFTP configuration file (typically found at /etc/default/tftpd-hpa) and set it up as follows:

```
$ vi /etc/default/tftpd-hpa
```

Paste the following content into the file.

```
# /etc/default/tftpd-hpa

TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/tftpboot"
TFTP_ADDRESS="0.0.0.0:69"
TFTP_OPTIONS="--secure"/tftpd-hpa
```

4. Restart the TFTP service to apply the changes.

```
$ sudo systemctl restart tftpd-hpa
```

Make sure the tftpd-hpa service is running:

```
$ sudo systemctl status tftpd-hpa
```

9.2 NFS Server Setup

NFS (Network File System) is a protocol that allows clients to access files over a network as if they were local. It enables multiple clients to share files from a central server, simplifying file management across machines.

In this setup, NFS will share the root file system (rootfs) with clients booting over the network. This allows client devices to dynamically retrieve their operating system files and configurations, making it ideal for embedded systems that require consistent file access without local storage.

1. Install the NFS server and NFS client packages if they are not already installed on your host PC:

```
$ sudo apt update
$ sudo apt install nfs-kernel-server nfs-common
```

2. Edit the /etc/exports file to specify the directories to be shared and their access permissions.

```
$ vi /etc/exports
```

For example, to share the /tftpboot directory, add the following line:

```
/tftpboot *(rw,no_root_squash,async)
```

Here, * allows access from any client. Consider replacing it with specific client IP addresses for better security.

3. After editing /etc/exports, run the following command to export the directories:

```
$ sudo exportfs -a
```

4. Start the NFS server and enable it to run at boot:

```
$ sudo systemctl start nfs-kernel-server
$ sudo systemctl enable nfs-kernel-server
```

9.3 U-Boot DHCP IP Configuration

In this subsection, the U-Boot environment will be configured for network settings, including the specification of the Ethernet device and the configuration of the server and device IP addresses.

1. Enter the U-Boot interactive command prompt for configuration by pressing any key when prompted with **Hit any key to stop autoboot**:

```
U-Boot 2021.10 (May 24 2024 - 07:26:08 +0000)

CPU:   Renesas Electronics CPU rev 1.0
Model: <Detected Board Model>
DRAM:  896 MiB
MMC:   sd@11c00000: 0
Loading Environment from SPIFlash... SF: Detected is25wp256 with page size 256
Bytes, erase size 4 KiB, total 32 MiB

In:    serial@1004b800
Out:   serial@1004b800
Err:   serial@1004b800
Net:   <Detected Ethernet Interface>
Hit any key to stop autoboot:  0
=>
```

2. Specify the Ethernet device (eth1) to use for the network connection.
For example:

```
=> setenv ethact ethernet@11c30000
```

3. Configure server and device IPs:

```
=> setenv serverip <server_ip>
=> setenv ipaddr <device_ip>
```

For example:

```
=> setenv serverip 192.168.5.86
=> setenv ipaddr 192.168.5.30
```

9.4 TFTP Boot

In this subsection, the boot arguments and commands for U-Boot will be configured to load the kernel image and device tree from the TFTP server.

Ensure all hardware connections are properly set up, as shown below

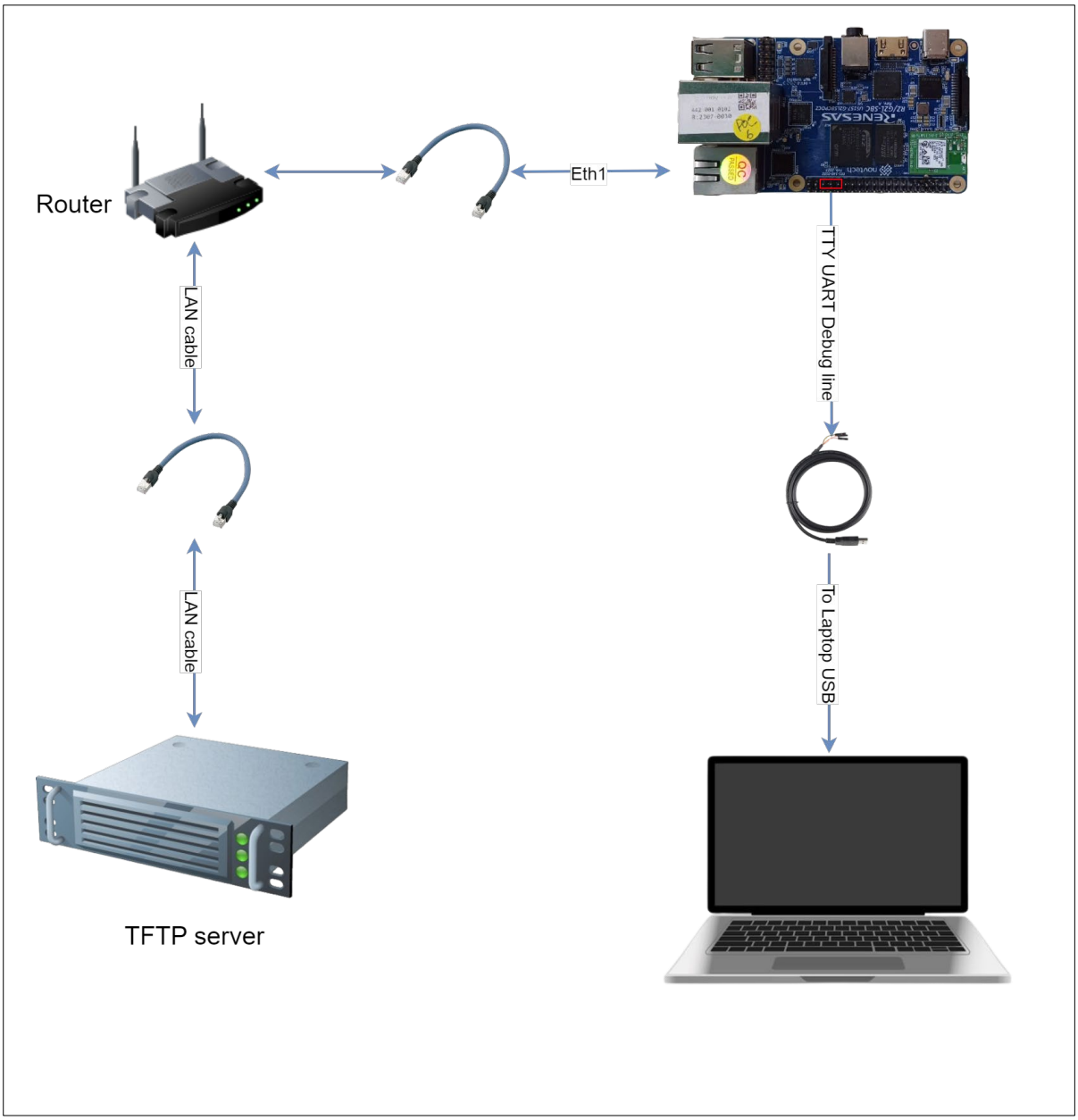


Figure 109. TFTP boot setup for RZ/G2L-SBC

9.4.1 Prepare boot images from the release package

After setting up the TFTP server and confirming the hardware connections, place the required boot images, such as the kernel image, device tree blob (DTB), device tree overlay (DTBO), and root file system, into the TFTP directory (/tftpboot). These files will be loaded during the boot process. The following steps outline the procedure for preparing and placing these images.

1. Locate the boot images in the release package.

   The release package provides the necessary components under target/images:

   - Kernel image: target/images/linux/Image
   - Device tree blobs (DTB): target/images/linux/dtbs/<board>.dtb
   - Device tree overlays (if applicable): target/images/linux/dtbs/overlays/*.dtbo
   - Root file system archives: target/images/rootfs/<image-name>.tar.bz2

(e.g., core-image-weston.tar.bz2, renesas-core-image-cli.tar.bz2, etc.)

2. Select the rootfs image
   Choose the appropriate root file system tarball (*.tar.bz2) depending on the desired image type. Refer to Supported Distributions section for the list of currently supported distributions. In this example, the core-image-weston.tar.bz2 root filesystem will be used
3. Extract the selected rootfs
   Create a subfolder inside the TFTP directory to keep the files organized, and then extract the chosen rootfs tarball into that folder.

```
renesas@builder-pc:~$ mkdir -p /tftpboot/rz-cmn/rootfs
```

   Extract the root file system into the TFTP directory

```
renesas@builder-pc:~$ sudo tar xf \
<path/to/release/package/rootfs/target/images/core-image-weston.tar.bz2 -C \
/tftpboot/rz-cmn/rootfs
```

   For example,

```
renesas@builder-pc:~$ sudo tar xf \
~/rz-cmn-srp/target/images/core-image-weston.tar.bz2 -C /tftpboot/rz-cmn/rootfs
```

4. Copy the kernel image

```
renesas@builder-pc:~$ sudo cp -f ~/rz-cmn-srp/target/images/linux/Image \
/tftpboot/rz-cmn
```

5. Copy the device tree and device tree overlays (if needed)

```
# Copy the device tree blob
renesas@builder-pc:~$ sudo cp ~/rz-cmn-srp/target/images/linux/dtbs/rzg2l-sbc.dtb \
/tftpboot/rz-cmn

# Copy the device tree overlays (RZG2L-SBC) may need these
renesas@builder-pc:~$ sudo cp -r ~/rz-cmn-srp/target/images/linux/dtbs/overlays/ \
/tftpboot/rz-cmn
```

6. Verify the TFTP directory contents
   Ensure the folder contains the expected items:
   - Image
   - <board>.dtb
   - Any required .dtbo files under the overlays folder
   - Expected rootfs directory

```
renesas@builder-pc:/tftpboot/rz-cmn/$ tree -L 2
.
├── Image
├── overlays
│   ├── rzg2l-sbc-can.dtbo
│   ├── rzg2l-sbc-dsi.dtbo
│   ├── rzg2l-sbc-ext-i2c.dtbo
│   ├── rzg2l-sbc-ext-spi.dtbo
│   └── rzg2l-sbc-ov5640.dtbo
├── rootfs
│   ├── bin -> usr/bin
│   ├── boot
│   ├── dev
│   ├── etc
│   ├── home
│   ├── lib -> usr/lib
│   ├── media
│   ├── mnt
│   ├── opt
│   ├── proc
│   ├── root
│   ├── run
│   ├── sbin -> usr/sbin
│   ├── snap
│   ├── srv
│   ├── sys
│   ├── tmp
│   ├── usr
│   └── var
└── rzg2l-sbc.dtb
```

9.4.2 Setup U-Boot boot arguments

Define the boot arguments to specify the network and root file system settings:

```
=> setenv bootargs 'consoleblank=0 strict-devmem=0
ip=<device_ip>:<server_ip>::::<eth_device> root=/dev/nfs rw
nfsroot=<server_ip>:</path/to/your/rootfs>,v3,tcp'
```

For example:

```
=> setenv setenv bootargs 'consoleblank=0 strict-devmem=0
ip=192.168.5.30:192.168.5.86::::eth1 root=/dev/nfs rw
nfsroot=192.168.5.86:/tftpboot/rz-cmn/rootfs,v3,tcp'
```

9.4.3 Configure U-Boot boot commands

Configure the boot command to load the kernel image and device tree files from the TFTP server.

```
=> setenv bootcmd 'tftp <load_address_kernel> <path/to/kernel_image>; tftp
<load_address_dtb> <path/to/device_tree_blob>; tftp <load_address_dtbo>
<path/to/dtbo file>; booti <load_address_kernel> - <load_address_dtb> -
<load_address_dtbo>'
```

For example, load '**Image', 'rzg2l-sbc.dtb',** and '**rzg2l-sbc-ext-spi.dtbo'** files.

```
=> setenv bootcmd 'tftp 0x48080000 rz-cmn/Image; tftp 0x48000000 rz-cmn/rzg2l-
sbc.dtb; tftp 0x48010000 rz-cmn/overlays/rzg2l-sbc-ext-spi.dtbo; booti 0x48080000
- 0x48000000 - 0x48010000'
```

Save the changes to the environment variables so they persist across reboots:

```
=> saveenv
```

Initiate the boot process by running **bootcmd**:

```
=> run bootcmd
```

If everything is set up correctly, the images will be booted from the network.

```
=> run bootcmd
Using ethernet@11c30000 device
TFTP from server 192.168.5.86; our IP address is 192.168.5.30
Filename rz-cmn/Image'.
Load address: 0x48080000
Loading: ##################################################################
         ##################################################################
         ##################################################################
         19.6 MiB/s
done
Bytes transferred = 18035200 (1133200 hex)
Using ethernet@11c30000 device
TFTP from server 192.168.5.86; our IP address is 192.168.5.30
Filename rz-cmn/rzg2l-sbc.dtb'.
Load address: 0x48000000
Loading: ####
         8.6 MiB/s
done
Bytes transferred = 44855 (af37 hex)
Using ethernet@11c30000 device
TFTP from server 192.168.5.86; our IP address is 192.168.5.30
Filename rz-cmn/overlays/rzg2l-sbc-ext-spi.dtbo'.
Load address: 0x48010000
Loading: #
         455.1 KiB/s
done
Bytes transferred = 932 (3a4 hex)
Moving Image from 0x48080000 to 0x48200000, end=493a0000
## Flattened Device Tree blob at 48000000
   Booting using the fdt blob at 0x48000000
   Loading Device Tree to 000000007bf1a000, end 000000007bf27f36 ... OK

Starting kernel ...
...
```

10. Using SSH and SCP for Remote Access and File Transfers

This section explains how to use SSH (Secure Shell) for secure remote access to the RZ/G2L-SBC and how to utilize SCP (Secure Copy Protocol) for file transfers. By default, OpenSSH is employed as it is a feature-rich and widely used SSH implementation that offers advanced capabilities for secure communication. While OpenSSH serves as the default option, Dropbear SSH can be considered for lightweight, resource-constrained environments, making it particularly suitable for embedded systems.

10.1 Differences Between Dropbear and OpenSSH

- **Resource Usage**: Dropbear is optimized for lower resource usage, making it ideal for embedded systems.
- **Feature Set**: OpenSSH has a more extensive feature set, including advanced options for authentication and configuration.
- **Key Authentication**: OpenSSH requires the use of SSH keys for authentication, while Dropbear can operate with both keys and passwords.

10.2 Using OpenSSH

OpenSSH is a widely used, full-featured SSH implementation that provides encrypted communication between hosts. It supports advanced authentication methods and secure remote administration, making it ideal for robust network security.

The RZ/G2L-SBC supports both password and key-based authentication methods. To enhance security by enforcing SSH key-based login, follow these steps to switch to key-based authentication:

1. Generate an SSH key pair on the local machine. Run the following command to generate a secure SSH key pair:

```
$ ssh-keygen -t rsa -b 4096
```

2. Copying an SSH public Key to the board using SSH: transfer your public key to the board with this command:

```
$ cat ~/.ssh/id_rsa.pub | ssh username@remote_host "mkdir -p ~/.ssh && cat >>
~/.ssh/authorized_keys"
```

For example:

```
$ cat ~/.ssh/id_rsa.pub | ssh root@192.168.5.30 "mkdir -p ~/.ssh && cat >>
~/.ssh/authorized_keys"
```

3. Authenticate using SSH keys:

```
$ ssh root@192.168.5.30
```

If this is the first time connecting to this host (as mentioned in the previous method), a message like the following may appear:

```
$ The authenticity of host 192.169.5.30 (192.168.5.30)' can't be established.
ED25519 key fingerprint is SHA256:esQPI0Ip9HZH9A6dvTsA9+k7eLjT4sqzpiF7znl0tyw.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

This indicates that the local computer does not recognize the remote host. Type **yes** and press **the Enter key** to proceed.

Step 4: Disable password authentication. If logging in to your account using SSH is successful without a password, SSH key-based authentication has been correctly configured. However, password-based authentication remains active, which leaves the server vulnerable to brute-force attacks.

Once the SSH connection is established, open the SSH daemon's configuration file:

```
$ vi /etc/ssh/sshd_config
```

Inside the file, search for a directive called **PasswordAuthentication**. This may be commented out. Uncomment the line by removing any '#' at the beginning of the line, and set the value to **no**. This will disable the ability to log in through SSH using account passwords: /etc/ssh/sshd

```
PasswordAuthentication no
```

Step 5: Restart the SSH service to apply the changes:

```
$ systemctl restart ssh
```

10.3 SSH Access

After configuring the authentication key, access to the RZ/G2L-SBC via SSH can be achieved using various tools available on both Windows and Linux platforms.

10.3.1 SSH from Windows Host

1. Using Git Bash.
   - Install Git for Windows if you have not already.
   - Open and use the following command:

   ```
   $ ssh username@<device_ip>
   ```

   For example:

   ```
   $ ssh root@192.168.5.30
   ```

   - Type **yes** to confirm the host's authenticity when prompted.

   ```
   $ ssh root@192.168.5.30
   The authenticity of host '192.168.5.30 (192.168.5.30)' can't be established.
   RSA key fingerprint is SHA256:v39PhjNp4F7HcQpwJmfNOYcC+ZZ3Yw8i1ICsL2mXUgg.
   This key is not known by any other names.
   Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
   Warning: Permanently added '192.168.5.30' (RSA) to the list of known hosts.
   ```

2. Use MobaXTerm.
   - Download and install MobaXTerm if you have not already.
   - Select "Session" > "SSH" and enter the device's IP address

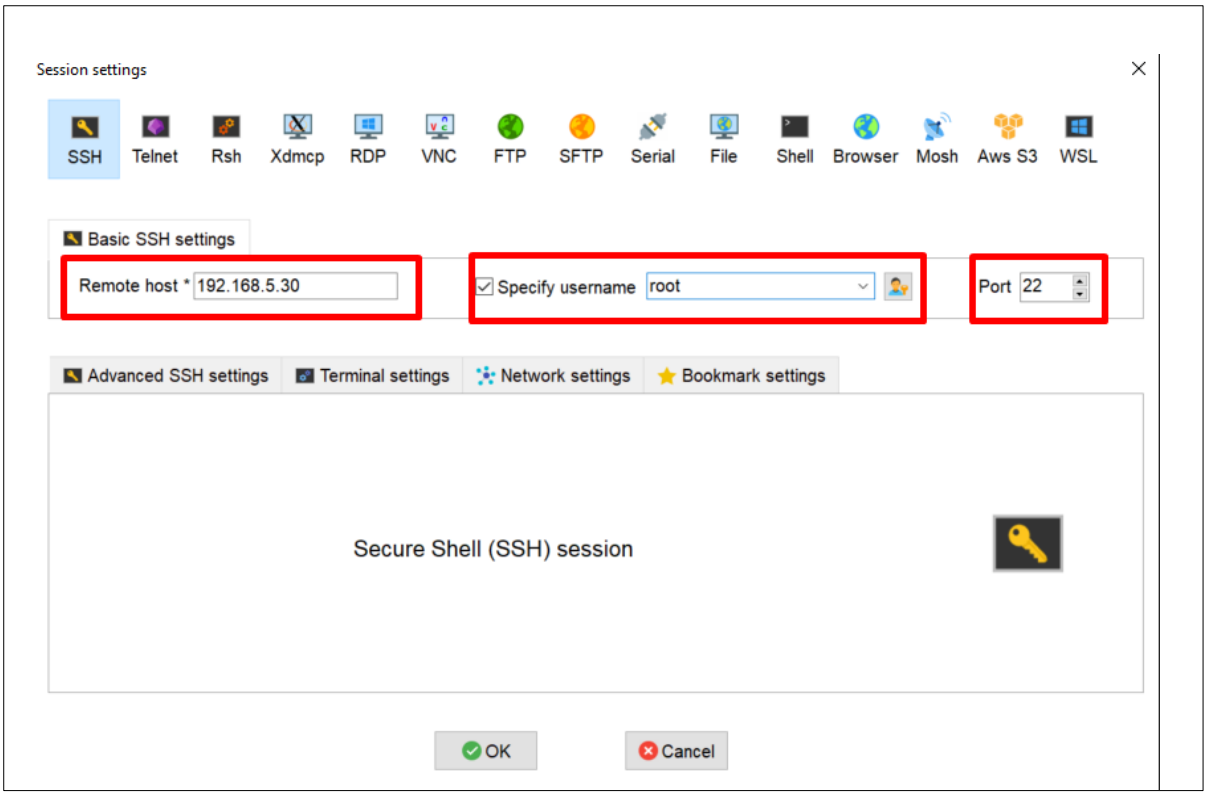


Figure 110. SSH settings in MobaXTerm

- Click ‘OK’ to save the setting.
- Click on the session to initiate an SSH connection.

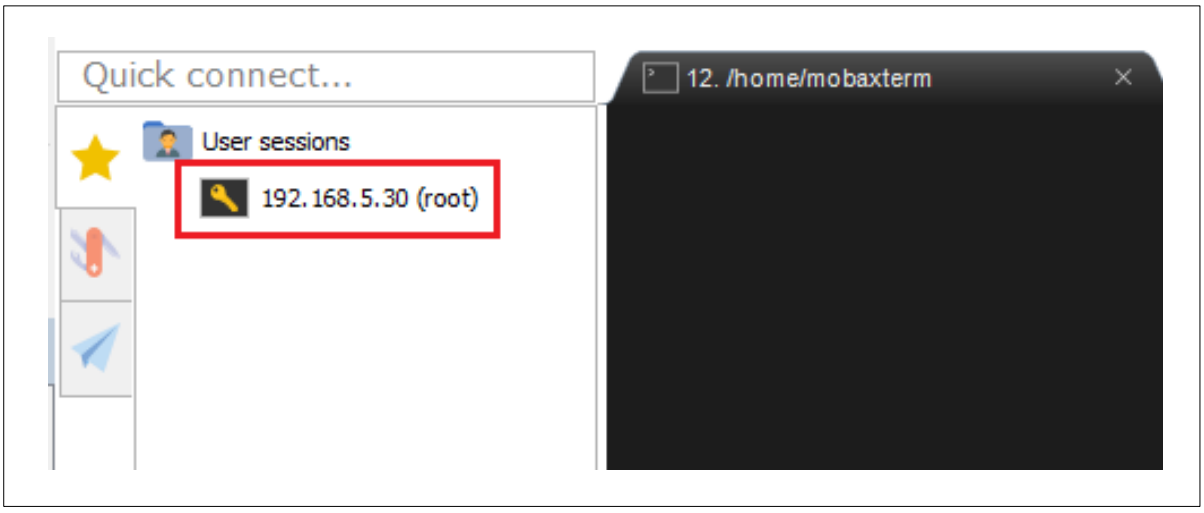


Figure 111. Connect to the SSH session in MobaXTerm

10.3.2 SSH from Linux Host

1. Open a terminal and run.

```
$ ssh username@<device_ip>
```

For example:

```
$ ssh root@192.168.5.30
```

2. Type **yes** to confirm the host's authenticity when prompted.

```
$ ssh root@192.168.5.30
The authenticity of host '192.168.5.30 (192.168.5.30)' can't be established.
RSA key fingerprint is SHA256:v39PhjNp4F7HcQpwJmfNOYcC+ZZ3Yw8i1ICsL2mXUgg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.5.30' (RSA) to the list of known hosts.
```

10.4 SCP (Secure Copy Protocol)

To securely transfer files between local and remote systems, SCP can be used on both Windows and Linux.

10.4.1 SCP from Windows Host

1. Using Git Bash:
   - Install Git for Windows if you have not already.
   - Use the following command:

     ```
     $ scp <local_file> username@<device_ip>:<remote_path>
     ```

     For example:

     ```
     $ scp hello-world root@192.168.5.30:home/root
     ```

   - Type **yes** to confirm the host's authenticity when prompted.
2. Use WinSCP.
   - Open WinSCP and select “New Session”.
   - Choose SCP as the protocol, then enter the remote device's IP address and the hostname.

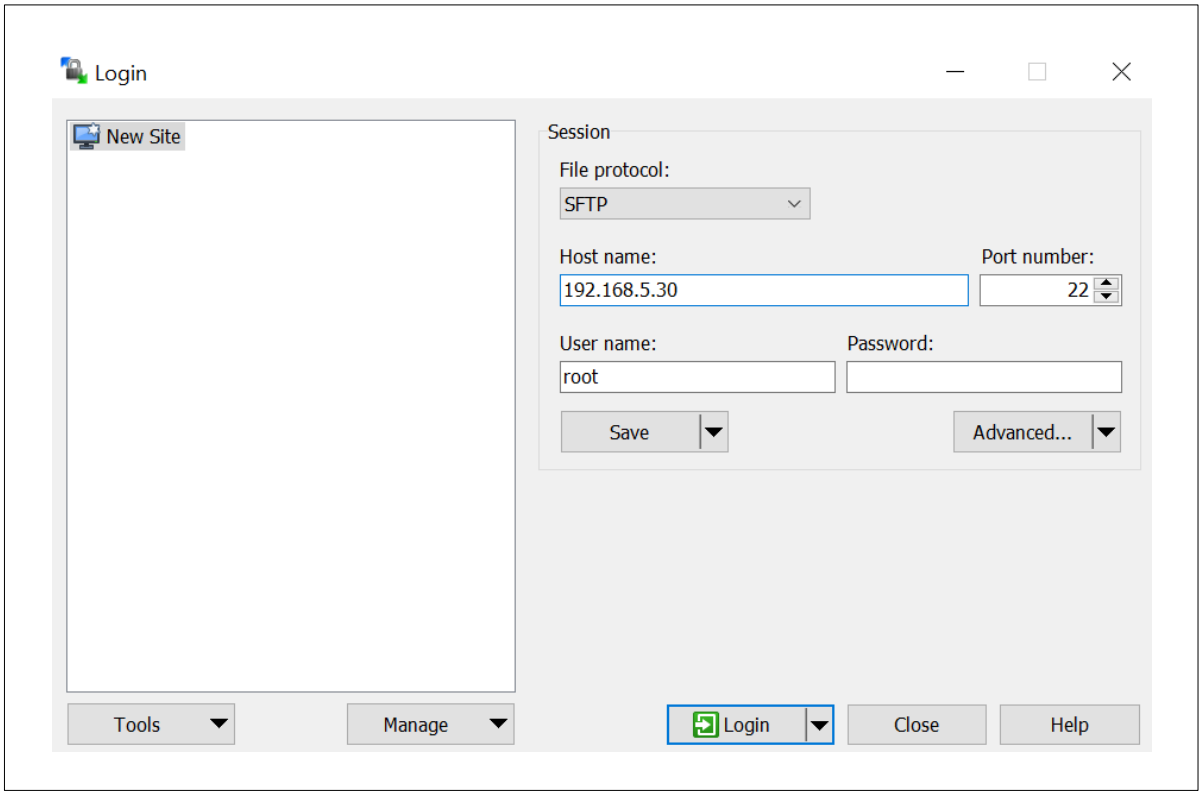


Figure 112. Setting up WinSCP for an SSH session

- Click ‘Login’ and select yes to confirm the host's authenticity when prompted.
- Drag and drop files between your local machine (Left) and the target board (Right) to transfer.

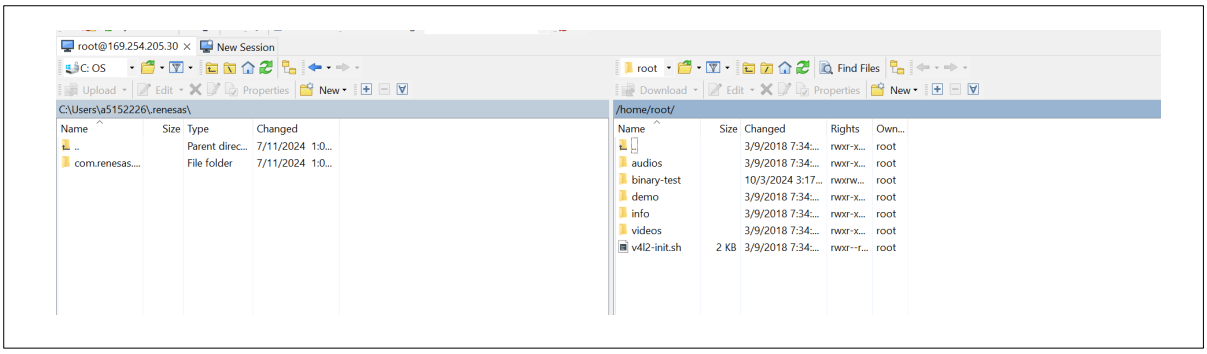


Figure 113. Using WinSCP to transfer files

10.4.2 SCP from Linux Host

1. Open a terminal and run.

```
$ scp <local_file> username@<device_ip>:<remote_path>
```

For example:

```
$ scp hello-world root@192.168.5.30:/home/root
```

2. Type **yes** to confirm the host's authenticity when prompted.

```
$ scp hello-world root@192.168.5.30:/home/root
The authenticity of host '192.168.5.30 (192.168.5.30)' can't be established.
RSA key fingerprint is SHA256:v39PhjNp4F7HcQpwJmfNOYcC+ZZ3Yw8i1ICsL2mXUgg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.5.30' (RSA) to the list of known hosts.
```

11. Building the eSDK

The extensible SDK makes it easy to add new applications and libraries to an image, modify the source for an existing component, test changes on the RZ board, and ease integration into the rest of the OpenEmbedded Build System.

The eSDK build generates an installer, which you will use to install the eSDK on the same PC where your Yocto environment is set up.

In Section 6.1, the support script was copied from the release package, which helped commence the image build. The script can also support eSDK build, run the following command to start the build with specific images:

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ IMAGE=<target-images> ./rzsbc_builder.sh
build-sdk
```

For example, to build the core-image-weston eSDK:

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ IMAGE=core-image-
weston ./rzsbc_builder.sh build-sdk
```

To build the eSDK for all supported images, run the following command:

```
renesas@builder-pc:~/renesas/rz-cmn-srp$ IMAGE=all-supported-
images ./rzsbc_builder.sh build-sdk
```

The resulting eSDK installer will be in `~/renesas/rz-cmn-srp/yocto_rzcmn_board/build/tmp/deploy/sdk`.

The eSDK installer will have the extension “.sh”.

Note: SDK build support is provided only for Yocto-based images. Ubuntu images are not associated with an SDK in the build system. For Ubuntu, the development environment must be set up manually using standard Ubuntu tools. The SDK generated by the build scripts applies exclusively to Yocto.

```
renesas@builder-pc:~/renesas/rz-cmn-srp/yocto_rzcmn_board/build/tmp/deploy/sdk$ ls
poky-glibc-x86_64-core-image-bsp-cortexa55-rz-cmn-toolchain-5.1.4.host.manifest
poky-glibc-x86_64-core-image-bsp-cortexa55-rz-cmn-toolchain-5.1.4.sh
poky-glibc-x86_64-core-image-bsp-cortexa55-rz-cmn-toolchain-5.1.4.target.manifest
poky-glibc-x86_64-core-image-bsp-cortexa55-rz-cmn-toolchain-5.1.4.testdata.json
poky-glibc-x86_64-core-image-minimal-cortexa55-rz-cmn-toolchain-5.1.4.host.manifest
poky-glibc-x86_64-core-image-minimal-cortexa55-rz-cmn-toolchain-5.1.4.sh
poky-glibc-x86_64-core-image-minimal-cortexa55-rz-cmn-toolchain-5.1.4.target.manifest
poky-glibc-x86_64-core-image-minimal-cortexa55-rz-cmn-toolchain-5.1.4.testdata.json
poky-glibc-x86_64-core-image-weston-cortexa55-rz-cmn-toolchain-5.1.4.host.manifest
poky-glibc-x86_64-core-image-weston-cortexa55-rz-cmn-toolchain-5.1.4.sh
poky-glibc-x86_64-core-image-weston-cortexa55-rz-cmn-toolchain-5.1.4.target.manifest
poky-glibc-x86_64-core-image-weston-cortexa55-rz-cmn-toolchain-5.1.4.testdata.json
poky-glibc-x86_64-renesas-core-image-cli-cortexa55-rz-cmn-toolchain-
5.1.4.host.manifest
poky-glibc-x86_64-renesas-core-image-cli-cortexa55-rz-cmn-toolchain-5.1.4.sh
poky-glibc-x86_64-renesas-core-image-cli-cortexa55-rz-cmn-toolchain-
5.1.4.target.manifest
```

```
poky-glibc-x86_64-renesas-core-image-cli-cortexa55-rz-cmn-toolchain-
5.1.4.testdata.json
poky-glibc-x86_64-renesas-core-image-weston-cortexa55-rz-cmn-toolchain-
5.1.4.host.manifest
poky-glibc-x86_64-renesas-core-image-weston-cortexa55-rz-cmn-toolchain-5.1.4.sh
poky-glibc-x86_64-renesas-core-image-weston-cortexa55-rz-cmn-toolchain-
5.1.4.target.manifest
poky-glibc-x86_64-renesas-core-image-weston-cortexa55-rz-cmn-toolchain-
5.1.4.testdata.json
poky-glibc-x86_64-renesas-quickboot-cli-cortexa55-rz-cmn-toolchain-5.1.4.host.manifest
poky-glibc-x86_64-renesas-quickboot-cli-cortexa55-rz-cmn-toolchain-5.1.4.sh
poky-glibc-x86_64-renesas-quickboot-cli-cortexa55-rz-cmn-toolchain-
5.1.4.target.manifest
poky-glibc-x86_64-renesas-quickboot-cli-cortexa55-rz-cmn-toolchain-5.1.4.testdata.json
poky-glibc-x86_64-renesas-quickboot-wayland-cortexa55-rz-cmn-toolchain-
5.1.4.host.manifest
poky-glibc-x86_64-renesas-quickboot-wayland-cortexa55-rz-cmn-toolchain-5.1.4.sh
poky-glibc-x86_64-renesas-quickboot-wayland-cortexa55-rz-cmn-toolchain-
5.1.4.target.manifest
poky-glibc-x86_64-renesas-quickboot-wayland-cortexa55-rz-cmn-toolchain-
5.1.4.testdata.json
```

Note: The SDK build may fail depending on the build environment. At that time, run the build again after a period of time.

12. Application Building, Packaging, and Running

The SDK allows you to develop and test custom applications for the RZ board on different systems. This section covers setting up your development environment and running your applications.

12.1 How to extract the eSDK

To get started, extract the eSDK and install the toolchain on your host PC. This step provides the necessary tools for cross-compiling your applications.

To set up your environment:

1. Install the toolchain on a Host PC.

For example, to install the toolchain, run the following command.

```
renesas@builder-pc:~/renesas/rz-cmn-
srp/yocto_rzcmn_board$ sh ./build/tmp/deploy/sdk/poky-glibc-x86_64-core-image-
weston-cortexa55-rz-cmn-toolchain-ext-5.1.4.sh
```

Note: You cannot install the eSDK as root because BitBake will not run with root privileges. Therefore, attempting to install the extensible SDK as root is counterproductive.

If the installation is successful, the following messages will appear:

```
renesas@builder-pc:~/renesas/rz-cmn-srp/yocto_rzcmn_board
$ sh ./build/tmp/deploy/sdk/poky-glibc-x86_64-core-image-weston-cortexa55-rz-cmn-
toolchain-ext-5.1.4.sh
Poky (Yocto Project Reference Distro) Extensible SDK installer version 5.1.4
==========================================================================
Enter target directory for SDK (default: ~/poky_sdk): ~/esdk/5.1.4
You are about to install the SDK to "/home/renesas/esdk/5.1.4". Proceed [Y/n]? Y
Extracting SDK.............done
Setting it up...
Extracting buildtools...
Preparing build system...
Parsing recipes: 100% |#########################################| Time: 0:00:52
Initialising tasks: 100% |######################################| Time: 0:00:00
Checking sstate mirror object availability: 100% |##############| Time: 0:00:00
Loading cache: 100% |###########################################| Time: 0:00:00
Initialising tasks: 100% |######################################| Time: 0:00:00
done

SDK has been successfully set up and is ready to be used.
Each time you wish to use the SDK in a new shell session, you need to source the
environment setup script e.g.
 $ . /home/renesas/esdk/5.1.4/environment-setup-cortexa55-poky-linux
```

2. Set up cross-compile environment. The following command assumes that you installed the SDK in `~/esdk/5.1.4`

```
renesas@builder-pc:~$ source ~/esdk/5.1.4/environment-setup-cortexa55-poky-linux
SDK environment now set up; additionally you may now run devtool to perform
development tasks.
Run devtool --help for further details.
```

Note: The user needs to run the above command once for each shell session. In addition, source' is a bash-specific call. The POSIX convention is to use'. ~/esdk/5.1.4/environment-setup-cortexa55-poky-linux'. Bash equates 'source' to '.'.

To begin working with the Extensible Software Development Kit (eSDK) in Yocto, consult the official documentation provided by the Yocto Project. This guide offers comprehensive instructions on configuring and utilizing the eSDK effectively.

Access the official eSDK documentation by following this URL: Using the Extensible SDK.

12.2 Build a sample application using the eSDK with CMake

CMake is cross-platform, free, and open-source software for building automation, testing, packaging, and installation of software by using a compiler-independent method.

If the host PC does not have CMake installed, it can be installed using the following command:

```
renesas@builder-pc:~$ sudo apt-get install cmake
```

The following steps will include instructions for setting up the project, editing the CMakeLists.txt file, and performing the build and installation.

1. Create the project structure:

```
renesas@builder-pc:~$ mkdir ~/cmake_helloworld
renesas@builder-pc:~$ cd ~/cmake_helloworld
renesas@builder-pc:~/cmake_helloworld$ mkdir build src
```

2. Organize the project structure as shown below:

```
renesas@builder-pc:~/cmake_helloworld$ tree
.
├── build
├── CMakeLists.txt
└── src
    └── helloworld.c
```

`CMakeLists.txt` and `helloworld.c` will be created later.

3. Create your application.

```
renesas@builder-pc:~/cmake_helloworld$ vi src/helloworld.c
```

Then, copy the contents below to the file:

```
#include <stdio.h>

int main(int argc, char** argv)
{
    printf("\nHello World!\n");
    return 0;
}
```

4. Create CMake configuration file.

```
renesas@builder-pc:~/cmake_helloworld$ vi CMakeLists.txt
```

Edit the following configuration file to match the SDK paths. The eSDK installation, as described in 13.1 How to extract the eSDK, is a prerequisite for this operation.

```
cmake_minimum_required(VERSION 3.10)
project(HelloWorld C)

# Set the path to your C compiler
set(CMAKE_C_COMPILER /path/to/your/sdk/bin/gcc)

# Set the path to your C++ compiler (if needed)
set(CMAKE_CXX_COMPILER /path/to/your/sdk/bin/g++)

# Define the sysroot path for cross-compilation
set(CMAKE_SYSROOT /path/to/your/sysroot)

# Add the executable target “helloworld”
add_executable(helloworld src/helloworld.c)
```

For example, if the SDK is installed in `/home/renesas/esdk/5.1.4`, the completed configuration file will resemble the following:

```
cmake_minimum_required(VERSION 3.10)
project(HelloWorld C)

set(CMAKE_C_COMPILER
/home/renesas/esdk/5.1.4/tmp/sysroots/x86_64/usr/bin/aarch64-poky-linux/aarch64-
poky-linux-gcc)
set(CMAKE_CXX_COMPILER
/home/renesas/esdk/5.1.4/tmp/sysroots/x86_64/usr/bin/aarch64-poky-linux/aarch64-
poky-linux-g++)

# Sysroot path
set(CMAKE_SYSROOT /home/renesas/esdk/5.1.4/tmp/sysroots/rzg2l-sbc)

add_executable(helloworld src/helloworld.c)
```

5. Build the application.

```
renesas@builder-pc:~/cmake_helloworld$ cd build/
renesas@builder-pc:~/cmake_helloworld/build$ cmake ../
-- The C compiler identification is GNU 9.4.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/renesas/cmake_helloworld/build
renesas@builder-pc:~/cmake_helloworld/build$ cmake --build .
[ 50%] Building C object CMakeFiles/hello.dir/src/helloworld.c.o
[100%] Linking C executable helloworld
[100%] Built target helloworld
```

After completing, confirm that the executed application `helloworld` is generated in the build folder.

```
renesas@builder-pc:~/cmake_helloworld/build$ ls
CMakeCache.txt  CMakeFiles  cmake_install.cmake  CPackConfig.cmake
CPackSourceConfig.cmake  helloworld  Makefile
```

Also, this application must be cross-compiled for aarch64.

```
renesas@builder-pc:~/cmake_helloworld/build$ file helloworld
helloworld: ELF 64-bit LSB pie executable, ARM aarch64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-aarch64.so.1,
BuildID[sha1]=436a40422c25d0eb57771b5cda061b49e5c197e7, for GNU/Linux 3.14.0,
with debug_info, not stripped
```

12.3 Package Programs with Cpack

This section provides a step-by-step guide on how to configure CMake to package your application into a .deb file, which is a Debian package file. You can install them on the RZ boards as an application. CPack is a CMake module that handles packaging.

12.3.1 Package a C Program

The following steps provide detailed instructions for using CPack to package a C program into a .deb file, including configuring CMake and preparing the necessary files for packaging.

1. Add CPack configuration to CMakeLists.txt from the previous chapter Build a sample application using the eSDK with **CMake**

```
renesas@builder-pc:~/cmake_helloworld$ vi CMakeLists.txt
```

Then, edit your CMakeLists.txt file to include CPack configuration

```
cmake_minimum_required(VERSION 3.10)
project(HelloWorld C)

set(CMAKE_C_COMPILER
/home/renesas/esdk/5.1.4/tmp/sysroots/x86_64/usr/bin/aarch64-poky-linux/aarch64-
poky-linux-gcc)
set(CMAKE_CXX_COMPILER
/home/renesas/esdk/5.1.4/tmp/sysroots/x86_64/usr/bin/aarch64-poky-linux/aarch64-
poky-linux-g++)

# Sysroot path
set(CMAKE_SYSROOT /home/renesas/esdk/5.1.4/tmp/sysroots/rz-cmn)

add_executable(helloworld src/helloworld.c)

# Specify the installation path
install(TARGETS helloworld DESTINATION /usr/local/bin)

# CPack configuration
set(CPACK_GENERATOR "DEB")
set(CPACK_PACKAGE_NAME "helloworld")
set(CPACK_PACKAGE_VERSION "1.0.0")
set(CPACK_DEBIAN_PACKAGE_ARCHITECTURE "arm64")
set(CPACK_PACKAGE_CONTACT "Your Name <your.email@example.com>")
set(CPACK_DEBIAN_PACKAGE_MAINTAINER "Your name")

include(CPack)
```

2. Package the C program into a Debian package installer.

```
renesas@builder-pc:~/cmake_helloworld$ cd build/
renesas@builder-pc:~/cmake_helloworld$ cmake ../
-- The C compiler identification is GNU <gnu-version>
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /home/renesas/esdk/5.1.4/sysroots/x86_64-
pokysdk-linux/usr/bin/aarch64-poky-linux/aarch64-poky-linux-gcc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Configuring done (0.1s)
-- Generating done (0.0s)
-- Build files have been written to: /home/tiennguyen/cmake/build
renesas@builder-pc:~/cmake_helloworld/build$ cpack
CPack: Create package using DEB
CPack: Install projects
CPack: - Run preinstall target for: HelloWorld
CPack: - Install project: HelloWorld []
CPack: Create package
-- CPACK_DEBIAN_PACKAGE_DEPENDS not set, the package will have no dependencies.
CPack: - package: /home/renesas/cmake_helloworld/build/helloworld-1.0.0-Linux.deb
generated.
```

After completing, confirm that the Debian package (.deb) is generated in the build folder.

```
renesas@builder-pc:~/cmake_helloworld/build$ ls
CMakeCache.txt  cmake_install.cmake  _CPack_Packages          helloworld
install_manifest.txt          CMakeFiles      CPackConfig.cmake
CPackSourceConfig.cmake  helloworld-1.0.0-Linux.deb  Makefile
```

3. Ship the Debian package installer to the target board.

The Debian package installer “helloworld-1.0.0-Linux.deb” can be transferred to the target board using the SCP tool as below, or other methods, such as a USB drive or NFS (Network File System).

```
renesas@builder-pc:~/cmake_helloworld/build$ scp helloworld-1.0.0-Linux.deb
root@<board_IP_address>:<destination>
```

For example:

```
renesas@builder-pc:~/cmake_helloworld/build$ scp helloworld-1.0.0-Linux.deb
root@192.168.5.58:/home/root
```

12.3.2 Package a Python Program

This section explains how to package Python scripts into a .deb file using CPack, focusing on the necessary configurations and packaging steps.

Two options are available for running a Python script:

1. Directly with Python: Use the command `python3 script.py` to execute the script directly.
2. By shell script: Use a shell script to run the Python script. This approach can be useful for adding additional setup or configuration steps.

To run the application without invoking the python3 command directly, create a shell script that contains the command to execute the Python script.

The steps below are similar to those for packaging a C program, with differences primarily in the source code and CPack configuration within the CMakeLists.txt file.

1. Create a workspace for CMake.

```
renesas@builder-pc:~$ mkdir ~/cmake_python
renesas@builder-pc:~$ cd ~/cmake_python
renesas@builder-pc:~/cmake_python$ mkdir build src
```

2. Organize the project structure as shown below:

```
renesas@builder-pc:~/cmake_python$ tree
.
├── build
├── CMakeLists.txt
└── src
    ├── tkinter_wrapper.sh
    └── main.py
```

`CMakeLists.txt`, `tkinter_wrapper.sh`, and `main.py` will be created later in the next steps.

3. Modify the Python program; this program is the same as Tkinter example in section Python GUI Programming with Tkinter.

Copy this example content and paste it into this Python file.

```
renesas@builder-pc:~/cmake_python$ vi src/main.py
```

4. Create a Tkinter wrapper shell script to run the application.

```
renesas@builder-pc:~/cmake_python$ vi src/tkinter_wrapper.sh
```

Then, copy the content below to the script.

```
#!/bin/bash

/usr/bin/python3 /usr/local/share/tkinter_example/main.py
```

5. Configure the CMakeLists.txt for packaging a Python program.

```
renesas@builder-pc:~/cmake_python$ vi CMakeLists.txt
```

Then, copy the contents below to the file:

```
cmake_minimum_required(VERSION 3.10)
project(TkinterExample)

# Define script and wrapper
set(SCRIPT_NAME "src/main.py")
set(WRAPPER_SCRIPT "src/tkinter_wrapper.sh")
set(EXEC_NAME "tkinter_example")

# Define installation paths
set(INSTALL_DIR "/usr/local/bin")
set(INSTALL_SCRIPT_DIR "/usr/local/share/tkinter_example")

# Install the wrapper script
configure_file(${CMAKE_SOURCE_DIR}/${WRAPPER_SCRIPT} ${CMAKE_BINARY_DIR}/${EXEC_NAME} @ONLY)
install(PROGRAMS ${CMAKE_BINARY_DIR}/${EXEC_NAME} DESTINATION ${INSTALL_DIR})
install(FILES ${CMAKE_SOURCE_DIR}/${SCRIPT_NAME} DESTINATION ${INSTALL_SCRIPT_DIR})

# Packaging configuration
set(CPACK_GENERATOR "DEB")
set(CPACK_PACKAGE_NAME "tkinter_example")
set(CPACK_PACKAGE_VERSION "1.0.0")
set(CPACK_PACKAGE_CONTACT "your-email@example.com")
set(CPACK_DEBIAN_PACKAGE_ARCHITECTURE "arm64")
include(CPack)
```

6. Packaging the program.

```
renesas@builder-pc:~/cmake_python$ cd build/
renesas@builder-pc:~/cmake_python/build$ cmake ../
-- The C compiler identification is GNU <gnu-version>
-- The CXX compiler identification is GNU <gnu-version>
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler:
/home/renesas/esdk/5.1.4/sysroots/x86_64/usr/bin/aarch64-poky-linux/aarch64-poky-
linux-gcc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler:
/opt/poky/5.1.1/sysroots/x86_64/usr/bin/aarch64-poky-linux/aarch64-poky-linux-g++
- skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done (0.3s)
-- Generating done (0.0s)
-- Build files have been written to: /home/renesas/cmake_python/build
renesas@builder-pc:~/cmake_python/build$ cpack
CPack: Create package using DEB
CPack: Install projects
CPack: - Run preinstall target for: TkinterExample
CPack: - Install project: TkinterExample []
CPack: Create package
-- CPACK_DEBIAN_PACKAGE_DEPENDS not set, the package will have no dependencies.
CPack: - package: /home/renesas/cmake_python/build/tkinter_example-1.0.0-
Linux.deb generated.
```

After completing, confirm that the Debian package (.deb) is generated in the build folder.

```
renesas@builder-pc:~/cmake_python/build$ ls
CMakeCache.txt  cmake_install.cmake  _CPack_Packages
install_manifest.txt  tkinter_example
CMakeFiles      CPackConfig.cmake    CPackSourceConfig.cmake  Makefile
tkinter_example-1.0.0-Linux.deb
```

Then, the Debian package installer “tkinter_example-1.0.0-Linux.deb” can be transferred to the target board using the SCP tool as below or other methods, such as a USB drive or NFS (Network File System).

```
renesas@builder-pc:~/cmake_python/build$ scp tkinter_example-1.0.0-Linux.deb
root@192.168.5.58:/home/root
```

12.4 Run Sample Applications

Power on the RZ/G2L-SBC and start the system. Once the system has booted, transfer the binary package that is built using the SDK with CMake, which is mentioned in section Build a sample application using the eSDK with **CMake**. Then, run the sample application as follows:

```
NOTICE:  BL2: <version>
NOTICE:  BL2: Built : <date>
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: <version>
NOTICE:  BL31: Built : <date>
…
rz-cmn login: root
root@rz-cmn:~# ./helloworld

Hello, World!
root@rz-cmn:~#
```

12.5 Install and Run Debian application packages by using DPKG

After shipping the Debian package installer to the target board, the package can be installed using dpkg.

The steps are:

1. List out all available .deb files to make sure all .deb files have been shipped to the target board.
2. Install the C program by running `dpkg -i helloworld-1.0.0-Linux.deb`.
3. Install the Python program by running `dpkg -i tkinter_example-1.0.0-Linux.deb`.

```
NOTICE:  BL2: <version>
NOTICE:  BL2: Built : <date>
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: <version>
NOTICE:  BL31: Built : <date>
…
rz-cmn login: root
root@rz-cmn:~# ls
audios  demo  helloworld-1.0.0-Linux.deb  tkinter_example-1.0.0-Linux.deb  images
info  v4l2-init.sh  videos
root@rz-cmn:~# dpkg -i helloworld-1.0.0-Linux.deb
Selecting previously unselected package helloworld.
(Reading database ... 4 files and directories currently installed.)
Preparing to unpack helloworld-1.0.0-Linux.deb ...
Unpacking helloworld (1.0.0) ...
Setting up helloworld (1.0.0) ...
root@rz-cmn:~#
root@rz-cmn:~# dpkg -i tkinter_example-1.0.0-Linux.deb
Selecting previously unselected package tkinter_example.
(Reading database ... 4 files and directories currently installed.)
Preparing to unpack tkinter_example-1.0.0-Linux.deb ...
Unpacking tkinter_example (1.0.0) ...
Setting up tkinter_example (1.0.0) ...
```

After installation, confirm that the package is correctly installed by running the following.

```
root@rz-cmn:~# dpkg -l
ii  helloworld     1.0.0        arm64        HelloWorld built using CMake
ii  tkinter_example 1.0.0       arm64        TkinterExample built using CMake
```

This completes the installation. The applications are ready for use.

There are a few ways to run the application:

1. Directly from the installation location.
2. Call it from /usr/local/bin/<your_application>.
3. Call it directly from anywhere if it is installed within the PATH search.

The following command lists all the files that were installed with their full paths:

```
root@rz-cmn:~# dpkg -L helloworld
/usr
/usr/local
/usr/local/bin
/usr/local/bin/hello
root@rz-cmn:~# /usr/local/bin/hello
Hello, World!
```

For applications that have a graphical interface, the display ID needs to be set in the environment. For this reason, export the DISPLAY if you are using an environment where the display is not automatically set, as shown below:

Switch to the user ‘weston’ to run the Tkinter application

```
root@rz-cmn:~# su - weston
```

```
root@rz-cmn:~$ export DISPLAY=:0
root@rz-cmn:~$ dpkg -L tkinter_example
/usr
/usr/local
/usr/local/bin
/usr/local/bin/tkinter_example
/usr/local/share
/usr/local/share/tkinter_example
/usr/local/share/tkinter_example/main.py
root@rz-cmn:~$ tkinter_example
```

12.6 Using devtool in the Yocto eSDK

12.6.1 Overview

The devtool utility is part of the Yocto Project’s Extensible SDK (eSDK). It provides an isolated workspace for modifying, testing, and maintaining recipes without altering upstream metadata.

This section focuses on the **core commands** used in day-to-day development, especially for the Linux kernel, device trees, and driver modifications on the Renesas RZ Common System.

12.6.2 Prerequisites

Before using the devtool, ensure the following:

1. The Yocto eSDK has been installed and extracted; see section How to extract the eSDK for detailed information.
2. The environment setup script has been sourced:

   ```
   $ source /path/to/poky_sdk/environment-setup-<arch>-poky-linux
   ```

   For example:

   ```
   renesas@builder-pc:~$ source ~/poky_sdk/environment-setup-cortexa55-poky-linux
   SDK environment now set up; additionally you may now run devtool to perform
   development tasks.
   Run devtool --help for further details.
   ```

12.6.3 Common Usage Scenarios

This section explains how to use the main **devtool** commands in the Yocto Extensible SDK, along with examples based on Linux kernel development for Renesas RZ boards.

12.6.3.1 devtool modify – Preparing a Workspace

The devtool modify command checks out the source code for a recipe into a local workspace, allowing changes without touching upstream layers. This is usually the first step in customizing the kernel, device trees, or drivers.

Syntax:

```
renesas@builder-pc:~$ devtool modify <recipe>
```

Example (modify linux-yocto recipe):

```
renesas@builder-pc:~$ devtool modify linux-yocto
```

When executed, this:

- Creates a workspace copy of the kernel source under:

  ~/poky_sdk/workspace/sources/linux-yocto/
- Generates a .bbappend for linux-yocto in the workspace's recipe area.
- Prepares the environment for kernel modifications.

(1) Adding Kernel Configuration

There are two possible methods to add kernel configuration for linux-yocto:

1. Edit the in- tree defconfig directly:

   ~/poky_sdk/workspace/sources/linux-yocto/arch/arm64/configs/renesas_defconfig
2. Add a configuration fragment (.cfg) file
   - Create the append structure

     ```
     renesas@builder-pc:~$ mkdir -p ~/poky_sdk/workspace/appends/<recipe-name>/files
     ```

     For example, create an append structure for linux-yocto

     ```
     renesas@builder-pc:~$ mkdir -p ~/poky_sdk/workspace/appends/linux-yocto/files
     ```

   - Place the fragment in files/, e.g., usb-serial-ftdi.cfg:

     ```
     CONFIG_USB_SERIAL=y
     CONFIG_USB_SERIAL_FTDI_SIO=y
     ```

   - Create/modify the bbappend

     ```
     renesas@builder-pc:~$ vim ~/poky_sdk/workspace/appends/linux-yocto/\
     linux-yocto_6.10.bbappend
     ```

     Add the usb-serial.cfg file to SRC_URI to tell the kernel to apply this fragment

     ```
     SRC_URI:append = " file://usb-serial-ftdi.cfg"
     ```

12.6.3.2 devtool build – Building the Recipe

After making changes in the workspace, use devtool build to compile the recipe.

Syntax

```
renesas@builder-pc:~$ devtool build <recipe>
```

Example:

```
renesas@builder-pc:~$ devtool build linux-yocto
```

What it does:

- Uses the workspace sources for <recipe> (created by devtool modify <recipe>).

- Runs the recipe's standard BitBake tasks (e.g., do_compile, do_install, packaging) with dependency handling.
- Produces deployable artifacts for the recipe (binaries, libraries, firmware, kernel modules, etc.), depending on the recipe's class.

What it does not do

- Does not rebuild a complete image. To include changes in an image, use devtool build-image <image>
- To integrate changes into a bootable image, use devtool build-image <image>.

Note: When artifacts generated by `devtool build` (for example, `Image`, DTBs, or kernel modules) are intended for use with Ubuntu-based root file systems (such as `ubuntu-core-image` or `ubuntu-lxde-image`), the build must be performed with `DISTRO=ubuntu-tiny`.

Export the variable before running the build:

$ export DISTRO=ubuntu-tiny

$ devtool build <recipe>

Otherwise, the generated artifacts will not be compatible with the Ubuntu image.

(1) Typical output (generic locations):

Work directory (per-recipe, per-machine):

- <sdk-root>/tmp/work/<machine>-poky-linux/<recipe>/<version>/
  Contains intermediate build results, the image/ directory with installed files, and logs.
- Deployed artifacts (if the recipe deploys output): <sdk-root>/tmp/deploy/

For example, when building linux-yocto, the artifacts can be collected from these locations:

- Kernel modules (.ko files):~/poky_sdk/tmp/work/rz-cmn-poky-linux/linux-yocto/6.18.20+git/image/usr/lib/modules/6.18.20-yocto-standard/kernel/
- Device tree and Image:
  - In work directory: ~/poky_sdk/tmp/work/rz-cmn-poky-linux/linux-yocto/6.18.20+git/image/boot/
  - In deploy directory (as the recipe deploys output): ~/poky_sdk/tmp/deploy/images/rz-cmn/target/images/linux

12.6.3.3 devtool reset – Cleaning Up

When experimentation is complete, use devtool reset to remove the workspace copy and restore the recipe to its original state. This ensures that subsequent builds use the unmodified upstream recipe.

Command:

```
renesas@builder-pc:~$ devtool reset <recipe>
```

Example:

```
renesas@builder-pc:~$ devtool reset linux-yocto
```

This command deletes the workspace sources and temporary append files, cleaning up after testing (e.g., kernel debug options). Any changes not preserved with devtool update-recipe will be lost.

12.6.3.4 devtool build-image – Building Target Image

The devtool build-image command builds a complete target image that includes all recipes currently under modification. This is useful to verify integration of changes into a full root filesystem, not just individual components.

Command:

```
renesas@builder-pc:~$ devtool build-image <image>
```

Example:

```
renesas@builder-pc:~$ devtool build-image core-image-weston
```

When executed, this command:

- Rebuilds the specified image recipe.
- Automatically includes outputs from any recipes currently modified in the workspace.
- Produces a bootable image in the deploy directory:
    - .wic file (complete bootable image):
      <sdk-root>/tmp/deploy/images/<machine>/target/images/
    - Compressed root filesystem (for flashing or NFS root):
      <sdk-root>/tmp/deploy/images/<machine>/target/images/rootfs

Use devtool build-image after testing an individual recipe (e.g., linux-yocto) to confirm integration into the final image.

- The resulting image can be flashed to the target device for full system validation.
- This is equivalent to running bitbake <image>

Note: If building the target image with devtool fails, it is likely that the workspace is not clean. Please clean the workspace (see Section devtool reset – Cleaning Up for instructions on how to reset the workspace for a specific recipe) and then rebuild the target image

13. Remote Debugging using GDBServer

GDBServer is utilized to facilitate remote debugging on the target board. GDBServer enables the debugging process to run on the target machine while being controlled from a different system (the host machine) via a network connection.

This setup is particularly beneficial for application development, as it allows the execution and debugging of programs on the target board while providing the capability to view and control the process from the host machine.

To ensure that all necessary tools and libraries for debugging are available, preparations must be made on both the host and target machines. With this preparation complete, the next step is to proceed with the remote debugging process.

13.1 Prepare GDB on the Host Machine

GDB has two components to work with. One is the host side ‘gdb’ debugger. The other is the target side ‘gdbserver’. The GDB (GNU debugger) is executed on the host side. It is executed on your host system to connect to the target system. It is always available within the eSDK. The eSDK installation as described in Section 12.1 is a prerequisite for this operation. To set up the environment that would use the GDB targeting the target board from the eSDK, simply run the poky environment script as follows:

```
renesas@builder-pc:~$ source ~/esdk/5.1.4/environment-setup-cortexa55-poky-linux
SDK environment now set up; additionally you may now run devtool to perform
development tasks.
Run devtool --help for further details.
```

Note: The user needs to run the above command once for each shell session. In addition, source’ is a bash-specific call. The POSIX convention is to use. ~/esdk/5.1.4/environment-setup-cortexa55-poky-linux’. Bash equates ‘source’ to ‘.’.

To confirm GDB is ready to use, run the following command and check the result:

```
renesas@builder-pc:~$ echo ${GDB}

aarch64-poky-linux-gdb
```

13.2 Install GDBServer on RZ

By default, GDBServer is not installed on the target board. It is necessary to install it using APT. Execute the following commands to install GDBServer:

```
root@rz-cmn:~# apt-get update

root@rz-cmn:~# apt-get install gdbserver
```

Ensure that internet access is available before executing apt-get update.

This concludes the preparation of the basic host environment. The next section will discuss the remote debugging process.

13.3 Remote Debugging Example

13.3.1 Remote Debugging on CLI

CLI (Command Line Interface) is a text-based user interface used to interact with computer programs and operating systems. Unlike graphical user interfaces (GUIs), where users interact with visual elements (like buttons and icons), a CLI requires users to input commands in text form. This is basically a shell environment used in all operating systems as a foundational method of interacting with the system. For the purposes of this section, we assume Ubuntu bash as the interactive application.

Firstly, run GDBServer with a specific network port (`2000`is the assigned port in this case) and the program `hello-gdbserver` as a parameter on the target as follows:

```
root@rz-cmn:~# gdbserver localhost:2000 hello-gdbserver

Process /home/root/hello-gdbserver created; pid = 358

Listening on port 2000
```

The content before compiling the `hello-gdbserver` program:

```
#include <stdio.h>

int main() {

        int i;

        printf("Program to demonstrate gdbserver debugging!\n");
        printf("Print from 1 to 10\n");

        for (i = 1;i <= 10;i++)
                printf("%d\n", i);

        printf("Program completed!\n");

        return 0;
}
```

The target's IP address is required for use on the host later. In this example, the IP address 169.254.43.30 will be used.

```
root@rz-cmn:~# ifconfig end1
end1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500  metric 1
        inet 169.254.43.30  netmask 255.255.0.0  broadcast 169.254.255.255
        inet6 fe80::1ea0:d3ff:fe20:119b  prefixlen 64  scopeid 0x20<link>
        ether 1c:a0:d3:20:11:9b  txqueuelen 1000  (Ethernet)
        RX packets 34497  bytes 2657706 (2.5 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 68954  bytes 97379412 (92.8 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 133
```

Next, launch GDB on the host.

```
renesas@builder-pc:~$ aarch64-poky-linux-gdb
GNU gdb (GDB) <GNU version>
Copyright (C) <year> Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux --target=aarch64-poky-linux".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb)
```

Use `target remote` with the IP address and the assigned network port to connect to the target.

```
(gdb) target remote 169.254.43.30:2000
Remote debugging using 169.254.43.30:2000
Reading /home/root/hello-gdbserver from remote target...
warning: File transfers from remote targets can be slow. Use "set sysroot" to
access files locally instead.
Reading /home/root/hello-gdbserver from remote target...
Reading symbols from target:/home/root/hello-gdbserver...
Reading /lib64/ld-linux-aarch64.so.1 from remote target...
Reading /lib64/ld-linux-aarch64.so.1 from remote target...
Reading symbols from target:/lib64/ld-linux-aarch64.so.1...
Reading /lib64/ld-2.31.so from remote target...
Reading /lib64/.debug/ld-2.31.so from remote target...
Reading /lib64/.debug/ld-2.31.so from remote target...
Reading symbols from target:/lib64/.debug/ld-2.31.so...
0x0000fffff7fcd0c0 in _start () from target:/lib64/ld-linux-aarch64.so.1
```

Then, add a breakpoint at the `main` function to stop the program at that function in the next step:

```
(gdb) b main

Breakpoint 1 at 0xaaaaaaaa07cc: file hello-gdbserver.c, line 7.
```

At this point, the `continue` command can be used to resume execution and jump to the main function.

```
(gdb) continue

Continuing.

Reading /lib64/libc.so.6 from remote target...

Reading /lib64/libc-<glibc-version>.so from remote target...

Reading /lib64/.debug/libc-<glibc-version>.so from remote target...

Reading /lib64/.debug/libc-<glibc-version>.so from remote target...


Breakpoint 1, main () at hello-gdbserver.c:7

warning: Source file is more recent than executable.

7               printf("Program to demonstrate gdbserver debugging!\n");
```

Then, type `continue` to execute the remainder of the program.

```
(gdb) continue

Continuing.

[Inferior 1 (process 342) exited normally]
```

Eventually, run `quit` to exit GDB and stop the debugging section.

```
(gdb) quit
```

In parallel, the output can be monitored on the target device.

```
Remote debugging from host ::ffff:169.254.43.86, port 40666

Program to demonstrate gdbserver debugging!

Print from 1 to 10

1

2

3

4

5

6

7

8

9

10

Program completed!


Child exited with status 0

root@rz-cmn:~#
```

13.3.2 Remote Debugging on Visual Studio Code

In the previous subsection, remote debugging using the command line was discussed, specifically with GDB and GDBServer. While this method is effective, it can be complex and challenging, particularly for developers who may not be familiar with command-line operations.

This section describes how to set up and use Visual Studio Code (VSCode) for remote debugging with the GDB (GNU Debugger) extension. Using VSCode simplifies the debugging process by providing a user-friendly graphical interface that streamlines the workflow, making it easier to troubleshoot and test C/C++ applications running on the target board.

Here's how to get started:

1. Install the C/C++ Extension (If it has not been installed yet):
   - Open VSCode.
   - Go to the Extensions tab on the left side (or press Ctrl + Shift + X).
   - Search for C/C++.
   - Click Install to add the extension.

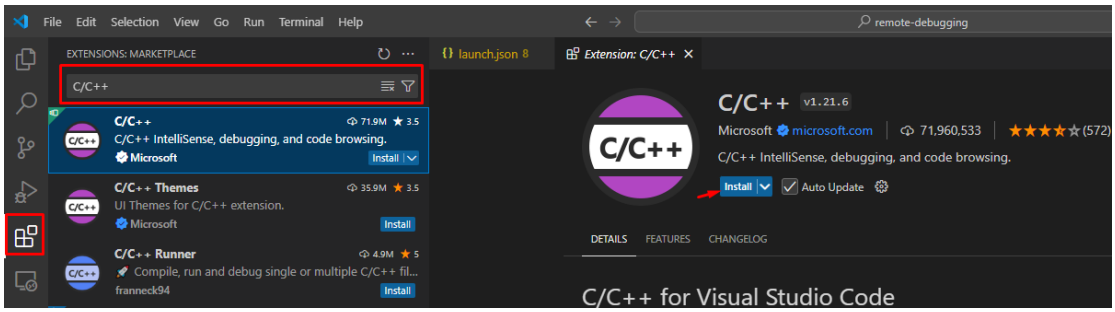


Figure 114. C/C++ Extension in VSCode

2. Create a Workspace:
   - Create a new workspace (you can name it remote debugging).
   - Create a folder within this workspace and place your program file, hello-gdbserver.c in it.
   - Build the execution file using eSDK, we assume that you have sourced the environment.

```
renesas@builder-pc:~/remote-debugging/program$ $CC $CFLAGS hello-gdbserver.c -o
hello-gdbserver
```

3. Set Up Debug Configuration:
   - Open the Run and Debug view in VSCode (or press Ctrl + Shift + D).
   - Click on create a launch.json file to configure the debugger.

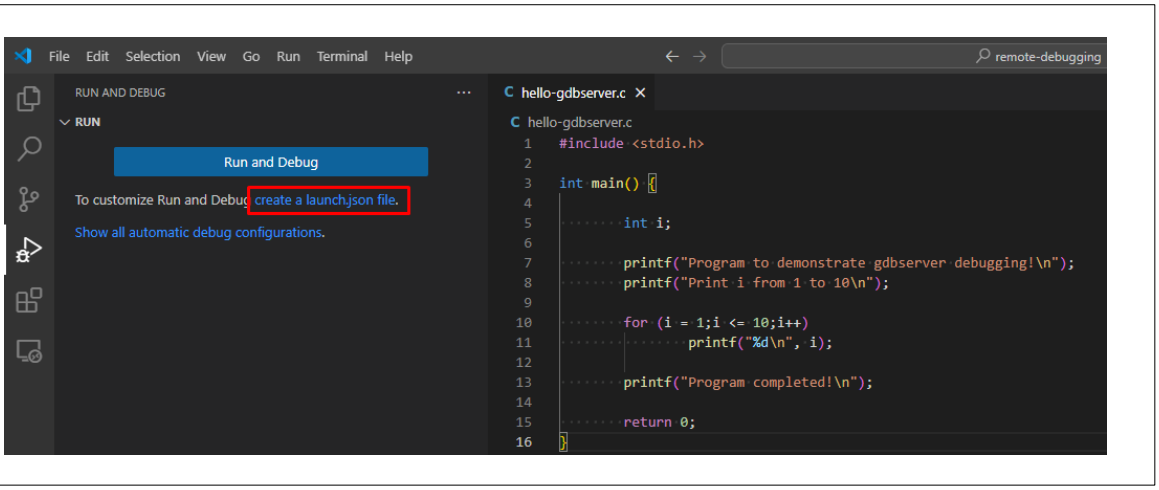


Figure 115. Create a launch.json file in VSCode Debugger

- Select the C++ (GDB) option and customize the configuration as needed.

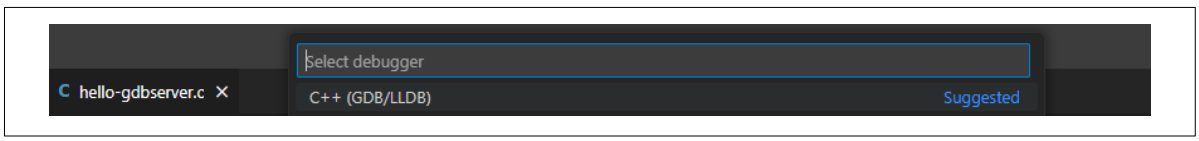


Figure 116. Select C++ GDB as Debugger

- Place the content below in launch.json file:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "gdb",
            "type": "cppdbg",
            "request": "launch",
            "program": "</local/path/to/the/executable>",
            "cwd": "${workspaceFolder}",
            "stopAtEntry": true,
            "stopAtConnect": true,
            "MIMode": "gdb",
            "miDebuggerPath": "</path/to/gdb>",
            "miDebuggerServerAddress": "<target_addr>:<port>",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

For example:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "gdb",
            "type": "cppdbg",
            "request": "launch",
            "program": "/home/renesas/remote-debugging/program/hello-gdbserver",
            "cwd": "${workspaceFolder}",
            "stopAtEntry": true,
            "stopAtConnect": true,
            "MIMode": "gdb",
            "miDebuggerPath":
"/home/renesas/esdk/5.1.4/tmp/sysroots/x86_64/usr/bin/aarch64-poky-linux/aarch64-
poky-linux-gdb",
            "miDebuggerServerAddress": "169.254.43.30:2000",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

- Ensure your workspace appears as follows:

```
renesas@builder-pc:~/remote-debugging$ tree -a

.
├── program
│   ├── hello-gdbserver
│   └── hello-gdbserver.c
└── .vscode
    └── launch.json

2 directories, 4 files
```

4. Connect to the Remote Target:
   - As with the CLI section, start the GDBServer on the remote device and specify the target application.

```
root@rz-cmn:~# gdbserver localhost:2000 hello-gdbserver

Process /home/root/hello-gdbserver created; pid = 358

Listening on port 2000
```

5. Start the debugging.
   - Back in VSCode, select your launch configuration.
   - Place the breakpoint within the hello-gdbserver.c file in VSCode.
   - Click the Start Debugging button (green play icon) to begin the debugging session.

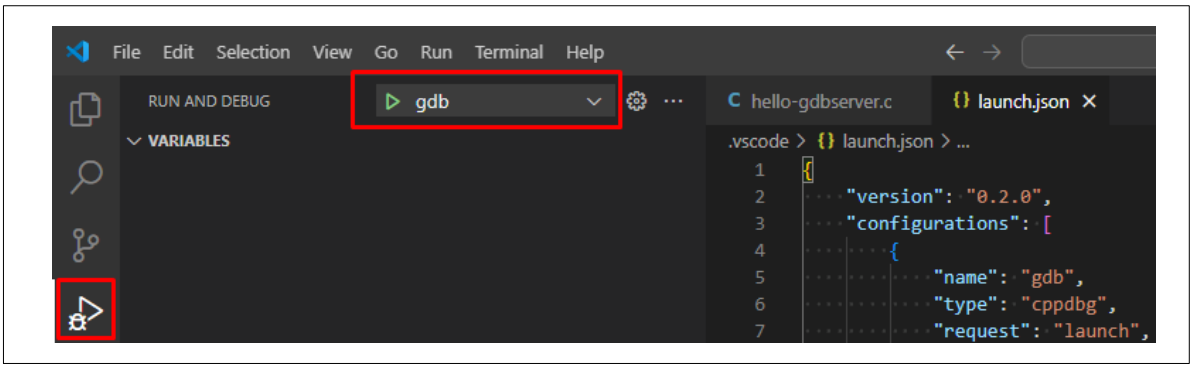


Figure 117. Running the Debugger for Remote Debugging in VSCode

   - Use F5 to continue execution, F10 to step over the current line, and F11 to step into functions.

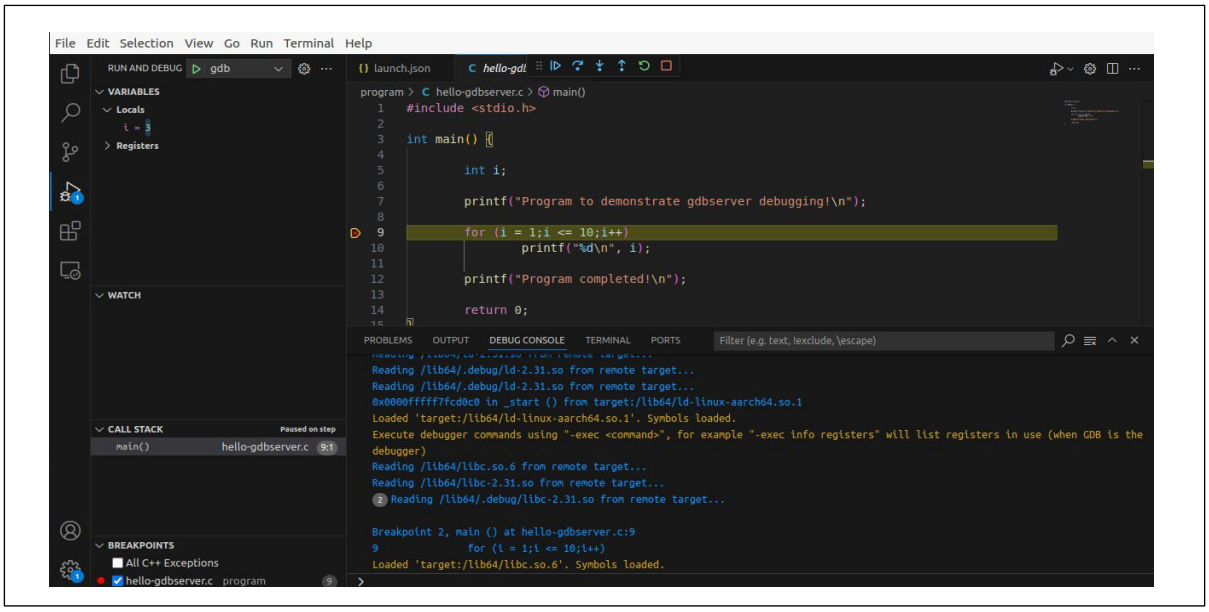


Figure 118. Step through each step in Debug Mode in VSCode

13.3.3 Remote Debugging on Eclipse IDE

In the previous section, the use of VSCode for remote debugging with GDB and GDBServer was discussed. While VSCode offers a modern and user-friendly environment, many developers prefer Eclipse IDE for its comprehensive toolset and robust support for C/C++ development. This section explains how to set up and use the Eclipse IDE for remote debugging with GDB.

1. Install the Eclipse IDE (if not already installed) by following the official instructions on the Eclipse website: Eclipse Installer 2024-09 R | Eclipse Packages
2. Create a C/C++ project:

- Open Eclipse and navigate to File > New > C/C++ Project.
- Create a new C Empty Project; choose Cross GCC.

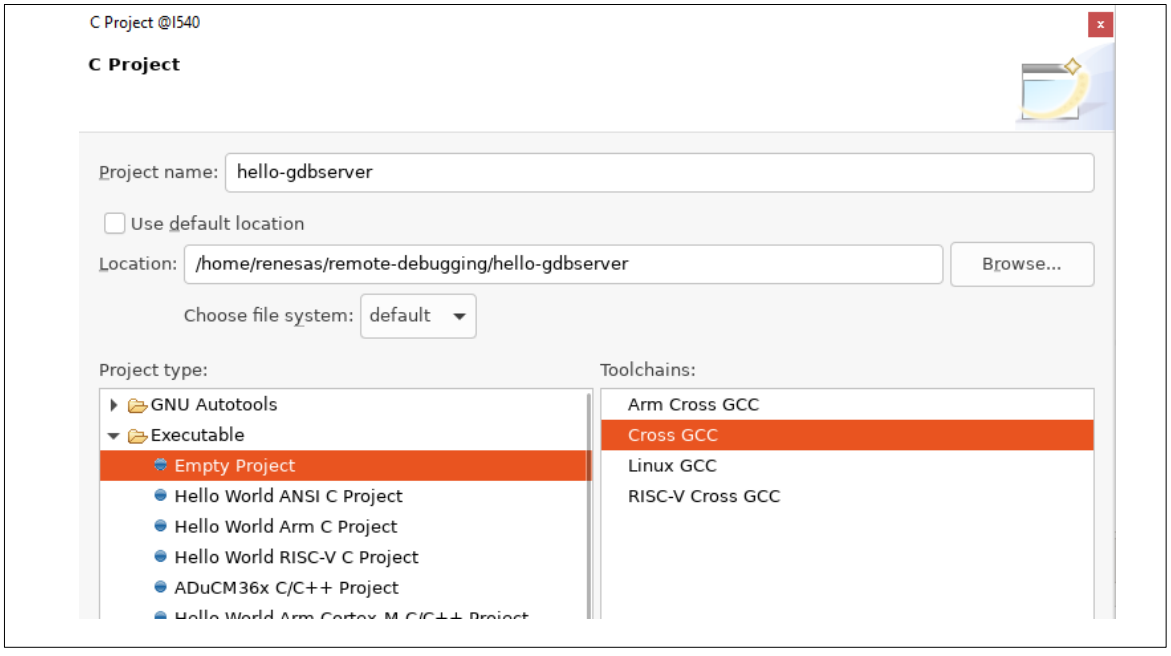


Figure 119. Create a C Project in Eclipse

Click Next, then Finish, and paste the content from hello-gdbserver.c into the C file.

3. Configure the Cross Toolchain.

- Go to Project > Properties.
- In the left pane, select C/C++ Build > Settings.

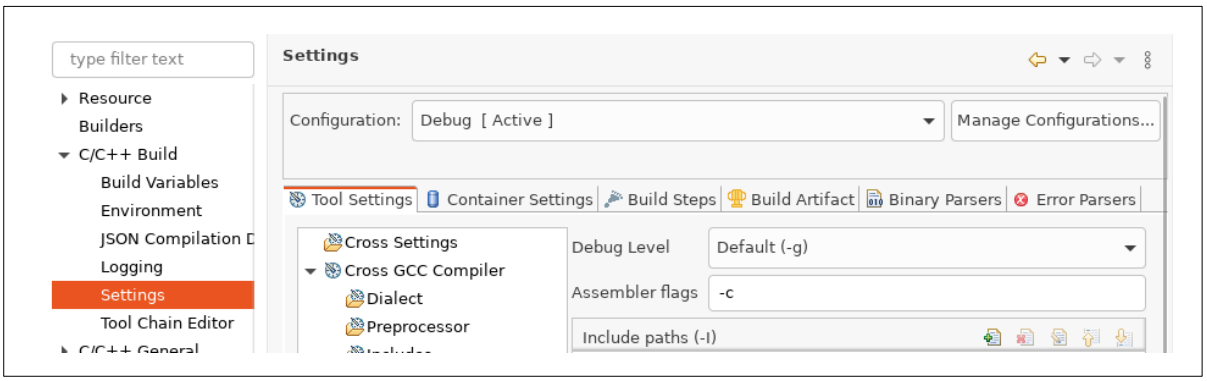


Figure 120. Configuring the cross toolchain in Eclipse project properties

- Under the Tool Settings tab, configure the Cross Settings as follows:
  - Prefix: aarch64-poky-linux-
  - Path: </path/to/your/aarch64-poky-linux>.

  For example:

  - Prefix: aarch64-poky-linux-
  - Path: /home/renesas/esdk/5.1.4/tmp/sysroots/x86_64/usr/bin/aarch64-poky-linux.

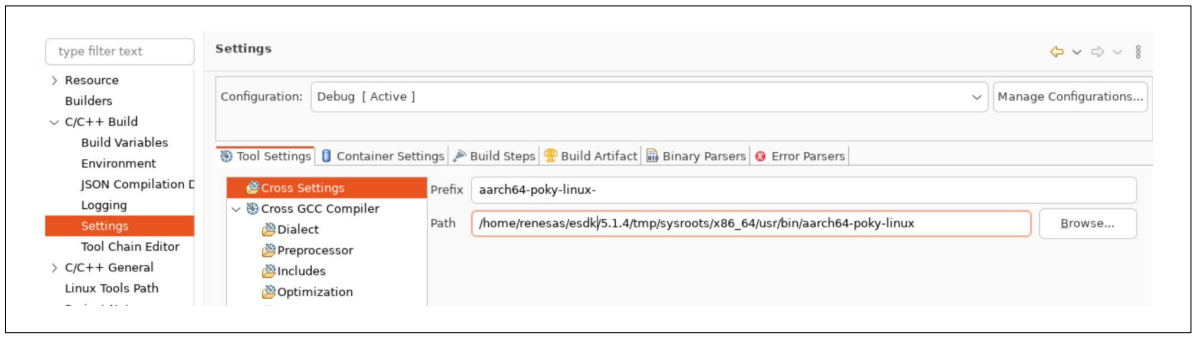


Figure 121. Configuring cross-compiler settings in the Eclipse tool settings

- In the Includes section, specify the include paths:
  - Include paths: /home/renesas/esdk/5.1.4/tmp/sysroots/rz-cmn/usr/include

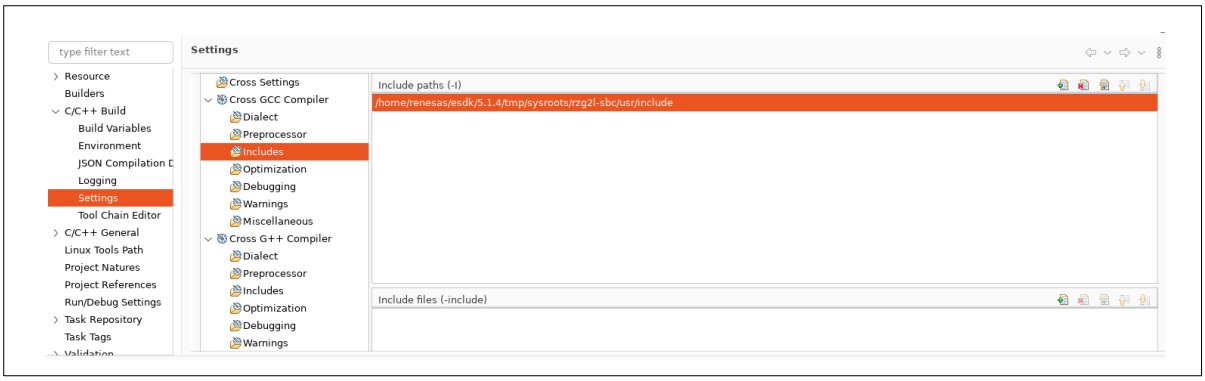


Figure 122. Configuring the include path in the Eclipse tool settings

- In the Cross GCC Linker section, go to Libraries and specify the library search path:
  - Library search path: /home/renesas/esdk/5.1.4/tmp/sysroots/x86_64/usr/lib

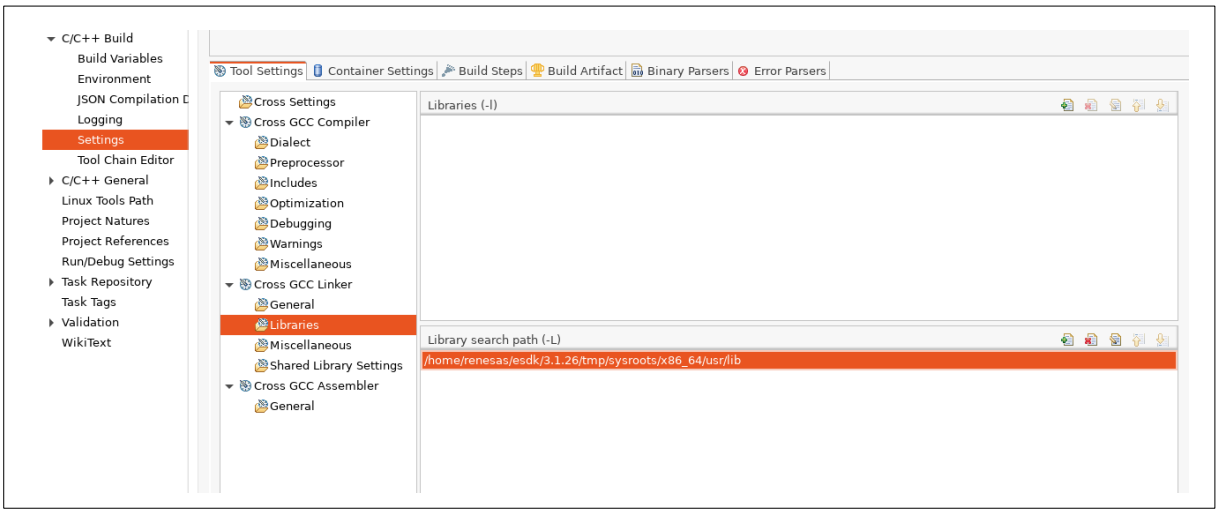


Figure 123. Configuring library paths in the Eclipse tool settings

- In the Miscellaneous section, specify the linker flags:
  - Linker flags: --sysroot=/home/renesas/esdk/5.1.4/poky_sdk/tmp/sysroots/rz-cmn

Figure 124. Configuring linker flags for Sysroot in Eclipse tool settings

4. Configure Eclipse to connect to the GDB Server:

- In Eclipse, go to the Run menu and select Debug Configurations.
- Under the Debugger tab, select C/C++ Remote Application.

- In the Main tab, click Edit to configure the target IP/hostname. After setting it up, the Remote option will appear under Connection Type; select Remote to enable the remote connection.

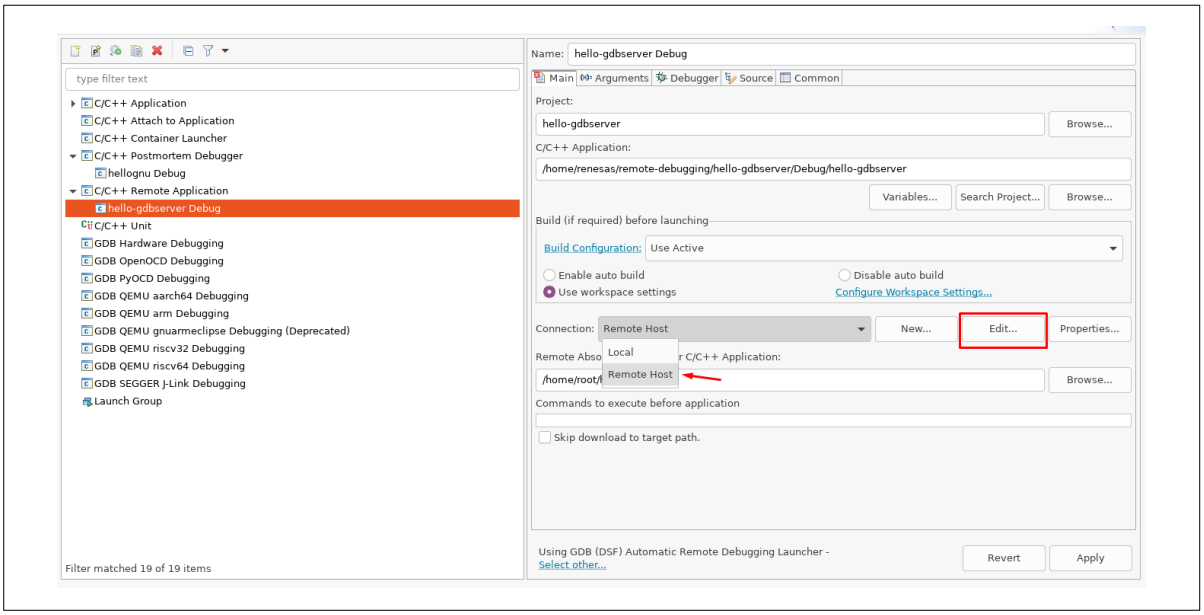


Figure 125. Debug configuration settings in Eclipse

- Host: Enter the IP address of the RZ/G2L-SBC.
- User: Enter the username of the RZ/G2L-SBC (typically root).
- Authentication: Choose between key-based authentication or password-based authentication, depending on your preference.
- Finally, click Finish to complete the setup for the SSH session.

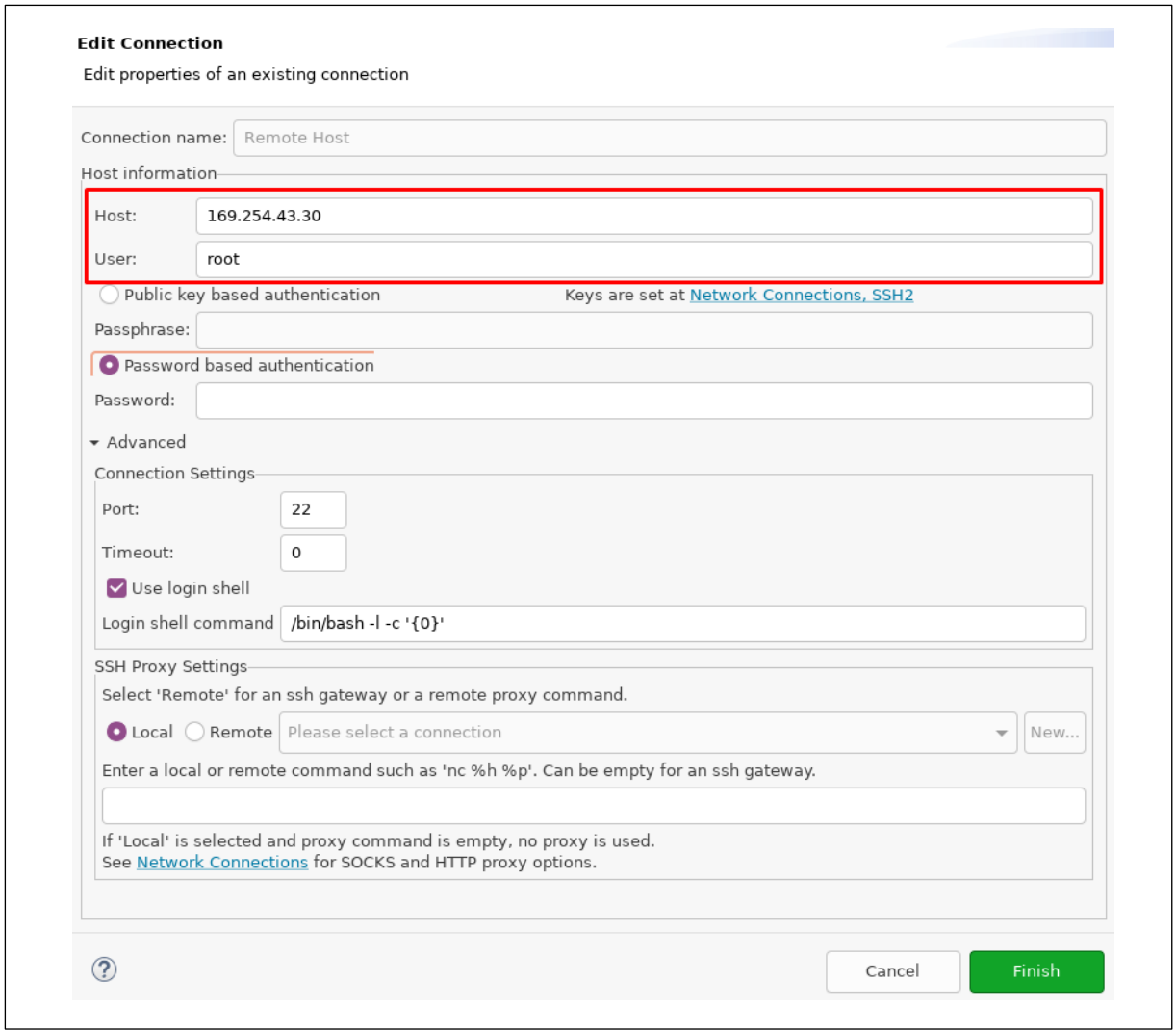


Figure 126. Configuring SSH connection settings in debug configurations

- In the Remote Absolute File Path field, specify the location where Eclipse will copy the program to the target board. Click Browse to connect via SSH and select the target location, or manually enter the path on the target board.

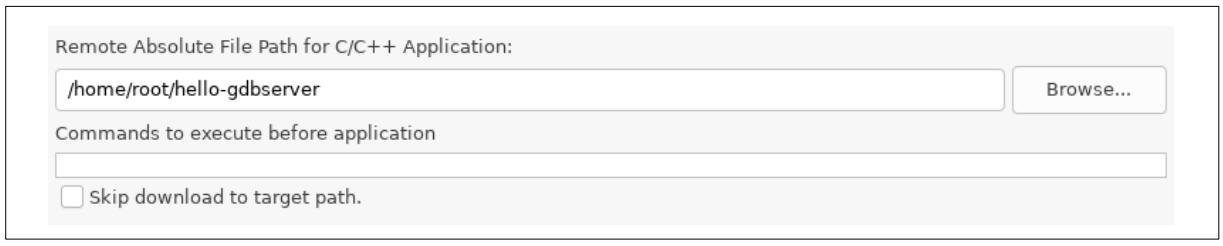


Figure 127. Configuring the remote absolute file path in debug configurations

- In the Debugger tab:
  - In GDB Debugger: Provide the path to your cross-compiled GDB (For example, /home/renesas/esdk/5.1.4/tmp/sysroots/x86_64/usr/bin/aarch64-poky-linux/aarch64-poky-linux-gdb).

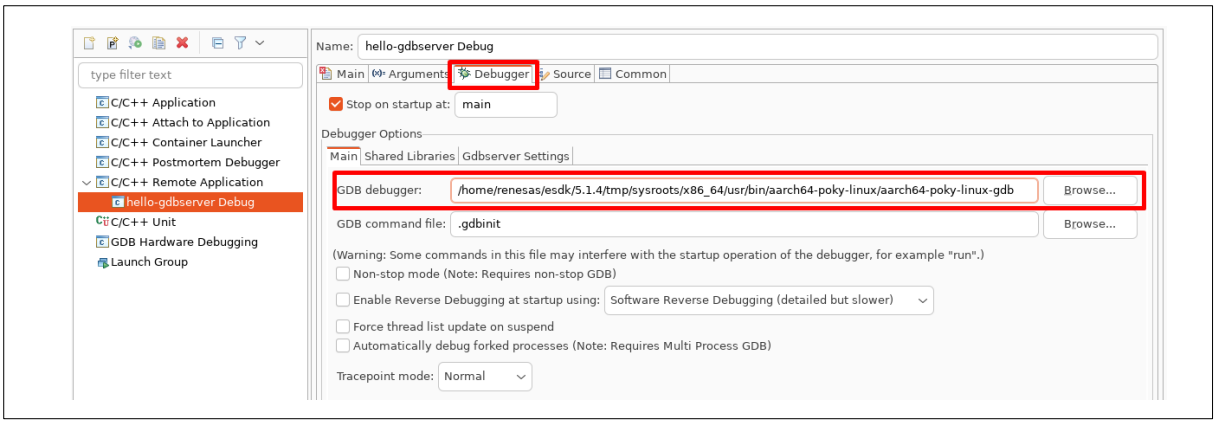


Figure 128. Configuring the GDB debugger in Eclipse debug configurations

5. Start the Debugging Session:

- After configuring the debug settings, click Apply and then Debug.
- Eclipse will attempt to connect to the GDB server running on the target device.
- If the connection is successful, it will be possible to set breakpoints, step through the code, and inspect variables just as in a local debugging session.

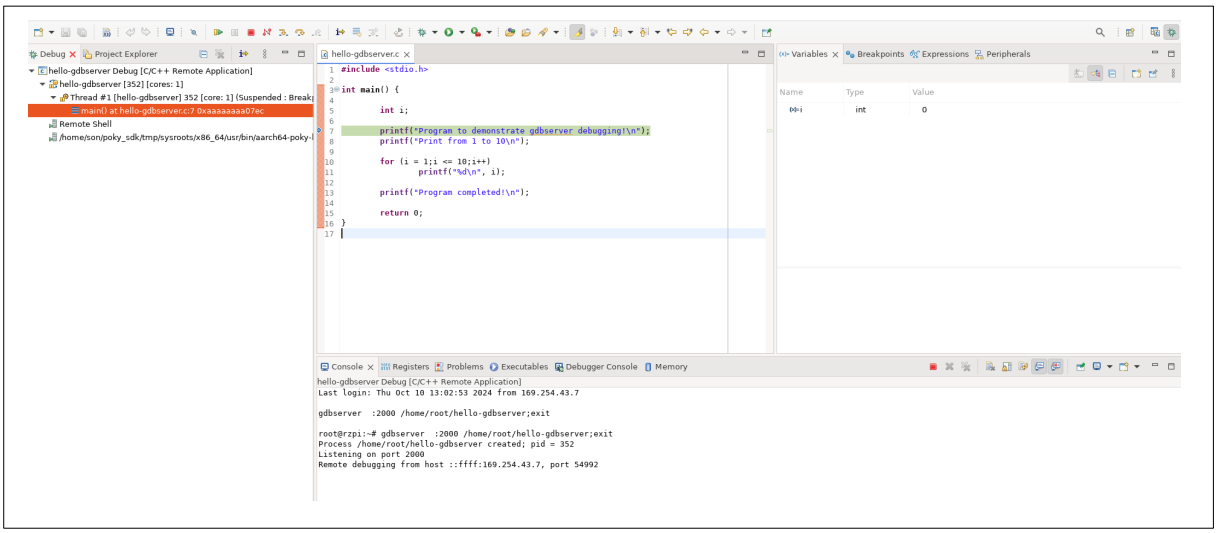


Figure 129. Start the debugging session in Eclipse

Press F5 to step into, F6 to step over, or F8 to resume and monitor the variables.

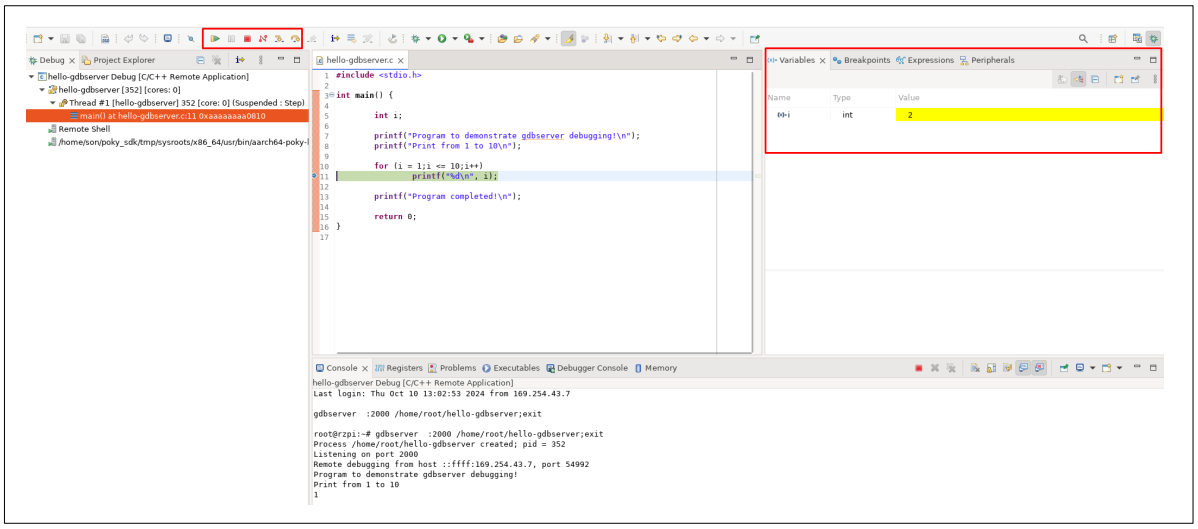


Figure 130. Go through each step in debug mode in Eclipse

The path of the compiler may need to be adjusted to reflect the specific system configuration.

13.4 Postmortem Analysis Example

This section provides an overview of postmortem analysis, a critical process for diagnosing application crashes by examining core dump files. It details how developers can analyze these core dumps to pinpoint the exact lines of code that led to an error, allowing for effective troubleshooting and resolution of issues.

13.4.1 Postmortem Analysis on CLI

This subsection describes how to perform a postmortem analysis using the command-line interface (CLI). It emphasizes the steps for loading core dump files with CLI tools, enabling developers to navigate directly to the lines of code where errors occurred. The section highlights the efficiency of command-line tools for diagnosing issues quickly.

1. Create a simple C program that intentionally causes a segmentation fault. For example, the file named `segfault_example.` has the following content:

```
#include <stdio.h>

int main() {
        int *ptr = NULL;

        printf("Attempting to dereference a NULL pointer...\r\n");

        *ptr = 42;

        return 0;
}
```

2. Source the environment and compile the segfault_example.c program.

```
renesas@builder-pc:~$ source ~/esdk/5.1.4/environment-setup-cortexa55-poky-linux
SDK environment now set up; additionally you may now run devtool to perform
development tasks.
Run devtool --help for further details.
renesas@builder-pc:~/remote-debugging/segfault_program$ $CC $CFLAGS
segfault_example.c -o segfault_example
```

3. Transfer the program to RZ/G2L-SBC.

```
renesas@builder-pc:~/remote-debugging/segfault_program$ scp segfault_example
root@169.254.43.30:/home/root
```

4. Ensure that the system allows core dumps. To set the core dump size to unlimited, run the following command:

```
root@rz-cmn:~# ulimit -c unlimited
root@rz-cmn:~# echo core | tee /proc/sys/kernel/core_pattern
```

5. Run the program to generate a core dump file or use remote debugging to obtain it.

```
root@rz-cmn:~# ./segfault_example
Attempting to dereference a NULL pointer...
Segmentation fault (core dumped)
```

When the segmentation fault occurs, a core dump file will be generated, usually named core or core.<pid>, for example, core.880 in my case.

```
root@rz-cmn:~# ls core*
core.880
```

Transfer the core dump file back to your host machine.

6. Using GDB to analyze the core dump file. Return to the remote machine and use the following command.

```
renesas@builder-pc:~/remote-debugging/segfault_program$ aarch64-poky-linux-gdb
</path/to/local_program> </path/to/core/dump/file>
```

For example:

```
renesas@builder-pc:~/remote-debugging/segfault_program$ aarch64-poky-linux-gdb
segfault_example core.810

GNU gdb (GDB) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux --target=aarch64-poky-linux".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from segfault...
[New LWP 810]

warning: Could not load shared library symbols for 2 libraries, e.g.
/lib64/libc.so.6.
Use the "info sharedlibrary" command to see the complete listing.
Do you need "set solib-search-path" or "set sysroot"?
Core was generated by `./segfault.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x0000aaaae3340794 in main () at segfault_example.c:8
--Type <RET> for more, q to quit, c to continue without paging--
8               *ptr = 42;
(gdb)
(gdb) quit
```

The segmentation fault occurred because the program attempted to dereference a NULL pointer at line 8 in segfault_example.c, where it tried to assign 42 to *ptr, resulting in an invalid memory access.

13.4.2 Postmortem Analysis on Visual Studio Code

In this subsection, the process of analyzing core dump files using Visual Studio Code (VSCode) is explored. It explains how to load core dumps and utilize VSCode's debugging features to automatically jump to the lines of code that caused the application to crash.

If subsection Remote Debugging on Visual Studio **Code** has been followed, the next step is to analyze the core dump file. A key addition is to include a line in the `launch.json` file that specifies the path to the core dump file for analysis. This adjustment enables full utilization of VSCode's features for inspecting the crash details.

For example, in launch.json, add the following line to specify the core dump file path:

```
"coreDumpPath": "</path/to/core/dump/file>,
```

Here’s a complete example of a launch.json

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "gdb",
            "type": "cppdbg",
            "request": "launch",
            "program": "/home/renesas/remote-debugging/program/segfault_example",
            "cwd": "${workspaceFolder}",
            "stopAtEntry": true,
            "stopAtConnect": true,
            "MIMode": "gdb",
            "miDebuggerPath":
"/home/renesas/esdk/5.1.4/tmp/sysroots/x86_64/usr/bin/aarch64-poky-linux/aarch64-
poky-linux-gdb",
            "miDebuggerServerAddress": "169.254.43.30:2000",
            "coreDumpPath": "/home/renesas/remote-debugging/segfault/core.810",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

After running the debugging session with the core dump file, the IDE (Visual Studio Code) automatically points to the exact line in the source code where the crash occurred.

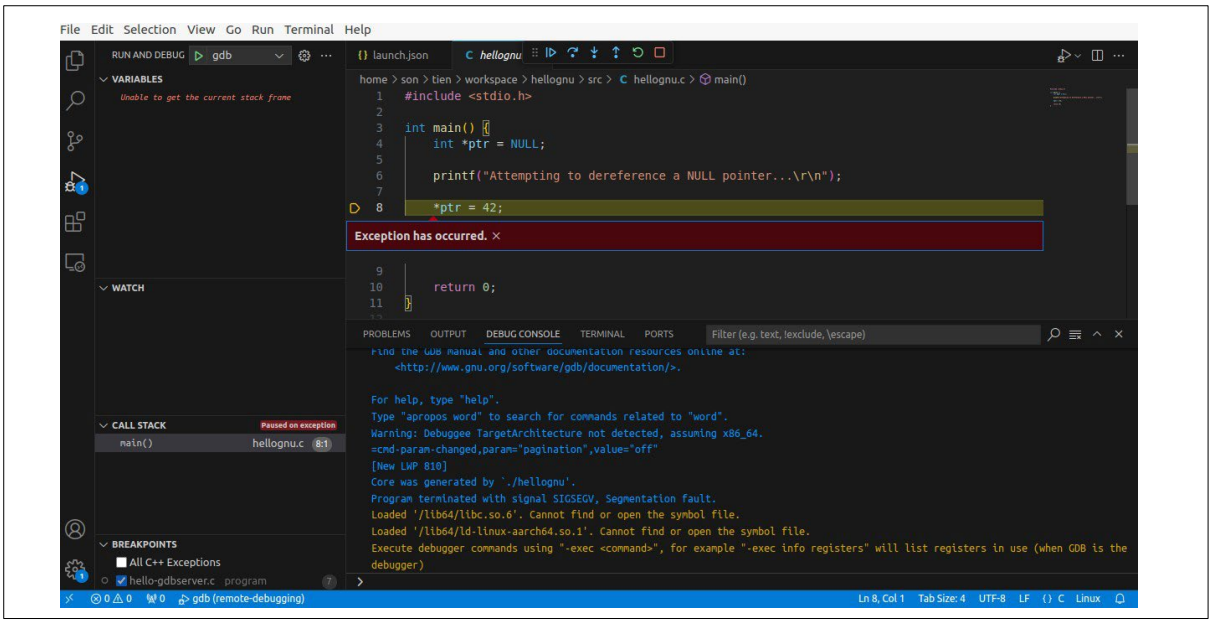


Figure 131. Starting analysis of the Core dump file in VSCode debug mode

13.4.3 Postmortem Analysis on Eclipse

This subsection describes postmortem analysis using the Eclipse IDE. Similar to Visual Studio Code, Eclipse allows loading a core dump to inspect the application's state at the time of a crash.

1. Configure Eclipse to connect to the GDB Server:

- In Eclipse, go to the Run menu and select Debug Configurations.
- Under the Debugger tab, select C/C++ Postmortem Debugger.
- In the Main tab, under the 'Core file' field, click and specify where the core dump file is.

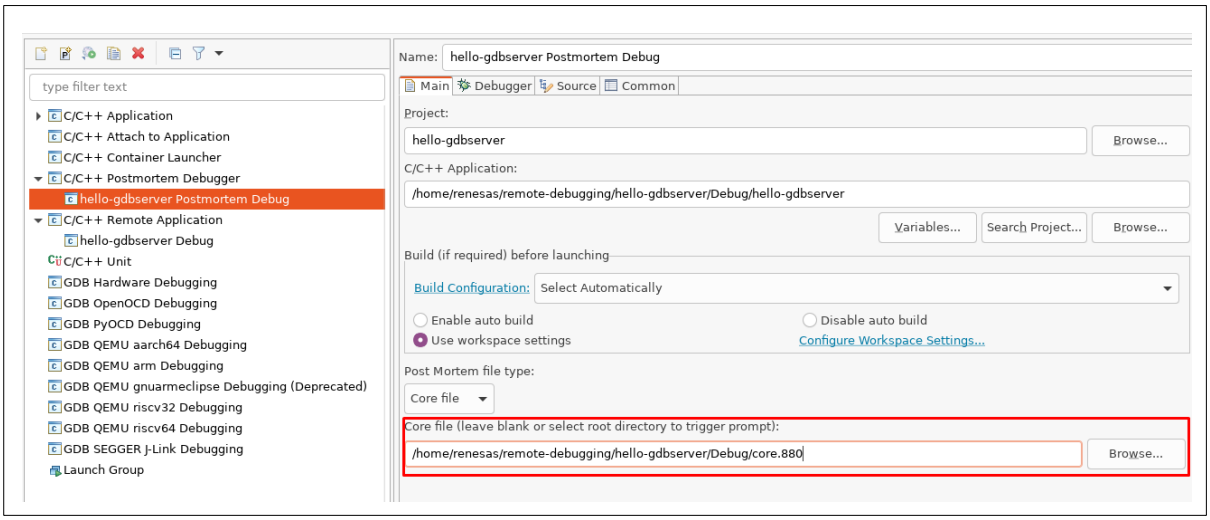


Figure 132. Specifying the Core File in Eclipse Debugger Settings

- In the Debugger tab:
  - In GDB Debugger: Provide the path to your cross-compiled GDB (For example, /home/renesas/esdk/5.1.4/tmp/sysroots/x86_64/usr/bin/aarch64-poky-linux/aarch64-poky-linux-gdb).

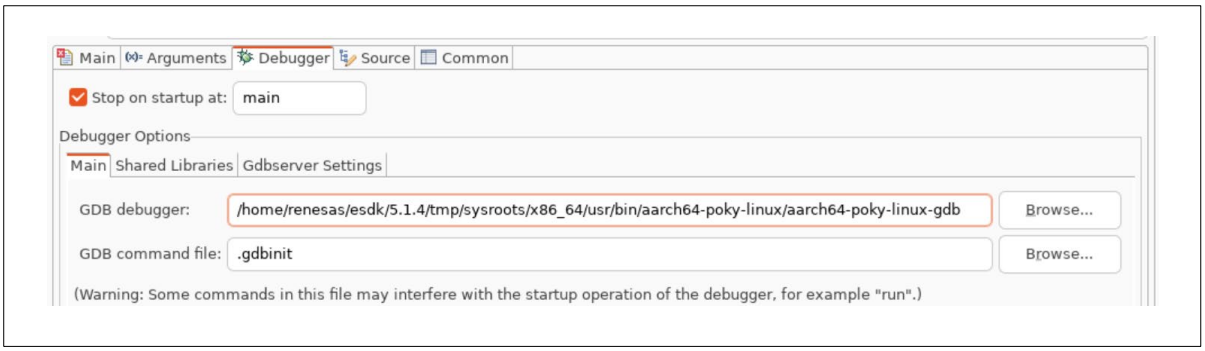


Figure 133. Specifying the GDB Debugger in Eclipse Debugger settings

2. Start the Debugging Session:

- Once the debugging session starts, Eclipse will show the line of code that caused the segmentation fault, along with the call stack.

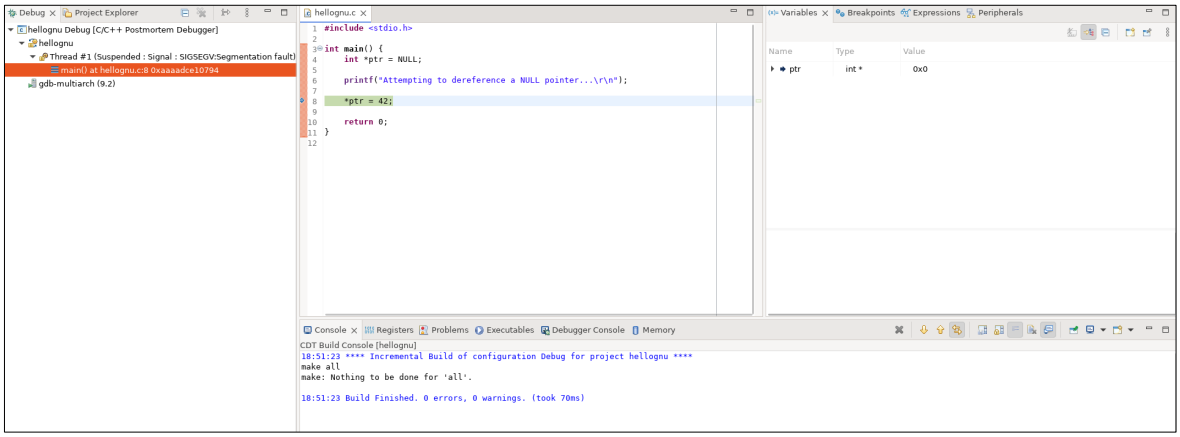


Figure 134. Starting analysis of the Core dump file in Eclipse debug mode

- Inspect the values of variables at that point in time by hovering over them or using the Variables view.
- Utilize the Expressions view to evaluate any expressions or check the state of specific variables.
- Navigate through the call stack to see the sequence of function calls leading to the crash. This can provide insight into how the program reached the faulting line.

14. Functional Overview

This chapter presents a high-level overview of the RZ/G2L, RZV2L, and RZV2H family of MPUs. It begins with a general introduction to the MPU architecture, followed by a detailed outline of the primary subsystems. Each subsection is designed to provide developers with foundational context prior to engaging in board-level integration or initiating software bring-up procedures.

14.1 MPU Architecture

This section describes the architectural foundation common across the RZ family. The MPU architecture defines the CPU clusters, interconnects, memory hierarchy, and peripheral integration that form the baseline of every device in the series.

14.1.1 RZ/G2L

The RZ/G2L MPU is a feature-packed SoC (System-on-Chip) that supports a variety of applications. Below is an overview of the MPU.

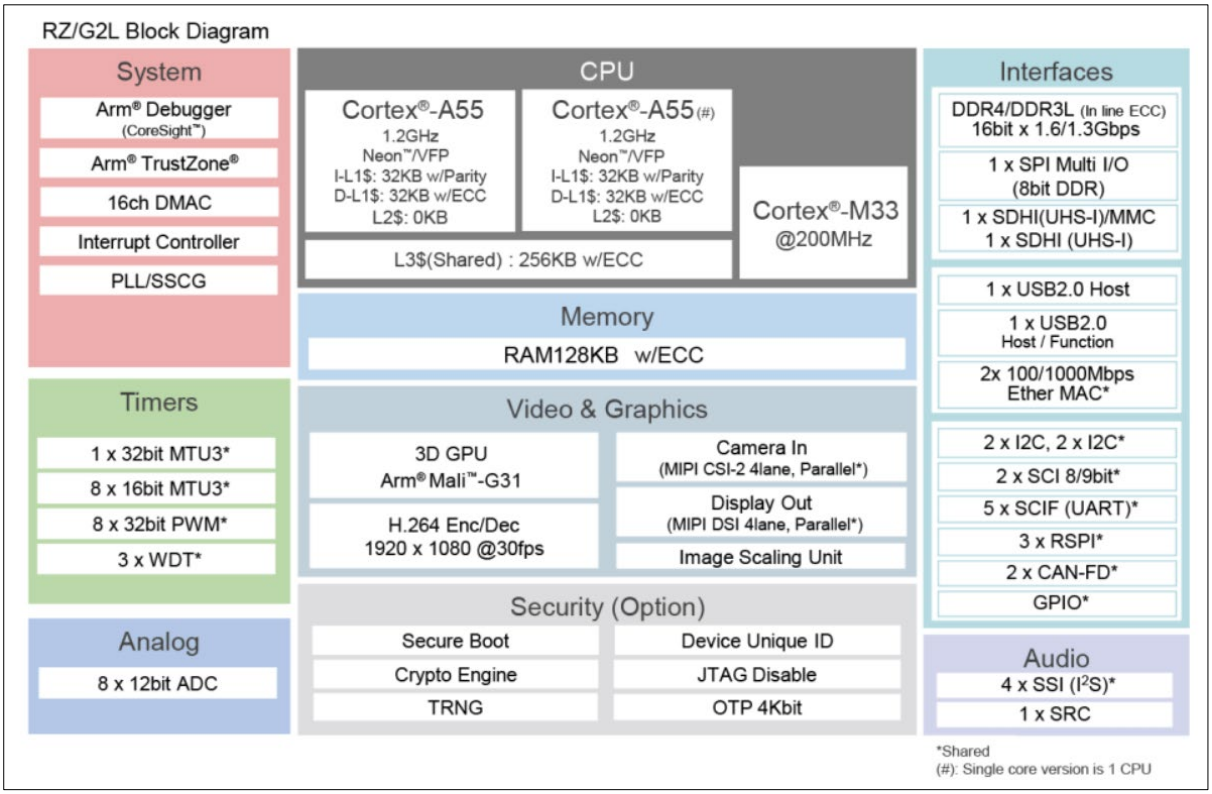


Figure 135. RZ/G2L Overview

14.1.1.1 GPIO Internals

The RZ/G2L SoC has a unique way of GPIO organization. It is not the typical banked GPIO interface that one might be used to. The RZ/G2L has individual GPIO LSI logic directly attached to the register outputs. This creates a notation for GPIO pins attached to ports, which are basically bits in a register.

Px_y :

P= port a.k.a 8-bit register set number.

x= port number

y= port bit

Each bit in a port control register corresponds to a single GPIO logic module. While each port has 8 bits, most of the ports (registers) are only using the lower two to three bits for GPIO line outs. The upper bits are used for other special functions at times. Table 58. GPIO-supported pins in RZ/G2L map all the available ports to bits.

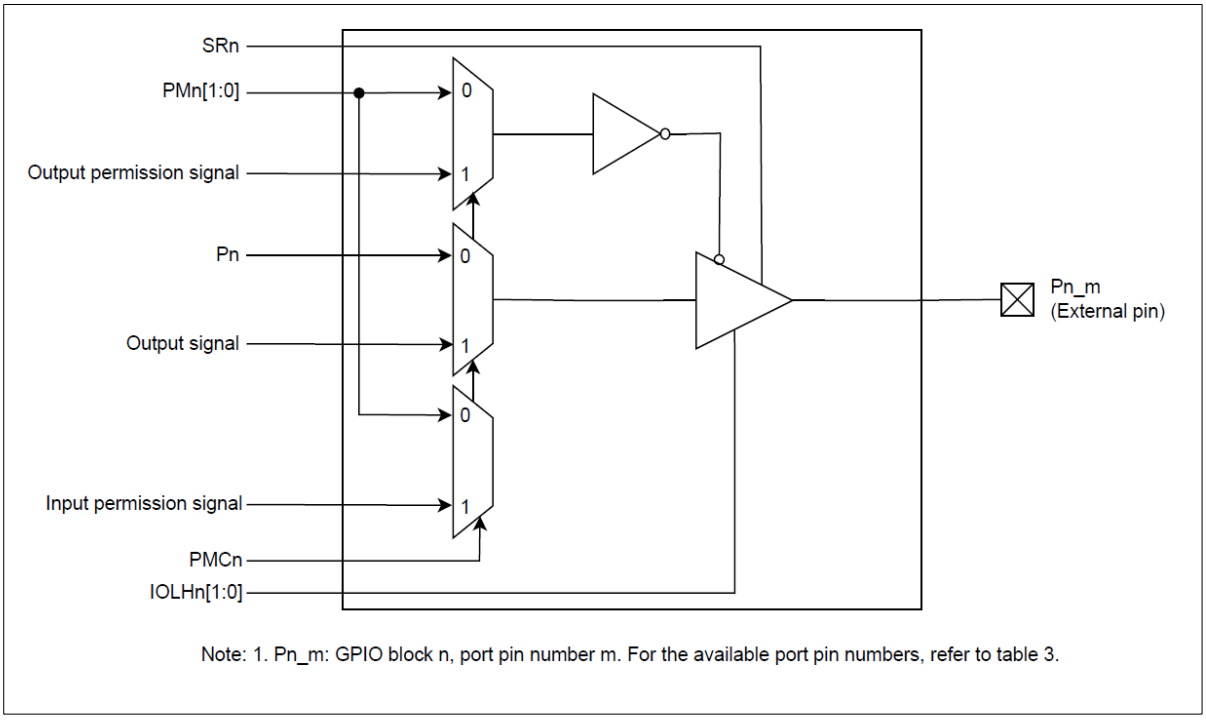


Figure 136. Multiplexed peripheral functions configuration diagram for GPIO pins

RZ/G2L can support up to 123 general-purpose I/O pins from 49 ports in Table 16. GPIO-supported pins in RZ/G2L:

Table 58. GPIO-supported pins in RZ/G2L

Port name	External Terminal Name					
	Bit7-5	Bit4	Bit3	Bit2	Bit1	Bit0
PORT 10	-	-	-	-	P0_1	P0_0
PORT 11	-	-	-	-	P1_1	P1_0
PORT 12	-	-	-	-	P2_1	P2_0
PORT 13	-	-	-	-	P3_1	P3_0
PORT 14	-	-	-	-	P4_1	P4_0
PORT 15	-	-	-	P5_2	P5_1	P5_0
PORT 16	-	-	-	-	P6_1	P6_0
PORT 17	-	-	-	P7_2	P7_1	P7_0
PORT 18	-	-	-	P8_2	P8_1	P8_0
PORT 19	-	-	-	-	P9_1	P9_0
PORT 1A	-	-	-	-	P10_1	P10_0
PORT 1B	-	-	-	-	P11_1	P11_0
PORT 1C	-	-	-	-	P12_1	P12_0
PORT 1D	-	-	-	P13_2	P13_1	P13_0
PORT 1E	-	-	-	-	P14_1	P14_0
PORT 1F	-	-	-	-	P15_1	P15_0
PORT 20	-	-	-	-	P16_1	P16_0
PORT 21	-	-	-	P17_2	P17_1	P17_0
PORT 22	-	-	-	-	P18_1	P18_0
PORT 23	-	-	-	-	P19_1	P19_0
PORT 24	-	-	-	P20_2	P20_1	P20_0
PORT 25	-	-	-	-	P21_1	P21_0
PORT 26	-	-	-	-	P22_1	P22_0
PORT 27	-	-	-	-	P23_1	P23_0
PORT 28	-	-	-	-	P24_1	P24_0

PORT 29	-	-	-	-	P25_1	P25_0
PORT 2A	-	-	-	-	P26_1	P26_0
PORT 2B	-	-	-	-	P27_1	P27_0
PORT 2C	-	-	-	-	P28_1	P28_0
PORT 2D	-	-	-	-	P29_1	P29_0
PORT 2E	-	-	-	-	P30_1	P30_0
PORT 2F	-	-	-	-	P31_1	P31_0
PORT 30	-	-	-	-	P32_1	P32_0
PORT 31	-	-	-	-	P33_1	P33_0
PORT 32	-	-	-	-	P34_1	P34_0
PORT 33	-	-	-	-	P35_1	P35_0
PORT 34	-	-	-	-	P36_1	P36_0
PORT 35	-	-	-	P37_2	P37_1	P37_0
PORT 36	-	-	-	-	P38_1	P38_0
PORT 37	-	-	-	P39_2	P39_1	P39_0
PORT 38	-	-	-	P40_2	P40_1	P40_0
PORT 39	-	-	-	-	P41_1	P41_0
PORT 3A	-	P42_4	P42_3	P42_2	P42_1	P42_0
PORT 3B	-	-	P43_3	P43_2	P43_1	P43_0
PORT 3C	-	-	P44_3	P44_2	P44_1	P44_0
PORT 3D	-	-	P45_3	P45_2	P45_1	P45_0
PORT 3E	-	-	P46_3	P46_2	P46_1	P46_0
PORT 3F	-	-	P47_3	P47_2	P47_1	P47_0
PORT 40	-	P48_4	P48_3	P48_2	P48_1	P48_0

-: unused pins

Note: The RZ/G2L has only one GPIO chip interface to control all the supported pins mentioned in section **GPIO (General Purpose I/O pins) with libgpiod**

14.1.2 RZ/V2L

RZ/V2L pairs a 1.2-GHz Arm® Cortex®-A55 with Renesas' DRP-AI vision engine (a combination of DRP and AI-MAC). It supports 16-bit DDR3L/DDR4, integrates an ARM Mali-G31 GPU, and includes an H.264 video codec.

The DRP-AI block delivers high inference efficiency, typically avoiding heatsinks or fans. Alongside neural-net acceleration, it provides real-time image-processing functions, e.g., color correction and noise reduction, so many camera systems can run without an external ISP. This keeps BOM and power low across consumer, industrial, and retail/POS designs.

RZ/V2L is package and pin-compatible with RZ/G2L, enabling a straightforward upgrade path to add AI capability with minimal board changes and low migration cost.

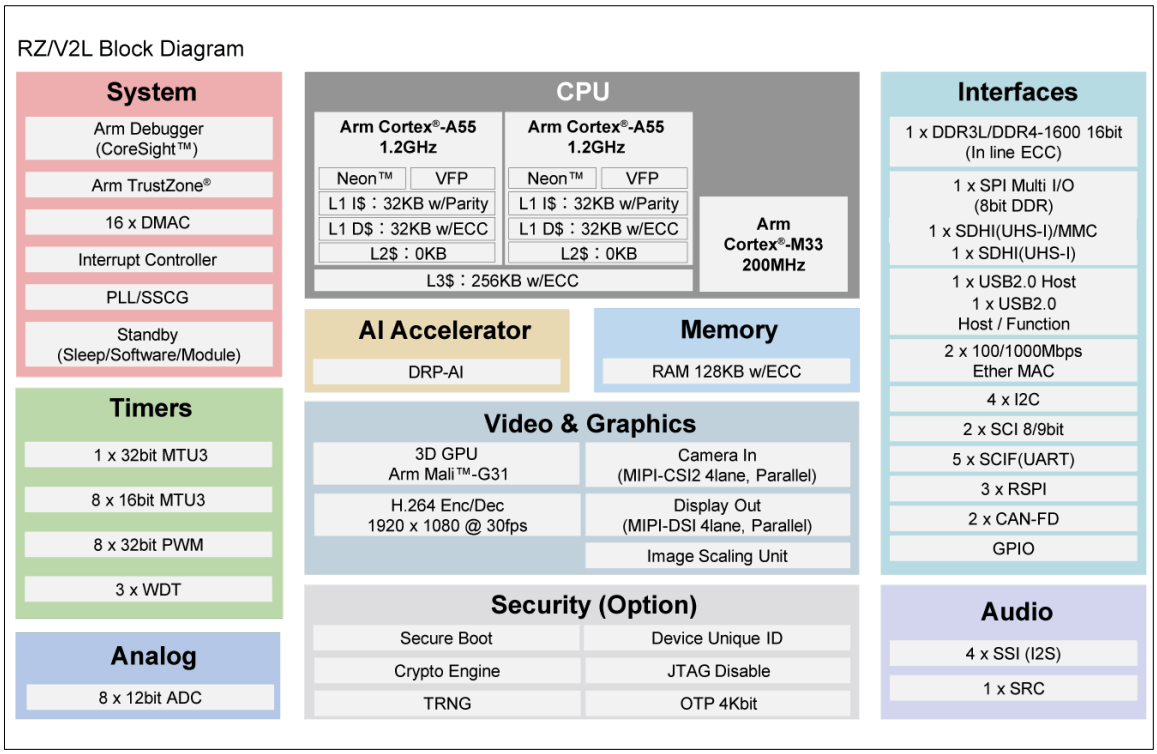


Figure 137. RZ/V2L Overview

14.1.3 RZ/V2H

The RZ/V2H is a high-performance MPU in the RZ/V series, optimized for AI inference and advanced multimedia while maintaining compatibility with the common RZ software stack. It expands on the G2L/V2L baseline with larger compute resources and dedicated accelerators.

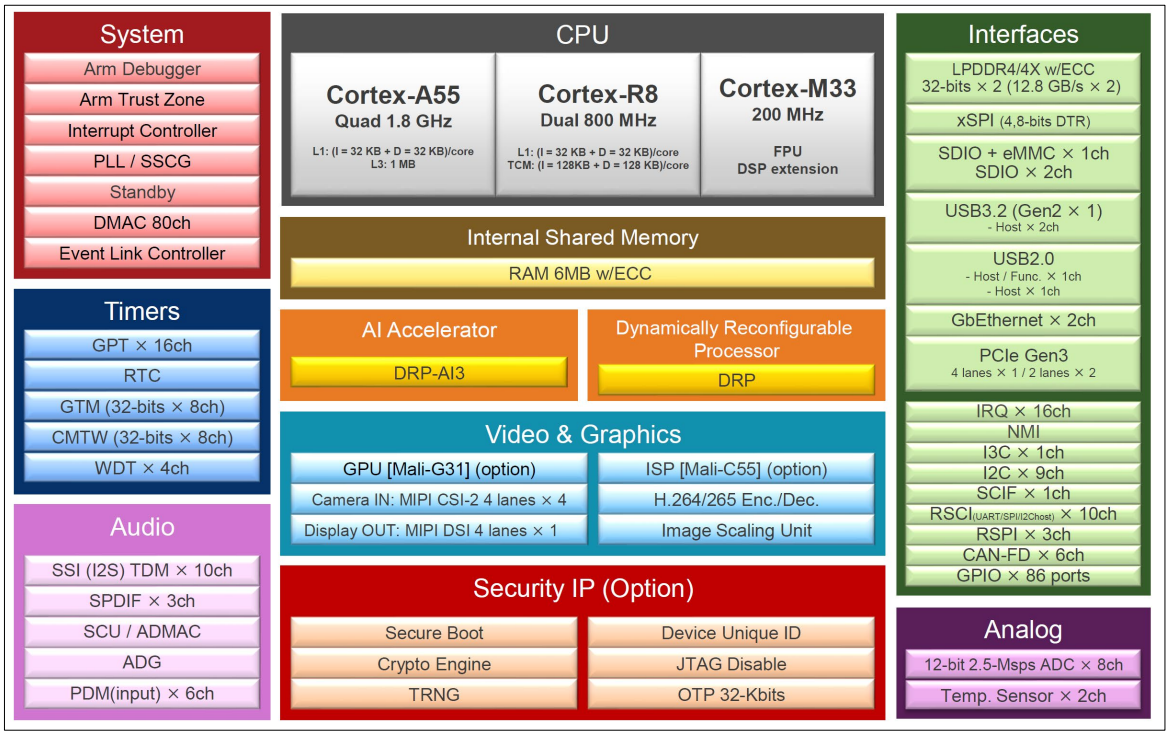


Figure 138. RZ/V2H Overview

14.2 RZ/G2L-SBC Board

This section delves into the functional and design aspects of the RZ/G2L-SBC. The image below highlights the key hardware components in the RZ/G2L SBC design.

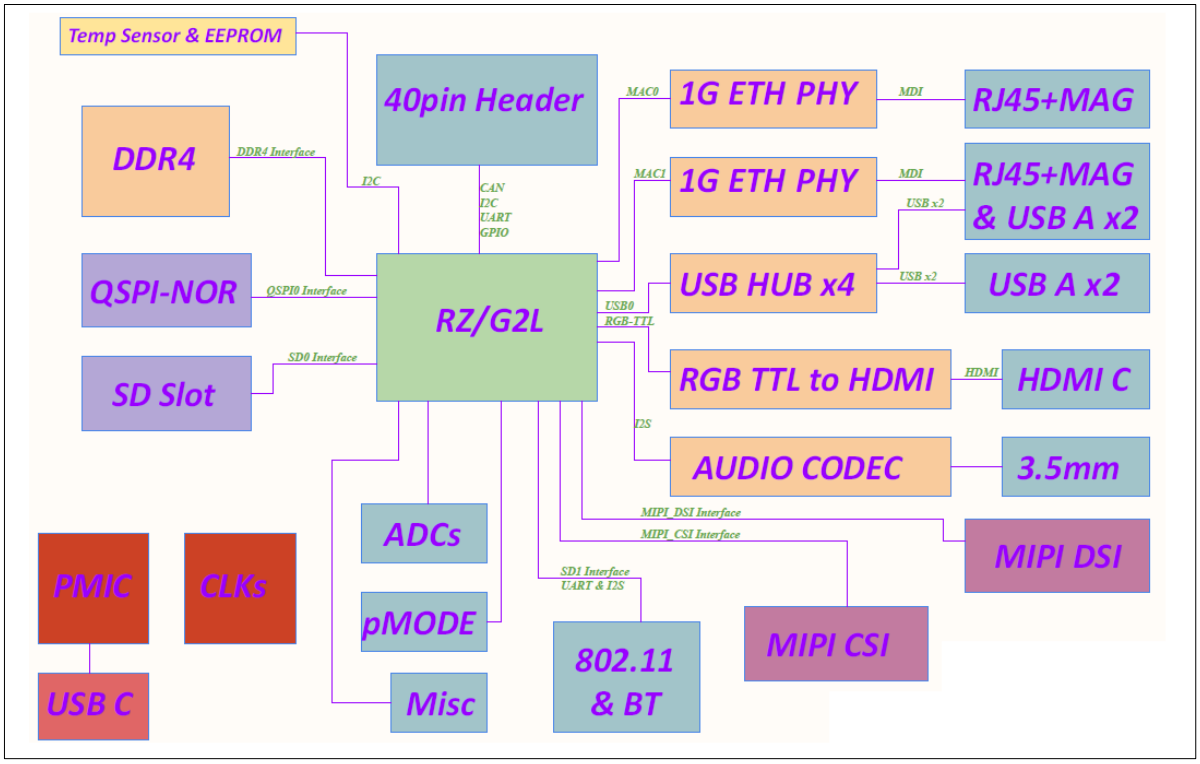


Figure 139. RZ/G2L-SBC System Overview

Table 59. Main Components on RZ/G2L-SBC

Component Number	Component Name	Type (Manufacturer)
U1	Temperature Sensor Digital, Local -55°C ~ 125°C 11 b 8-HWSON (2x3)	CAT34TS02 (Onsemi)
U2	USB Controller	UPD720115K8-611-BAK-A-ND (Renesas Electronics)
U3	MPU RZ/G2L	R9A07G044L23GBG (Renesas Electronics)
U4	DDR4 SDRAM 512MB	IS43QR16256A-093PBLI-TR (ISSI)
U5	Ethernet Phy 10BASE-TE, 100BASE-TX, 1GBASE-T	PEF7071VV16 (MaxLinear)
U6	VersaClock® Programmable Clock Generator	5P35023B-000NLGI8 (Renesas Electronics)
U7	HDMI Transmitter	SiI9022A/4A – QFN (SiliconImage)
U8	PMIC	RAA215300 (Renesas Electronics)
U9	AND GATE 2IN SOT-23-5 Vcc 1.65V to 5.5V	7UL1G08FS (Toshiba)
U10	Ethernet Phy 10BASE-TE, 100BASE-TX, 1GBASE-T	PEF7071VV16 (MaxLinear)

U11	Audio Codec with Advanced Accessory Detect	DA7219 (Renesas Electronics)
U12	Dual USB Port Power Supply Controller - Covering the Industrial Temperature Range of -40 °C to +85 °C	ISL61852FIRZ (Renesas Electronics)
U13	QSPI Flash 512MBIT SPI/QUAD 8WSON	S25FS512SDSNFB010 (Infineon)
U14	Dual USB Port Power Supply Controller - Covering the Industrial Temperature Range of -40 °C to +85 °C	ISL61852FIRZ (Renesas Electronics)
M1	Integrated 802.11 b/g/n Wi-Fi Module	iWi-L-WB (Laird)
Y1	Crystal resonator for XIN	XRCGB24M000F0L00R0 (Murata)
Y2	Crystal resonator for XIN	ST3215SB32768H5HPWAA (Kyocera-AVW)

Table 60. Primary connectors on RZ/G2L-SBC

Components Number	Component Name	Type (Manufacturer)
J1	USB 2 & 3	USB-A-D-RA (Adam Tech)
J2	PMOD	PPPC062LFBN-RC (Sullins)
J3	40-Pin Header (Raspberry Pi 3B compliant)	-
J4	USB 0 &1, 10/100/1000 Ethernet 2	YKGU-6101NL (Ingke)
J5	MIPI-CSI	1-1734248-5 (TE Connectivity)
J6	MIPI-DSI	1-1734248-5 (TE Connectivity)
J7	10/100/1000 Ethernet 1	YKGD-8069NL (Ingke)
J8	Audio I/O (Speaker/Microphone)	ASJ-192-Y (Adam Tech)
J9	Mini-HDMI	10029449-001RLF (FCI)
J10	USB-Type-C Power Input	C-ARA1-AK515 (CNC Tech)
J11	20-pin JTAG connector	3221-10-0300-00 (CNC Tech)
J12	Expansion Connector to Display adapter & boot strapping pins	528850274 (Molex)
J13	Expansion Connector to Display adapter & boot strapping pins	528850274 (Molex)
P1	microSD card slot	MEM2051-00-195-00-A (GCT)

14.2.1 Functional Overview

The RZ/G2L-SBC is a power-efficient, graphics-enabled development board in a popular single-board computer format with well-supported expansion interfaces. This Renesas RZ/G2L processor-based platform is ideal for developing cost-efficient HMI, industrial, robotics, and a range of energy-efficient design applications. The RZ/G2L processor has two 1.2GHz Arm® Cortex®-A55 cores, a 200MHz Cortex-M33 core, a MALI 3D GPU, and an Image Scaling Unit. This processor SoC is equipped with an on-chip plus H.264 video (1920 x 1080) encode/decode function in silicon, making it ideal for implementing cost-effective embedded vision and display applications.

The RZ/G2L-SBC is engineered in a compact Raspberry Pi form factor with a versatile set of expansion interfaces, including Gigabit Ethernet, 801.11ac Wi-Fi, four USB 2.0 host ports, a MIPI DSI display with touch and CSI camera interfaces, a CANFD interface, a PMOD interface, a Pi-HAT-compatible 40-pin expansion header, and two expansion sockets for a daughter card.

The board supports analog audio applications via an audio codec and a stereo headphone jack. It also pins out five 12-bit ADC inputs for interfacing with analog sensors through an expansion module (not included). A 5V input power is sourced via a USB-C connector and managed via a single-chip Renesas RAA215300 PMIC device.

The onboard memory includes 1GB DDR4, 64 MiB QSPI NOR flash memory, and a microSD slot for removable boot media.

Software enablement includes a Linux 6.18 kernel-based BSP, along with reference designs showcasing demo implementations of HMI applications. Onboard 10-pin JTAG/SWD mini-SMT header (unpopulated) and 40-pin GPIO header enable the use of an external debugger and USB-serial cable

14.2.2 Physical View

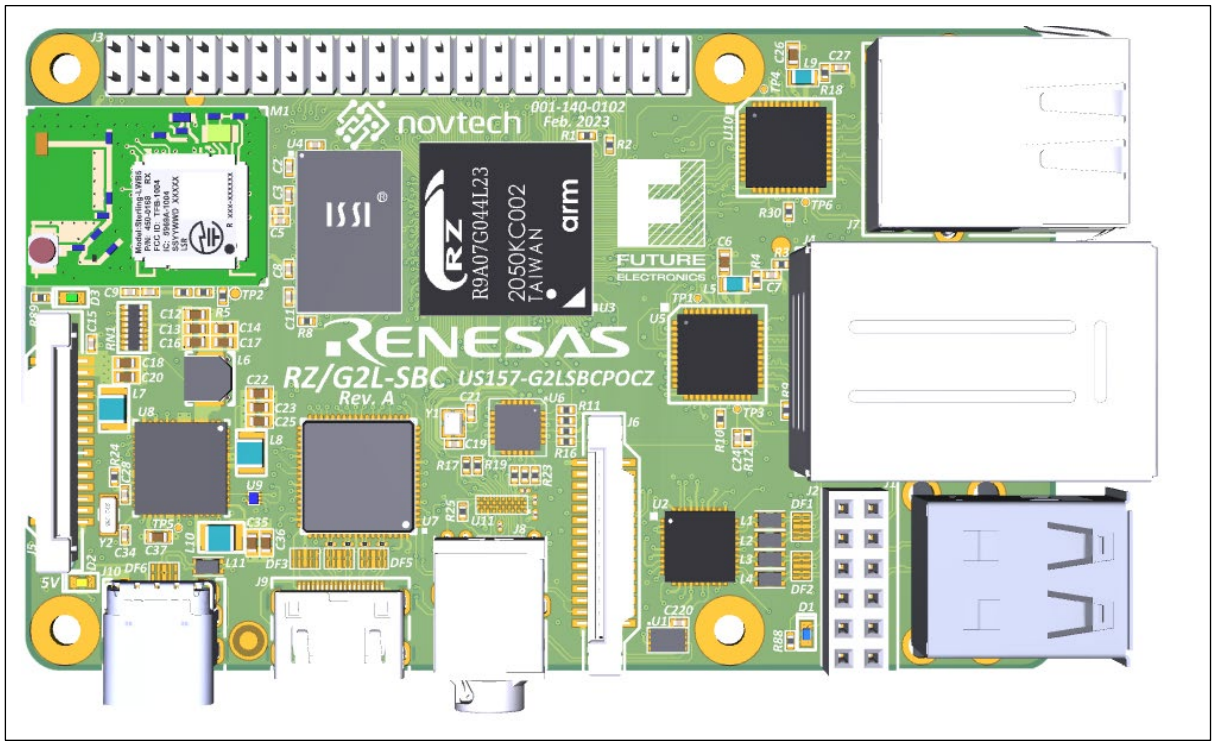


Figure 140. Top-side view of the RZ/G2L-SBC

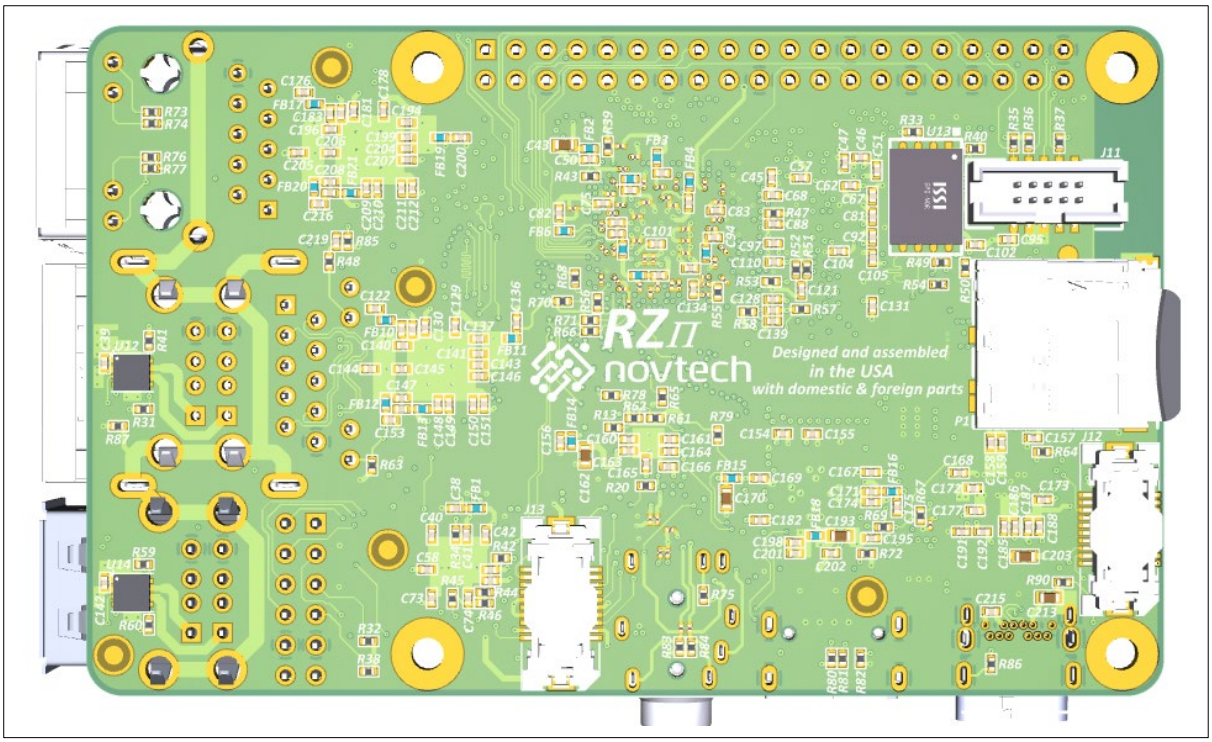


Figure 141. Bottom side of the RZ/G2L-SBC

14.2.3 Overview of Connectors

Given below is the basic positioning of the top-level connectors.

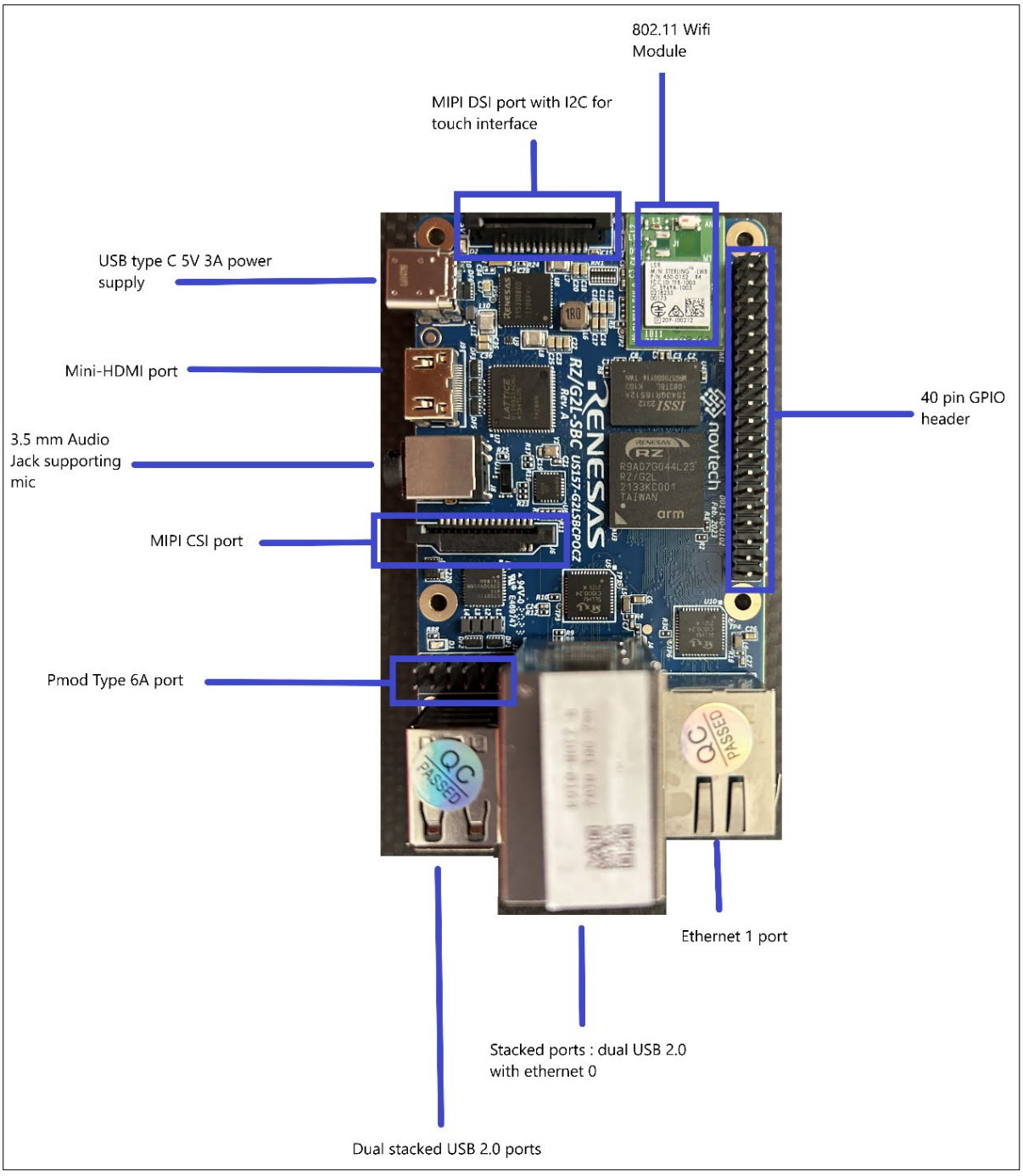


Figure 142. RZ/G2L-SBC top side connectors.

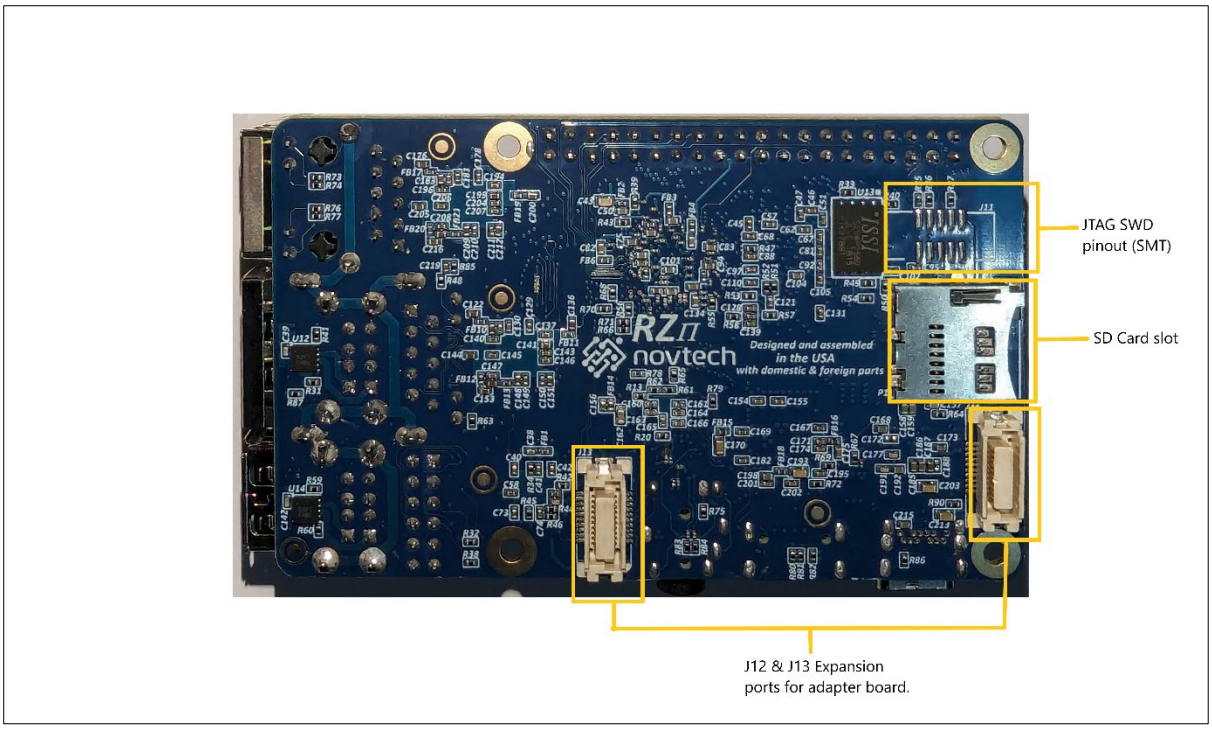


Figure 143. RZ/G2L-SBC Bottom view connectors.

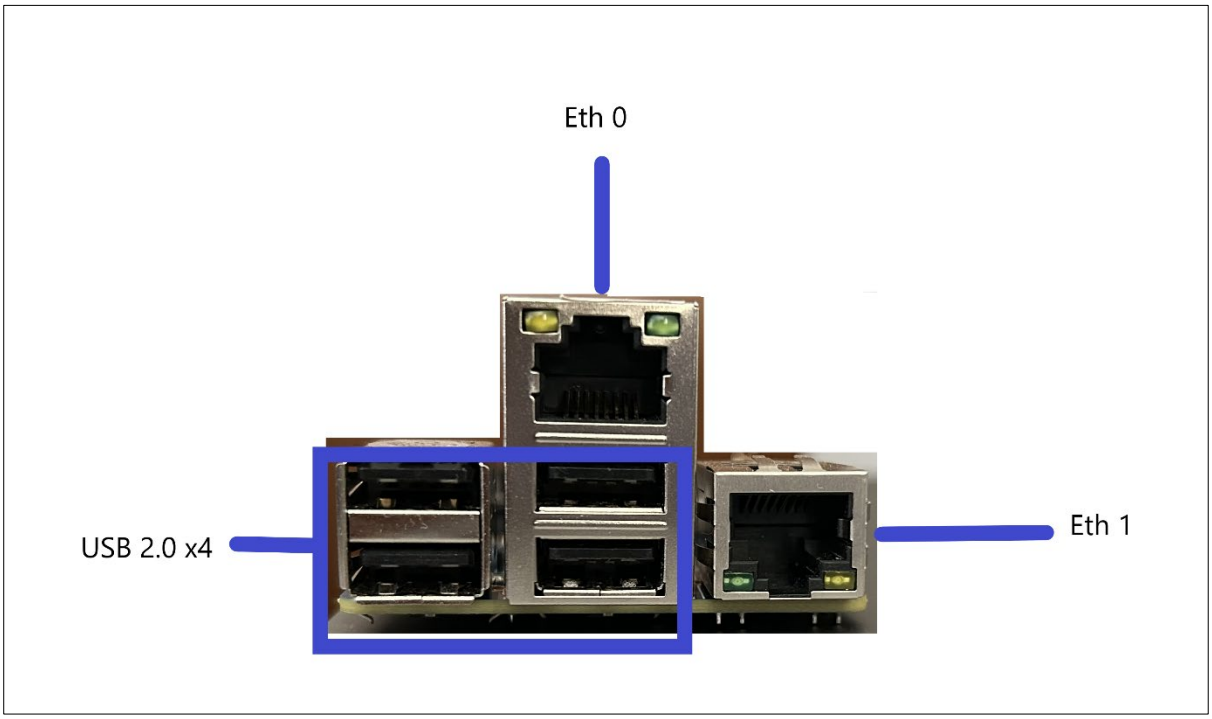


Figure 144. RZ/G2L-SBC side view I/O ports.

14.2.4 Power Supply

This section delves into the RZ/G2L-SBC's power supply architecture. The RZ/G2L-SBC uses a simple design with a 5V supply as the single external power source.

14.2.4.1 USB Type-C Power

This board has one USB Type-C receptacle for power input with USB Power Delivery. The USB Type–C power connector is meant to connect to a 5V power supply. The RZ/G2L-SBC requires a minimum of 3A power to prevent brownouts. However, we recommend a 4.5 -5A power supply as several ports support peripherals that consume substantial power.

14.2.4.2 Power Rails

Given below is the basic power supply design. It is a simple design that uses an input power supply from USB-C or one of the routed pins marked as 5V in the 40-pin GPIO or the adapter board and routes it through a series of converters to generate different power lines.

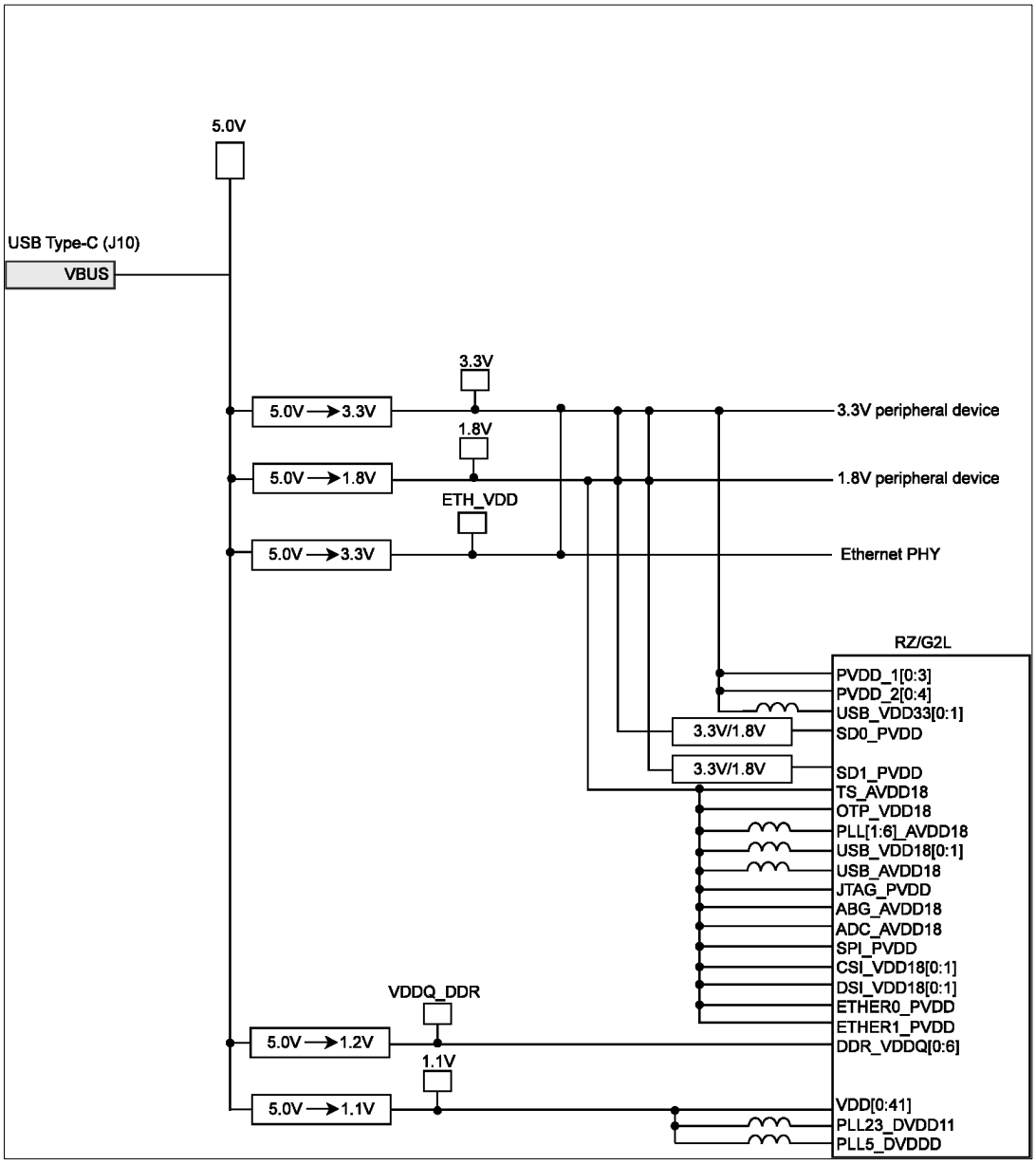


Figure 145. Power supply rails.

The Input power of 5V is used to generate five independent power lines:

- Two independent 3.3 V lines for peripherals and Ethernet.
- A 1.8V master supply line
- A 1.2V master supply line
- A 1.1V master supply line

The 1.2V line is used by the RZ/G2L SoC and the DDR4 SDRAM, while the 1.1V line is exclusively used by the RZ/G2L SoC. The RZ/G2L also draws power to its internal IP blocks from the 1.8V line.

This design is aimed at simplicity and hence omits the use of any power and reset switches. POR behavior is strictly controlled by the PMIC and its passives.

14.2.4.3 Power Supply Regulation

The power supply is regulated by Renesas RAA215300, a low-cost nine-channel PMIC IC.

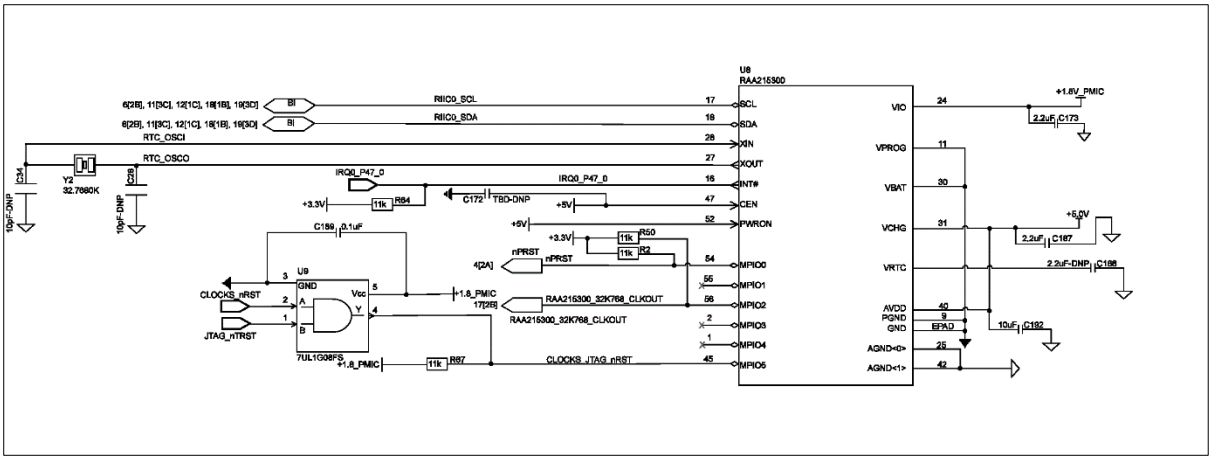


Figure 146. Block Diagram of Power Supply Regulation using RAA215300.

14.2.5 Power Management Integrated Circuit- PMIC

All LDOs are cycled as per the POR cycle. Any control is exercised by the RZ/G2L through the I²C interface. However, the LDOs are always turned on post-POR.

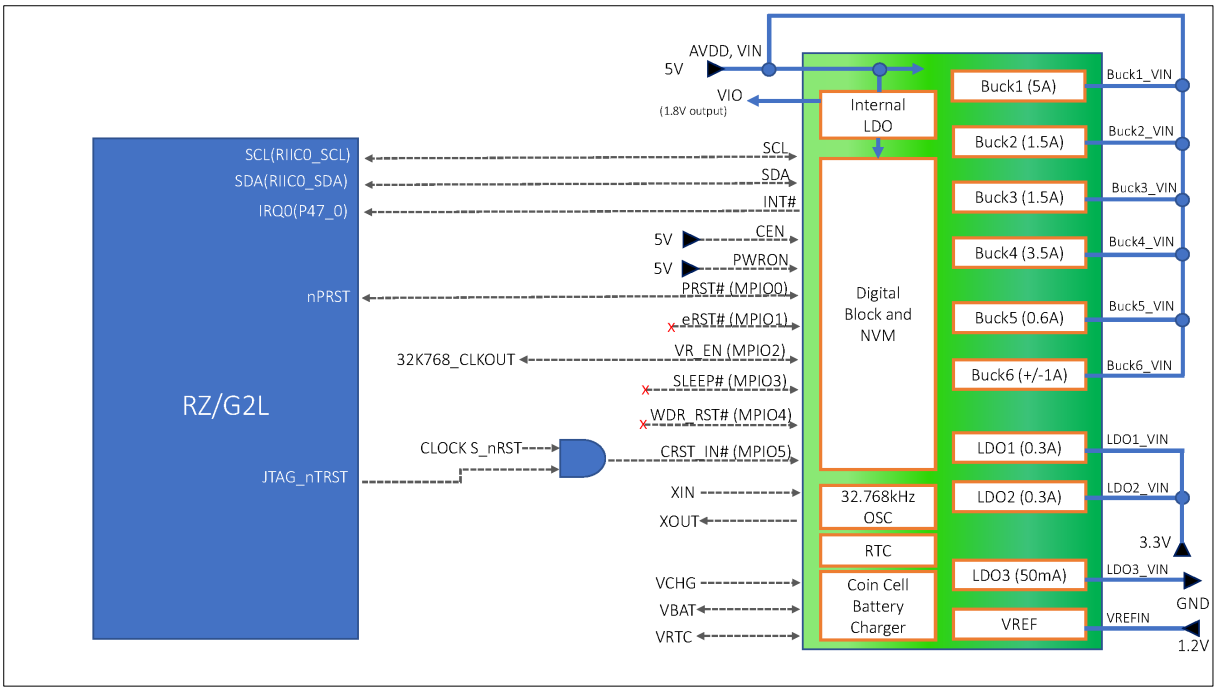


Figure 147. Block diagram of PMIC interface to RZ/G2L

14.2.6 RESET Control

The RZ/G2L-SBC has simplified POR behavior. It is by default set up to boot from QSPI0, which is achieved through external pull-up and pull-down resistors to a default code of 011. The default boot mode of 011 is for booting from QSPI0, but setting the operating voltage to 1.8V. The bootstrapping lines can be accessed externally through the J12 port at the bottom (through an adapter board). This makes it possible to alter the boot flow using these pins.

In addition to the boot order, the SoC has two more lines: DEBUGEN (BE) & BSCANP (BS). These lines control the boot mode, which can be JTAG boundary scan or debug mode. Figure **141. Reset Control Logic**

Note**:** Debug mode and boundary scan are mutually exclusive. Only one of DEBUGEN / BSCANP can be active at a time. Do not enable both.

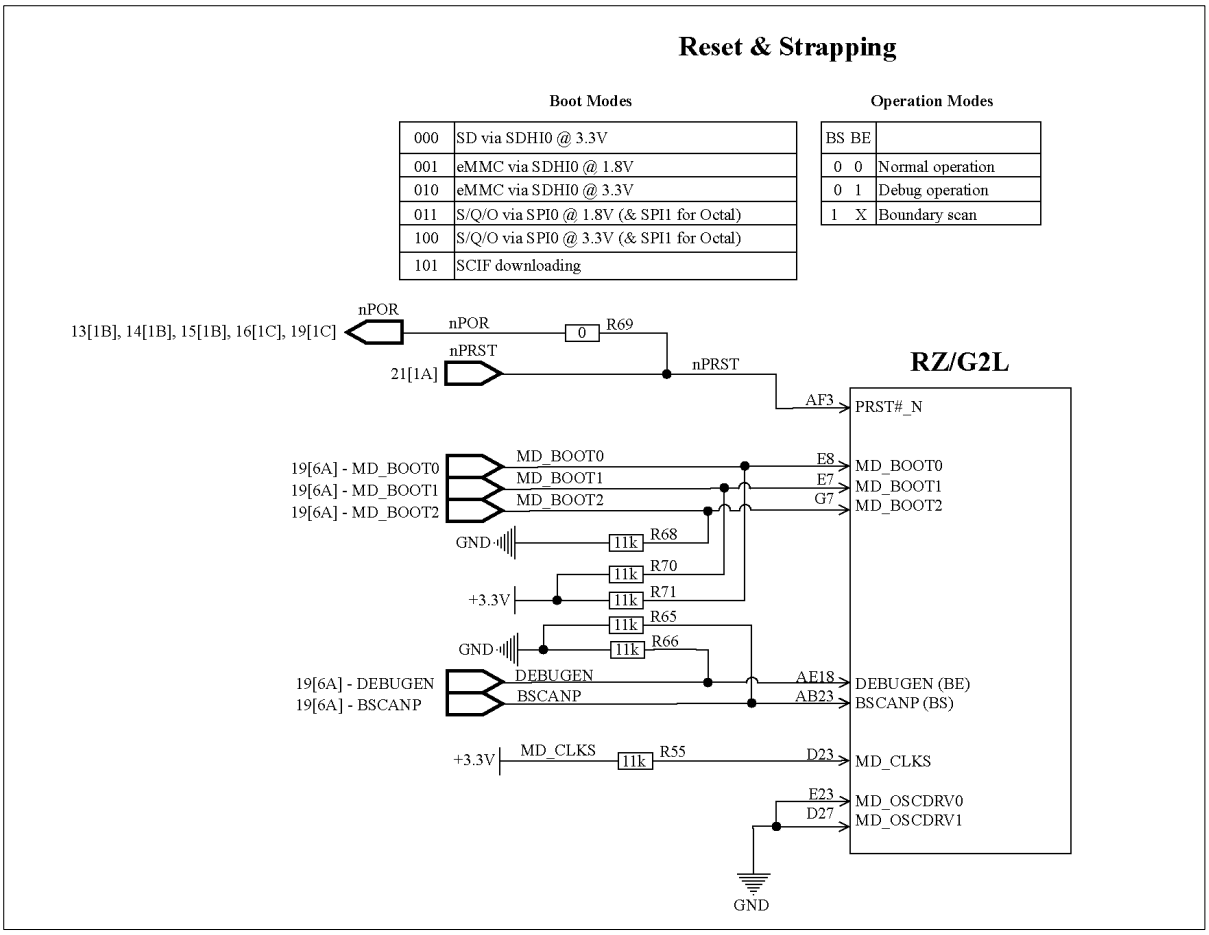


Figure 148. Reset Control Logic

14.2.7 Clock Configuration

The RZ/G2L-SBC design uses a Renesas VersaClock-3S as a singular programmable clock generator as the master clock source for the entire board. It drives the source clock for not just the RZ/G2L-SoC but all other devices that use an external clock input. This reduces the design complexity by reducing the use of passives and PLLs per peripheral while using a single 24 MHz crystal XTALL.

Note:

1. MIPI DSI interface supports operations up to FHD@60 fps rates.
2. SD Interface supports UHS-I mode of 50MBps (SDR50) and 104MBps (SDR104)

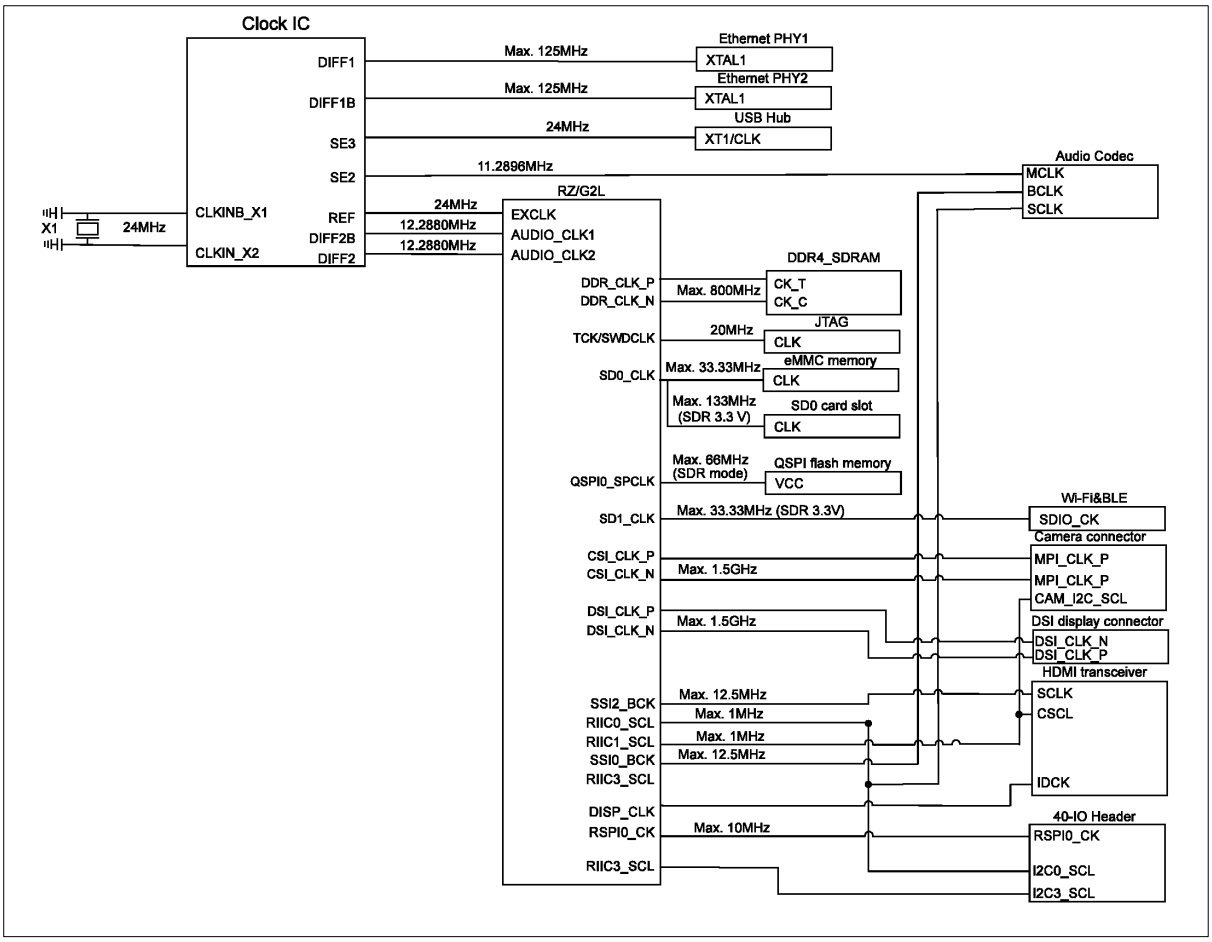


Figure 149. Block diagram of Clock interfacing.

14.2.8 Peripheral Interface

14.2.8.1 Gigabit Ethernet

The RZ/G2L-SBC comes with two Gigabit Ethernet ports. They are identified as Eth 0 and Eth 1 in the Linux environment. They are both gigabit-capable. The Gigabit Ethernet Interface is controlled by the Ethernet controller (E-MAC) that conforms to the definition of the MAC (Media Access Control) layer that is built into the RZ/G2L. The Ethernet clock is sourced from a clock generator connected to the Ethernet PHY.

This interface complies with IEEE802.3 PHY RGMII.

ETH0 is connected to PHY 2, and ETH1 is connected to PHY1. Take note of the order.

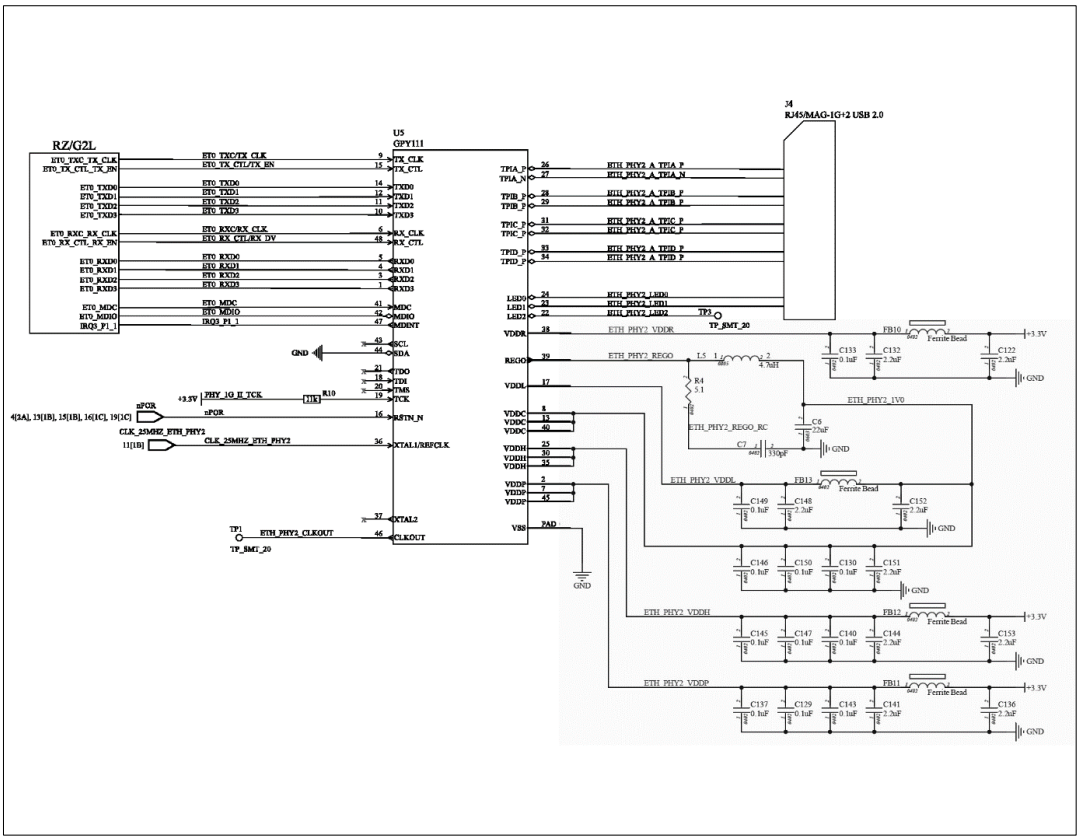


Figure 150. Ethernet 0 PHY interfacing.

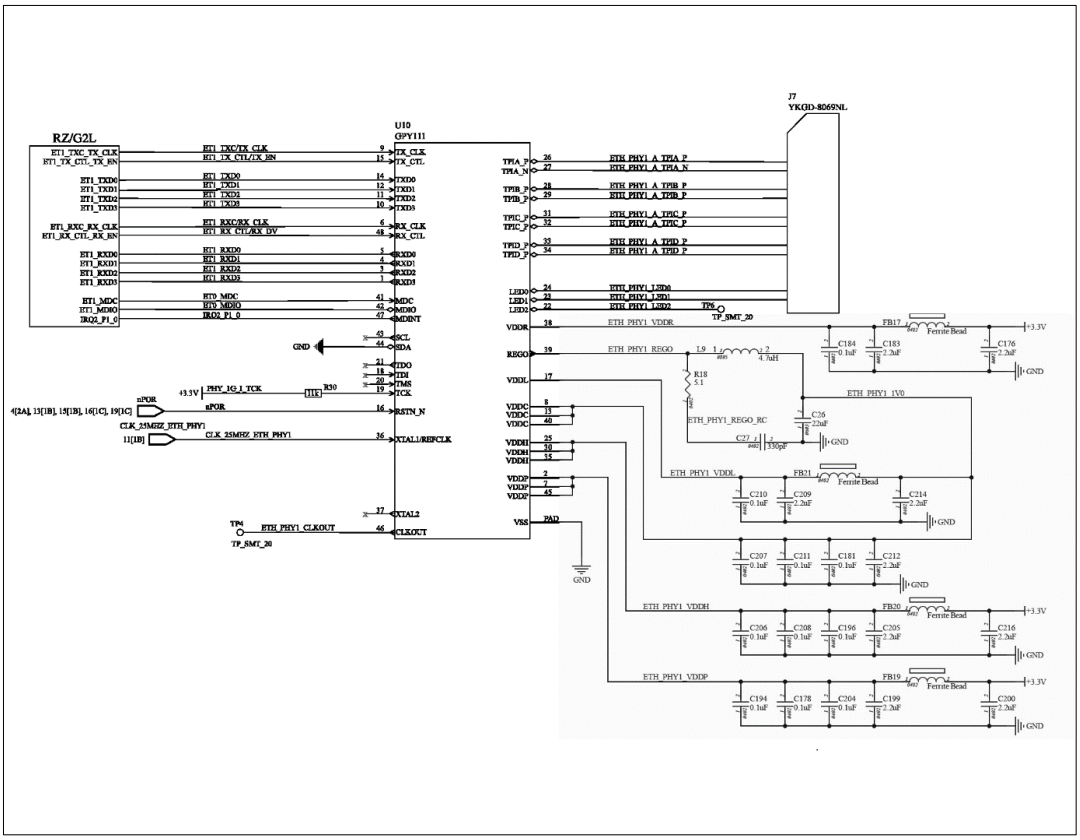


Figure 151. Ethernet 1 PHY interfacing.

14.2.8.2 USB 2.0 Ports

The SBC has 4 USB 2.0 ports, which are of type A. The primary USB hub is the Renesas UPD720115 (µPD720115), which is a 4-port hub conforming to the USB battery charging specification version 1.2. It has one upstream port and four downstream ports. The USB hub is connected directly to the RZ/G2L SoC's USB 1 data ports. The RZ/G2L SoC has a single USB 2.0 Host Interface channel.

The USB 0 channel (OTG interface) is routed to the USB-C power supply port. However, the actual OTG lines are not connected, and only the data lines are routed to the USB-C port. When the board is powered through the 40-pin IO or the bottom expansion connectors, it frees up the USB-C port. It can then be used for connecting peripherals.

Note: The USB-C has not been tested as a peripheral interface so far.

The power supply to the four USB 2.0 ports downstream is controlled through two external power regulators: Renesas ISL6185. The ISL6185 isolates and protects the internal circuit from the external USB peripheral while providing higher levels of 5V power through supply sourcing.

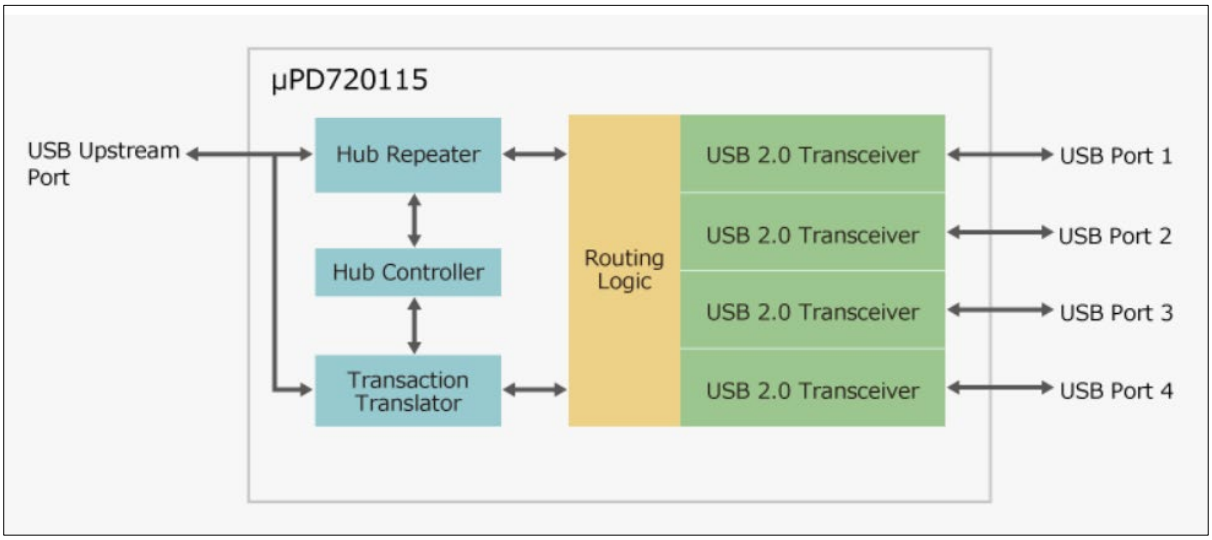


Figure 152. UPD720115 block diagram.

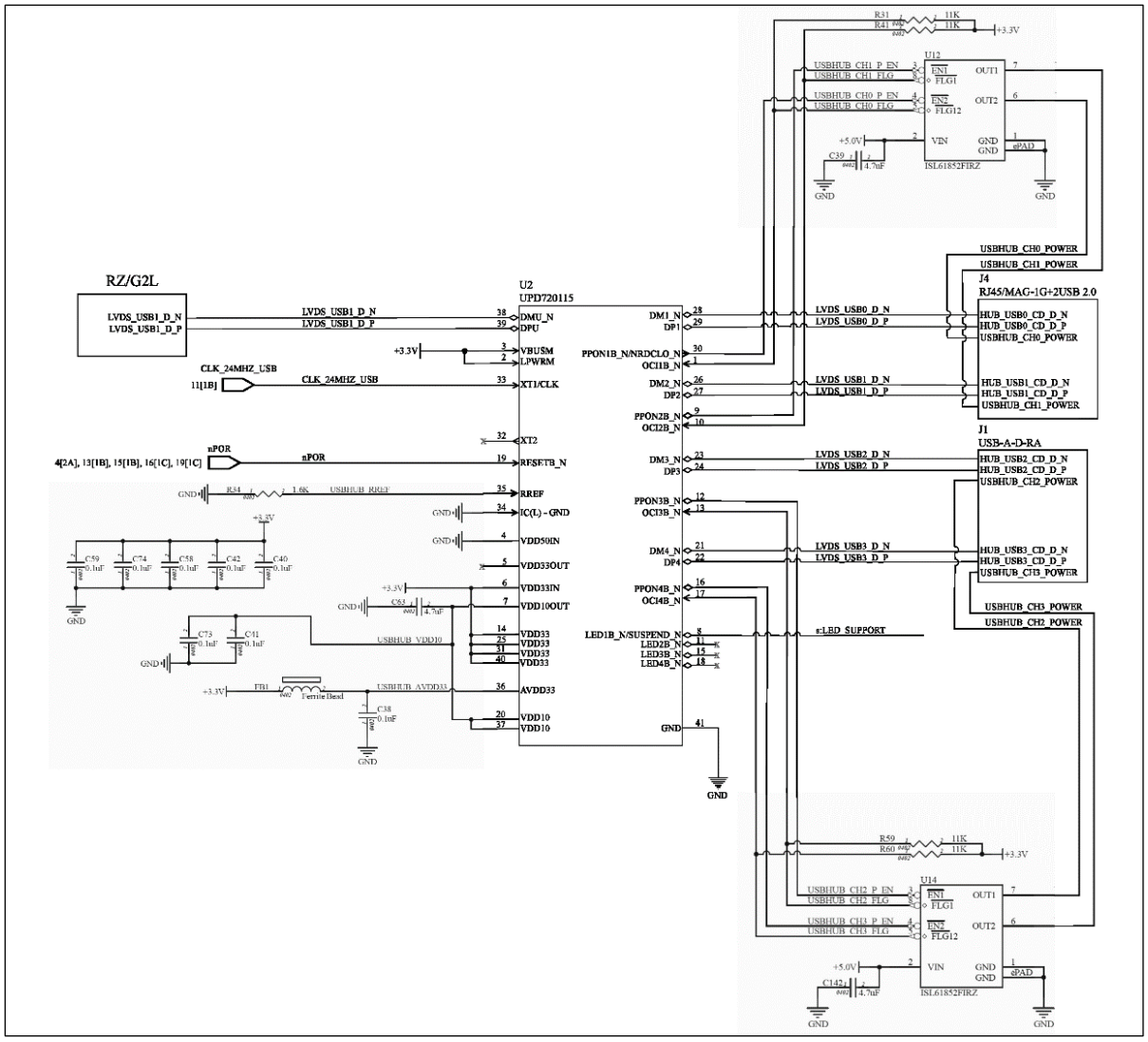


Figure 153. USB 2.0 Hub Block Diagram

14.2.8.3 MIPI CSI Interface

The RZ/G2L-SBC comes with a dual-channel MIPI CSI port labeled as J6. It is located right next to the 3.5 mm audio jack. The CSI port 15-pin camera port is verified to work with the OV5640 camera module. It supports two data channels and one I^2C channel. It is directly interfaced to the RZ/G2L SoC.

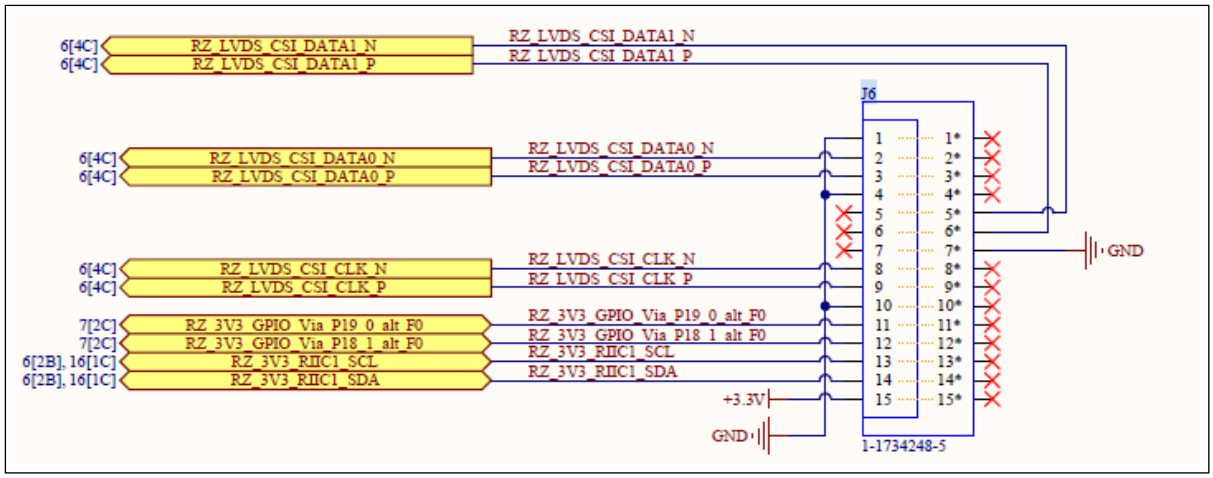


Figure 154. CSI Interface Schematic

14.2.8.4 MIPI DSI Interface

The RZ/G2L-SBC comes with a dual-channel MIPI DSI port labeled as J5. It is located toward the edge of the board next to the Wi-Fi chipset. The 15-pin display port is verified to work with a Waveshare 5” DSI display with a capacitive touch interface module. It supports two data channels and one I²C channel. It is directly interfaced to the RZ/G2L SoC.

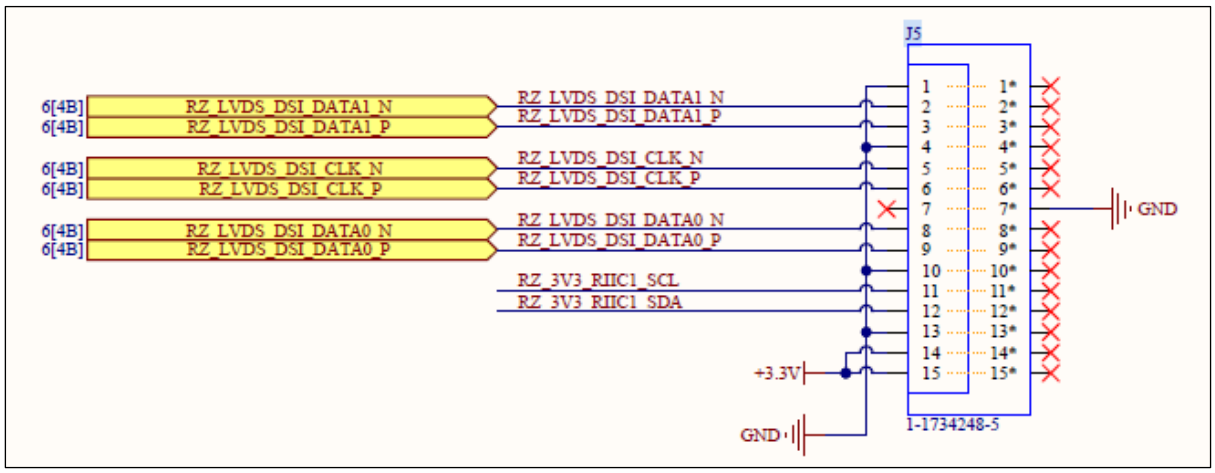


Figure 155. DSI Schematic

14.2.8.5 Audio DAC with 3.5mm Jack

The RZ/G2L-SBC comes with an onboard audio DAC from Renesas: DA7219. The audio DAC is interfaced to the RZ/G2L SoC to its SSI1 and I²C 0. The SSI1 is used for audio streaming of I2S data, while the I²C interface is used for mux and peripheral control.

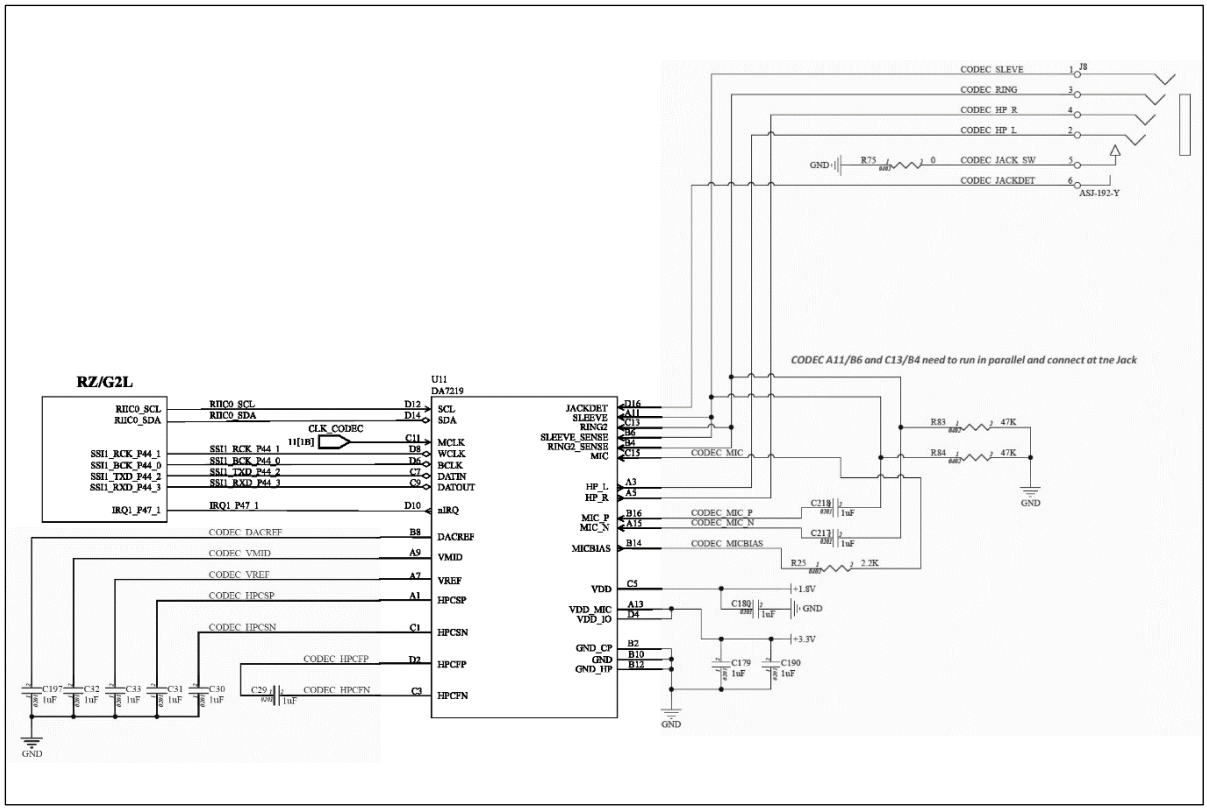


Figure 156. Audio CODEC Interface Block Diagram

14.2.8.6 HDMI Display Subsystem

The RZ/G2L-SBC comes with an HDMI display output, which is derived from the RGB parallel interface from the RZ/G2L SoC through an RGB to HDMI converter interface IC. The physical HDMI port is a mini-HDMI type (not micro). The HDMI signal source is the RGB parallel LVDS interface. An RGB to HDMI bridge IC is used to convert RGB to the HDMI protocol. The bridge is fully supported, and HDMI is enabled with the EDID feature.

Note: The LVDS interface is the source for both the external HDMI bridge and the on-chip DSI IP blocks. So, only one interface may be active at a time. Under no circumstances should both interfaces be turned on at the same time, as there is a limitation regarding the ISP unit.

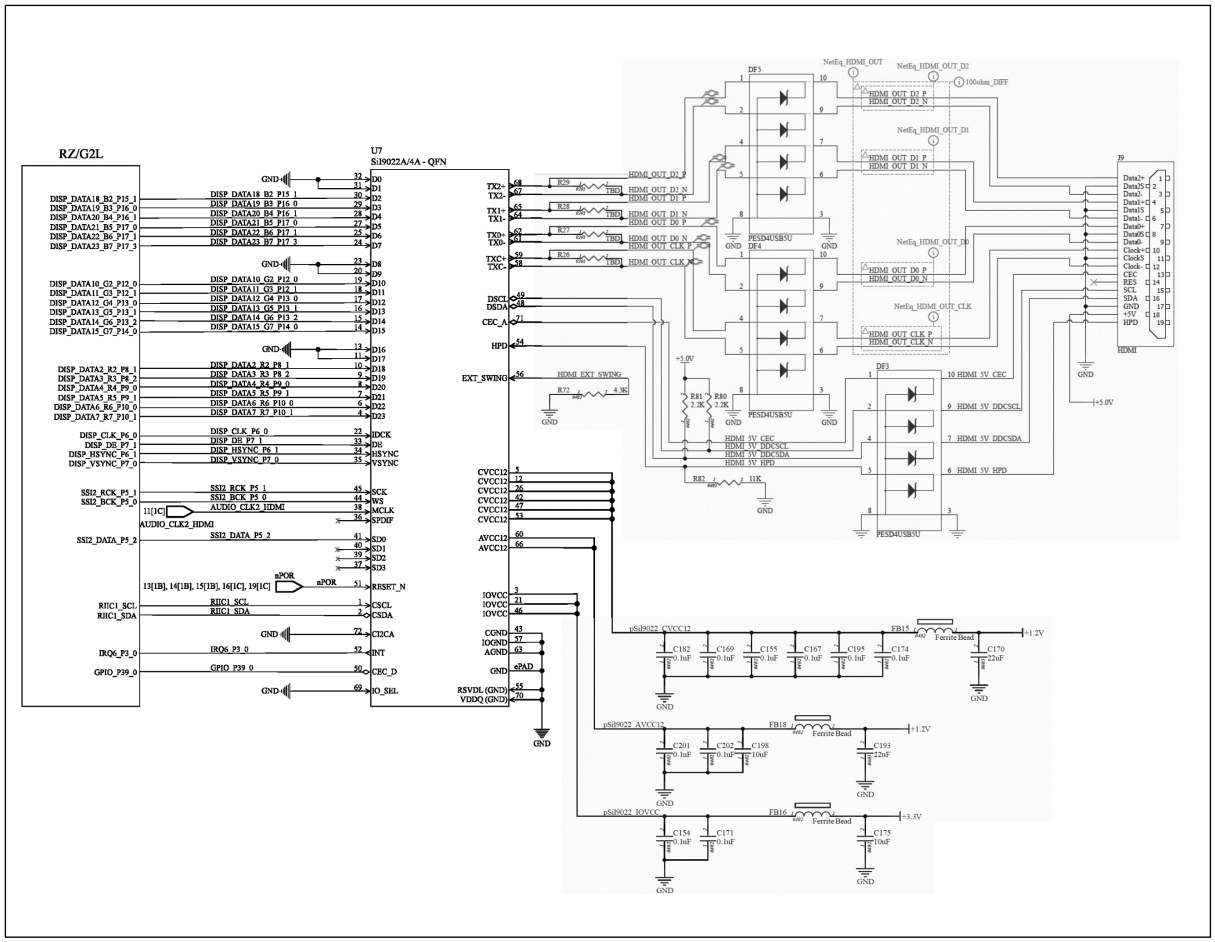


Figure 157. HDMI Bridge and mini-HDMI port interfacing.

14.2.8.7 40-pin I/O Header

The RZ/G2L-SBC comes with a 40-pin GPIO interface, which is broadly compliant with the Raspberry Pi 3 40-pin GPIO interface and provides additional interfaces like two CAN ports. The diagram below shows the pin configuration along with marking of the bottom I/O ports for reference to the orientation of the board

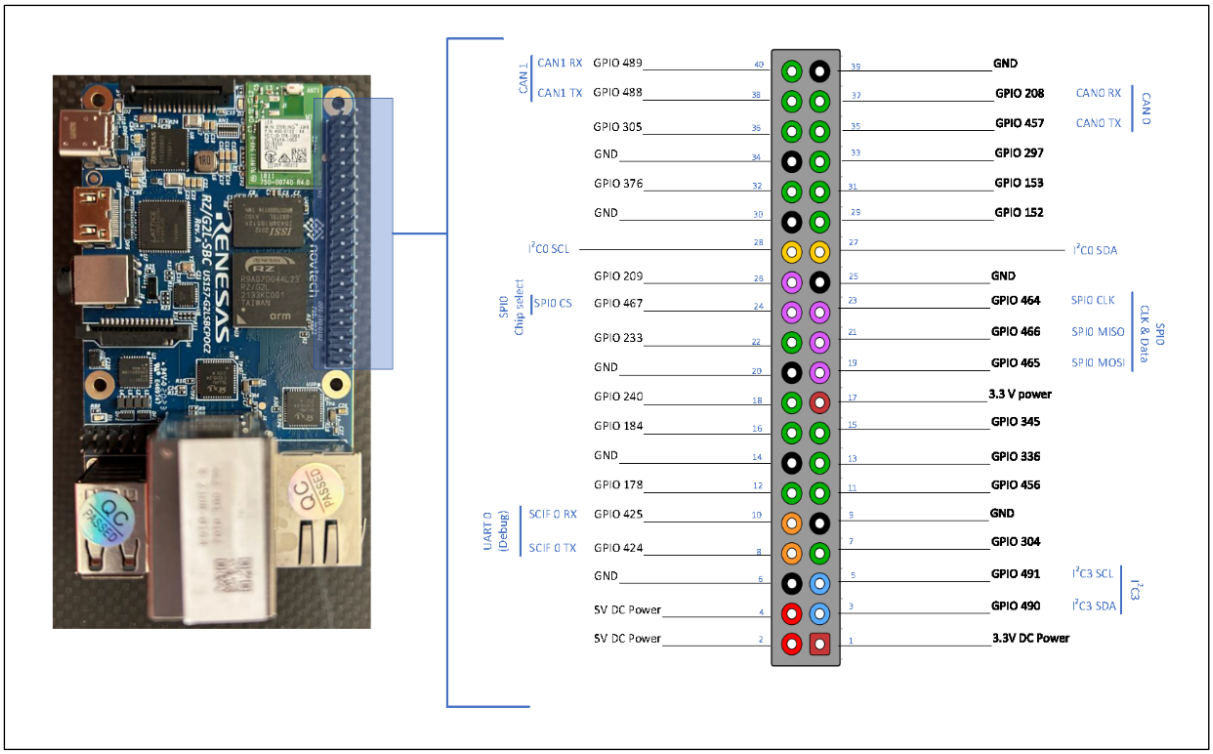


Figure 158. 40 PIN GPIO map with orientation details.

14.2.8.8 PMOD Type 6A Standard Interface

The RZ/G2L-SBC is equipped with a 2x6-pin header routed to the PMOD Type-6A interface conforming to the 1.3.0 specification of PMOD. It includes the alternate pin functions from the specification.

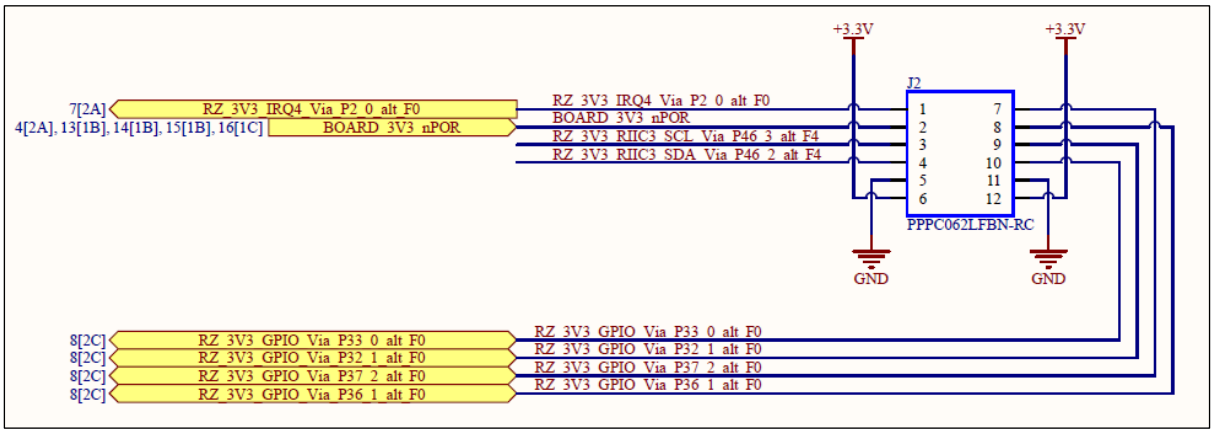


Figure 159. Schematic of PMOD Type 6A pin header J2.

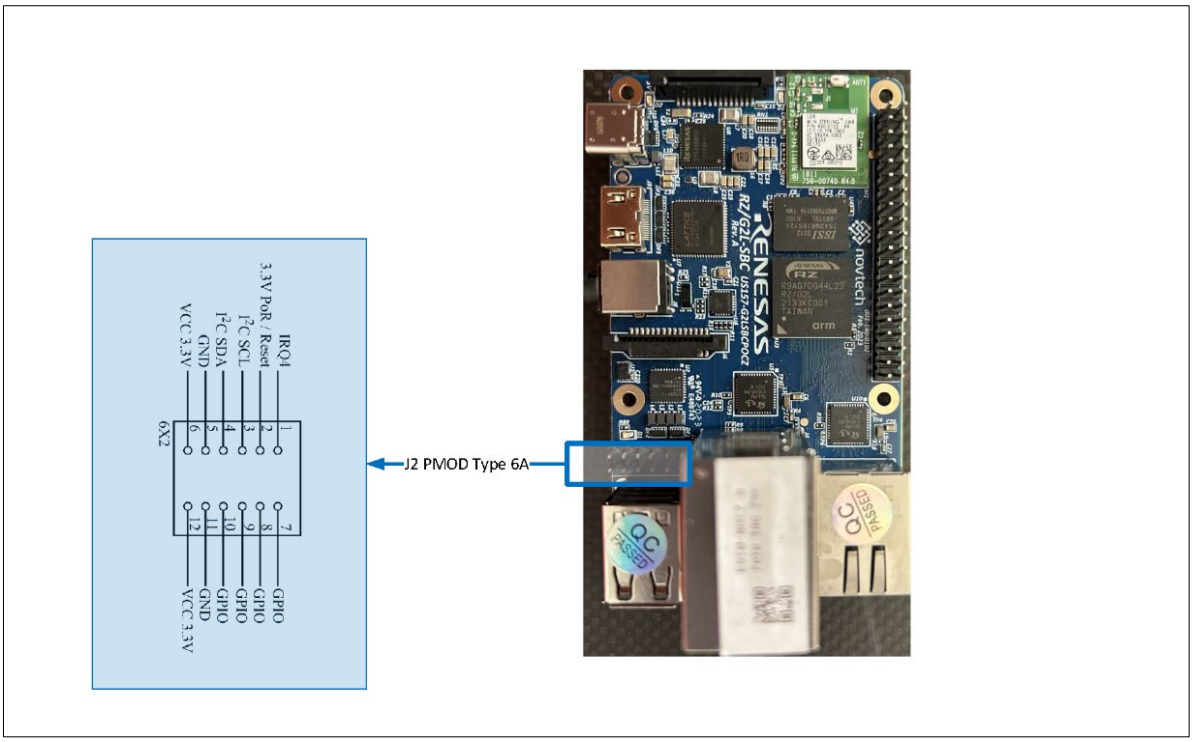


Figure 160. PMOD Type 6A 2x6 0.1 mm pinout with orientation details.

14.2.8.9 uSD-Card Interface

The RZ/G2L-SBC comes with a spring-loaded micro-SD card slot. This is intended to be the primary storage as well as the OS boot device. The SD card is connected to channel 0 of the RZ/G2L SoC SD/MMC interface. The SoC SDIO interface is compliant with memory card standard version 3.0 and supports UHS-1 mode of 50 MB/s (SDR50) and 104 MB/s (SDR104).

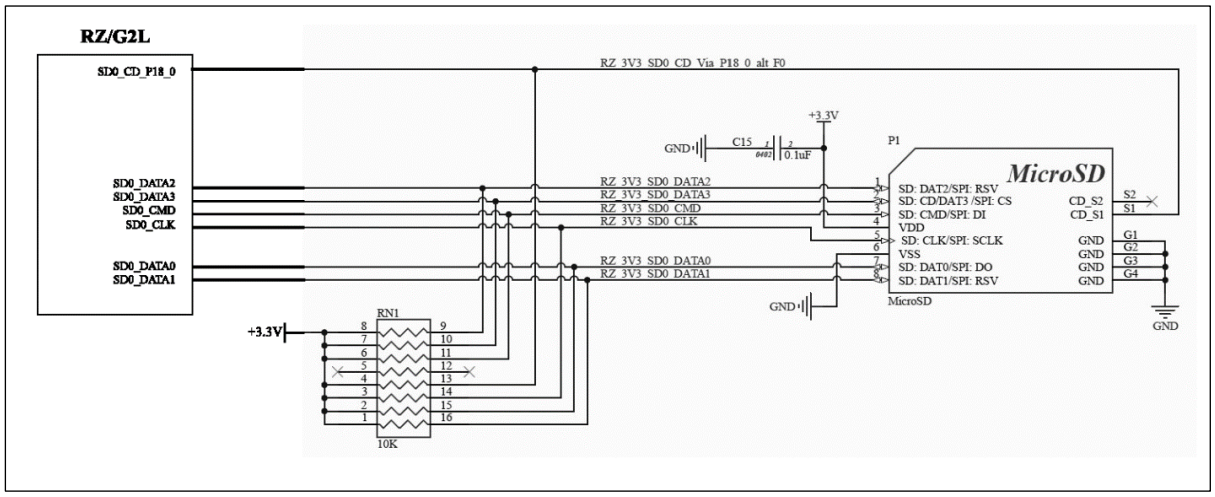


Figure 161. uSD-Card interface block diagram.

14.2.8.10 JTAG SWD Debug

The JTAG/SWD interface is an SMT pin-out on the bottom side of the board marked J11. It uses the standard 10-pin interface when populated. By default, this is not populated on the board. In addition to populating the pins of J11, the use of the J12 port to set BSCANP is necessary to trigger the JTAG

boundary scan of the RZ/G2L SoC. The SBC by itself will not be able to initiate the JTAG boundary scan mode. All the interface lines have pull-ups.

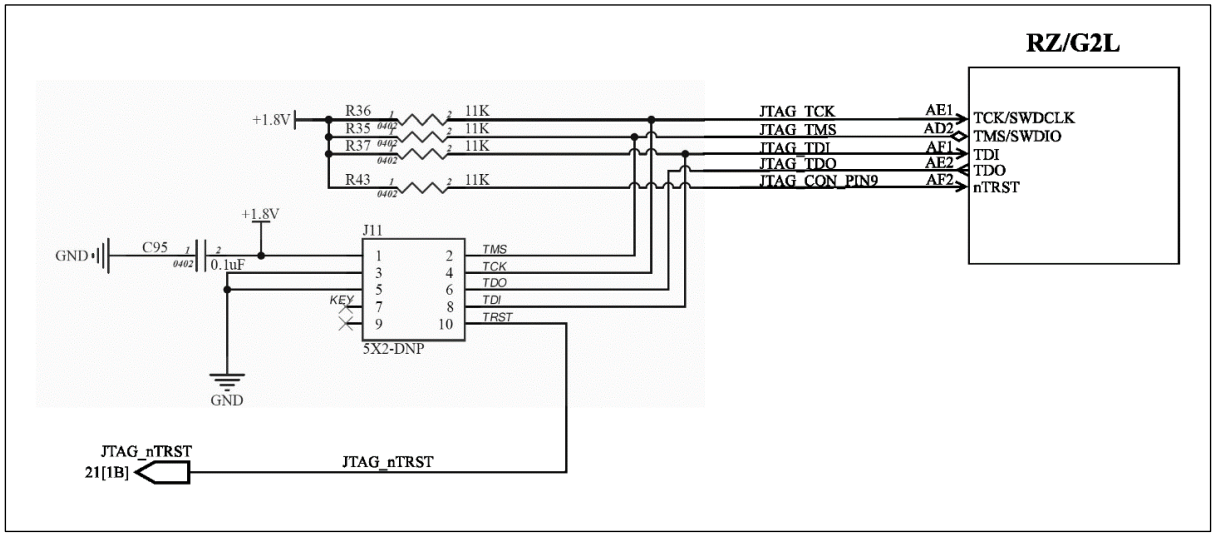


Figure 162. JTAG/SWD Block Diagram

14.2.8.11 Expansion Connector

The RZ/G2L-SBC has two connectors at the bottom, J12 and J13, that contain pinouts for the ADC inputs, Bootstrapping (boot mode selection), and the QSPI1 interface, in addition to a few GPIOs. This is meant to be used in conjunction with an adapter/daughter board. The primary uses of this are mostly custom versions, where factory flashing and other manufacturing functions are controlled by these lines. The ADC input lines are also mapped to the J13 connector.

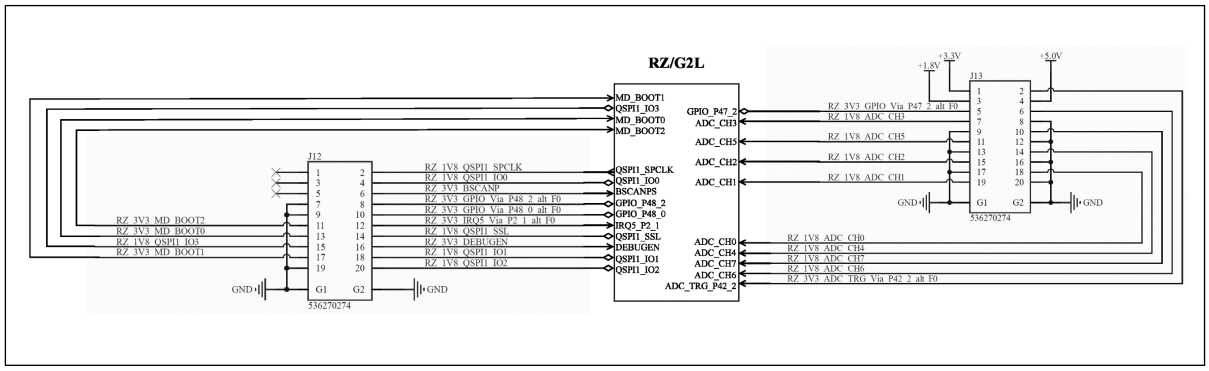


Figure 163. Block diagram of Bottom Connectors.

Refer to the appendix for details on the adapter board and flashing tools.

14.2.9 Memory

The RZ/G2L-SBC design uses four types of memory.

1. QSPI NOR Flash
2. DDR4 SDRAM
3. EEPROM
4. SD-Card

14.2.9.1 QSPI Flash

ss The QSPI flash memory is controlled by the SPI multi-I/O bus controller (SPIBSC) that is built into the RZ/G2L. This memory supports both single data rate (SDR) and double data rate (DDR) transfers at 66 MHz and 50 MHz clock frequency. QSPI0 interfaces to a Cyprus S25FS512SDSNFB010 64MiB NOR

Flash module. The QSPI is the default boot device, which contains the firmware: Arm Trusted Firmware (ATF), OPTEE (loaded but disabled by default), and U-Boot.

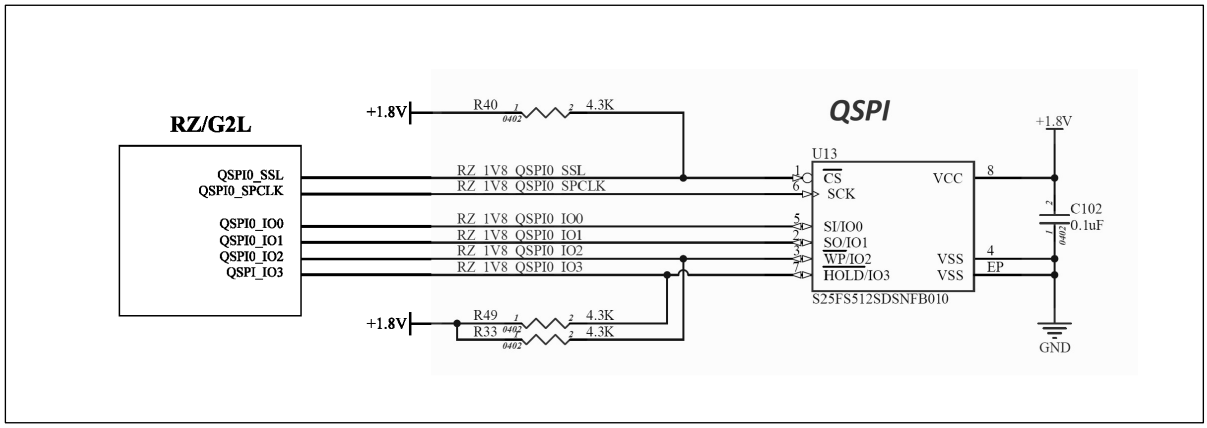


Figure 164. QSPI interface.

Note: The pull-up resistor on the clock line “QSPI0_SPCLK” is optional and is omitted in this design.

14.2.9.2 DDR4 SDRAM

The DDR4 SDRAM is controlled by the DDD3L/DDR4 SDRAM Memory Controller (MEMC) that is built into the RZ/G2L. This interface supports up to DDR4-1600 SDRAM, a 16-bit data bus width, and inline ECC.

This interface complies with JEDEC STANDARD JESD79-4C.

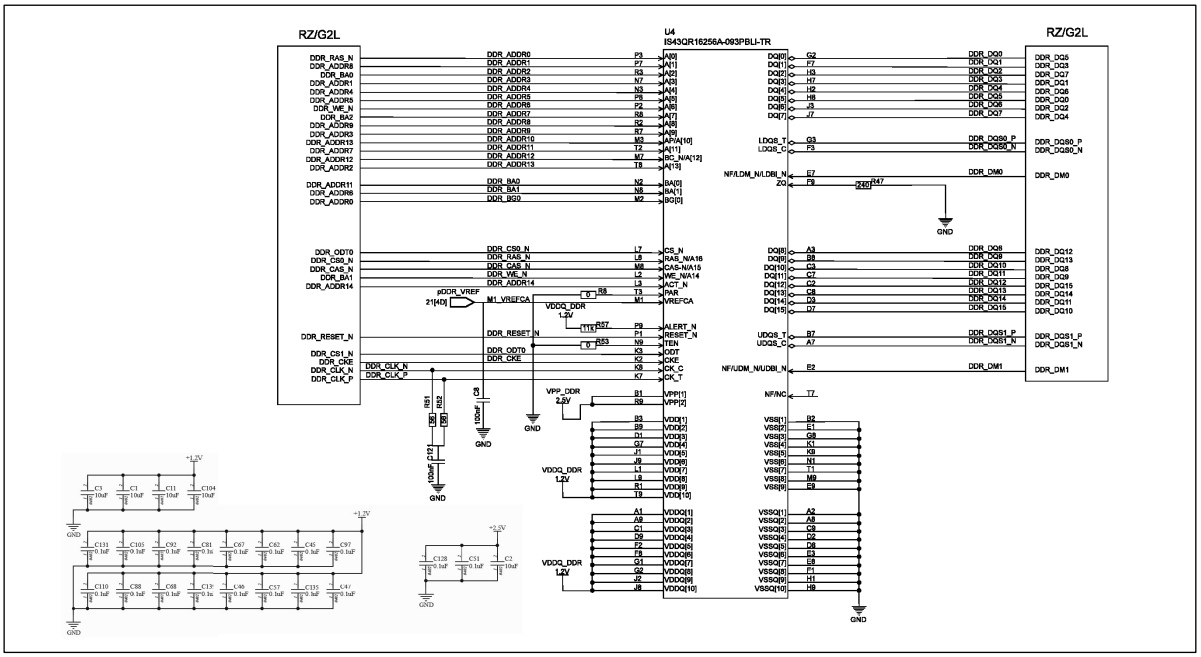


Figure 165. DDR4 SDRAM Interface

14.2.9.3 EEPROM with Temperature Sensor.

The RZ/G2L-SBC has an onboard CAT34TS02 I²C Temperature sensor with on-chip EEPROM, which is meant to hold factory data like Serial number, manufacturer name, etc. It is currently only used to

hold the Ethernet MAC IDs. Each board has its own registered MAC ID, which is stored on the EEPROM and read by u-boot during bootup. The EEPROM also has a built-in temperature sensor that can be read over the I²C interface. The EEPROM is configured as 16 pages of 16 bytes each for a total of 256 KiB (2 kilobits) of memory. Currently, two MAC IDs occupy 6 bytes of memory each for a total of 12 bytes.

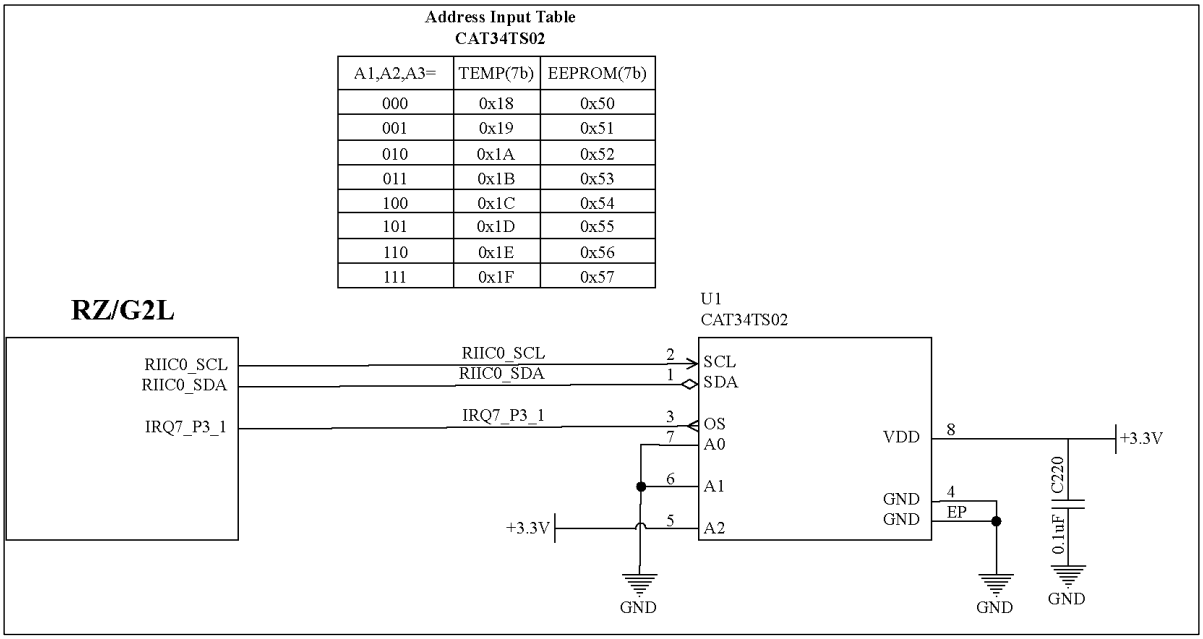


Figure 166. I²C EEPROM Block Diagram

Table 61. EEPROM Parameters

Parameter	Value	Description
I²C speed	100KHz / 400 kHz	It supports the standard and fast modes of operation.
EEPROM memory size	2 kib / 256 bytes	
EEPROM memory ordering	16 pages of 16 bytes each	Page bank array configuration
Temperature range	-20 °C to +125 °C	
Operating Voltage	3.3V	
Temperature alarm	Programmable over I²C	Three programmable trigger settings for high, low, and critical temperatures to raise an interrupt over the IRQ 7 line.

14.3 RS-G2L100 Board

The Geniatech AHAURA RS-G2L100 Development Board is based on Renesas' low-power, highly efficient RZ/G2L SoC, which is jointly designed & manufactured in close collaboration with Geniatech & Renesas.

This section delves into the functional and design aspects of the RS-G2L100. The image below highlights the key hardware components in the RS-G2L100 design.

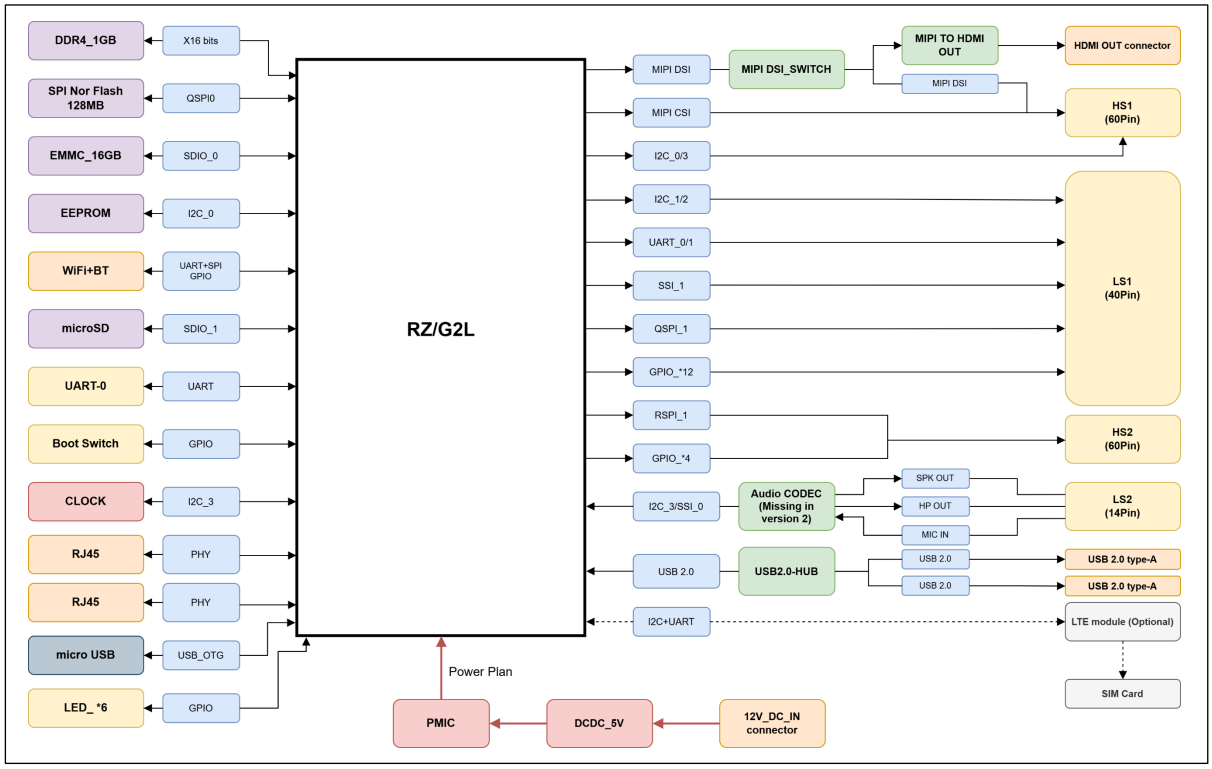


Figure 167. RS-G2L100 high-level overview

14.3.1 Notice

The hardware design, schematics, and detailed technical specifications of the RS-G2L100 board are proprietary to Geniatech and are protected under a Non-Disclosure Agreement (NDA).

The information provided in this section is minimal for reference purposes only and may be limited to publicly available or non-confidential details. It does not represent the complete hardware specification of the board.

For access to complete hardware documentation or further technical details, please contact Geniatech directly:

- Website: https://www.geniatech.com
- Email: sales@geniatech.com
- Product page: Renesas RZ 96Boards-CE SBC Linux Yocto Embedded Board - Geniatech
- Hardware Datasheet: ‘Download’ tab in product page.

14.3.2 Overview

Geniatech AHAURA RS-G2L100 key features include a high-performance 64-bit Dual-core processor, HDMI out display support at resolutions up to 1080P, hardware video decoding at up to 1080P, up to 4GB of RAM, dual-band 2.4/5.0 GHz wireless LAN, Bluetooth 4.0, two Gigabit Ethernet ports, two USB 2.0, and 4G/LTE Modem(Optional)

Processor

- RZ/G2L microprocessor (R9A07G044L24GBG)
- Arm® Cortex®-A55 × 2 cores, Arm® Cortex®-M33 × 1 core

- Multimedia and peripheral subsystems integrated

Memory and Storage

- DDR4 SDRAM: 2 GB × 1 device
- QSPI NOR Flash: 512 Mb × 1 device
- eMMC: 64 GB × 1 device
- microSD card slot: supported.

Clock and Power

- Clock Generator: 5-output programmable clock generator (5P35023)

Connectivity

- Ethernet PHY: integrated, supporting 10/100/1000 Mbps
- The HDMI to HDMI connector is a high-speed serial interface for a digital video module.
- 40-Pin Header (Raspberry Pi 3B compliant)
- High-Speed Connector: supports different high-speed interfaces such as MIPI-DSI / MIPI-CSI / USB2.0

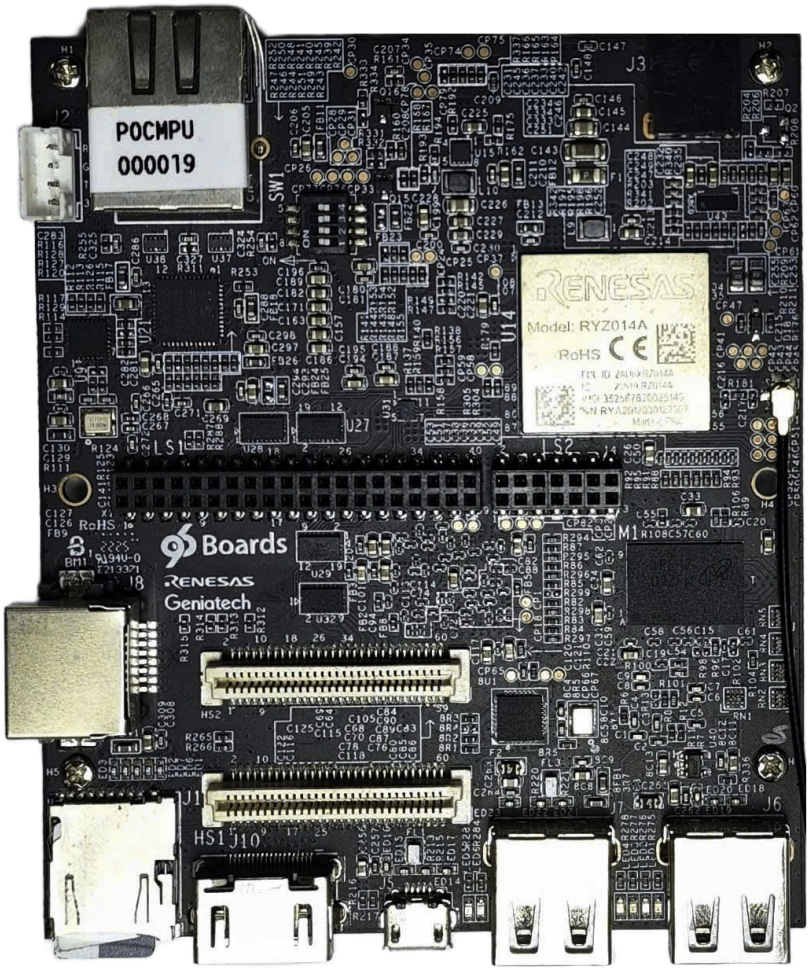


Figure 168. RS-G2L100 Top view

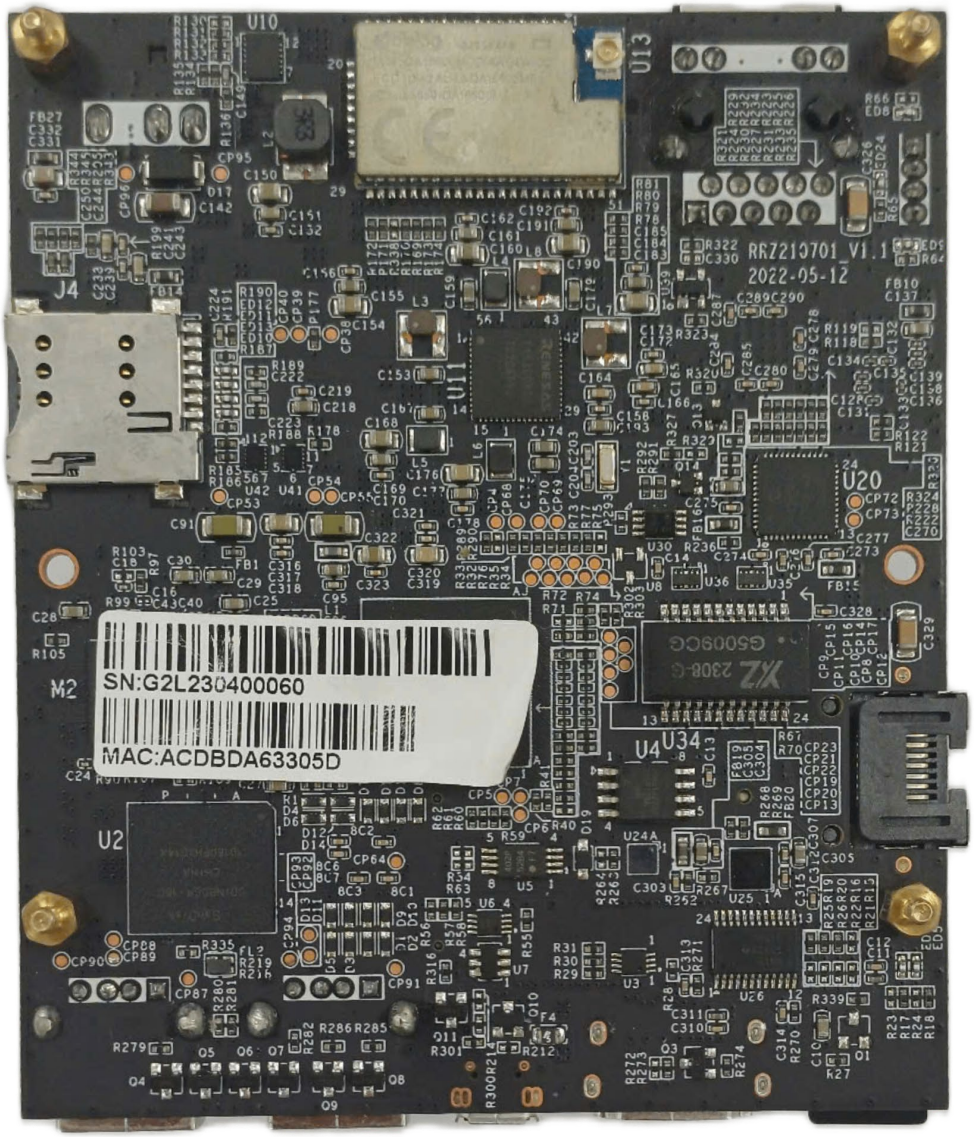


Figure 169. RS-G2L100 Bottom View

14.3.3 Overview of Connectors

Figure 170. RZ/G2L-EVK Overview of Connectors

14.3.4 Power Supply

14.3.4.1 DC Power Jack

The board includes a DC power jack designed for connection to a 12 V power source. A minimum current supply of 3 A is required to ensure stable operation and avoid brownouts. For systems using multiple peripherals, a power source rated at 4.5–5 A is recommended to provide sufficient margin and maintain reliable performance.

Figure 171. RS-G2L100 Power Supplies

14.3.4.2 Power Rails

This section describes how the board distributes power from the DC 12V 3A input jack into the various voltage rails required by the SoC and peripheral devices. Each functional block of the SoC is powered by a dedicated regulator output to ensure stable and reliable operation. The 12 V input provides sufficient headroom for efficient regulation and supports systems with multiple peripherals when paired with a power source rated at 4.5–5 A.

- The board is powered through a DC jack connector (J3) that accepts a 12 V input.
- The 12 V supply serves as the main input source and is protected by a reverse-polarity diode (D17) and a fuse (F1 rated at 12 V/2 A).
- From the 12 V input, a synchronous buck converter (ISL85005) generates a regulated 5 V rail capable of delivering up to 5 A for downstream power distribution.

Regulated rails

From the 5 V input, multiple voltage rails are generated by on-board DC-DC converters and LDOs:

- 3.3V rail:
  - Supplies 3.3 V peripheral devices and SoC I/O domains.
- 1.8 V rail:
  - Supplies low-voltage peripherals and interface logic.
- 1.2 V rail
  - Supports internal SoC power domains and DDR-related circuits.
- 1.1 V rail
  - Supplies SoC core and digital logic domains.

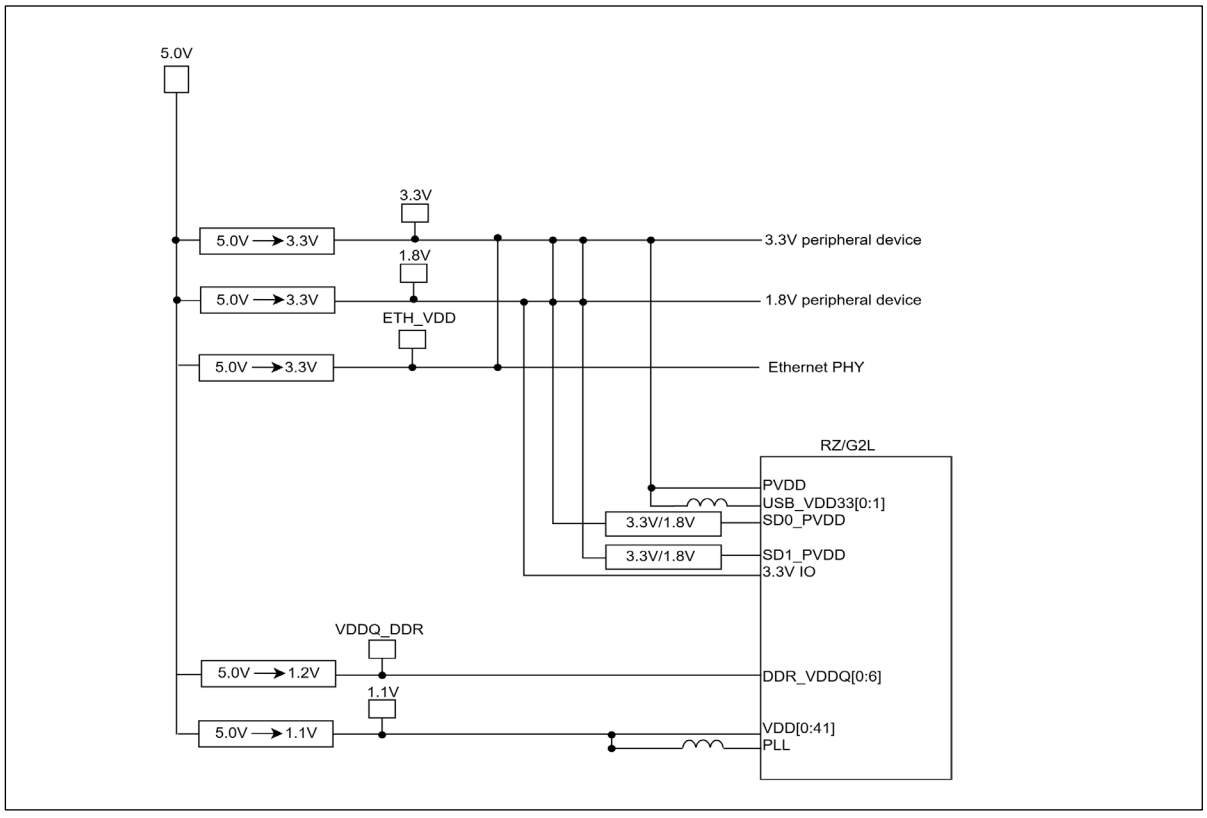


Figure 172. RS-G2L/100 Power Rails

14.3.5 RESET Control

This section explains how reset signals are managed on the RS-G2L100 board. Reset control ensures that the SoC, memory, and peripheral devices start in a defined state during power-up or after a manual reset.

Reset sources.

- Power-on reset – generated automatically by the PMIC (RAA215300) when the 5 V input supply is applied and stable.
- The RS-G2L100 board does not provide a physical reset button. System reset is triggered only by power cycling or by software-initiated reset through the SoC.

14.3.6 Clock Configuration

The RS-G2L100 provides a flexible clock distribution network to support the SoC, DDR, storage, and peripheral interfaces. A 24 MHz reference oscillator feeds a clock generator IC, which distributes clocks to the SoC, Ethernet PHYs, audio codec, and high-speed interfaces.

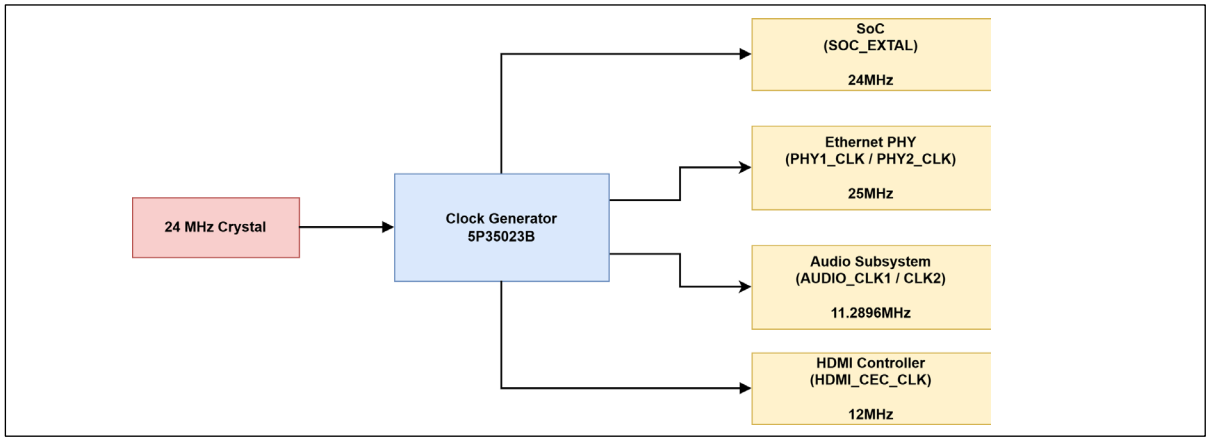


Figure 173. Block diagram of the clock configuration of RS-G2L100

14.3.7 Memory

The RS-G2L100 board's design uses four types of memory.

1. QSPI NOR Flash
2. eMMC Flash
3. DDR4 SDRAM
4. SD-Card

14.3.7.1 QSPI NOR Flash

The QSPI flash device is managed by the integrated SPI multi-I/O bus controller (SPIBSC) in the RZ/G2L. It is capable of operating in both single data rate (SDR) and double data rate (DDR) modes, supporting transfer speeds of up to 66 MHz in SDR and 50 MHz in DDR.

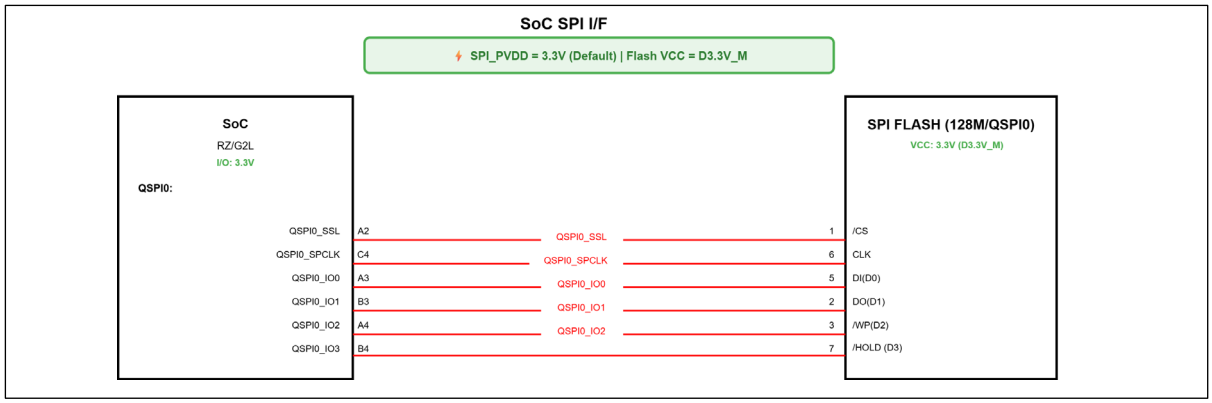


Figure 174. QSPI Block Diagram of RS-G2L100

14.3.7.2 DDR4 SDRAM

The RS-G2L100 board integrates DDR4 SDRAM connected to the integrated memory controller of the Renesas RZ/G2L SoC.

The memory subsystem:

- Supports DDR4 operation
- Complies with JEDEC DDR4 standards
- It is configured to support typical system memory requirements for the platform

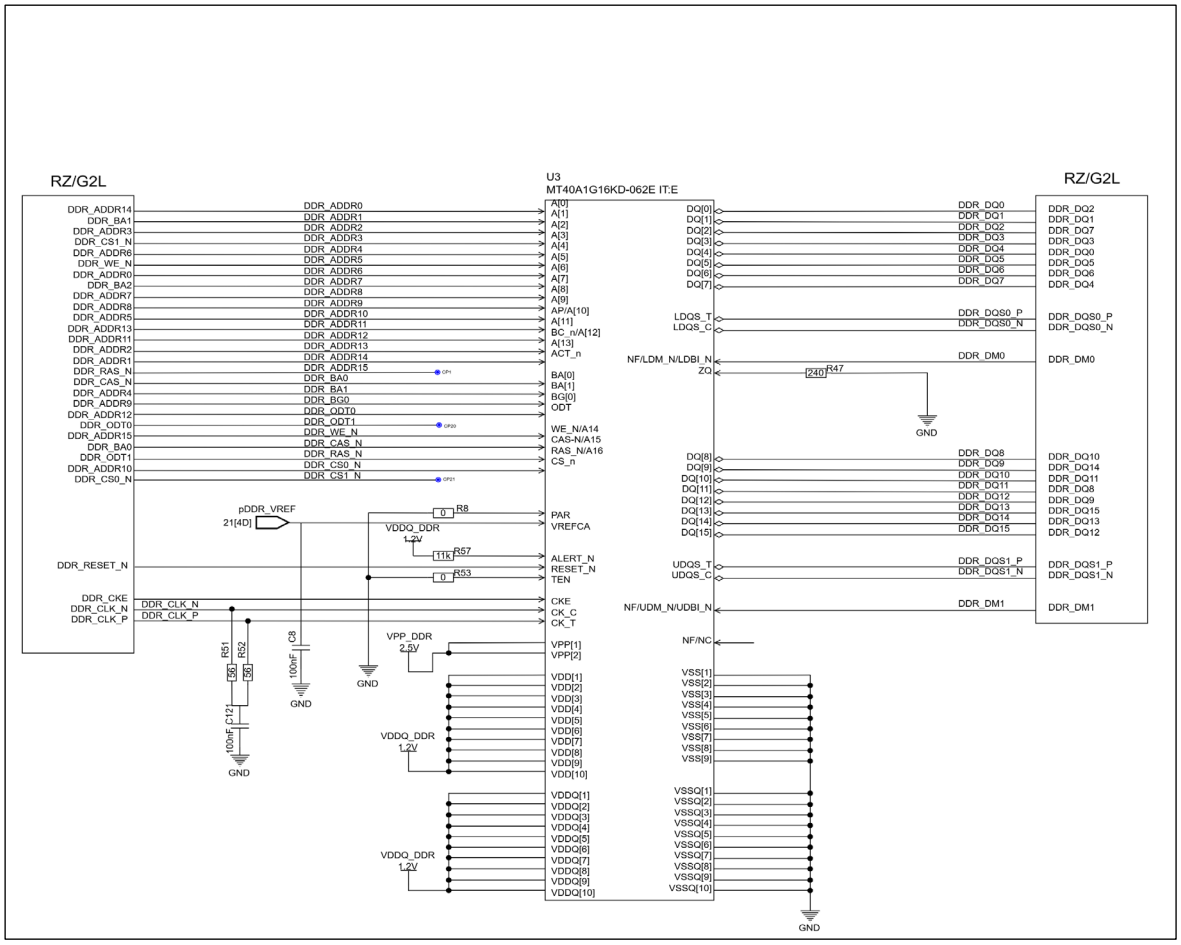


Figure 175. DDR4 SRAM interface

14.3.7.3 eMMC Flash

The on-board eMMC device is connected to the SD0 interface of the integrated SD/MMC controller on the RZ/G2L. It operates alongside the microSD card interface as an alternative storage option. This interface follows the JEDEC v5.1 specification and supports HS400 transfer mode.

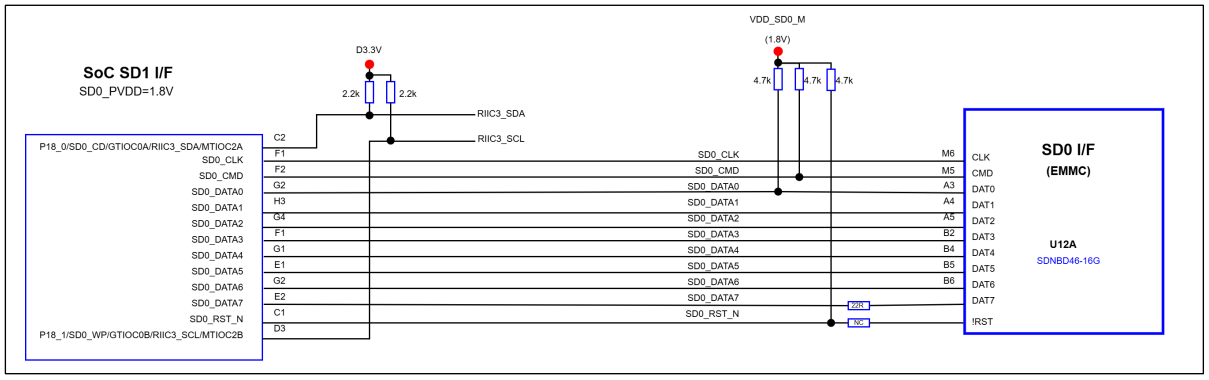


Figure 176. eMMC Block Diagram of RS-G2L100

14.4 RZ/G2L-EVK & RZ/V2L-EVK Board

The RZ/G2L-EVK and RZ/V2L-EVK share an identical hardware design. Both evaluation kits use the same PCB layout, connectors, and peripheral configuration, differing only in the SoC mounted on the board. Unless otherwise specified, the descriptions in this section apply equally to both EVKs.

14.4.1 Overview

The RZ/G2L SMARC Module Board provides a compact evaluation platform for the Renesas RZ/G2L microprocessor (R9A07G044L23GBG). Its design follows the SMARC v2.1 standard and is intended for functional evaluation, performance testing, and application software development.

The companion RZ/V2L SMARC Module Board (RTK9754L23C01000BE) is identical in hardware layout but mounts the R9A07G054L23GBG device. Since both processors are pin-compatible, the two boards are functionally equivalent.

Processor

- RZ/G2L microprocessor (R9A07G044L23GBG)
- Arm® Cortex®-A55 × 2 cores, Arm® Cortex®-M33 × 1 core
- Multimedia and peripheral subsystems integrated

Memory and Storage

- DDR4 SDRAM: 2 GB × 1 device
- QSPI NOR Flash: 512 Mb × 1 device
- eMMC: 64 GB × 1 device
- microSD card slot: supported, usable for booting as eSD

Clock and Power

- Clock Generator: 5-output programmable clock generator (5P35023)
- PMIC: Renesas RAA215300, supplies all SoC and board rails

Connectivity

- Ethernet PHY: integrated, supporting 10/100/1000 Mbps
- SMARC Edge Connector: 314-pin, 0.5 mm pitch; exposes unused SoC signals for carrier board expansion

- The Micro-HDMI connector via DSI/HDMI conversion module is mounted as standard for connection to a high-speed serial interface for the digital video module.
- The audio codec is mounted as standard for advanced development of an audio system. A 3.5 mm audio jack is provided for connection to audio equipment.
- MIPI-CSI interface: supports external camera modules (e.g., Google Coral Camera). Image recognition processing can be used with images input from an MIPI camera.

Debug and Expansion

- Debugging: 10-pin header for ARM Cortex debug interface
- Analog: 10-pin header for ADC signals
- Carrier board support: SMARC connector enables use with standard or custom carrier boards

14.4.2 Layout Parts

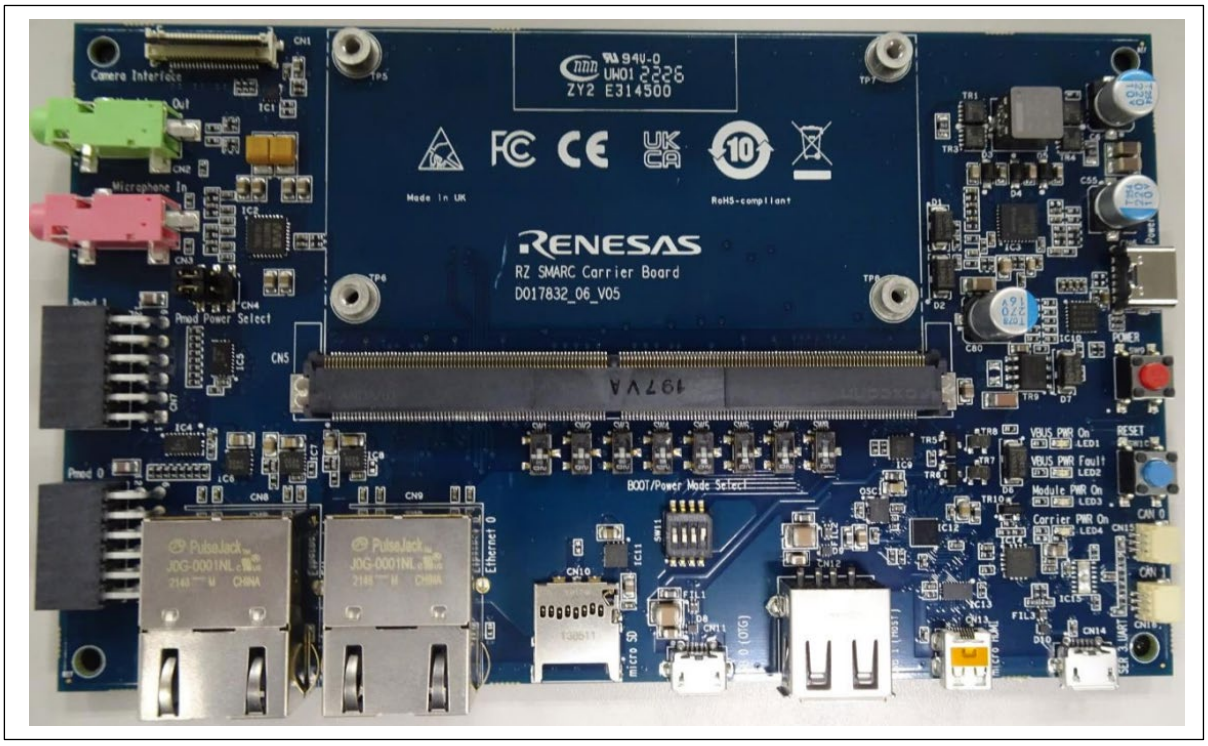


Figure 177. Top View of the RTK97X4XXXB00000BE

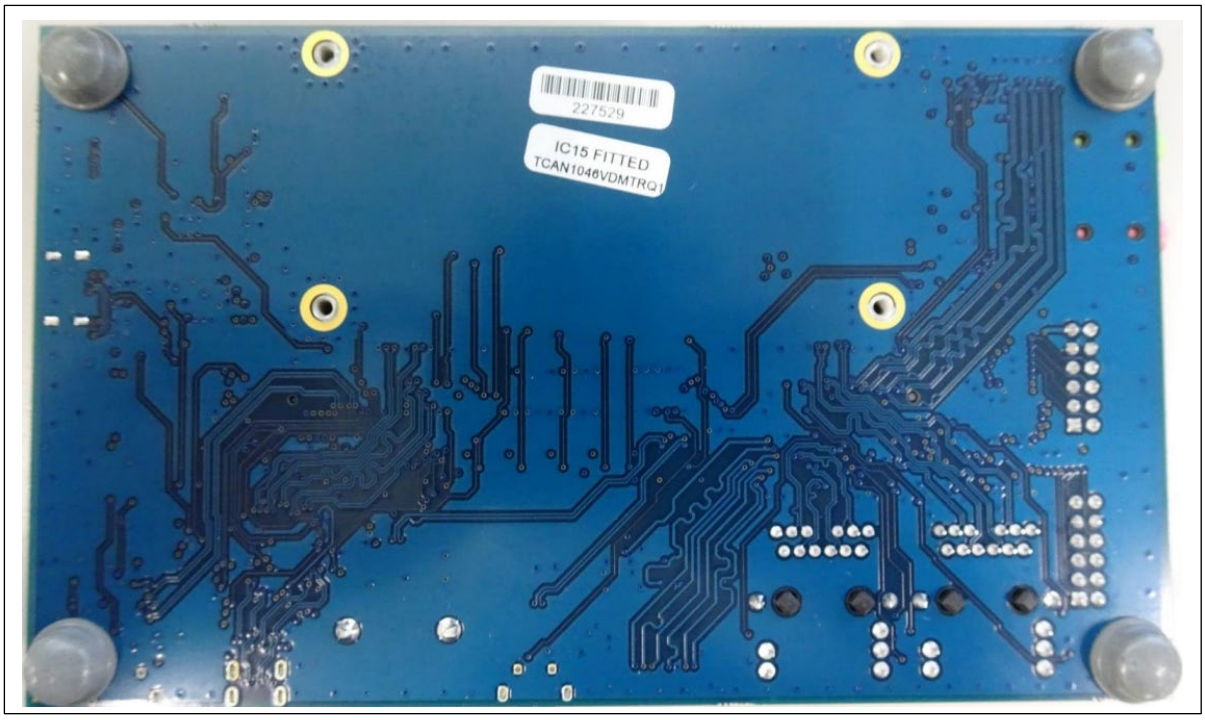


Figure 178. Bottom View of the RTK97X4XXXB00000BE

14.4.3 Overview of Connectors

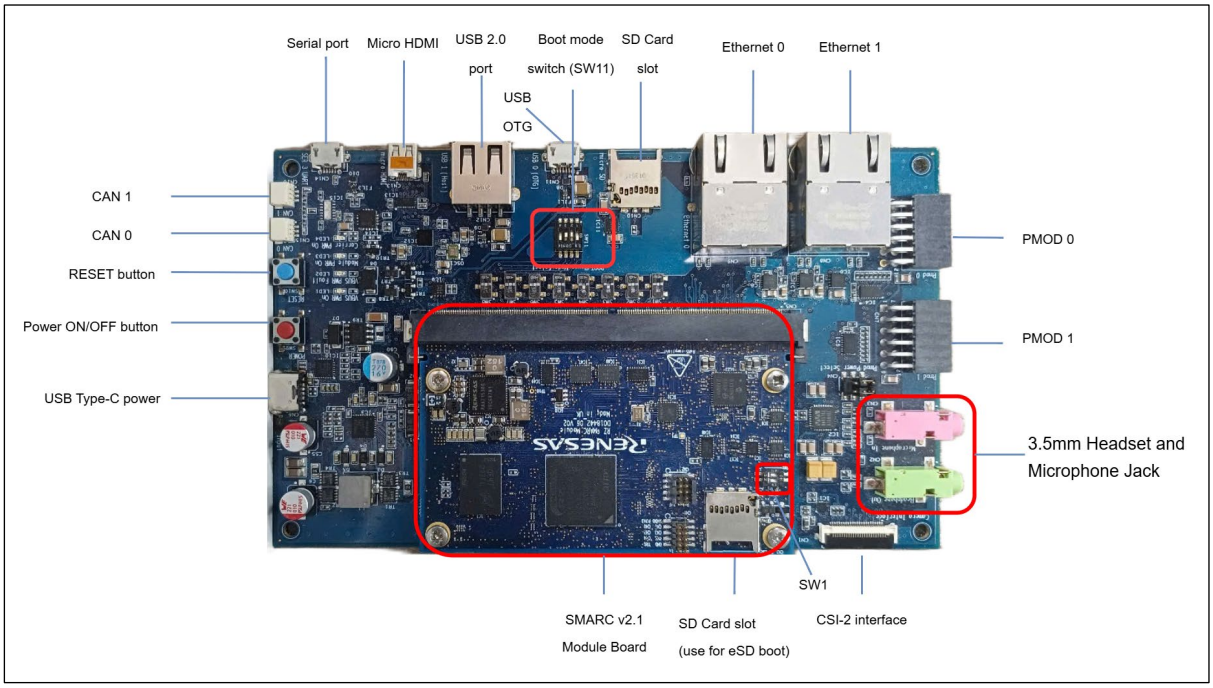


Figure 179. RZ/G2L-EVK Overview of Connectors

14.4.4 Power Supply

14.4.4.1 USB Type-C Power

The board includes a USB Type-C receptacle for power input with USB Power Delivery support. The connector is designed for connection to a 5V power source. A minimum current supply of 3A is required to ensure stable operation and avoid brownouts. For systems using multiple peripherals, a power source rated at 4.5–5A is recommended to provide sufficient margin.

14.4.4.2 Power Rails

This section describes how the RZ/G2L-EVK and RZ/V2L-EVK boards distribute power from the USB Type-C input into the different voltage rails required by the SoC and peripheral devices. Each functional block of the SoC is supplied by a dedicated regulator output to ensure stable operation.

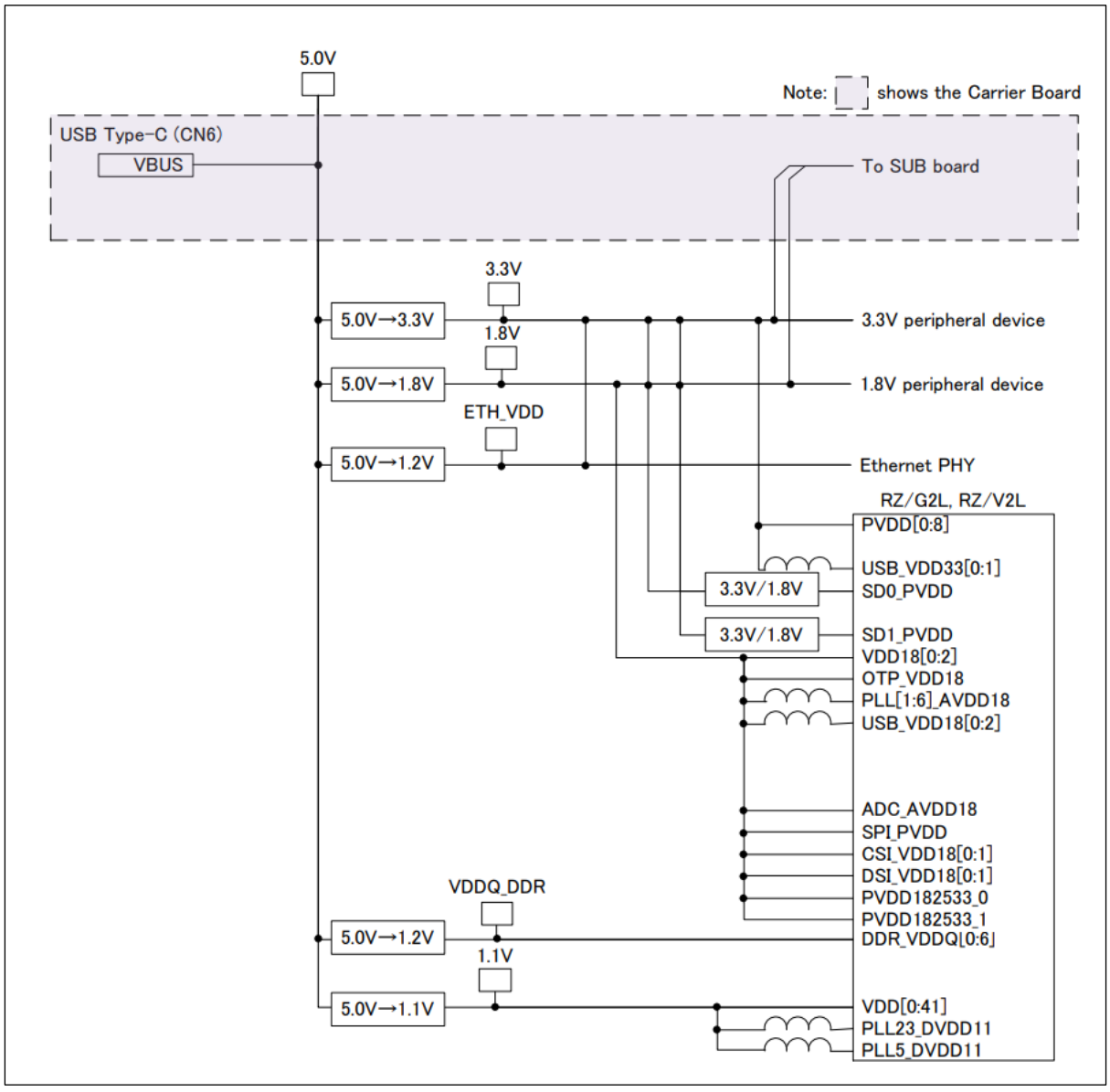


Figure 180. RZG2L-EVK Block Diagram of Power Rails

Input source

- The board is powered through a USB Type-C connector (CN6).
- VBUS 5V from USB Power Delivery serves as the main input supply.

Regulated rails

From the 5V input, multiple voltage rails are generated by on-board DC-DC converters and LDOs:

- 3.3V rail:
  - Supplies 3.3 V peripheral devices and SoC I/O domains.
  - Also, powers USB_VDD33, SD0_PVDD, and other 3.3 V blocks.
- 1.8 V rail:
  - Supplies low-voltage peripherals and interface logic.
  - Provides power to SD1_PVDD, OTP_VDD18, USB_VDD18, and Ethernet PHY.
- 1.2 V rail
  - Used by SoC internal PVDD domains and some PLL circuits.
  - Also feeds DDR VDDQ and selected core logic.
- 1.1 V rail
  - Supplies SoC digital cores (VDD11, DVDD11).
  - Used for PLL and analog domains requiring 1.1 V.

14.4.4.3 Power Supply Regulation

This section describes the on-board power management and regulation for the RZ/G2L-EVK and RZ/V2L-EVK. The board uses a dedicated Power Management IC (PMIC) to generate and sequence the supply rails required by the SoC and peripherals.

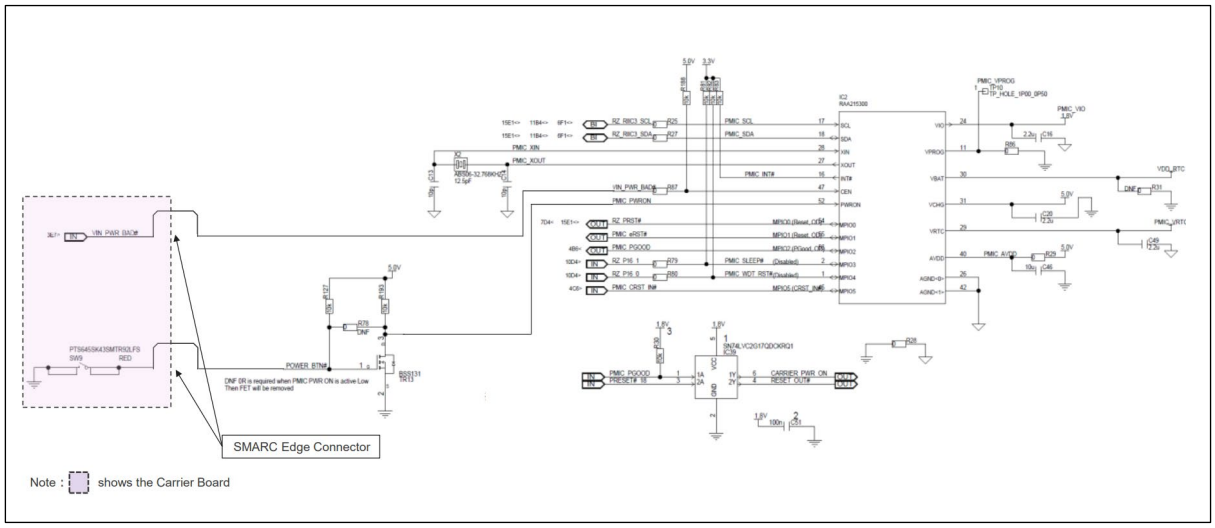


Figure 181. RZG2L-EVK Block Diagram of Power Supply Regulation

14.4.5 Power Management Integrated Circuit (PMIC)

LDO1 and LDO2 provide low-current regulated outputs (up to 0.3 A each) used to supply SoC subsystems or peripherals that require stable voltage rails. Their output voltages are not fixed in hardware but are determined by the state of configuration pins on the SoC.

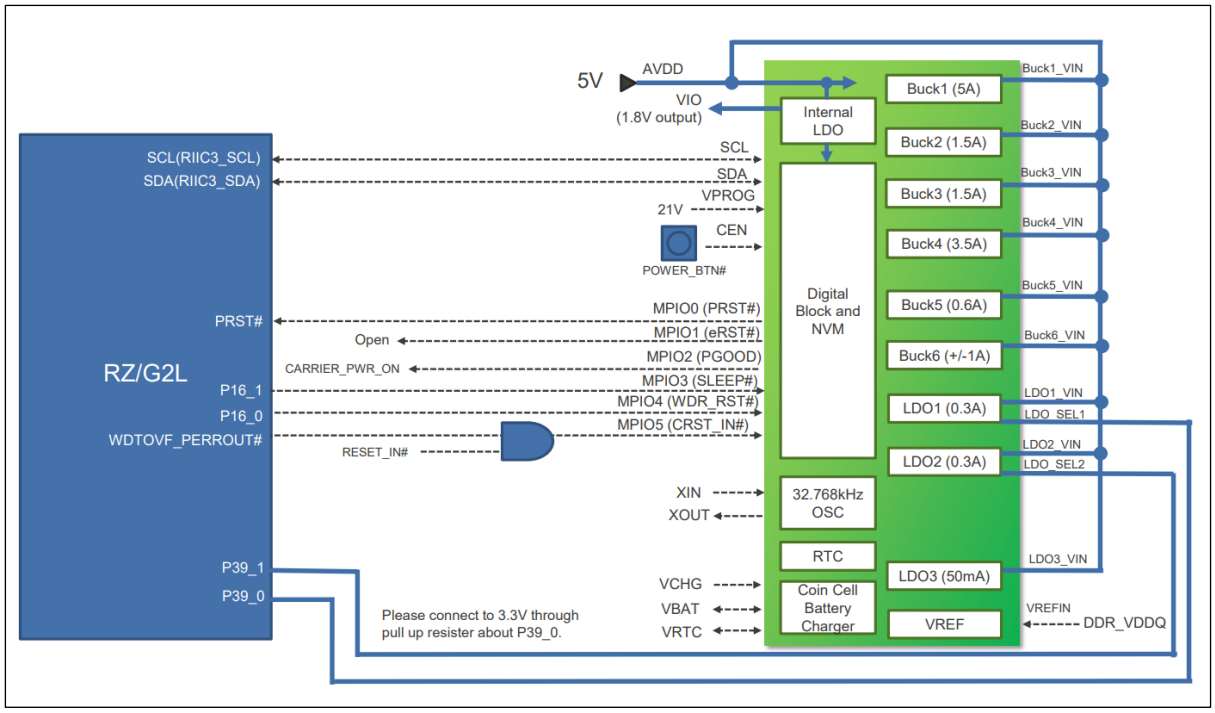


Figure 182. RZG2L-EVK Block diagram of PMIC

14.4.6 RESET Control

This section explains how reset signals are managed on the RZ/G2L-EVK and RZ/V2L-EVK boards. Reset control ensures that the SoC, memory, and peripheral devices start in a defined state during power-up or after a manual reset.

Reset sources.

- Power-on reset — generated automatically by the PMIC (RTK9744L23C01000BE / RAA215300) when the 5V input supply is applied and stable.
- Manual reset — initiated by pressing the reset button (RESET_BTN#), which asserts RESET_IN#.

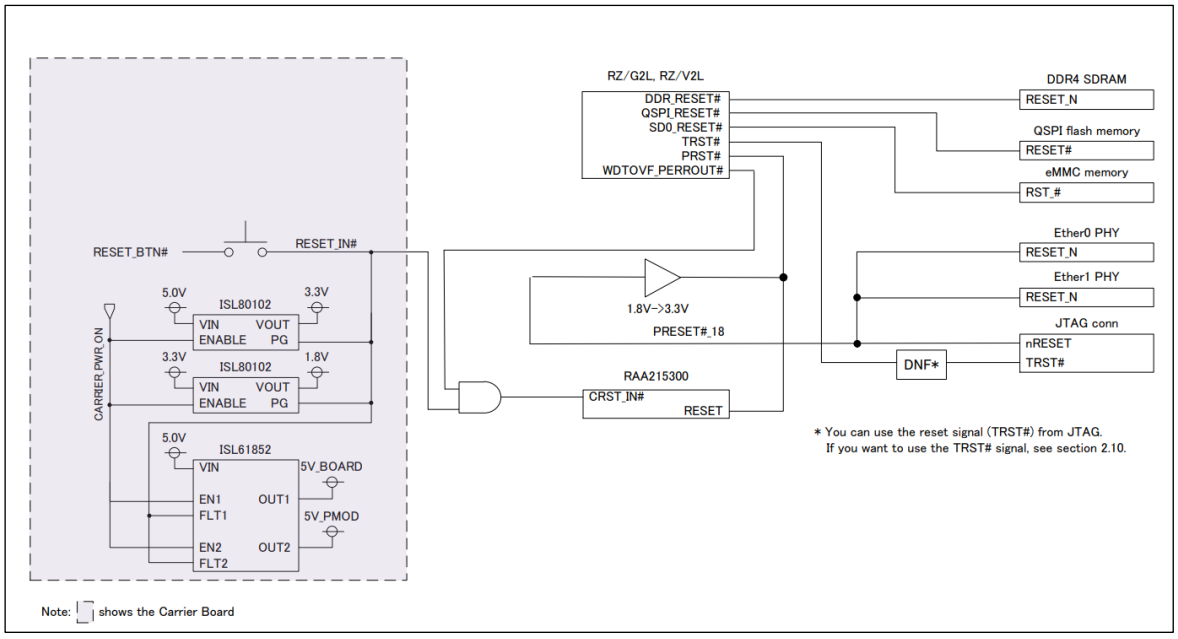


Figure 183. RZG2L-EVK Block Diagram of RESET Control

14.4.7 Clock Configuration

The RZ/G2L-EVK provides a flexible clock distribution network to support the SoC, DDR, storage, and peripheral interfaces. A 24 MHz reference oscillator feeds a clock generator IC, which distributes clocks to the SoC, Ethernet PHYs, audio codec, and high-speed interfaces.

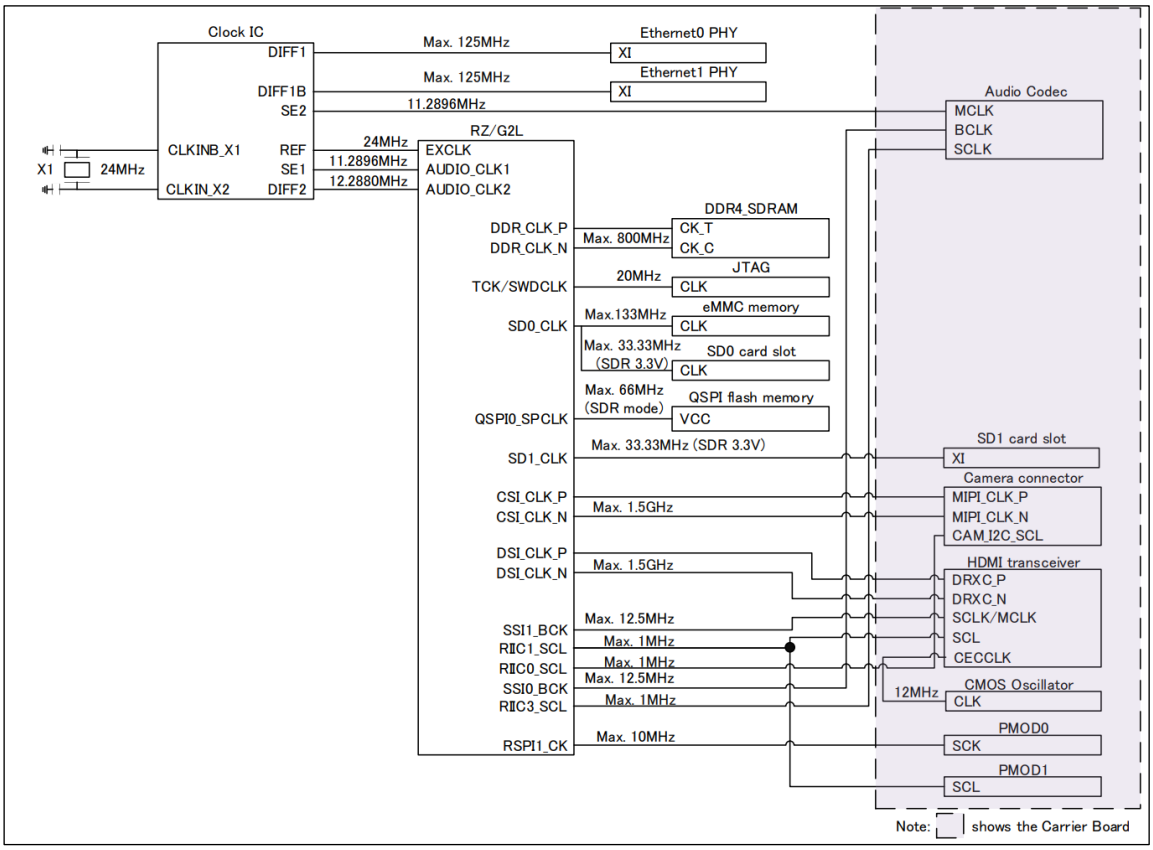


Figure 184. Block Diagram of Clock Configuration

14.4.8 Power Management Integrated Circuit (PMIC)

The RAA215300 is a high-performance, low-cost, 9-channel PMIC designed for 32-bit and 64-bit MCU and MPU applications. It supports DDR3, DDR3L, DDR4, and LPDDR4 memory power interfaces. The internally compensated regulators, built-in Real-Time Clock (RTC), 32kHz crystal oscillator, and coin cell battery charger provide a highly integrated, small-footprint power solution ideal for System-On-Module (SOM) applications.

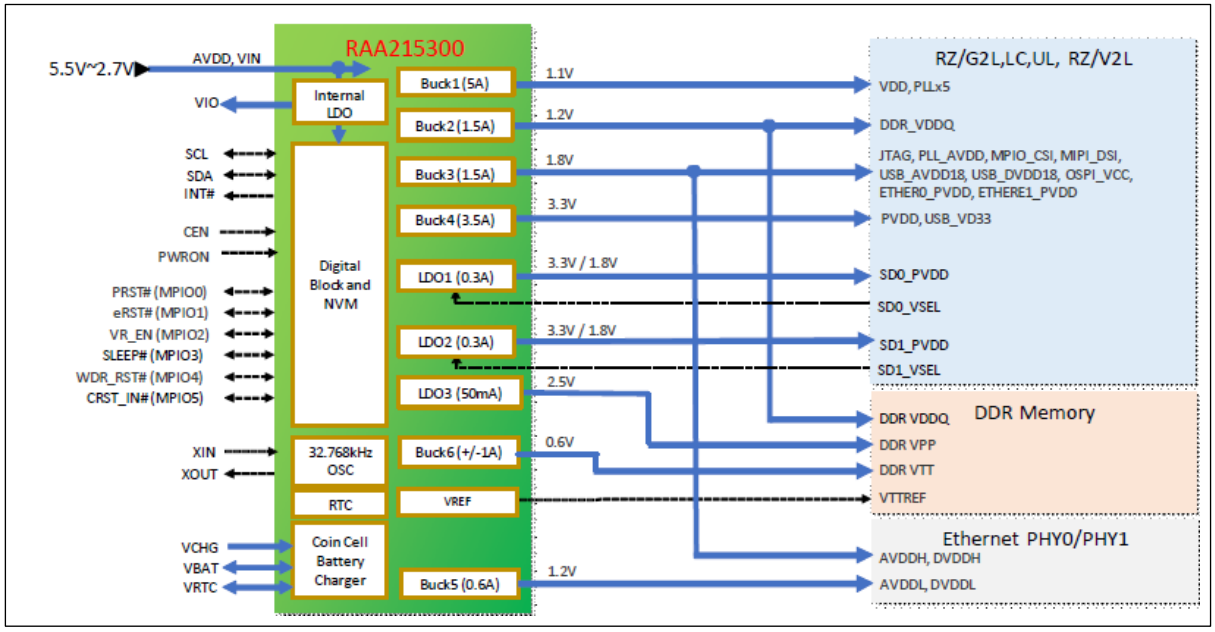


Figure 185. RAA215300 Block Diagram

14.4.9 Peripheral Interface

14.4.9.1 Gigabit Ethernet

The RZ/G2L-EVK integrates a Gigabit Ethernet interface to provide high-speed network connectivity. The SoC contains a built-in Ethernet Media Access Controller (E-MAC) that implements the MAC layer functions. This controller connects externally to a dedicated Ethernet PHY, which handles the physical layer signaling.

The Ethernet subsystem requires a reference clock, which is supplied from an external clock generator to the PHY. This ensures stable timing for both transmission and reception of data.

Standards compliance.

- The interface follows the IEEE 802.3 specification for Ethernet operation.
- The link between the SoC and the PHY uses the Reduced Gigabit Media-Independent Interface (RGMII) standard.
- Supported data rates include 10 Mbps, 100 Mbps, and 1000 Mbps.

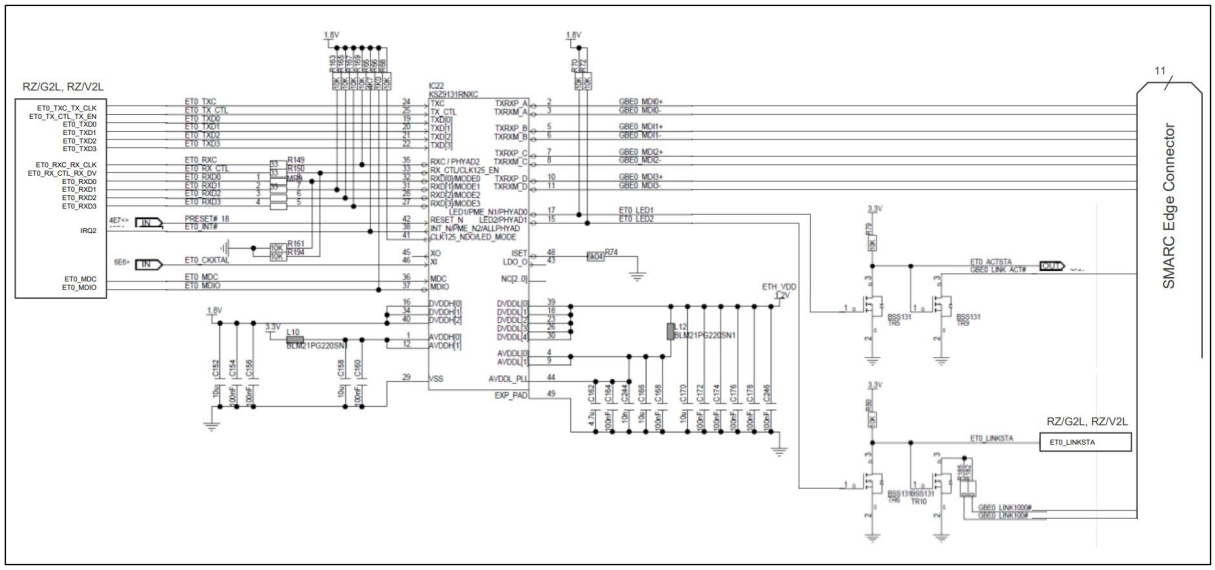


Figure 186. RZG2L-EVK Block Diagram of Ethernet

14.4.9.2 USB 2.0 Ports

The RZ/G2L-EVK provides two USB 2.0 interfaces:

- USB0 (OTG Port)
  - Configurable as either Host or Device (On-The-Go mode).
  - Supports VBUS detection and ID pin control for role switching.
  - Exposed through a Micro-USB connector for direct peripheral or host connection.
- USB1 (Host Port)
  - Dedicated Host-only port, intended for USB peripherals such as keyboards, storage devices, or Wi-Fi dongles.
  - Exposed through a standard USB Type-A connector.

Both ports comply with the USB 2.0 specification, supporting high-speed (480 Mbps) and full-speed (12 Mbps) operation. Protection circuitry is implemented with ESD diodes and EMI filters to ensure stable operation and compliance with EMI/ESD requirements.

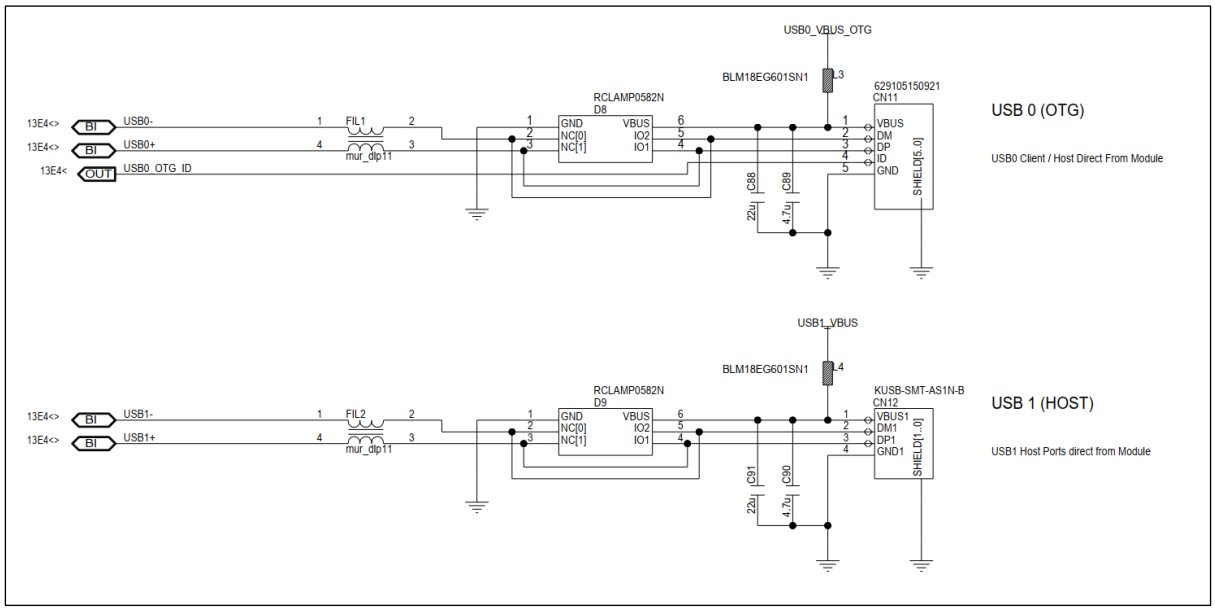


Figure 187. RZG2L-EVK Block Diagram of USB 2.0 Ports

14.4.9.3 MIPI CSI Interface

The RZ/G2L-EVK includes a MIPI CSI-2 camera interface for attaching external image sensors. The interface provides high-speed serial lanes optimized for video capture and vision processing tasks. The interface is designed to work with standard MIPI CSI-2 camera modules. For example, the Google Coral Camera Module can be connected directly, enabling rapid prototyping of computer vision and AI workloads.

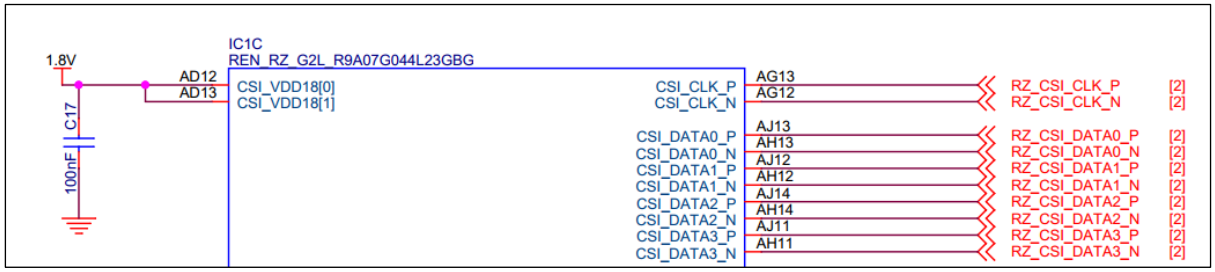


Figure 188. RZ/G2L-EVK Schematic MIPI CSI

14.4.9.4 Audio Subsystem with 3.5mm Jack

The RZ/G2L-EVK integrates an audio subsystem based on the WM8978 audio codec, providing both input and output capabilities.

- Microphone Input (MIC):
  - A dedicated microphone interface is available through a 3.5 mm jack, with biasing and filtering circuitry for stable audio capture.
- Headphone Output:
  - The codec drives a stereo headphone output via a 3.5 mm jack, supporting audio playback for development and testing.
- Codec Integration:
  - The audio codec is powered by 1.8 V and 3.3 V rails and interfaces with the SoC over the I²S bus. Control is performed via an I²C interface, allowing configuration of audio paths and gain settings.

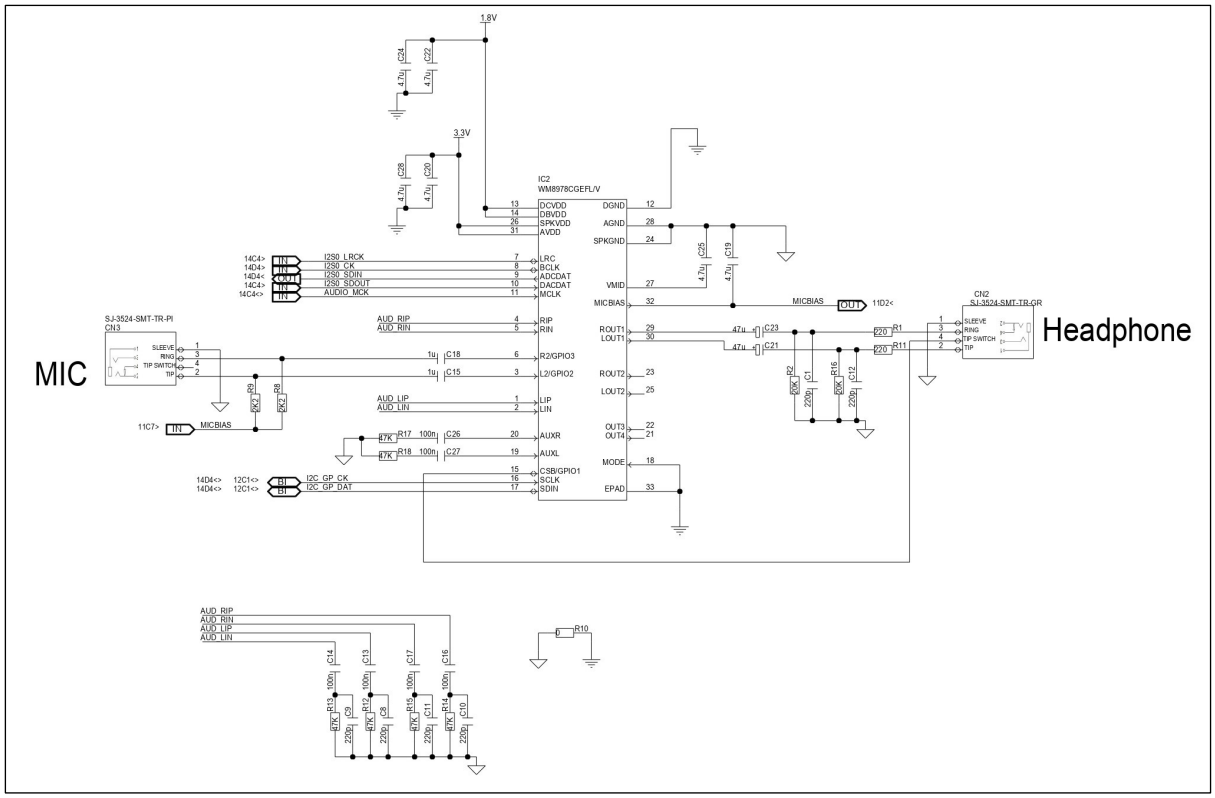


Figure 189. Audio Subsystem Block Diagram

14.4.9.5 HDMI Display Subsystem

The RZ/G2L-EVK includes a micro-HDMI Type-D connector that provides display output through a dedicated HDMI transmitter (ADV7533). The interface uses the SoC's MIPI-DSI signals, which are converted to HDMI TMDS by the transmitter.

The HDMI block is supported by a 27 MHz reference clock, I²C control lines for configuration, and hot-plug detection. Power is supplied through multiple rails (1.8V, 3.3V, and 5V), with appropriate filtering and decoupling to ensure signal integrity.

This implementation complies with HDMI 1.4 specifications, enabling resolutions up to 1080p at 60 Hz and supporting features such as audio over HDMI and Consumer Electronics Control (CEC).

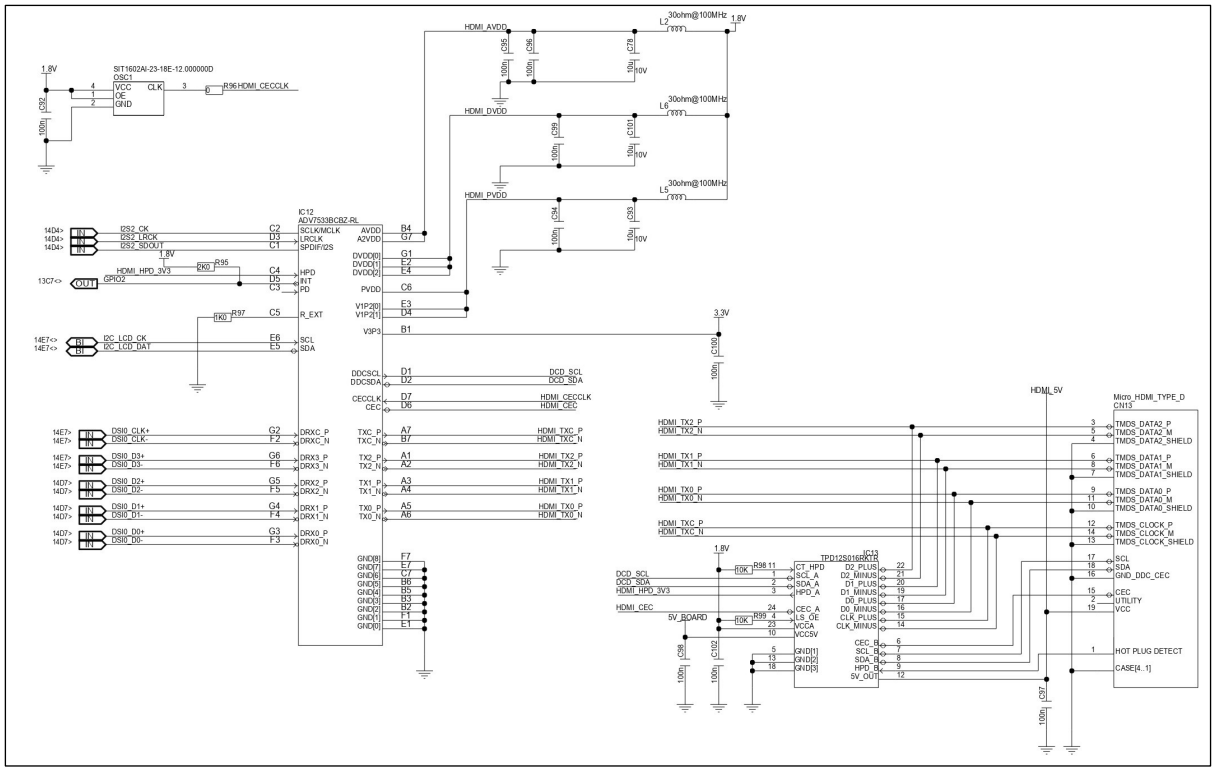


Figure 190. HDMI and Micro HDMI Interface Block Diagram

14.4.9.6 uSD-card Interface

The evaluation board includes a microSD slot connected to channel 0 of the SoC's SD/MMC interface. This interface allows the use of a microSD card alongside the onboard eMMC device.

For boot mode switches, please refer to Section 16.2 Boot Mode Reference (Non-SCIF) RZG2L-EVK & RZV2L-EVK for more detailed information.

Standards compliance.

- Conforms to the SD Memory Card Standard v3.0.
- Supports UHS-I transfer modes, including:
  - SDR50: up to 50 MB/s
  - SDR104: up to 104 MB/s

The microSD interface is typically used for removable storage, firmware updates, or system boot in development setups where eMMC is not required.

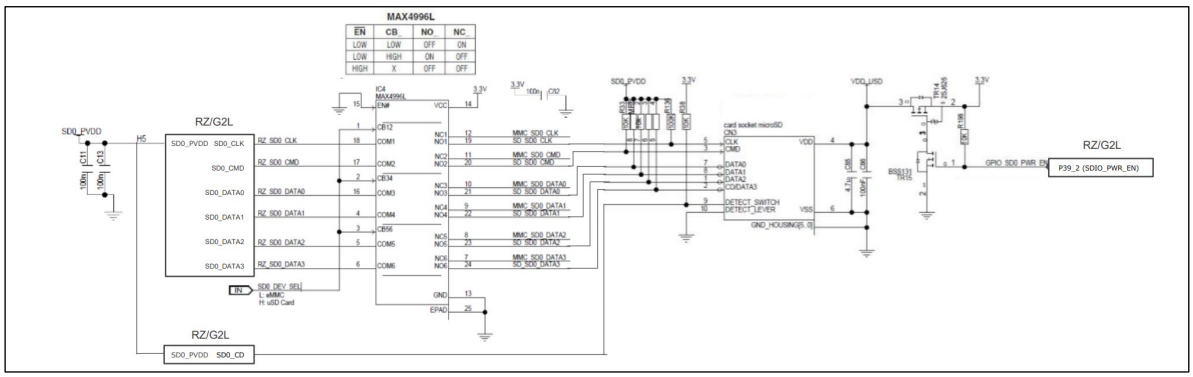


Figure 191. Block Diagram of uSD-card

14.4.9.7 PMOD Interface

The RZ/G2L-EVK provides two PMOD connectors (PMOD0 and PMOD1) to allow flexible peripheral expansion. These connectors follow the Digilent PMOD standard and support different functional configurations such as SPI, UART, and I²C, enabling engineers to attach a wide variety of external modules.

PMOD0 – Type 2A (Expanded SPI)

- Provides an SPI interface with CS, MOSI, MISO, and SCK signals.
- Includes additional GPIO pins for interrupts, reset, and chip select extensions.
- Suitable for SPI-based expansion modules such as sensors, displays, or communication ICs.

PMOD1 – Type 3A / Type 6A (Expanded UART/I²C)

- Configurable as Type 3A (UART) with TXD, RXD, CTS, RTS signals plus a general-purpose GPIO.
- Alternatively supports Type 6A (I²C) with SCL, SDA, interrupt, reset, and extra GPIO lines.
- Offers flexibility for modules requiring serial communication or I²C-based expansion devices.

Power and Pinout

- Both PMOD connectors provide 3.3V and 5V power supply options, selectable via jumpers.
- Pinouts conform to PMOD specifications, ensuring compatibility with a wide range of third-party modules.
- Each connector exposes up to 12 pins, combining data, control, and power signals.

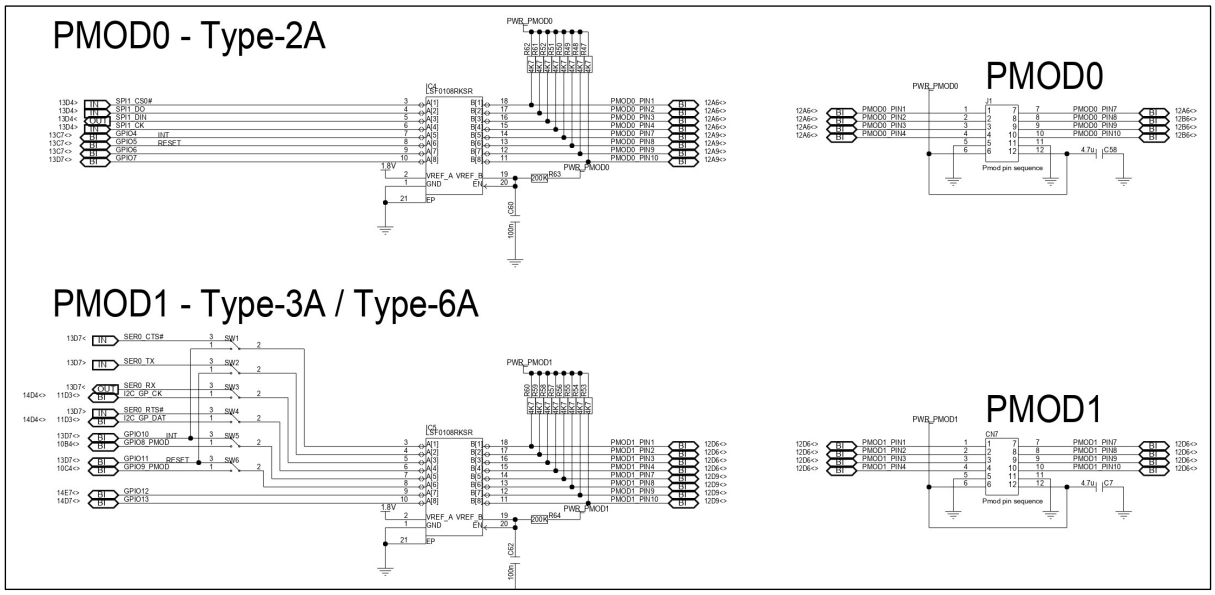


Figure 192. PMOD Type 2A, 3A and 6A Interfaces

14.4.9.8 Analog to Digital Converter

The RZ/G2L SMARC Module Board (RTK9744L23C01000BE) provides access to the on-chip Analog-to-Digital Converter (ADC) through a 10-pin header. This interface allows developers to evaluate analog signal acquisition features of the RZ/G2L processor.

Features:

- Up to six ADC input channels (CH0–CH5) are available via the header.
- Each channel supports 1.8 V analog input (ADC_AVDD domain).

- Signals are routed directly from the RZ/G2L SoC pins to the connector for external measurement or application-specific wiring.
- Additional pins are provided for ADC trigger input and power references (1.8 V and 3.3 V).

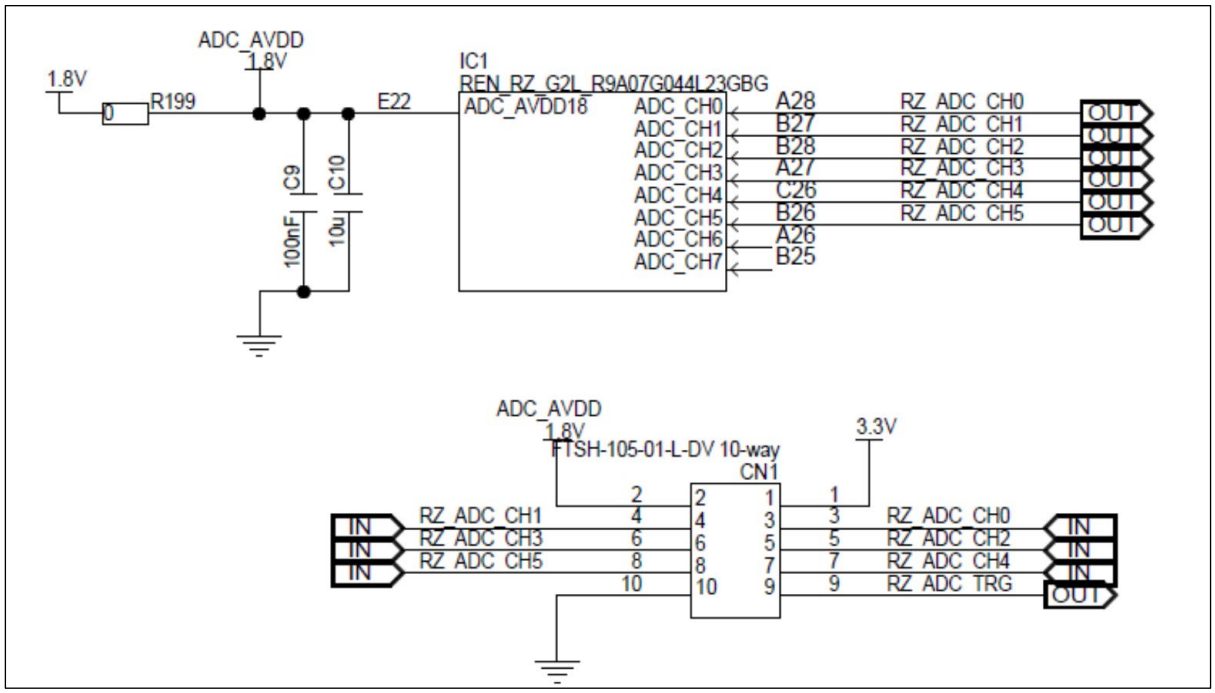


Figure 193. RZG2L-EVK Block Diagram of ADC

14.4.9.9 JTAG SWA Debug

The RZ/G2L-EVK provides a 10-pin header (CN2) for connection to an external debugger. This interface enables hardware-level debugging and supports both JTAG and Serial Wire Debug (SWD) protocols. Debug access is available for the Cortex-A55 application cores and the Cortex-M33 real-time core.

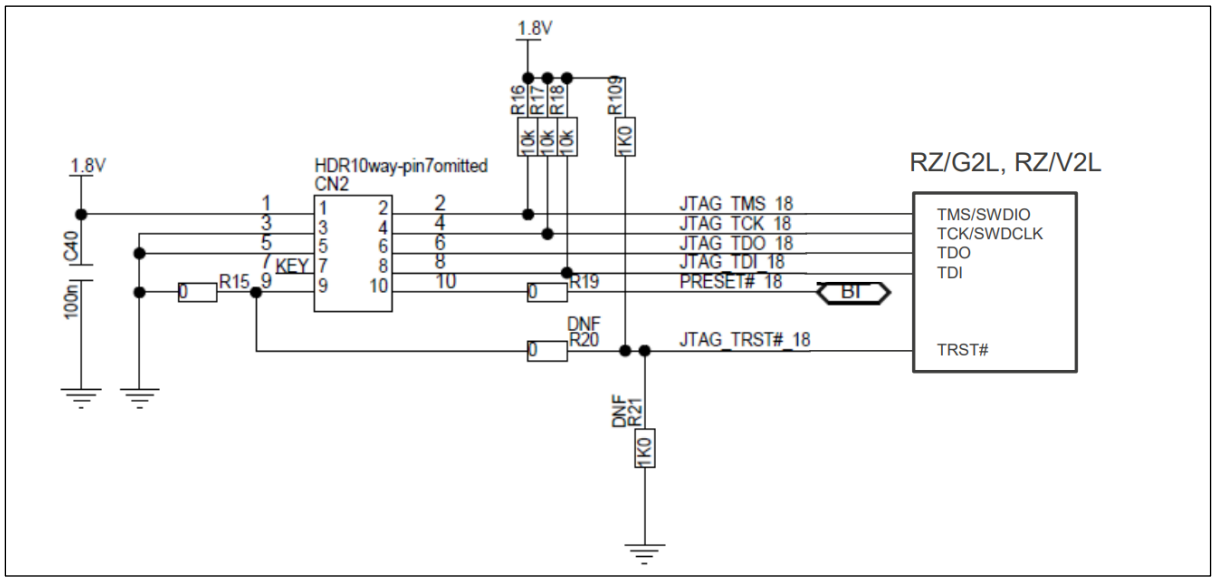


Figure 194. RZG2L-EVK Block Diagram of JTAG SWA Debug

14.4.10 Memory

The EVK's board design uses four types of memory.

1. QSPI NOR Flash
2. eMMC Flash
3. DDR4 SDRAM
4. SD-Card

14.4.10.1 QSPI NOR Flash

The QSPI flash device is managed by the integrated SPI multi-I/O bus controller (SPIBSC) in the RZ/G2L. It is capable of operating in both single data rate (SDR) and double data rate (DDR) modes, supporting transfer speeds of up to 66 MHz in SDR and 50 MHz in DDR.

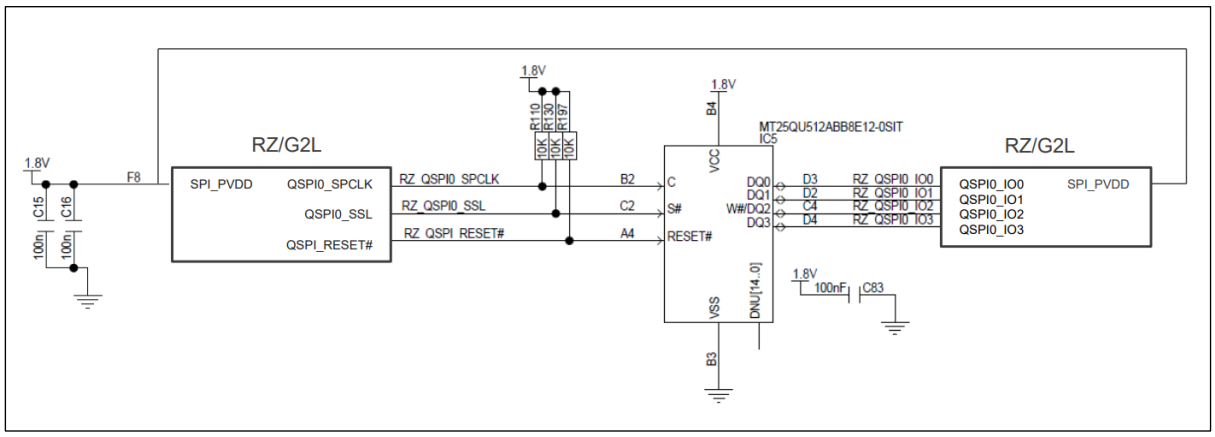


Figure 195. RZG2L-EVK Block Diagram of QSPI NOR Flash

14.4.10.2 DDR4 SDRAM

The DDR4 SDRAM is managed by the integrated DDR3L/DDR4 Memory Controller (MEMC) within the RZ/G2L. The controller supports DDR4-1600 devices with a 16-bit data bus and inline ECC capability. The interface is fully compliant with the JEDEC JESD79-4C standard.

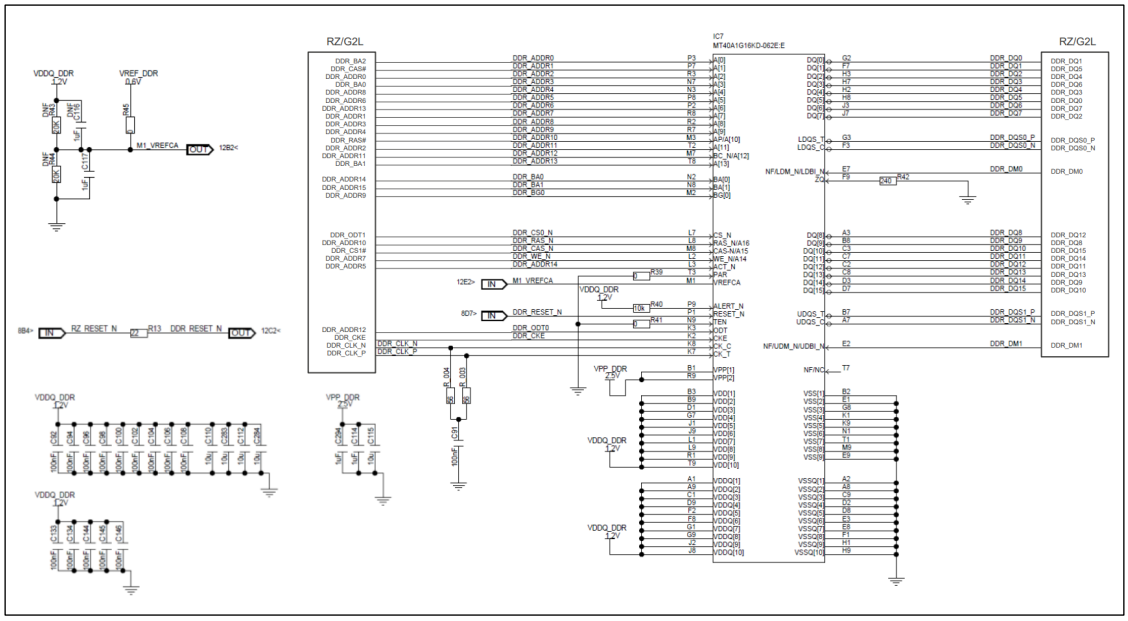


Figure 196. RZG2L-EVK Block Diagram of DDR4 SDRAM

14.4.10.3 eMMC Flash

The on-board eMMC device is connected to channel 0 of the integrated SD/MMC interface on the RZ/G2L. It operates alongside the microSD card interface as an alternative storage option. The eMMC can be selected when the SW_SD0_DEV_SEL switch is enabled (SW1-2 set to OFF, selecting SD/MMC). This interface follows the JEDEC v4.51 specification and supports HS200 transfer mode.

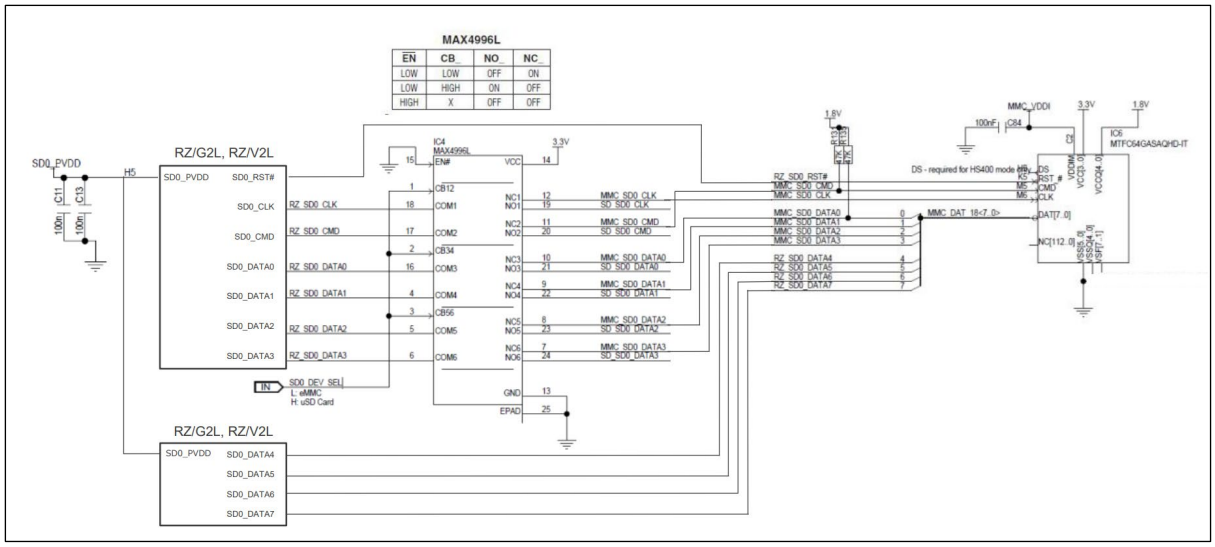


Figure 197. RZG2L-EVK eMMC Flash

14.5 RZ/V2H-EVK Board

14.5.1 Overview

Evaluation platform for the Renesas RZ/V2H, intended for functional evaluation, performance testing, and application software development. The kit follows a module-plus-carrier approach: a Secure Evaluation Board (CPU board) hosts the SoC and core functions, and an EVK Expansion Board (EXP board) adds user I/O (HDMI, audio, Pmod, etc.).

Processor

- RZ/V2H application MPU (Arm® Cortex®-A55 cluster with integrated peripheral subsystems and security features per device option).
- Suitable for vision/HMI/industrial use cases.

Memory and Storage

- System DRAM: LPDDR4X, 64 Gb × 2 on the CPU board.
- Boot/firmware: NOR flash 512 Mb.
- Removable: dual microSD card slots on the EXP board.
- eMMC (if fitted by variant): supported via SDHI.

Display and Camera

- Camera: up to 4× MIPI® CSI-2® connectors for image sensors.
- Display: HDMI® Type-A ×1 on the EXP board; MIPI-DSI via module (if supported by variant/bridge).

Connectivity

- Ethernet: 2× Gigabit Ethernet connectors (RGMII/RMII per wiring and PHY).
- USB:

  - 2× USB 3.2 Gen 2 Type-A
  - 1× USB 2.0 micro-AB
  - 1× USB 2.0 Type-A
- Serial: USB micro-B debug UART.
- I²C/SPI/GPIO: exposed on board headers and via SMARC edge interconnect.

High-Speed Expansion

- PCIe® x4 slot ×1 on the EXP board for add-in cards or accelerators.

Audio

- MIC input ×1, Headphone output ×1, AUX line input ×1 on the EXP board.
- Codec and jack population follow the kit BOM.

Power and Clocks

- Power input: USB-PD Type-C ×1 (kit default).
- PMIC: generates SoC rails (core/IO/analog) and peripheral supplies.
- Clocking: programmable clock generator provides system/peripheral references.

Debug and Expansion

- Debugger interface: 10-pin Arm® Cortex® JTAG/SWD header.
- Pmod™ connectors: 4× general-purpose expansion headers.
- SMARC edge connector exposes additional SoC interfaces for custom carriers.

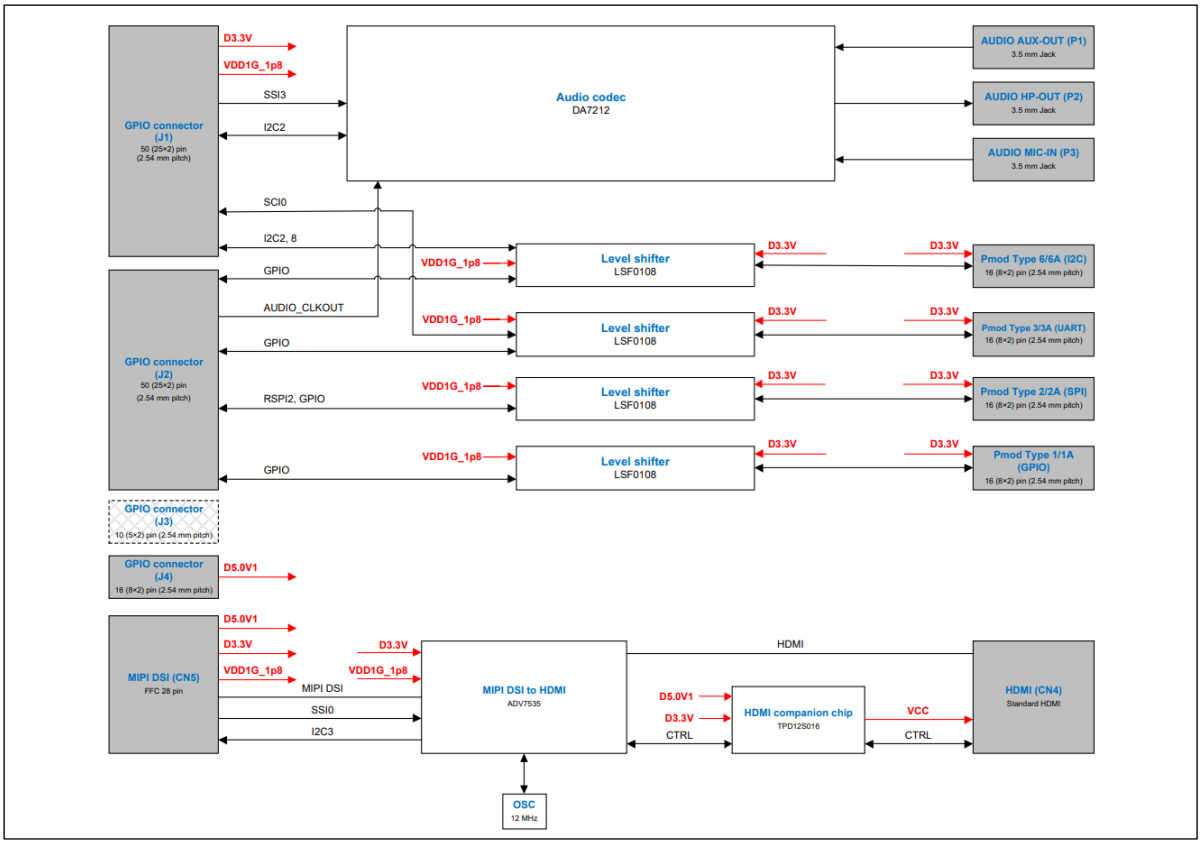


Figure 198. RZ/V2H-EVK Board Block Diagram (Source: *RZ/V2H Evaluation Board Kit (Secure Type) Hardware Manual*, p. 12)

14.5.2 Part Layout

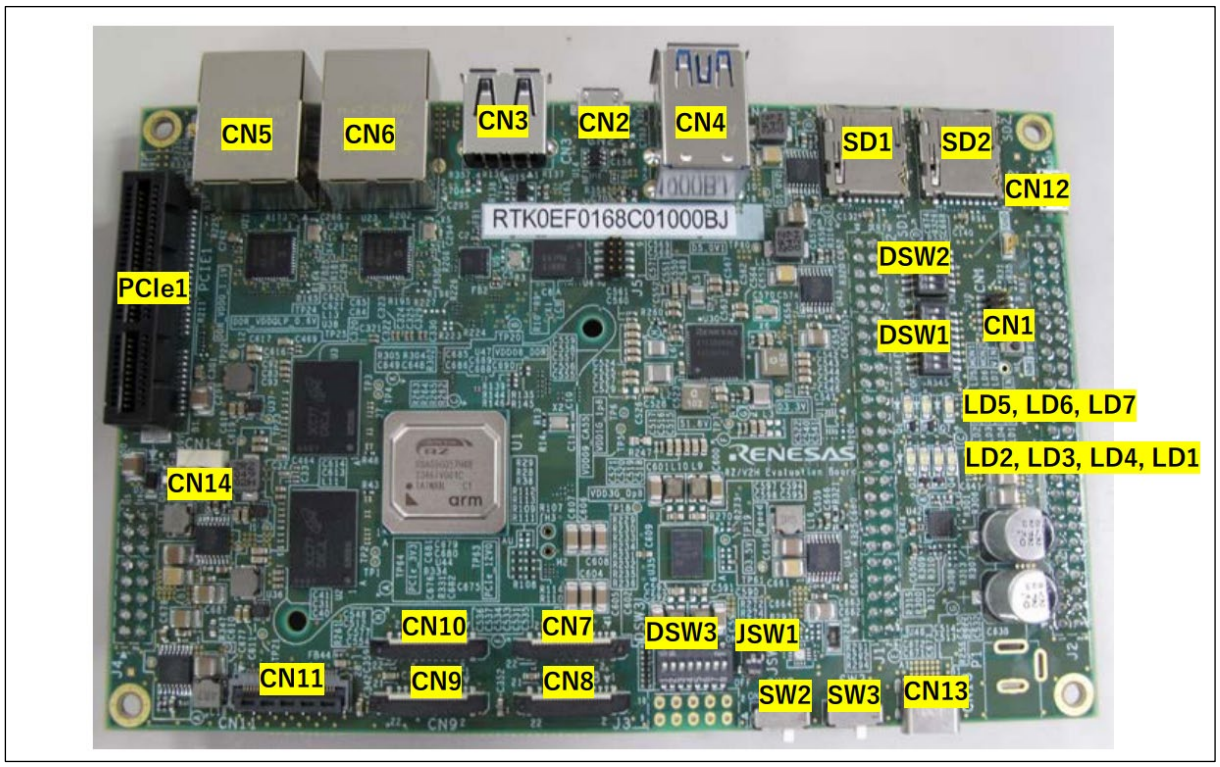


Figure 199. RZ/V2H-EVK Part Side

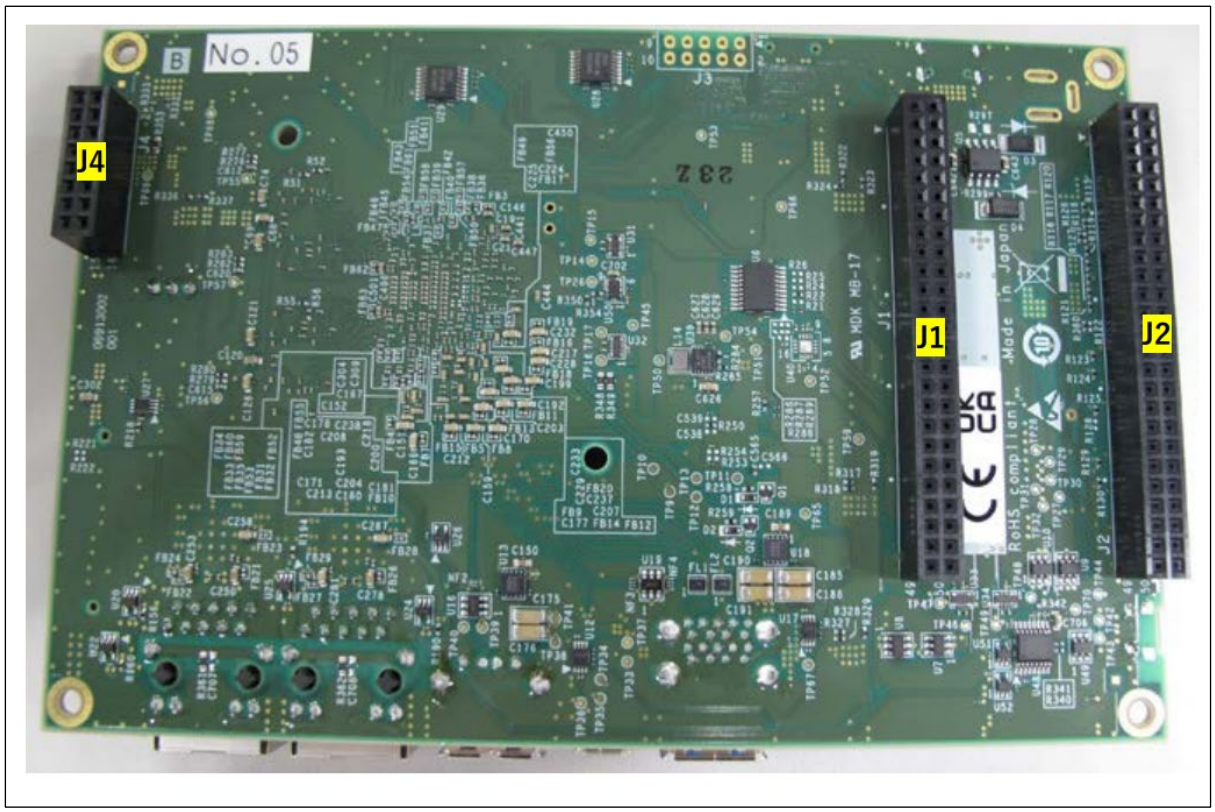


Figure 200. RZ/V2H-EVK Solder Side

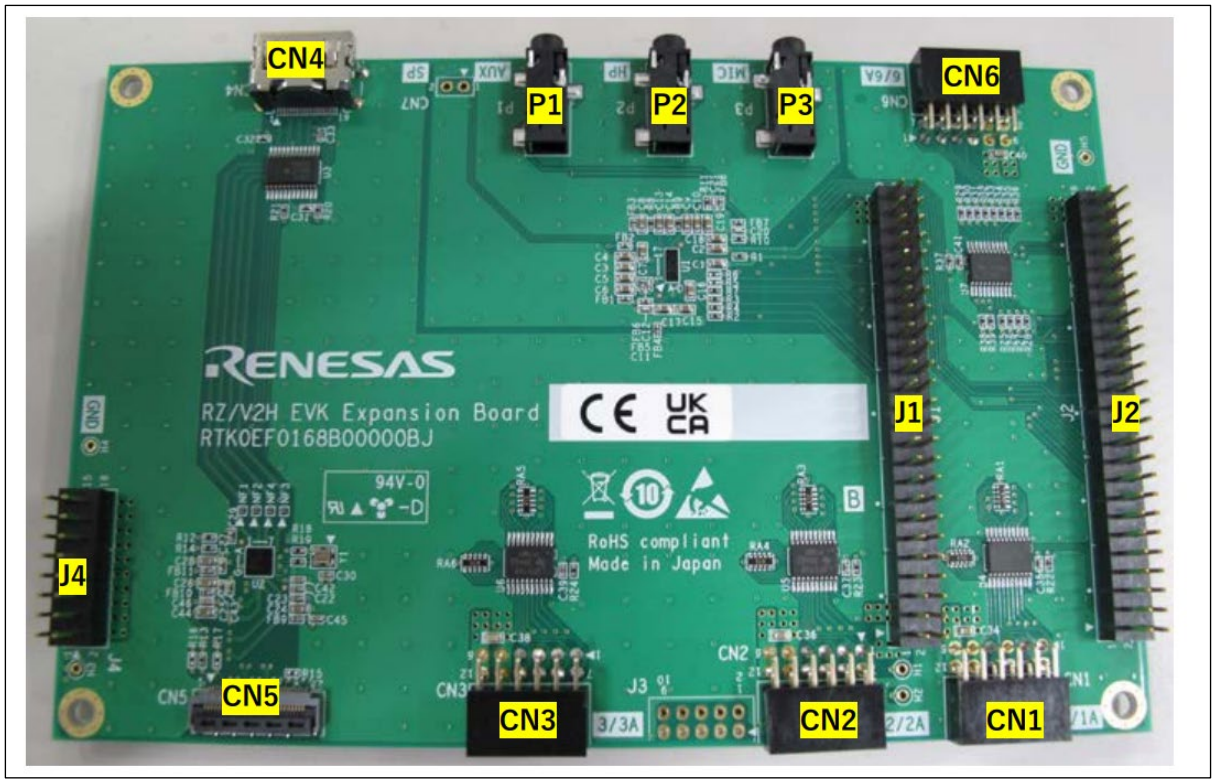


Figure 201. RZ/V2H-EVK Part Side

14.5.3 Overview of Connectors

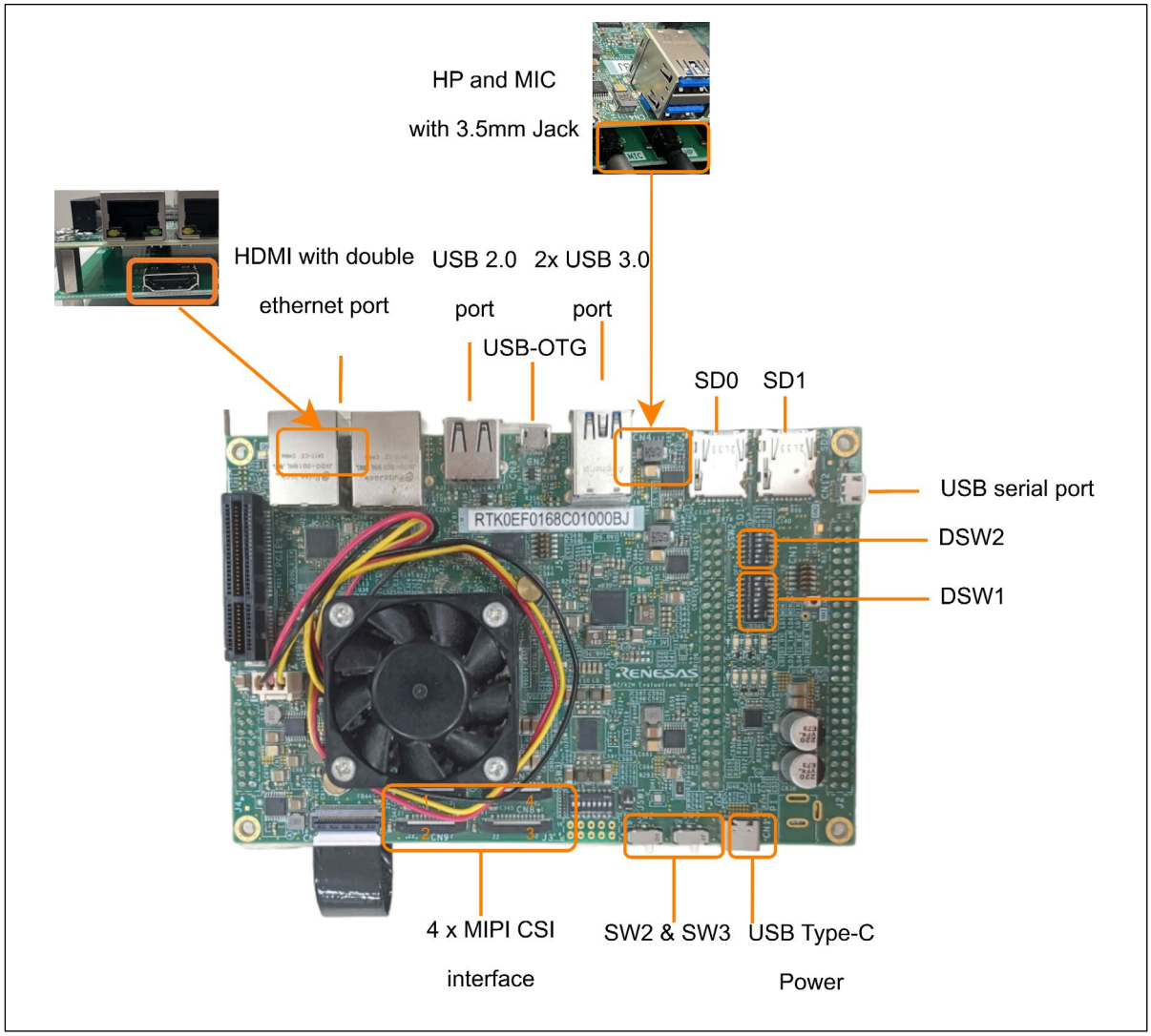


Figure 202. RZ/V2H-EVK Overview of Connectors

14.5.4 Power Supply

14.5.4.1 USB Type-C Power

The RZ/V2H-EVK is powered through a USB Type-C receptacle controlled by an on-board USB Power Delivery (PD) controller that negotiates the input profile before enabling rails. For full-feature operation (PCIe card, dual GbE, multi-camera, HDMI, multiple USB3 devices), use a USB-PD 100 W adapter (20 V × 5 A) with a 5 A e-marked USB-C cable.

14.5.4.2 Power Supply Voltage

Table 62. RZ/V2H-EVK Power Supply Voltage

Item	Description
Power supply voltage	• VDD (core): 0.8 V • VDD (CA55): 0.8 V or 0.9 V • VDD (ADC, TSU, OTP): 1.8 V • VDD (DDR IO): 1.1 V, 0.6 V (only 0.6 V: for LPDDR4X) • VDD (MIPI DPHY): 1.2 V, 1.8 V (only 1.8 V: for MIPI CSI-2) • VDD (others): 1.8 V, 3.3 V

14.5.4.3 Power Supply Regulation

The RZ/V2H-EVK board uses a dedicated Power Management IC (PMIC) to generate and sequence the supply rails required by the SoC and peripherals. See Section 14.4.5 Power Management Integrated Circuit (PMIC) for more detailed information.

14.5.5 RESET Control

Describe the power/reset sequence and the roles of the on-board slide switches and indicators on the RZ/V2H Secure Evaluation Board (CPU board) when used with the EVK Expansion Board.

Power/Reset Sequence

1. Set DIP switches to the desired boot configuration before applying power.
2. Verify the power slide switches SW2 and SW3 are OFF.
3. Connect the USB Type-C power cable to CN13 on the CPU board.
4. Turn SW3 = ON to enable USB-PD input. LD2 and LD7 should illuminate.
5. Turn SW2 = ON to enable the PMIC outputs. LD1, LD3, and LD4 illuminate when the RZ/V2H boots.

Table 63. RZ/V2H-EVK Switches Function

Switch	Name/Function	ON	OFF
SW2	PMIC (RAA215300) Enable	Enables PMIC outputs (rails sequenced and released).	Disables PMIC outputs (rails off).
SW3	USB-PD Input Enable	Enables USB-PD power input from the Type-C port	Disables USB-PD power input.

LED cues:

- LD2, LD7 = USB-PD power path active (after SW3 = ON).
- LD1, LD3, LD4 = PMIC rails enabled and system starting (after SW2 = ON).

14.5.6 Clock Configuration

The RZ/V2H integrates a Clock Pulse Generator (CPG) and a Power Management Unit (PMU) to manage all timing and power sequencing across the SoC. Together, they provide:

- Generation and distribution of system and peripheral clocks.
- Reset sequencing for reliable initialization.
- Boot-time configuration of CPU and peripheral frequencies.
- Dynamic power management, including clock gating and domain-level power control.

14.5.6.1 Clock Generating and Control Functions

The following functions are provided.

- The clocks to be supplied to various units are generated from external input clocks or PLL output clocks.
- Especially when the CA55 is cold-booted, the CA55 operating frequency by the external pin is set.
- The selection of the frequency divider ratio is implemented (the setting can be changed by the register).
- The clock path is selected by the clock selector (the setting can be changed by the register).
- Clock supply ON/OFF control (the setting can be changed by the register)
- SSCG control of PLL, multiplication ratio setting, and ON/OFF control (the setting can be changed by the register)

Table 64. PLL Table Overview

PLL	Details	Initial Frequency	SSCG Default Value
PLLCM33	For the MCPU bus and various units (PD_AWO area)	1,600 MHz	OFF (fixed)
PLLCLN	For buses other than the MCPU bus and various units	1,600 MHz	OFF (fixed)
PLLDTY	For buses other than the MCPU bus and various units	1,600 MHz	Depends on MD_CLKS
PLLCA55	CA55	1,700 MHz	Depends on MD_CLKS
PLLVDO	CRU / ISP / GE3D / CA55	1,260 MHz	Depends on MD_CLKS
PLLETH	GBETH / DRP1 / DRP-AI / DSI	1,000 MHz	OFF (fixed)
PLLDSI	DSI / LCDC	297 MHz	OFF (fixed)
PLLDDR0	For DDR ch. 0	800 MHz	OFF (fixed)
PLLDDR1	For DDR ch. 1	800 MHz	OFF (fixed)
PLLGPU	GE3D	1,260 MHz	ON (fixed)
PLLDRP	DRP1 / DRP-AI / CA55	1,260 MHz	Depends on MD_CLKS

14.5.6.2 Clock Pin Specification

The RZ/V2H device provides a wide range of dedicated clock input and output pins to support system timing, peripheral interfaces, and external device synchronization.

This table lists all clock-related pins on RZ/V2H—showing pin name, I/O direction, function, maximum supported frequency, and the supplying/receiving unit

Table 65. RZ/V2H-EVK Clock Pin Specification

Pin Name	Input	Output	Frequency*1	Supply Source/ Destination Unit
LSI Clock				
QXTAL	Output	24 MHz main clock	24 MHz	CPG
QEXTAL	Input	24 MHz main clock	24 MHz	CPG
RTXOUT	Output	32.768 kHz real-time clock	32.768 kHz	RTC
RTXIN	Input	32.768 kHz real-time clock	32.768 kHz	RTC
AUDIO_XTAL	Output	4 to 48 MHz audio clock (Internal OSC mode)	48 MHz	ADG
AUDIO_EXTAL	Input	4 to 48 MHz audio clock (Internal OSC mode) 4 to 50 MHz audio clock (Input clock from the external device)	50 MHz	ADG
AUDIO_CLKB*2	Input	4 to 50 MHz audio clock-in	50 MHz	ADG
AUDIO_CLKC*2	Input	4 to 50 MHz audio clock-in	50 MHz	ADG
AUDIO_CLKOUT*2	Output	2 to 25 MHz audio clock-out (Half frequency of input clock)	25 MHz	ADG

Debug Interface				
TCK_SWCLK	Input	Test clock pin for the on-chip emulator functions as the SWCLK pin in serial wire debug (SWD) mode	20 MHz	CST
Expanded serial peripheral interface (xSPI)				
XSPI0_CKP*3	Output	Clock output pins (positive)	133 MHz	xSPI
XSPI0_CKN*3	Output	Clock output pins (negative)	133 MHz	xSPI
SD/MMC interface				
SD0CLK	Output	Output the clock signal to the external SD/MMC device	200 MHz	SD0
SD Interface				
SD1CLK	Output	Output the clock signal to the external SD device	200 MHz	SD1
SD2CLK	Output	Output the clock signal to the external SD device	200 MHz	SD2
PCIe Gen3				
PCIE_REFCLKP0	I/O	Differential reference clock (positive), 100 MHz ±300 ppm	100 MHz	PCIE0
PCIE_REFCLKN0	I/O	Differential reference clock (negative), 100 MHz ±300 ppm	100 MHz	PCIE0
PCIE_REFCLKP1	I/O	Differential reference clock (positive), 100 MHz ±300 ppm	100 MHz	PCIE1
PCIE_REFCLKN1	I/O	Differential reference clock (negative), 100 MHz ±300 ppm	100 MHz	PCIE1
Gbit Ethernet ch. 0				
ET0_MDC	Output	Output management data clocks	2.5 MHz	GBETH0
ET0_RXC_RXCLK	Input	RX Clocks	125 MHz	GBETH0
ET0_TXC_TXCLK	I/O	TX Clocks	125 MHz	GBETH0
Gbit Ethernet ch. 1				
ET1_MDC	Output	Output management data clocks	2.5 MHz	GBETH1
ET1_RXC_RXCLK	Input	RX Clocks	125 MHz	GBETH1
ET1_TXC_TXCLK	I/O	TX Clocks	125 MHz	GBETH1
MIPI DSI				
DSI_DPCLK	Output	Output clocks (positive)	750 MHz	DSI
DSI_DNCLK	Output	Output clocks (negative)	750 MHz	DSI
MIPI CSI-2 ch. n (n = 0 to 3)				
CSIn_CLKP	Input	Input clocks (positive)	1050 MHz	CRUn
CSIn_CLKN	Input	Input clocks (negative)	1050 MHz	CRUn
Serial peripheral interface (RSPI) ch. n (n = 0, 1, 2)				
RSPCKn	I/O	Synchronous clock signal	50 MHz	RSPIn
Serial communication interface (RSCI) ch. n (n = 0 to 9)				
SCKn	I/O	Clock pins (simple SPI mode)	37.5 MHz	RSCIn
SCLn	I/O	I^2C clocks (simple I^2C mode)	400 kHz	RSCIn
I^2C bus interface (RIIC) ch. n (n = 0 to 8)				
SCLn	I/O	Clock pins with N-ch open drain	1 MHz	RIICn
I3C bus interface (I3C)				
SCL30	I/O	Clock Pin	3 MHz	I3C0
Pulse density modulation interface (PDM) ch. n (n = 0 to 5)				
PDMCLK00	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK01	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK02	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK10	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK11	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK12	Output	Output PDM sampling clocks	2.4 MHz	PDM0
Serial sound interface (SSIU) ch. n (n = 0 to 9)				
SSIn_SCK	I/O	Output clocks	12.5 MHz	SSIU

- Note 1. The clock frequencies are either variable or fixed. In the case of the variable clocks, the maximum frequencies that can be output are listed for the output clocks, and the maximum frequencies that can be input to this LSI are listed for the input clocks.

- Note 2. For the use of these pins, the function selection of multiplexed function pins is required.
- Note 3. CKP and CKN waves have opposite phases.

14.5.7 Peripheral Interface

14.5.7.1 Gigabit Ethernet

The CPU board of the V2HEVK has two Ethernet interfaces. The RJ-45 connectors (CN5 and CN6) are connected to the Ethernet interface of the RZ/V2H via the Ethernet PHY IC. Connecting this interface to a public line is prohibited.

14.5.7.2 USB 2.0 Ports

USB 2.0 Host (CH1): Type-A connector CN3 is wired to USB2.0 channel 1 on the RZ/V2H. Use this port to connect full-/high-speed USB peripherals (flash drives, keyboards, Wi-Fi dongles, etc.).

USB 2.0 DRD (CH0): Micro-AB connector CN2 is wired to USB2.0 channel 0 and operates as a Dual-Role Device (host/device) depending on cable/ID pin. Suitable for OTG use, device mode console, or host mode with an adapter.

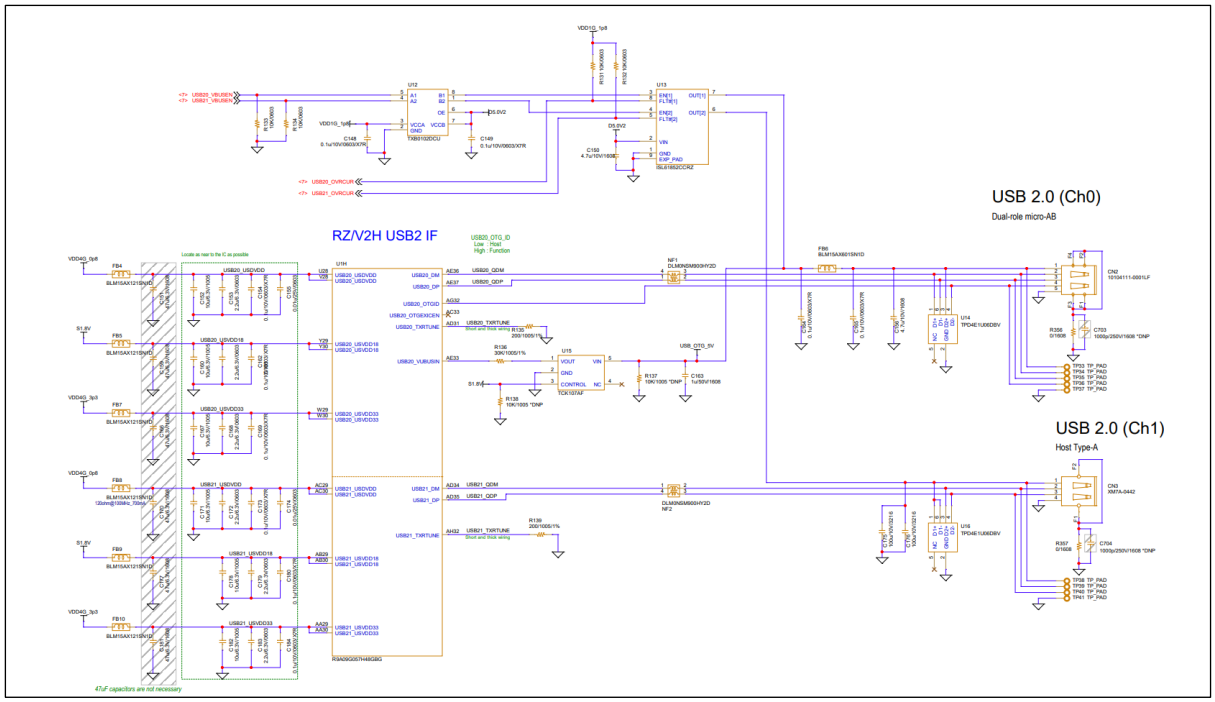


Figure 203. RZ/V2H-EVK USB 2.0 Port Schematic

14.5.7.3 USB 3.2 Ports

The RZ/V2H-EVK provides one USB 3.2 Gen2 Type-A host connector (CN4), supporting Superspeed+ (10 Gbps) signaling. This interface allows connection of high-speed external devices such as SSDs, cameras, and expansion adapters.

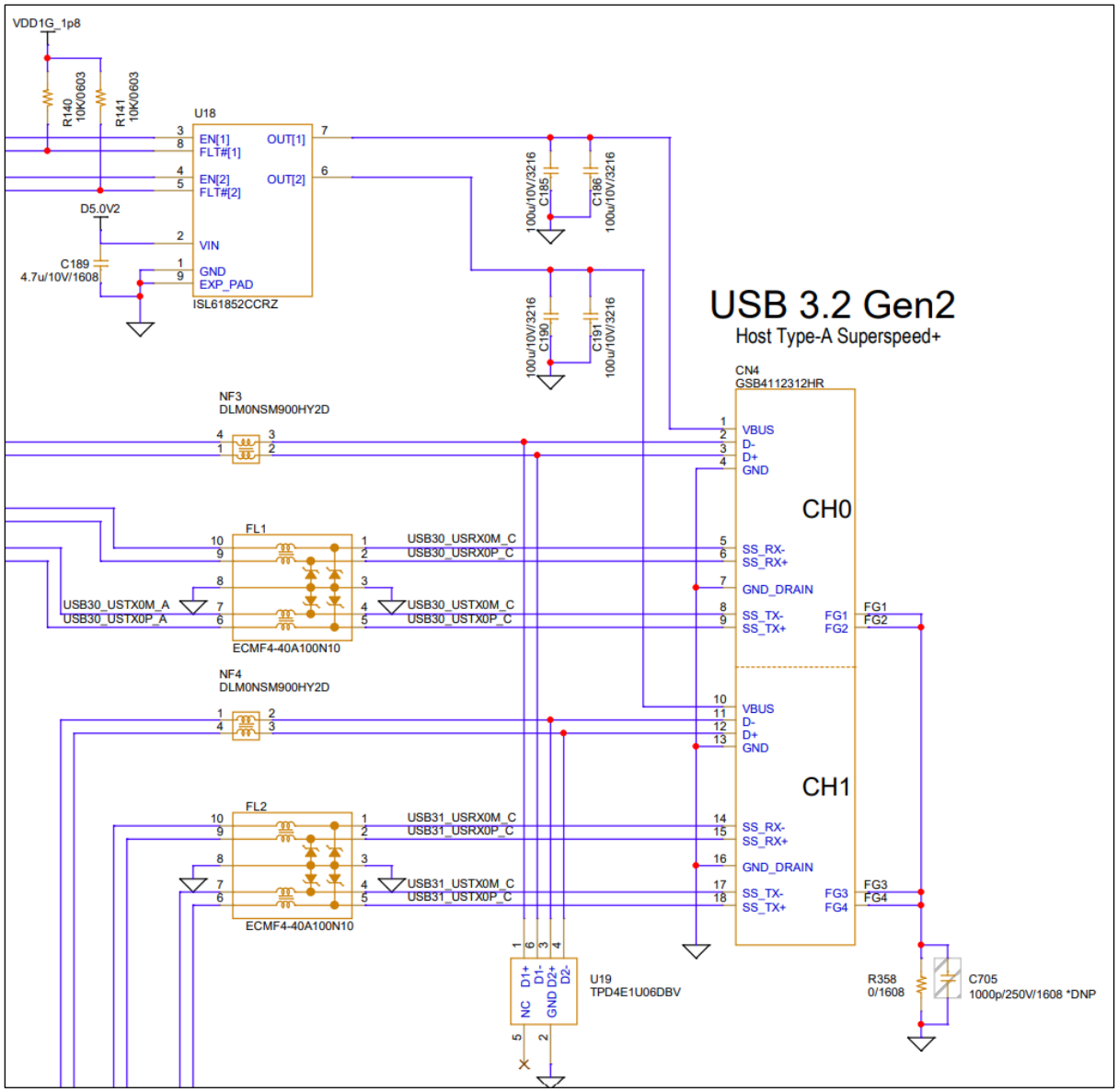


Figure 204. RZ/V2H-EVK USB 3.2 Module

14.5.7.4 MIPI CSI Interface

The RZ/V2H-EVK exposes four independent MIPI-CSI-2 ports (CH0–CH3) for image sensors. Each channel is brought to its own 22-pin camera connector (CN7/CN8/CN9/CN10) and is protected with ESD arrays and common-mode chokes. High-speed pairs are length-matched and routed as 100-Ω differential lines.

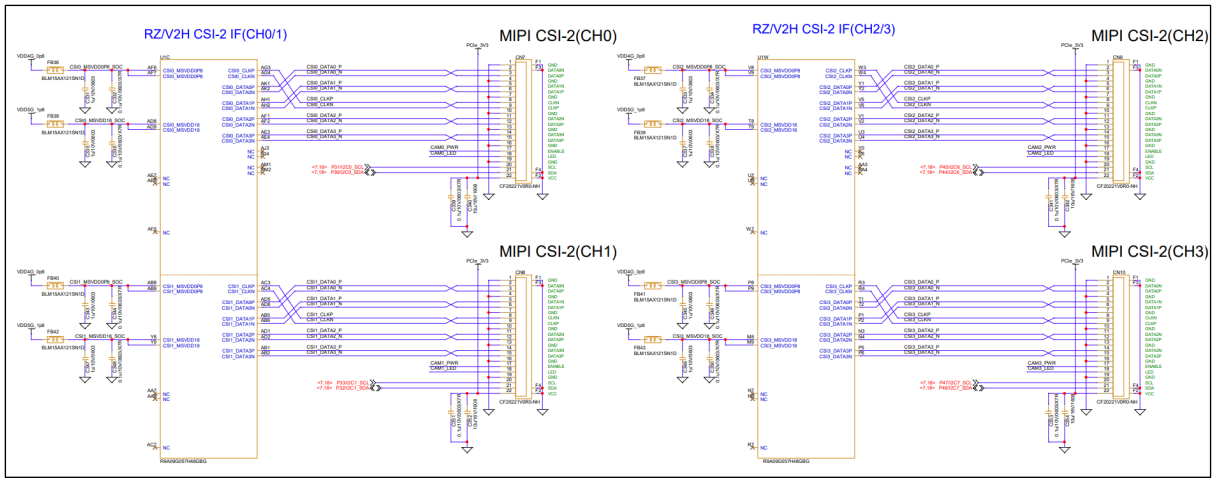


Figure 205. RZ/V2H MIPI CSI-2 Interface Schematic

14.5.7.5 Audio Subsystem with 3.5mm Jack

The EVK's audio path is codec-centric: a DA7212 codec on the EXP board connects to the SoC over I²C (control) and SSI3 (I²S data), with AUDIO_CLKx from the ADG providing the master clock. It exposes three 3.5-mm jacks—MIC-IN, HP-OUT, and AUX-OUT—and can be extended with PDM mics or S/PDIF as needed.

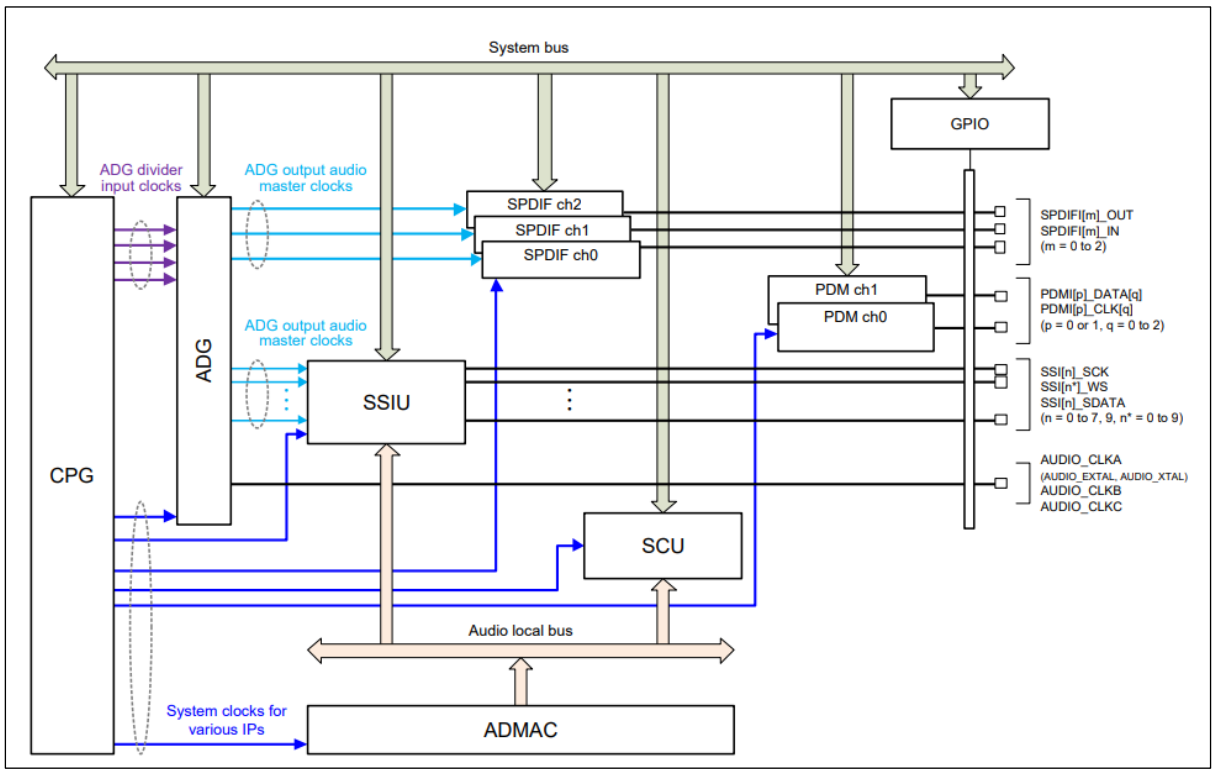


Figure 206. RZ/V2H-EVK Block Diagram of Audio Module

14.5.7.6 HDMI Display Subsystem

RZ/V2H-EVK drives HDMI through a MIPI-DSI → HDMI bridge. The SoC's MIPI-DSI interface connects to an ADV7535 bridge, which converts DSI video to TMDS (three data pairs plus one clock pair). These TMDS pairs route to the micro-HDMI Type-D connector.

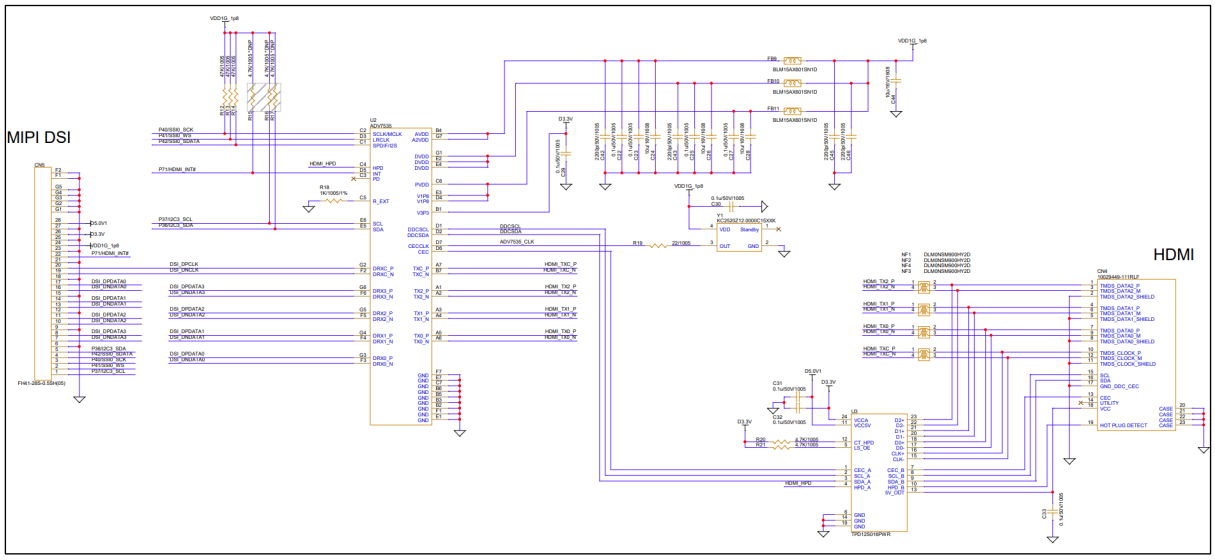


Figure 207. RZ/V2H-EVK HDMI Interface

14.5.7.7 uSD-card Interface

The interface shown below reflects RZ/V2H-EVK Rev.1: two microSD connectors

- SD1 → SDHI0 (boot-capable). This interface supports ROM boot. Insert the card before power-up/reset and select SD boot with the board's boot DIP settings. Refer to Section 16.2. Boot Mode Reference (Non-SCIF) of RZV2H-EVK for more detailed information.
- SD2 → SDHI1 for removable storage.

In EVK Rev.2, the layout changes to one microSD slot plus one on-board eMMC device (boot-capable), replacing the second microSD.

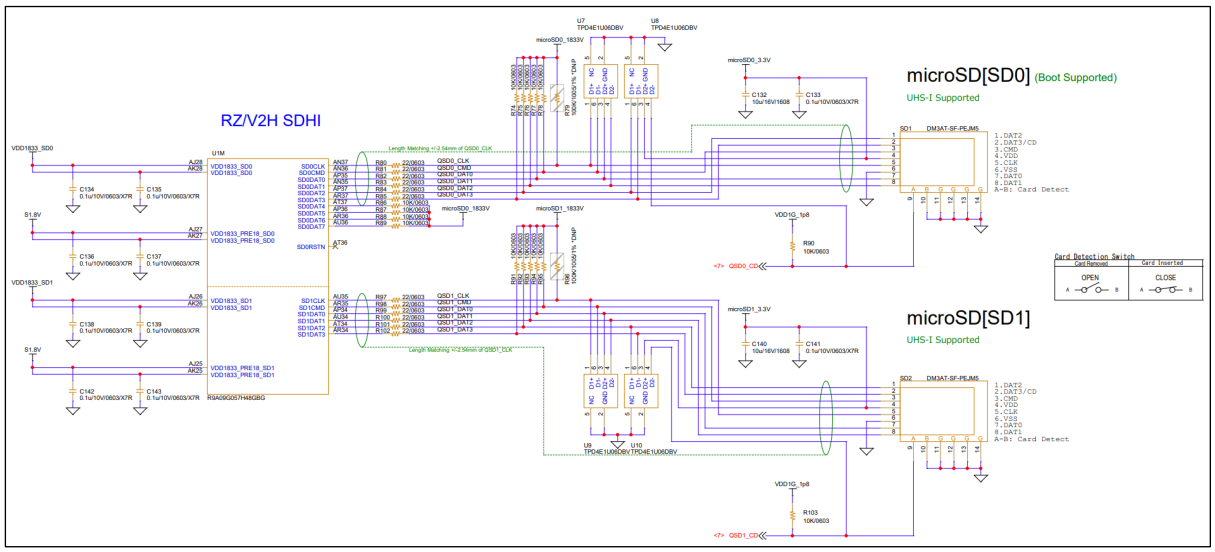


Figure 208. RZ/V2H-EVK microSD interface

14.5.7.8 PMOD Interface

The expansion board provides four PMOD headers, each translated from the SoC's 1.8 V I/O to 3.3 V through LSF0108 level shifters. All headers follow the 2×6 PMOD pin format with +3.3 V and GND on each row.

- PMOD Type-1A (GPIO) – General-purpose digital I/O. Eight bidirectional lines for LEDs, buttons, or simple logic control.
- PMOD Type-2A (SPI) – SPI host interface (SCLK, MOSI, MISO, CS) plus two auxiliary GPIOs for IRQ/RESET. Suitable for ADC/DAC, sensors, and small displays.
- PMOD Type-3A (UART) – Asynchronous serial interface (TXD, RXD, CTS, RTS) at 3.3 V levels; remaining pins available as GPIO.
- PMOD Type-6/6A (I²C) – I²C host interface (SCL, SDA) with 3.3 V pull-ups and extra pins for INT/RESET or GPIO on many modules.

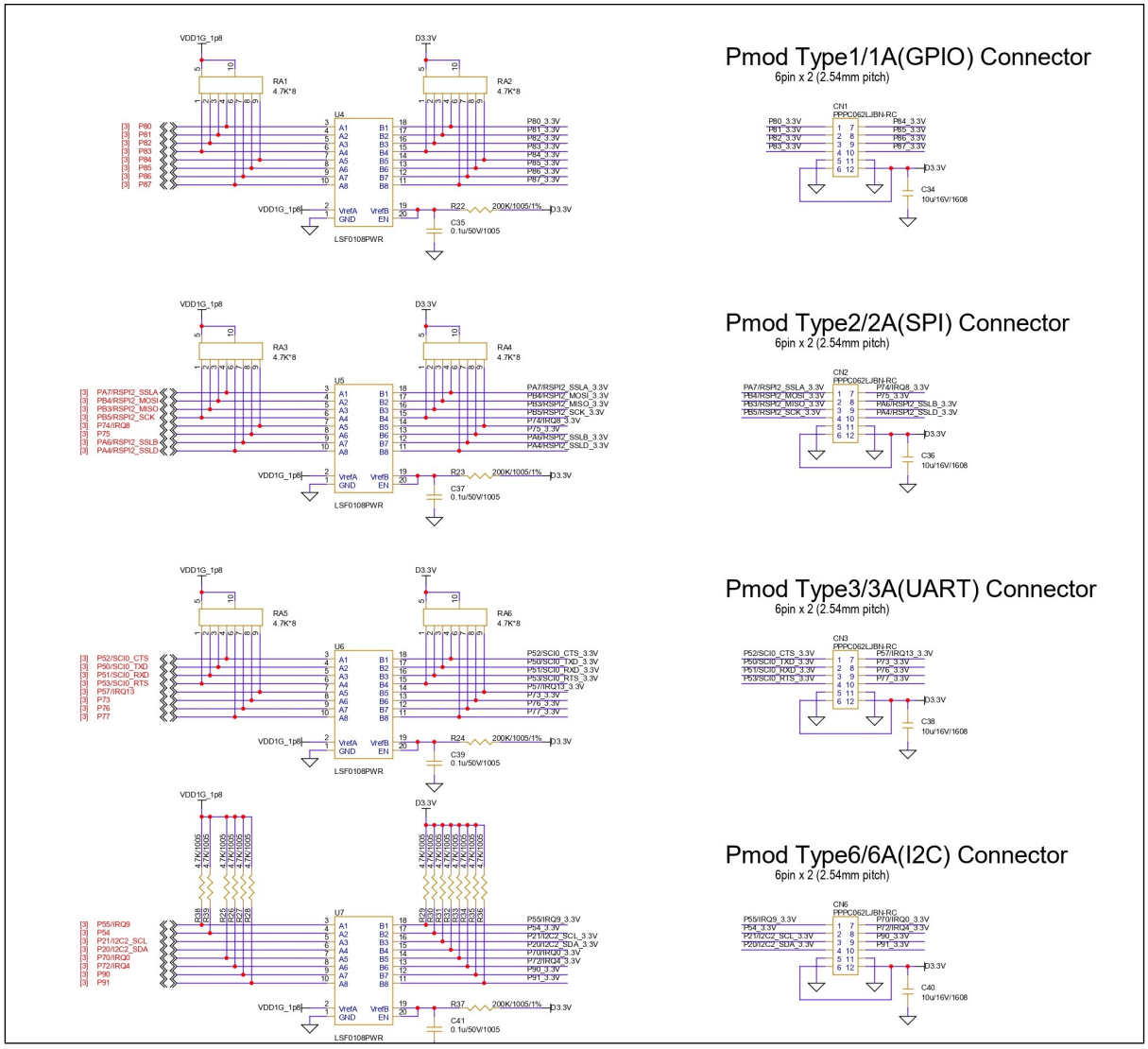


Figure 209. RZ/V2H-EVK PMOD Interfaces

14.5.7.9 Analog to Digital Converter

RZ/V2H-EVK exposes the SoC's on-chip SAR ADC for basic voltage sensing and evaluation. The CPU board routes up to 8 single-ended input channels (ANI000–ANI007).

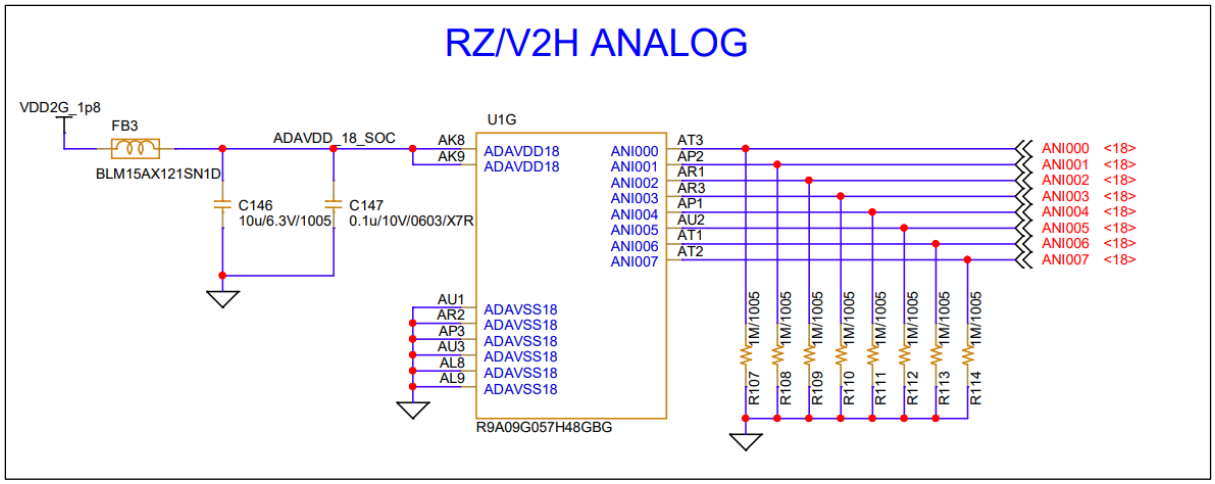


Figure 210. RZ/V2H Analog Module

14.5.7.10 Debug Interface

Connect the ICE to the Arm JTAG connector (CN1). For the ICE, having confirmed the connection, contact Renesas Electronics' sales representative.

Connector

- 10-pin, 1.27 mm (0.05") Arm Cortex Debug header (half-pitch).
- Voltage reference: 1.8 V (VDD1G_1p8 on the board). Use probes/cables that support 1.8 V I/O.

Supported modes

- SWD (2-wire): TMS/SWDIO, TCK/SWCLK, nRESET, VTref, GND.
- Full JTAG (4-wire): adds TDI and TDO.
- nTRST is brought out via NTRST_CN; it's optional and may be left open by default.

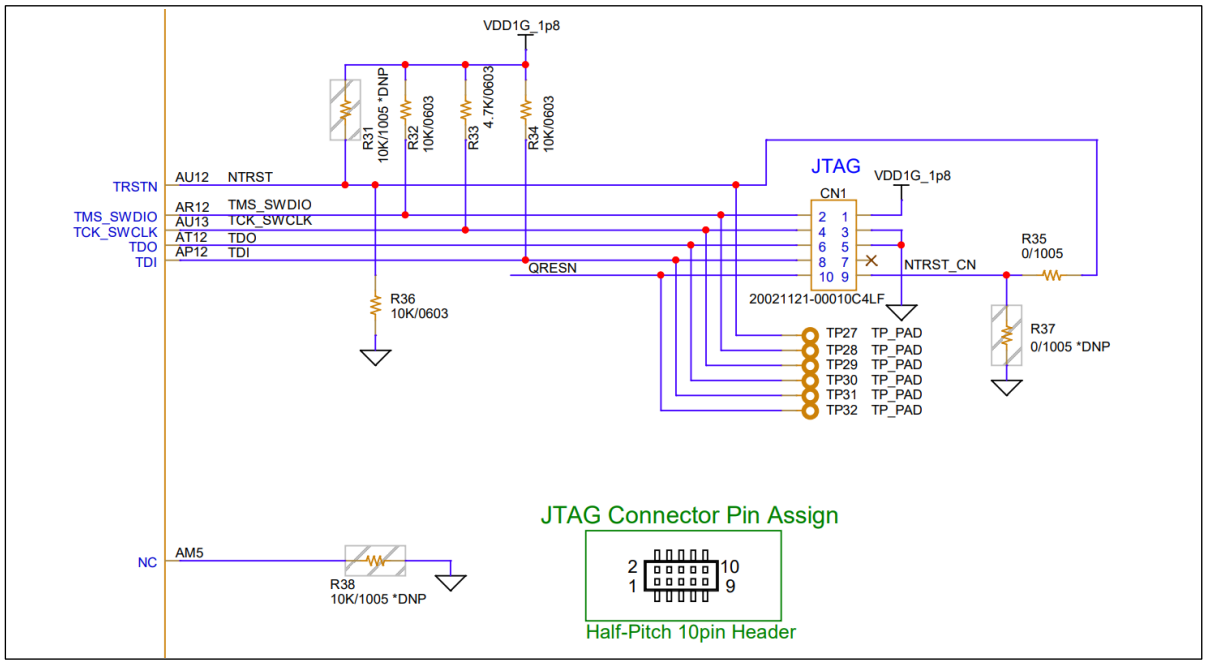


Figure 211. RZ/V2H-EVK JTAG Connector Schematic

14.5.8 Memory

14.5.8.1 xSPI Flash

The CPU board wires xSPI0 (octal/DDR-capable Serial Flash interface) to a 512-Mbit (64-MB) SPI-NOR device. The interface is powered at 1.8 V (VDD1G_1p8) and is ROM-boot supported—the SoC can fetch BL2/FIP directly from this flash.

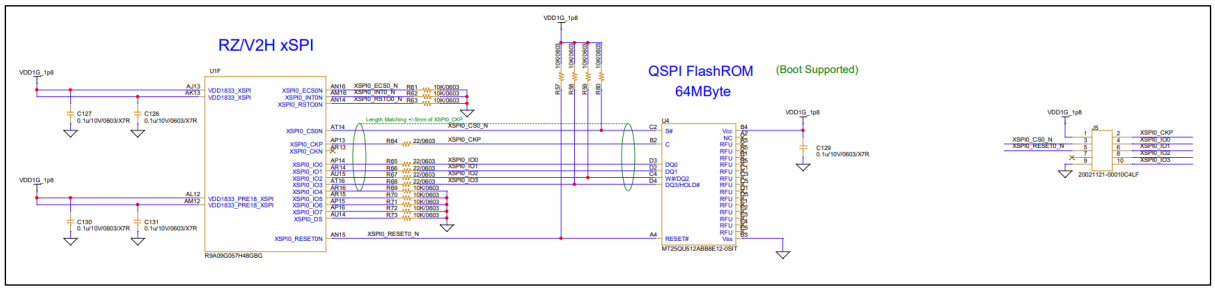


Figure 212. RZ/V2L-EVK xSPI Flash

14.5.8.2 LPDDR4

This block is the external memory subsystem for LPDDR4/LPDDR4X devices. It runs at 3200 Mb/s (1600 MHz) and uses a 32-bit data bus (two x 16-bit channels). The subsystem is split into a DDR Controller (MC) and a DDR PHY. Initialization sequences and timing tables are provided in the Linux package for this device.

Supported standards

- LPDDR4 — JEDEC JESD209-4D
- LPDDR4X — JEDEC JESD209-4-1A

DRAM interface

- Data rate: 3200 Mb/s (LPDDR4/LPDDR4X)
- Bus width: 32-bit (2 × 16-bit channels)
- Ranks: 1 or 2
- Capacity: up to 8 GB (byte-mode not supported)

Controller (MC) highlights

- Fully pipelined command/read/write paths
- Bank look-ahead to maximize throughput
- Programmable registers for timings/protocol (include auto-pre-charge)
- Full DRAM bring-up on controller reset
- Weighted round-robin arbitration across ports
- ECC: single-bit correction; single/double-bit error detect/report; selectable ECC storage (not on #AC0/#BC0)
- Built-in self-test (BIST) for external DRAM

Low-power features

- Power-down, self-refresh, and I/O retention modes
- Automatic entry/exit or software-controlled operation

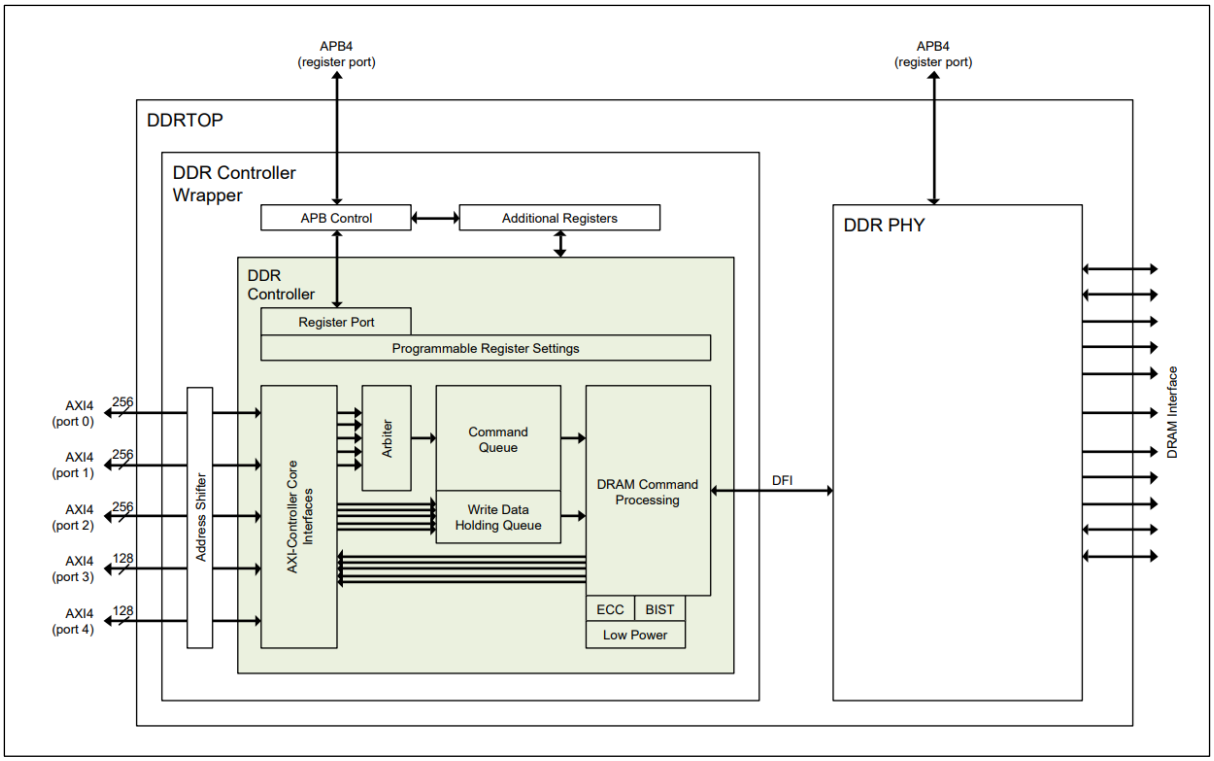


Figure 213. RZ/V2H-EVK Block Diagram of LPDDR4 (Source *RZ/V2H Group User's Manual: Hardware (Non-Agreement)*, p.278)

14.5.8.3 eMMC Flash

Support for on-board eMMC depends on the RZ/V2H-EVK hardware revision.

Board variants

- Version 1 – No eMMC fitted. The CPU board provides two microSD slots (SD1 on SDHI0, boot-capable; SD2 on SDHI1 for data).
- Version 2 – One eMMC device is populated (wired to the MMC/SDHI host), plus one microSD slot for removable media.

When eMMC is present, it may be used as the primary boot/storage device; otherwise, boot from SD1 (SDHI0).

14.6 RZ/V2H-RDK Board

14.6.1 Overview

This platform is powered by Renesas's next-generation RZ/V2H MPU for AI applications, delivering up to 8 TOPS of AI inference performance and a multi-core CPU capable of running multiple operating systems simultaneously to support high-performance AI image processing. It also provides a wide range of interfaces, making it well-suited for development and integration across various applications, especially robotics.

Processor

- RZ/V2H application MPU (Arm® Cortex®-A55 cluster with integrated peripheral subsystems and security features per device option).

- Suitable for vision/HMI/industrial use cases.

Memory and Storage

- System DRAM: LPDDR4X, 8 Gb × 2 on the CPU board.
- Boot/firmware: NOR flash 512 Mb.
- Removable: microSD card slot ×1

Display and Camera

- Camera: up to 2× MIPI® CSI-2® connectors for image sensors.
- Display: Micro HDMI® Type-D ×1 on board.

Connectivity

- Ethernet: 1× Gigabit Ethernet connector (RGMII/RMII per wiring and PHY).
- USB:
  - 2× USB 3.2 Gen 2 Type-A
- Serial: USB micro-B debug UART.
- CANFD: 2 x CANFD Transceiver port
- I^2C/SPI/GPIO: exposed on 40-pin GPIO.

High-Speed Expansion

- PCIe® 3.0 ×1 (Root Complex) (16-pin/0.5mm)

Power and Clocks

- Power input: 12V 1A AC/DC Power Supply.
- PMIC: generates SoC rails (core/IO/analog) and peripheral supplies.
- Clocking: programmable clock generator provides system/peripheral references.

Debug and Expansion

- Debugger interface: 10-pin Arm® Cortex® JTAG/SWD header.
- 40-pin GPIO header.

14.6.2 Part Layout

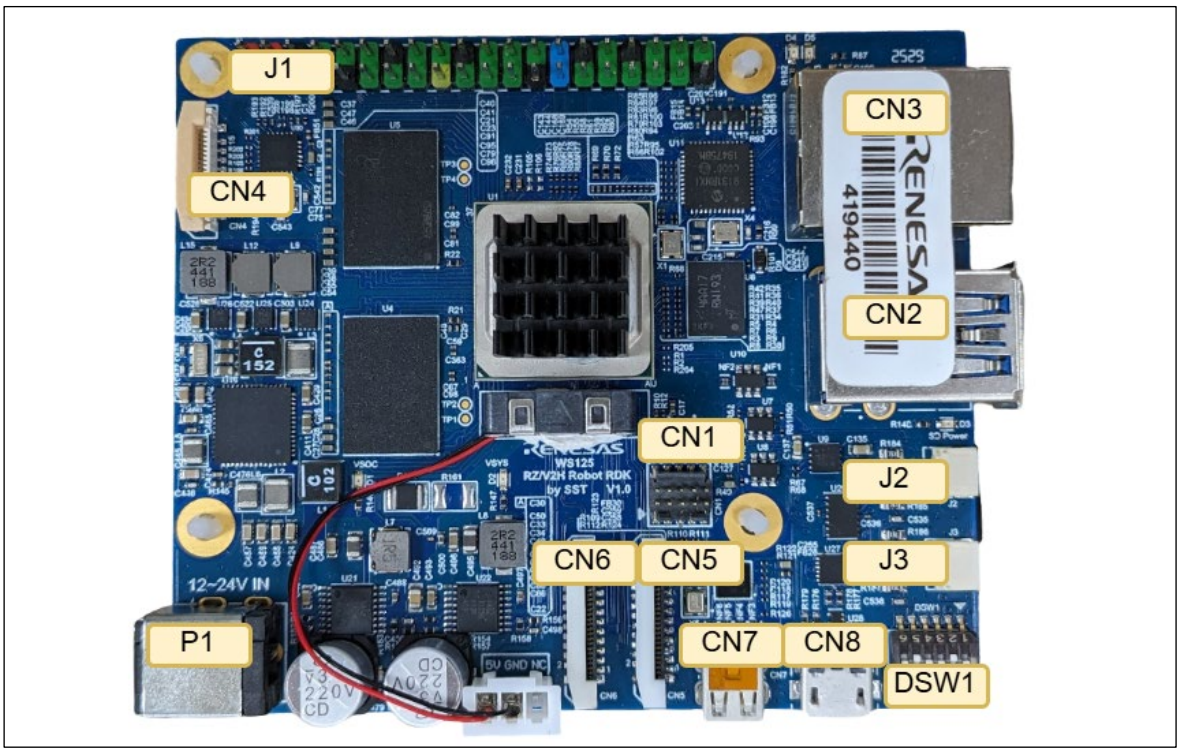


Figure 214. RZ/V2H-RDK Part Side

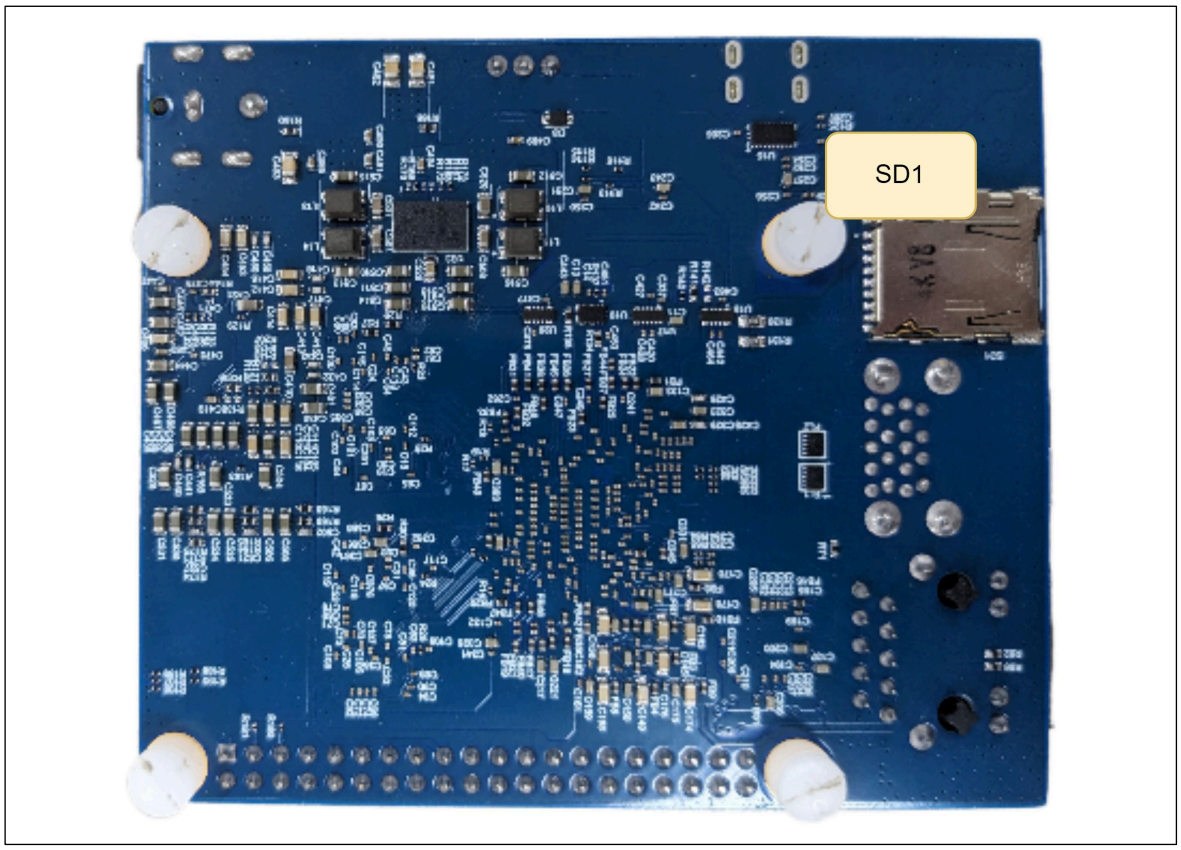


Figure 215. RZ/V2H-RDK Solder Side

14.6.3 Overview of Connectors

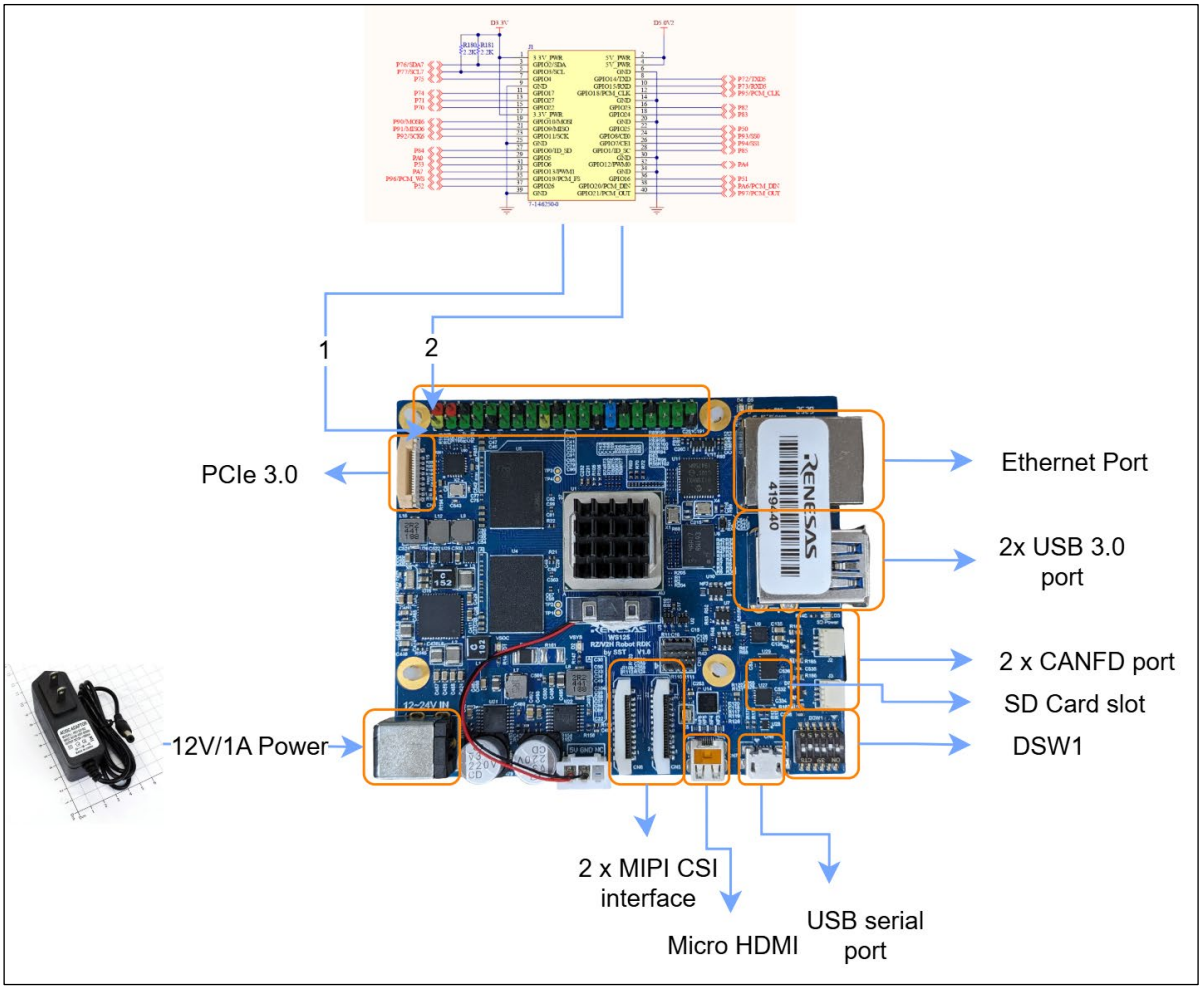


Figure 216. RZ/V2H-RDK Overview of Connectors

14.6.4 Power Supply

14.6.4.1 Power

The RZ/V2H-RDK is powered through a DC jack (5.5 mm × 2.1 mm barrel connector) using a 12V 1A power supply.

14.6.4.2 Power Supply Voltage

Table 66. RZ/V2H-RDK Power Supply Voltage

Item	Description
Power supply voltage	• VDD (core): 0.8 V • VDD (CA55): 0.8 V or 0.9 V • VDD (ADC, TSU, OTP): 1.8 V • VDD (DDR IO): 1.1 V, 0.6 V (only 0.6 V: for LPDDR4X) • VDD (MIPI DPHY): 1.2 V, 1.8 V (only 1.8 V: for MIPI CSI-2) • VDD (others): 1.8 V, 3.3 V

14.6.4.3 Power Supply Regulation

The RZ/V2H-RDK board uses a dedicated Power Management IC (PMIC) to generate and sequence the supply rails required by the SoC and peripherals. See Section 14.4.5 Power Management Integrated Circuit (PMIC) for more detailed information.

14.6.5 Power Management Integrated Circuit (PMIC)

The RAA215300 is a high-performance, low-cost, 9-channel PMIC designed for 32-bit and 64-bit MCU and MPU applications. It supports DDR3, DDR3L, DDR4, and LPDDR4 memory power interfaces. The internally compensated regulators, built-in Real-Time Clock (RTC), 32kHz crystal oscillator, and coin cell battery charger provide a highly integrated, small footprint power solution ideal for System-On-Module (SOM) applications.

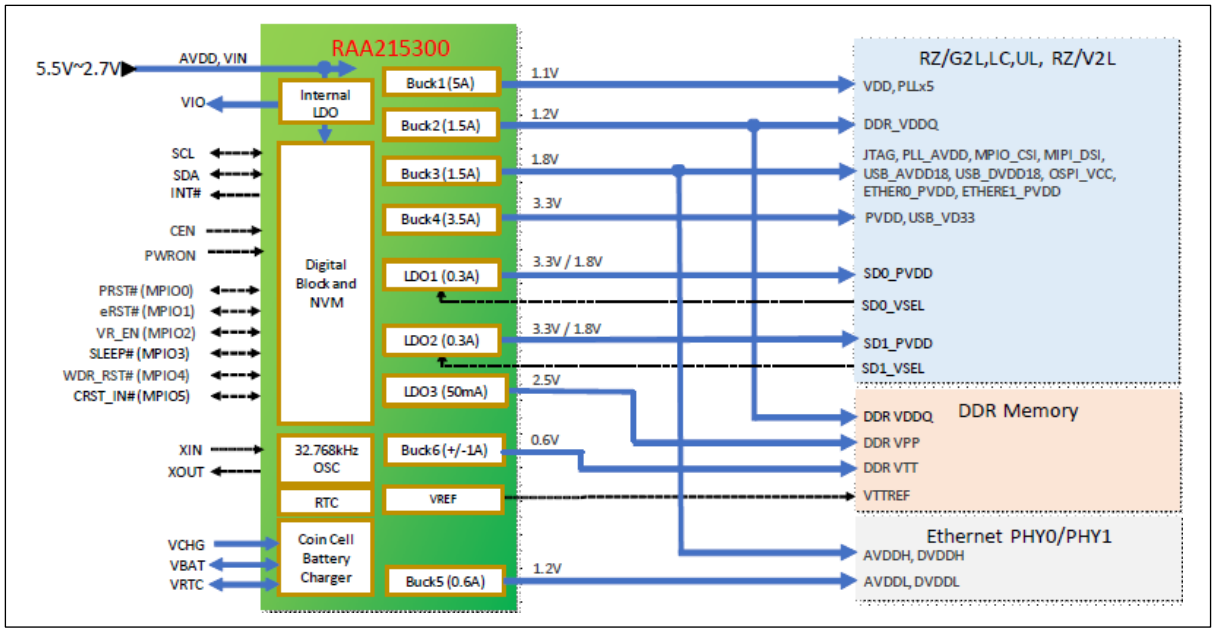


Figure 217. RAA215300 Block Diagram

14.6.6 Clock Configuration

The RZ/V2H integrates a Clock Pulse Generator (CPG) and a Power Management Unit (PMU) to manage all timing and power sequencing across the SoC. Together, they provide:

- Generation and distribution of system and peripheral clocks.
- Reset sequencing for reliable initialization.
- Boot-time configuration of CPU and peripheral frequencies.
- Dynamic power management, including clock gating and domain-level power control.

14.6.6.1 Clock Generating and Control Functions

The following functions are provided.

- The clocks to be supplied to various units are generated from external input clocks or PLL output clocks.
- Especially when the CA55 is cold-booted, the CA55 operating frequency by the external pin is set.
- The selection of the frequency divider ratio is implemented (the setting can be changed by the register).

- The clock path is selected by the clock selector (the setting can be changed by the register).
- Clock supply ON/OFF control (the setting can be changed by register)
- SSCG control of PLL, multiplication ratio setting, and ON/OFF control (the setting can be changed by the register)

Table 67. PLL Table Overview

PLL	Details	Initial Frequency	SSCG Default Value
PLLCM33	For the MCPU bus and various units (PD_AWO area)	1,600 MHz	OFF (fixed)
PLLCLN	For buses other than the MCPU bus and various units	1,600 MHz	OFF (fixed)
PLLDTY	For buses other than the MCPU bus and various units	1,600 MHz	Depends on MD_CLKS
PLLCA55	CA55	1,700 MHz	Depends on MD_CLKS
PLLVDO	CRU / ISP / GE3D / CA55	1,260 MHz	Depends on MD_CLKS
PLLETH	GBETH / DRP1 / DRP-AI / DSI	1,000 MHz	OFF (fixed)
PLLDSI	DSI / LCDC	297 MHz	OFF (fixed)
PLLDDR0	For DDR ch. 0	800 MHz	OFF (fixed)
PLLDDR1	For DDR ch. 1	800 MHz	OFF (fixed)
PLLGPU	GE3D	1,260 MHz	ON (fixed)
PLLDRP	DRP1 / DRP-AI / CA55	1,260 MHz	Depends on MD_CLKS

14.6.6.2 Clock Pin Specification

The RZ/V2H device provides a wide range of dedicated clock input and output pins to support system timing, peripheral interfaces, and external device synchronization.

This table lists all clock-related pins on RZ/V2H—showing pin name, I/O direction, function, maximum supported frequency, and the supplying/receiving unit

Table 68. RZ/V2H-RDK Clock Pin Specification

Pin Name	Input	Output	Frequency*1	Supply Source/ Destination Unit
LSI Clock				
QXTAL	Output	24 MHz main clock	24 MHz	CPG
QEXTAL	Input	24 MHz main clock	24 MHz	CPG
RTXOUT	Output	32.768 kHz real-time clock	32.768 kHz	RTC
RTXIN	Input	32.768 kHz real-time clock	32.768 kHz	RTC
AUDIO_XTAL	Output	4 to 48 MHz audio clock (Internal OSC mode)	48 MHz	ADG
AUDIO_EXTAL	Input	4 to 48 MHz audio clock (Internal OSC mode) 4 to 50 MHz audio clock (Input clock from the external device)	50 MHz	ADG
AUDIO_CLKB*2	Input	4 to 50 MHz audio clock-in	50 MHz	ADG
AUDIO_CLKC*2	Input	4 to 50 MHz audio clock-in	50 MHz	ADG
AUDIO_CLKOUT*2	Output	2 to 25 MHz audio clock-out (Half frequency of input clock)	25 MHz	ADG
Debug Interface				
TCK_SWCLK	Input	Test clock pin for the on-chip emulator functions as the SWCLK pin in serial wire debug (SWD) mode	20 MHz	CST

Expanded serial peripheral interface (xSPI)				
XSPI0_CKP*3	Output	Clock output pins (positive)	133 MHz	xSPI
XSPI0_CKN*3	Output	Clock output pins (negative)	133 MHz	xSPI
SD/MMC interface				
SD0CLK	Output	Output the clock signal to the external SD/MMC device	200 MHz	SD0
SD Interface				
SD1CLK	Output	Output the clock signal to the external SD device	200 MHz	SD1
SD2CLK	Output	Output the clock signal to the external SD device	200 MHz	SD2
PCIe Gen3				
PCIE_REFCLKP0	I/O	Differential reference clock (positive), 100 MHz ±300 ppm	100 MHz	PCIE0
PCIE_REFCLKN0	I/O	Differential reference clock (negative), 100 MHz ±300 ppm	100 MHz	PCIE0
PCIE_REFCLKP1	I/O	Differential reference clock (positive), 100 MHz ±300 ppm	100 MHz	PCIE1
PCIE_REFCLKN1	I/O	Differential reference clock (negative), 100 MHz ±300 ppm	100 MHz	PCIE1
Gbit Ethernet ch. 0				
ET0_MDC	Output	Output management data clocks	2.5 MHz	GBETH0
ET0_RXC_RXCLK	Input	RX Clocks	125 MHz	GBETH0
ET0_TXC_TXCLK	I/O	TX Clocks	125 MHz	GBETH0
Gbit Ethernet ch. 1				
ET1_MDC	Output	Output management data clocks	2.5 MHz	GBETH1
ET1_RXC_RXCLK	Input	RX Clocks	125 MHz	GBETH1
ET1_TXC_TXCLK	I/O	TX Clocks	125 MHz	GBETH1
MIPI DSI				
DSI_DPCLK	Output	Output clocks (positive)	750 MHz	DSI
DSI_DNCLK	Output	Output clocks (negative)	750 MHz	DSI
MIPI CSI-2 ch. n (n = 0 to 3)				
CSIn_CLKP	Input	Input clocks (positive)	1050 MHz	CRUn
CSIn_CLKN	Input	Input clocks (negative)	1050 MHz	CRUn
Serial peripheral interface (RSPI) ch. n (n = 0, 1, 2)				
RSPCKn	I/O	Synchronous clock signal	50 MHz	RSPIn
Serial communication interface (RSCI) ch. n (n = 0 to 9)				
SCKn	I/O	Clock pins (simple SPI mode)	37.5 MHz	RSCIn
SCLn	I/O	I^2C clocks (simple I^2C mode)	400 kHz	RSCIn
I^2C bus interface (RIIC) ch. n (n = 0 to 8)				
SCLn	I/O	Clock pins with N-ch open drain	1 MHz	RIICn
I3C bus interface (I3C)				
SCL30	I/O	Clock Pin	3 MHz	I3C0
Pulse density modulation interface (PDM) ch. n (n = 0 to 5)				
PDMCLK00	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK01	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK02	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK10	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK11	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK12	Output	Output PDM sampling clocks	2.4 MHz	PDM0
Serial sound interface (SSIU) ch. n (n = 0 to 9)				
SSIn_SCK	I/O	Output clocks	12.5 MHz	SSIU

- Note 1. The clock frequencies are either variable or fixed. In the case of the variable clocks, the maximum frequencies that can be output are listed for the output clocks and the maximum frequencies that can be input to this LSI are listed for the input clocks.
- Note 2. For the use of these pins, the function selection of multiplexed function pins is required.
- Note 3. CKP and CKN waves have opposite phases.

14.6.7 Peripheral Interface

14.6.7.1 Gigabit Ethernet

The RZV2H-RDK has one Ethernet interface. The RJ-45 connector (CN3) is connected to the Ethernet interface of the RZ/V2H via the Ethernet PHY IC.

14.6.7.2 USB 3.2 Ports

The RZ/V2H-RDK provides one USB 3.2 Gen2 Type-A host connector (CN4), supporting Superspeed+ (10 Gbps) signaling. This interface allows connection of high-speed external devices such as SSDs, cameras, and expansion adapters.

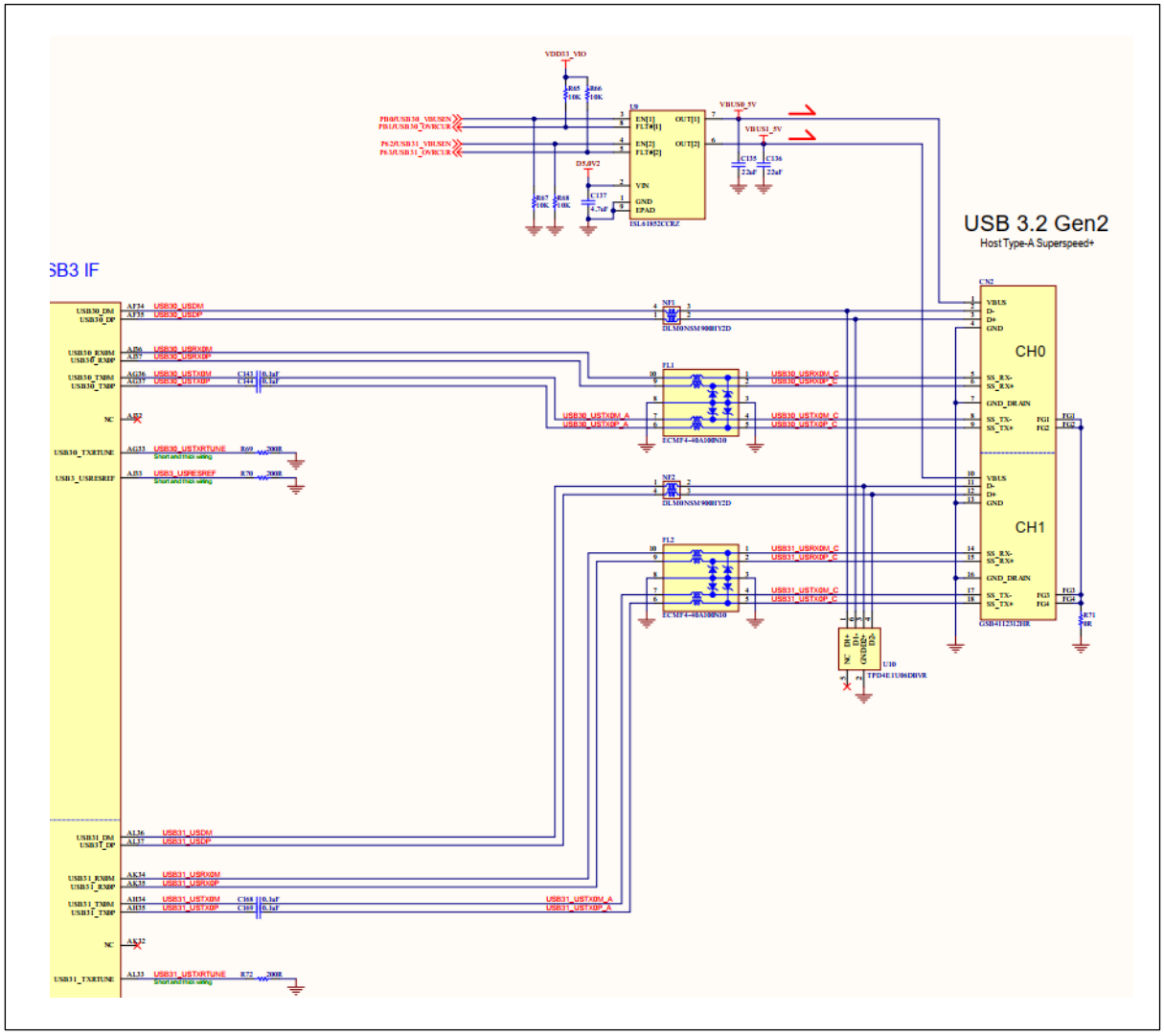


Figure 218. RZ/V2H-RDK USB 3.2 Module

14.6.7.3 MIPI CSI Interface

The RZ/V2H-RDK exposes two independent MIPI-CSI-2 ports (CH0–CH1) for image sensors. Each channel is brought to its own 22-pin camera connector (CN5/CN6) and is protected with ESD arrays and common-mode chokes. High-speed pairs are length-matched and routed as 100-Ω differential lines.

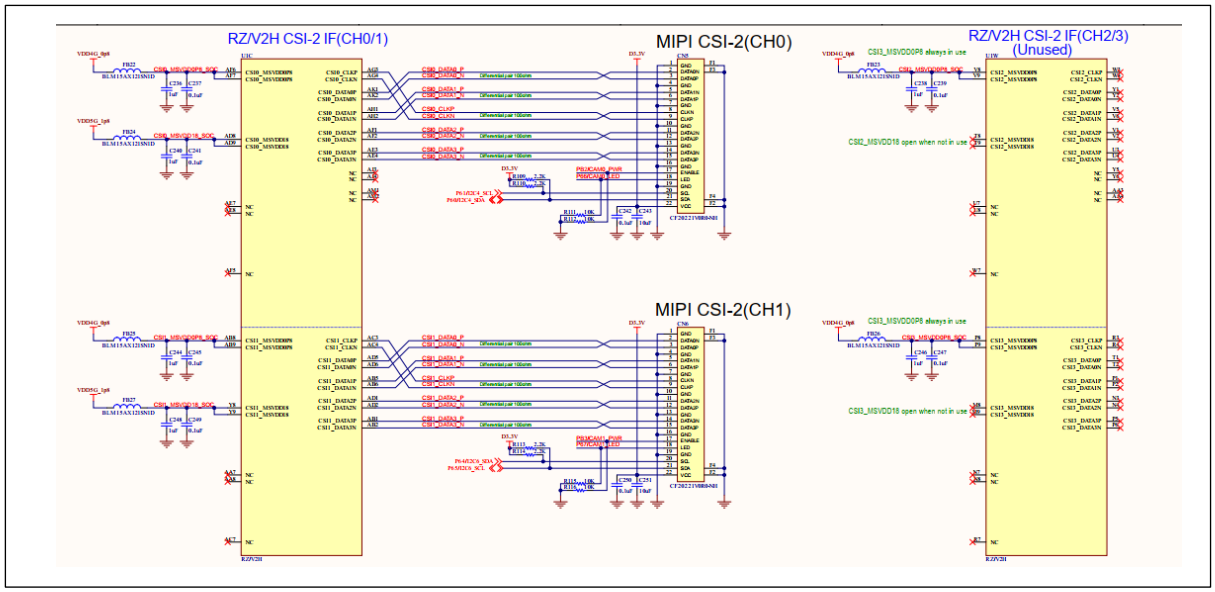


Figure 219. RZ/V2H-RDK MIPI CSI-2 Interface Schematic

14.6.7.4 CANFD Interface

RZV2H-RDK provides two CANFD interfaces through TCAN1046VDMTRQ1 transceivers. The SoC's CANFD controller connects to the TCAN1046VDMTRQ1 transceiver, which converts differential CAN signals to twisted-pair wiring.

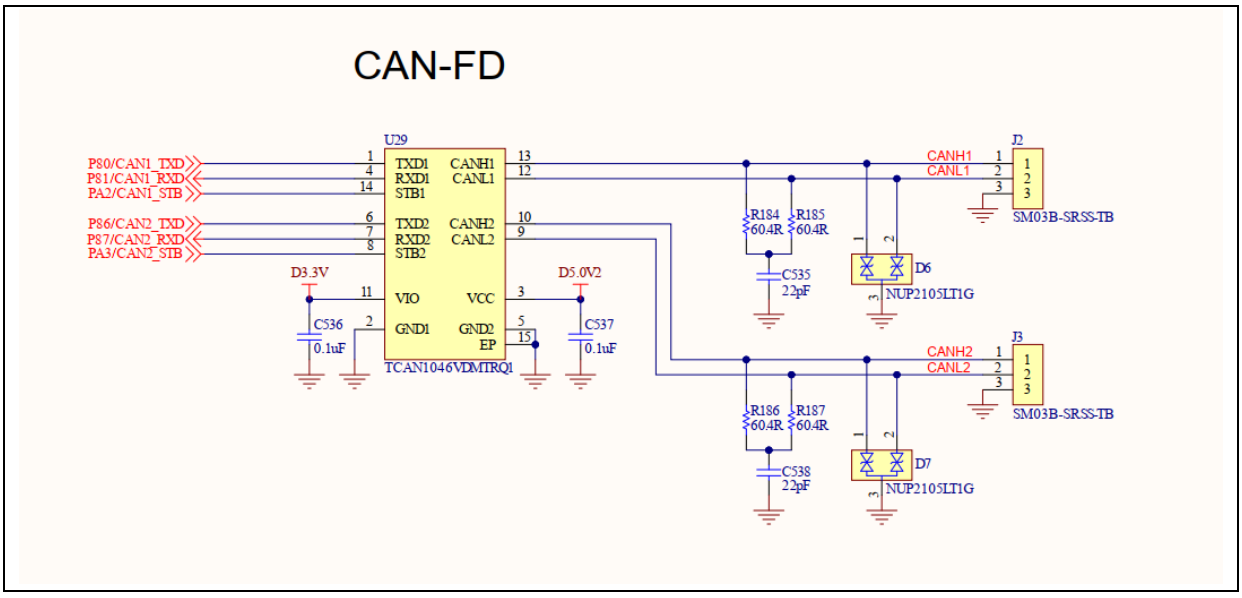


Figure 220. RZ/V2H-RDK CAN FD Interface Schematic

14.6.7.5 HDMI Display Subsystem

RZ/V2H-EVK drives HDMI through a MIPI-DSI → HDMI bridge. The SoC's MIPI-DSI interface connects to an ADV7535 bridge, which converts DSI video to TMDS (three data pairs plus one clock pair). These TMDS pairs route to the micro-HDMI Type-D connector.

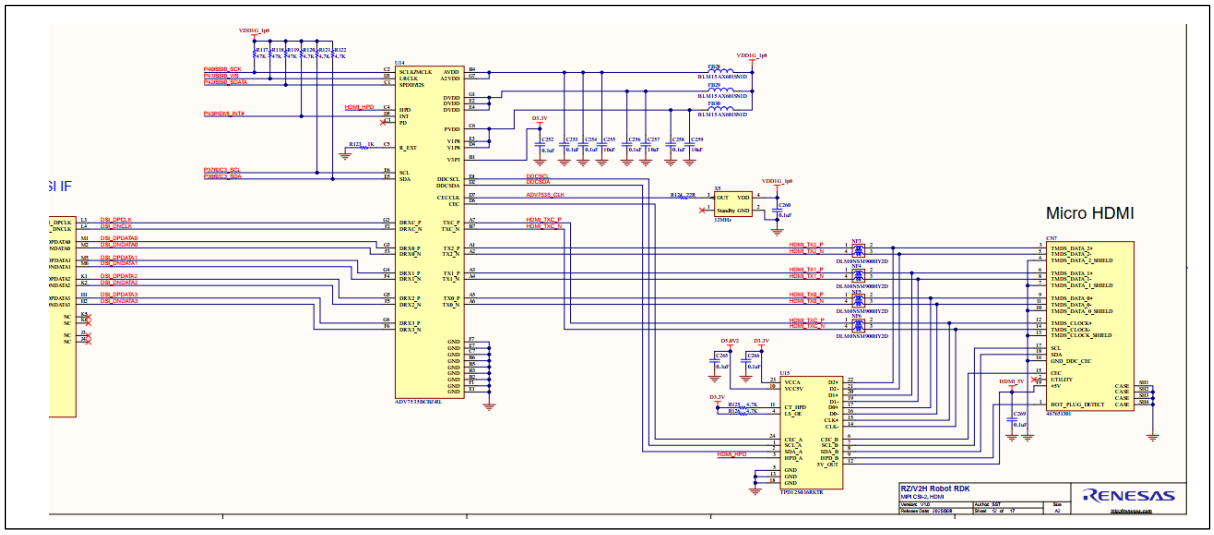


Figure 221. RZ/V2H-RDK Micro-HDMI Interface

14.6.7.6 uSD-card Interface

The interface shown below reflects RZ/V2H-RDK Rev.1: one microSD connector

- SD1 → SDHI0 (boot-capable). This interface supports ROM boot. Insert the card before power-up/reset and select SD boot with the board's boot DIP settings. Refer to Section 16.2. Boot Mode Reference (Non-SCIF) of RZV2H-RDK for more detailed information.

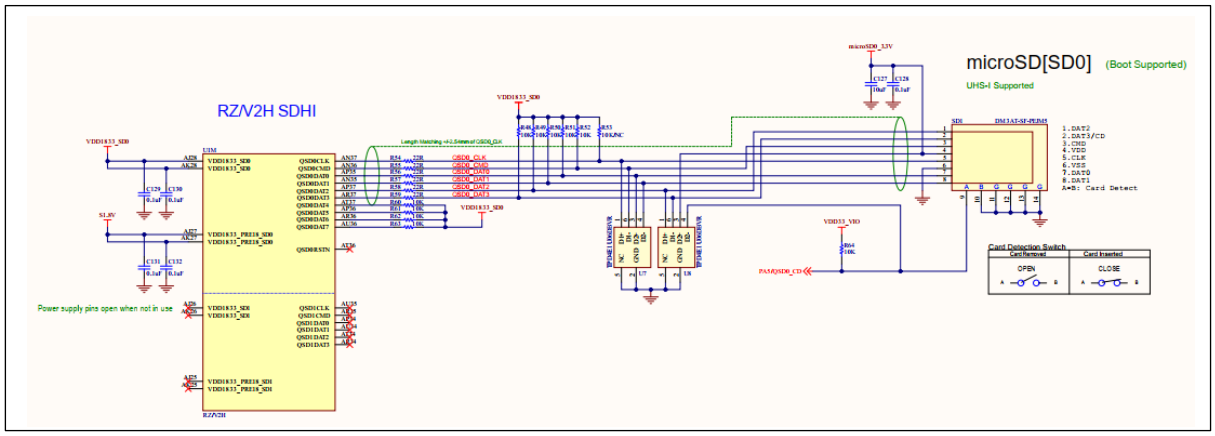


Figure 222. RZ/V2H-RDK microSD Interface

14.6.7.7 Debug Interface

Connect the ICE to the Arm JTAG connector (CN1). For the ICE to confirm the connection, contact Renesas Electronics' sales representative.

Connector

- 10-pin, 1.27 mm (0.05") Arm Cortex Debug header (half-pitch).
- Voltage reference: 1.8 V (VDD1G_1p8 on the board). Use probes/cables that support 1.8 V I/O.

Supported modes

- SWD (2-wire): TMS/SWDIO, TCK/SWCLK, nRESET, VTref, GND.
- Full JTAG (4-wire): adds TDI and TDO.
- nTRST is brought out via NTRST_CN; it's optional and may be left open by default.

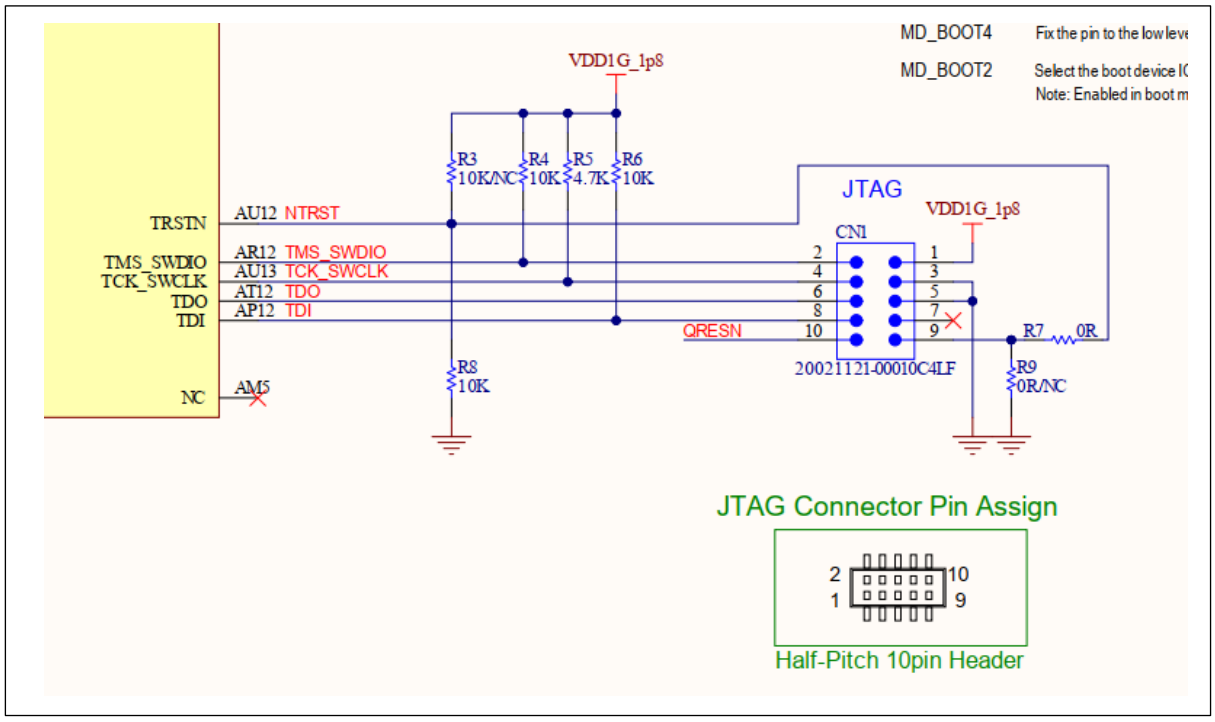


Figure 223. RZ/V2H-RDK JTAG Connector Schematic

14.6.8 Memory

14.6.8.1 xSPI Flash

The CPU board wires xSPI0 (octal/DDR-capable Serial Flash interface) to a 512-Mbit (64-MB) SPI-NOR device. The interface is powered at 1.8 V (VDD1G_1p8) and is ROM-boot supported—the SoC can fetch BL2/FIP directly from this flash.

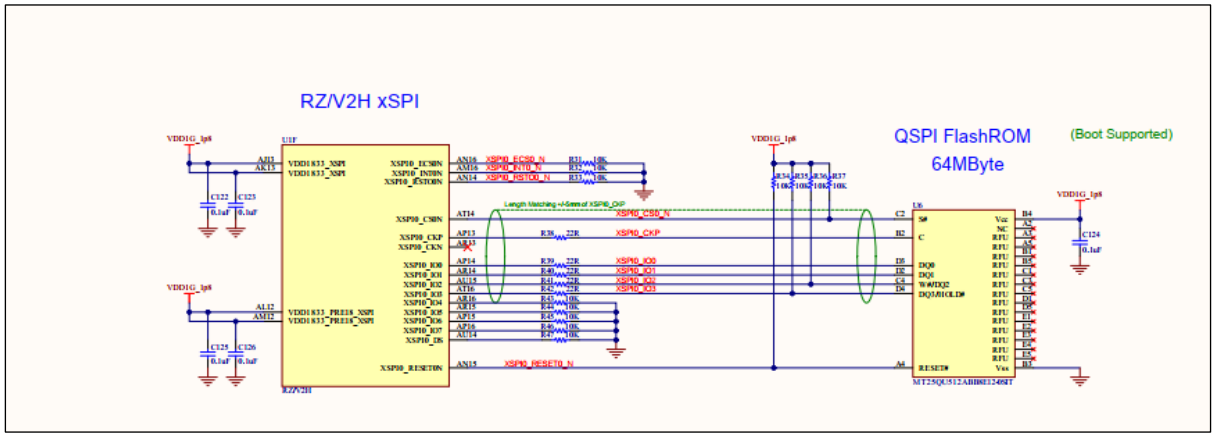


Figure 224. RZ/V2H-RDK xSPI Flash

14.6.8.2 LPDDR4

This block is the external memory subsystem for LPDDR4/LPDDR4X devices. It runs at 3200 Mb/s (1600 MHz) and uses a 32-bit data bus (two x 16-bit channels). The subsystem is split into a DDR Controller (MC) and a DDR PHY. Initialization sequences and timing tables are provided in the Linux package for this device.

Supported standards

- LPDDR4 — JEDEC JESD209-4D
- LPDDR4X — JEDEC JESD209-4-1A

DRAM interface

- Data rate: 3200 Mb/s (LPDDR4/LPDDR4X)
- Bus width: 32-bit (2 × 16-bit channels)
- Ranks: 1 or 2
- Capacity: up to 8 GB (byte-mode not supported)

Controller (MC) highlights

- Fully pipelined command/read/write paths
- Bank look-ahead to maximize throughput
- Programmable registers for timings/protocol (include auto-precharge)
- Full DRAM bring-up on controller reset
- Weighted round-robin arbitration across ports
- ECC: single-bit correction; single/double-bit error detect/report; selectable ECC storage (not on #AC0/#BC0)
- Built-in self-test (BIST) for external DRAM

Low-power features

- Power-down, self-refresh, and I/O retention modes
- Automatic entry/exit or software-controlled operation

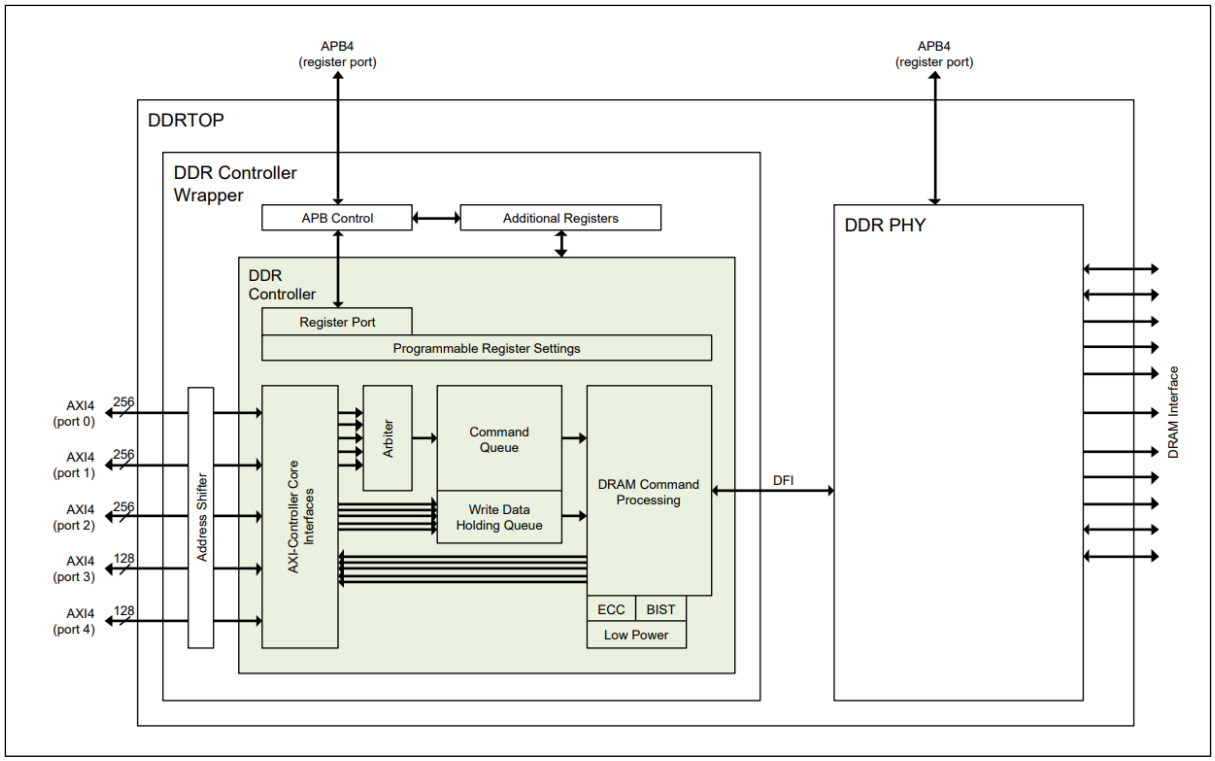


Figure 225. RZ/V2H-RDK Block Diagram of LPDDR4 (Source *RZ/V2H Group User's Manual: Hardware (Non-Agreement)*, p.278)

14.6.8.3 eMMC Flash

RZV2H-RDK does not have on-board eMMC.

14.7 IMDT V2H-SBC Board

14.7.1 Notice

The IMDT V2H-SBC board hardware design and detailed specifications are proprietary and subject to a Non-Disclosure Agreement (NDA). Therefore, hardware design details, schematics, and other confidential information cannot be included in this manual.

The information provided in this section is minimal for reference purposes only and may be limited to publicly available or non-confidential details. It does not represent the complete hardware specification of the board.

For detailed hardware documentation or additional information, please contact IMDT directly:

- Website: https://www.imd-tec.com
- Email: info@imd-tec.com
- Product page: IMDT - V2H SBC
- Hardware Datasheet: V2H_V2N - SBC rev2.1.pdf

14.7.2 Overview

The IMDT V2H SBCs (Single Board Computers) carrier board transforms IMDT SOMs based on Renesas RZ/V2H into powerful, compact mini-computers. Designed for AI-driven vision applications, these SBCs provide an optimal balance of performance, efficiency, and cost while offering extensive assembly options and comprehensive onboard connectivity to meet diverse development needs. Their

energy-efficient design and small footprint make them the ideal choice for developers building compact, high-performance applications.

Processor

- RZ/V2H application MPU (Arm® Cortex®-A55 cluster with integrated peripheral subsystems and security features per device option).
- Suitable for vision/HMI/industrial use cases.

Memory and Storage

- System DRAM: LPDDR4X, 64 Gb × 2 on the CPU board.
- Removable: microSD card slot.
- eMMC (if fitted by variant): supported via SDHI.

Display and Camera

- Camera: up to 4× MIPI® CSI-2® connectors for image sensors.
- Display: MIPI-DSI via an adapter.

Connectivity

- Ethernet: 2× Gigabit Ethernet connectors (RGMII/RMII per wiring and PHY).
- USB:
  - 1× USB 3 Type-A
  - 1× USB 2.0 micro-AB
  - 1× USB 2.0 Type-A
- Serial: USB Type-C debug UART.
- I^2C/SPI/GPIO

Audio

- Headphone output ×1

Power and Clocks

- Power input: USB-PD Type-C ×1, DC-Jack x1
- PMIC: generates SoC rails (core/IO/analog) and peripheral supplies.
- Clocking: programmable clock generator provides system/peripheral references.

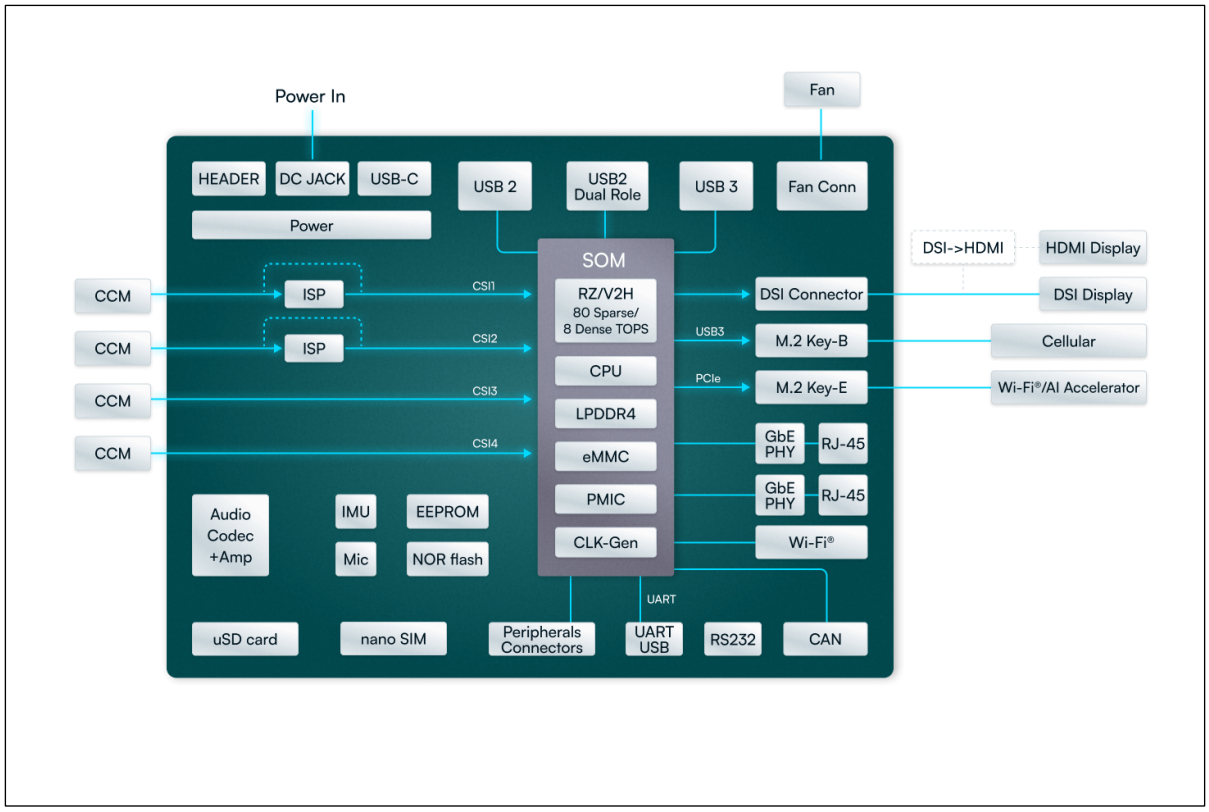
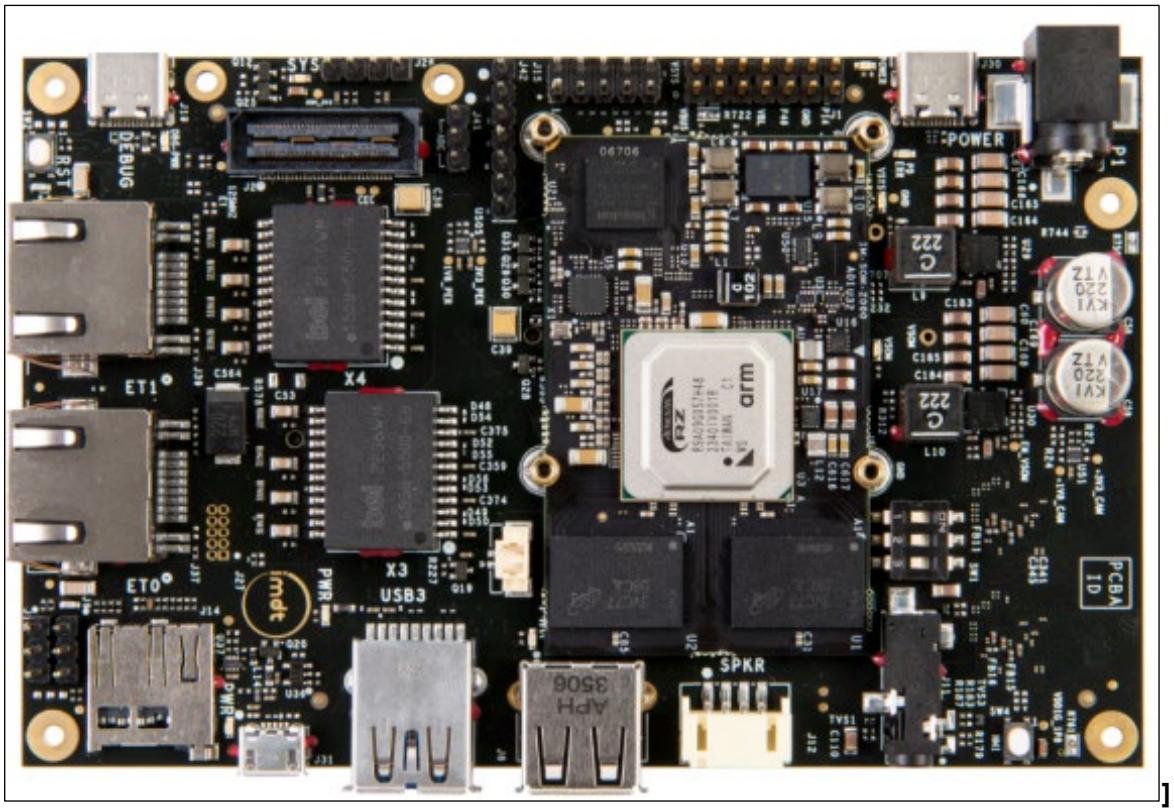


Figure 226. IMDT V2H-SBC System-level Block Diagram



]

Figure 227. IMDT V2H-SBC Top Side

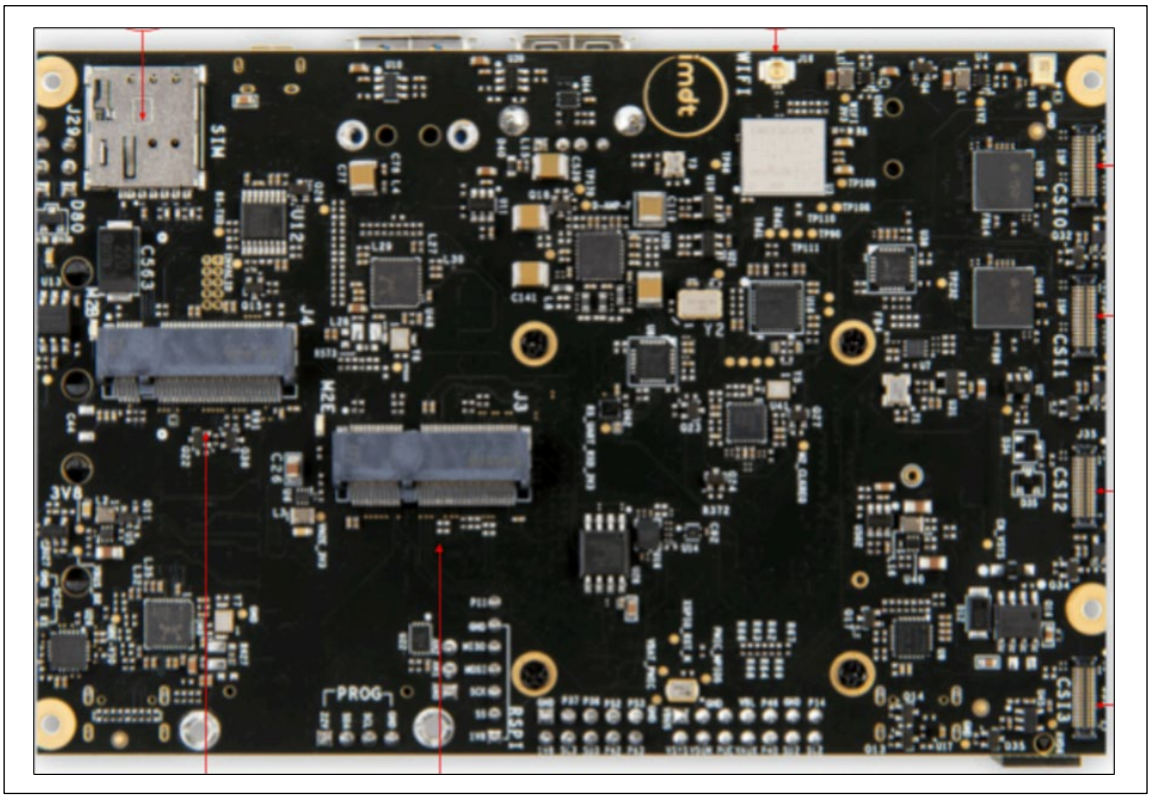


Figure 228. IMDT V2H-SBC Bottom Side

14.7.3 Overview of Connectors

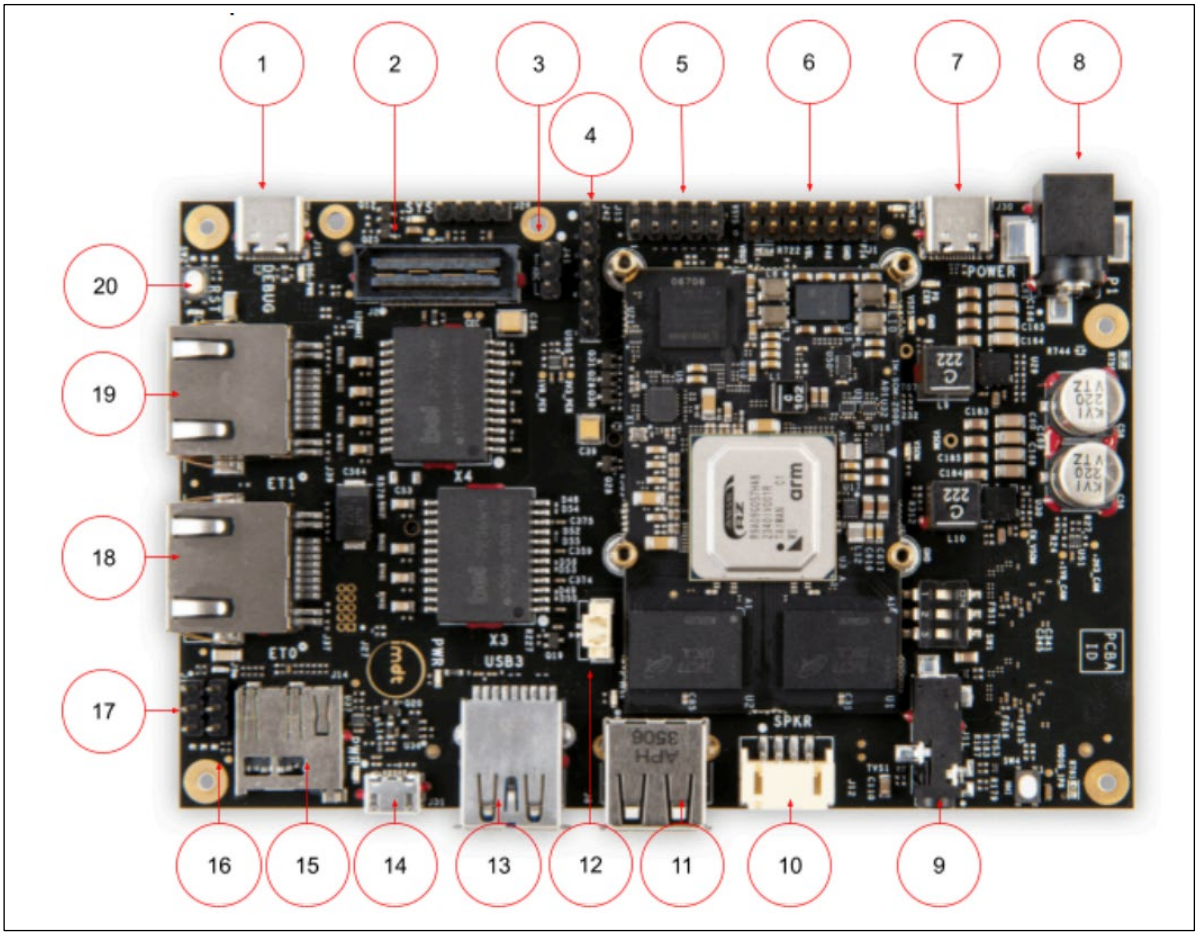


Figure 229. Top View

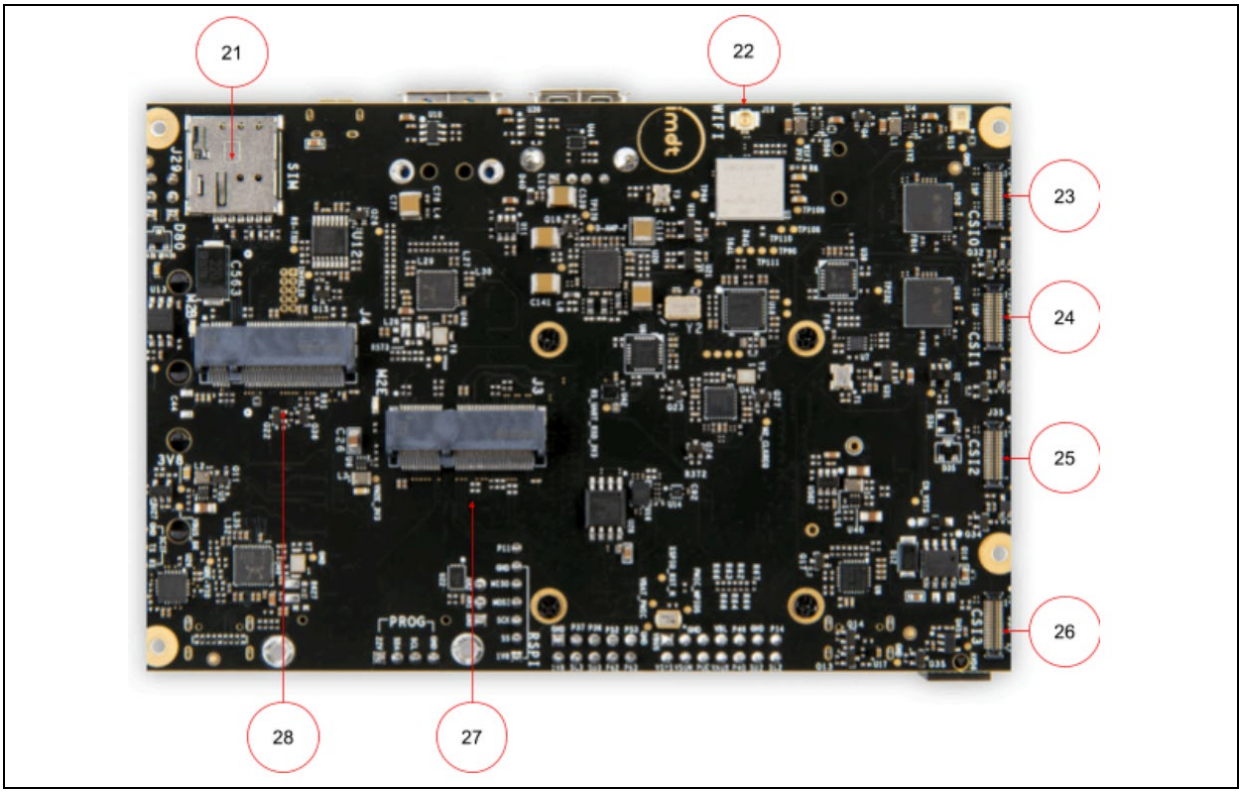


Figure 230. Bottom View

Table 69. Board Functionality Overview

Number	Functionality
1	DEBUG
2	Display
3	Analog Input
4	Auxiliary 1
5	Auxiliary 2
6	PSM (Power Supply Module)
7	Power In - USB-PD
8	Power In - DC Jack
9	3.5mm Audio Jack
10	Speaker Out
11	USB 2
12	Fan
13	USB 3
14	USB Micro-B
15	Micro SD Card
16	RS-232
17	CAN
18	Ethernet 0
19	Ethernet 1
20	Reset Button
21	Nano SIM Card
22	WiFi Antenna
23	CSI 0 Connector
24	CSI 1 Connector
25	CSI 2 Connector

26	CSI 3 Connector
27	M.2 Key-E
28	M.2 Key-B

14.7.4 Power Supply

The IMDT V2H SBC can be powered in two different ways:

Table 70. IMDT V2H-SBC Power Supply Voltage

Connector	Power Requirement
DC Barrel Jack	DC12V- 24V/2A min
USB-C PD	DC9V/3A min

14.7.4.1 Power Supply Regulation

The IMDT V2H-SBC board uses a dedicated Power Management IC (PMIC) to generate and sequence the supply rails required by the SoC and peripherals. See the section below for more detailed information.

14.7.5 Power Management Integrated Circuit (PMIC)

The RAA215300 is a high-performance, low-cost, 9-channel PMIC designed for 32-bit and 64-bit MCU and MPU applications. It supports DDR3, DDR3L, DDR4, and LPDDR4 memory power interfaces. The internally compensated regulators, built-in Real-Time Clock (RTC), 32kHz crystal oscillator, and coin cell battery charger provide a highly integrated, small-footprint power solution ideal for System-On-Module (SOM) applications.

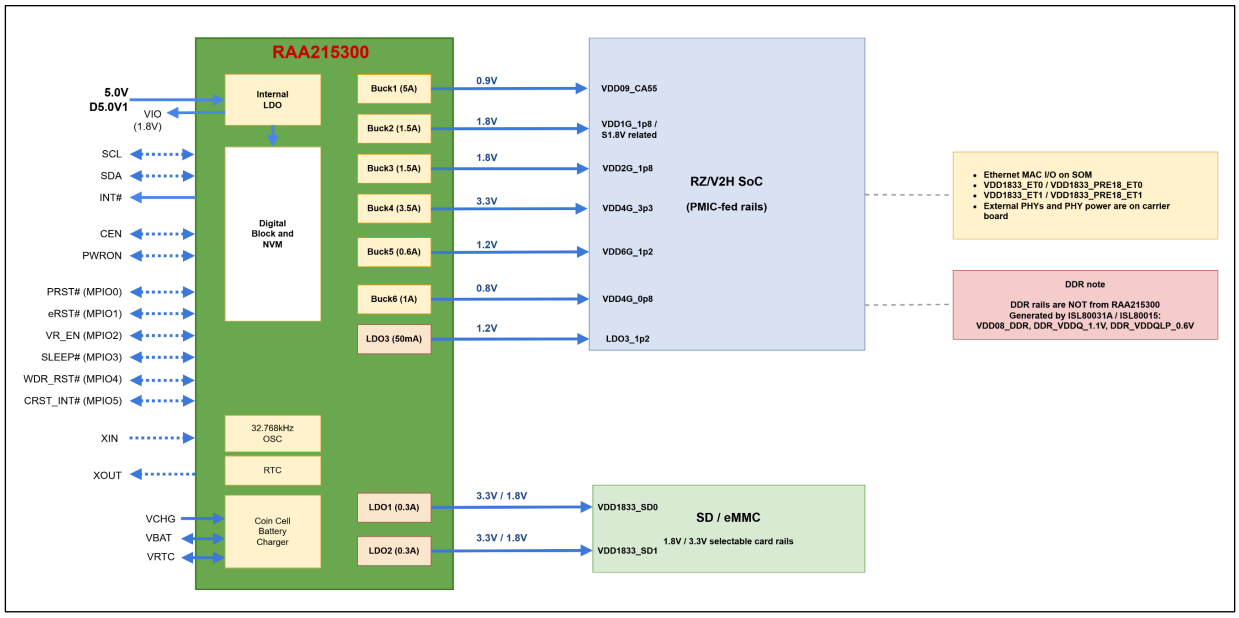


Figure 231. RAA215300 Block Diagram

14.7.6 Power Tree / Internal Power Architecture

The IM-SOM power architecture consists of the main 5 V input, the onboard PMIC, and several dedicated regulators used to generate the SoC, DDR, storage, and peripheral supply rails. In addition to the RAA215300 PMIC, the SOM uses supplementary regulators for selected low-voltage SoC and DDR rails, while the carrier board provides additional peripheral power rails as required

The power tree diagram provides a system-level view of how the main input supply is distributed across the SOM and carrier subsystems, including the major 0.8 V, 1.2 V, 1.8 V, 3.3 V, and 5 V domains.

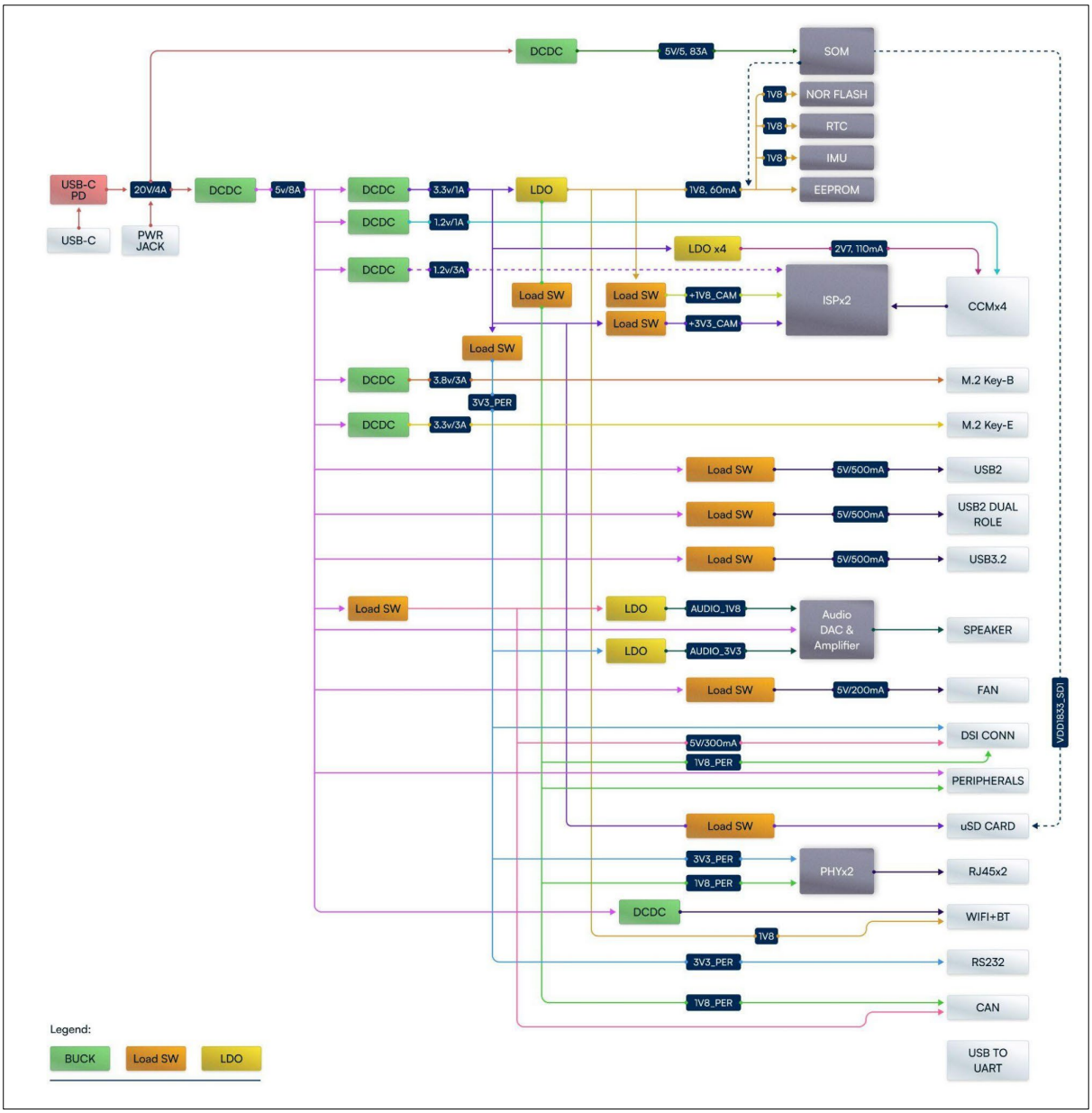


Figure 232. IM-SOM-2000 Power Tree / Internal Power Architecture (Source: V2H_V2N-SBC-rev2.1)

14.7.7 Clock Configuration

The RZ/V2H integrates a Clock Pulse Generator (CPG) and a Power Management Unit (PMU) to manage all timing and power sequencing across the SoC. Together, they provide:

- Generation and distribution of system and peripheral clocks.
- Reset sequencing for reliable initialization.
- Boot-time configuration of CPU and peripheral frequencies.
- Dynamic power management, including clock gating and domain-level power control.

14.7.7.1 Clock Generating and Control Functions

The following functions are provided.

- The clocks to be supplied to various units are generated from external input clocks or PLL output clocks.
- Especially when the CA55 is cold-booted, the CA55 operating frequency is set by the external pin.
- The selection of the frequency divider ratio is implemented (the setting can be changed by the register).
- The clock path is selected by the clock selector (the setting can be changed by the register).
- Clock supply ON/OFF control (the setting can be changed by the register)
- SSCG control of PLL, multiplication ratio setting, and ON/OFF control (the setting can be changed by the register)

Table 71. PLL Table Overview

PLL	Details	Initial Frequency	SSCG Default Value
PLLCM33	For the MCPU bus and various units (PD_AWO area)	1,600 MHz	OFF (fixed)
PLLCLN	For buses other than the MCPU bus and various units	1,600 MHz	OFF (fixed)
PLLDTY	For buses other than the MCPU bus and various units	1,600 MHz	Depends on MD_CLKS
PLLCA55	CA55	1,700 MHz	Depends on MD_CLKS
PLLVDO	CRU / ISP / GE3D / CA55	1,260 MHz	Depends on MD_CLKS
PLLETH	GBETH / DRP1 / DRP-AI / DSI	1,000 MHz	OFF (fixed)
PLLDSI	DSI / LCDC	297 MHz	OFF (fixed)
PLLDDR0	For DDR ch. 0	800 MHz	OFF (fixed)
PLLDDR1	For DDR ch. 1	800 MHz	OFF (fixed)
PLLGPU	GE3D	1,260 MHz	ON (fixed)
PLLDRP	DRP1 / DRP-AI / CA55	1,260 MHz	Depends on MD_CLKS

14.7.7.2 Clock Pin Specification

The RZ/V2H device provides a wide range of dedicated clock input and output pins to support system timing, peripheral interfaces, and external device synchronization.

This table lists all clock-related pins on RZ/V2H—showing pin name, I/O direction, function, maximum supported frequency, and the supplying/receiving unit

Table 72. IMDT V2H-SBC Clock Pin Specification

Pin Name	Input	Output	Frequency*1	Supply Source/ Destination Unit
LSI Clock				

QXTAL	Output	24 MHz main clock	24 MHz	CPG
QEXTAL	Input	24 MHz main clock	24 MHz	CPG
RTXOUT	Output	32.768 kHz real-time clock	32.768 kHz	RTC
RTXIN	Input	32.768 kHz real-time clock	32.768 kHz	RTC
AUDIO_XTAL	Output	4 to 48 MHz audio clock (Internal OSC mode)	48 MHz	ADG
AUDIO_EXTAL	Input	4 to 48 MHz audio clock (Internal OSC mode) 4 to 50 MHz audio clock (Input clock from the external device)	50 MHz	ADG
AUDIO_CLKOUT*2	Output	2 to 25 MHz audio clock-out (Half frequency of input clock)	25 MHz	ADG
Debug Interface				
TCK_SWCLK	Input	Test clock pin for the on-chip emulator functions as the SWCLK pin in serial wire debug (SWD) mode	20 MHz	CST
Expanded serial peripheral interface (xSPI)				
XSPI0_CKP*3	Output	Clock output pins (positive)	133 MHz	xSPI
XSPI0_CKN*3	Output	Clock output pins (negative)	133 MHz	xSPI
SD/MMC interface				
SD0CLK	Output	Output the clock signal to the external SD/MMC device	200 MHz	SD0
SD Interface				
SD1CLK	Output	Output the clock signal to the external SD device	200 MHz	SD1
SD2CLK	Output	Output the clock signal to the external SD device	200 MHz	SD2
PCIe Gen3				
PCIE_REFCLKP0	I/O	Differential reference clock (positive), 100 MHz ±300 ppm	100 MHz	PCIE0
PCIE_REFCLKN0	I/O	Differential reference clock (negative), 100 MHz ±300 ppm	100 MHz	PCIE0
PCIE_REFCLKP1	I/O	Differential reference clock (positive), 100 MHz ±300 ppm	100 MHz	PCIE1
PCIE_REFCLKN1	I/O	Differential reference clock (negative), 100 MHz ±300 ppm	100 MHz	PCIE1
Gbit Ethernet ch. 0				
ET0_MDC	Output	Output management data clocks	2.5 MHz	GBETH0
ET0_RXC_RXCLK	Input	RX Clocks	125 MHz	GBETH0

ET0_TXC_TXCLK	I/O	TX Clocks	125 MHz	GBETH0
Gbit Ethernet ch. 1				
ET1_MDC	Output	Output management data clocks	2.5 MHz	GBETH1
ET1_RXC_RXCLK	Input	RX Clocks	125 MHz	GBETH1
ET1_TXC_TXCLK	I/O	TX Clocks	125 MHz	GBETH1
MIPI DSI				
DSI_DPCLK	Output	Output clocks (positive)	750 MHz	DSI
DSI_DNCLK	Output	Output clocks (negative)	750 MHz	DSI
MIPI CSI-2 ch. n (n = 0 to 3)				
CSIn_CLKP	Input	Input clocks (positive)	1050 MHz	CRUn
CSIn_CLKN	Input	Input clocks (negative)	1050 MHz	CRUn
Serial peripheral interface (RSPI) ch. n (n = 0, 1, 2)				
RSPCKn	I/O	Synchronous clock signal	50 MHz	RSPIn
Serial communication interface (RSCI) ch. n (n = 0 to 9)				
SCKn	I/O	Clock pins (simple SPI mode)	37.5 MHz	RSCIn
SCLn	I/O	I2C clocks (simple I2C mode)	400 kHz	RSCIn
I2C bus interface (RIIC) ch. n (n = 0 to 8)				
SCLn	I/O	Clock pins with N-ch open drain	1 MHz	RIICn
I3C bus interface (I3C)				
SCL30	I/O	Clock Pin	3 MHz	I3C0
Pulse density modulation interface (PDM) ch. n (n = 0 to 5)				
PDMCLK00	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK01	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK02	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK10	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK11	Output	Output PDM sampling clocks	2.4 MHz	PDM0
PDMCLK12	Output	Output PDM sampling clocks	2.4 MHz	PDM0
Serial sound interface (SSIU) ch. n (n = 0 to 9)				
SSIn_SCK	I/O	Output clocks	12.5 MHz	SSIU

- Note 1. The clock frequencies are either variable or fixed. In the case of the variable clocks, the maximum frequencies that can be output are listed for the output clocks, and the maximum frequencies that can be input to this LSI are listed for the input clocks.
- Note 2. For the use of these pins, the function selection of multiplexed function pins is required.
- Note 3. CKP and CKN waves have opposite phases.

14.7.8 Peripheral Interface

14.7.8.1 Gigabit Ethernet

The CPU board of the IMDT V2H-SBC has two Ethernet interfaces. The RJ-45 connectors are connected to the Ethernet interface of the RZ/V2H via the Ethernet PHY IC. Connecting this interface to a public line is prohibited.

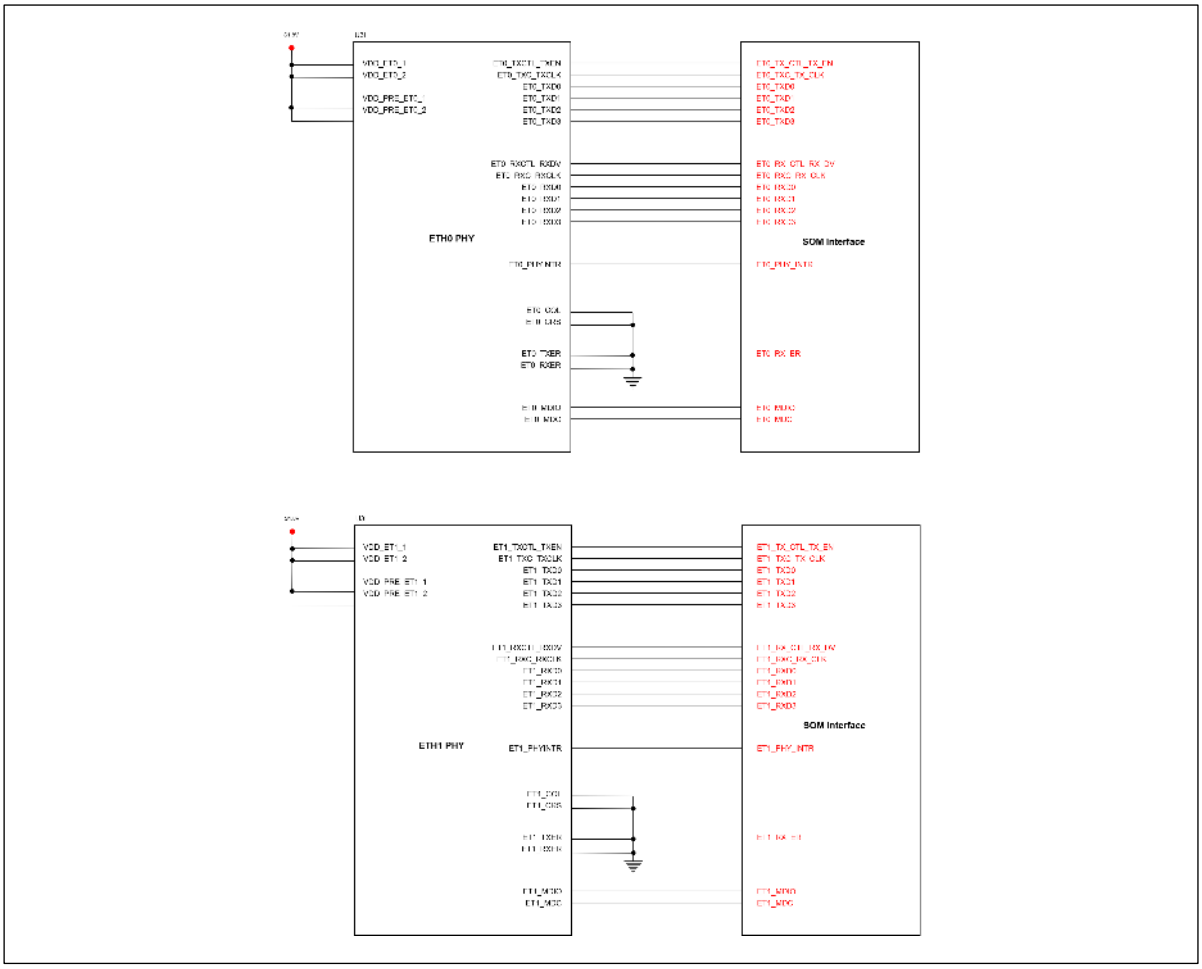


Figure 233. ETH0,1 Block Diagram of IMDT-V2H-SBC

14.7.8.2 USB 2.0 Ports

USB 2.0 Host Type-A connector CN3 is wired to USB 2.0 channel 1 on the RZ/V2H. Use this port to connect full-/high-speed USB peripherals (flash drives, keyboards, Wi-Fi dongles, etc.).

USB 2.0 DRD (CH0): Micro-AB connector CN2 is wired to USB 2.0 channel 0 and operates as a Dual-Role Device (host/device) depending on the cable/ID pin. Suitable for OTG use, device mode console, or host mode with an adapter.

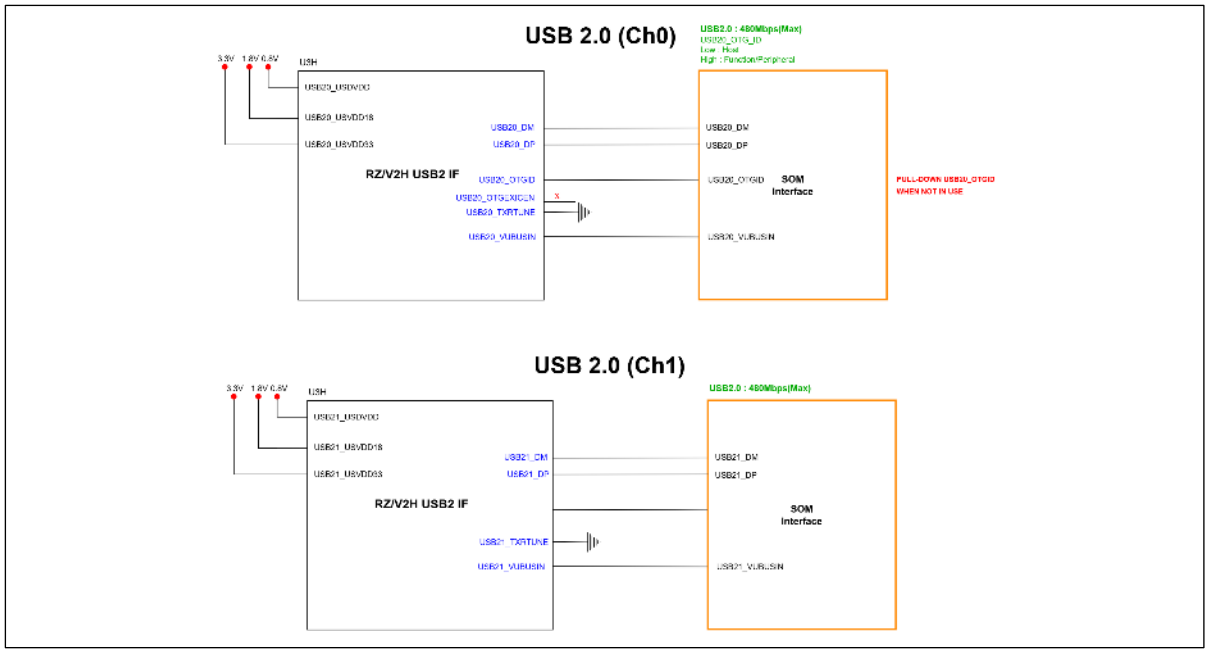


Figure 234. USB 2.0 Block Diagram

14.7.8.3 USB 3.2 Ports

The IMDT V2H-SBC provides one USB 3.2 Gen2 Type-A host connector, supporting Superspeed+ (10 Gbps) signaling. This interface allows connection of high-speed external devices such as SSDs, cameras, and expansion adapters.

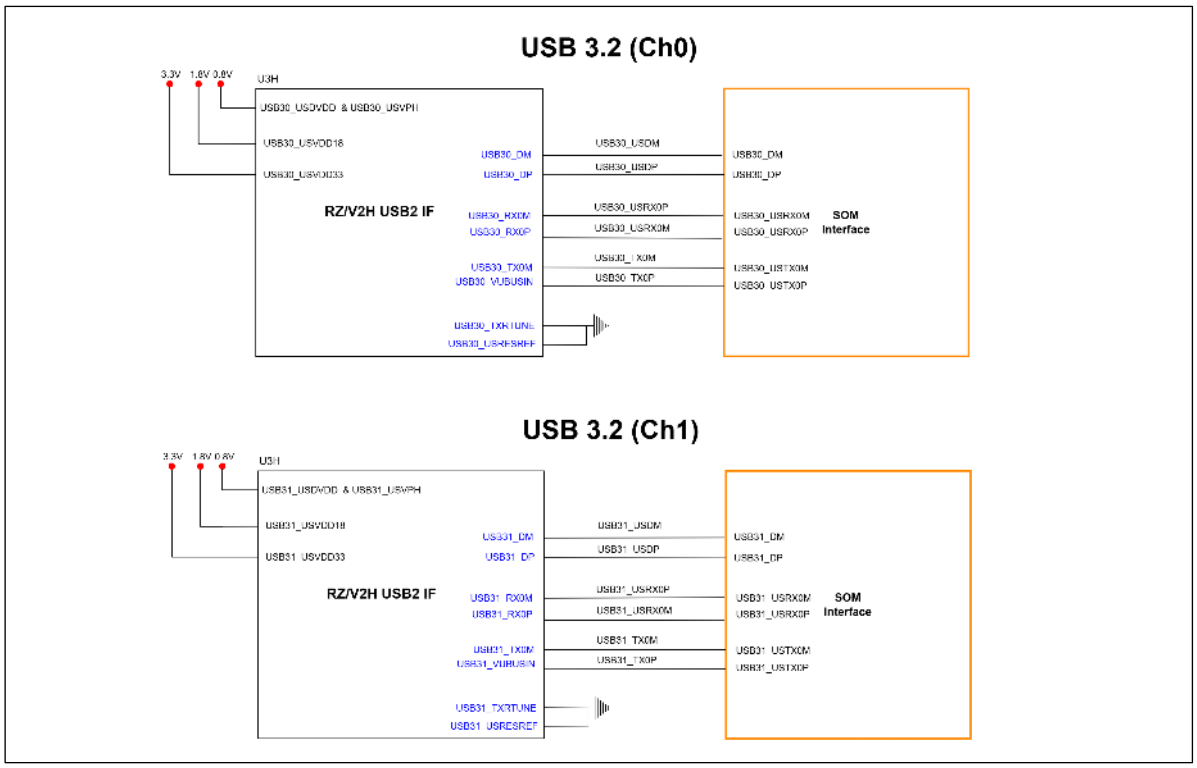


Figure 235. USB 3.2 Block Diagram

14.7.8.4 Audio Subsystem with 3.5mm Jack

The audio path is codec-centric: a codec connects to the SoC over I²C (control) and SSI3 (I²S data), with AUDIO_CLKA (AUDIO_CLKB and AUDIO_CLKC are not connected on this board) from the ADG providing the master clock. It features a 3.5-mm audio jack.

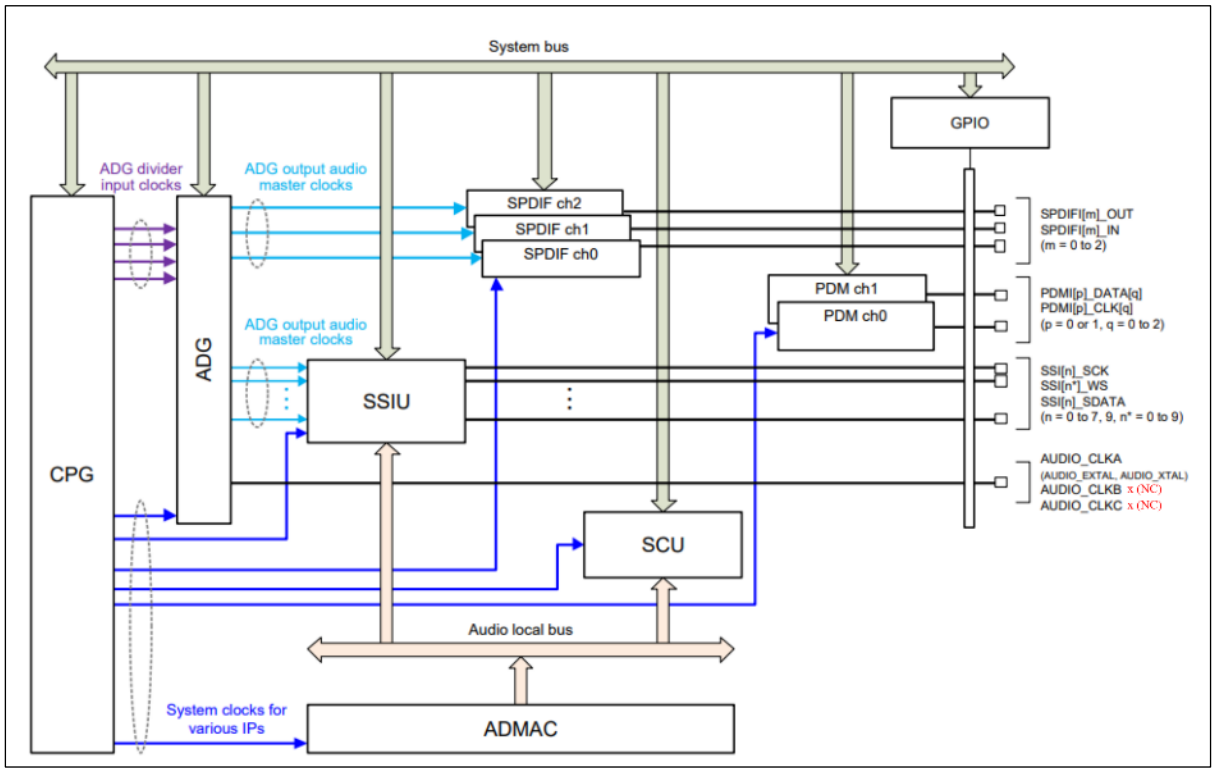


Figure 236. IMDT-V2H-SBC Audio Subsystem

14.7.8.5 MIPI CSI Interface

The IMDT V2H-SBC provides four independent MIPI-CSI-2 interfaces (CH0–CH3) for image sensor connectivity. Each channel is routed to a dedicated 34-pin IAS camera connector (CN0–CN3).

The four CSI interfaces are not equivalent in terms of ISP integration:

- CSI0 and CSI1 are connected via on-board OnSemi AP1302 ISPs
- CSI2 and CSI3 are connected directly to the CPU and can utilize the internal CPU ISP

This distinction should be considered when selecting camera sensors and when planning features that require ISP-based image processing.

This configuration allows connection of 4 cameras with 4 ISPs

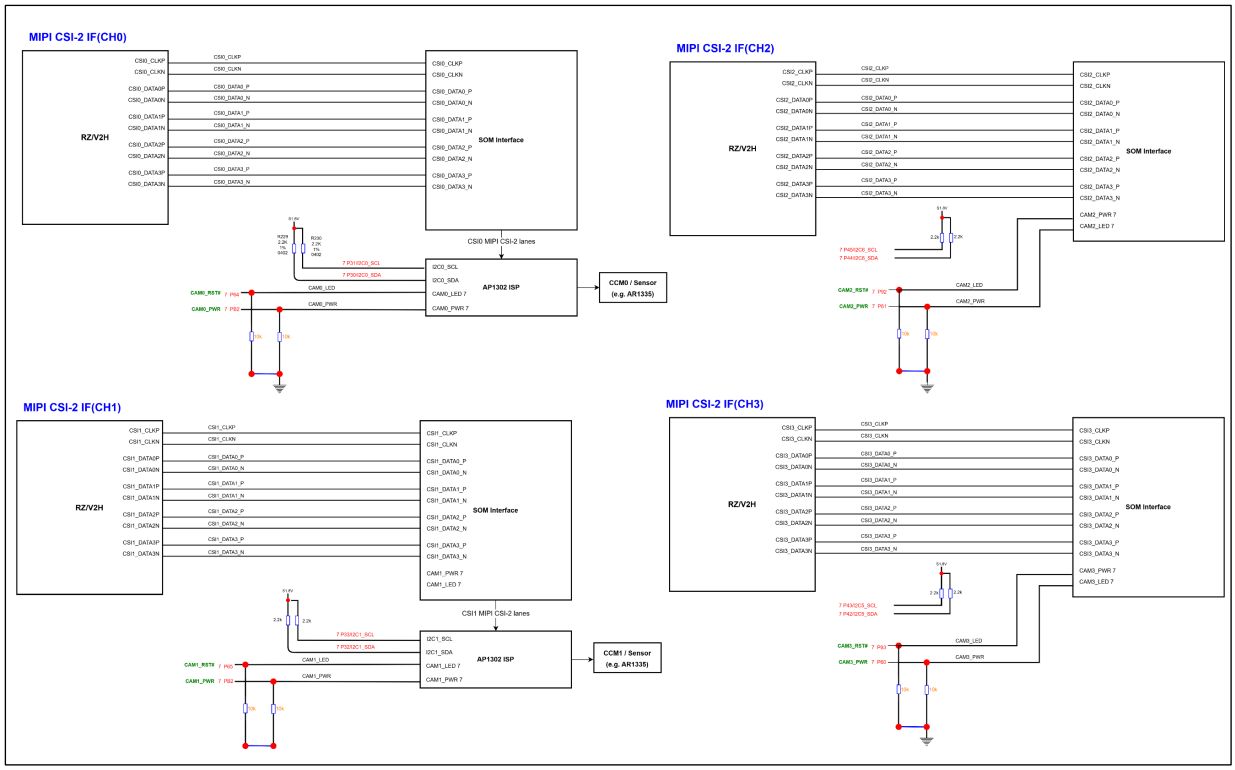


Figure 237. CSI-2 IF Block Diagram of IMDT-V2H-SBC

14.7.8.6 uSD-card Interface

The interface shown below reflects the IMDT V2H-SBC:

- SDHI1 connected to a microSD slot (user-removable)

Both interfaces support ROM boot. Insert the microSD card before power-up/reset and select SD boot via DIP switch.

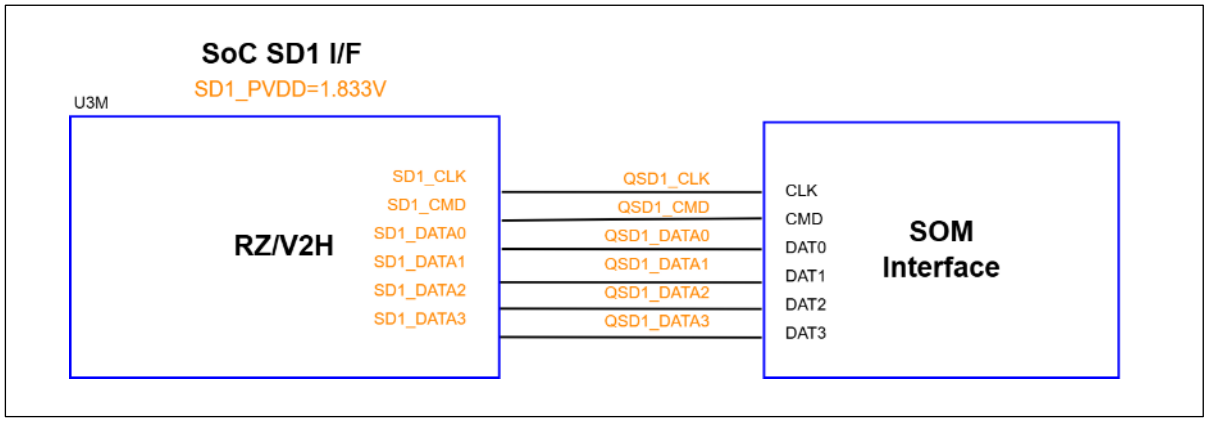


Figure 238. SD1 I/F Block Diagram

14.7.8.7 Analog to Digital Converter

IMDT V2H-SBC exposes the SoC's on-chip SAR ADC for basic voltage sensing and evaluation. The CPU board routes up to 8 single-ended input channels.

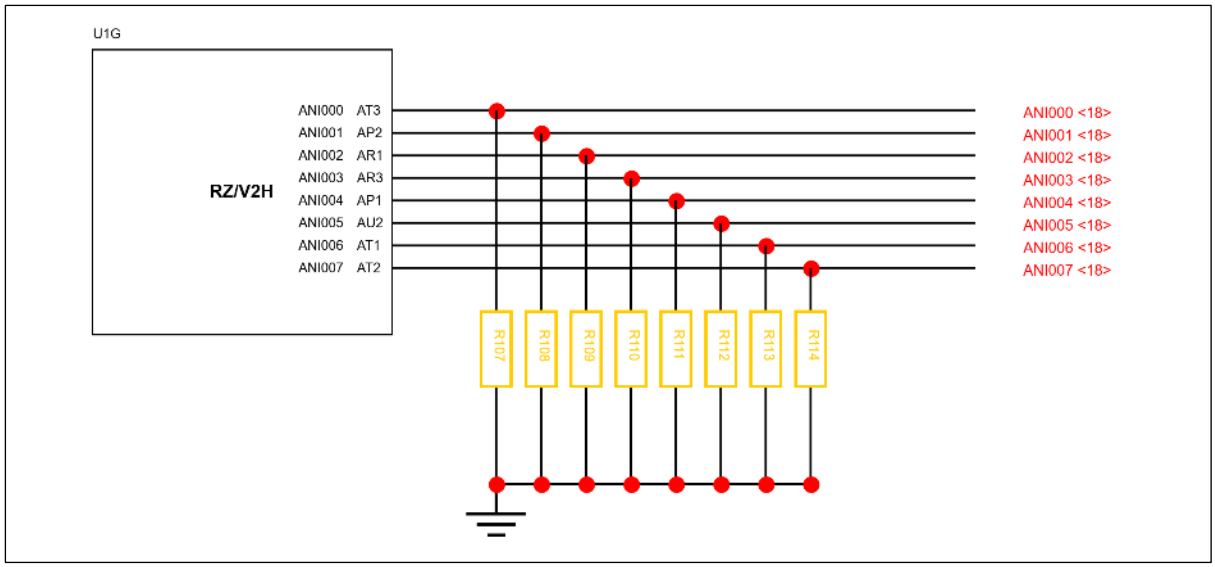


Figure 239. IMDT V2H-SBC Analog to Digital Converter

14.7.9 Memory

14.7.9.1 xSPI Flash

The CPU board includes a 64 MB SPI NOR flash device connected to the SoC serial flash interface.

The flash operates at 1.8 V and supports ROM-based boot. The SoC is capable of fetching early boot firmware (e.g., BL2/FIP) directly from this device.

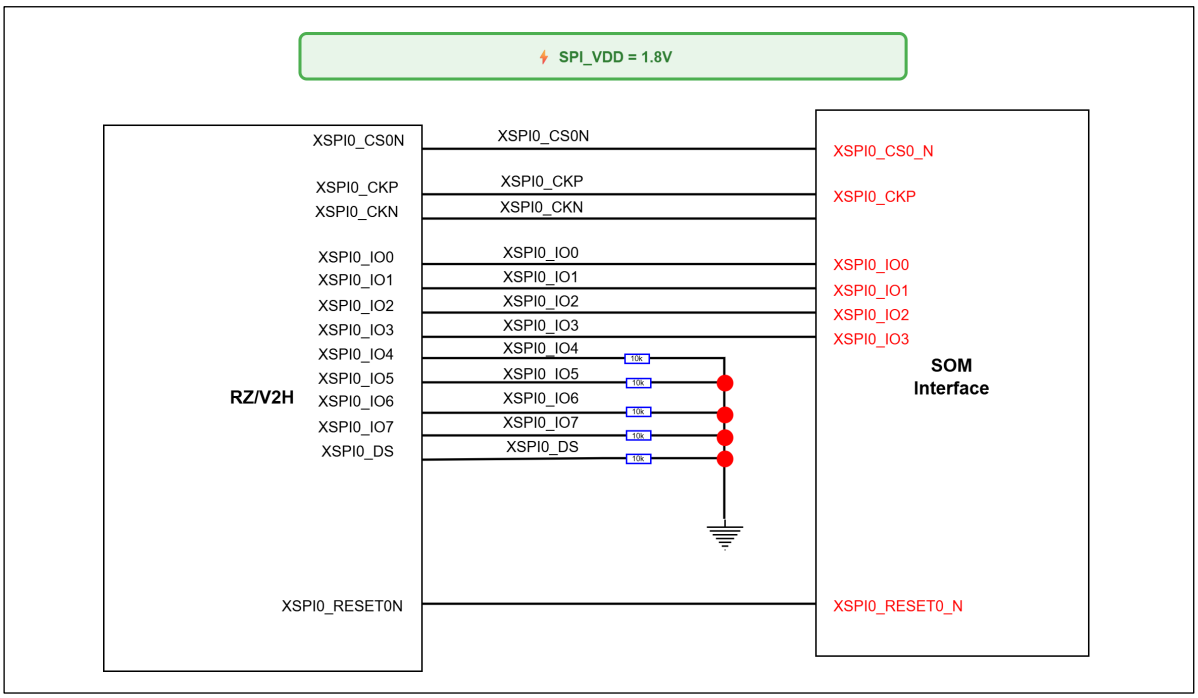


Figure 240. Block Diagram of xSPI Flash of IMDT-V2H-SBC

14.7.9.2 LPDDR4

The SoC integrates an external memory subsystem supporting LPDDR4 and LPDDR4X devices. The subsystem consists of a Memory Controller (MC) and a DDR PHY.

The controller supports data rates up to 3200 Mb/s (1600 MHz) with a 32-bit data bus configuration (2 × 16-bit channels).

Supported standards

- LPDDR4 — JEDEC JESD209-4D
- LPDDR4X — JEDEC JESD209-4-1A

DRAM interface

- Data rate: up to 3200 Mb/s (LPDDR4/LPDDR4X)
- Bus width: 32-bit (2 × 16-bit channels)
- Ranks: 1 or 2
- Capacity: up to 8 GB (byte-mode not supported)

Controller (MC) highlights

- Fully pipelined command/read/write paths
- Bank look-ahead to maximize throughput
- Programmable registers for timings/protocol (include auto-pre-charge)
- Full DRAM bring-up on controller reset
- Weighted round-robin arbitration across ports
- ECC: single-bit correction; single/double-bit error detect/report
- Built-in self-test (BIST) for external DRAM

Low-power features

- Power-down, self-refresh, and I/O retention modes
- Automatic entry/exit or software-controlled operation

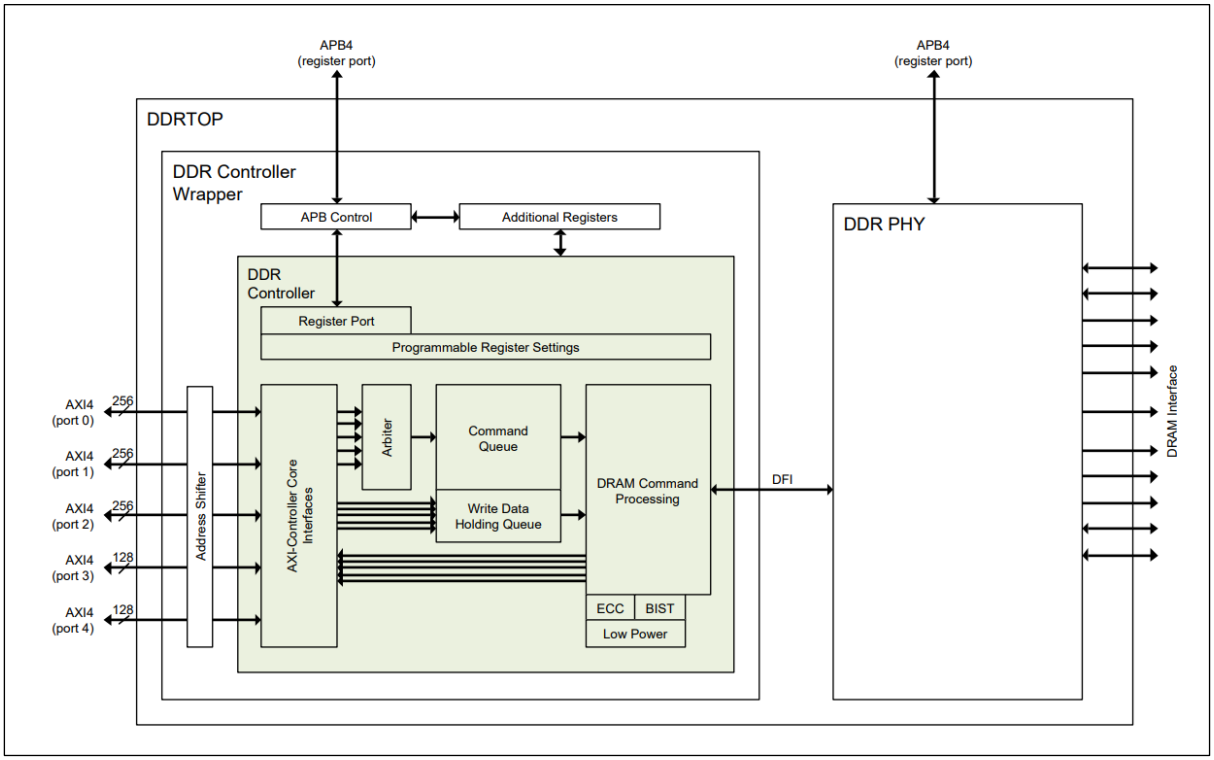


Figure 241. IMDT V2H-SBC Block Diagram of LPDDR4

14.7.9.3 eMMC Flash

One eMMC device is populated (wired to the MMC/SDHI host), plus one microSD slot for removable media. eMMC may be used as the primary boot/storage device; otherwise, boot from SD1 (SDHI0).

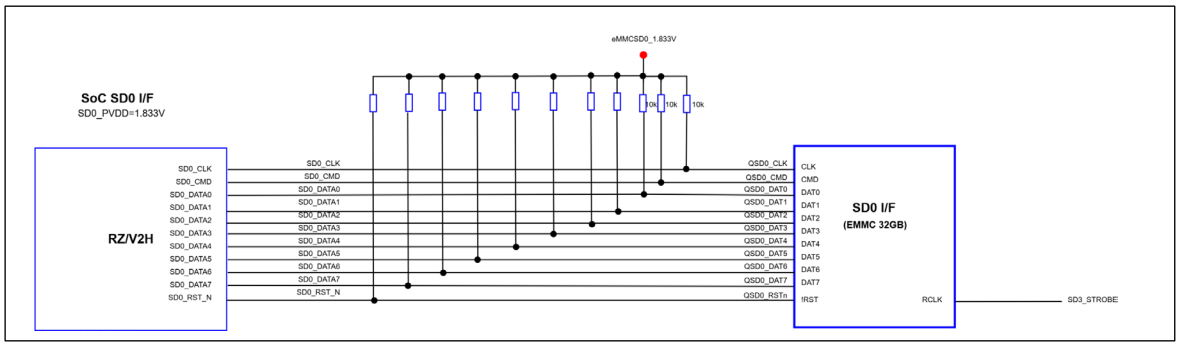


Figure 242. SD0 (eMMC) Block Diagram

15. Adding a New Board (Porting Guide)

This section describes how to add a new board to the RZ Common System. It covers naming, device trees, Flash-writer, TF-A, U-Boot, Linux Kernel settings, and Yocto/Ubuntu integration.

For boards in the same RZ series (RZG2L / RZV2L / RZV2H), changes are typically device tree, platform-specific features, and packaging. Adding new MPUs to the RZ Common System often requires driver or platform code updates.

The overall process includes:

- Defining board metadata (JSON and board-info binary).
- Porting Flash Writer for storage and DDR initialization.
- Supplying TF-A (BL2/BL31) with a board configuration via FCONF DT.
- Creating a U-Boot device tree and minimal board hooks.
- Creating a Linux device tree (.dts) and enabling required drivers.
- Updating flash scripts and build recipes to package the new board.

Each subsection below describes these steps in detail.

15.1 Board Identification

Board information is a structured set of metadata that identifies a supported Renesas RZ-based board and provides board-specific boot parameters to the firmware stack. Typical board information includes the board model ID, PCB revision, model string, and boot parameters such as MMC device, partition, and image load addresses.

In the RZ Common System, the board identification mechanism uses this information to allow firmware to detect the board and select the correct configuration during boot.

This mechanism has two parts:

- Host side: Board metadata is defined in a human-readable JSON file and converted into a binary file by a binmake tool, which will be described in a later section.
- Target side: The generated binary file is flashed into persistent storage, and firmware reads it during boot.

The JSON file is therefore only the authoring format used on the host PC. It is not parsed directly on the target. Instead, the JSON description is converted into a binary file, typically named board-info.bin, using the binmake tool. This binary file is then flashed into a dedicated region of QSPI, eMMC, or SD card. During boot, Trusted Firmware-A (TF-A) and U-Boot read this binary structure to:

- detect the board model and board revision
- select the correct board configuration and device tree
- obtain board-specific boot parameters such as MMC device, partition, and image load addresses.

15.1.1 End-to-End Flow

The board identification process is shown below. This subsection provides a high-level overview of the complete flow; each step is described in more detail in the following subsections.

1. On the host PC, define a board entry in the JSON configuration file.
2. On the host PC, run the binmake tool to convert the JSON entry into board-info.bin.
3. Flash board-info.bin into a predefined storage region on the target board using Flash Writer or the universal flashing script.
4. During boot, TF-A reads the binary board information from storage.

5. U-Boot reads the same information or receives the selected parameters from earlier firmware stages and populates board-related environment variables.
6. Linux is then booted using the board-specific configuration selected by the firmware.

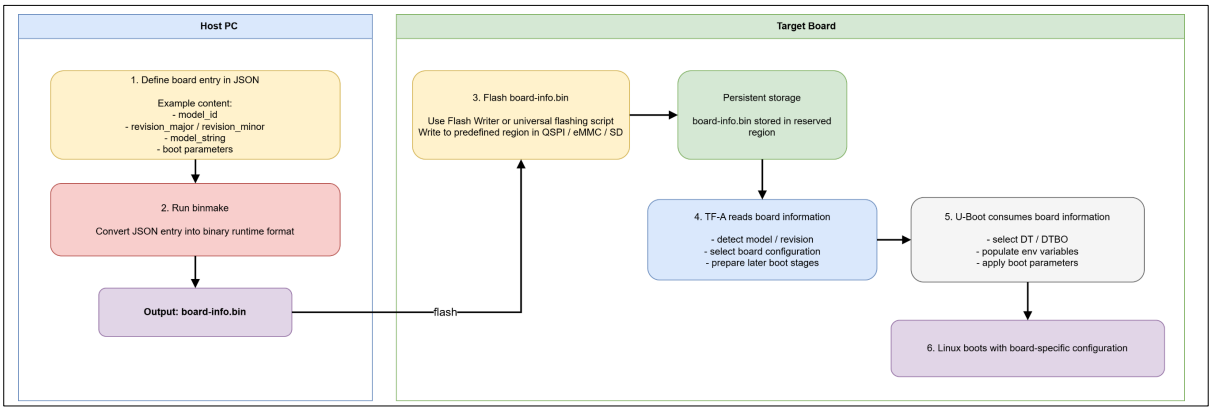


Figure 243. Board Identification End-to-End Flow

15.1.2 Host-side JSON Specification

On the host side, each board is described by a JSON entry. This JSON file is the editable source format used by developers and manufacturing flows. It allows board metadata to be maintained in a structured and human-readable form before being converted into the binary runtime format.

In the JSON file:

- numeric values such as model_id are defined in decimal,
- firmware internally may use or display these values in hexadecimal,
- only the generated binary format is used at runtime on the target.

Table 73. JSON specification table

Field		Description	Example	Usage
model_id		Unique numeric identifier for the board. Used by TF-A and U-Boot to select the correct configuration. In the JSON file, the value is defined in decimal; it is converted to hexadecimal in the firmware.	16 (0x10) = RZ/G2L-EVK 17 (0x11) = RZ/G2L-SBC 32 (0x20) = RZ/V2L-EVK 48 (0x30) = RZ/V2H-EVK	Used
Revision number	Major	Major PCB revision number. Increment when the board design changes significantly.	Used	Used
	Minor	Minor PCB revision number. Increment for small changes or fixes.	Used	Used
model_string		Human-readable string for the board name. Must uniquely match model_id.	"rzg2l-evk"	Used
mfg_name		Manufacturer name.	"Renesas"	Reserved/ Not Used
bl2_loc		Storage class (device type) where BL2 is located	0 = QSPI 1 = MMC	Reserved/ Not Used
bl2_dtb_loc		Storage class (device type) where BL2 DTB is located. It must be the same as bl2_loc	0 = QSPI 1 = MMC	Reserved/ Not Used
u_boot_loc		Storage class where U-Boot is located.	0 = QSPI 1 = MMC	Reserved/ Not Used

u_boot_dtb_loc	Storage class where U-Boot DTB is located. It must be the same as u_boot_loc	0 = QSPI 1 = MMC	Reserved/ Not Used
kernel_loc	Storage class where the Linux kernel is located.	0 = QSPI 1 = MMC	Reserved/ Not Used
kernel_dtb_loc	Storage class where the Linux DTB is located. It must be the same as kernel_loc	0 = QSPI 1 = MMC	Reserved/ Not Used
bl2_id	Device number within the selected storage class for BL2 and BL2 DTB.	If bl2_loc=1 (MMC) and bl2_id=0, BL2 is on mmc0.	Reserved/ Not Used
bl2_dtb_id	Device number within the selected storage class for BL2 DTB.	If bl2_loc=1 (MMC) and bl2_dtb_id=0, BL2 DTB is on mmc0.	Reserved/ Not Used
u_boot_id	Device number for U-Boot and its DTB.	0 (mmc0)	Reserved/ Not Used
u_boot_dtb_id	Device number for U-Boot DTB.	0 (mmc0)	Reserved/ Not Used
kernel_id	Device number for Linux kernel.	0 (mmc0)	Reserved/ Not Used
kernel_dtb_id	Device number for Linux DTB.	0 (mmc0)	Reserved/ Not Used
bl2_desc	Extra parameters for BL2 (offsets, load addresses). Usually, empty.	[]	Reserved/ Not Used
bl2_dtb_desc	Extra parameters for BL2 DTB (offsets, load addresses). Usually, empty.	[]	Reserved/ Not Used
u_boot_desc	U-Boot-specific parameters. Indexed values: [0] mmcdev [1] mmcpart [2] rootfs_mmcdev [3] rootfs_mmcpart [4] image load address [5] environment load address	["0","1","0","2","0x48080000","0x58000000"]	Used
u_boot_dtb_desc	U-Boot DTB and DTBO load addresses. First entry = DTB, optional second entry = DTBO. [0] u-boot DTB address [1] u-boot DTBO address	["0x48000000","0x48010000"]	Used
kernel_desc	Reserved for Linux kernel extra parameters. Usually left empty.	[]	Reserved/ Not Used
kernel_dtb_desc	Reserved for Linux DTB extra parameters. Usually left empty.	[]	Reserved/ Not Used

Note: Reserved/Not Used fields exist for forward compatibility with legacy layouts and future boot paths. Leave them at defaults or empty [] unless a future release marks them Used.

Example JSON for a new board

```
  "rzg2l-sbc": {
      "model_id": 17,
      "revision_minor": 0,
      "revision_major": 1,
      "model_string": "rzg2l-sbc",
      "mfg_name": "Renesas",

      "bl2_loc": 0,
      "bl2_dtb_loc": 0,
      "u_boot_loc": 3,
      "u_boot_dtb_loc": 3,
      "kernel_loc": 1,
      "kernel_dtb_loc": 1,

      "bl2_id": 2,
      "bl2_dtb_id": 3,
      "u_boot_id": 6,
      "u_boot_dtb_id": 7,
      "kernel_id": 14,
      "kernel_dtb_id": 15,

      "bl2_desc": [],
      "bl2_dtb_desc": [],
      "u_boot_desc": ["0", "1", "0", "2", "0x48080000", "0x58000000"],
      "u_boot_dtb_desc": ["0x48000000", "0x48010000"],
      "kernel_desc": [],
      "kernel_dtb_desc": []
  }
```

15.1.3 Host-Side Binary Generation

Once the JSON configuration has been updated with a new board entry, it must be converted into a binary format and then flashed into persistent storage (QSPI, eMMC, or SD card). At boot time, Trusted Firmware-A (ATF) and U-Boot can read the board information directly from hardware and correctly identify the platform, load the right device tree, and initialize BL2, U-Boot, and Linux Kernel with the proper parameters.

This binary file is the artifact that must be flashed to the target. Neither TF-A nor U-Boot reads the JSON file directly.

1. Create the JSON
   Define a new board entry in the JSON file. See section JSON Specification for details on the required fields and examples.
2. Build **binmake** tool
   - Prerequisites
     - On Windows
       - CMake: To install, please refer to this page CMake to download the setup file for a specific platform.
       - gcc

- Download and install MinGW-w64
- Install to the default location (**C:/MinGW**)
- Add the following path to the Windows Environment Variables → Path: **C:/MinGW/bin**

  - On Linux

```
sudo apt install build-essential cmake
```

- Build tool
  - **On Linux**: Open a terminal and run the following commands to clone the repository

```
renesas@builder:~/$ git clone git@github.com:Renesas-SST/rz-utils.git -b \
styhead/rz-cmn
renesas@builder:~/$ cd rz-utils/tools/binmake
renesas@builder:~/rz-utils/tools/binmake$ mkdir build
renesas@builder:~/rz-utils/tools/binmake$ cd build
```

Then compile the binary

```
renesas@builder:~/rz-utils/tools/binmake/build$ cmake ..
-- The C compiler identification is GNU 11.4.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/renesas/rz-utils/tools/binmake/build
renesas@builder:~/rz-utils/tools/binmake/build$ make
[ 16%] Building C object CMakeFiles/binmake.dir/src/binmake.c.o
[ 33%] Building C object CMakeFiles/binmake.dir/src/revision.c.o
[ 50%] Building C object CMakeFiles/binmake.dir/src/json_utils.c.o
[ 66%] Building C object CMakeFiles/binmake.dir/src/output_utils.c.o
[ 83%] Building C object CMakeFiles/binmake.dir/cjson/cJSON.c.o
[100%] Linking C executable binmake
[100%] Built target binmake
```

- **On Windows:** Open one of the following terminals with Administrator privileges:
  - PowerShell
  - Git Bash
  - MobaXterm

  The step is similar to that of Linux users. First, clone the repository and then navigate to the rz-utils/tools/binmake directory. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> git clone git@github.com:Renesas-SST/rz-utils.git -b \
styhead/rz-cmn
PS C:\Users\renesas> cd rz-utils/tools/binmake
PS C:\Users\renesas\rz-utils\tools\binmake> mkdir build
PS C:\Users\renesas\rz-utils\tools\binmake> cd build
```

Then compile the binary

```
PS C:\Users\renesas\rz-utils\tools\binmake\build> cmake -G "MinGW Makefiles" ..
-- The C compiler identification is GNU 6.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: C:/MinGW/bin/gcc.exe - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Configuring done (2.0s)
-- Generating done (0.0s)
PS C:\Users\renesas\rz-utils\tools\binmake\build> make
-- Configuring done (0.2s)
-- Generating done (0.1s)
-- Build files have been written to: C:/Users/renesas/rz-
utils/tools/binmake/build
[ 33%] Building C object CMakeFiles/binmake.dir/binmake.c.obj
[ 66%] Building C object CMakeFiles/binmake.dir/cjson/cJSON.c.obj
[100%] Linking C executable binmake.exe
[100%] Built target binmake
```

3. Basic usage
Depending on the OS, the compiled binary in the build folder will be
   - Windows: binmake.exe
   - Linux: binmake

   Quick check after building: verify that the tool runs correctly. If it shows the usage or help, then the binary is compiled and runs properly.

```
# Linux
./binmake –help

# Windows
./binmake.exe --help
```

   - Single board / single revision builds

```
# Linux
./binmake --input=../platform_info.json --board=rzg2l-sbc --output=board-info.bin

# Windows (PowerShell)
./binmake.exe --input=../platform_info.json --board=rzg2l-sbc --output=board-
info.bin
```

   - All revisions of a board family: If the board has multiple revisions, use the all-revisions flag to generate one binary for each revision.

```
# Linux
./binmake --input=../platform_info.json --board=rzg2l-sbc --all-revisions

# Windows
./binmake.exe --input=../platform_info.json --board=rzg2l-sbc --all-revisions
```

The exact command-line syntax depends on the tool version. In all cases, the result is a binary board identification file that can later be flashed to the target.

15.1.4 Target-Side Flashing

Once board-info.bin has been generated on the host, it must be programmed into persistent storage on the target board.

Supported target storage typically includes:

- QSPI flash
- eMMC

The binary must be written to the predefined board-information region defined by the platform flash layout. This step can be performed using the universal flashing script

When using the universal flashing flow, add the binary name of the board identification to flash_images.json so it is written to the correct offset or partition.

Refer to section flash_images.json – File Overview and Usage for detailed steps on how to use the universal flashing script with this file, specifically how to define it in flash_images.json.

15.1.5 Target-Side Runtime Usage

During boot, firmware reads the flashed board information binary from storage and interprets it using a fixed structure.

TF-A usage: Trusted Firmware-A reads the board-info binary during early boot to:

- identify the board model,
- identify the board revision,
- select the correct board configuration,
- pass board-specific information to later stages where required.

U-Boot usage: U-Boot uses the board-info data to:

- identify the board,
- select the correct device tree,
- populate board-related boot variables such as MMC device, partition, and image addresses.

Based on the JSON field descriptions in the above section, the following U-Boot variables are populated from the board identification file on every reboot. Therefore, any manual changes made with setenv ...; saveenv will be overwritten on the next boot.

Table 74. Auto-managed U-boot environment variables

Variable	Example
model_string	rzg2l-sbc
revision_major	1
revision_minor	0
mmcdev	0
mmcpart	1
image_addr	0x48080000
env_addr	0x48000000
dtb_addr	0x480C0000
dtbo_addr	0x48100000

15.2 Flash Writer

The Flash Writer is a small, self-contained utility used to erase, partition, and program non-volatile storage (QSPI/OSPI via RPC, eMMC/SD via SDHI) before TF-A, U-Boot, and Linux are in place. It operates without a board DTB; instead, it relies on board-specific C definitions (pins, clocks, voltages, storage layout). Only the minimum set of blocks required to communicate on the console and access one storage path must be enabled.

15.2.1 What must be ported for a new board

1. New board on an already-supported SoC
   - Serial console (SCIF)
     - Usually reused from the existing SoC support.
     - Only change if the board uses a different SCIF instance or pins.
   - Storage path (RPC for QSPI/OSPI, or SDHI for eMMC/SD)
     - If the board uses a new flash device (such as a new SPI-NOR part or a different eMMC vendor), a device profile must be added if one does not already exist.
   - DDR initialization
     - Required in most cases, since each board usually uses a different DDR device and layout. Port the DDR parameter tables and training code for the specific DRAM on the board.
     - For very small images, the writer can run in SRAM-only mode, in which case DDR initialization can be skipped.

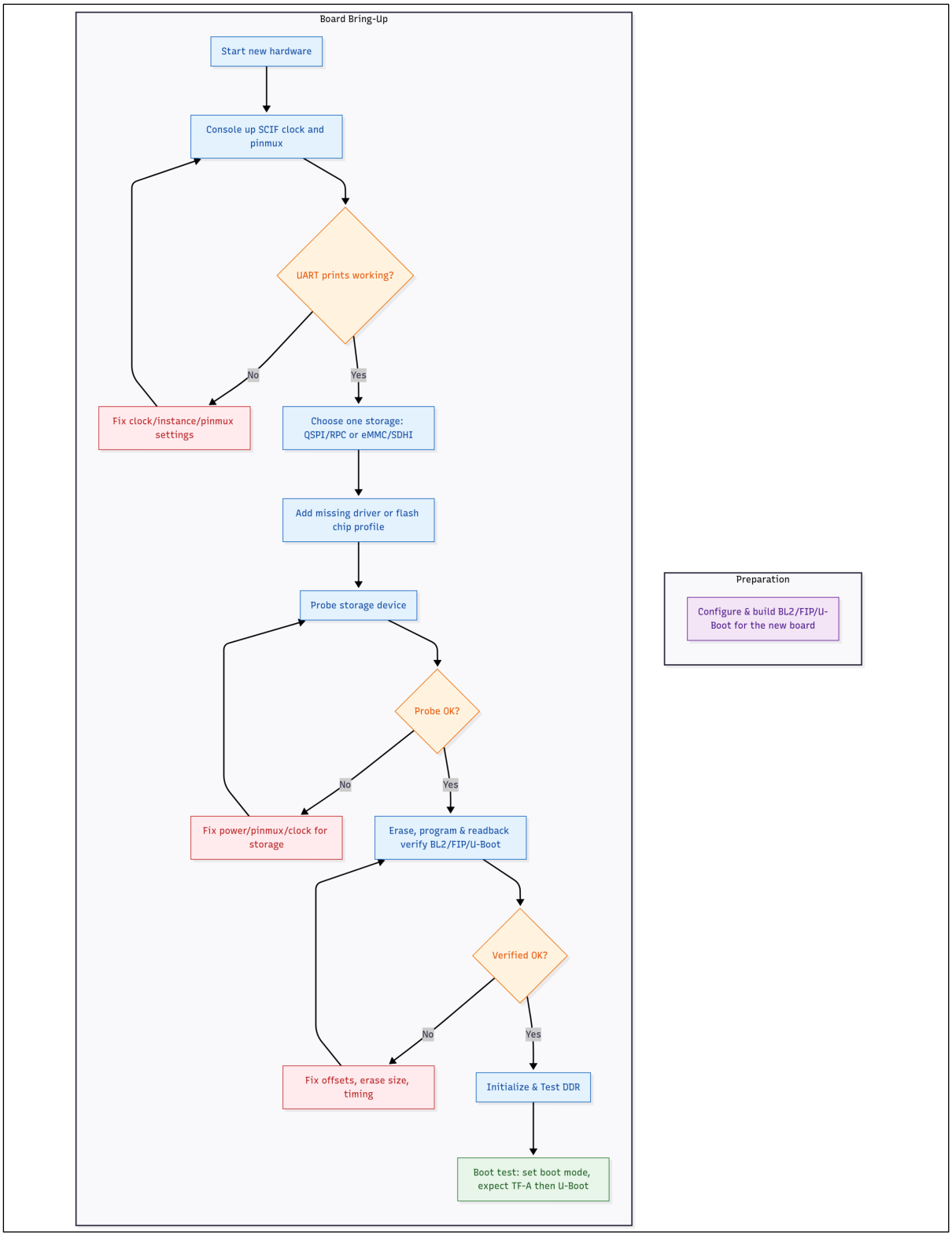


Figure 244. Flash-writer porting sequence

2. First board on a new SoC (SoC not yet supported):

This is the more comprehensive porting process and should be performed once per SoC. Individual boards can then reuse the resulting implementation. All items from case (1) plus SoC-level bring-up

- Base addresses & module IDs
  - Define SCIF, RPC/OSPI, SDHI, GPIO, reset, and clock registers for this SoC.
- Clock tree & resets
  - Implement CPG/Module Stop control for SCIF and storage blocks.
  - Configure dividers/sources required for console and storage.
- Low-level init
  - Provide delay/timer sources, watchdog handling, and cache/MMU policy.
  - Add handling for SoC-specific quirks (e.g., RPC PHY toggles, SDHI tuning/voltage switching).
- DDR init framework
  - First set of DDR parameter tables for this SoC family.

15.2.2 DDR bring-up for the Flash Writer

Flash Writer includes a DDR initialization library (e.g., drivers/ddr/common/ddr.c) that pulls in pre-generated parameter files. When external DRAM is required, add board-specific DDR parameter files and select them via preprocessor switches.

Unless the DDR connection and topology are identical to those on a Renesas reference board, parameters must be adapted. The DDR configuration generation tool, provided by Renesas as an Excel workbook, is used for this purpose.

Note: On some boards (e.g., RZ/V2H-EVK), Flash Writer operates entirely in internal SRAM. In these cases, DDR initialization is not required, and no DDR parameter files are needed. This section can be skipped when porting for such boards.

15.2.2.1 How to get DDR tools and Documentation for Flash Writer DDR

1. Go to renesas.com and navigate to the product page for the target board.
2. Look for the Easy Download Guide on the product or evaluation board page.
3. Inside the Easy Download Guide, look specifically for documents such as: DDR Configuration Generation Tool (usually an Excel workbook)

Step-4	Custom board development	Contents Type	Contents Name	RZ/G2L Link	RZ/G2LC Link	Note	Access levels
4-1	Documents and Simulation Models required for custom board design and development	Document	PCB Design and Verification Flow (for DDR-IF)	Please contact Disti/Sales		-	Non limitetion
		Document	Board circuit design checklist	Link		-	Non limitetion
		Document	PCB Design Guidelines for MIPI-CSI, MIPI-DSI and USB2.0	Link		-	Non limitetion
		Document,Tool	PCB Design Guideline for DDR4/DDR3L, DDR Config Generation Tool	Link		-	Non limitetion
		Document,Model	PCB verification guide for Core VDD, PDN model	Link		-	Non limitetion
		Document,Model	PCB verification guide for DDR4/DDR3L	Link	Link	-	Non limitetion
		Design Data	SMARC Module Board Design Data(Schematics,Layout,BOM)	Link	Link	Disclaimer*4	MyRenesas
		Design Data	Carrier Board Design Data(Schematics,Layout,BOM)	Link		Disclaimer*4	MyRenesas
		Document	RZ Family DRAM List	Link		-	Non limitetion
		Model	IBIS	Link	Link	Disclaimer*4	MyRenesas
		File	BSDL	Link		Disclaimer*4	MyRenesas
		Document	Guide to Using RGMII in Making an Ethernet-IF Connection	Link		-	Non limitetion
		Document	Power Consumption Measurement	Link	-	-	Non limitetion
		Document	Reference power consumption guide	Link	Link	-	Non limitetion
		Document	Thermal Management Guideline	Link	Link	-	Non limitetion
		Document	Mechanical Handling Guideline	Link		-	Non limitetion
		Document	Mount Conditions Reflow Soldering Conditions	Link		-	Non limitetion
		Document	Mount Conditions Baking Conditions	Link		-	Non limitetion
		Document	Semiconductor Package Mount Manual	Link		-	Non limitetion
		Document	Lifetime Guideline	Link		-	Non limitetion
		Device Info	Power Management IC	Link		-	Non limitetion
		Device Info	Clock Generator IC	Link		-	Non limitetion
		Document	R9A07G044XXXGBG Reliability Report	Link	Link		Non limitetion
		Document	Power calculation sheet and guide	Please contact Disti/Sales		-	Please contact Disti/Sales
		Document	Low Power Guideline	Please contact Disti/Sales		-	-

*1: Letter of Acknowledgement
*2: End User Lisence Agreement
*3: Software Lisence Agreement
*4: Disclaimer Letter

Figure 245. Example DDR generation tool of RZ/G2L-EVK

Download the ZIP file and open the Excel workbook to generate DDR parameter files (e.g., param_mc.c, param_swizzle.c) using the Renesas-style generator. Follow the next section for details.

15.2.2.2 DDR parameter generation with the Excel Workbook

The workbook generates C source files that encode DDR initialization parameters for a specific hardware design. These files are compiled into Flash Writer to apply the exact electrical and mapping settings expected by the board.

(1) Prerequisites

- Schematic and PCB layout finalized (or at least stable DDR routing and terminations).
- Known DDR device configuration (bus width, density, rank count, timing grade).
- Results or guidance from signal-integrity/layout analysis for drive strength and impedance.
- A target SoC family path in Flash Writer (ddr/g2l, ddr/v2l, or ddr/v2h).

(2) Configuration Model (how selections work)

There are two mutually exclusive ways to define the configuration:

6. Condition/Connection pairing
   - Condition: Describes operating constraints and memory component characteristics.
   - Connection: Describes board-level wiring (lane usage, byte lanes, swap patterns).
   - Use this path when a published Condition/Connection pair exactly matches the hardware.
7. Topology
   - A consolidated preset that captures the wiring pattern when a matching Condition/Connection pair is unavailable.
   - Use this path when the design deviates from predefined Condition/Connection entries.

Note: choose either a Condition/Connection pair or a Topology, not both.

(3) Tab-by-Tab Guide

1. Guide (usage notes) tab
   - Provides general workbook instructions.
   - Code-change examples on this tab target Trusted Firmware-A (TF-A). Integration steps for Flash Writer differ; follow the Flash Writer placement guidance in this document when adding the generated files.
2. 01_Condition (reference) tab
   - Lookup sheet for valid Condition# values.
   - No edits are expected on this tab.
3. 02_Connection (reference) tab
   - Lookup sheet for valid Connection# values.
   - No edits are expected on this tab.
4. 03_Topology (reference) tab
   - Lookup sheet for valid Topology# values.
   - No edits are expected on this tab.
5. 04_Analog (board-specific electricals) tab
   - Editable fields for drive strength, impedance, ODT, and related analog parameters.
   - Values should mirror what the layout/SI analysis determined for the design so that DDR init writes consistent register settings.
   - Modify only the column corresponding to the selected entry in GenParam; other columns are ignored during generation.
6. 05_CA_Remap (command/address mapping) tab
   - Adjust when the command/address pin mapping differs from Renesas EVK examples

- Update only the column matching the active Condition/Connection or Topology selection in GenParam.
- Keep mappings for pins that are unused on the board (e.g., smaller DDR device). DDR pins are dedicated; blank cells can trigger validation errors in the workbook.

7. GenParam (control panel) tab
   - All selections are made here via dropdowns in highlighted cells.
   - For Condition/Connection flow: set Topology = “Other”, then pick Condition# and Connection# entries.
   - For Topology flow: select a Topology#; the Condition/Connection indicators on the page update automatically for reference.
   - Generate param files to export the C sources to the workbook’s folder once all the configuration is complete

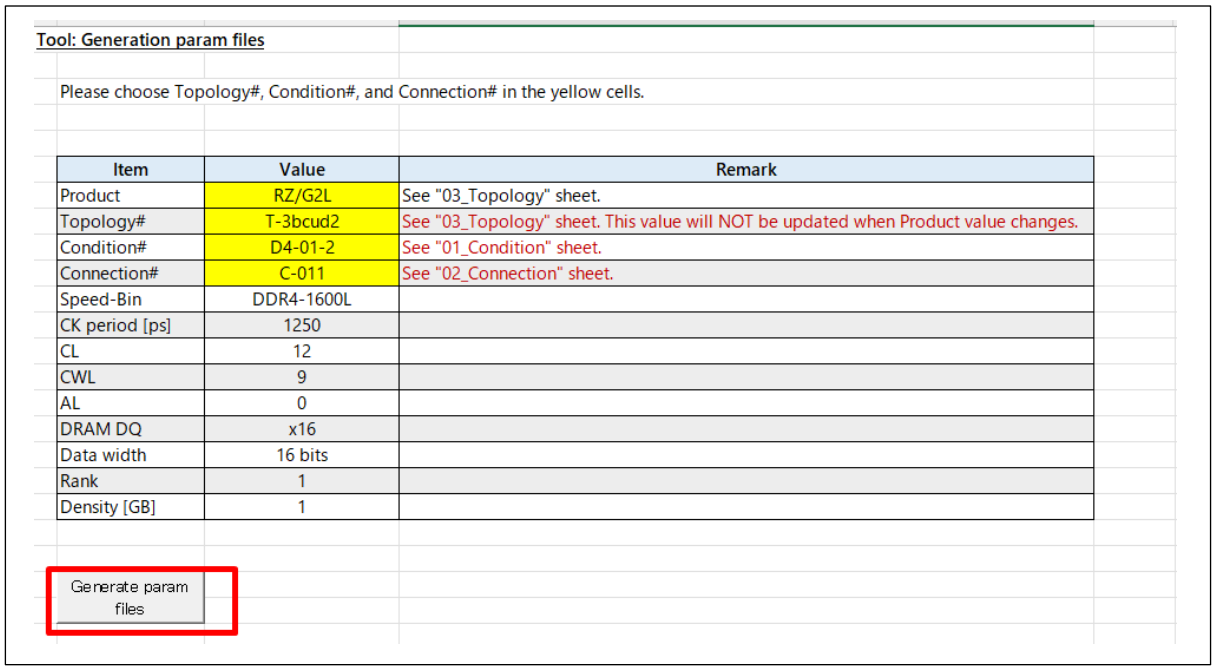


Figure 246. DDR GenParam sheet

15.2.2.3 Collect output and integrate into Flash-writer sources

When the **Generate param files** button is pressed, two C files are produced in the workbook’s directory:

- param_mc.c – memory controller configuration (timings, drive/ODT, controller registers).
- param_swizzle.c – lane/bit mapping and any address/command swizzles.

(1) Recommended naming (follow guide tab)

Use identifiers from GenParam to make variants explicit:

- param_mc.c → param_mc_{Connection#}_{Condition#}.c
  Example: param_mc_C-011_D4-01-2.c
- param_swizzle.c → param_swizzle_{Topology#}.c
  Examples: param_swizzle_T3bcud.c, param_swizzle_other.c

Place the generated files in the Flash-Writer source tree; reuse an existing equivalent file if it matches the required parameters, otherwise add a new, distinctly named file (avoid overwriting unless intentionally replacing it for all boards).

- param_mc_*.c → flash-writer/drivers/ddr/g2l/ (or ddr/v2l or ddr/v2h, matching the SoC family)

- param_swizzle_*.c → flash-writer/drivers/ddr/common/

(2) Add a new board to the Flash-Writer Makefile

Open the Flash-Writer makefile and add a board stanza that sets the device family, flash type, DDR type/size, and swizzle/topology label.

Example inputs

- Board name: RZV2L_EVK_A1
- DEVICE (SoC family): RZV2L
- FLASH_TYPE: QSPI
- DDR_TYPE: DDR4
- DDR_SIZE: 2GB
- SWIZZLE: T3BCUD

If an identical board stanza already exists in the makefile, reuse it; otherwise, add a new stanza with the correct values for the hardware.

Open makefile

```
renesas@builder-pc ~/flash-writer$ vim makefile
```

makefile stanza (example)

```
# Default
ifeq ("$(BOARD)", "")
BOARD = RZG2L_SMARC
endif

# ------------------------------------
# NEW BOARD (example): RZV2L_EVK_A1
# ------------------------------------
ifeq ("$(BOARD)", "RZV2L_EVK_A1")
FILENAME_ADD = _RZV2L_EVK_A1
DEVICE      = RZV2L          # SoC family → controls param_mc path (ddr/v2l)
FLASH_TYPE  = QSPI           # or EMMC, as used by the board
DDR_TYPE    = DDR4           # memory type
DDR_SIZE    = 2GB            # 512MB | 1GB | 2GB | 4GB | …
SWIZZLE     = T3BCUD         # must correspond to an available param_swizzle_*.c
Endif

# Map to ddr.c macros
ifeq ($(DDR_TYPE),DDR4)
CFLAGS += -DDDR4=1
endif

# DDR size
ifeq ($(DDR_SIZE),2GB)
CFLAGS += -DDDR_SIZE_2GB=1
endif

# Swizzle/topology
ifeq ($(SWIZZLE),T3BCUD)
CFLAGS += -DSWIZZLE_T3BCUD=1
endif
```

(3) Map Makefile variables to the C preprocessor macros used by ddr.c

Ensure the build defines exactly one DDR size macro and one swizzle macro that matches cases in ***drivers/common/ddr.c.***

Before adding new entries, check whether the required size and swizzle/topology are already supported:

- If an existing definition and file already exist
  - Confirm that ddr.c contains a matching macro (e.g., DDR_SIZE_2GB, SWIZZLE_T3BCUD) and the corresponding #include "param_mc_….c" / #include "param_swizzle_….c".
  - In the makefile, enable exactly one matching size macro and one swizzle macro (via CFLAGS or variable-to-macro mapping).
  - Generated files may be omitted if the existing shipped files match the design.
  - No changes to ddr.c are required in this case.
- If no matching definition/file exists

- Extend ddr.c by adding one #elif in the size selector and one #elif in the swizzle selector to include the new filenames.

```
/* Size selector (add near other DDR_SIZE_* cases) */
#if (DDR4 == 1)
#if (DDR_SIZE_2GB == 1)
#include "param_mc_C-010_D4-01-2.c" // controller params for 2GB
#endif
#if (SWIZZLE_T3BCUD == 1)
#include "param_swizzle_T3bcud.c" // swizzle/topology T3BCUD
#endif
#endif
```

Perform a clean rebuild of Flash Writer for a new example board named RZV2L_EVK_A1 to ensure the new parameters are compiled in.

```
renesas@builder-pc ~/flash-writer$ make BOARD=RZV2L_EVK_A1 -j"$(nproc)"
```

Before running the build command, ensure the cross-compile environment is set up. Please follow section How to extract the eSDK

15.2.3 Add a New SPI Flash

This subsection outlines the minimal changes required to register an additional SPI/QSPI device.

15.2.3.1 Header Updates

Confirm that the manufacturer identifier for the target flash vendor is present in the file **(include/dg_modul4.h)**. Typical entries include:

```
#define CYPRESS_MANUFACTURER_ID         0x01    /* Cypress      */
#define WINBOND_MANUFACTURER_ID         0xEF    /* Winbond      */
#define MACRONIX_MANUFACTURER_ID        0xC2    /* Macronix     */
#define MICRON_MANUFACTURER_ID          0x20    /* Micron       */
#define DIALOG_MANUFACTURER_ID          0x1F    /* Dialog       */
```

If the vendor is not listed, add:

- A manufacturer ID macro: *_MANUFACTURER_ID
- A device ID macro: DEVICE_ID_*
- A sector-erase size macro (SA) if a new granularity is required

```
/* Vendor and device identifiers */
#define ISS_MANUFACTURER_ID 0x9D         /* Example vendor ISS */
#define DEVICE_ID_IS25WP512 0x701A       /* Example Device ID */

/* Sector erase size (SA) definitions */
#define SA_4KB 0x01000 /* 4 KiB = 4096 bytes */
```

15.2.3.2 Register the Device (table-driven flow)

Once the manufacturer and device IDs are defined (see section Header Updates), the device must be registered in the Flash Writer. Registration is done by updating the vendor's device table in

dg_modul4.c. If the vendor is not yet supported, create a new device table and add an entry to supportedManufacturers[].

Check if the vendor already exists

Look in the SPI flash driver sources for an existing device table such as winbondDevices, macronixDevices, micronDevices, or issDevices. The currently supported vendors are:

- Cypress
- Winbond
- Macronix
- Micron
- Dialog
- ISS (Integrated Silicon Solution, Inc.)

(1) Add a Device to an Existing Vendor

If the vendor is already listed above, extend the corresponding device table by adding one line for the new device. Example (adding a new ISS device):

```
// Example: extend ISS table with an additional part
const FlashDeviceConfig issDevices[] __attribute__((section(".rodata"))) = {
    { DEVICE_ID_IS25WP256, "IS25WP256", SA_4KB,  TOTAL_SIZE_32MB },
    { DEVICE_ID_IS25WP512, "IS25WP512", SA_4KB,  TOTAL_SIZE_64MB }  // ← new device
};
```

- When adding a device, choose values according to the datasheet:
  - Sector erase size: one of SA_* macros, per datasheet.
  - Total size: one of TOTAL_SIZE_* macros, per datasheet.
  - Name: short, human-readable part name for logs (e.g., "IS25WP512").

(2) Add a New Vendor

If the vendor does not exist in the list above, create a new device table and register it in supportedManufacturers[].

Example (new vendor “AcmeSemi”):

```
// New device table
const FlashDeviceConfig acmesemiDevices[] __attribute__((section(".rodata"))) = {
    { DEVICE_ID_ACME128, "ACME128", SA_64KB, TOTAL_SIZE_16MB },
    { DEVICE_ID_ACME256, "ACME256", SA_64KB, TOTAL_SIZE_32MB },
};

// Add to supportedManufacturers[]
const Manufacturer supportedManufacturers[] __attribute__((section(".rodata"))) = {
    { CYPRESS_MANUFACTURER_ID,  "Cypress",  cypressDevices,  sizeof(cypressDevices)/sizeof(cypressDevices[0]) },
    { WINBOND_MANUFACTURER_ID,  "Winbond",  winbondDevices,  sizeof(winbondDevices)/sizeof(winbondDevices[0]) },
    { MACRONIX_MANUFACTURER_ID, "Macronix", macronixDevices, sizeof(macronixDevices)/sizeof(macronixDevices[0]) },
    { MICRON_MANUFACTURER_ID,   "Micron",   micronDevices,   sizeof(micronDevices)/sizeof(micronDevices[0]) },
    { DIALOG_MANUFACTURER_ID,   "Dialog",   dialogDevices,   sizeof(dialogDevices)/sizeof(dialogDevices[0]) },
    { ISS_MANUFACTURER_ID,      "ISS",      issDevices,      sizeof(issDevices)/sizeof(issDevices[0]) },
    { ACMESEMI_MANUFACTURER_ID, "AcmeSemi", acmesemiDevices, sizeof(acmesemiDevices)/sizeof(acmesemiDevices[0]) } // new
vendor
};
```

After rebuilding, the Flash Writer will automatically detect and configure devices from this vendor or a new device.

15.3 TF-A (BL2/BL31)

As described in section Arm Trusted Firmware-A (TF-A), TF-A provides the BL2 and BL31 stages of the boot flow. In the RZ Common System, TF-A ensures early memory, clock, and boot device initialization before handing control to U-Boot and Linux.

The RZ Common BL2/BL31 binary supports multiple SoCs (RZ/G2L, RZ/V2L, RZ/V2H) and is designed to remain board-agnostic. Board-specific details such as DDR initialization, SPI/xSPI flash parameters, and system configuration are provided at runtime through a Firmware Configuration Framework (FCONF) device tree.

When adding a new board, the task is to provide these parameters in a structured way. To simplify this process, TF-A includes the dts_creator tool, which reads a YAML board configuration, applies the appropriate SoC template, extracts DDR and SPI parameters, and generates a valid DTS for BL2.

15.3.1 Firmware Configuration Framework

FCONF (Firmware Configuration Framework) is a TF-A mechanism that allows platform settings to be passed at runtime through a structured device tree blob (DTB) rather than being hard-coded into firmware binaries. Instead of recompiling BL2 for every new board, generic binaries (BL2/BL31) remain board-agnostic, while the FCONF DT provides:

- Memory parameters (DDR type, initialization tables)
- Boot device parameters (SPI/xSPI, eMMC, SD offsets)
- Clock and pinctrl defaults
- Security and system controller options

At boot, BL2 loads the FCONF DTB and uses its properties to configure hardware before handing off to BL31 and U-Boot. This makes it possible to support many boards within the same SoC family using one common BL2 binary — only the FCONF DT changes per board.

15.3.2 Tool Location and Layout

The dts_creator tool is provided under rz-atf/tools/renesas/ and automates the generation of board device tree files for BL2. Instead of manually editing complex DTS templates, users describe their board in a YAML config, and the tool fills in the correct values.

```
rz-atf/tools/renesas/dts_creator
├── configs/              # Board configuration YAMLs
│   └── <board-name>.yaml
├── dts_creator.py        # Main generator script
├── schemas/              # YAML schema for validation
│   └── rz-boards.yaml
└── template/             # SoC template DTS files
    └── <board-name>.dts
```

Each part has a defined role:

- configs: describe boards
- schemas: validate generated DTS
- template: provide SoC boilerplate
- Python script merges everything into the final DTS.

15.3.3 Prerequisites

Before using dts_creator, the build environment must have a supported Python runtime and required libraries as follows:

- Python 3.8 or later – required to run the script
- PyYAML – used for parsing YAML configs.
- Core requirements:
  - **On Windows**, MSYS2 is required to build and run dtc and related tools.
    - Download and install MSYS2 from MSYS2 (check the Installation section and download the .exe installer)
    - Follow the setup steps to complete the installation
    - Once the installation is complete, open the MSYS2 shell and run:

```
PS C:\Users\renesas> pacman -S base-devel mingw-w64-x86_64-gcc
PS C:\Users\renesas> git clone https://git.kernel.org/pub/scm/utils/dtc/dtc.git
PS C:\Users\renesas> cd dtc
PS C:\Users\renesas> make dtc
```

After building, dtc.exe will be in the dtc directory; you can run it from Windows CMD/PowerShell or add its location to your PATH.

- dtschema for dt-validate. Firhbgbst, install Microsoft Visual C++ Build Tools
  - Download the installer from: Microsoft C++ Build Tools - Visual Studio
  - Run the installer and select the Desktop development with C++ workload.
  - After installation, verify by running:

```
PS C:\Users\renesas> cl.exe
```

It should print the Microsoft compiler version (14.0 or newer).

  - Install Python packages. From PowerShell, run:

```
PS C:\Users\renesas> pip install pyyaml dtschema
```

  - **On Linux**, install system packages first. Open the terminal and run the following commands. If Python 3.12 is in use, set up a virtual environment first.

```
renesas@builder-pc:~/$ sudo apt install python3.12-venv
renesas@builder-pc:~/$ python3 -m venv .venv
renesas@builder-pc:~/$ source .venv/bin/activate
```

Then run the following commands

```
renesas@builder-pc:~$ sudo apt install device-tree-compiler swig python3-dev
renesas@builder-pc:~$ pip3 install pyyaml dtschema
```

15.3.4 Adding a New Board

When adding a new board, the starting point is a YAML configuration file under tools/renesas/dts_creator/configs/. This file describes the SoC family, board name, memory type, and boot flash device. dts_creator uses these fields to select the right DTS template and fill in hardware parameters.

3. Create a new configuration file (myboard.yaml) for your board under:

rz-atf/tools/renesas/dts_creator/configs

Replace myboard.yaml with a descriptive name for the actual board (for example, rzg2l_sbc.yaml, rzv2h_custom.yaml). By default, the DTS generated will be written to:

```
rz-atf/fdts/<soc>-<board>.dts
```

4. Field in the required fields:

```
soc: rzg2l            # SoC family: rzg2l | rzv2l | rzv2h
board: sbc            # Board identifier: evk | sbc
# output: fdts/myboard.dts   # Optional override of the output path

spi:
  protocol: spi_multi # Boot flash protocol: spi_multi (QSPI) | xspi (Octa)
  flash_chip: IS25WP256  # Exact flash family name supported by TF-A

ddr:
  type: ddr4          # Memory type: ddr4 | lpddr4
  ddr sources:
    - plat/renesas/rz/soc/g2l/drivers/ddr/param_mc_myboard.c
    - plat/renesas/rz/common/drivers/ddr/param_swizzle_myboard.c
```

This example defines a board based on RZ/G2L, booting from SPI flash, with DDR4 memory, and DDR parameters coming from two .c files under the TF-A source tree.

YAML Fields

- soc: Defines the SoC family of the board. Supported values are:
  - rzg2l — RZ/G2L family (includes G2L, G2LC, G2UL variants)
  - rzv2l — RZ/V2L family
  - rzv2h — RZ/V2H family
- board: A simple identifier for the board type, e.g., evk, sbc, or a custom name. Used to name the generated DTS.
- output (optional): Path where the generated .dts will be written. If omitted, a default path under fdts/ is used.
- spi.protocol:

Specifies the boot flash protocol:
  - spi_multi — Standard QSPI/quad-SPI flash
  - xspi — Octa-SPI flash (used in RZ/V2H and some RZ/V2L designs)
- spi.flash_chip: The specific flash device part number (e.g., IS25WP256). This must match the supported flash driver names in TF-A. For new parts, add a matching header with spi_multi_reg_values.h under plat/renesas/rz/common/include/drivers/spi_multi.
- ddr.type:
  Memory type used on the board:
  - ddr4 — DDR4 SDRAM
  - lpddr4 — Low-power DDR4
- ddr sources: A list of .c files containing DDR configuration tables (memory controller init, PHY init, swizzle settings). These files are parsed by dts_creator and embedded into the generated DTS.
  - Each new board with a unique memory device must provide its own DDR parameter sources under plat/renesas/rz/.../drivers/ddr/.
  - Refer to section DDR bring-up for the Flash Writer for detailed guidance on how to create new DDR parameter files for a new board.

15.3.5 Usage

After preparing a board YAML configuration, the next step is to run the dts_creator tool. The tool parses the YAML file, selects the appropriate SoC template, extracts DDR and SPI parameters from the specified sources, and generates a complete DTS for BL2 FCONF.

From the root of the Linux TF-A repository:

On Linux: Open a terminal and run the following commands:

```
renesas@builder:~/$ cd rz-atf/tools/renesas/dts_creator
renesas@builder:~/rz-atf/tools/renesas/dts_creator$ python3 dts_creator.py -c \
configs/myboard.yaml
```

On Windows: Open one of the following terminals with Administrator privileges:

- PowerShell
- Git Bash
- MobaXterm

Navigate to the host/tools/bootloader_flasher directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas\Desktop> cd ~/rz-atf\tools\renesas\dts_creator
PS C:\Users\renesas\rz-atf\tools\renesas\dts_creator$ py dts_creator.py -c \
configs\myboard.yaml
```

Optional: -o <output-path> overrides the output declared in the config.

After generating a DTS, validate it against the Renesas schema:

```
$ cd ~/rz-atf/fdts
$ dtc -I dts -O dtb -o <temp>.dtb <path-to-generated.dts>
$ dt-validate --schema tools/renesas/dts_creator/schemas/rz-boards.yaml\
<temp>.dtb
```

The DTS should pass without warnings or errors before it is committed.

Example:

```
# On Linux
renesas@builder-pc:~$ cd ~/rz-atf/fdts
renesas@builder-pc:~/rz-atf/fdts$ dtc -I dts -O dtb -o myboard.dtb myboard.dts
renesas@builder-pc:~/rz-atf/fdts$ dt-validate \
--schema ../tools/renesas/dts_creator/schemas/rz-boards.yaml myboard.dtb


# On Windows
PS C:\Users\renesas> cd ~/rz-atf/fdts
PS C:\Users\renesas\rz-atf/fdts> dtc -I dts -O dtb -o myboard.dtb myboard.dts
PS C:\Users\renesas\rz-atf\fdts> dt-validate \
--schema ../tools/renesas/dts_creator/schemas/rz-boards.yaml myboard.dtb
```

After generating the device tree, it can be passed to the universal flashing script, which merges it into the common BL2 at flash time. Refer to section flash_images.json – File Overview and Usage for detailed steps on how to use the universal flashing script with this file, specifically how to define it in flash_images.json (atf_dtb) field.

15.4 U-Boot (BL33)

This release ships a SoC-common U-Boot binary without a DTB (u-boot-nodtb.bin). Adding a new board usually requires only minor modifications to the U-Boot source code, mainly in the early initialization phase (e.g., board_early_init_f(), misc_init_r(), or ft_board_setup()) to handle board-specific logic such as power sequencing or clock pinmuxing.

U-Boot separates responsibilities into three scopes, aligned with initialization phases:

- SoC (early init): minimal, silicon-wide steps to reach a console and parse the Device Tree.
- BSP / drivers (late init): device behavior configured by the Device Tree.
- Board (late init): wiring- and policy-specific choices read from the Device Tree.

Because most configurations are expressed in the Device Tree, the U-Boot binary remains portable across boards, and only minimal source changes are needed for unique initialization requirements.

15.4.1 Initialization Model

15.4.1.1 SoC-specific - early init (s_init)

Purpose: stabilize silicon so that U-Boot can run, print, and parse DT.

Typical content

- Minimal clocks/resets and bus enable.
- Essential UART pinmux (only if required for early console).
- Watchdog safety (SoC-level).
- Mandatory SoC errata/workarounds that must precede any driver’s access.

15.4.1.2 BSP-specific (drivers)

Purpose: reusable device support for multiple boards.

Typical content

- SPI-NOR/PMIC/PHY drivers and generic quirks.
- DT-configurable features shared across platforms.

15.4.1.3 Board-specific — late init (board_late_init / last_stage_init)

Apply behavior that depends on wiring or fitted options and can wait until devices are probed.

Typical content

- Read board EEPROM (e.g., MAC/IDs) and set env.
- PMIC/I²C sequences defined by DT.
- Enable USB/Ethernet once pinctrl/rails.
- Optional PHY tuning, LEDs, diagnostics (often in last_stage_init).

15.4.2 How soc_id and board_id are used

TF-A provides two identifiers that U-Boot consumes:

- extern u64 soc_id; identifies RZ/G2L, RZ/V2L, or RZ/V2H.
  - RZ_SOC_RZG2L: RZ/G2L SoC
  - RZ_SOC_RZV2L: RZ/V2L SoC
  - RZ_SOC_RZV2H: RZ/V2H SoC
- extern u64 board_id; identifies the specific board (EVK, SBC, custom, etc.). This value is obtained from the Board Identification data, which corresponds to the model_id field defined in the JSON file. See Board Identification

board/renesas/rz-cmn/rz-cmn.c branches on these IDs to select the minimal per-board actions.

15.4.3 Steps to Add a Board

1. Create a board device tree for the board
   - Add arch/arm/dts/<board>.dts
   - Include the correct SoC .dtsi and any common carrier/module *-u-boot.dtsi
2. Describe the hardware
   - Enable the console UART (chosen.stdout-path).
   - Enable only peripherals that exist on the board (SDHI/eMMC, Ethernet, I²C, USB, RPC/SPI-NOR, etc.).
   - Provide pinctrl, clocks, and regulators as needed.
3. Example: Minimal board DT template

```
// SPDX-License-Identifier: GPL-2.0
/dts-v1/;

/* TODO: pick the right SoC include:
 *   RZ/G2L: r9a07g044l.dtsi
 *   RZ/V2L: r9a07g054l.dtsi
 *   RZ/V2H: r9a09g057h.dtsi
 *   .. other SoCs
 */
#include "r9a07g044l.dtsi"
#include "r9a07g044l-u-boot.dtsi"

 / {
```

```
        model = "<board name>";
        /* Recommended: "<vendor-prefix>, <board-name>", then the SoC compatible */
        /* Example */
        compatible = "acme,foo-rzg2l", "renesas,r9a07g044l", "renesas,rzg2l";

        chosen {
                /* TODO: set to the UART wired (serial0/1/etc.) and baud */
                stdout-path = "serial0:115200n8";
        };

        /* TODO: set actual DRAM base/size for the board
         * Replace the Marco with actual DDR base and size
        /*
        memory@48000000 {
                device_type = "memory";
                reg = <0 DDR_BASE_ADDR 0 DDR_AVAILABLE_SIZE>;
        };
};

/* ---------- Enable and adjust only what is actually present on the board ---------- */

/* Console UART */
&scif0  { status = "okay"; };

/* SD/eMMC */
&sdhi0  { status = "disabled"; /* bus-width = <4>; or <8>; */ };
&sdhi1  { status = "disabled"; };

/* Ethernet (add pinctrl, phy-mode, and phy node if used) */
&eth0   { status = "disabled"; };

/* I2C (add devices if used) */
&i2c0   { status = "disabled"; };

/* RPC/SPI-NOR (add flash child if used) */
&spibsc { status = "disabled"; };

/* USB (EHCI) */
&ehci0  { status = "disabled"; };
&ehci1  { status = "disabled"; };

/* Watchdog */
&wdt0   { status = "disabled"; };
```

4. Build and assemble

```
# Build DTB
make ARCH=arm CROSS_COMPILE=<triplet-> arch/arm/dts/<board>.dtb

# Assemble final image (OF_SEPARATE)
cat u-boot-nodtb.bin arch/arm/dts/<board>.dtb > u-boot.bin
```

15.4.4 Board-Specific Code Hooks

This section describes the three optional places to add minimal board code in board/renesas/rz-cmn/rz-cmn.c.

15.4.4.1 Early init – s_init()

Do the minimum register work required so U-Boot can run reliably, print to console (if needed), and parse the device tree—before any driver accesses hardware.

- A clock/reset/pinmux must be applied before DM/driver probe (e.g., gating a domain that holds bus fabric).
- SoC errata/workarounds that must precede any peripheral access.
- Early console pinmux only if necessary for debug.

```
static void s_init_myboard(void)
{
    /* Minimal, SoC-safe tweaks only */
    /* e.g., toggle PFC write-protect, enable one clock/reset, etc. */
}

void s_init(void)
{
    /* Existing code */
    if (board_id == myboard) {
      s_init_myboard();
    }
}
```

15.4.4.2 Board Init – board_init()

Perform a concise per-board setup after relocation and basic subsystems are ready, but before late policies. Typical use: a PHY/USB/PMIC enables a sequence that drivers can't (or shouldn't) do automatically.

```
int board_init(void)
{
    gd->bd->bi_boot_params = CONFIG_SYS_TEXT_BASE + 0x50000;

    if (board_id == BOARD_ID_MYBOARD) {
        // board_usb_init_myboard();   /* short bring-up sequence */
        // board_pmic_i2c_init();      /* minimal sanity config */
    }
    return 0;
}
```

15.4.4.3 Board_late_init() – policies & ID after drivers' probe

This is an optional stage that applies board policies and wiring-dependent actions after devices and buses exist under the driver model. If the added board does not require any late policies beyond what the device tree and drivers already provide, this stage can be omitted entirely for a new board.

Common uses.

- Read board EEPROM (e.g., MAC addresses) and export to env
- Watchdog re-init
- GPIO indicators or simple diagnostics

```
int board_late_init(void)
{
    /* Existing code */
    if (board_id == BOARD_ID_MYBOARD) {
        /* Final, board-specific tweaks (e.g., PHY delay configuration). */
        /* TODO: add your writes/calls here */
    }
    return 0;
}
```

15.4.4.4 last_stage_init() – Final tweaks

last_stage_init() is an optional U-Boot board hook that runs near the end of BL33 bring-up, after driver model devices have been probed and board_late_init() has executed.

- PHY register tuning or strap fixes
- Enabling auxiliary clocks/peripherals not strictly required for boot
- Final diagnostics

```
int last_stage_init(void)
{
    /* Existing code */
    if (board_id == BOARD_ID_MYBOARD) {
        /* Example: PHY delay configuration */
    }
    return 0;
}
```

15.4.5 Adding BSP Drivers

Add or modify BSP components only when the hardware cannot be supported by existing drivers and Device Tree (DT) alone.

When a driver update/addition is required

- New or unsupported device: SPI-NOR without usable SFDP/ID, new PMIC/regulator, clock chip, Ethernet PHY, USB/bridge device.
- Mandatory behaviors that cannot be modeled in DT: required register sequences, timing workarounds, addressing modes, or errata not exposed via DT properties.
- Probe succeeds but operation fails; device binds yet needs a quirk/opcode/map fix to function reliably.

15.5 Linux Kernel

Linux on the RZ Common System is device tree (DT) centric. The SoC .dtsi file describes common silicon blocks, while each board contributes a small .dts file that wires those blocks together (pins, voltages, clocks, PHYs, GPIOs, supplies). Most bring-up work involves authoring a board device tree and enabling the correct drivers.

Direct register writes are not used in Linux DT. Instead, standard bindings and driver properties should be used.

15.5.1 Steps to Add a Board

1. **Create a board DT**
    - Create arch/arm64/boot/dts/renesas/<board>.dts (path may vary if using an out-of-tree DTS package).
    - Include the correct SoC .dtsi (e.g., r9a07g044l.dtsi, r9a07g054l.dtsi, …) and any board-shared .dtsi you maintain.
2. **Describe the hardware**
    - **Memory:** add a memory@… node with the actual base/size; declare any reserved-memory regions needed by firmware.
    - **Chosen/console:** set /chosen { stdout-path = "serialX:115200n8"; }; to match the wired UART.
    - **Pin control:** define pin groups under &pinctrl and reference them from each peripheral via pinctrl-0.
    - **Clocks/resets:** reference SoC clock providers via clocks = <&…> (no hardcoded clock writes).
    - **Regulators:** model rails (fixed or PMIC-based) and hook them up via vmmc-supply, vqmmc-supply, vdd-supply, phy-supply, etc.
    - **Peripherals:** enable only what exists on the board; provide properties the drivers expect:
        - **MMC/SDHI:** bus-width, cd-gpios, wp-gpios, vmmc-supply, vqmmc-supply, non-removable (for eMMC), timing caps (cap-sd-highspeed, mmc-hs200-1_8v, etc.).
        - **Ethernet:** phy-mode (rgmii, rmii, …), phy-handle (or fixed-link), optional internal delay properties (rx-internal-delay-ps, tx-internal-delay-ps) depending on PHY/MAC, and MDIO bus if external PHYs are present.
        - **I²C:** mark controllers okay; add on-bus devices (EEPROMs, PMICs, sensors) with their compatible and reg.
        - **USB:** enable controller(s); add vbus-supply if power switching is controlled by a regulator/GPIO.
        - **SPI/RPC (xSPI/NOR):** enable the bus and add the flash node with compatible = "jedec,spi-nor" (or vendor-specific); configure bus widths and modes per binding.
        - **Watchdog, GPIO LEDs/keys, UARTs:** enable and wire as needed.
3. **Add to Makefile**
    - Kconfig/SoC enablement

- Ensure the correct CONFIG_ARCH_* option is enabled for the SoC family (e.g., CONFIG_ARCH_R9A07G044, CONFIG_ARCH_R9A07G057). This ensures the kernel builds DTS files for the right SoC.

- Device tree Makefile
  - Edit the arch/arm64/boot/dts/renesas/Makefile
  - Add a new .dts so it is compiled into a .dtb
  - Example

```
# For R9A07G044 SoC
dtb-$(CONFIG_ARCH_R9A07G044) += <new-board>.dtb
```

4. **Build the DTBs**
   Run the kernel build to generate device trees

```
renesas@builder-pc:~/linux-rz$ make renesas_defconfig
renesas@builder-pc:~/linux-rz$ make ARCH=arm64 \
CROSS_COMPILE=aarch64-poky-linux- \
dtbs
```

   This way, when building the kernel, the board's DTB will be automatically generated in the arch/arm64/boot/dts/renesas/ output directory.

15.5.2 Kernel Device Tree Selection

After completing all BSP steps, the final task is to let U-Boot know which board's device tree to load. U-Boot's environment already supports user-defined hooks. To load a new device tree for a new board without rebuilding U-Boot, add a rule in uEnv.txt that sets fdtfile when the board's identifiers match.

Open the uEnv.txt placed under partition 1 (FAT32) and add the following condition

```
# Set fdtfile to "new-board.dtb" when the board model and revision match.
# This tells U-Boot which device tree to load and pass to the kernel at boot.
# Replace "model-string", "1", and "0" with your board's actual identifiers.

fdt_user_cases=if test "${model_string}" = "model-string" -a "${revision_major}"
= "1" -a "${revision_minor}" = "0"; then setenv fdtfile new-board.dtb; fi
```

How it works:

- fdt_user_cases: Executed after the built-in selection table. When fdtfile remains unset, this hook can be used to assign a DTB.
- Identifiers: model_string, revision_major, and revision_minor come from the board-info blob generated from platform_info.json in the build system (see section JSON Specification. These values must match the identifiers for the target hardware.

> Note: If the condition matches the board identification, then new-board.dtb will be loaded during kernel boot. The new-board.dtb file should be placed under partition 1 (FAT32) at /dtb/renesas/new-board.dtb.

16. Appendix

16.1 Factory Firmware Flashing Using Serial Downloader (SCIF) Mode

In most cases, the RZ boards are preloaded with the latest firmware. The preferred method of updating the firmware is through the SD card flashing method, as described in section 7. Programming / Flashing .

However, there are cases where you might require the use of a serial downloader. This is more common in a factory environment where the boards are being programmed for the first time or in cases where the board is bricked.

This is considered hardware flashing because it requires the board to be put into the serial download mode (called SCIF mode) by altering the bootstrapping pins.

16.1.1 RZ/G2L-SBC

This section describes the firmware flashing process for the RZ/G2L-SBC board.

Note: The RZ/G2L-SBC does not have any interfaces on the main board to alter the boot mode. The bootstrapping pins are routed through the bottom connectors J12 & J13. Hence, the process requires the use of an adapter board, which is not included in the package.

16.1.1.1 Required Hardware

This flashing process requires the use of a boot mode change, which is achieved using an adapter board that is not included in the package.

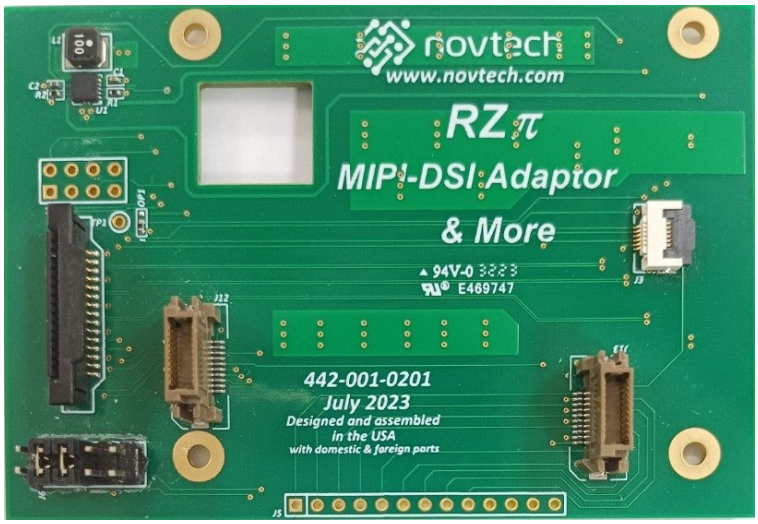


Figure 247. Adaptor board

After setting up the required hardware and configuring the boot mode through connectors J12 and J13 on the RZ/G2L-SBC, refer to Section 7.3.2 for instructions on using the universal script to flash the firmware.

When flashing completes, power off the board and restore the boot mode strapping on connectors J12 and J13 from SCIF Download Mode to the normal boot configuration for the RZ/G2L-SBC. Then power on the board again.

16.1.2 RS-G2L100

This section describes the firmware flashing process and boot mode configuration for each supported board. Firmware flashing is required to write bootloaders (BL2 and FIP) to the onboard flash memory. The process uses Renesas' Flash Writer tool and requires setting the board into SCIF Download Mode.

The RS-G2L100 features onboard DIP switches that allow direct selection of boot modes.

16.1.2.1 DIP Switch Settings

Use the DIP switch SW1 to configure the boot mode:

Table 75. SCIF Download Mode - RS-G2L100

Switch	SCIF Download Mode
SW1-1	OFF
SW1-2	ON
SW1-3	OFF
SW1-4	OFF

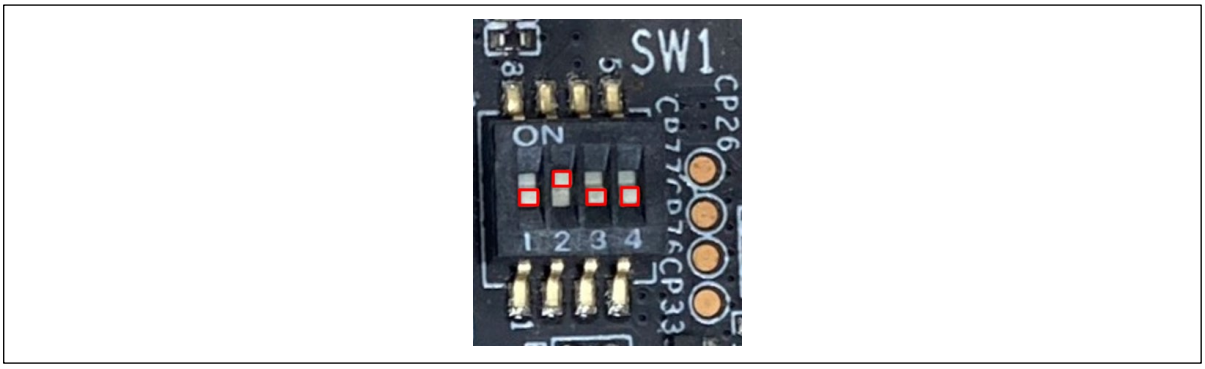


Figure 248. SW1 SCIF Download Mode - RS-G2L100

After setting up the required hardware and configuring the SCIF download mode for the RS-G2L100, refer to Section 7.3.2 for instructions on using the universal script to flash the firmware.

When flashing completes, power off the board and restore the switch settings to the corresponding normal boot mode in Section 16.2.1 before powering on again

16.1.3 RZ/G2L-EVK and RZ/V2L-EVK

This section describes the firmware flashing process and boot mode configuration for each supported board. Firmware flashing is required to write bootloaders (BL2 and FIP) to the onboard flash memory. The process uses Renesas' Flash Writer tool and requires setting the board into SCIF Download Mode.

Unlike the RZ/G2L-SBC, which requires external hardware to configure boot mode, the RZ/G2L-EVK and RZ/V2L-EVK feature onboard DIP switches that allow direct selection of boot modes. This simplifies the flashing process and eliminates the need for manual signal strapping.

16.1.3.1 DIP Switch Settings

Use the DIP switch SW11 to configure the boot mode:

Table 76. SCIF Download Mode - RZ/G2L-EVK & RZ/V2L-EVK

Switch	SCIF Download Mode
SW11-1	OFF
SW11-2	ON
SW11-3	OFF
SW11-4	ON

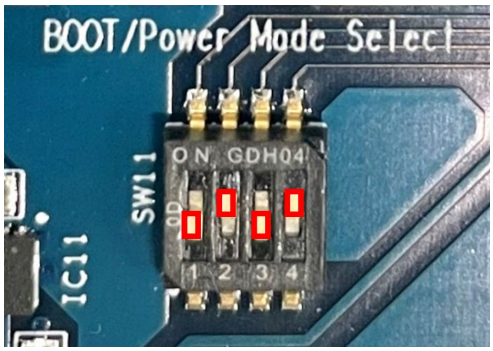


Figure 249. SW11 SCIF Download Mode - RZ/G2L-EVK & RZ/V2L-EVK

Use DIP switch 1 (SW1) to select eMMC as the boot device.

Table 77. Select eMMC as the boot device

Switch	Select eMMC
SW1-1	ON
SW1-2	OFF

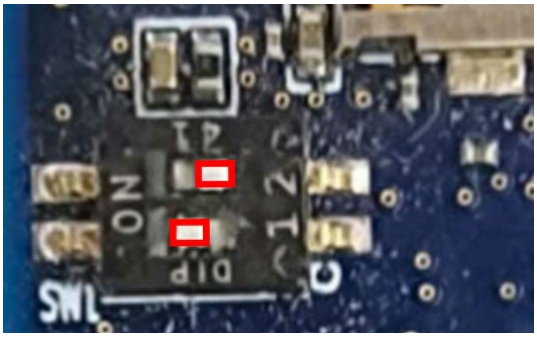


Figure 250. SW1 Settings for eMMC Boot - RZ/G2L-EVK & RZ/V2L-EVK

After setting up the required hardware and configuring the SCIF download mode for the RZ/G2L-EVK or RZ/V2L-EVK, refer to Section 7.3.2 for instructions on using the universal script to flash the firmware.

When flashing completes, power off the board and restore the switch settings to the corresponding normal boot mode in Section 16.2.2 before powering on again

16.1.4 RZ/V2H-EVK

To configure the boot mode on the RZ/V2H-EVK board, use DIP switch DSW1 to set DSW1-4 to OFF and DSW1-5 to ON. The BootROM interprets these two switches as a pair to select the boot source. All other switches must remain in their default positions unless CPU selection, boot frequency, or JTAG debug mode changes are required.

Table 78. SCIF Download Mode - RZ/V2H-EVK

Switch	Status	Function
DSW1-1	ON	Select the cold boot CPU - OFF: CM33 - ON: CA55 (default)
DSW1-2	OFF	Input the CA55 frequency at the CA55 cold boot - [OFF: OFF]: 1.6GHz - [OFF: ON]: 1.7GHz (default) - [ON: OFF]: 1.1GHz - [ON: ON]: 1.5GHz
DSW1-3	ON	
DSW1-4	OFF	- [OFF: OFF]: xSPI - [OFF: ON]: SCIF - [ON: OFF]: SD (default) - [ON: ON]: eMMC
DSW1-5	ON	
DSW1-6	OFF	OFF: SSCG ON (default), ON: SSCG OFF
DSW1-7	OFF	OFF: Normal mode, ON: Debug mode
DSW1-8	OFF	Fixed to OFF

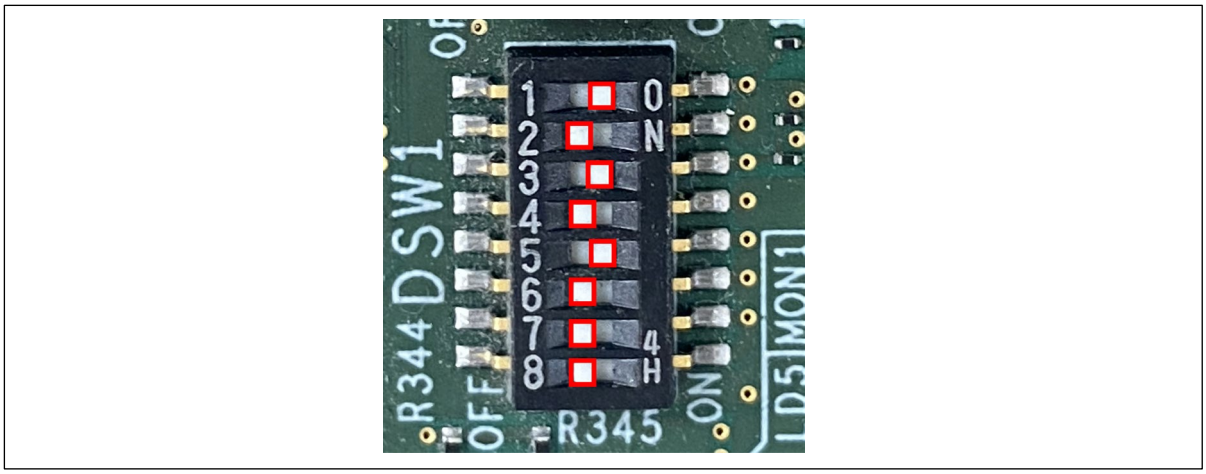


Figure 251. DSW1 - SCIF Boot Mode

To enable SCIF Download Mode, set DSW1-4 and DSW1-5 according to the SCIF configuration in the Table SCIF Download Mode

Other switches (DSW1-1, DSW1-2, DSW1-3, DSW1-6, and DSW1-7) should remain in their default positions unless you need to change CPU selection, boot frequency, SSCG, or debug mode as described in the table.

After configuring the DIP switches for SCIF Download Mode for RZ/V2H-EVK, refer to Section 7.3.2 for instructions on using the universal script to flash the firmware.

When flashing completes, power off the board and restore the switch settings to the corresponding normal boot mode in Section 16.2.3 before powering on again

16.1.5 RZ/V2H-RDK

RZ/V2H-RDK also uses DIP switch DSW1 to configure the boot mode, but unlike RZ/V2H-EVK, it has fewer switches. To enable SCIF Download Mode, set DSW1-4 to OFF and DSW1-5 to ON. These two switches are interpreted as a pair by the BootROM to select the boot source. Other switches should

remain in their default positions unless CPU selection, boot frequency, or JTAG debug mode must be changed

Table 79. DSW1 Switch - RZ/V2H-RDK

Switch	Default settings	Function
DSW1-1	ON	Select the cold boot CPU - OFF: CM33 - ON: CA55 (default)
DSW1-2	OFF	Input the CA55 frequency at the CA55 cold boot - [OFF: OFF]: 1.6GHz - [OFF: ON]: 1.7GHz (default) - [ON: OFF]: 1.1GHz - [ON: ON]: 1.5GHz
DSW1-3	ON	
DSW1-4	OFF	- [OFF: OFF]: xSPI - [OFF: ON]: SCIF - [ON: OFF]: SD (default) - [ON: ON]: eMMC
DSW1-5	ON	
DSW1-6	OFF	Select JTAG debug mode - OFF: normal mode (default) - ON: JTAG

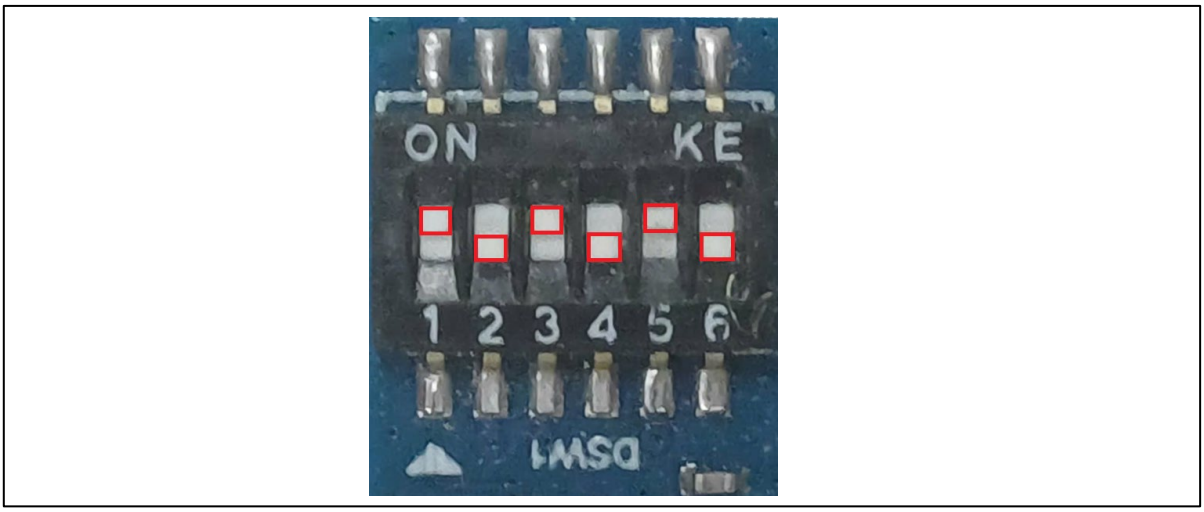


Figure 252. RZ/V2H-RDK - DSW1 SCIF Download Mode

After configuring the DIP switches for SCIF Download Mode for RZ/V2H-RDK, refer to Section 7.3.2 for instructions on using the universal script to flash the firmware.

When flashing completes, power off the board and restore the switch settings to the corresponding normal boot mode in Section 16.2.4 before powering on again.

16.1.6 IMDT V2H-SBC

Use the DIP switch DSW1 to configure the boot mode:

Table 80. SCIF Download Mode – IMDT V2H-SBC

DSW1-Switch	Behavior
Figure 253. SCIF Download Mode	In this configuration, the board is placed in SCIF Download mode. This mode is used for programming the bootloader into the xSPI Flash or the onboard eMMC, over USB Serial.

To enable SCIF Download Mode, set DSW1-1, DSW1-2, and DSW1-3 according to the SCIF configuration in the Table 80. SCIF Download Mode – .

After setting up the required hardware and configuring the SCIF download mode for the IMDT V2H-SBC, refer to Section 7.3.2 for instructions on using the universal script to flash the firmware.

When flashing completes, power off the board and restore the switch settings to the corresponding normal boot mode in Section 16.2.5 before powering on again

16.2 Boot Mode Reference (Non-SCIF)

This section summarizes the switch/strap settings required for normal boot and boot-device selection across supported boards. Use these settings after completing factory flashing or when switching the boot device during bring-up.

16.2.1 RS-G2L100

The RS-G2L100 provides on-board DIP switches for boot mode selection.

The settings below tell the BootROM which device to read the initial firmware from, i.e., where to fetch BL2 (and Boot Parameter, if used) and subsequently the FIP.

Use these settings after factory flashing to boot from the programmed device.

Table 81. SW1 – Boot Device Selection (Normal Boot)

Boot device	SW1-1	SW1-2	SW1-3	SW1-4	Description
eMMC	OFF	ON	ON	OFF	Boot from on-board eMMC (BootROM loads BL2/BL2+BP from eMMC, then FIP).
QSPI	OFF	OFF	ON	OFF	Boot from QSPI NOR flash

Table 82. RS-G2L100 SW1 Switch Mode Position

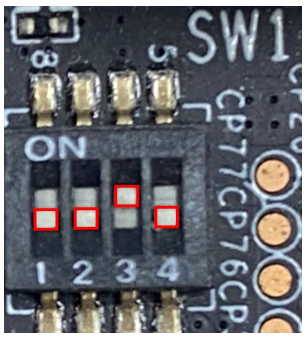


Figure 254. SW1 – QSPI Boot Mode

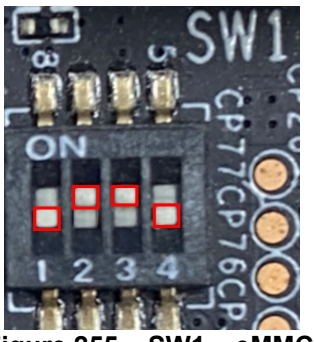


Figure 255. SW1 – eMMC Boot Mode

16.2.2 RZ/G2L-EVK & RZ/V2L-EVK

These EVKs provide on-board DIP switches for boot mode and boot device selection.

The settings below tell the BootROM which device to read the initial firmware from, for example, where to fetch BL2 (and Boot Parameter, if used) and subsequently the FIP.

Use these settings after factory flashing to boot from the programmed device.

Table 83. SW11 – Boot Device Selection (Normal Boot)

Boot device	SW11-1	SW11-2	SW11-3	SW11-4	Description
eMMC	ON	OFF	OFF	ON	Boot from on-board eMMC (BootROM loads BL2/BL2+BP from eMMC, then FIP).
QSPI	OFF	OFF	OFF	ON	Boot from QSPI NOR flash
SD / eSD	ON	ON	OFF	ON	Boot from SD/eSD card (slot media)

Table 84. RZ/G2L-EVK & RZ/V2L-EVK SW11 Switch Mode Position

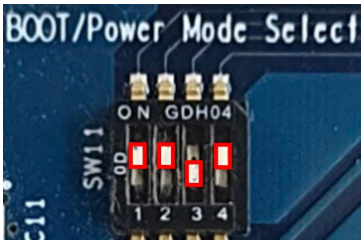


Figure 256. SW11 - eSD Boot Mode

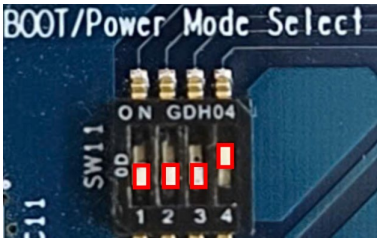


Figure 257. SW11 – QSPI Boot Mode

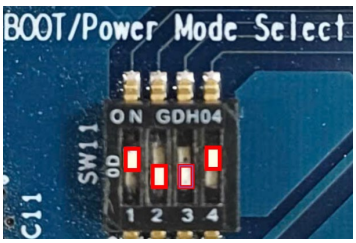


Figure 258. SW11 – eMMC Boot Mode

On the SOM module, SW1 selects eMMC or microSD boot mode. Please refer to the table below for the boot-mode options for each switch setting.

Table 85. SW1 – SOM module Switch mode

Boot device	ON	OFF

SW1-1	Normal Operation	JTAG debug mode
SW1-2	Select microSD slot on RTK9744L23C01000BE	Select eMMC on RTK9744L23C01000BE

Table 86. RZ/G2L-EVK & RZ/V2L-EVK SW1 Switch Mode Position

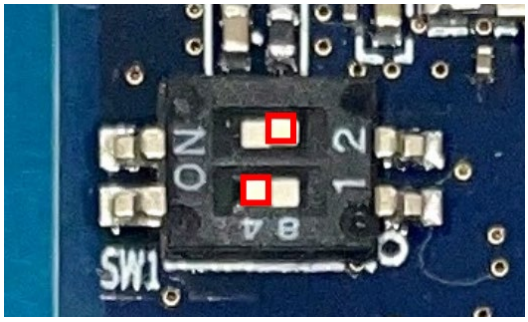


Figure 259. eMMC selection with normal operation

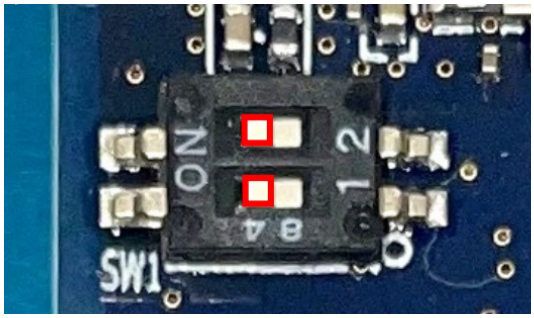


Figure 260. Enable the SD slot CN3 for normal operation

16.2.3 RZ/V2H-EVK

The RZ/V2H-EVK also provides on-board DIP switches for boot mode and boot device selection.

Use these settings after factory flashing to boot from the programmed device.

Table 87. DSW1 – Boot Device Selection (Normal Boot)

Boot device	DSW1-1	DSW1-2	DSW1-3	DSW1-4	DSW1-5	DSW1-6	DSW1-7	DSW1-8
eMMC	ON	OFF	ON	ON	ON	OFF	OFF	OFF
xSPI	ON	OFF	ON	OFF	OFF	OFF	OFF	OFF
SD / eSD	ON	OFF	ON	ON	OFF	OFF	OFF	OFF

Table 88. RZ/V2H-EVK DSW1 Switch Mode Position

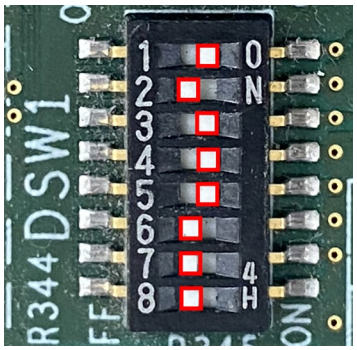


Figure 261. DSW1 - eMMC Boot Mode

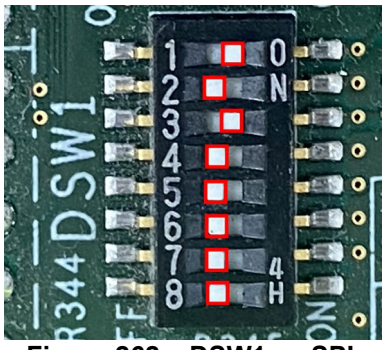


Figure 262. DSW1 – xSPI Boot Mode

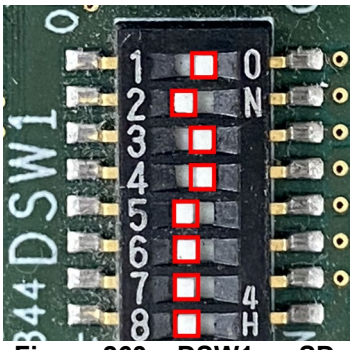


Figure 263. DSW1 – eSD Boot Mode

DSW2 controls output from the on-board clock generator (5P35023B) and one protected utility signal.

Table 89. DSW2 - Audio Clock / Utility DIP

Switch	Signal	OFF (default)	ON

1	Audio_CLKB_OE	Disables 5P35023B Audio_CLKB output	Enables Audio_CLKB output
2	Audio_CLKB	Audio_CLKB not supplied	Audio_CLKB is driven
3	Audio_CLKC_OE	Disables 5P35023B Audio_CLKC output	Enables Audio_CLKC output
4	Audio_CLKC	Audio_CLKC not supplied	Audio_CLKC is driven
5	NEN_VPROG	Must remain OFF	Prohibited — do not set ON
6	—	—	—

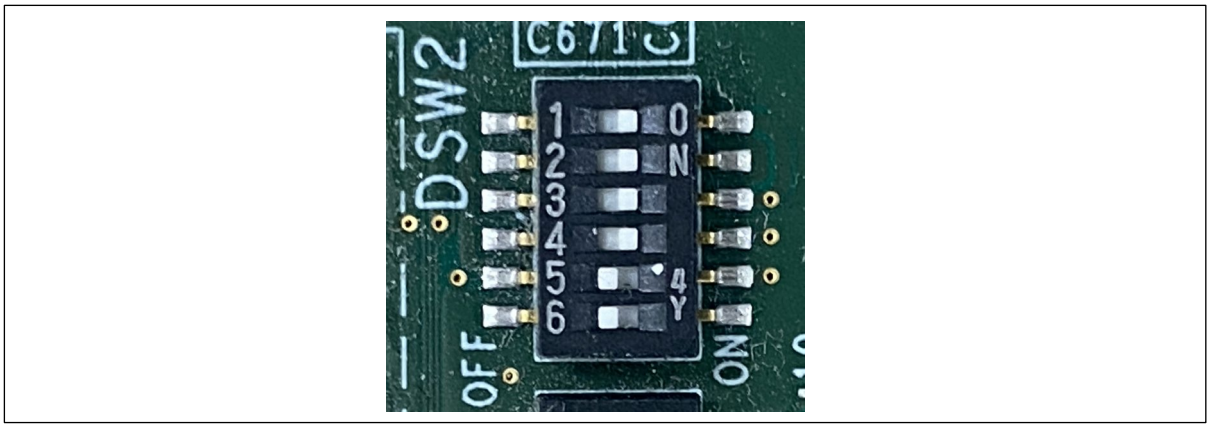


Figure 264. DSW2 - Audio Clock / Utility DIP

The table below lists the settings of the DIP switch (JSW1 on the RZ/V2H Secure Evaluation Board) and its functions.

Table 90. JSW1 Functions

Switch	Function
1-2	MIPI CSI-2 camera interface voltage: 1.8 V
2-3	MIPI CSI-2 camera interface voltage: 3.3 V (default)

Note: Set this switch according to the interface voltage of the camera module to be connected.

16.2.4 RZ/V2H-RDK

The RZ/V2H-RDK also provides on-board DIP switches for boot mode and boot device selection.

Use these settings after factory flashing to boot from the programmed device.

Table 91. DSW1 – Boot Device Selection (Normal Boot)

Boot device	DSW1-1	DSW1-2	DSW1-3	DSW1-4	DSW1-5	DSW1-6
eMMC	ON	OFF	ON	ON	ON	OFF
xSPI	ON	OFF	ON	OFF	OFF	OFF
SD / eSD	ON	OFF	ON	ON	OFF	OFF

Table 92. RZ/V2H-RDK DSW1 Switch Mode Position

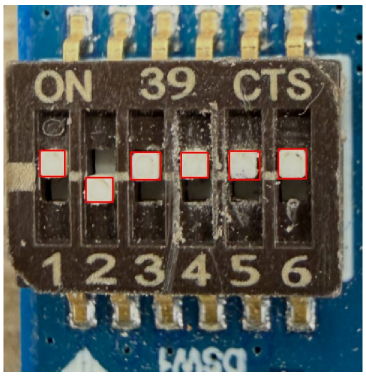


Figure 265. DSW1 - eMMC Boot Mode (not supported)

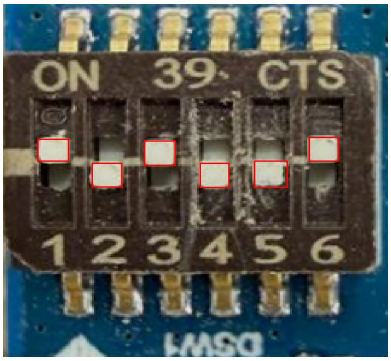


Figure 266. DSW1 – xSPI Boot Mode

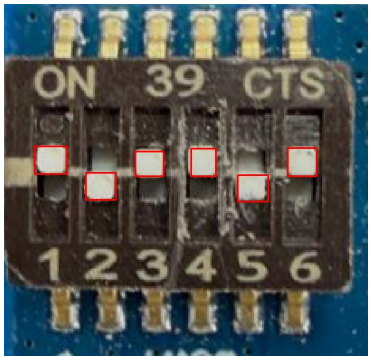


Figure 267. DSW1 – eSD Boot Mode

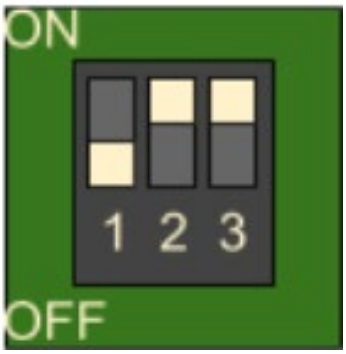
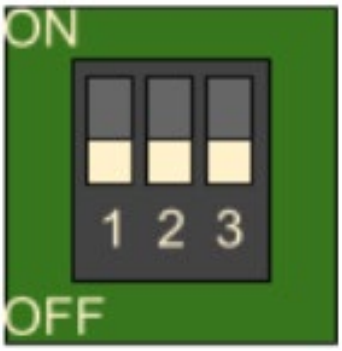
16.2.5 IMDT V2H-SBC

The IDMT-V2H-SBC also provides on-board DIP switches for boot mode and boot device selection.

Use these settings after factory flashing to boot from the programmed device.

Table 93. DSW1 – Boot Device Selection (Normal Boot)

DSW1-Switch	Behavior

 **Figure 268. IMDT V2H-SBC xSPI Boot Mode Switch**	In this configuration, the bootROM attempts to load the bootloader from the xSPI Flash. If the Boot ROM fails to boot from the xSPI Flash, the board will enter SCIF Download mode. OFF, ON, ON
 **Figure 269. IMDT V2H-SBC eMMC Boot Mode Switch**	In this configuration, the bootROM attempts to load the bootloader from the eMMC. If the Boot ROM fails to boot from the eMMC Flash, the board will enter SCIF Download mode. OFF, OFF, OFF

16.3 Prepare the eMMC root filesystem

The onboard eMMC can be used as the main root filesystem, but it must first be initialized with a valid Linux rootfs. Since the eMMC is initially empty or unformatted, it cannot be used directly.

During preparation, the system must boot Linux using an SD card rootfs, while BL2/FIP are loaded from either QSPI or eMMC, depending on the boot mode. Once Linux is running, the onboard eMMC (for example, /dev/mmcblk0) becomes accessible and can be partitioned, formatted, and written with the rootfs.

Step 1. Prepare the rootfs archive

- Obtain the provided root filesystem archive (e.g., core-image-weston.tar.bz2). Obtain the provided root filesystem archive, which is included in the release images under:
  ```
  rz-cmn-srp/target/images/rootfs
  ```
- Copy the archive to an SD card. This SD card will later be used as the source for writing the rootfs into the onboard eMMC.

Step2. Boot the board in QSPI/eMMC mode

- Set the DIP switch to select the boot source: QSPI or eMMC. Refer to section Boot Mode Reference (Non-SCIF) for the correct switch positions for the target boards.
- Insert the prepared SD card (from Step 1) into the correct slot.

Table 94. SD card slot used for rootfs preparation

Board	SD card slot used for rootfs preparation	Notes

RZ/G2L-EVK	CN10 (Carrier microSD slot)	SW1-2 must be set to **ON (eMMC)**
RZ/V2L-EVK	CN10 (Carrier microSD slot)	SW1-2 must be set to **ON (eMMC)**
RZ/V2H-EVK	SoM microSD slot (on CPU board, underneath the module)	-
RS-G2L100	J1 (Carrier microSD slot)	-
IMDT V2H-SBC	J14 (Carrier microSD slot)	-

Note: Some early RZ/V2H-EVK revisions included two SD slots, and either slot could be used for rootfs preparation. The production version provides only the SoM slot underneath the CPU board. Always check the hardware manual for the board revision in use.

- Power on the board

Step 3. Identify the eMMC device

Determine which MMC index corresponds to the onboard eMMC. This can be checked in either U-Boot or Linux.

- In U-boot: Run the following commands:
  1. Reboot or power on the board
  2. When the message “Hit any key to stop autoboot” appears, press a key to interrupt boot and enter the U-Boot console (=>).
  3. Run the following commands:

```
=> mmc rescan
=> mmc list
```

After rescanning, the list will show which device is the eMMC. The example below is for the RZ/G2L-EVK, where **mmc0** is identified as the eMMC device and **mmc1** as the SD card (CN10).

```
=> mmc list
sd@11c00000: 0
sd@11c10000: 1

=> mmc rescan
=> mmc list
sd@11c00000: 0 (eMMC)
sd@11c10000: 1
```

- Run *lsblk* to list all block devices. The numbering in Linux matches the U-Boot mapping: for example, if U-Boot shows mmc0 = eMMC, then in Linux the eMMC will appear as /dev/mmcblk0.

```
root@rz-cmn:~# lsblk
```

Example output:

```
mmcblk0      179:0    0 59.3G  0 disk
|-mmcblk0p1  179:1    0  100M  0 part
`-mmcblk0p2  179:2    0  2.5G  0 part
```

Step 4. Create the partition table

```
root@rz-cmn:~# fdisk /dev/mmcblk0
```

Inside fdisk, create the partition table as follows:

1. Press o → create a new DOS disklabel.
2. Press n, then p, then ENTER for defaults, then type +500M → creates Partition 1 (boot).
3. Press n, then p, then ENTER twice for defaults → creates Partition 2 (rootfs).
4. Press t, then select partition 1, then type b → set Partition 1 type to W95 FAT32.
5. Press w → write and exit.

Afterward, check the partition table with the command below:

```
root@rz-cmn:~# fdisk -l /dev/mmcblk0
```

Expected table:

```
root@rz-cmn:~# fdisk -l  /dev/mmcblk0
Disk /dev/mmcblk0: 59.28 GiB, 63652757504 bytes, 124321792 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x076c4a2a

Device         Boot  Start     End Sectors  Size Id Type
/dev/mmcblk0p1 *        32  204831  204800  100M  c W95 FAT32 (LBA)
/dev/mmcblk0p2      204832 5376159 5171328  2.5G 83 Linux
```

Step 5. Format the partitions

Format Partition 1 as FAT32 (boot):

```
root@rz-cmn:~# mkfs.vfat -F 32 -n boot /dev/mmcblk0p1
```

Format Partition 2 as ext4 (rootfs)

```
root@rz-cmn:~# mkfs.ext4 -L rootfs /dev/mmcblk0p2
```

Step 6. Populate the rootfs and kernel Image

Once the partitions are formatted, the eMMC must be populated with the Linux root filesystem and the kernel image.

4. Mount the partitions
```
root@rz-cmn:~# mkdir -p /mnt/boot
root@rz-cmn:~# mkdir -p /mnt/rootfs

root@rz-cmn:~# mount /dev/mmcblk0p1 /mnt/boot
root@rz-cmn:~# mount /dev/mmcblk0p2 /mnt/rootfs
```
5. Extract the rootfs archive into the rootfs partition
```
root@rz-cmn:~# cd /mnt/rootfs
root@rz-cmn:~# tar xf /home/root/core-image-weston.tar.bz2 .
root@rz-cmn:~# sync
```
6. Copy the kernel Image, DTBs, and user environment (uEnv.txt) to the boot partition
```
root@rz-cmn:~# cp -rf /boot/* /mnt/boot/
```
7. Unmount the partitions

```
root@rz-cmn:~# umount /mnt/boot
root@rz-cmn:~# umount /mnt/rootfs
```

At this point, the eMMC contains:

- Partition 1 (boot): Kernel Image + DTBs + user environment (uEnv.txt)
- Partition 2 (rootfs): Full Linux root filesystem

The board can now boot using eMMC as the root filesystem.

16.4 How To Get the Console After Bootup

Once the RZ boards have booted, on the UART terminal, you will be able to login using the default user ‘root’. There is no password. Leave the password field empty and just hit the return / enter key.

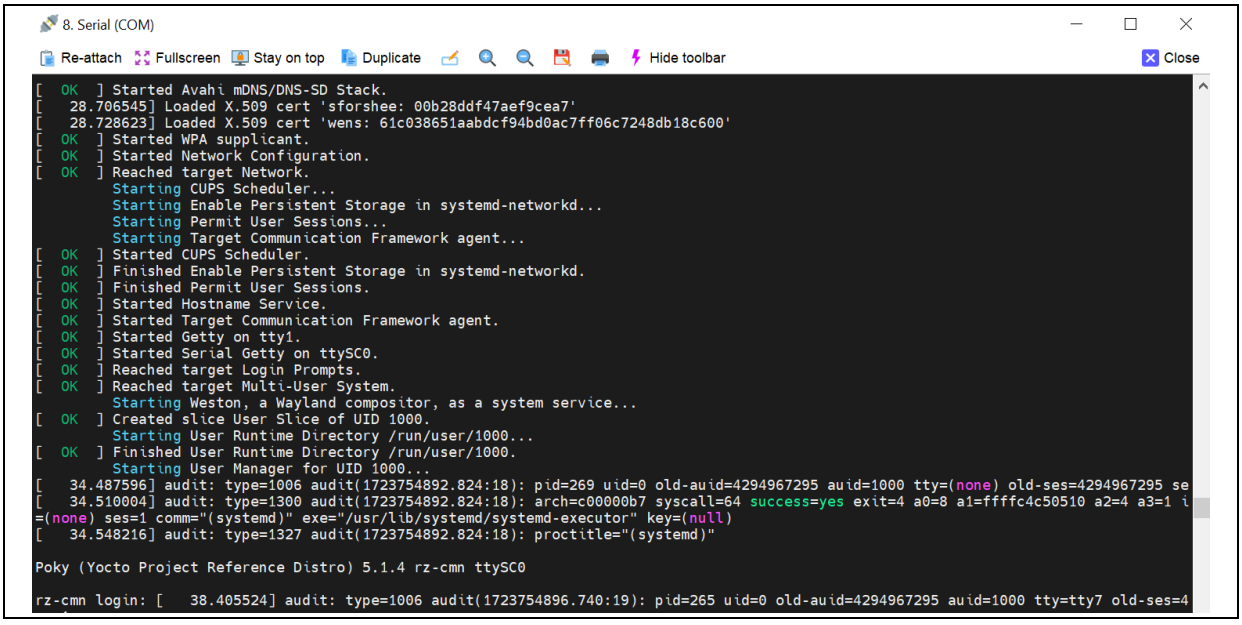


Figure 270. Root login of Linux console over UART.

16.5 BSP Interfaces

Renesas provides a dedicated BSP Manual Set for the RZ/G2L, RZ/V2L, and RZ/V2H Evaluation Kits (EVKs), offering technical guidance on SoC configuration, supported drivers, and Linux system integration.

It is a key reference for developers working with the Verified Linux Package (VLP) on these platforms.

- Download the BSP Manual Set here: RZ/G2L, RZ/Five, RZ/V2L BSP Manual Set (v4.00)
- Download the RZ/V2H BSP Manual Set: RZ/V2H BSP Manual Set (v1.01)

16.6 Creating Remote RPMsg Firmware

This appendix describes a basic workflow for rebuilding or modifying remote RPMsg firmware for use with this release package.

In this release package, prebuilt remote firmware binaries are provided for demonstration and evaluation. Users who need to rebuild or modify the remote RPMsg firmware should obtain the corresponding example project from the Renesas RZ Multi-OS Package and use it as the starting point.

The remote firmware runs on a remote processor core, such as Cortex-M33 or Cortex-R8, and communicates with Linux through OpenAMP/RPMsg.

16.6.1 Prerequisites and Software Requirements

Before creating or rebuilding remote RPMsg firmware, prepare the required development environment.

1. Download e² studio v2026-04.1 including RZ FSP from this link setup_rzfsp_v4_1_0_e2s_v2026-04.1.exe
2. Install e² studio IDE by opening the installer above. First, select Custom Install and then click Next to continue.
3. Change the installation directory if needed, then click Next

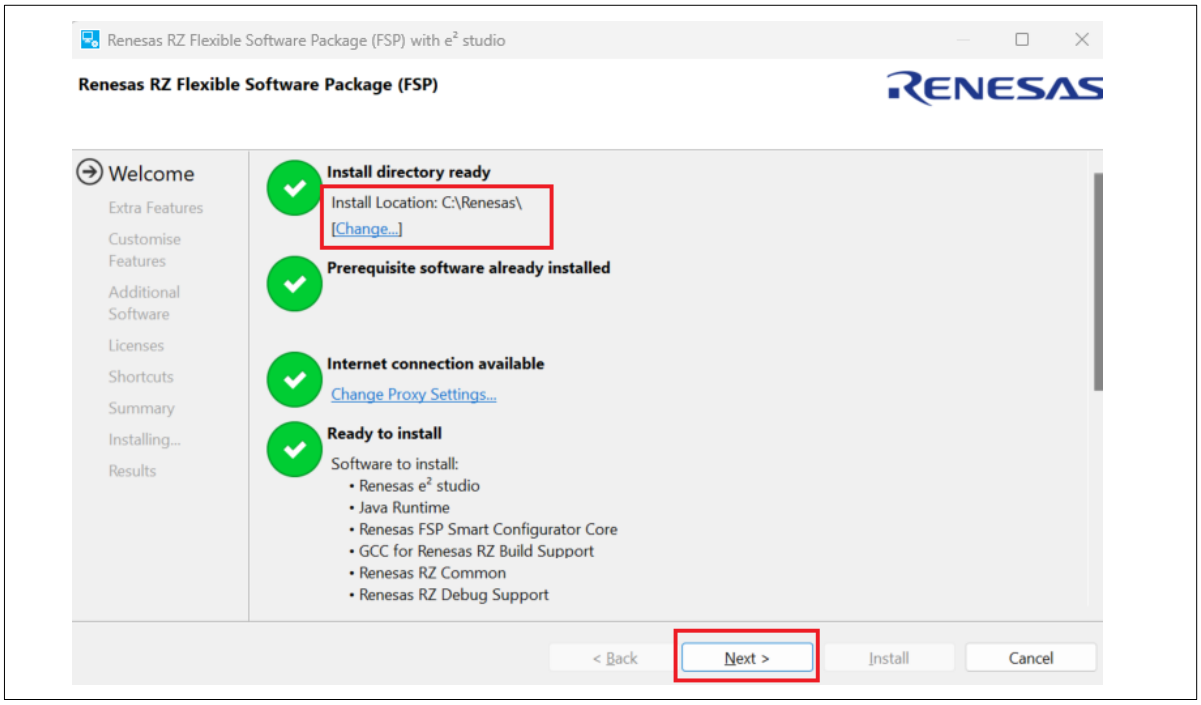


Figure 271. e²studio installation - select install location

4. Select the additional features to install. If support for non-English languages is required, select the appropriate language packages at this step. Click Next to continue.

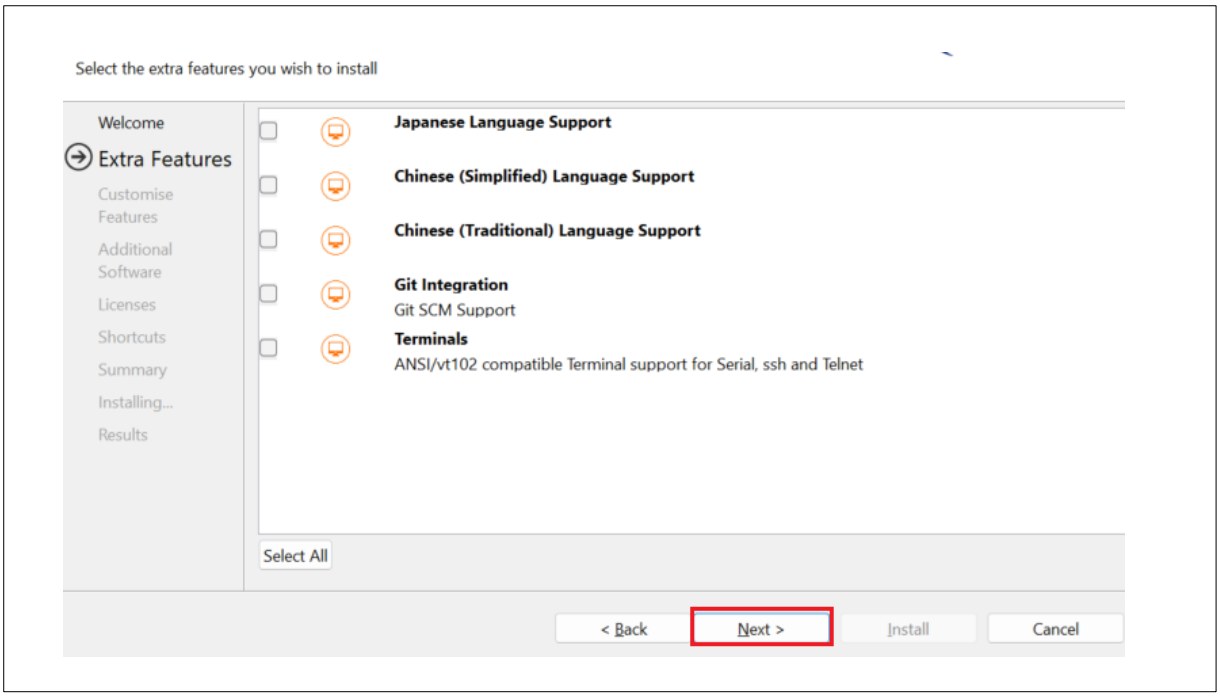


Figure 272. e2studio installation - Extra Features Screen

5. On the Customize Features screen, make sure that both FSP Smart Configure Core and GCC for Renesas RZ Build Support are checked (they are enabled by default), then click Next

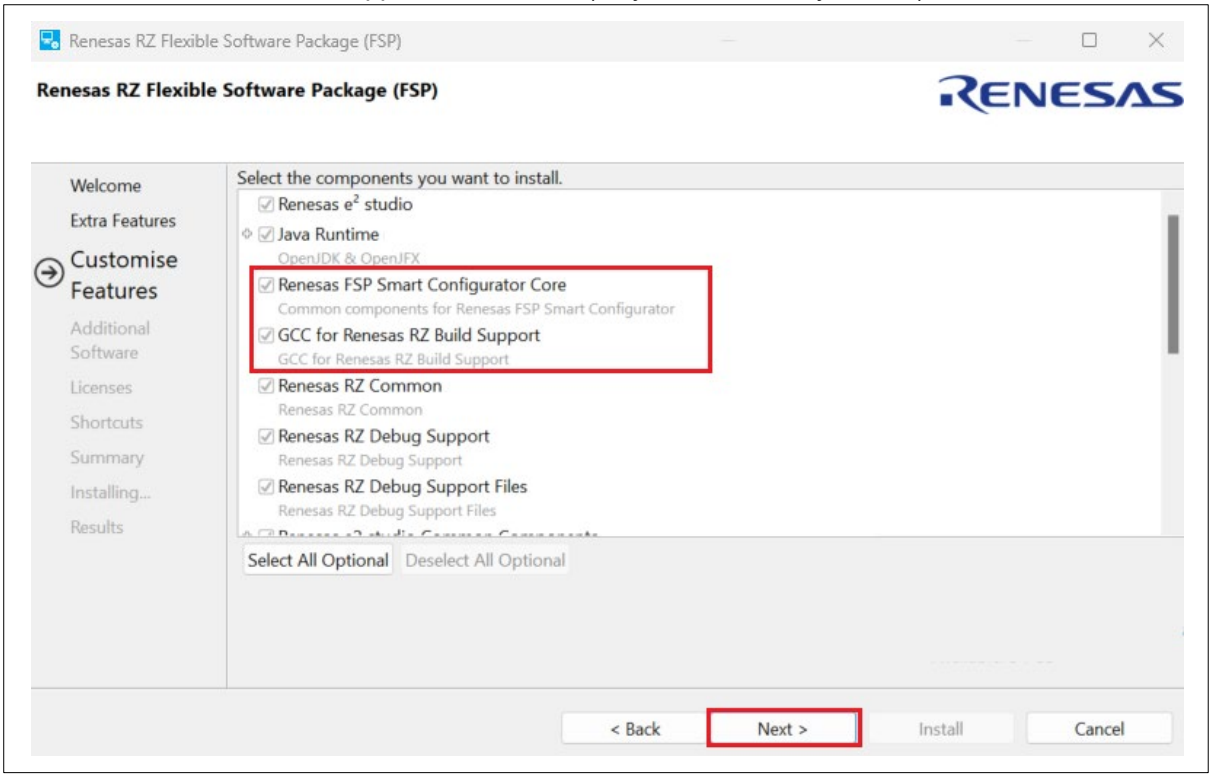


Figure 273. e²studio installation - Customize Features Screen

6. Next, on the Additional Software screen, make sure that both Renesas RZ FSP and Renesas RZ Toolchains are checked, then click Next.

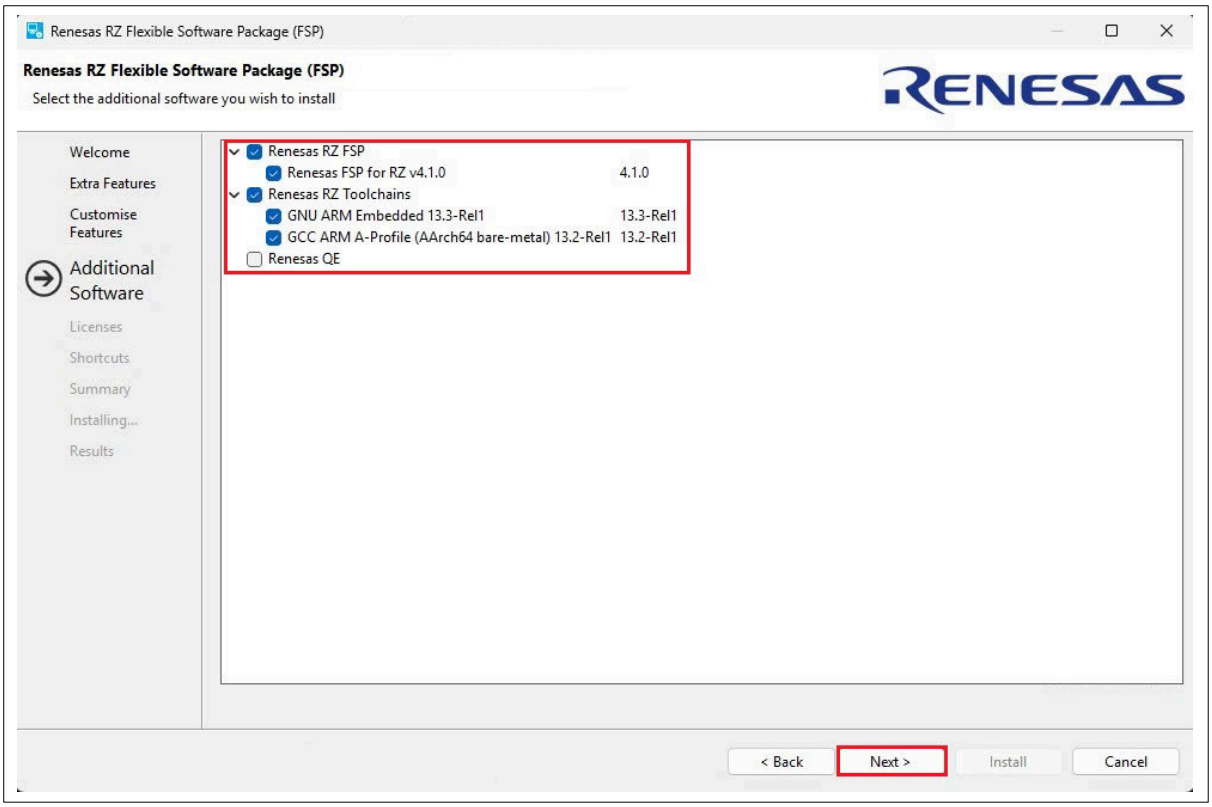


Figure 274. e²studio installation - Additional Software Screen

7. The Summary screen will appear. Simply click Install to proceed.
8. During the installation, if a Windows Security prompt appears asking to install the device software, click Install.

Figure 275. Windows Security prompt

16.6.2 RPMsg Example Project Workflow

This subsection describes a simple workflow for using the RPMsg example project from the Renesas RZ Multi-OS Package as the starting point for firmware rebuild or modification. It covers project selection, import into the workspace, optional modification, build, and use of the generated firmware output.

16.6.2.1 Import Existing Project into Workspace

Use the following workflow to rebuild or modify the provided RPMsg example project.

1. Download and extract the Multi-OS package: Download the Renesas RZ Multi-OS Package and extract it to the development host.
2. Open the RPMsg example directory: After extraction, open the rz-rpmsg-examples directory included in the package. This directory contains RPMsg example projects for the supported boards, organized by board name.

```
rz-rpmsg-examples/
   ├── evb_rzn2h
   ├── evb_rzt2h
   ├── evk_rzg2l
   ├── evk_rzg2lc
   ├── evk_rzg2ul
   ├── evk_rzg3e
   ├── evk_rzg3s
   ├── evk_rzv2h
   ├── evk_rzv2l
   └── evk_rzv2n
```

3. Select the directory for the target board: Open the subdirectory corresponding to the target board. For example, for the RZ/G2L EVK, open: *rz-rpmsg-examples/evk_rzg2l*. This directory contains the project archive *rzg2l_cm33_rpmsg_linux_rtos_example.zip*, which is the CM33 example project for RZ/G2L.
4. Import the project into e² studio: Open e² studio, then import the selected project into the workspace by selecting File > Import. In the General category, select Existing Projects into Workspace, then click Next to continue.

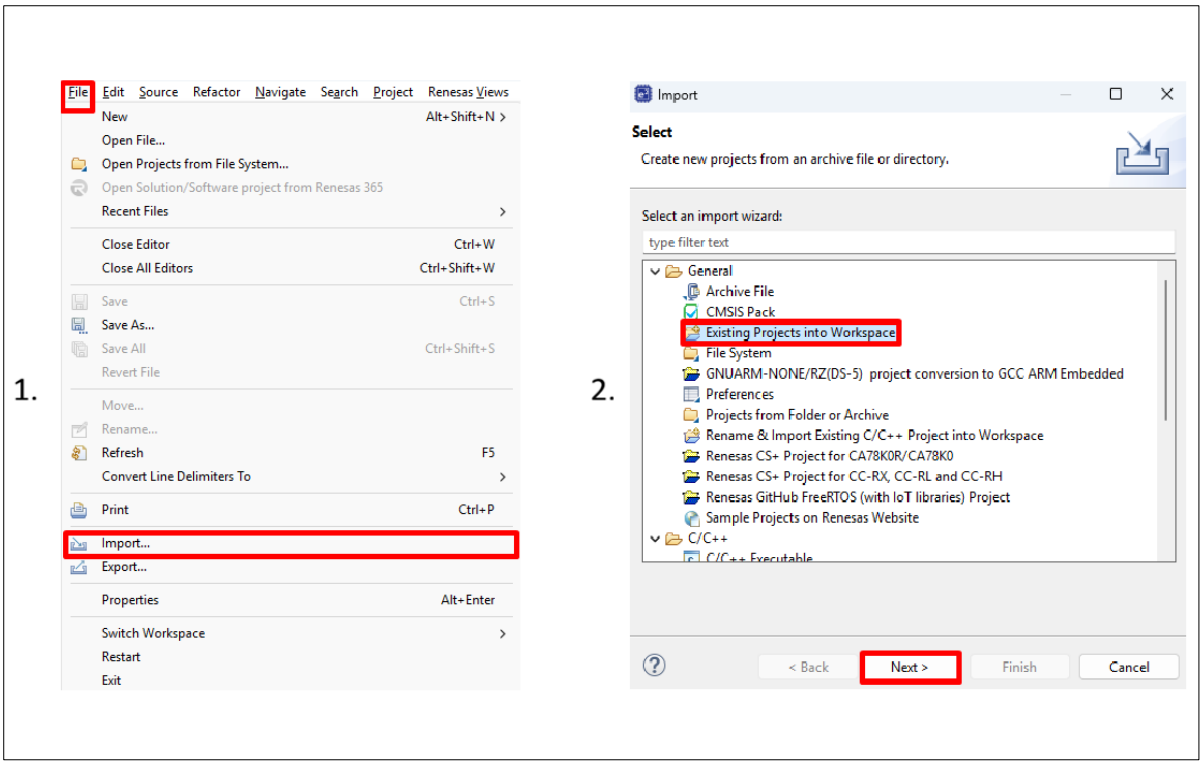


Figure 276. File drop-down menu - Import project

5. After completing the previous steps, the project import dialog will appear. Select Archive File, browse to the ZIP file identified in Step 3, and then click Finish. Alternatively, the project may also be imported from its extracted folder by selecting Select root directory and browsing to the project's root directory.

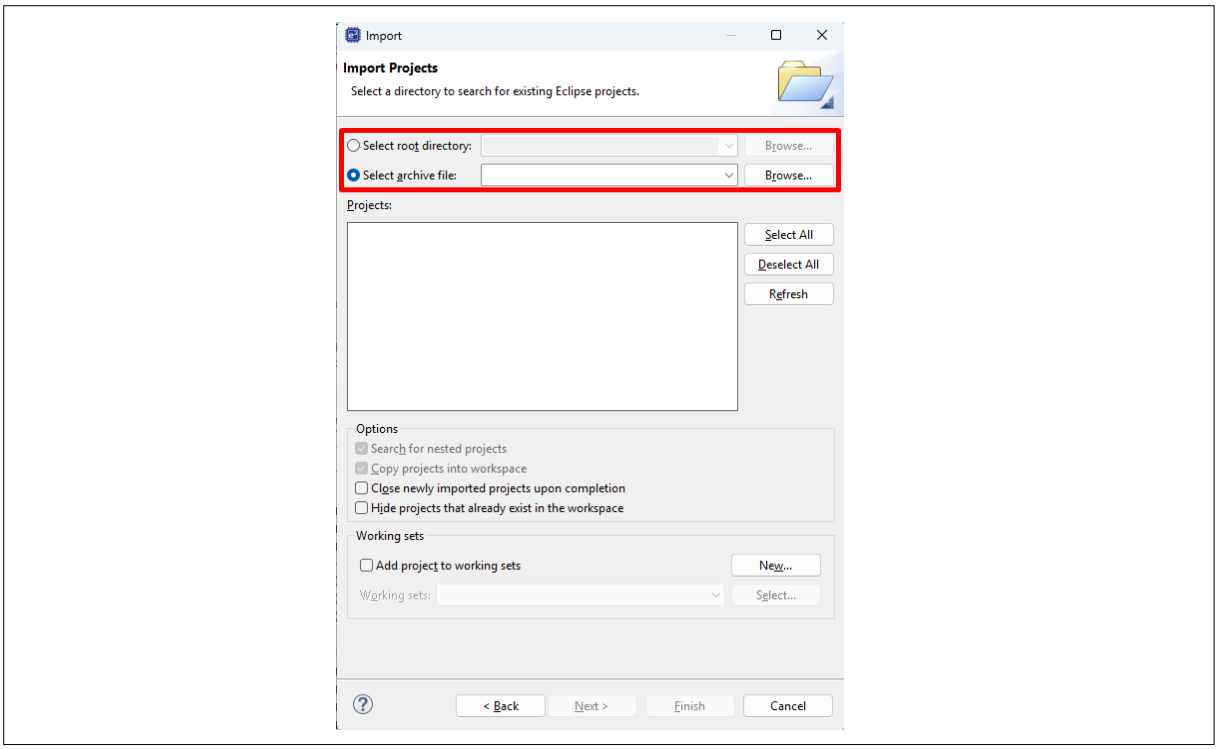


Figure 277. Import Existing Project Dialog

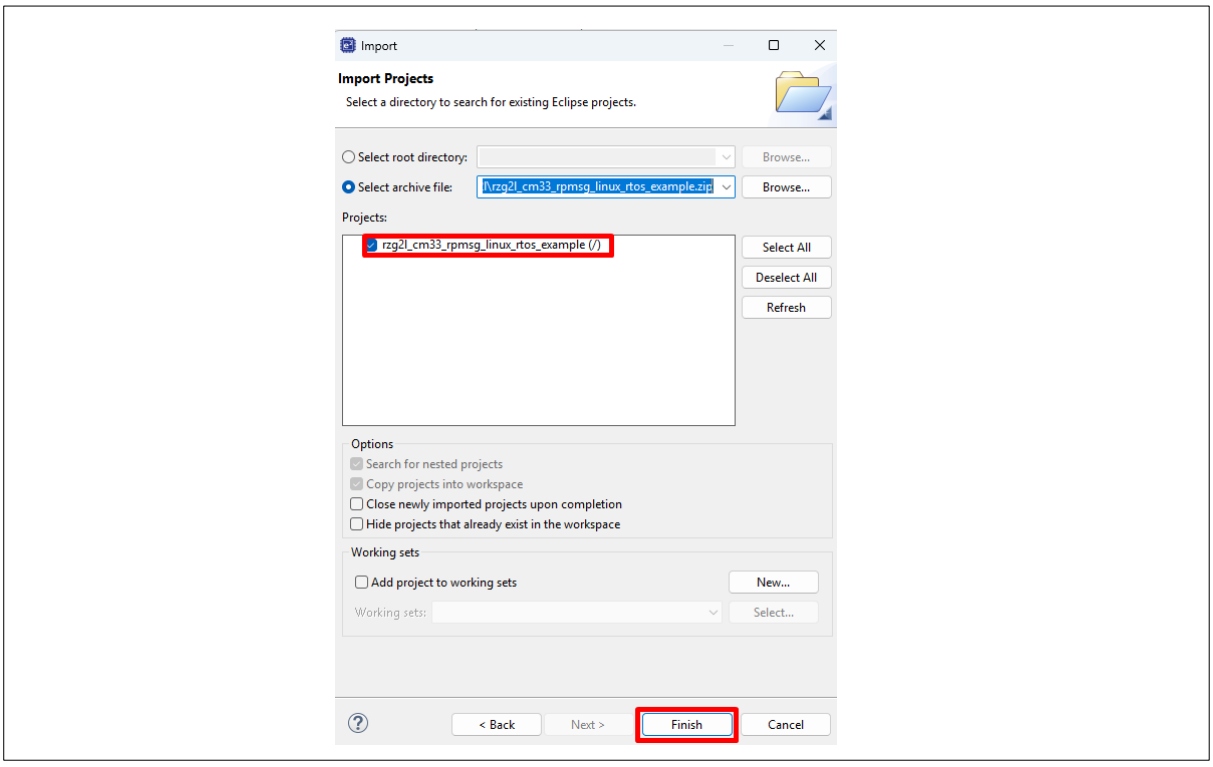


Figure 278. Select project and Finish

16.6.2.2 Modifying the Project from Sample Project

The RZ/V2H CR8 RPMsg example projects provided in the Renesas RZ Multi-OS Package use a memory layout that differs from the layout used in this release package. Before building the project,

update the linker script, RPMsg configuration, MPU settings, and resource-table configuration to match the memory layout in this release package.

Apply the changes described in the following sections to the project that corresponds to the target CR8 core:

- rzv2h_cr8_core0_rpmsg_linux_rtos_example for CR8 core 0
- rzv2h_cr8_core1_rpmsg_linux_rtos_example for CR8 core 1

(1) Modifying the CR8 Core 0 Project

For the CR8 core 0 project, modify the following files.

a. Update the linker script

Open script/rzv2h_evk_cr.ld and update the OpenAMP memory definitions as follows:

```
OPENAMP_RSCTBL_START    = DEFINED(CR8_0) ? 0x42F02000 : 0x42F04000;
MHU_SHMEM_START         = DEFINED(CR8_0) ? 0x42F03000 : 0x42F05000;
OPENAMP_VRING_START     = DEFINED(CR8_0) ? 0x43800000 : 0x44000000;
```

b. Update the RPMsg configuration

Open src/OpenAMP_RPMsg_cfg.h and update the RPMsg memory base address, vring addresses, shared-memory addresses, and device names as follows:

```
/* Memory region reserved between 0x43800000 and 0x43FFFFFF for RPMsg */
#define RPMSG_MEM_BASE (0x43800000U)
#define RPMSG_MEM_SIZE (0x00800000U)
```

Update the channel 0 settings:

```
#define CFG_VRING0_BASE0_PA (0x43800000U)
#define CFG_VRING1_BASE0_PA (0x43850000U)
#define CFG_VRING_SHM_BASE0_PA (0x43A00000U)
#define CFG_VRING_CTL_NAME0 "43800000.vring-ctl0-c0"
#define CFG_VRING_SHM_NAME0 "43a00000.vring-shm0-c0"
```

Update the channel 1 settings:

```
#define CFG_VRING0_BASE1_PA (0x43900000U)
#define CFG_VRING1_BASE1_PA (0x43950000U)
#define CFG_VRING_SHM_BASE1_PA (0x43D00000U)
#define CFG_VRING_CTL_NAME1 "43900000.vring-ctl1-c0"
#define CFG_VRING_SHM_NAME1 "43d00000.vring-shm1-c0"
```

c. Update the MPU region size

Open src/mpu_region_table.c. In both MPU configuration tables, change the size of region 8, which covers the OpenAMP resource table and MHU shared-memory area, from BSP_MPU_SIZE_8KB to BSP_MPU_SIZE_16KB.

```
/* region 8 (Open AMP RSCTBL + MHU SHMEM) */
{8,
0x42F00000,
BSP_MPU_SIZE_16KB,
```

d. Update the resource-table configuration

Open src/rsc_table.h and update the resource-table device name and physical address:

```
#define CFG_RSCTBL_DEV_NAME "42f02000.rsctbl"
#define CFG_RSCTBL_MEM_PA (0x42F02000U)
```

(2) Modifying the CR8 Core 1 Project

For the CR8 core 1 project, modify the following files.

a. Update the linker script

Open script/rzv2h_evk_cr.ld and update the OpenAMP memory definitions as follows:

```
OPENAMP_RSCTBL_START = DEFINED(CR8_0) ? 0x42F02000 : 0x42F04000;
MHU_SHMEM_START = DEFINED(CR8_0) ? 0x42F03000 : 0x42F05000;
OPENAMP_VRING_START = DEFINED(CR8_0) ? 0x43800000 : 0x44000000;
```

b. Update the RPMsg configuration

Open src/OpenAMP_RPMsg_cfg.h and update the RPMsg memory base address, vring addresses, shared-memory addresses, and device names as follows:

```
/* Memory region reserved between 0x44000000 and 0x447FFFFF for RPMsg */
#define RPMSG_MEM_BASE (0x44000000U)
#define RPMSG_MEM_SIZE (0x00800000U)
```

Update the channel 0 settings:

```
#define CFG_VRING0_BASE0_PA (0x44000000U)
#define CFG_VRING1_BASE0_PA (0x44050000U)
#define CFG_VRING_SHM_BASE0_PA (0x44200000U)
#define CFG_VRING_CTL_NAME0 "44000000.vring-ctl0-c1"
#define CFG_VRING_SHM_NAME0 "44200000.vring-shm0-c1"
```

Update the channel 1 settings:

```
#define CFG_VRING0_BASE1_PA (0x44100000U)
#define CFG_VRING1_BASE1_PA (0x44150000U)
#define CFG_VRING_SHM_BASE1_PA (0x44500000U)
#define CFG_VRING_CTL_NAME1 "44100000.vring-ctl1-c1"
#define CFG_VRING_SHM_NAME1 "44500000.vring-shm1-c1"
```

c. Update the MPU region size

Open src/mpu_region_table.c and apply the following changes in both MPU configuration tables:

- Change the size of region 8 from BSP_MPU_SIZE_8KB to BSP_MPU_SIZE_32KB.

- Change the base address of region 9, which covers the OpenAMP vring area, from 0x43000000 to 0x44000000

```
/* region 8 (Open AMP RSCTBL + MHU SHMEM) */
{8,
0x42F00000,
BSP_MPU_SIZE_32KB,
```

```
/* region 9 (Open AMP VRING) */
{9,
0x44000000,
BSP_MPU_SIZE_16MB,
```

d. Update the resource-table configuration

Open src/rsc_table.h and update the resource-table device name and physical address:

```
#define CFG_RSCTBL_DEV_NAME "42f02000.rsctbl"
#define CFG_RSCTBL_MEM_PA (0x42F02000U)
```

16.6.2.3 Build the Project

After the example project has been imported into the workspace, build the project in e² studio.

If the project structure does not appear in the IDE, open the Project Explorer view by selecting Window > Show View > Project Explorer.

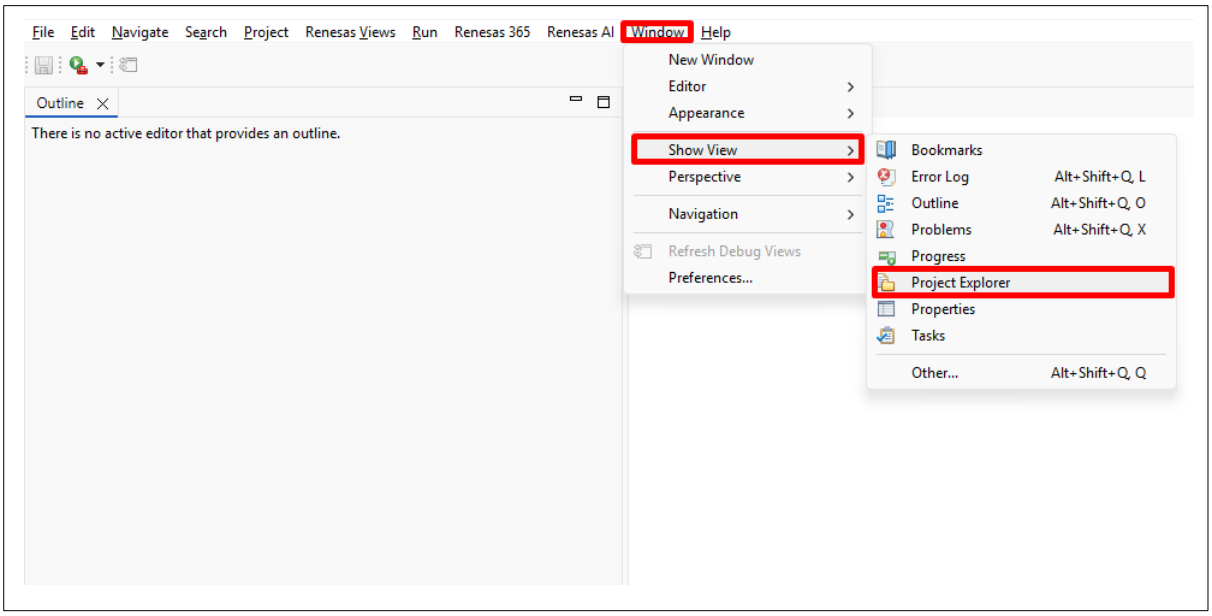


Figure 279. Open Project Explorer in e² studio

Before building, confirm that the intended build configuration is selected and that all required toolchain and project settings are valid. If a small firmware change is required, such as changing a returned RPMsg message string or modifying a simple application action, make that change before starting the build.

Run the build in e² studio by right-clicking the project in the Project Explorer view and selecting Build Project.

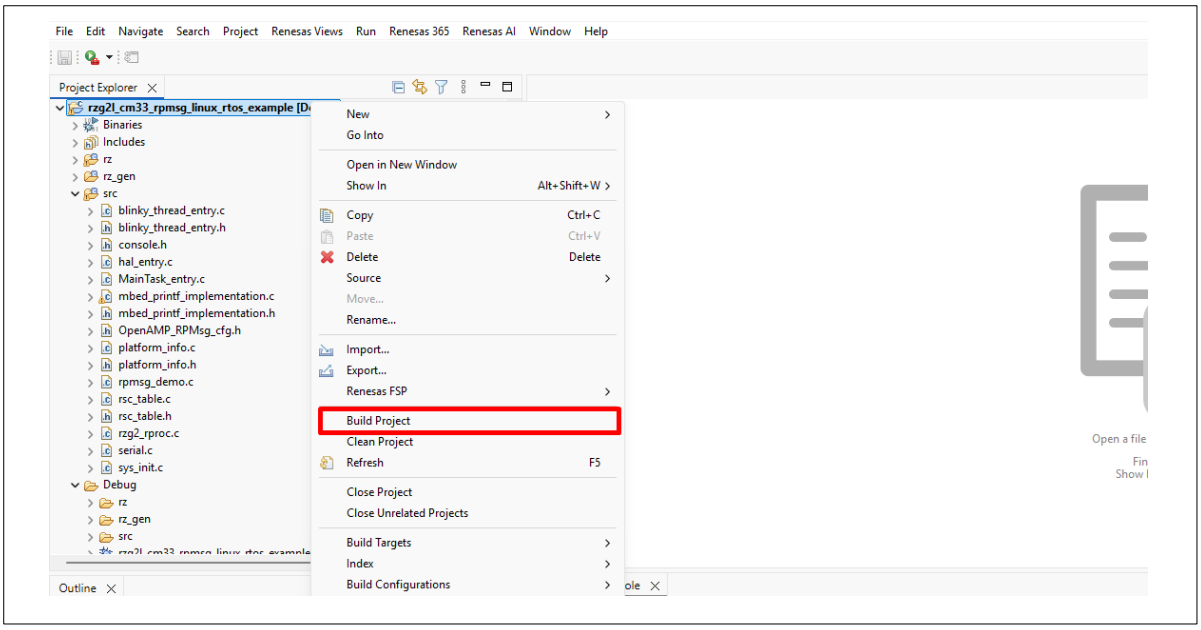


Figure 280. Build Project Drop-down

Confirm that the build is completed without errors. If the build is successful, the firmware output files required for remote processor loading are generated in the Debug folder.

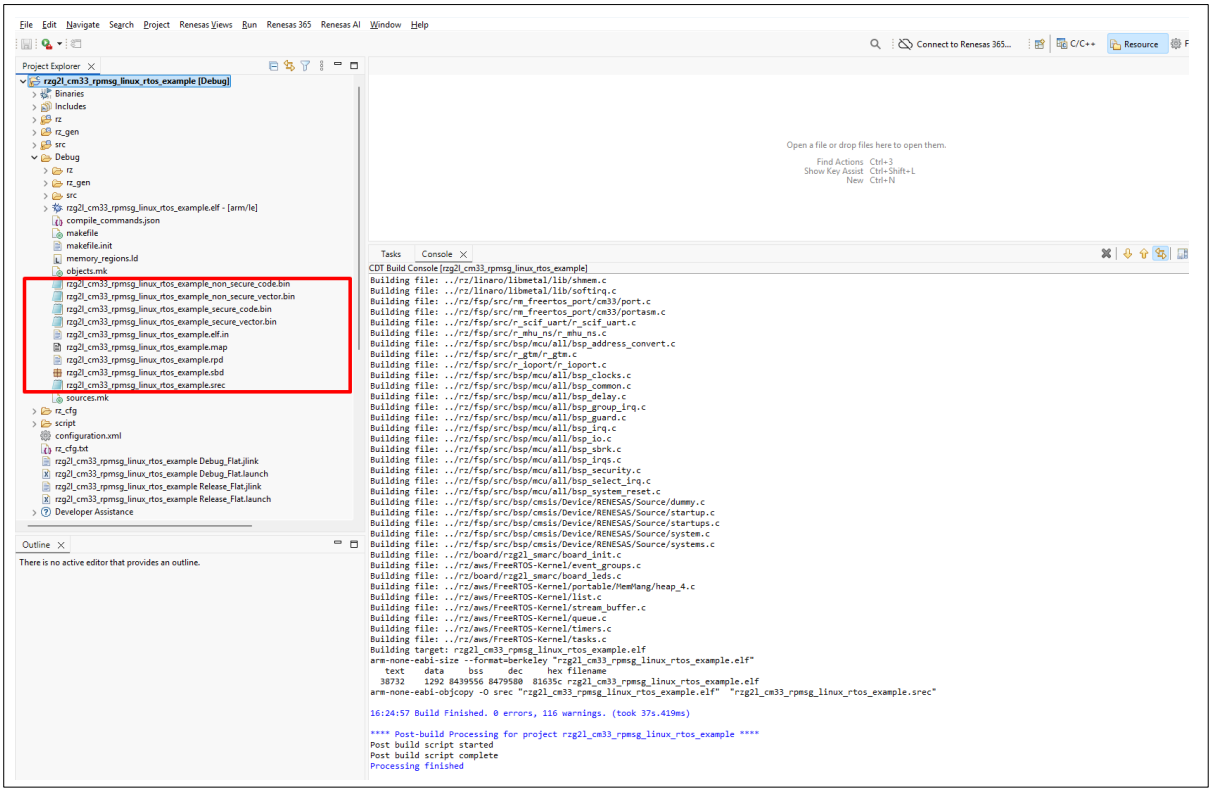


Figure 281. Successful build log and generated firmware artifacts in the Debug folder

Depending on the target device and project configuration, the build output may be generated as:

- a single firmware binary,
- multiple binaries for secure and non-secure regions,

- or multiple binaries for separate memory sections such as ITCM, SRAM, and SDRAM.

16.6.2.4 Use Generated Firmware

After the firmware output files have been identified, use them in place of the corresponding prebuilt demonstration firmware.

Depending on the target platform and selected workflow, the rebuilt firmware may be loaded by:

- U-Boot, or
- Linux remoteproc.

After loading and starting the rebuilt firmware, run the Linux RPMsg sample application to confirm communication between Linux and the remote processor.

For information on which firmware image to use for each target board and the applicable load address, refer to section **Firmware Load Methods**. For instructions on running the sample application, refer to section **Running the RPMsg Sample Application**.

17. Troubleshooting

17.1 Unable To Support Scripts for Bootloader/Firmware Flashing on Linux

Not all Linux distributions ship with the Python3 package and its modules, which are required to run the support scripts described in section Programming / Flashing Images

To make sure your Linux machine can run the support scripts successfully, execute the following commands (This example is for Ubuntu 20.04):

```
$ sudo apt update
$ sudo apt install -y python3 python3-pip
$ pip3 install pyserial==3.5 argparse==1.4.0
```

These commands update system packages, install Python 3 and its pip tool, and then install the required Python modules ('pyserial' and 'argparse') at the specified versions for the support scripts.

17.2 Flashing Tools Failing Halfway

The flashing tools are used to update the core firmware in the QSPI memory, which forms the core part of the booting process. This should never fail. When a firmware flashing tool fails, the result is often an unbootable 'bricked' device. The only way to recover from this is to use a SCIF boot and the respective flashing process described in the appendix section Factory Firmware Flashing Using Serial Downloader (SCIF) Mode

17.3 DHCP Failure

DHCP depends on the network infrastructure and sometimes takes over 30 seconds or fails completely. When DHCP fails, the SBC will self-assign an IP address from the 169.254.x.y address range. This series of addresses is called the automatic private IP addresses.

This is often a network issue. At times, eth0 can take longer to get the IP address. If eth0 is not responding, recheck with eth1. Your individual network topology will affect the board's ability to get an IP address through DHCP.

17.4 'Ifconfig' doesn't list the Wi-Fi interface

The Wi-Fi is not active by default at boot. While all the drivers and subsystems are loaded, the Wi-Fi must be enabled with the command 'enable Wi-Fi' in the conmanctl utility as described in Wi-Fi 802.11 Module.

17.5 IP Configuration

An IP address is a bit tricky to get right. It often won't show up unless the port is powered up, and it gets complicated to identify the interface name and ensure there is an address on it. There is some trial and error involved in this step for flashing the system image. You can manually assign the IP address to your host if necessary. Refer to the following for more info on Windows IP settings:

1. How to configure a static IP on Windows 10 or 11 | Windows Central
2. Change TCP/IP settings - Microsoft Support

17.6 Stuck in U-boot with error "Bad Linux ARM64 Image magic!"

There is a very rare situation in which a board might refuse to boot the Linux image. It usually displays the following in the UART:

```
NOTICE:  BL2: v2.5(release):
NOTICE:  BL2: Built : 14:13:21, Aug  7 2023
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: v2.5(release):
NOTICE:  BL31: Built : 22:50:40, Aug 27 2023


U-Boot 2020.10 (Sep 08 2023 - 17:04:31 -0400)

CPU: Renesas Electronics E rev 15.4
Model: <Detected Board>
DRAM:  896 MiB
MMC:   sh-sdhi: 0
Loading Environment from SPIFlash... SF: Detected is25wp256 with page size 256
Bytes, erase size 4 KiB, total 32 MiB
*** Warning - bad CRC, using default environment

In:    serial@1004b800
Out:   serial@1004b800
Err:   serial@1004b800
Net:   <Detected Ethernet Interface>
Hit any key to stop autoboot:  0
Failed to load 'boot/Image.gz'
44855 bytes read in 20 ms (2.1 MiB/s)
Error: Bad gzipped data
Bad Linux ARM64 Image magic!
=>
```

This is a board not updated with the newest U-Boot. This is also your chance to try the steps from section Flashing the Bootloader. Once “u-boot” is updated, this issue will be resolved.

18. References

18.1 Git Repositories

Build scripts: Renesas-SST/rz-build-scripts: Build scripts for rz projects (github.com)

Yocto board meta layer: Renesas-SST/meta-renesas: Yocto meta layer for Renesas System Solutions (github.com)

Linux Kernel: Renesas-SST/linux-rz: Linux kernel for System and Solutions Products (github.com)

Arm Trusted Firmware – A: Renesas-SST/rz-atf: Arm Trusted Firmware implementation for System & Solutions products (github.com)

u-boot: Renesas-SST/u-boot: A u-boot supporting System & Solutions Products (github.com)

flash-writer: Renesas-SST/flash-writer: Serial flashing utility to load into blank boards supporting System & Solutions Products (github.com)

rz-utils: Renesas-SST/rz-utils: Collection of utilities for various workflows related to Renesas RZ-based devices (github.com)

18.2 RZ/G MPU Series

Product page: RZ/G Series (Linux-based MPU) | Renesas

Wiki: RZ/G - Renesas-wiki - Confluence

Other RZ topics: RZ Topics - Renesas-wiki - Confluence

18.3 RZ/V MPU Series

Product page: RZ/V Embedded AI MPUs | Renesas

Wiki: RZ/V - Renesas-wiki - Confluence

Other RZ topics: RZ Topics - Renesas-wiki - Confluence

18.4 External Resources

18.4.1 Confluence Page

System & Solution Team Confluence page: System & Solutions Team Home

18.4.2 QT Development

Qt official page: Qt | Tools for Each Stage of Software Development Lifecycle

Qt documentation: Qt Documentation | Home

18.4.3 Yocto Project

Official Yocto manual: Yocto Project Reference Manual — The Yocto Project ® 5.1.4 documentation

18.4.4 Linux Kernel Documentation

The Linux Kernel Documentation — The Linux Kernel documentation

18.4.5 Arm Developer Documentation

Main page: https://developer.arm.com/documentation/

Armv8 Architecture manual: Arm Architecture Reference Manual for A-profile architecture

Generic Interrupt Controller (GIC) architecture specification: Arm Generic Interrupt Controller (GIC) Architecture Specification

Armv8-A Register manual: Arm Armv8-A Architecture Registers

Armv8-A Known issues: Arm Architecture Reference Manual for A-profile architecture: Known issues

Arm Yocto SystemReady IR implementation: Deploying Yocto on SystemReady IR compliant hardware (arm.com)

Arm TrustZone SMCC protocol: SMC Calling Convention (SMCCC) (arm.com)

Arm 64-bit ISA architecture: Arm A64 Instruction Set Architecture

18.4.6 JEDEC DDR4

DDR4 SDRAM STANDARD | JEDEC

18.4.7 PMOD Specification

Wiki: Pmod Interface - Wikipedia

Specification document: pmod-interface-specification-1_3_1.pdf (digilent.com)

18.4.8 Essential Linux Tutorial

Linux/Unix Tutorial (geeksforgeeks.org)

UNIX / LINUX Tutorial (tutorialspoint.com)

18.4.9 Packaging

CMake Reference Documentation — CMake 3.30.2 Documentation

CPack — CMake 3.30.2 Documentation

18.4.10 Using Extensible SDK

Using the Extensible SDK

18.4.11 Install Eclipse IDE

Eclipse Installer 2024-09 R | Eclipse Packages

18.4.12 Linux Kernel Development

HOWTO do Linux kernel development — The Linux Kernel documentation

Linux Kernel - GeeksforGeeks

The Linux Kernel Module Programming Guide (sysprog21.github.io)

A Beginner's Guide to Linux Kernel Development (LFD103) - Linux Foundation - Training

18.4.13 Linux Kernel Driver Development

Basic intro: Device Drivers in Linux - GeeksforGeeks

Driver docs: Driver Basics — The Linux Kernel documentation

Kernel docs: Device Drivers — The Linux Kernel documentation

Lab: Character device drivers — The Linux Kernel documentation (linux-kernel-labs.github.io)

18.4.14 Real-time kernel

Jane W. S. Liu, *Real-Time Systems, Prentice Hall, 2000*.

Giorgio C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, Springer.

Real-time Linux Project: Real-Time Linux

Linux Foundation documentation: realtime:documentation:start [Wiki]

18.4.15 IMD Technologies

IMDT V2H/V2N SBC – Getting Started Guide, 14 April 2025. © 2025 IMD Technologies.

Hardware Datasheet: V2H_V2N - SBC rev2.1.pdf

18.4.16 Renesas RZ Multi-OS package

Reference package providing RPMsg example projects and related documentation for supported RZ devices: RZ Multi-OS Package

Revision History

Rev.	Date	Description	
		Page	Summary
1.00	Mar.03.25	—	Initial release
1.10	Jun.04.25	—	Ubuntu release
2.00	Jul.02.25	—	Yocto Styhead release
3.00	Sep.25.25	—	Expanded release with support for multiple boards
3.10	Apr.17.26	—	Add partner boards to the current RZ common system
3.20	May.05.26	—	Add Real time kernel and eSD support
3.30	Sep.11.26	—	Add Multi-OS support

System Release Package, RZ Series – User Manual

Publication Date: Sep.11.26

Published by: Renesas Electronics Corporation

RZ Family/ RZ/G/V Series

Renesas Electronics Corporation

R12UZ0204EU0330